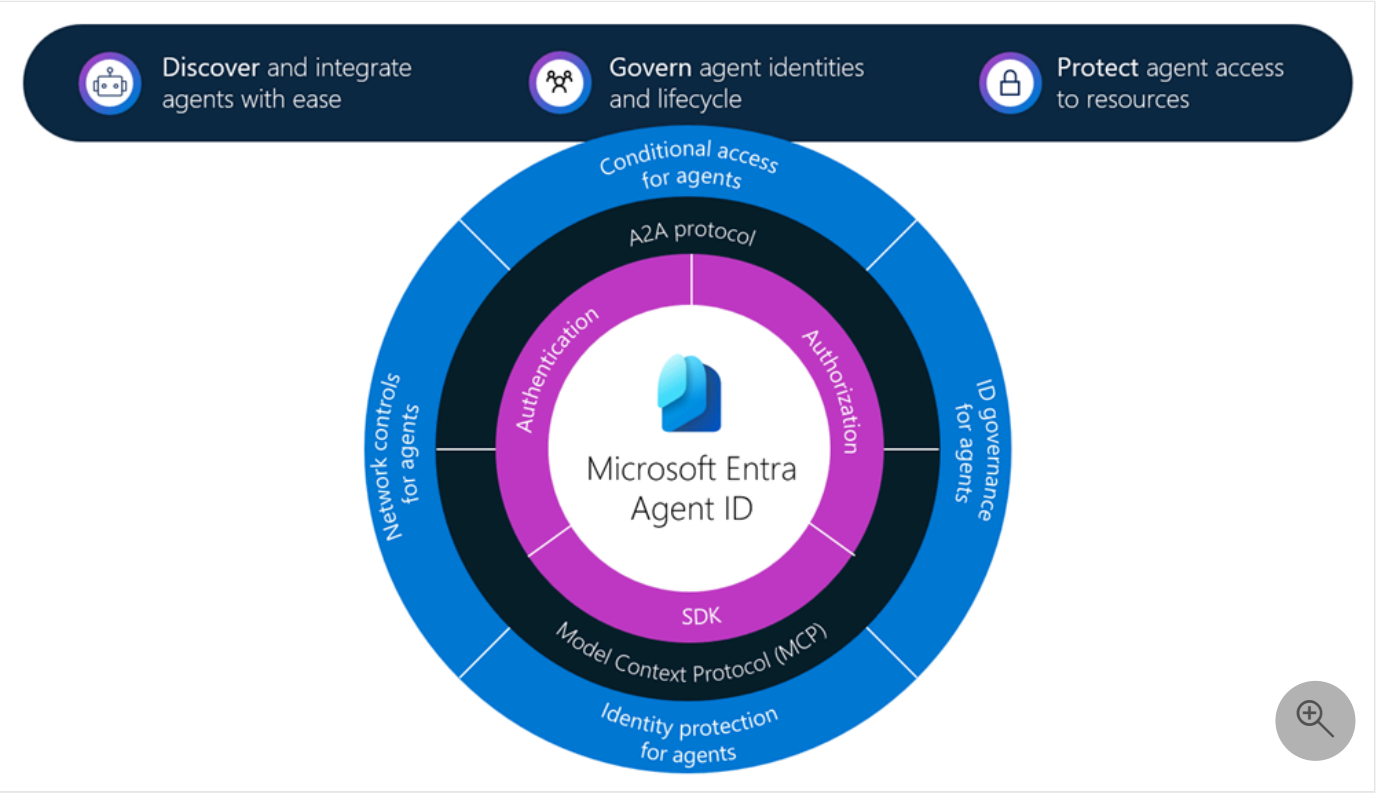
What is Microsoft Entra Agent ID?

Microsoft Entra Agent ID is an identity and security framework that extends Microsoft Entra capabilities to AI agents. As organizations deploy assistive, autonomous, and user-like agents, they need purpose-built identity constructs to authenticate, authorize, govern, and protect these nonhuman identities. Microsoft Entra Agent ID addresses these needs by providing a unified platform for managing agent identities at enterprise scale.



Microsoft Entra Agent ID brings together identity management, access protection, governance, and compliance for AI agents.

Agent identity platform

The Microsoft Entra Agent identity platform enables developers to create and manage agent identities, which are specialized identity constructs built for AI agents. Agent identity blueprints serve as templates for creating individual agent identities with parent-child relationships, enabling consistent security policies across large numbers of agents. The platform supports standard protocols such as OAuth 2.0, Model Context Protocol (MCP), and agent-to-agent (A2A) for authentication and agent-to-agent communication.

Microsoft Entra Agent ID works with agents built on Microsoft and non-Microsoft platforms. Organizations can integrate third-party agents from platforms such as AWS Bedrock and n8n by

using the Microsoft Entra ID Auth SDK (sidecar) or workload identity federation, giving every agent a governed identity regardless of where it was built.

Security and governance for agents

Microsoft Entra Agent ID extends existing Microsoft Entra security and governance capabilities to agent identities. Agents receive the same identity-driven protections as users and workloads, including adaptive access policies, real-time risk detection, lifecycle management, and network-level controls. All agent authentication and activity is logged for compliance and audit.

For details on how these capabilities work for agents, see:

- Microsoft Entra security for AI overview
- Conditional Access for agents
- Identity Protection for agents
- Identity governance for agents
- Network controls for agents
- Sign-in and audit logs for agents

How to get started

Microsoft Entra Agent ID is a product within Microsoft Entra that provides the platform for creating and managing agent identities and agent identity blueprints. Agent ID is available for all Microsoft Entra customers.

Microsoft Agent 365 enables agents to operate across Microsoft 365 services and enterprise workflows, which requires a **Microsoft Agent 365** license for each user. For pricing details, see Microsoft Agent 365 plans and pricing.

Extending Microsoft Entra security features to agents requires Microsoft Agent 365. Agent 365 is included with Microsoft 365 E7 and is available as an add-on to Microsoft E5/A5/Business Premium (or Microsoft Defender Suite + Microsoft Purview Suite). See our latest Agent 365 product terms for more details.

Related content

- Microsoft Entra security for AI overview
- What are agent identities?
- What is the Microsoft Entra Agent identity platform?

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/24/2026

What is the Microsoft agent identity platform

The Microsoft agent identity platform is an identity and authorization framework built to address the unique authentication, authorization, and governance challenges posed by AI agents operating in enterprise environments.

Unlike nonagentic application identities designed for web services or user identities designed for humans, the Microsoft agent identity platform is purpose-built for AI agents. The platform provides specialized components that enable AI agents to authenticate securely and access resources appropriately. It also enables agents to discover other agents and operate within enterprise-grade governance frameworks.

This overview introduces the core components of the Microsoft agent identity platform. It focuses on the identity constructs, authentication mechanisms, token systems, and discovery capabilities that form the foundation of agent identity management in enterprise environments.

How to get started

Microsoft Entra Agent ID is a product within Microsoft Entra that provides the platform for creating and managing agent identities and agent identity blueprints. Agent ID is available for all Microsoft Entra customers.

Microsoft Agent 365 enables agents to operate across Microsoft 365 services and enterprise workflows, which requires a **Microsoft Agent 365** license for each user. For pricing details, see Microsoft Agent 365 plans and pricing.

Extending Microsoft Entra security features to agents requires **Microsoft 365 E7** (includes Agent 365 and Microsoft Entra Suite) or **Microsoft 365 E5** paired with a **Microsoft Agent 365** license. Customers without E5 or E7 can use the following standalone licensing options with a **Microsoft Agent 365** license:

- **Conditional Access for agents**: Microsoft Entra ID P1
- **ID Protection for agents**: Microsoft Entra ID P2
- **ID Governance for agents**: Microsoft Entra ID P1
- **Network controls for agents**: Microsoft Entra Internet Access, included in Microsoft Entra Suite or licensed separately. For more information, see What is Global Secure Access.

Platform architecture overview

The Microsoft agent identity platform is built on several foundational technical components that work together to provide a complete identity and authorization solution for AI agents:

- **Authentication service**: An OAuth 2.0 and OpenID Connect (OIDC) standard-compliant authentication service that enables secure, standards-based authentication for agents. This service issues tokens that agents use to authenticate to resources and APIs, supporting both application-only and delegated access scenarios. There are three objects that form the core identity constructs in the platform: agent identity blueprint, agent identity, and agent's user account.

- **SDKs**: Software development kits that enable developers to integrate with the Microsoft agent identity platform. SDKs abstract the complexity of token acquisition and protocol handling, making it straightforward for platforms that build agents to incorporate identity management into their applications. Microsoft agent identity platform includes two SDKs: Microsoft Identity Web (.NET) and the Microsoft Entra ID Auth SDK (sidecar).

- **Agent management**: A comprehensive agent metadata store and administrative interface within the Microsoft Entra admin center that enables administrators to discover, view, configure, and manage agents. The platform provides the agent registry that is a centralized repository for registering and managing agents across an organization.

These technical components work together with the identity constructs described in the following section to provide complete platform functionality.

Authentication and authorization

The Microsoft agent identity platform uses OAuth for authorization and OpenID Connect (OIDC) for authentication.

- **OpenID Connect (OIDC)** enables agents to authenticate and verify the identity of other entities they communicate with, establishing secure trust relationships.

- **OAuth 2.0** allows agents to request access tokens that authorize them to access resources on behalf of themselves or users, supporting both application-only and delegated access scenarios.

For more information, see OAuth protocols.

Tokens are the fundamental security mechanism enabling secure communication and authorization in the Microsoft agent identity platform. The platform supports multiple token flow patterns designed for specific operational scenarios. For more information, see tokens in Microsoft agent identity platform

Integration and interoperability

The Microsoft agent identity platform is designed to work seamlessly across the Microsoft ecosystem and beyond. It integrates with:

- **Microsoft Entra ID**: The platform extends Microsoft Entra ID capabilities to support agent scenarios, using existing identity infrastructure and policies.
- **Platforms and services that create agents**: Platforms that create and manage agents can integrate with the Microsoft agent identity platform to secure agents. This includes Microsoft-owned platforms like Copilot Studio and non-Microsoft platforms such as Amazon Web Services (AWS) Bedrock, n8n, and other agent frameworks that support OAuth 2.0 and OpenID Connect. Organizations can onboard agents from these platforms by using the Microsoft Entra Auth SDK (sidecar) or workload identity federation, with no platform-specific credential management required.
- **Extended Microsoft identity and security products**: Integration with Conditional Access, identity protection, identity governance, Global Secure Access, and other security services enable comprehensive agent security.

This interoperability ensures that organizations can build, deploy, and manage agent identities consistently regardless of where agents are created or deployed. For step-by-step guidance on integrating non-Microsoft agents, see Integrate third-party agents with Microsoft Entra Agent ID.

Last updated on 06/16/2026

Microsoft Entra security for AI overview

AI agents—autonomous software systems that perceive their environment, make decisions, and take actions—expand organizational capabilities but introduce security challenges that differ from traditional application security. These capabilities require identity-based security controls that authenticate AI workloads, enforce access policies, and provide governance over nonhuman identities.

Microsoft Entra provides the identity control plane for securing AI systems. It extends identity capabilities to AI agents, applications, and services so that organizations can apply consistent authentication, authorization, and governance controls across human and nonhuman identities.

This article explains why AI systems need identity-based security, introduces the key Microsoft Entra capabilities that address these challenges, and links to detailed documentation for each area.

Why AI systems need identity-based security

Organizations are deploying AI agents for increasingly diverse tasks, and each deployment model presents distinct security challenges:

Assistive agents (interactive)

Assistive agents (also called interactive agents) perform specific tasks on demand on behalf of a signed-in user, often through a chat interface. Tasks include analyzing customer data for sales recommendations or answering support questions with escalation to human representatives. These agents are granted Microsoft Entra delegated permissions that allow them to act on behalf of users. Common scenarios include customer support assistants, research helpers, and real-time collaboration agents.

Autonomous agents

Autonomous agents operate independently using their own identity, not a human user's identity. These agents run in the background, making decisions and taking actions without human intervention, such as monitoring network logs for security operations, managing infrastructure deployments with autoscaling, or processing scheduled maintenance tasks. Autonomous agents authenticate directly with the Microsoft Entra ID platform using their agent identity and the client credentials flow.

Agent's user account

An agent's user account is an optional account that pairs 1:1 with an agent identity. It functions with human user characteristics, including persistent identities and access to organizational resources such as mailboxes, calendars, Teams channels, and documents. Use an agent's user account only when the agent must access systems that require a user object. An agent's user account doesn't replace the agent identity — both must exist. For more information, see Agent's user accounts.

Security challenges

Unlike applications that execute predetermined logic, AI agents make dynamic decisions and adapt behavior based on training data, input, and environment conditions. This adaptive behavior expands the organizational attack surface:

- **External accessibility**: Many AI agents interact with external users, third-party systems, or the public internet. This exposure creates potential pathways for adversaries to compromise agents and access organizational systems.
- **Permission escalation risk**: Agents are often provisioned with broad permissions to ensure capability. An agent analyzing financial data might receive access to all financial records, expense reports, and vendor contracts, which is broader than necessary for the specific task.
- **Autonomous decision-making**: Compromised agents that make autonomous decisions can take harmful actions. A supply chain agent with purchasing authority could place unauthorized orders. An infrastructure management agent could delete critical systems.
- **Prompt injection attacks**: AI agents are vulnerable to attacks that don't affect traditional applications. Prompt injection attacks manipulate agent behavior by inserting malicious instructions into data processed by the agent.
- **Agent-to-agent propagation**: Agents that interact with other agents can propagate compromise. If an orchestration agent is compromised, it can potentially target other agents to perform malicious actions.

Organizations must also demonstrate that AI systems operate within governance frameworks, comply with privacy regulations, and maintain audit trails documenting agent actions and decisions.

Agent sprawl

Agent proliferation creates a governance challenge termed "agent sprawl", which is the uncontrolled expansion of agents across an organization without adequate visibility, management, or lifecycle controls. Agent sprawl emerges when business units create agents without formal IT oversight (shadow AI), when agents created for temporary purposes remain in production indefinitely, and when agent permissions exceed actual requirements and are never reviewed.

Uncontrolled agent sprawl leads to increased security risk from over-privileged agents with unclear ownership, compliance challenges when auditors expect governance over AI systems, and incident response difficulties when organizations can't quickly identify compromised agents.

Agent security scenarios

Agent security challenges manifest differently depending on the agent's purpose and deployment context:

Expand table

Scenario	Description	Security challenge	Risk
User-initiated agent	Agents act on behalf of users, inheriting user capabilities and access rights.	Prevent agents from misusing inherited permissions; maintain user control and enable access revocation.	Compromised agents might perform unauthorized actions as the user, such as accessing files, sending communications, or manipulating data.
Autonomous agent	Agents operate with their own identities and permissions, independent of users.	Grant only necessary permissions for intended tasks; prevent agents from exceeding authorized scope.	Compromised agents might operate without constraint, placing unauthorized orders, modifying data, or accessing sensitive information.
Agent's user account	An agent's user account enables the agent to function as a human user with persistent identity, mailbox, and access to collaborative systems.	Maintain appropriate permission scope; prevent compromised agents from using team access to spread malware or manipulate decisions.	Compromised agents might access documents, participate in meetings under false pretenses, or send communications as trusted team members.
Agent-to-agent	Agents interact with other agents, such as an orchestration agent that delegates tasks to specialized agents.	Establish authenticated agent communication; ensure agents interact only with legitimate agents; maintain audit trails of interactions.	Unsecured communication allows adversaries to inject malicious agents or intercept or manipulate agent interactions.

Secure identities for AI agents

AI agents require purpose-built identity constructs that differ from traditional application identities. Microsoft Entra Agent ID provides an identity and security framework designed for AI agents.



Microsoft Entra Agent ID enables organizations to:

- **Register and manage agents**: Create and manage agent identity blueprints as templates and agent identities as individual instances with parent-child relationships, enabling centralized metadata management and automatic organization into security collections.
- **Assign secure, scalable identities**: The Microsoft Entra Agent identity platform enables you to assign identities to agents, autodiscover them across your organization, and manage all agent metadata in one place including capabilities, tasks, and protocols. It provides agent-to-agent discovery and authorization based on standard protocols such as MCP and A2A.
- **Log and monitor agent activity**: All authentication and actions performed by agents are logged in Microsoft Entra ID and viewable through the Microsoft Entra admin center for compliance and audit purposes.

For more information about agent identities as identity constructs, including how they differ from application and user identities, see What are agent identities?.

Control AI access with Zero Trust

Microsoft Entra applies Zero Trust identity controls to AI identities through Conditional Access and Identity Protection.

Conditional Access for agents

Conditional Access enables organizations to define and enforce adaptive policies that evaluate agent context and risk before granting access to resources:

- Enforce adaptive access control policies for all agent patterns across assistive, autonomous, and agent user account types.
- Use real-time signals such as agent identity risk to control agent access to resources, with Microsoft Managed Policies providing a secure baseline by blocking high-risk agents.
- Deploy Conditional Access policies at scale using custom security attributes, while still supporting fine-grained controls for individual agents.

For more information, see Conditional Access for agents.

Identity Protection for agents

Microsoft Entra ID Protection detects and blocks threats by flagging anomalous activities involving agents:

- Detect agent identity risk derived from user risk and based on agents' own actions, including unusual or unauthorized activities.
- Provide risk signals to Conditional Access to enforce risk-based policies and session management controls.
- Provide risk signals to inform agent discoverability and access, with automatic remediation of compromised agents using preconfigured policies.

For more information, see Identity Protection for agents.

Govern AI identities and permissions

As organizations deploy more AI agents, identity governance becomes critical to prevent agent sprawl and maintain compliance. Microsoft Entra ID Governance extends lifecycle management, access reviews, and entitlement management to agent identities:

- Govern agent identities at scale, from deployment to expiration.
- Ensure sponsors and owners are assigned and maintained for each agent identity, preventing orphaned agent identities.

- Enforce that agent access to resources is intentional, auditable, and time-bound through access packages.

For more information, see Identity governance for agents.

For a general overview of Microsoft Entra ID Governance, see What is Microsoft Entra ID Governance?.

Secure AI services and automation

Many AI workloads run as applications, services, or automation pipelines rather than as interactive agents. These workloads need secure, credential-free authentication to access enterprise resources.

Microsoft Entra workload identities provide identity and access management for software workloads. Organizations can use managed identities and federated credentials to authenticate AI services without managing secrets.

For more information, see What are workload identities?.

Secure generative AI architectures

Enterprise generative AI architectures require identity controls at every layer, from the client application to the AI model and the data sources it accesses. Microsoft Entra enforces authentication, authorization, and governance controls across these architectural components.

Network-level controls through Global Secure Access enforce consistent network security policies across users and agents:

- Log agent network activity to remote tools for audit and threat detection, and apply web categorization to control access to APIs and MCP servers.
- Restrict file uploads and downloads using file-type policies to minimize risk, and automatically block and alert on malicious destinations using threat intelligence-based filtering.
- Detect and block prompt injection attacks that attempt to manipulate agent behavior through malicious instructions.

For more information, see Secure Web and AI Gateway for agents.

Related content

- What is Microsoft Entra Agent ID?
- What is the Microsoft Entra Agent identity platform?

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/01/2026

What's new in Microsoft Entra Agent ID

Microsoft Entra Agent ID is now generally available. This release brings first-class identity and access management to AI agents, enabling organizations to authenticate, authorize, govern, and protect agent identities at enterprise scale. Microsoft Entra Agent ID extends Zero Trust principles to AI workloads with purpose-built identity constructs, specialized OAuth flows, and comprehensive security controls.

This article summarizes the key capabilities and documentation currently available.

Manage AI agents at scale

Microsoft Entra Agent ID introduces new identity constructs and authentication protocols designed specifically for AI agents. Notable updates include:

- Key concepts - New concepts added to further define core agent identity concepts and their relationships.
- Administrative relationships - Deeper definitions and clarified differences between owners, sponsors, and managers of agent identities, blueprints, and agents' user accounts.
- Design patterns (New) - Common architectural patterns for agent identity deployments.
- Best practices (New) - Recommended approaches for agent identity management.
- Plan your agent identity architecture (New) - Guidance for planning agent identity deployment at scale.
- Create an agent identity blueprint and Create an agent identity (Preview) - Use the new "wizard" to create agent identity blueprints and agent identities in the Microsoft Entra admin center.
- AI-guided setup (New) - Automate onboarding with an AI coding agent that walks you through blueprint creation, credential configuration, and agent identity provisioning.
- Agent identity deletion (New) - Learn about the automated cascade cleanup process and soft-delete functionality for agent identities.
- Authentication with the Auth SDK (sidecar) (New) - Overview of the sidecar pattern for agent authentication.
- Configure SDK for agent identities - SDK configuration for token acquisition.
- Run the sidecar for local development (New) - Local development setup for the Auth SDK.
- Validate agent tokens in a downstream API (New) - Token validation guidance for APIs that receive agent tokens.

- Configure non-Microsoft agents with Agent ID (New) - Integration patterns (sidecar and federation) for platforms like AWS, GCP, and n8n.
- Secure an Amazon Bedrock agent (New) - Step-by-step guide for securing Bedrock agents with Agent ID.
- Secure an n8n agent (New) - Deploy n8n on Azure with Agent ID integration.
- Migrate custom app registrations (New) - Move agents using standard app registrations to Agent ID.
- Migrate Copilot Studio agents (New) - Move Microsoft Copilot Studio agents to Agent ID.

To simplify agent management across the enterprise, agent registry experiences are converging under Microsoft Agent 365. This change gives customers one place to discover and manage all agents, while Microsoft Entra continues to provide the identity foundation through Agent ID. For more information, see Agent Registry convergence with Microsoft Agent 365.

Govern agent identities and lifecycle

Microsoft Entra ID Governance extends lifecycle and access management capabilities to agent identities:

- Governing agent identities overview - Overview of how Microsoft Entra governs agent identity lifecycle and access.
- Access packages for agent identities - Govern agent access through policy-based access packages and permission assignment for both on-behalf-of (OBO) and autonomous (non-OBO) scenarios.
- Sponsor lifecycle workflows (New) - Automate sponsor maintenance and reassignment for agent blueprints and agent identities.
- Manage your agent identities (New) - View and control agent identities you own or sponsor.
- Agent identity sponsor templates (New) - Two new lifecycle workflow templates for notifying managers and cosponsors, and automatically transfer sponsorship when an agent identity sponsor changes roles or leaves the organization, to prevent orphaned agents.

Protect agent access to resources

Conditional Access and ID Protection features extend Microsoft Entra Agent ID to help secure agent identities and their access to resources:

- Conditional Access for agents (Updated)- Improved guidance and detailed scenarios for Conditional Access policies, plus new templates for agent-specific policies.

- Block access for high-risk agent identities (New) - Conditional Access template to block sign-ins from risky agent identities.
- Block access for high-risk agent's user account (New) - Conditional Access template to block sign-ins from risky agents' user accounts.
- Autonomous agent access policy (New) - Conditional Access template for autonomous agents without user context.
- On behalf of agent access policy (New) - Conditional Access template for agents acting on behalf of users.

Related content

- Security for AI overview
- Microsoft Agent 365
- FAQ

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/01/2026

Microsoft Entra Agent ID key concepts

The Microsoft agent identity platform provides specialized identity constructs designed specifically for AI agents operating in enterprise environments. These identity constructs enable secure authentication and authorization patterns that differ from traditional user and application identities, addressing the unique requirements of autonomous AI systems.

This article explains the core concepts that form the foundation of agent identity management: agent identities, agent identity blueprints, and their supporting components. Understanding these concepts is essential for developers who need to implement secure, scalable authentication patterns for AI agents.

The agent identity architecture follows a hierarchical model where agent identity blueprints serve as templates for creating multiple agent instances, each with distinct identities and capabilities. This approach enables centralized management while providing the flexibility needed for diverse AI agent deployment scenarios.

Core identity concepts

The following concepts form the foundation of Microsoft Entra Agent ID and the Microsoft agent identity platform.

Agent identity

An agent identity is the primary identity an AI agent uses to authenticate to systems and access resources. Unlike user accounts, agent identities don't have credentials of their own. They authenticate using tokens issued by their agent identity blueprint. For more information, see Agent identities.

Agent identity blueprint

An agent identity blueprint is an object in Microsoft Entra ID that serves as the template and authentication foundation for one or more agent identities. The blueprint holds credentials and uses them to acquire tokens on behalf of all agent identities created from it. Policies and settings applied to a blueprint, such as Conditional Access, take effect for all its agent identities. For more information, see Agent identity blueprints.

Agent identity blueprint principal

When a blueprint is added to a tenant, Microsoft Entra creates a corresponding principal object. An agent identity blueprint principal is the Microsoft Entra object that records a blueprint's presence in a tenant and enables it to acquire tokens and appear in audit logs. For more information, see Agent identity blueprint principals.

Traditional service principal (not recommended for AI agents)

Traditional service principals were designed for static, deterministic workloads. Microsoft Entra Agent ID exists because service principals lack the governance infrastructure AI agents need. There's no enforced sponsorship, no agent-aware audit entries, and no blueprint-managed lifecycle. For more information, see Agent identities, service principals, and applications.

Regular user account (not recommended for AI agents)

Regular Microsoft Entra user accounts are designed for human sign-in patterns. Assigning them to AI agents causes failures across every Zero Trust enforcement layer: Conditional Access policies built for humans don't apply correctly to agents, ID Protection detections are degraded, and identity governance processes can incorrectly remove agent access. For more information, see Plan your agent identity architecture.

Agent operation patterns

The agent identity platform supports the following patterns for how agents operate and authenticate, each serving different use cases and security requirements.

Interactive agents

Interactive agents perform specific tasks on demand on behalf of a signed-in user, often through a chat interface. Tasks include analyzing customer data for sales recommendations or answering support questions with escalation to human representatives. These agents are granted Microsoft Entra delegated permissions that allow them to act on behalf of users using the on-behalf-of (OBO) authentication flow. Common scenarios include customer support assistants, research helpers, and real-time collaboration agents.

Autonomous agents

Autonomous agents operate independently using their own identity, not a human user's identity. These agents run in the background, making decisions and taking actions without human intervention, such as monitoring network logs for security operations, managing infrastructure

deployments with autoscaling, or processing scheduled maintenance tasks. Autonomous agents authenticate directly with the Microsoft Entra ID platform using their agent identity and the client credentials flow.

Agent's user account

An agent's user account is an optional account that pairs 1:1 with an agent identity. It functions with human user characteristics, including persistent identities and access to organizational resources such as mailboxes, calendars, Teams channels, and documents. Use an agent's user account only when the agent must access systems that require a user object. An agent's user account doesn't replace the agent identity — both must exist. For more information, see Agent's user accounts.

Agent owners, sponsors, and managers

The agent identity platform introduces an administrative model that separates technical administration from business accountability, ensuring operational control and compliance oversight without excessive permissions. The agent administrative roles include owners, managers, and sponsors.

- **Owners** serve as technical administrators for agents, handling operational and configuration aspects.
- **Sponsors** provide business accountability for agents, making lifecycle decisions without technical administrative access.
- **Managers** are human users who are designated as the hiring manager or operational owner for an agent's user account.

For more information, see Administrative relationships for agent identities (Owners, sponsors, and managers)

Microsoft Entra ID Auth SDK (sidecar)

The Microsoft Entra ID Auth SDK (sidecar) is a containerized web service that handles token acquisition, validation, and secure downstream API calls for agents registered in the Microsoft identity platform. It runs as a companion container alongside your application, allowing you to offload identity logic to a dedicated service. For more information, see Microsoft Entra ID Auth SDK (sidecar).

Related content

- What is an agent ID?
- Microsoft Entra Agent ID oauth protocols
- Inheritable permissions

Last updated on 06/16/2026

What are agent identities

Agent identities are identity accounts within Microsoft Entra ID that provide unique identification and authentication capabilities for AI agents. As organizations increasingly deploy autonomous AI systems, identity models designed for human users and applications prove insufficient. Agent identities address this gap by providing a specialized identity construct built specifically for the unique requirements of AI agents operating at enterprise scale.

What is an agent

In its most fundamental form, an agent is an application that attempts to achieve a goal by understanding its environment / context, making decisions and acting on them autonomously using available tools. Agents can act with or without human intervention when provided with proper goals or objectives.

Key components of an agent include:

- **Model**: Model based agents have language models that serve as the centralized decision maker for agent processes. Models can be general purpose, multimodal, or fine-tuned based on specific agent architecture needs.
- **Orchestration Layer**: The cyclical process that governs how the agent takes in information, performs internal reasoning, and uses that reasoning to inform its next action. This loop continues until the agent reaches its goal or a stopping point. Complexity varies from simple decision rules to chained logic.
- **Memory**: Memory in agents provides agents with more dynamic and up-to-date information, ensuring responses are accurate and relevant. This memory allows developers to provide more data in its original format to an agent, without requiring data transformations, model retraining, or fine-tuning. It differs from typical large language models (LLMs) that remain static, retaining only the knowledge they were initially trained on.
- **Tools**: Tools enable agents to interact with their environment and extend their capabilities. Tools can include web search, database access, APIs, file systems, or integrations with other software. Each tool requires careful consideration of security, permissions, and error handling. By using tools, agents can perform complex tasks, access information, and control external systems, making them more effective and adaptable.

Agent workflows are in whole or part planned and driven autonomously and independently of a human user. There are many types of agent workflows, from the ones initiated directly by a user, to the ones that operate autonomously. Subcomponents, skills, tools, or APIs used in those workflows might or might not themselves be agents.

Why agent identities exist

The emergence of AI agents as autonomous enterprise systems introduces security and operational challenges that existing identity models can't adequately address. Agent identities help to address certain security challenges posed by AI agents, such as:

- The need to distinguish operations performed by AI agents from operations performed by workforce, customer, or workload identities.
- Enabling AI agents to gain right-sized access across systems.
- Preventing AI agents from gaining access to the most critical security roles and systems.
- Scaling identity management to large numbers of AI agents that might be quickly created and destroyed.

Agent identities versus application identities

Application identities, typically represented as service principals in Microsoft Entra ID, were designed for services built and maintained by organizations. These identities carry the expectation of long-term stability, known ownership, and managed lifecycle.

Agents are often created dynamically through automation, user actions in tools like Copilot Studio, or API orchestration. An agent might exist for minutes during a specific task, or might be created and destroyed thousands of times per day as part of an automated workflow. Managing this level of dynamism with existing application identities creates operational complexity and security challenges.

Agent identities embrace this dynamic nature while providing appropriate security controls. Organizations can create agent identities in bulk, apply consistent policies to all agents, and retire agents without leaving orphaned credentials or permission assignments behind. The identity model is designed for scale and ephemerality rather than permanence.

Agent identities versus human user identities

Human user identities are tied to authentication mechanisms humans use daily, such as passwords, multifactor authentication, and passkeys. Human users have data associated with

them like mailboxes, teams, and organizational hierarchy.

Agent identities represent software systems, not human beings. They don't use human authentication mechanisms. However, certain scenarios require agents to appear and operate as if they're human users. For these scenarios, agent identities can be paired with agents' user accounts - special Microsoft Entra user accounts that maintain a one-to-one relationship with their paired agent identity. This distinction allows organizations to provide agents with a user identity when necessary for system compatibility, while still maintaining clear separation and appropriate security policies for AI-driven operations.

What agent identities enable

Agent identities provide the foundation for secure, scalable AI agent deployment by enabling several key capabilities. Like other accounts, agent identities are primarily a means to access apps, web services, and other systems in your organization. An AI agent can use agent identities to:

- **Access web services**. Agents can request access tokens from Microsoft Entra, and use those tokens to access web services. Services can include Microsoft services such as Microsoft Graph, services built by your organization, and services purchased from third-party vendors.
- **Autonomous access**. Agents can act autonomously, using access rights given directly to the agent identity. Access rights can include Microsoft Graph permissions, Azure RBAC roles, Microsoft Entra directory roles, Microsoft Entra app roles, and more.
- **Delegated access**. Agents can act on behalf of human users, using access rights given to the user. The user has control over which rights are delegated to the agent identity.
- **Authenticate incoming messages**: Agents can accept requests from other clients, users, and agents. Those requests can be secured using access tokens issued by Microsoft Entra ID, allowing the agent to reliably identify the caller and make authorization decisions.

Agent identities in practice

Several Microsoft products already use agent identities for authenticating AI agents. Two examples include:

- Entra conditional access optimization agent: When this agent is enabled in your tenant, it's given an agent identity. The Microsoft Entra conditional access optimization agent then uses its agent identity to query Microsoft Entra systems and inspect the tenant's configuration. All queries made by the agent are recorded as having been performed by an

AI agent. The agent's activity is subject to any security policies enforced for agent identities. You can view the agent's identity in the Microsoft Entra admin center in the **Agent identities** tab.

- **Agents built in Copilot Studio**: In organizations that use Copilot Studio, users can quickly create AI agents using Copilot Studio's low-code tools. Each time an AI agent is created, the agent gets an agent identity in the Microsoft Entra tenant. The user who created the agent is recorded as its sponsor. The agent can then use its agent identity to send/receive chat messages to users, and connect to various systems like SharePoint or Dataverse. Any authentication performed by these agents is logged in Microsoft Entra ID as AI agents and viewable through the Microsoft Entra admin center.

How to get started

Microsoft Entra Agent ID is a product within Microsoft Entra that provides the platform for creating and managing agent identities and agent identity blueprints. Agent ID is available for all Microsoft Entra customers.

Microsoft Agent 365 enables agents to operate across Microsoft 365 services and enterprise workflows, which requires a **Microsoft Agent 365** license for each user. For pricing details, see Microsoft Agent 365 plans and pricing.

Extending Microsoft Entra security features to agents requires **Microsoft 365 E7** (includes Agent 365 and Microsoft Entra Suite) or **Microsoft 365 E5** paired with a **Microsoft Agent 365** license. Customers without E5 or E7 can use the following standalone licensing options with a **Microsoft Agent 365** license:

- **Conditional Access for agents**: Microsoft Entra ID P1
- **ID Protection for agents**: Microsoft Entra ID P2
- **ID Governance for agents**: Microsoft Entra ID P1
- **Network controls for agents**: Microsoft Entra Internet Access, included in Microsoft Entra Suite or licensed separately. For more information, see What is Global Secure Access.

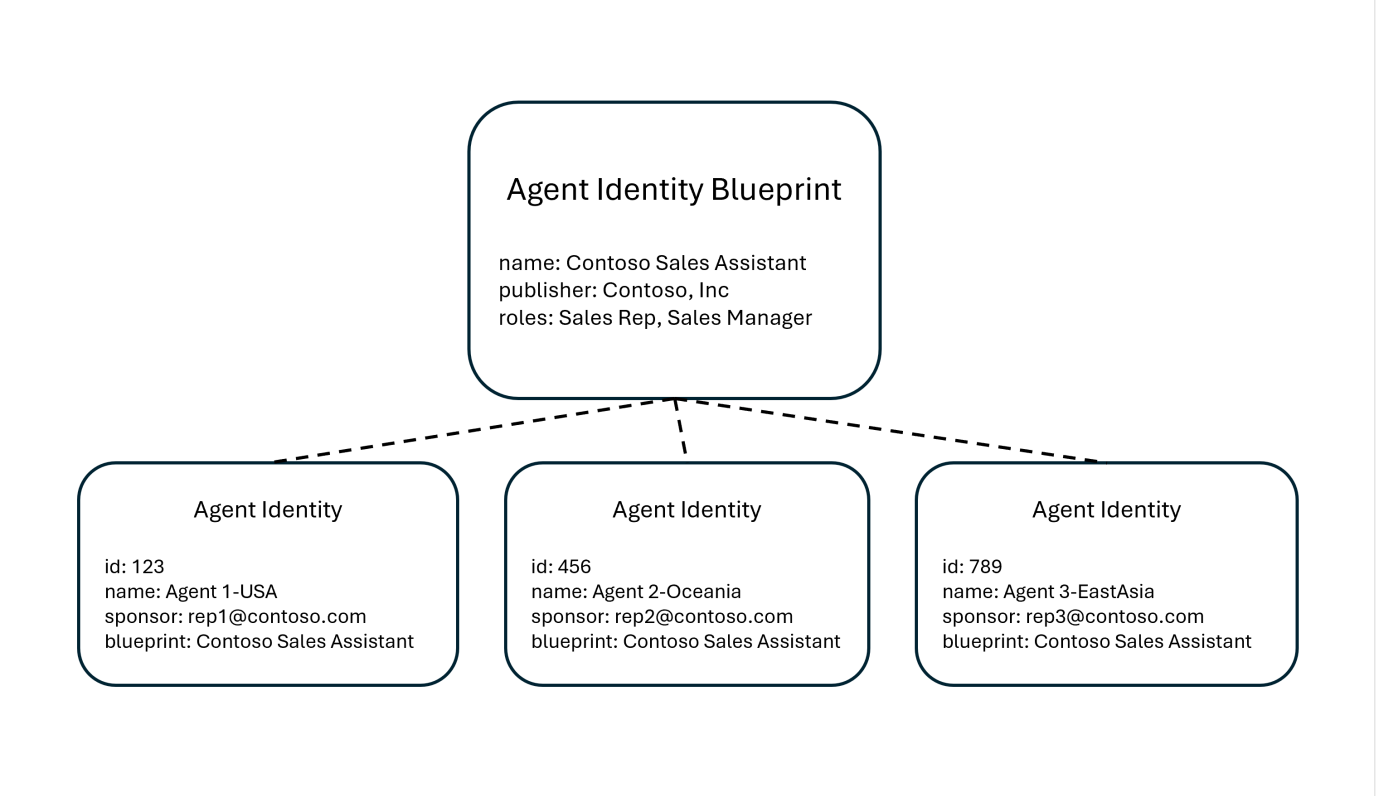
Related content

- Agent identities in Microsoft Entra Agent ID
- What is Microsoft Entra Agent ID?
- Microsoft Entra security for AI overview
- What is the Microsoft Entra Agent identity platform

Last updated on 06/15/2026

Agent identity blueprints in Microsoft Entra Agent ID

An agent identity blueprint is an object in Microsoft Entra ID that serves as a template for creating agent identities. It establishes the foundation for how agents are created, authenticated, and managed within an organization. The agent identity blueprint is a key component of the Microsoft agent identity platform that enables secure development and administration of AI agents at scale.



Agent identity blueprints are a powerful component of the Microsoft Entra Agent ID platform. They hold critical information about how agent identities are created and authenticated, and they serve as a container for managing agent identities at scale.

Agent identity blueprints

An agent identity blueprint is more than just a template for agent identities, just like a blueprint for a building is more than just a drawing. Where architectural blueprints include plumbing, electrical, and structural details, agent identity blueprints include important information about authentication, permissions, and activity logs.

Agent identity blueprints can be used to deploy multiple agents of the same type. If multiple agents are created from one agent identity blueprint, each agent has its own identity, credentials, and permissions but they share common characteristics that are defined by the blueprint. This allows organizations to create a consistent configuration for all agents of a particular type, while still maintaining the flexibility to customize individual agents as needed.

An agent identity blueprint has the following properties that are shared across all its agent identities:

- **Description**: A brief summary of the agent's purpose and functions.
- **App roles**: Define the roles that can be given to users and other principals when using the agent.
- **Verified publisher**: The organization that built the agent.
- **Settings for authentication protocols**: Configure which information is included in access tokens issued to the agent, such as **OptionalClaims**.

The full schema is available in the Microsoft Graph API reference documentation.

Key characteristics of agent identity blueprints

In addition to standard properties, the agent identity blueprint includes key characteristics that are important to understand before creating agent identities from your agent identity blueprint.

Credentials

Credentials used to authenticate an agent identity are configured on the agent identity blueprint. When an AI agent wants to perform an operation, the credentials configured on the agent identity blueprint are used to request an access token from Microsoft Entra ID. OAuth permissions granted to a agent identity blueprint are granted to all agent identities created from that blueprint. There are several credentials types that can be used for agent identities. For more information on these, see credentials for agent identities. For auth protocols, see Agent ID authentication protocols

Security

The blueprint provides a logical container for agent identities in which many different identity administration operations can be performed. This capability helps administrators scale their security efforts to large numbers of AI agents.

Identity administrators can apply policies and settings to agent identity blueprints that take effect for all agent identities created from the blueprint. You can apply Conditional Access policies an agent identity blueprint that targets all agent identities created from that blueprint. Disabling an agent identity blueprint prevents all its agent identities from authenticating.

Used to create agent identities

Blueprints don't just hold information. They're also a special identity type in a Microsoft Entra ID tenant. A blueprint can perform exactly one operation in the tenant: provision or deprovision agent identities. All agent identities in a Microsoft Entra ID tenant are created from an agent identity blueprint. To create an agent identity, a blueprint has:

- An OAuth client ID: a unique ID used to request access tokens from Microsoft Entra ID.
- Credentials: used to request access tokens from Microsoft Entra ID.
- `AgentIdentity.CreateAsManager`: a special Microsoft Graph permission that enables the blueprint to create agent identities in the tenant.

A service uses the blueprint's client ID, credentials, and permissions to send agent identity creation requests via Microsoft Graph APIs. The agent identities created by a blueprint share common characteristics.

For more information, see create agent identities.

Inheritable permissions and required resource access

Agent identity blueprints include two important permission-related configurations that control how agent identities access resources:

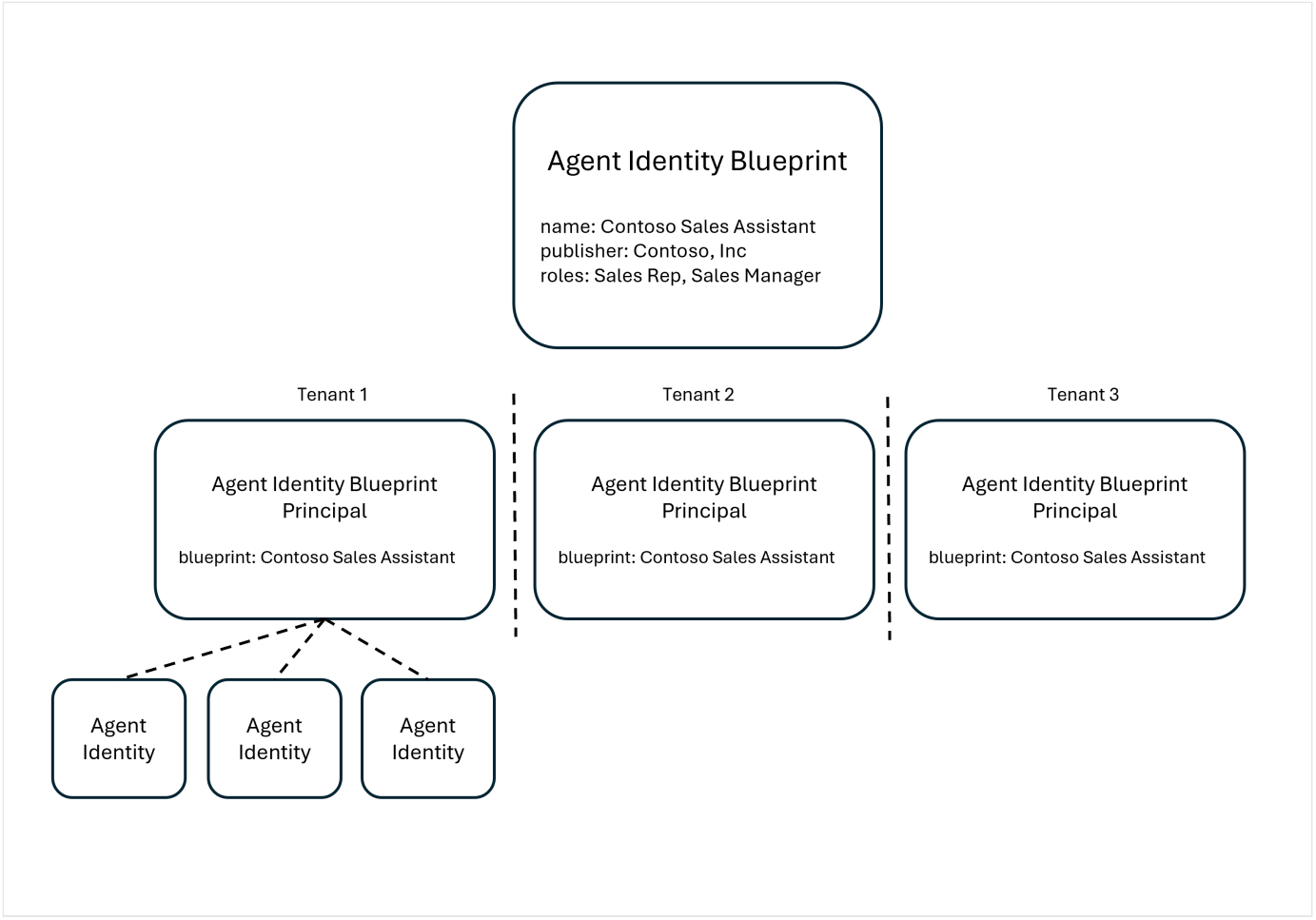
- **Required resource access** declares the APIs and permissions the agent needs to function. This list is visible to administrators during consent review and helps them evaluate whether to approve the agent.
- **Inheritable permissions** define which resource apps can have their permissions automatically inherited by agent identities created from the blueprint. When an administrator grants permissions on the blueprint principal from an inheritable resource app, all agent identities in that organization receive those permissions automatically.

These configurations are declarations that don't grant authorization by themselves. Administrators must still consent to permissions on the blueprint principal or individual agent identities.

For more information, see Inheritable permissions.

Agent identity blueprint principals

An agent identity blueprint principal is an object in Microsoft Entra Agent ID that represents the presence of an agent identity blueprint within a specific tenant. When an agent identity blueprint application is added to a tenant, Microsoft Entra creates a corresponding principal object, which is the agent identity blueprint principal.



This principal serves several important roles:

- **Token Issuance**: When the agent identity blueprint is used to acquire tokens within a tenant, the resulting token's `oid` (object ID) claim references the agent identity blueprint principal. It ensures that any authentication or authorization performed by the agent identity blueprint is traceable to its principal object in the tenant.
- **Audit Logging**: Actions performed by the agent identity blueprint, such as creating agent identities, are recorded in audit logs as being executed by the agent identity blueprint principal. It provides clear accountability and traceability for operations initiated by the agent identity blueprint.

Agent identity blueprints are always created in a Microsoft Entra tenant. An agent identity blueprint is often used to create agent identities in that same tenant. These agent identity blueprints are called "single-tenant." Agent identity blueprints can also be configured as "multitenant" and published to potential customers via Microsoft catalogs. Customers can then add these blueprints to their tenant, so that they can be used to create agent identities.

In either case, an agent identity blueprint principal is always created when a blueprint is added to a tenant. The presence of this principal indicates that a blueprint exists in a tenant and can be used to create agent identities. Customers can remove a blueprint from their tenant by deleting the agent identity blueprint principal.

Related content

- Agent identity blueprint creation channels
- Create an agent identity blueprint
- Credentials for agent identities.

Last updated on 05/01/2026

The agent's user account in Microsoft Entra Agent ID

The agent's user account is a specialized identity type designed to bridge the gap between agents and human user capabilities. The agent's user account enables AI-powered applications to interact with systems and services that require user identities, while maintaining appropriate security boundaries and management controls. It allows organizations to manage the agent's access using similar capabilities as they do for human users.

Example of agent's user account scenarios

Sometimes it's not enough for an agent to perform tasks on behalf of a user or operate as an autonomous application. In certain scenarios, an agent needs to act as a user, functioning essentially as a digital worker. The following are example scenarios where the agent's user account would be applicable:

- The organization needs long-term digital employees that function as team members with mailboxes, chat access, and inclusion in HR systems.
- The agent needs to access APIs or resources that are only available to user identities
- The agent needs to participate in collaborative workflows as a team member

For these reasons, the agent's user account is created. The agent's user account is optional and should only be created for interactions where the agent needs to act as a user or access resources restricted to user accounts.

The agent's user account

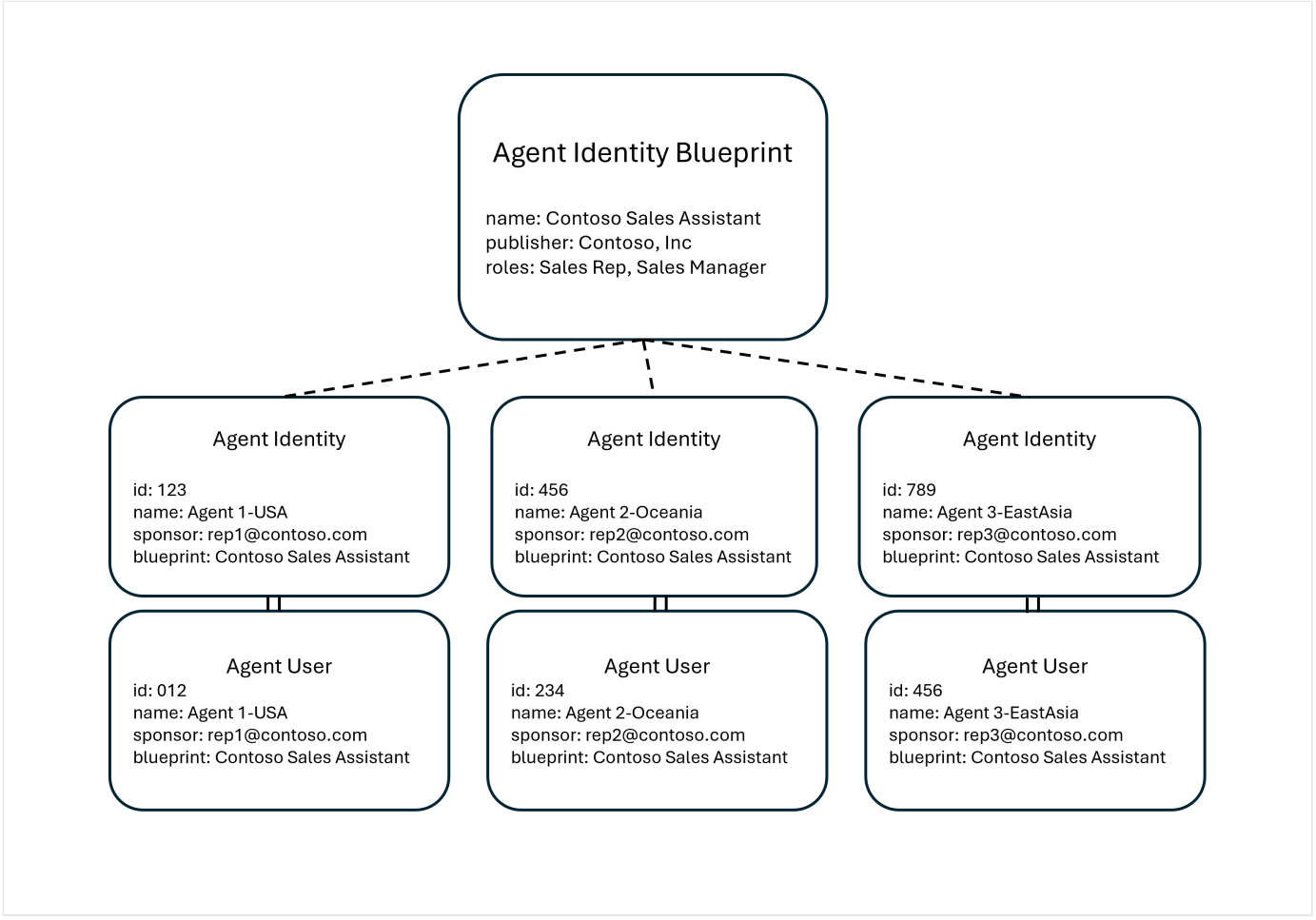
The agent's user account represents a subtype of user identity within Microsoft Entra. These identities are designed to enable agent applications to perform actions in contexts where a user identity is required. Unlike nonagentic service principals or application identities, the agent's user account receives tokens with claim `idtyp=user`, allowing it to access APIs and services that specifically require user identities. It also maintains security constraints necessary for nonhuman identities.

An agent's user account isn't created automatically. It requires an explicit creation process that connects it to its parent agent identity. This parent-child relationship is fundamental to understanding how the agent's user account functions and is secured in Microsoft Entra. Once

established, this relationship is immutable and serves as a cornerstone of the security model for the agent's user account. The relationship is a one-to-one (1:1) mapping. Each agent identity can have at most one associated agent's user account, and each agent's user account is linked to exactly one parent agent identity, itself linked to exactly one agent identity blueprint application.

The agent's user account:

- Is also created using an agent identity blueprint.
- Is always associated to a specific agent identity, specified upon creation.
- Has distinct unique identifiers, separate from the agent identity.
- Can only authenticate by presenting a token issued to the associated agent identity.



Agent's user account and agent identity relationship

The agent identity blueprint doesn't have the permission by default to create the agent's user account because this capability is optional and not always needed. It's a permission that must be explicitly granted to the agent identity blueprint.

The agent's user account is created using the agent identity blueprint. When granted proper permissions, the agent identity blueprint can create an agent's user account and establish a

parent relationship with a specific agent identity. The agent identity is considered the parent of the agent's user account.

Admins manage the lifecycle of an agent's user account. An admin user can delete the agent's user account once its functionalities aren't needed anymore.

Authentication and security model

The authentication model for the agent's user account differs significantly from human-user accounts:

- **Federated identity credentials**: Authentication happens through credentials assigned to the agent's user account. In production systems, use Federated Identity Credentials (FIC). These credentials are used for authenticating both the agent identity blueprint and the agent identity. The credential assigned to the user is used for authenticating across the agent ecosystem.
- **Restricted credential model**: The agent's user account doesn't have regular credentials like passwords. Instead, it's restricted to using the credentials provided through its parent relationship. This restriction on credentials, along with restrictions on interactive sign-in, ensures that the agent's user account can't be used like a standard user account.
- **Impersonation mechanism**: The associated agent identity can impersonate its child agent's user account. It allows the parent's business logic to obtain tokens and act as the agent's user account when needed.

Capabilities of the agent's user account

The agent's user account possesses capabilities that allow it to function effectively within Microsoft 365 and other environments:

- An agent's user account can be added to Microsoft Entra groups, including dynamic groups, enabling it to inherit permissions granted to those groups. It can't, however, be added to role-assignable groups.
- The agent's user account can access resources and utilize other collaborative features typically reserved for human users.
- The agent's user account can be added to administrative units, similar to human users.

- The agent's user account can be assigned licenses, which is often necessary for provisioning Microsoft 365 resources.

Security constraints

The agent's user account operates under specific security constraints to ensure appropriate use:

- Credential limitations: The agent's user account can't have credentials like passwords or passkeys. The only credential type it supports is the agent identity reference to its parent. So even if the agent's user account behaves as a user, its credentials are confidential client credentials.
- Administrative role restrictions: The agent's user account can't be assigned privileged administrator roles. This limitation provides an important security boundary, preventing potential elevation of privileges.
- Permission model: The agent's user account typically has permissions similar to guest users, with more capabilities for enumerating users and groups. The agent's user account can't be assigned privileged admin roles. Custom role assignment and role-assignable groups aren't available to the agent's user account. For more information, see Microsoft Graph permissions reference

Provisioning agent user accounts for Microsoft 365

To fully provision an agent's user account with digital worker capabilities such as a mailbox, Teams presence, or HR system integration, create the agent through Microsoft Teams. Agent 365 and the Agent 365 SDK provide the foundation for agent user accounts to fully participate in Microsoft 365.

> **Note**
>
> Creating an agent's user account directly through the Microsoft Graph API establishes the identity in Microsoft Entra but doesn't provision Microsoft 365 capabilities. Use the Graph API approach only for scenarios that don't require Microsoft 365 participation.

For more information, see the Microsoft 365 Agents SDK documentation.

Related content

- Agent's user account OAuth flow
- Plan your agent identity architecture

Last updated on 06/11/2026

Agent identities, service principals, and applications

Agent identities are built on the same service principal infrastructure that applications use throughout Microsoft Entra, but they're a distinct object type. Standard service principals were designed for static, deterministic workloads and operate using their own credentials directly. Agent identities add a delegation model where the blueprint acquires tokens on behalf of each agent identity, support a one-to-many blueprint relationship, require an assigned sponsor, and generate agent-specific audit entries.

This article explains the following concepts:

- The service principal types in the agent app model
- How they relate to agent identity blueprints
- How they differ from standard application service principals

Agent identity blueprint principal

Agent identity blueprint principals are created automatically when an agent identity blueprint is instantiated in a tenant. These service principals provide the runtime *representation* of the agent identity blueprint within the tenant's directory and enable the agent identity blueprint to perform operations, such as creating instances and managing lifecycle operations.

The creation process involves consent operations that require permissions like `AgentIdentity.Create` and `ServicePrincipal.Manage.OwnedBy`. The agent identity blueprint principal enables the agent identity blueprint to obtain app-only tokens for Microsoft Graph calls necessary to create and manage agent identities. This process is essential because agent blueprints themselves can't directly obtain tokens for Microsoft Graph operations.

Agent identity as service principal

Agent identities are modeled as single-tenant service principals with a new "agent" subtype classification. This design uses existing Microsoft Entra ID service principal infrastructure while adding agent-specific behaviors and constraints.

Agent identities inherit their protocol properties from their parent agent identity blueprint through the ParentID relationship. Unlike standard service principals that operate independently,

agent identities require their parent agent identity blueprint for impersonation and token exchange operations.

Agent identities can be granted permissions directly and appear in sign-in logs when tokens are issued for agent operations. They serve as the primary identity that customers reason about when managing agent permissions and access.

The agent identity blueprint principal creates agent identity service principals using Microsoft Graph calls with app-only tokens and appropriate roles. The creation process establishes the parent-child relationship and configures the necessary Federated Identity Credential (FIC) relationships for impersonation.

Agents built using application service principals

Before the Microsoft agent identity platform was introduced, some Microsoft applications, including earlier versions of Microsoft Copilot Studio and Azure AI Foundry, used standard application service principals to secure their agents. These service principals appear alongside agent identity objects in the Microsoft Entra admin center.

You can distinguish them by filtering the **All agent identities** list. Agents using standard service principals behave identically to other application service principals and don't have the agent-specific behaviors described in this article. For filtering options, see Select viewing options.

Key differences

The following sections describe how the agent app model differs from standard application service principals.

Impersonation model

Standard service principals operate using their own credentials and identity. Agent service principals use an impersonation model: the agent identity blueprint acquires tokens on behalf of each agent identity, so the agent identity appears as the client in the resulting token and in audit logs, even though the blueprint performed the actual token exchange.

This separation means the blueprint holds the authentication credentials while the agent identity holds the permissions and the audit identity. A compromise of the blueprint's credentials affects all agent identities under it, which is why blueprint count is a security boundary decision.

Multi-instance relationship

Standard applications have a one-to-one relationship between application and service principal. The agent app model is one-to-many: a single agent identity blueprint can produce multiple agent identity service principals within a tenant or across tenants. Each agent identity sources its protocol properties from the blueprint but can hold its own permissions on downstream resources.

Credential management

Standard service principals manage their own credentials (certificates, secrets, or managed identities) and use them to obtain tokens directly. Agent identities don't manage credentials. The blueprint holds all credentials and uses them to impersonate each agent identity.

Managed identities are a supported credential type on blueprints (the most secure option for agents running on Azure), but they're a credential *on the blueprint*, not a replacement for the agent identity itself.

Permissions and role assignment

Agent service principals support both application permissions (for autonomous operations) and delegated permissions (for interactive, on-behalf-of operations). Permissions can be assigned directly to an agent identity or inherited from the blueprint.

When `InheritDelegatedPermissions` is enabled, agent identities inherit delegated permissions from the blueprint, which simplifies consent management across many agent identities. Use inherited permissions for the shared baseline and direct assignment for role-specific access.

Agent identities can be assigned Azure RBAC roles and Microsoft Entra built-in roles, the same as standard service principals. Agent identity blueprints can't be assigned Azure RBAC roles.

Audit and logging

Agent service principals maintain distinct identities in audit logs and sign-in reports. When an agent identity performs an operation, logs show it as the acting client and indicate the relationship to its blueprint. Sign-in logs differentiate between agent identity blueprints, agent identities, and agents' user accounts, enabling clear identification of the credential source, the acting identity, and the subject of each operation.

Last updated on 04/30/2026

Administrative relationships in Microsoft Entra Agent ID (Owners, sponsors, and managers)

Microsoft Entra Agent ID introduces an administrative model that separates technical administration from business accountability, ensuring operational control and oversight without excessive permissions. This document explains the administrative relationships for Microsoft Entra Agent ID identity types. This guidance applies to agent identities, agent identity blueprints, agent identity blueprint principals, and agents' user accounts. The article covers owners, sponsors, and managers and their importance in maintaining secure operations.

The administrative relationships available in Agent ID include:

- **Owners**: Technical administrators responsible for operational management of agent identity blueprints and agent identities, including setup, configuration, and credential management.
- **Sponsors**: Business representatives accountable for the agent's purpose and lifecycle decisions, including access reviews and agent retention, without technical administrative access. At least one sponsor is required for each agent identity and agent identity blueprint.
- **Managers**: User responsible for the agent within the organization's hierarchy, able to request access packages for their reporting agents.

These administrative relationships must be configured for each Agent ID object and are separate from the administrative rights granted by Microsoft Entra Role Based Access Control (RBAC) roles, like Agent ID Administrator.

Owners

Owners usually serve as technical administrators for agents, handling operational and configuration aspects. Individual users (including guest users) and service principals can be assigned as owners. Groups aren't supported as owners. Service principals as owners enable automated management of agent identities. Owners are optional for all agent identity objects.

Owner responsibilities

Owners can modify properties that the sponsor can't, like authentication properties. Owners can also add or update other owners and sponsors for the agent identities. Like sponsors, they can

disable and delete agent identities that are no longer needed. Unlike sponsors, owners can re-enable an agent identity that is disabled, restore soft-deleted identities, or hard-delete identities.

Owner access and permissions

Owners have administrative privileges scoped to their assigned agent identity blueprint or agent identity. They can edit settings, manage credentials, change configurations, and assign more owners.

Owners of an agent identity blueprint or agent identity blueprint principal can also create agent identities from that blueprint using delegated permissions, without needing an Agent ID Administrator or Agent ID Developer role. The calling application must be granted one of the following delegated permissions: `AgentIdentity.Create.All`, `AgentIdentity.ReadWrite.All`, or `AgentIdentity.ReadWrite.ManagedBy`.

Owner typical personas

Owners are typically developers or IT professionals with technical knowledge to manage application identities. They might be agent creators, technical application owners, or IT administrators for critical agents. Multiple owners can be assigned for backup coverage.

Service principals can also be set as owners when some other managing service needs the ability to modify or delete specific agent identities without user intervention.

Sponsors

Sponsors provide business accountability for agents, making lifecycle decisions without technical administrative access. They understand the business purpose of the agent, and they can determine whether an agent is still needed or requires access. Sponsors are required for agent identity blueprints and agent identities, ensuring every agent has a designated business owner.

Sponsorship should be maintained to ensure succession when an employee who's a sponsor moves or leaves. Users (including guest users) and groups can be assigned as sponsors. When a group is assigned, all members of the group have sponsor rights over the Agent ID object. Not all group types are supported as sponsors. The following group types are allowed:

- Dynamic membership groups (security or Microsoft 365)
- Assigned membership groups (Microsoft 365)

The following group types aren't allowed as sponsors:

- Role-assignable groups (security or Microsoft 365)
- Assigned membership groups (security)

Sponsor responsibilities

Sponsors make decisions about the agent lifecycle, including renewal, extension, or removal based on business need. They request access packages on behalf of agents, and they provide business justification for access requests. During security incidents, sponsors might determine whether agent behavior is expected and authorize appropriate responses including suspension or permission adjustments.

Sponsor access and permissions

Sponsors operate under least-privilege with limited administrative permissions. They can't modify application settings on agent blueprints or agent identities. Access is limited to nondestructive lifecycle operations: enabling and disabling agent identities, modifying the identity's sponsors, or soft-deleting.

Sponsor typical personas

Sponsors are usually business owners, product managers, team leads, or stakeholders who understand the agent's purpose. For unpublished agents, creators often serve as sponsors. For published agents, sponsors typically come from teams using the agent.

Agent identity sponsors vs. agent's user account sponsors

In Microsoft Agent ID, the agent's identity, blueprint, and blueprint principal may all have sponsors associated with them. In addition, agents can have an agent's user account created in order to access user-oriented services. While the Entra user has a sponsor relationship, there are differences between the user account sponsors and sponsors of the agent identity, blueprint, or blueprint principal.

The sponsor relationship on a user is primarily intended for the sponsors of B2B guests. They are not authorized to make any changes to their sponsored users, but they can request access on the user's behalf and may be involved in approval flows. In contrast, sponsors of agent identities, blueprints, and blueprint principals have limited access to manage those identities directly and can also request access or give approvals in lifecycle workflows.

When an agent is represented by both an agent identity object and an agent user account, we recommend maintaining the agent identity sponsor as the primary user or group responsible for

the agent.

If you require different access or authorization for an associated user account than from its agent identity, then sponsors for each object can request access packages on behalf of the identity they sponsor. If you need to set a sponsor on an agent's user account, then the same user or group should be set as the sponsor on both objects to ensure they can request the appropriate access for both the agent identity and the agent's user account as needed.

Expand table

	Agent user account sponsors	Agent identity, blueprint, blueprint principal sponsors
Allowed types	Users (including guests), groups (any)	Users (including guests), select groups (dynamic membership, Microsoft 365). Role-assignable groups not supported.
Limits	Maximum 5 sponsors	Maximum 100 sponsors, with no more than 5 groups
Authorization	No direct authorization to modify sponsored users	Delete or disable the agent identity and modify its sponsors
Required	Not required	Required on create for agent identities and agent blueprints

Managers

Managers are individual users responsible for an agent identity within the organizational hierarchy. For agents that are active in user scenarios, consider setting a manager on the agent's user account. Managers can request access packages for their agents' user accounts, and will see agents designated as reporting to them in the Microsoft Entra admin center. Managers don't have authorization to modify or delete agents; owners, sponsors, or administrators are required to take those actions.

Requirements and constraints

The administrative model enforces specific requirements and constraints to ensure effective oversight and accountability.

Creation requirements

A sponsor is required when creating an agent identity or agent blueprint. Agent identity blueprint principals are exempt from the sponsor requirement during creation. Owners and managers are

always optional.

Assignment policies

For delegated creation requests where both an application and user context exist, the calling user automatically becomes the sponsor if no sponsors are explicitly specified. However, if one or more other sponsors are designated during creation, the calling user isn't automatically added. Users with Agent ID admin roles aren't made sponsor automatically during creation. This avoids unintentionally overburdening admins with direct responsibility for individual agents.

For app-only create requests, the creating service must set one or more users or supported groups as the sponsor.

Last updated on 07/21/2026

Authorization in Microsoft Entra Agent ID

The introduction of agent identities is driven by the rise of AI-powered agents in organizations. Traditional identity types (like standard app registrations or user accounts) aren't ideal for autonomous agents. AI agents have unique security concerns because their autonomous decision-making, dynamic learning capabilities, and potentially access to sensitive data can introduce unpredictable behaviors.

Microsoft Entra Agent ID was created to fill this gap. Built on the Microsoft Entra ID platform, it provides a dedicated authentication and authorization framework for AI agents that enables them to safely access services and APIs while giving administrators a central way to monitor and control their actions. In short, agent identities let organizations discover, manage, and secure AI agents operating in a tenant, with proper policy enforcement, instead of treating agents as either full users or generic apps.

This article explains how authorization in Microsoft Entra Agent ID works for AI agents by providing information about roles, permission controls, and best practices for managing agent access.

Why agent identity authorization is important

AI agents can execute tasks quickly and at scale. Many high-privilege capabilities in Microsoft Entra ID (for example, the ability to manage users or roles) assume a human administrator with careful intent. An unrestrained agent with high privileges could perform unexpected administrative tasks with far reaching impact (such as deleting users or changing security settings).

For this reason, Microsoft Entra ID limits what agent identities can do. For example, Microsoft Entra blocks agents from being granted many high privilege roles or permissions. Users and administrators aren't allowed to consent to those powerful permissions for an agent. This design recognizes that agents should operate with least privilege. By preventing agents from receiving sensitive privileges, the system minimizes the risk that an AI agent could escalate access. The list of allowed roles and permissions will evolve over time.

Microsoft Entra role assignments for agent identities

From an authorization standpoint, an agent identity behaves somewhat like an application or a user with extra safeguards. Each agent identity has a service principal or a user in Microsoft Entra ID, and it can be assigned certain Microsoft Entra roles.

For example, an agent's identity can be assigned a Microsoft Entra role to give it administrator privileges, but many highly privileged directory roles are blocked for agents. Roles such as Global Administrator, Privileged Role Administrator, or User Administrator can't be assigned to agent identities. Only lower privileged roles (such as a reader role) can be assigned to an agent. You can't assign any custom roles to agent identities. Also, agent identities can't be members of role-assignable groups.

Microsoft has created the Agent ID Administrator and Agent ID Developer roles for managing and creating agents themselves.

Microsoft Entra roles allowed for agents

The following is the list of Microsoft Entra roles that can be assigned to agent identities:

- AI Administrator
- Attack Payload Author
- Attack Simulation Administrator
- Attribute Assignment Reader
- Attribute Definition Reader
- Attribute Log Administrator
- Attribute Log Reader
- Azure DevOps Administrator
- Azure Information Protection Administrator
- B2C IEF Policy Administrator
- Billing Administrator
- Cloud App Security Administrator
- Compliance Administrator
- Compliance Data Administrator
- Customer LockBox Access Approver
- Desktop Analytics Administrator
- Directory Readers
- Directory Synchronization Accounts
- Dynamics 365 Administrator
- Dynamics 365 Business Central Administrator
- Edge Administrator

- Exchange Administrator
- Exchange Recipient Administrator
- Extended Directory User Administrator
- External ID User Flow Administrator
- External ID User Flow Attribute Administrator
- Fabric Administrator
- Global Reader
- Global Secure Access Log Reader
- Insights Administrator
- Insights Analyst
- Insights Business Leader
- IoT Device Administrator
- Kaizala Administrator
- Knowledge Administrator
- Knowledge Manager
- License Administrator
- Message Center Privacy Reader
- Message Center Reader
- Microsoft 365 Backup Administrator
- Microsoft 365 Migration Administrator
- Microsoft Entra Joined Device Local Administrator
- Microsoft Graph Data Connect Administrator
- Microsoft Hardware Warranty Administrator
- Microsoft Hardware Warranty Specialist
- Network Administrator
- Office Apps Administrator
- Organizational Branding Administrator
- Organizational Data Source Administrator
- Organizational Messages Approver
- Organizational Messages Writer
- People Administrator
- Places Administrator
- Power Platform Administrator
- Printer Administrator
- Printer Technician
- Purview Workload Content Administrator
- Purview Workload Content Reader

- Purview Workload Content Writer
- Reports Reader
- Search Administrator
- Search Editor
- Security Reader
- Service Support Administrator
- SharePoint Administrator
- SharePoint Embedded Administrator
- Skype for Business Administrator
- Teams Administrator
- Teams Communications Administrator
- Teams Communications Support Engineer
- Teams Communications Support Specialist
- Teams Devices Administrator
- Teams Reader
- Teams Telephony Administrator
- Tenant Creator
- Usage Summary Reports Reader
- User Experience Success Manager
- Virtual Visits Administrator
- Viva Glint Tenant Administrator
- Viva Goals Administrator
- Viva Pulse Administrator
- Windows 365 Administrator
- Windows Update Deployment Administrator
- Yammer Administrator

Microsoft Graph permissions for agent IDs

For OAuth2 permissions, agent IDs (specifically, agent identity blueprints and agent identity blueprint principals) can use the same Microsoft Graph permission model as other apps. Agents can request delegated permissions (acting on behalf of a user via consent) or application permissions (app-only privileges granted by an administrator).

However, a set of high-risk Microsoft Graph API permissions are explicitly blocked for agents. For example, an agent can't be granted the following permissions:

Expand table

Blocked permission	Notes
`Application.ReadWrite.All`	Allows managing all applications.
`RoleManagement.ReadWrite.All`	Includes full control over users, groups, roles, directory settings, and other critical operations.
`User.ReadWrite.All`	Grants full control of all user accounts.
`Directory.AccessAsUser.All`	Grants access to information in the directory as the signed-in user. Ensures that an agent can't circumvent security by asking for sweeping Microsoft Graph access – even an administrator can't consent to give an agent those permissions.

Agent IDs can still be granted lower-privilege permissions as appropriate. For example, if an agent needs to read a user's mailbox or OneDrive file on that user's behalf, it can request a delegated permission like `Mail.Read` or `Files.Read` and the user (or administrator) can consent. Those aren't considered high-privilege in the tenant-wide sense; they're bounded to that user's data.

What's blocked are the tenant-scoped privileges that go beyond a single user or involve administrative control. Agents operate under a principle of limited scope. Agents can only do what a regular user could consent to, or what an administrator explicitly grants in a controlled, scoped manner.

When to Use Azure roles, Microsoft Entra roles, or Microsoft Graph permissions

Depending on what an agent needs to do, administrators can grant access in different ways to keep the scope appropriate. This includes assigning Azure roles, Microsoft Entra roles, OAuth permission grants including Graph permissions, application role assignments, access package assignments, and group memberships.

Azure roles

If an agent needs to access Azure resources: Assign Azure roles for those specific resources. For example, to let an agent read an Azure Key Vault, give its identity a Key Vault Reader role on that vault. This keeps scope narrow (just that resource or resource group) and uses least privilege. For more information, see Assign Azure roles using the Azure portal.

Microsoft Entra roles

If an agent needs to perform directory-level actions: Use Microsoft Entra roles only if a suitable lower privileged role exists. For example, if an agent just needs to read basic directory info, you might use a Directory Reader-type role. If you need to grant an agent write access, review the implications and select the least privileged role. There might not be an appropriate built-in role that isn't blocked. In those cases, you might choose to rely on Microsoft Graph permissions instead (with the understanding of their limits). For more information, see Assign Microsoft Entra roles.

Delegated Microsoft Graph permissions

If an agent acts on behalf of a user (user-centric scenarios): Use delegated Microsoft Graph permissions. This option requires interactive user consent but ensures the agent can't exceed that user's access. For example, an agent scheduling meetings for Alice would use delegated calendar API permissions; Alice consents, and the agent can only manage Alice's calendar (just as Alice could by themselves). For more information, see Overview of permissions and consent in the Microsoft identity platform.

Microsoft Graph application permissions

If an agent runs autonomously across the tenant (service scenarios): Use Microsoft Graph application permissions sparingly. Grant only the specific app permissions needed, and only if they aren't high-privilege. For example, an agent that generates organizational org-charts might need User.Read.All app permission to read all profiles – that could be acceptable (and isn't on the blocked list), whereas User.ReadWrite.All would be rejected.

Always review the permission's scope: tenant-wide read access might be fine for certain data, but tenant-wide write or control isn't allowed for agents. Administrators must explicitly consent to any app permission an agent gets, so there's an opportunity to review these requests carefully. For more information, see Overview of permissions and consent in the Microsoft identity platform.

Inheritable permissions for agent identities

Agent identity blueprints support inheritable permissions, which let administrators grant permissions once at the blueprint level and have those grants automatically apply to all agent identities created from the blueprint. This capability reduces repeated consent prompts across multiple deployments and environments.

For more information about how inheritable permissions, required resource access, and direct grants work together, see Inheritable permissions.

Last updated on 06/12/2026

Inheritable permissions and required resource access in Microsoft Entra Agent ID

When you build an agent identity blueprint, there are two key permission-related configurations: *required resource access* and *inheritable permissions*. These configurations work together to define what an agent needs, what administrators review during consent, and how permissions flow to agent identities.

Understanding the relationship between these configurations and how they affect authorization is essential for both developers designing agent blueprints and administrators onboarding agents into their organizations.

Required resource access

Required resource access is the agent identity blueprint's initial declaration of the APIs and permissions that the blueprint's child agent identities need to operate. It's expressed as a set of target resource applications and the specific delegated scopes and application roles the agent requests.

Required resource access serves as the agent's list of static consent permissions. When a tenant administrator reviews the agent for approval, this list makes the consent decision explicit and reviewable. It answers the question: "*What does this agent need to function?*"

Dynamic consent can still grant permissions that are inherited when the permission is explicitly requested and the resource app is configured as inheritable. However, dynamically requested permissions aren't visible up front unless they're also declared in required resource access.

Key characteristics of required resource access:

- Declares the baseline set of permissions the agent needs for its initial experience.
- Is visible to tenant administrators during the consent and onboarding process.
- Is a declaration, not a permission grant. Authorization requires administrator consent.

Inheritable permissions

Inheritable permissions is a list of resource apps configured on an agent identity blueprint that defines which permissions can be automatically inherited by agent identities created from that blueprint. When an administrator grants permissions to the agent identity blueprint principal and

those permissions are from resource apps listed as inheritable, all present and future agent identities created from that blueprint in the organization automatically receive those permissions with their tokens.

Inheritable permissions address a common deployment challenge: when you have multiple instances of the same agent across environments or business units, you don't want administrators to re-consent for the same permissions on every agent identity. Inheritable permissions let the administrator approve once at the blueprint level and have that approval apply automatically.

Two conditions for inheritance

For a permission to be inherited by an agent identity, *both* of the following conditions must be met:

- The **resource scopes, roles, or both must be listed** in the inheritable permissions configuration on the agent identity blueprint.
- The **permission must be granted** with:
  - static consent using *required resource access*, or
  - dynamic consent with the permissions explicitly declared on the consent request.

If either condition is missing, inheritance doesn't occur.

What inheritable permissions include

Inheritable permissions support both:

- **Delegated scopes**: Appear in the agent's delegated application permission access token `scp` claim.
- **Application roles**: Appear in the agent's application permission token `roles` claim.

Inheritance patterns

The following patterns are supported per resource app:

Expand table

Pattern	Description
All allowed	Inherit all available delegated scopes or application roles for the specified resource app. Newly granted scopes or roles on the blueprint principal are automatically included.

Pattern	Description
None	Inherit no scopes or roles for the specified resource app. Use this pattern to explicitly disable inheritance for scopes or roles independently.

You can configure scopes and roles independently on the same resource app. For example, you can inherit all scopes while inheriting no roles, or vice versa.

Declaration, grant, and inheritance

Required resource access and inheritable permissions are *configurations*: they don't grant any authorization by themselves. It's important to understand the distinction between what's declared, what's granted, and what's inherited.

Expand table

Layer	What it is	Who controls it	Effect
Required resource access	The list of APIs and permissions the agent needs to function	Developer (on the blueprint)	Visible to admins during consent review. Doesn't grant access.
Inheritable permissions	The list of resource apps eligible for inheritance	Developer (on the blueprint)	Defines which resource apps can have permissions flow to agent identities. Doesn't grant access.
Consent on blueprint principal	Permissions granted by an admin to the blueprint principal in a tenant	Tenant administrator	Grants authorization. If the resource app is also listed as inheritable, the permission flows to all agent identities.
User or agent consent on agent identity	Permissions granted directly to a specific agent identity	Tenant administrator	Grants authorization for that specific agent identity only.
Effective permissions in token	The merged set of inherited + directly granted permissions	Platform (at token issuance)	What the agent identity can actually do at runtime.

> **Note**
>
> Inherited permissions aren't visible as permissions on agent identities in the Microsoft Entra admin center or through Microsoft Graph. They're only observable in the token contents at

runtime. The platform merges inherited and directly granted permissions during token issuance.

Permission configuration cheat sheet

Use these quick rules:

- Static consent at the blueprint level depends on the permission being in required resource access.
- Dynamic consent at the blueprint level can work even when the permission isn't in required resource access, but the permission must be explicitly requested.
- Inheritance to agent identities depends on whether the resource app is configured as inheritable.
- Up-front visibility depends on whether the permission is in required resource access.

Static consent (blueprint principal)

Expand table

Permission in required resource access?	Resource app is inheritable?	Inherited by agent identities?	Visible to admins up front?
Yes	Yes	Yes	Yes
Yes	No	No	Yes
No	Yes	No	No
No	No	No	No

Dynamic consent (blueprint principal, permission explicitly requested)

Expand table

Permission in required resource access?	Resource app is inheritable?	Inherited by agent identities?	Visible to admins up front?
Yes	Yes	Yes	Yes
Yes	No	No	Yes
No	Yes	Yes	No

Permission in required resource access?	Resource app is inheritable?	Inherited by agent identities?	Visible to admins up front?
No	No	No	No

Direct consent on an agent identity remains available in all cases, but those grants apply only to that specific agent identity.

Best practices

When you configure required resource access and inheritable permissions for agent blueprints, balance security, usability, and future scalability.

- **Minimize up-front permissions.** Only include resource access that's essential for the agent's core functionality in the required resource access. Requesting unnecessary permissions at installation increases friction and reduces trust with tenant administrators.
- **Predeclare potential future permissions.** Specify resource apps that might be needed for future agent features in the inheritable permissions list. This transparency enables admins to anticipate future consent requests and facilitates smoother deployments across environments.
- **Use inheritable permissions for reusability.** Use inheritable permissions to let admins grant consent once at the blueprint level and have that approval automatically apply to all agent identities, including across multiple deployments and environments. If you're going to require a permission, it's good practice to also make its resource app inheritable so administrators don't have to grant it on each agent identity individually.
- **Keep governance simple and predictable.** Explicitly defining which permissions are required and which might be requested later helps organizations maintain clear access control and avoid unexpected permission escalations.
- **Review security implications.** Ensure that inheritable permissions don't grant excessive access or expose sensitive resources beyond what's necessary. Regularly audit permission lists to maintain compliance and minimize risk.

Example scenarios

The following scenarios illustrate how different permission configurations serve different deployment needs.

Scenario 1: Agent has optional features that require permissions later

Priya is building an IT help desk agent that answers questions from a knowledge base. Priya expects customers to later enable optional actions like creating incidents or posting to Teams. Priya leaves required resource access empty or minimal. She defines the resource apps her agent uses in the inheritable permissions list. When her company enables an action feature, the admin grants the needed permission once on the blueprint principal, and that approval is reused for all deployments.

Scenario 2: Agent requires permissions up front that should be inheritable

Mateo is building a new hire onboarding agent that needs Microsoft Graph access to read user profiles and create tasks. Mateo lists the baseline Graph permissions in required resource access and also adds the Graph resource app to the inheritable permissions list. When his company rolls the agent out to multiple business units, the admin review is consistent: the same permissions are requested every time, and the inheritable designation reduces repeated approval effort.

Scenario 3: Agent requires permissions that should not be inheritable

Lin is building privileged operations agent used by a small team of admins to perform sensitive tasks. The agent needs high-privilege permissions immediately. Lin includes these in required resource access but intentionally doesn't add them to the inheritable permissions list. For her company, each installation requires a fresh, explicit admin decision, reducing permission sprawl for highly privileged access.

Scenario 4: Agent requires different permissions in different organizations

Aisha is building a compliance evidence collector agent. Some tenants need it to pull from Microsoft 365 audit sources; others need it to pull from SharePoint sites. Aisha defines a small core set in required resource access and lists the full menu of possible resources in the inheritable permissions list. Each organization grants only the permissions that match their architecture, and the inheritable approach reduces repeated approvals during rollout.

Related content

- Configure inheritable permissions for agent identity blueprints
- Agent identity blueprints in Microsoft Entra Agent ID
- Authorization in Microsoft Entra Agent ID
- Agent ID design patterns
- Best practices for Microsoft Entra Agent ID

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/01/2026

Tokens in Microsoft agent identity platform

Tokens are the fundamental security mechanism that enables secure communication and authorization in the Microsoft agent identity platform. This article explains how tokens work in agent app scenarios.

In agent app scenarios, tokens carry enhanced claims to support the unique requirements. Unlike nonagentic application tokens, tokens used by agents include specialized claims that identify entity types, delegation relationships, and authorization contexts specific to agent operations.

These tokens enable secure communication between:

- Agent identity blueprints and their agent identities
- Agent identities and resource APIs
- Agents' user accounts and the services they interact with
- Complex delegation chains involving multiple agent entities

Token claims entity identifiers

Agent tokens include specialized claims that identify the type and role of entities participating in authentication flows:

- Actor facet claims (`xms_act_fct`): Identify the entity performing actions within the token flow. These claims enable systems to understand who is actually requesting access or performing operations.
- Subject facet claims (`xms_sub_fct`): Identify the ultimate subject for whom operations are being performed. It enables proper attribution even in complex delegation scenarios.
- Identity type claims (`idtyp`): Distinguish between user and application contexts, enabling appropriate policy application and security enforcement.
- Identity relationship claims (`xms_idrel`): Describe the relationship between the token subject and the resource tenant, supporting multi-tenancy and guest access scenarios.

Token flow patterns

Agents participate in several distinct token flow patterns, each designed for specific operational scenarios. For more information, see auth protocols in Microsoft agent identity platform.

Tokens in user delegation scenarios

User delegation scenarios enable agent applications to operate on behalf of users. In this scenario, tokens preserve user identity while identifying the agent identity blueprint as the acting entity. The OBO protocol requires token audience to match the client ID. However, for agent identity (agent ID), the incoming token has the audience of the agent identity blueprint. In this scenario, tokens have the following key characteristics:

- Maintain user identity context throughout operations
- Include delegated permissions granted to the agent identity blueprint
- Enable policy evaluation at both user and application levels
- Support both interactive and background operations

The agent identity receives delegated permissions that can be directly assigned or inherited from the parent agent identity blueprint when impersonation is used.

Tokens in application-only scenarios

Application-only scenarios represent autonomous operations where agent identity blueprints act on their own behalf without user context.

In this scenario, tokens have the following key characteristics:

- Represent the agent identity blueprint's own identity
- Include application-level permissions directly assigned by the tenant administrator
- Enable unscoped access within granted permission boundaries. Delegated permissions aren't applied.
- Support fully autonomous background operations

Tokens in agent's user account impersonation scenarios

Agent's user account impersonation scenarios enable the agent's user account to operate like a human user. These tokens support scenarios where agents need user-like context but with controlled, predefined identities.

In this scenario, tokens have the following key characteristics:

- Use specialized agent's user account identities

- Maintain user-context behavior patterns
- Include scoped delegated permissions
- Require explicit assignment to agent identities

In this scenario, the agent identity blueprint impersonates the agent identity, which then impersonates the assigned agent's user account. Access is scoped to delegated permissions assigned to the agent identity, ensuring the agent can't exceed its granted permissions even when operating with user context. The agent's user account can only be used when assigned to an agent identity and can't authenticate independently.

Nonagentic API integration

Both agents and nonagentic OBO clients carry forward subject facets. It enables resource servers to apply appropriate policies and logging for agent subjects. It also enables proper attribution even when tokens pass through nonagentic intermediaries.

Builder application claims

Platforms that create agents and integrate with Microsoft Entra Agent ID don't receive special token claims. These platforms use standard token claims appropriate for their authentication method and don't participate in the specialized agent claim structure.

Tenancy models and token behavior

All tokens contain a tenant ID (`tid`) representing the organization's tenant. Tokens are bounded within the tenant of the agent identity. Each agent identity receives tokens scoped to its operational tenant. Agent identities can't access resources outside their assigned customer tenant. Permission inheritance flows directly from parent to child entities.

Token validation

Clients using agent identities are expected to treat the access tokens issued to them to use at resource servers as opaque, and not try to parse them. However, resource servers that receive access tokens issued to agents need to parse the tokens to validate them and extract claims for authorization purposes.

Resource servers should validate agent tokens by:

- Verifying standard OAuth claims (aud, exp, iss)

- Checking agent facet claims for proper entity identification
- Validating permissions based on token type (delegated vs app-only)
- Ensuring tenant boundary compliance

Token claims enable policy engines to:

- Identify agents vs nonagentic clients
- Distinguish between different agent scenarios
- Apply appropriate conditional access policies
- Generate accurate audit logs

Example of the validation process would be to do the following actions:

- Check if a token was issued for an agent identity and for which agent blueprint.

  ```C#
  HttpContext.User.GetParentAgentBlueprint()
  ```

- Check if a token was issued for an agent's user account identity.

  ```C#
  HttpContext.User.IsAgentUserIdentity()
  ```

These two extensions methods, apply to both `ClaimsIdentity` and `ClaimsPrincipal`.

Audit and logging integration

Token claims provide the foundation for comprehensive audit trails:

- `azp` identifies the requesting agent identity for client attribution
- `oid` identifies the subject for resource access attribution
- `xms_act_fct` and `xms_sub_fct` enable detailed flow analysis
- `tid` ensures proper tenant context in logs

Related content

Agent token claims

Last updated on 04/08/2026

Authentication protocols in agents

Agents use OAuth 2.0 protocols with specialized token exchange patterns enabled by Federated Identity Credentials (FIC). All agent auth flows involve multi-stage token exchanges where the agent identity blueprint impersonates the agent identity to perform operations. This article explains the authentication protocols and token flows used by agents. It covers delegation scenarios, autonomous operations, and federated identity credential patterns. Microsoft recommends that you use our SDKs like Microsoft Entra ID Auth SDK (sidecar) since implementing these protocol steps isn't easy.

All agent entities are confidential clients that can also serve as APIs for On-Behalf-Of scenarios. Interactive flows aren't supported for any agent entity type, ensuring that all authentication occurs through programmatic token exchanges rather than user interaction flows.

> ⚠ **Warning**
>
> Microsoft recommends using the approved SDKs like Microsoft.Identity.Web and Microsoft Entra ID Auth SDK (sidecar) libraries to implement these protocols. Manual implementation of these protocols is complex and error-prone, and using the SDKs helps ensure security and compliance with best practices.

Prerequisites

If you aren't familiar already, go through the following protocol docs.

- Microsoft identity platform and OAuth 2.0 On-Behalf-Of flow
- Microsoft identity platform and the OAuth 2.0 client credentials flow

Supported grant types

The following are the supported grant types for agent applications.

Agent identity blueprint

Agent identity blueprints support `client_credentials` enabling secure token acquisition for impersonation scenarios. The `jwt-bearer` grant type facilitates token exchanges in On-Behalf Of

scenarios, allowing for delegation patterns. `refresh_token` grants enable background operations with user context, supporting long-running processes that maintain user authorization.

Agent identity

Agent identities use `client_credentials` for app-only autonomous operations, enabling independent functionality without user context, and impersonation for a user agent identity. The `jwt-bearer` grant type supports both client credential flow and On-Behalf Of (OBO) flow providing flexibility in delegation patterns. `refresh_token` grants facilitate background user-delegated operations, allowing agent identities to maintain user context across extended operations.

Unsupported flows

- The agent application model explicitly excludes certain authentication patterns to maintain security boundaries. Agents aren't supported for interactive (`/authorize`) flows, ensuring all authentication occurs programmatically.
- Public client capabilities aren't available, requiring all agents to operate as confidential clients.
- A web redirect URI can be configured on a blueprint for consent flows only (`response_type=none`), but can't be used for interactive token acquisition. Full redirect URI functionality is configured on the client application.

Core protocol patterns

Agents can operate in three primary modes:

- Agents operating on behalf of regular users in Microsoft Entra ID (interactive agents). This is a regular on-behalf-of flow.
- Agents operating on their own behalf using service principals created for agents (autonomous).
- Agents operating on their own behalf using user principals created specifically for that agent (for instance agents having their own mailbox).

Managed identities integration

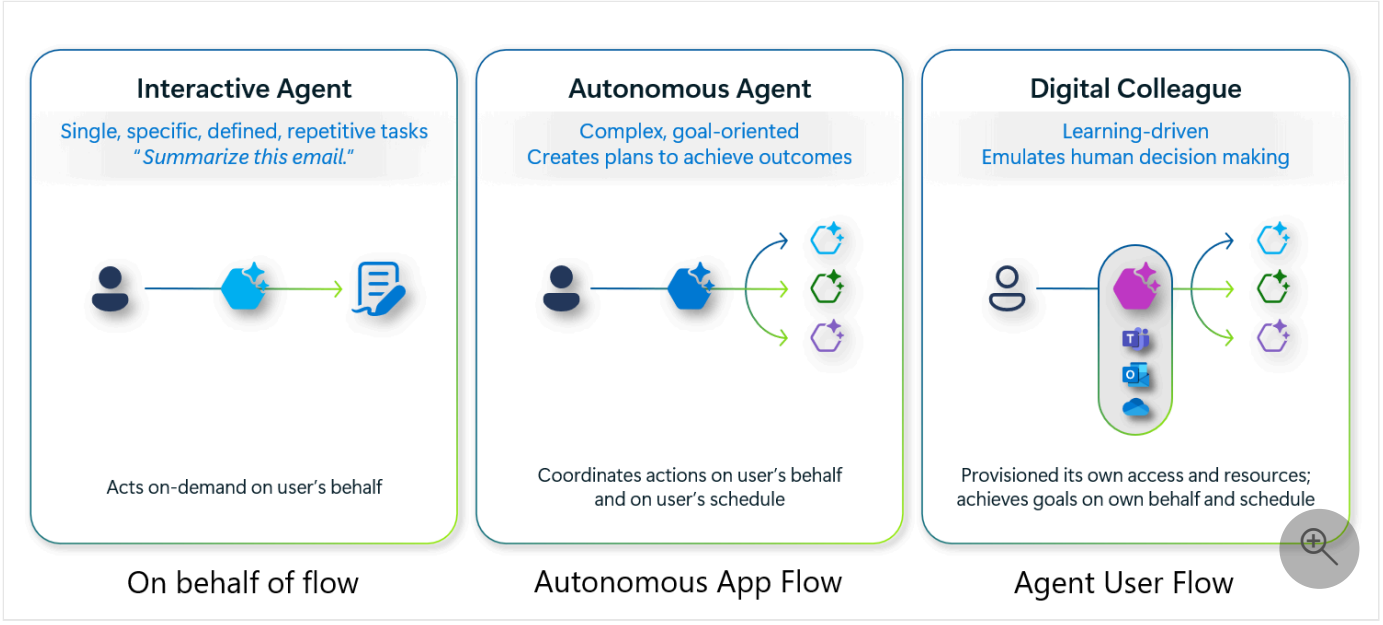
Managed identities are the preferred credential type. In this configuration, the managed identity token serves as the credential for the parent agent identity blueprint, while standard MSI protocols apply for credential acquisition. This integration allows the agent ID to receive the full

benefits of MSI security and management, including automatic credential rotation and secure storage.

> ⚠ **Warning**
>
> Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

Oauth protocols

There are three agent oauth flows:



On behalf of flow | Autonomous App Flow | Agent User Flow

- Agent on-behalf of flow: Agents operating on behalf of regular users (interactive agents).
- Autonomous app flow: App-only operations enable agent identities to act autonomously without user context.
- Agent's user account flow: Agents operating on their own behalf using user principals created specifically for agents.

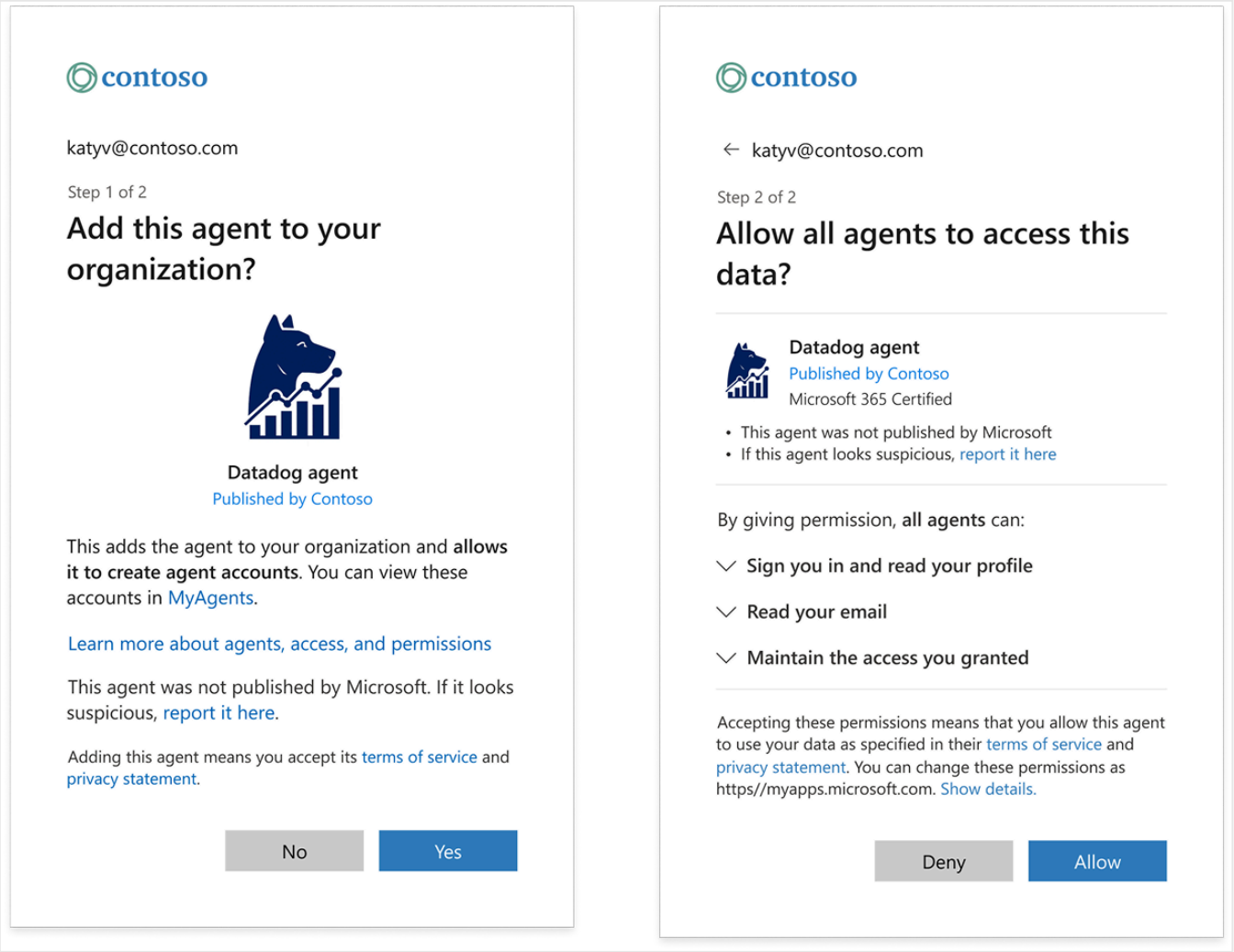
Related content

- Grant agents access to Microsoft 365 resources - admin-focused guidance on consent, manual authorization, and other authorization systems.
- Agent identity token claims - claims reference for tokens issued to agents.

Last updated on 06/11/2026

Microsoft Entra Agent ID sign-in process

This article is intended for nontechnical users who want to learn more about the Microsoft Entra Agent ID sign-in process. When signing in to an agent, product, or service that uses Microsoft Entra Agent ID for authentication and data access, you might see a consent page like the following:



Sign-in to an agent

An AI agent is software that can perform tasks for you or your team. It might summarize information, watch for issues, send alerts, or prepare drafts you can review. Unlike a regular app that waits for you to select or type, an agent can act on its own within the limits you approve.

To sign you in, AI agents must register their product or service with Microsoft by creating an Agent ID. Each AI agent has a unique Agent ID.

An Agent ID allows the AI agent to:

- Sign you in and identify you.

- Request access to data and actions you allow.

Consent pages

You see consent pages because agents need permissions before they can operate in your organization. This process has two steps:

1. Add this agent to your organization. This allows the agent to participate as a member of your organization. This page is only shown the first time someone in your organization logs in to the agent.

2. Allow agent to access this data. This shows the specific types of information or actions the agent is requesting (for example, reading your profile or email). You choose to allow or deny.

This page gives you control over how AI agents use your accounts and data you have access to.

Trust an agent

The consent page includes information to help you assess the trustworthiness of an agent that you sign in to:

- The name of the agent, product, or service
- Whether the agent has completed Microsoft's publisher verification process
- The organization that published the agent
- Whether the agent is built by Microsoft

For more information about the details of the consent page, view this article.

If you did not intentionally sign-in to this agent, always deny access. Never sign-in to apps or agents you're unfamiliar with.

Add agent to your organization

AI agents can't participate in your organization unless a member approves them. When you add an agent to your organization, the agent becomes available to you and other employees or members to use. The agent is able to sign-in other members and request access to their data and information.

Adding an agent to your organization also allows it to create agent accounts. Like human users, AI agents can have accounts in your organization. These accounts can be used to give AI

agents access to access apps, tools, files, and systems in your organization. They also help track what actions an AI agent takes.

When you add an agent, the agent gains the ability to create its own accounts in your organization and use them for access. The agent's accounts are always limited to the permissions and access granted by you or an administrator.

If you're uncertain whether to add an agent to your organization, pause and contact your administrator.

Allow an agent to access data

Approving the agent to access data means the following:

- The agent can use the listed permissions without asking again each time.
- The agent can run in the background to carry out tasks you or your organization configured.

It doesn't give the agent unlimited access. It only gets the specific items shown and can't go outside them unless you or an administrator later grant more permissions.

When in doubt, pause and ask your administrator if permissions are safe to grant to an agent.

Disable an agent or its permissions

You (or an administrator) can remove or disable the agent or its permissions. For more information, see manage agents in end user experience. For global removal, administrators can delete the agent identity from the Microsoft Entra admin center.

Criteria to report a suspicious agent

Report an agent if:

- The publisher name looks wrong or suspicious.
- Permissions seem far too broad for the described task.
- You were prompted unexpectedly and no recent change explains it.
- The agent appears to misuse data.

Use the **report it here** link on the consent screen or notify your security desk.

Need help?

If you're unsure, don't approve. Capture a screenshot, note the agent name, and contact your helpdesk or security team for guidance.

Last updated on 04/03/2026

Conditional Access for agents

Conditional Access for agents is an extension of the Conditional Access policy engine that controls how agents access resources protected by Microsoft Entra ID. It brings together real-time signals such as user's and agent's context, device, location, and risk information to determine when to allow, block, or limit access, or require more verification steps.

Understanding the agent's access pattern helps you target the correct identity. An agent can act on behalf of a signed-in user, use its own agent identity, or use its own agent user account.

Learn about Conditional Access for agents:

- High-level overview of Conditional Access: What is Conditional Access?
- Guide to managing agent identities across your organization: Manage agent identities in your organization.
- How to target agent identities in Conditional Access
- Secure autonomous agents with Conditional Access
- Secure agent users with Microsoft Entra Conditional Access

Requirements and licensing

Microsoft Entra ID Conditional Access for agents requires one of the following license plans:

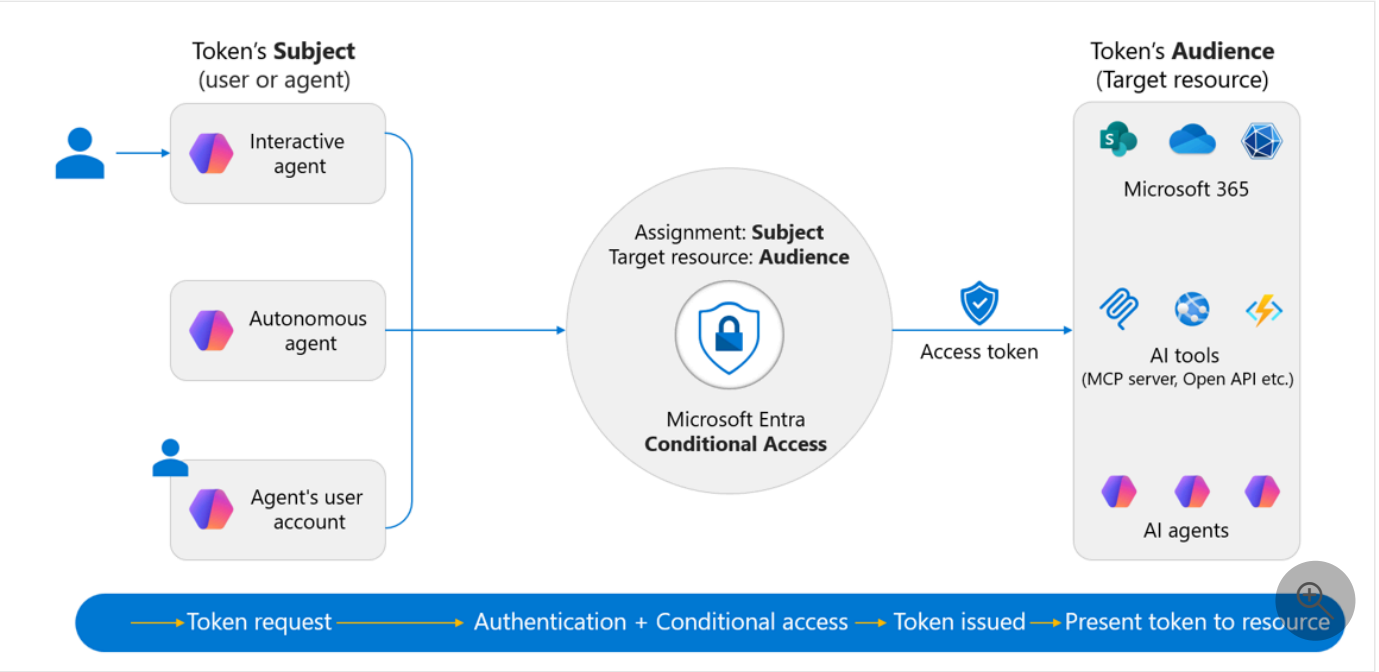
- **Microsoft 365 E7**, which includes Agent 365 and Microsoft Entra Suite.
- **Microsoft Agent 365** license paired with at least Microsoft Entra P1 or Microsoft 365 E3.

For more information, see Microsoft Agent 365 plans and pricing.

How Conditional Access evaluates agent access

To access a resource such as a SharePoint file, MCP server, or Open API service, a user or agent requests an access token from Microsoft Entra ID.

When a Conditional Access policy applies, Microsoft Entra ID evaluates the policy requirements before it issues the token. If the requirements are satisfied, Microsoft Entra ID issues the token. The target resource validates the token and uses its claims to make authorization decisions.



Each access token has one subject and one audience:

- **Subject**: The identity that receives the token.
  - In delegated access, the token represents the user while also identifying the calling application or agent.
  - In application-only access, the agent identity is the subject.
  - In agent-user access, the agent's user account is the subject.
- **Audience**: The target resource that the token is for, which must be registered in Entra ID. If a subject accesses multiple resources, it typically needs a separate token for each resource.

Conditional Access evaluates both the subject that requests access and the audience being accessed. It evaluates policies when Microsoft Entra ID issues or refreshes an access token. Some resources also support Continuous Access Evaluation, which can trigger near-real-time enforcement for specific events.

How Conditional Access decisions are made

Conditional Access policies operate as if-then statements:

- If the conditions defined in a policy are met, the configured access controls are enforced.
- If the required controls are satisfied, access is granted.
- If the required controls are not satisfied, access is denied.

For example, an organization may require multifactor authentication before a user can authorize an agent to access their email. Similarly, an organization may configure a policy to block access from agents identified as high risk.

When Conditional Access is evaluated

Conditional Access is evaluated whenever Microsoft Entra ID issues or refreshes an access token. Some resources also support Continuous Access Evaluation, which can trigger near-real-time enforcement for specific events.

Agent access patterns

Agents can access Microsoft Entra-protected resources using one of the following patterns:

Expand table

If the agent	Access pattern	Policy target	Guidance
Accesses downstream resources for a signed-in user	On-behalf-of (OBO), also known as delegated access	Users and groups	Agent OAuth flows: On-behalf-of
Accesses resources with its own agent identity and no signed-in user	Application-only, also known as client credentials or app-only access	Agent identity or agent identity blueprint	Secure autonomous agents with Conditional Access
Accesses resources through its own user account	Agent-user access	Agent's user account	Secure agents that act as users with Conditional Access

An agent can use more than one access pattern. Create separate policies for each token subject that the agent uses. A policy that targets an agent identity doesn't apply to the agent's user account, and a policy that targets the agent's user account doesn't apply to the agent identity.

Agents that act on behalf of a user

The most common access pattern is the on-behalf-of (OBO) flow. In the OBO flow, a user signs in to an agent application. The agent accesses downstream resources with the user's identity and delegated permissions. For example, when an agent reads your emails, it accesses your mailbox *on your behalf*. For more information about how the OBO flow works for agents, see Agent OAuth flows: On-behalf-of.

> **Note**
>
> The on-behalf-of flow is also known as delegated access. "On-behalf-of" describes the authentication flow, not the type of agent. These interactive agents involve a user interface

for human interaction. Any agent can use this flow when a signed-in user is present and the agent needs to access resources with that user's identity and permissions.

In this flow, the agent can't reuse the user's original token because it was issued for a different audience. Instead, the agent uses the OBO flow to exchange tokens with Microsoft Entra ID, obtaining a new token scoped to the target resource. This token exchange is also evaluated by Conditional Access, letting admins enforce granular controls over which resources agents can access on behalf of the user.

Because the signed-in user is the token subject in this flow, Conditional Access evaluates policies assigned to that **user**, whether the user is targeted directly, through a group, or through another supported user-targeting mechanism. Selecting an agent identity as the policy subject doesn't cover agent requests made on behalf of the user.

Agents that act as applications

Agents might access resources without a signed-in user. In this case the agent accesses the resource with its own identity. This flow is also known as client credentials flow, or app only access. All types of agents might use this flow. For more information about how agents authenticate with their own identity, see Agent OAuth flows: Autonomous apps.

This flow applies in the following common scenarios:

- **Autonomous agents that operate independently** run in the background, respond to events, or run on a schedule.
  - For example, an agent that generates a daily report and sends the result to a group of employees.
  - In this scenario, there's no user present, and the agent operates on its own.
- **Interactive agents that use their own identity** don't always access resources on a user's behalf; sometimes they use their own identity.
  - For example, if an agent calls a backend SMS service that users don't have access to, the OBO flow doesn't apply, and the agent authenticates directly as itself.
- **Agents published on the web for public use** don't authenticate the user or don't support delegating the user's context to corporate resources.

In these scenarios, the agent requests an access token using its own agent identity and credentials managed through the agent identity blueprint. The token is issued to the agent identity (not the user). Therefore, Conditional Access policies are scoped to the agent identity

rather than the user. For step-by-step policy configuration, see Secure autonomous agents with Conditional Access.

Agents that act as users

Sometimes it's not enough for an agent to perform tasks on behalf of a user or operate with its own identity. In certain scenarios, an agent has its own agent's user account that functions as a digital worker with its own mailbox, access to chat, and the ability to participate in collaborative workflows as a team member.

In this model, an admin creates a user account in the directory and links it to the agent's identity. From there, it's like any other user account. Licenses can be assigned to access Microsoft 365 resources such as a mailbox and calendar. The account can be added to administrative units and security groups just like a human user account.

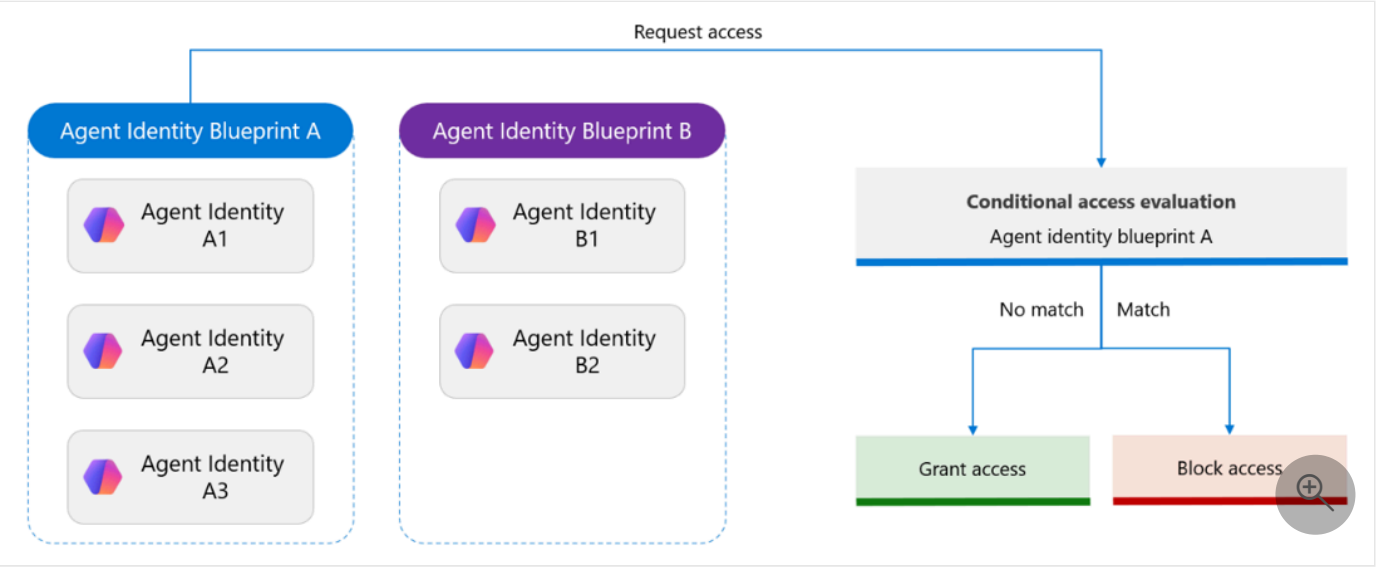
Agents using this flow are also considered autonomous agents as they don't involve a user interface for human interaction. In this model, the access token is issued to the agent's user account (the token subject), and policy is evaluated against the agent's user account, not the agent identity. For step-by-step policy configuration, see Conditional Access for autonomous agents. For more information about the agent user OAuth flow, see Agent user OAuth flow.

Agents running on managed endpoints like Windows 365 Cloud PCs for Agents can also be subject to device compliance and compliant network controls. Use the **Agent execution environments (Preview)** condition to scope these policies to endpoint-based sessions only. For more information, see Require a compliant device for agents' user accounts.

Conditional Access policies and agent identity blueprints

In addition to the specific agent access patterns, you can also select agent identity blueprints to apply Conditional Access policies to a class of agents. An agent identity blueprint defines the configuration and governance model for agent identities created from it. A policy that targets a blueprint applies to all agent identities created from that blueprint, including agent identities created later.

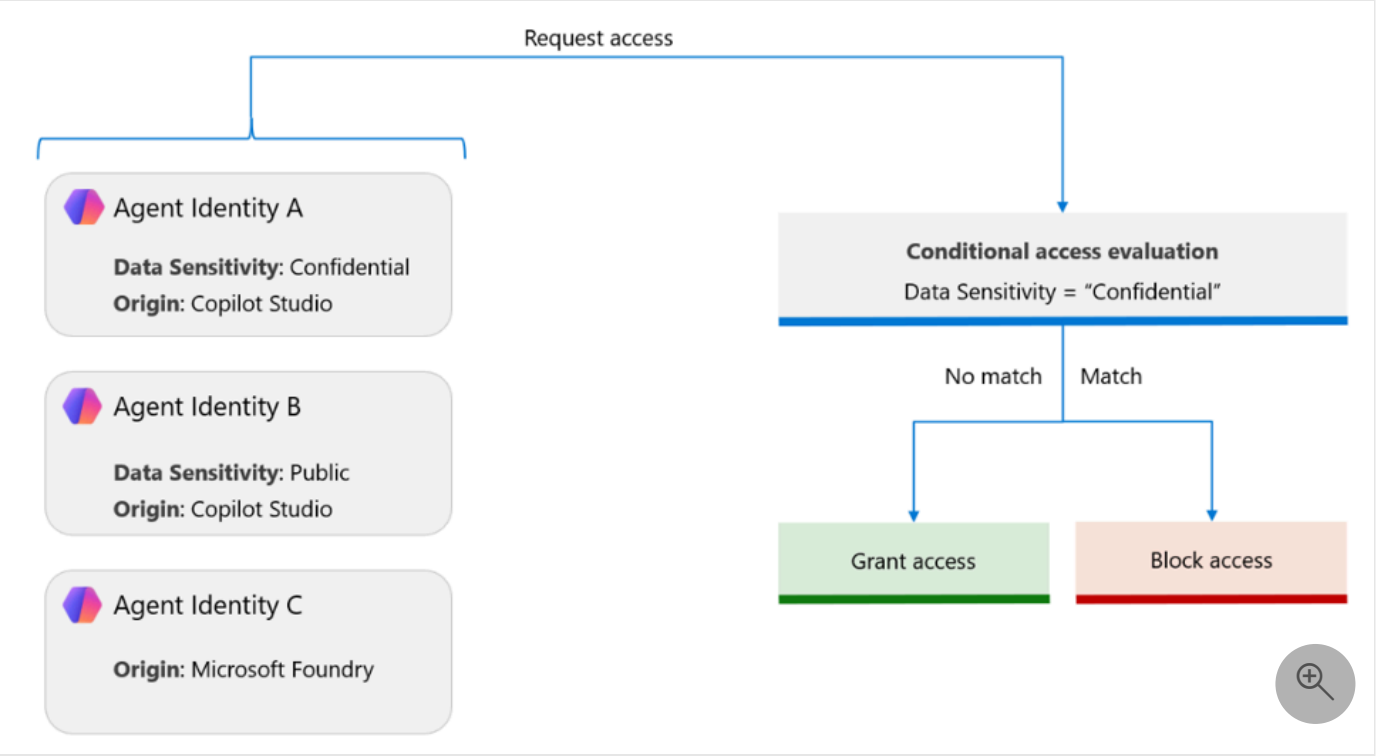
The following diagram shows that only agent identities associated with blueprint "A" are granted access; all other agents are excluded and blocked.



For example, imagine a project where you have several agents, each with its own purpose. Some operate independently, while others collaborate with other agents (A2A) to complete tasks. If they're all created under the same blueprint, a single policy applied to that blueprint enforces consistent access controls across the entire collection.

Attribute-driven Conditional Access

As the number of agent identities grows, individually managing each one across every policy becomes unsustainable. Custom security attributes let you categorize agent identities and resources with business-specific labels, then target those attributes in Conditional Access policies. Policies automatically apply to every agent with matching attributes, including ones added in the future.



For a policy example, see Allow approved agents by using custom security attributes.

Boundaries and limitations

Conditional Access policies don't apply when:

- An agent identity blueprint acquires a token for Microsoft Graph to create an agent identity or agent's user account.
  - Agent blueprints have limited functionality. They can't act independently to access resources and are only involved in creating agent identities and agents' user accounts.
  - Agent tasks are always performed by the agent identity.
- An agent identity blueprint or agent identity performs an intermediate token exchange at the `AAD Token Exchange Endpoint: Public` endpoint (Resource ID: `fb60f99c-7a34-4190-8149-302f77469936`).
  - Tokens scoped to the `AAD Token Exchange Endpoint: Public` can't call Microsoft Graph.
  - Agent flows are protected because Conditional Access protects token acquisition from the agent identity or agent's user account.
- Security defaults are enabled.
- Conditional Access only protects resources secured by Microsoft Entra ID. For example, if an agent accesses resources using an API key, it bypasses the Microsoft Entra ID authentication and token issuance pipeline entirely and Conditional Access policies won't apply to them.

The following configurations aren't currently supported:

- Policies scoped through the "Users" assignment don't apply to agent user accounts. This boundary applies whether the policy targets all users, selected users, groups, directory roles, or external users. To protect agent user accounts, create a policy that targets "Agent Users."
- Scoping a Conditional Access policy to include or exclude agent's user account based on their group membership.
- A Conditional Access policy targeting agent identities won't apply to the agent's user account.
- A Conditional Access policy targeting agent identities using agent identity blueprint covers only the agent identity, not the agent's user account.

Instead, to scope policies to agent users, under **Assignments** > **Users, agents, or workload identities**, select **Agents**, and then target all agent users or specific agent users.

Related content

- Target agent identities in Conditional Access policies
- Secure autonomous agents with Conditional Access
- Secure agents that act as users with Conditional Access

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/30/2026

Governing Agent Identities

Microsoft Entra allows you to ensure that the right people have the right access to the right apps and services at the right time. With the addition of the Microsoft agent identity platform, managing the access rights of agents in the same way is just as important in the governance lifecycle of your organization's identities. The Microsoft agent identity platform introduces the concept of Agent Identities (IDs). Agent identities are accounts within Microsoft Entra ID that provide unique identification and authentication capabilities for AI agents.

This allows agent identities to be governed with Microsoft Entra features in the same style as you would govern human identities. With Agent identities, you can govern and manage the identity and access lifecycle of agents, ensuring the agents have a responsible person providing oversight throughout the agent lifecycle and agent's access does not persist longer than it is needed. This article provides an overview of how Microsoft Entra can be utilized to govern agent identities.

License requirements

Using Microsoft Entra ID Governance for agent identities requires one of the following license plans:

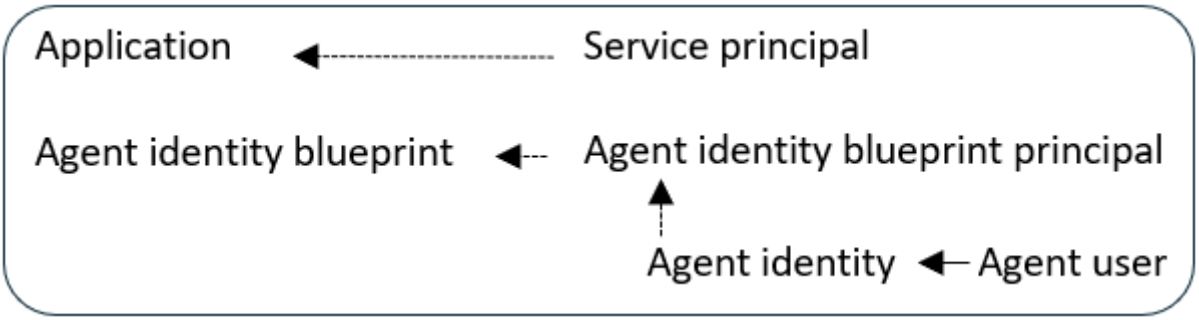
- **Microsoft 365 E7**, which includes Agent 365 and Microsoft Entra Suite, to provide governance of user and agent identities.
- **Microsoft Agent 365** license paired with at least Microsoft Entra P1 or Microsoft 365 E3.

For more information, see Microsoft Agent 365 plans and pricing. For the full list of agent-specific capabilities, refer to the **Microsoft Agent 365** column in the Microsoft Entra ID Governance licensing table.

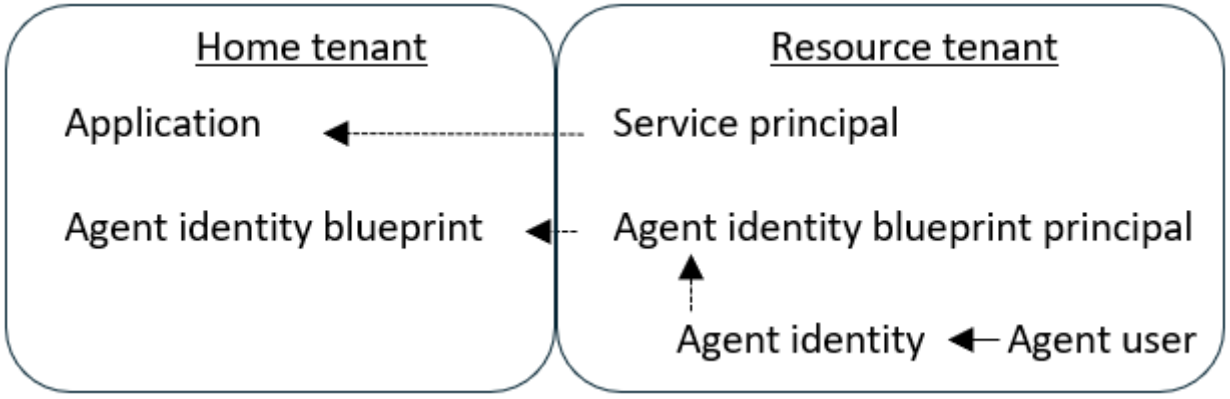
Agent identities basics

Historically, AI agents would rely upon tools to interact with various applications and systems, and each of those tools would have their own identities in those applications and systems. Some of those tools would use service principals to authenticate to Microsoft services via Microsoft Graph or Microsoft Azure APIs. Microsoft Entra Agent ID introduces support for identities for the agents themselves, with four new types of object: agent identity blueprint, agent identity blueprint principal, agent identity, and agent user. Through the agent identity blueprint, the

agent can create one or more agent identities, and optionally an agent user for each agent identity. Each agent identity and agent user can have distinct access rights.



For a multitenant-capable agent, an agent identity blueprint principal can be brought into the tenant with resources so it can create agent identities in that tenant, similar to how a multitenant application can have a service principal in each tenant.



The agent identity and the agent user allow AI agents to take on digital identities within Microsoft Entra. Once agent identities are created, these agent identities are able to be governed using lifecycle and access features. Sponsors can be assigned to agent identities after creation. Sponsors of agent identities are human users accountable for making decisions about its lifecycle and access. For more information about the role of a sponsor of agent identities, see: Administrative relationships for agent IDs.

Agent identities in other Microsoft products and portals

- **Microsoft Foundry** automatically provisions and manages agent identities throughout the agent lifecycle. When the first agent in a Foundry project is created, Microsoft Foundry provisions a default agent identity blueprint and a default agent identity for the project, and agents in the project authenticate by using the shared project's agent identity. Publishing an agent automatically creates a dedicated agent identity blueprint and agent identity, and the agent will authenticate by using the unique agent identity. Foundry supports use of the

agent identity for authentication in Model Context Protocol (MCP) and Agent-to-Agent (A2A) tools. For more information, see Agent identity concepts in Microsoft Foundry.

- You can configure an **Azure App Service or Azure Functions app** to use the Microsoft Entra agent identity platform to securely connect to resources as an agent. For more information, see How to use an agent identity in App Service and Azure Functions.
- Agents created in **Microsoft Copilot Studio** can be configured to automatically be assigned to an agent identity. When an agent identity is first created in a Power Platform environment after enabling this setting, a Microsoft Copilot Studio agent identity blueprint, and an agent identity blueprint principal, are automatically created. For more information, see Automatically create Entra agent identities for Copilot Studio agents (preview).
- For agents in the **Microsoft Teams** platform, a developer can create and manage agent identity blueprints in the Developer Portal for Teams. For more information, see Manage your apps in Developer Portal.
- **Microsoft Agent 365** gives each AI agent its own Microsoft Entra Agent ID, for identity, lifecycle, and access management. For more information, see Agent identity platform capabilities for Agent 365.

Assigning access to agent identities

When created, agent identities have limited permissions, such as OAuth 2 delegated permission scopes inherited from their parent agent identity blueprint. In addition, agent identities can have resource access assigned to them directly via access packages. Agents can request an access package for own agent IDs, or have their owner or sponsor request one on their behalf. With access packages, you're able to assign agent identities access to the following resources:

- Security Group memberships
- Application OAuth API permissions, including Graph application permissions
- Microsoft Entra roles

To use access packages for agent identities, configure an access package with the required policy settings. When creating an access package assignment policy, in the **Who can get access** section, select **For users, service principals, and agent identities in your directory**, and then select the option of **All agents**.

> ⓘ **Note**

If your agents aren't using Microsoft Entra agent IDs, then also create an access package assignment policy with the option **All Service principals** to allow service principals in your directory to be able to request this access package.

Agents can then be assigned access packages through three different request pathways.

- The agent identity itself can programmatically request an access package when needed for its operations, by creating an accessPackageAssignmentRequest.
- The agent's sponsor can request access on behalf of the agent ID, providing human oversight in the access request process. For more information, see Request an access package on behalf of an agent identity.
- An administrator can directly assign the agent identity or agent user to the access package.

After submission, the access request is routed to designated approvers based on the access package configuration.

When the agent identity has received an access package assignment with an expiry date, and if a sponsor is set on the agent identity, as the expiry date approaches, the sponsor receives notifications about the pending expiration. The sponsor then has two options: they can request an extension of the access package (if permitted by policy), or they can allow the access package assignment to expire. If the sponsor requests an extension, this request can trigger a new approval cycle, where approvers again confirm whether continued access is appropriate. If the sponsor takes no action, the access package assignment automatically expires on its end date, and the agent identity loses access to the target resources.

For a guide on creating an access package for agents, see: access packages for agent identities in Microsoft Entra Agent ID. For a guide on assigning identities to an existing access package, see: View, add, and remove assignments for an access package in entitlement management.

Conditional Access for agent identities

Entitlement management controls which resources an agent identity can be assigned. To also control the conditions under which an agent identity can access those resources, you can apply Conditional Access to agent identities. Conditional Access policies evaluate agent context and risk—including agent identity risk from Microsoft Entra ID Protection—before granting access. You can apply these policies at the agent identity blueprint level so that all agent identities created from a blueprint inherit them.

For more information, see Conditional Access for agents and Identity Protection for agents.

Management of agents

When agent identities are created, owners and sponsors of the agent can manually make decisions for the agent identity via both the My Account portal, and the My Access Portal.

From the My Account portal, Sponsors and Owners are able to manage the identity lifecycle of agents such as enabling and disabling the agent. You are also able to see information about its access, activity, and lifecycle. For more information about Managing agents, see: Manage Agents in Microsoft Entra ID.

From the My Access portal, Sponsors and Owners of agent identities are able to request access packages on behalf of their agent identities. For a guide on requesting access packages, see: Request an access package on behalf of an agent identity.

Agent identities sponsor administration

One of the most important parts of governing agent identities is making sure that a delegated human user is always assigned to make sure the agent identity's access to resources are current. If the sponsor is leaving the organization, sponsorship of the agent identities is automatically transferred to their manager. With sponsorship transferred, there's always a human user accountable for managing the access and lifecycle of the agent identities. Microsoft Entra ID Governance features can help streamline this process within your organization. Lifecycle workflows include multiple tasks around notifying cosponsors, and managers of sponsors, of impending sponsorship changes. For a guide on setting up a workflow for agent identities sponsors, see: Agent identity sponsor tasks in Lifecycle Workflows.

Related content

- Manage agent identities in your organization - Overview of agent identity management across the full lifecycle.
- Agent identity sponsor tasks in Lifecycle Workflows - Automate sponsor change notifications using Lifecycle Workflows.
- Manage agents in end user experience - Sponsors can manage their agents from the My Account portal.
- What is entitlement management?
- What is Microsoft Entra ID Governance?

ⓘ**Note:** The author created this article with assistance from AI. Learn more

Last updated on 06/16/2026

Access packages for Agent identities

Microsoft Entra entitlement management provides access packages as a governance mechanism. Access packages ensure that agent access assignments are intentional, auditable, and time-bound. Access packages represent a structured approach to managing agent identity permissions, contrasting with ad-hoc permission assignments that might lack appropriate governance controls. Access packages enable standardized access for many AI Agents with the same access needs, for example, a fleet of customer support AI Agents. Through access packages, organizations can establish consistent governance practices for agent identity, agent's user account, and service principal access to resources. For more information, see Governing agent identities.

Prerequisites

Before creating an access package, confirm the following prerequisites are met in your organization:

1. Agents are using Microsoft Entra Agent ID agent identities, or service principals, for authorization to access resources.

2. The authorization is one of:

    - Agents need their identity to be assigned OAuth *application permissions* for a target resource, such as Microsoft Graph or an application, to be able to access a target resource's APIs.
    - Agents need their identity to be assigned OAuth *delegated permissions* for a target resource, such as Microsoft Graph or an application, to be able to assist a user when accessing a target resource's APIs.
    - Agents need their identity to be assigned as members of groups.
    - Agents need their identity to be assigned to directory roles. The allowable roles are listed in Microsoft Entra roles allowed for agents.

3. You have or can create an entitlement management catalog suitable to hold those resources. The access package that you'll be creating, and any resources included in it, will be added to the catalog. For more information, see create a catalog.

    > ⓘ **Note**

If you'll be adding OAuth API permissions or directory roles to the access package as resource roles, then the catalog will be marked as privileged when they are added to the access package.

Create an access package for agent identities

To use access packages for agents, the IT admin first configures a new access package with the relevant resources, including Entra roles, group memberships, and OAuth permission grants to application APIs. Then the admin configures in the access package the required policy settings. These settings define who can get access, who can request access, approvals, access expiration, and extension.

As agent identities and service principals can't be added through access packages to application roles, SAP roles, or SharePoint Online site roles, you won't be able to reuse an existing access package that contains any of those resource roles. Instead, create a new access package.

1. Sign in to the Microsoft Entra admin center as at least an Identity Governance Administrator.

   > **Tip**
   >
   > If you'll be adding OAuth API permissions or directory roles to the access package as resource roles, then you will need to be a Global Administrator.

2. Browse to **ID Governance** > **Entitlement management** > **Access packages**.

3. Select **New access package**.

4. On the **Basics** tab, you give the access package a name, and description and specify which catalog to create the access package in. In the **Catalog** dropdown list, select the catalog where you want to put the access package.

5. Select **Next: Resource roles**. On the **Resource roles** tab, you select the resource roles to include in the access package. Access packages for agent identities can have security group memberships, directory roles, or API permissions as resource roles. For more information, see add a group, add a Microsoft Entra role, and add an API permission. Don't add application roles, SAP roles, or SharePoint Online site roles to an access package for agent identities.

> **Tip**
>
> If you're not sure which resource roles to include, you can skip adding them while creating the access package, and then **add them** later. For API permissions, resource ownership validation occurs both when you onboard permissions to an access package and when you add them to an access package. This validation helps ensure that only authorized resource owners can introduce or expand access to API permissions through access packages.

6. Select **Next: Requests**. On the **Requests** tab, you create the first policy to specify who can request the access package. in the **Who can get access** section, select **For users, service principals, and agent identities in your directory**. In **Select specific scope**, select the option of **All agents**.

   > **Note**
   >
   > If your agents are using service principals rather than Microsoft Entra agent identities, then also create an access package assignment policy with the option **All Service principals** to allow service principals in your directory to be able to request this access package.

7. Determine how many approval stages are needed. Set the **How many stages** toggle to **1** for single-stage approval, **2** for two-stage approval, or **3** for three-stage approval. Then configure approval stages and who the approvers should be. For more information, see single stage approval.

8. Once you have specified each approval stage, select **Next: Requestor Information**.

9. Select **Next: Lifecycle**. Specify how long an access package assignment should remain before it expires.

10. Select **Next: Rules**.

11. Select **Next: Review + create**. On the **Review + create** tab, you can review your settings and check for any validation errors.

12. Select **Create** to create the access package and its initial policy.

In addition to using the Microsoft Entra Admin Center, you can also create an access package programmatically, through Microsoft Graph, and through the PowerShell cmdlets for Microsoft Graph. For more information, see Create an access package programmatically.

Access request and approval process

Agents can then be assigned access packages through three different request pathways.

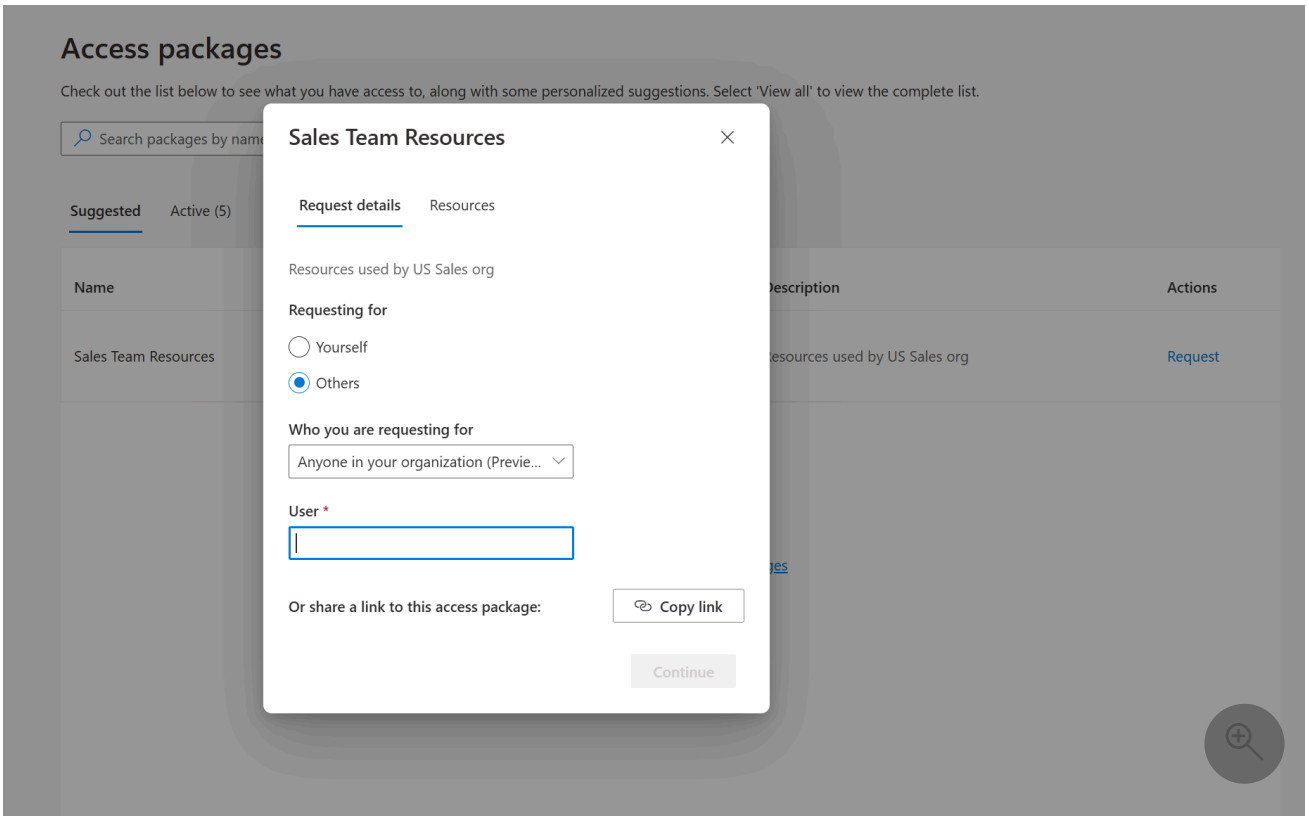
- The agent identity itself can programmatically request an access package when needed for its operations, by creating an accessPackageAssignmentRequest.
- The agent's sponsor can request access on behalf of the agent ID, providing human oversight in the access request process. For more information, see Request an access package on behalf of an agent identity.
- An administrator can directly assign the agent identity or agent's user account to the access package.

After submission, the access request is routed to designated approvers based on the access package policy configuration.

Request an access package on behalf of another user

Users with the appropriate permissions can request access packages on behalf of another user in their organization.

1. Sign in to the My Access portal at https://myaccess.microsoft.com. For US Government, the domain in the My Access portal link is `myaccess.microsoft.us`.
2. On the My Access Portal page, select **Access packages**.
3. On the Access packages page, locate the access package you want to request for a direct report and select **Request**.
4. On the Request pane under **Request details**, select requesting for **Others**.
5. Select **Anyone in your organization** and search for the user you want to submit the request for.



6. Continue with the remaining request steps to complete the access package request on behalf of the user.

Access assignment lifecycle

Once an approver accepts the access package assignment request, the agent identity receives time-bound access to the specified resources. The access is granted according to the resource roles defined in the access package. This establishes a clear start and end date for the access the agent might need.

If the assignment is to an agent identity, and a sponsor is set on the agent identity, as the expiry date approaches, the sponsor receives notifications about the pending expiration. The sponsor then has two options: they can request an extension of the access package (if permitted by policy), or they can allow the access package assignment to expire.

If the sponsor requests an extension, this request can trigger a new approval cycle, where approvers again confirm whether continued access is appropriate. If the sponsor takes no action, the access package assignment automatically expires on its end date, and the agent identity loses access to the target resources.

Related content

- Governing agent identities

- Create an access package in entitlement management
- Conditional access for agent identities

Last updated on 08/31/2026

ID Protection for agents

As organizations adopt, build, and deploy autonomous AI agents, the need to monitor and protect those agents becomes critical. Microsoft Entra ID Protection helps protect your organization by automatically detecting and responding to identity-based risks on agents that have agent identities provided by Microsoft Entra Agent ID.

Prerequisites

Roles

To use our Risky Agent reports, you must have one of the following administrator roles assigned.

- Security Administrator
- Security Operator
- Security Reader

To configure policies that use Agent Risk as a condition, you must have the Conditional Access Administrator role assigned.

Licensing

Starting soon, ID Protection for agents will require a Microsoft Agent 365 license to extend protection to agents through Microsoft Entra Agent ID.

How it works

Because agents can operate autonomously and on behalf of a user, they can display unique sign-in behavior. Agents can take initiative, interact with sensitive data, and operate at scale. Microsoft Entra ID Protection for agents identifies and mitigates risks associated with these capabilities. **Learning Mode** automatically suppresses behavioral alerts for agents that lack sufficient activity history, preventing false positives during onboarding and after periods of inactivity. A separate detection runs in parallel to ensure genuinely malicious early-life behavior is still caught. Once an agent exhibits suspicious behavior, ID Protection flags the activity as risky.

Activities contributing to risk

The following table provides the anomalous activities that can contribute to the agent being flagged for risk. At this time, all risk detections for risky agents are offline.

> **Note**
>
> In on-behalf-of (OBO) flows, where an agent acts using a user's delegated permissions, risky activity is attributed to the **user** rather than the agent. This approach targets remediation at the compromised user session without disrupting the agent for other users. Unless noted otherwise, risk detections in this table apply only to autonomous agent activity.

Expand table

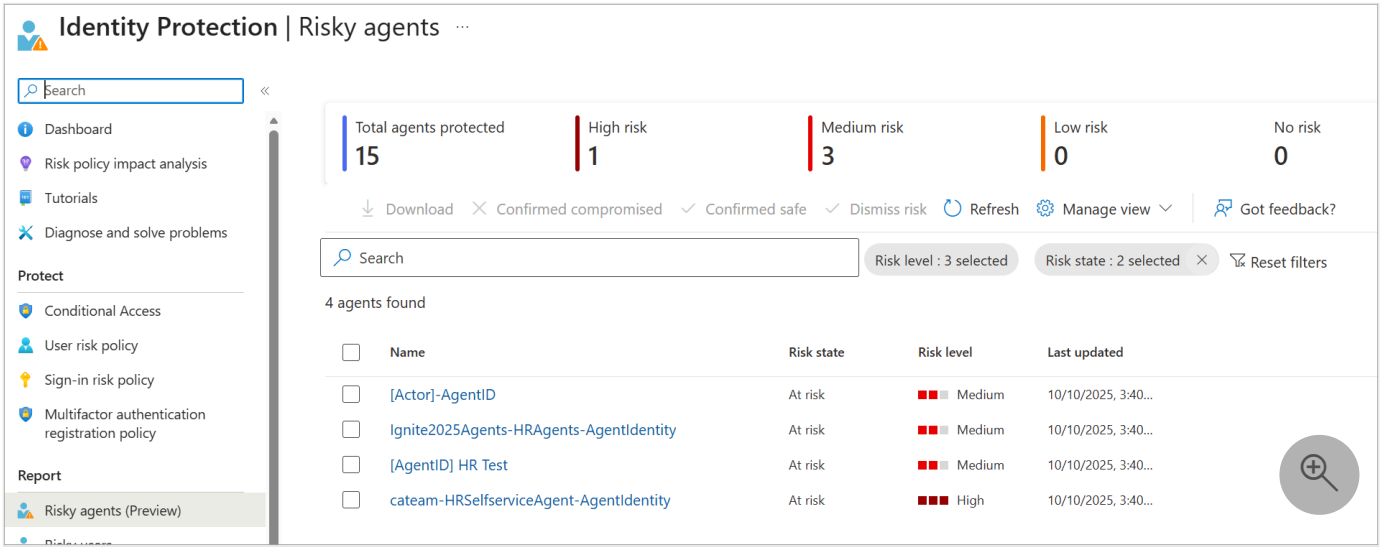
Agent risk detection	Description	riskEventType
Confirmed compromised	Admin confirmed agent compromised	adminConfirmedAgentCompromised
Early life malicious activity	Newly created agent immediately exhibited multiple suspicious behavior patterns, acting like an attacker.	earlyLifeMaliciousActivity
Entra Directory Reconnaissance	Agent performed suspicious reconnaissance or high-risk directory operations.	entraDirectoryReconnaissance
Failed access attempt	Agent attempted and failed to access resources for which it isn't authorized. This detection can indicate an attacker is attempting to replay an agent's token against an unauthorized resource.	failedAccessAttempt
Microsoft Entra threat intelligence	Microsoft identified activity that is consistent with known attack patterns based on its internal and external threat intelligence sources.	threatIntelligenceAccount
Sign-in spike	Agent made a higher number of sign-ins compared to its usual sign-in frequency. This spike can be an indicator that an attacker is using automation or a toolkit.	signInSpike
Suspicious credential usage	This detection flags when new credentials are added to agent blueprints and then actually used.	suspiciousCredentialUsage
Unfamiliar resource access	Agent targeted resources that it doesn't usually access. This detection can mean that an	unfamiliarResourceAccess

Agent risk detection	Description	riskEventType
	attacker is trying to access sensitive resources beyond the agent's intended purpose.	

View the risky agent report

The **Risky Agents** report provides a list of all agents that were flagged for risky behavior. A summary of risky agents appears on the ID Protection Dashboard. This snapshot view provides an overview of the number of agents flagged for risk by risk level. Select **View risky agents** to open the full report.

You can also navigate directly to the **Risky Agents** report from the ID Protection navigation menu. Filter and sort to find specific agents, risk states, or risk levels.



You can take action on agents directly from the report, including:

- **Confirm compromise**: Select after manual investigation or automated detection confirms the account is compromised. This step is useful as part of incident response to prevent further damage. Confirm compromise automatically sets the risk level to High and creates an event in the agent's **Risk detections**. This action triggers risk-based Conditional Access policies that are configured to block access on High Agent Risk.
- **Confirm safe**: Marks the user as safe after investigation and clears any active risk state for that user by setting risk level to None. Use this option when you want to mark a false positive and for the system to avoid flagging similar activity.
- **Dismiss risk**: Tells the system that the detected risk for an agent is no longer relevant after investigation, or is a benign true positive where you want the system to continue to flag similar activity.

- **Disable**: Prevents all sign-ins for that agent across Microsoft Entra ID and connected apps.

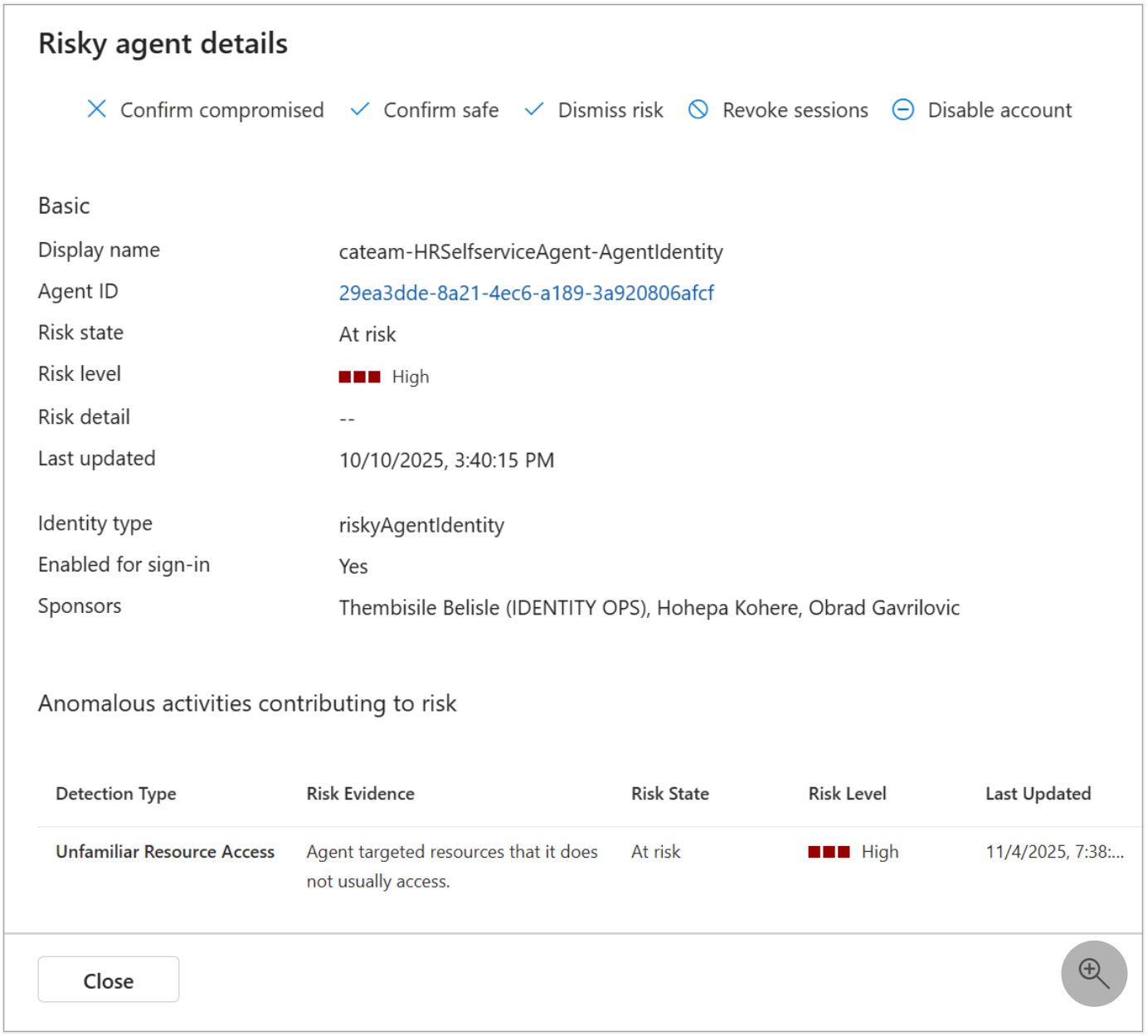
View the risky agent details

In the risky agent report, select an entry to view the full details including the corresponding risk detections for that agent. Just like with all ID Protection risk reports, you can take action on the agent directly from the report or from the details view.

Risky agent details include:

- Agent display name and ID
- Risk state and risk level
- Agent type and sponsors (if specified)

You can also navigate to the **Risk Detections** report and select the **Agent detections** tab to view a full list of the detection risk events from up to the past 90 days. Risk detections are retained for up to 90 days for investigation purposes.



Risk-based Conditional Access for agents

Use Conditional Access for agents to set risk policies that block risky agents from accessing resources or other agents. Use this Conditional Access template to simplify deploying this policy in your organization.

Microsoft Graph

You can also query risky agents using the Microsoft Graph API. There are two collections in the ID Protection APIs.

- `riskyAgents`
- `agentRiskDetections`

Export risk data

You can export data by configuring diagnostic settings in Microsoft Entra ID to send risk data to a Log Analytics workspace, archive it to a storage account, stream it to an event hub, or send it to a SIEM solution.

Related content

- Manage agent identities in your organization - Overview of agent identity management across the full lifecycle.
- Conditional Access for Agent ID - Enforce Zero Trust policies across agent identity token acquisition flows.
- Governing Agent Identities - Preventive governance approaches including access packages and sponsor oversight.

Last updated on 06/17/2026

Learn about Secure Web And AI Gateway for Microsoft Copilot Studio agents

As organizations adopt autonomous and interactive AI agents to perform tasks previously handled by humans, administrators need visibility and control over agent network activity. Global Secure Access for agents provides network security controls for Microsoft Copilot Studio agents, enabling you to apply the same security policies to agents that you use for users.

With Global Secure Access for agents, you can regulate how agents use knowledge, tools, and actions to access external resources. You can apply network security policies including web content filtering, threat intelligence filtering, and network file filtering to agent traffic.

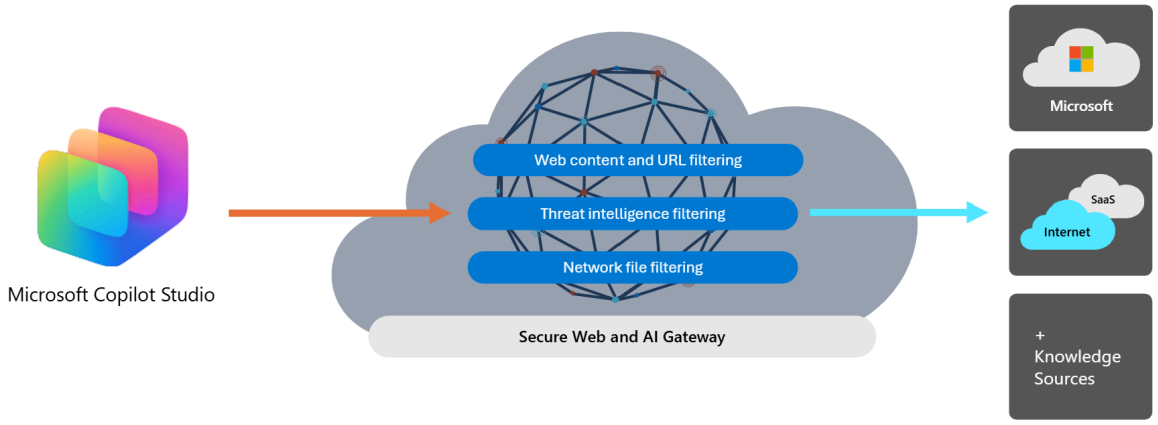
How network security for Copilot Studio agents works

To enforce network security controls on Copilot Studio agents, you forward agent traffic to Global Secure Access's globally distributed proxy service. You enable traffic forwarding in the Power Platform Admin Center on a per-environment or per-environment-group basis.

Agent traffic forwarding applies to multiple traffic types, including:

- HTTP Node traffic
- Custom connectors
- MCP Server Connector

Once agent traffic is forwarded to Global Secure Access, you can apply security policies to the traffic. The service evaluates agent traffic against your configured security policies, similar to how it evaluates user traffic.



Security policies for agents

Security policies for agents are configured using the baseline profile in Global Secure Access. The baseline profile applies security policies at the tenant level, ensuring consistent security controls across all agent traffic.

How to get started

Microsoft Entra Agent ID is a product within Microsoft Entra that provides the platform for creating and managing agent identities and agent identity blueprints. Agent ID is available for all Microsoft Entra customers.

Microsoft Agent 365 enables agents to operate across Microsoft 365 services and enterprise workflows, which requires a **Microsoft Agent 365** license for each user. For pricing details, see Microsoft Agent 365 plans and pricing.

Extending Microsoft Entra security features to agents requires **Microsoft 365 E7** (includes Agent 365 and Microsoft Entra Suite) or **Microsoft 365 E5** paired with a **Microsoft Agent 365** license. Customers without E5 or E7 can use the following standalone licensing options with a **Microsoft Agent 365** license:

- **Conditional Access for agents**: Microsoft Entra ID P1
- **ID Protection for agents**: Microsoft Entra ID P2
- **ID Governance for agents**: Microsoft Entra ID P1

- **Network controls for agents**: Microsoft Entra Internet Access, included in Microsoft Entra Suite or licensed separately. For more information, see What is Global Secure Access.

Next steps

- Configure network security for Microsoft Copilot Studio agents
- Learn about Global Secure Access
- Learn about traffic forwarding profiles

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/15/2026

Microsoft Entra Agent ID logs

As the usage, capabilities, and scope of AI agents grow, it's important to understand how activities associated with agent identities are logged in Microsoft Entra ID. As an IT admin, you need to know what information is available in the audit and sign-in logs and how to use that information to monitor agent activity. The audit and sign-in logs are helpful when you need to investigate the following types of scenarios:

- Sign-in logs for agents or agent related traffic
- Audit logs for when an agent identity or agent's user account is either the initiator or performer of an operation in my tenant

This article describes how agent identity activities are logged in Microsoft Entra ID and how to access those logs using the Microsoft Entra admin center and Microsoft Graph.

Audit logs

Agent activity is logged under the base identity type from where the activity originates. There are currently three agent identity types that appear in the audit logs, each correlating to an existing identity type in Microsoft Entra ID.

- **Agent identity blueprint** activities appear as *application events* (for example, "Add application" or "Delete application").
- **Agent identity** activities appear as *service principal events* (for example, "Add service principal" or "Update service principal").
- **Agent's user account** activities appear as *user events* (for example, "Add user").

To identify whether an audit event involves an agent identity, check the `agentType` property on the `initiatedBy`, `performedBy`, and `targetResources` fields. An agent identity and an agent's user account can both be represented in the `initiatedBy` and `performedBy` events. A value other than `notAgentic` indicates agent involvement.

agentType

The `agentType` value further clarifies agent identity actions. The `agentType` property indicates whether the identity involved in an audit event is an agent identity, agent identity blueprint, or an agent's user account.

The `agentType` property appears on the following resources:

- `auditAppIdentity`
- `auditUserIdentity`
- `targetResource`
- `auditActivityPerformer`

Expand table

Value	Description
`notAgentic`	The identity isn't an agent. This is a standard app, user, or service principal.
`agenticApp`	An agent identity blueprint, which is the template or definition for an agent. Analogous to an app registration.
`agenticAppInstance`	An agent identity, which is a specific running instance of an agent. Analogous to a service principal.
`agentIdentityBlueprintPrincipal`	The blueprint principal itself, which is the service principal that represents the blueprint.
`agentIDuser`	An agent's user account. This identity lets an agent act as a user with user-delegated permissions.
`unknownFutureValue`	Evolvable enumeration sentinel value. Don't use.

The following table summarizes how these agent activities and values map together and appear in the audit logs:

Expand table

Agent action	Audit activity	agentType value
Create an agent identity blueprint	Add application	`agenticApp`
Create an agent identity	Add service principal	`agenticAppInstance`
Create an agent's user account	Add user	`agentIDuser`
Update an agent identity blueprint	Update application	`agenticApp`
Update an agent identity	Update service principal	`agenticAppInstance`
Delete an agent identity blueprint	Delete application	`agenticApp`
Delete an agent identity	Delete service principal	`agenticAppInstance`

> **Tip**
>
> To receive `agentIdentityBlueprintPrincipal` and `agentIDuser` values in Microsoft Graph responses, include the `Prefer: include-unknown-enum-members` request header. These values are part of an **evolvable enumeration**.

blueprintId

The `blueprintId` property is the object ID of the agentIdentityBlueprint associated with the agent. Use this property to correlate an agent identity (instance) back to its blueprint (template). This property appears on `auditAppIdentity`, targetResource, and `auditActivityPerformer`.

The relationship between `blueprintId` and the agent identity is similar to the relationship between an app registration and its service principal. Multiple agent identities can share the same blueprint.

What changed in the audit log schema

The following changes to the audit log schema enable agent identity tracking:

- The `initiatedBy.app` property now uses the `auditAppIdentity` resource type, which includes `agentType` and `blueprintId` alongside the existing `appId`, `displayName`, `servicePrincipalId`, and `servicePrincipalName` properties.
- The targetResource resource type now includes `agentType` and `blueprintId` properties.
- The auditUserIdentity resource type now includes an `agentType` property.
- A new `auditActivityPerformer` resource type provides `agentType`, `appId`, and `blueprintId` for the actor of an audit event.

Sign-in logs

The `agentSignIn` sign-in event type contains properties about the agent, such as if the agent is an app or an instance of an app. Because agents can sign in with either user-delegated or app-only permissions, their sign-ins might appear across each of the four sign-in log types.

The `agentSignIn` sign-in event type is available in the Microsoft Entra admin center and the Microsoft Graph API.

Microsoft Entra admin center

1. Sign in to the Microsoft Entra admin center as at least a Reports Reader.
2. Browse to **Entra ID** > **Monitoring & health** > **Sign-in logs**.
3. Use the following filter options to view agent sign-ins:

    - **Agent type**: Choose from **Agent ID user**, **Agent Identity**, **Agent Identity Blueprint** or **Not Agentic**
    - **Is Agent**: Choose from **No** or **Yes**

Last updated on 05/01/2026

How are agent identities created?

Administrators who work in identity and access management and security need clear visibility into how AI agent identities are added to a tenant, how they're added, and what they do when they're there. This article describes the various channels through which agent identities can be created in your Microsoft Entra tenant, the roles and permissions required for each channel, and strategies for monitoring and controlling agent identity creation.

Overview of creation channels

Agent identity blueprints can enter your tenant through multiple channels. Each channel implies different monitoring and control points:

Expand table

Channel	Typical Actors	Can be controlled by
*Microsoft Entra admin center / Azure portal	Developers, Administrators	Role assignments
Microsoft Graph API	Automation, DevOps pipelines, integration services	Microsoft Graph permission grants
CLI, PowerShell, Infrastructure as Code	DevOps, Administrators	Role assignments
*Microsoft product integrations	Users of Microsoft agent platforms	Administrative controls for each product
*Microsoft Entra ID consent experience	Employees, members of organization	App consent policies

- Indicates a managed experience channel

When an agent identity blueprint is added to your tenant through one of the managed experiences, an **agent identity blueprint principal** object is also created in the tenant. This principal is assigned privileges to create agent identities and agents' user accounts in the tenant. If an agent identity blueprint is added through the Microsoft Graph API, CLI, PowerShell, or Infrastructure as Code tools channels, the associated principal must be manually created.

In addition to the channels listed in the previous table, any system with an agent identity blueprint principal in your tenant becomes a channel for the agent identity and agent's user

account creation:

Expand table

Channel	Typical Actors	Can be controlled by
Agent identity blueprint principals	Microsoft-built agents and non-Microsoft agents added to your tenant	Microsoft Entra application permissions

The following sections provide more details on each of these channels.

Agent identity creation workflows

The following table gives three examples of ways that agent identities are added to a tenant, using one of the channels described in the previous sections.

Expand table

	Built in Copilot Studio	Developer built	Non-Microsoft built
Agent blueprint source	Automatically added when Copilot Studio is enabled	Developer creates via Microsoft Entra admin center	Added when employee consents to agent
Who creates the agent	Employee in Copilot Studio	Developer via code using an agent blueprint	Non-Microsoft agent (after purchase/consent)
Identity creation	Copilot Studio creates identity in tenant	Code creates identity via Microsoft Graph	Agent creates identity via Microsoft Graph

Microsoft Entra admin center, Microsoft Graph, and CLI tools

Developers and other members of your organization can be given rights to create agent identity blueprints via the Microsoft Entra admin center, Azure portal, Microsoft Graph PowerShell / CLI, Bicep / ARM templates, and other tools. These tools all create agent identity blueprints by calling Microsoft Graph APIs.

Creating an agent identity blueprint with these tools requires one of the following permissions:

Expand table

Scenario	Permissions required
User creates blueprint via Microsoft Entra admin center, CLIs	User must be assigned **Agent ID Developer** or **Agent ID Administrator** roles.
Client creates blueprint via Microsoft Graph, using delegated permissions	User must have the roles mentioned in the previous row. Client must be granted *at least* the **AgentIdentityBlueprint.Create** delegated permission.
Client creates blueprint via Microsoft Graph, using application permissions	Client must be granted **AgentIdentityBlueprint.Create** application permission.

Creating an agent identity with these tools requires one of the following permissions:

Expand table

Scenario	Permissions required
User creates agent identity via Microsoft Entra admin center, Microsoft Graph, or PowerShell/CLI	User must be assigned **Agent ID Developer**, **Agent ID Administrator**, or **AI Administrator** role.
Client creates agent identity via Microsoft Graph, using delegated permissions	User must be assigned **Agent ID Administrator** role. Client must be granted **AgentIdentity.Create.All** or **AgentIdentity.ReadWrite.AI** delegated permission.
Client creates agent identity via Microsoft Graph, using delegated permissions	User must be an **owner** of the agent identity blueprint or agent identity blueprint principal (owners added at any time have this permission). Client must be granted **AgentIdentity.Create.All**, **AgentIdentity.ReadWrite.All**, or **AgentIdentity.ReadWrite.ManagedBy** delegated permission. No Agent ID role is required.
Client creates agent identity via Microsoft Graph, using application permissions	Client must be granted **AgentIdentity.Create.All** application permission.

Creating an agent's user account with these tools requires one of the following permissions:

Expand table

Scenario	Permissions required
User creates agent's user account via Microsoft Entra admin center, Microsoft Graph, or PowerShell/CLI	User must be assigned **Agent ID Administrator** or **User Administrator** role.
Client creates agent's user account via Microsoft Graph, using delegated permissions	User must have the roles in the previous row. Client must be granted **AgentIdUser.ReadWrite.All** delegated permission.
Client creates agent's user account via Microsoft Graph, using application permissions	Client must be granted **AgentIdUser.ReadWrite.All** application permission.

Administrators can control agent identity creation via these channels by constraining which users and clients are assigned the permissions in the previous table. For more information, see Agent identity permissions reference.

Microsoft product integrations

Employees can create agents via many Microsoft products. These products are integrated with Microsoft Entra Agent identity platform and can create agent identity blueprints, agent identities, and agents' user accounts in tenants. Microsoft products integrated with Microsoft Entra Agent ID include:

Expand table

Product	Documentation
Microsoft Copilot Studio	Copilot Studio documentation
Microsoft Security Copilot	Security Copilot documentation
Azure AI Foundry	Azure AI Foundry documentation

Administrators can control agent identity creation via these channels by using the administrative tools for each respective product.

Microsoft Entra ID consent experience

When users sign in for the first time to third-party agents, the Microsoft Entra ID sign-in experience displays a "consent" page to the user. The consent page allows the employee to add the agent to their tenant, which results in the creation of an agent identity blueprint principal.

Agent identity blueprint principals

Once an agent identity blueprint principal is created in your tenant, the principal is assigned privileges to create agent identities and agents' user accounts. The principal can also update and delete any of the identities it creates. The following table describes permissions that can be given to these principals:

Expand table

Entity	Permission	Comments
Agent identity	`AgentIdentity.CreateAsManager` (application permission)	This permission is automatically granted to any agent identity blueprint principal in the tenant. It allows the principal to create agent identities, and update and delete the agent identities it creates. This permission can't be revoked. To prevent a principal from creating agent identities, you must disable it or remove the principal from the tenant. Principals are limited to a maximum of 250 agent identity creations.
Agent's user account	`AgentIdUser.ReadWrite.IdentityParentedBy` (application permission)	A Microsoft Entra ID administrator must grant this permission to an agent identity blueprint principal. Once granted, the principal can create agents' user accounts, and update and delete any users it creates. The permission can be revoked.

Administrators can control agent identity creation via this channel by limiting which agent identity blueprints are added to their tenant as agent identity blueprint principals. For more information on these permissions, see Agent identity permissions reference.

Auditing and monitoring agent identity creation

All agent identity creations are recorded in Microsoft Entra audit logs. Using audit logs, you can determine which channel was used to create an agent identity blueprint, agent identity blueprint principal, agent identity, or agent's user account.

Expand table

Objective	How to Implement	Tooling / Source
Detect new agent identity creations	Subscribe to audit events filtering on agent identity object type / creation operations	Microsoft Entra audit logs

Objective	How to Implement	Tooling / Source
Inventory all agent identities	Use Microsoft Graph APIs to query all agent identity objects by object type.	Microsoft Graph docs

Related content

- Create and delete agent identities
- Create agent identity blueprints

Last updated on 05/01/2026

Microsoft Entra Agent ID design patterns

Microsoft Entra Agent ID introduces new identity constructs as well as new ways of thinking about existing authentication and authorization patterns. To better understand how these constructs fit together, it's helpful to look at common AI agent deployment patterns and how they map to Microsoft Entra Agent ID.

This article describes common AI agent deployment patterns and how they map to Microsoft Entra Agent ID. The article starts with a review of key identity concepts, describes permisssions and trust boundaries, and then walks through common deployment patterns.

For step-by-step decision guidance on how many blueprints and agent identities to create, see Plan your agent identity architecture.

> **Important**
>
> Microsoft Entra Agent ID is currently in PREVIEW. This information relates to a prerelease product that may be substantially modified before it's released. Microsoft makes no warranties, expressed or implied, with respect to the information provided here.

Key concepts

The following components are the foundation of Microsoft Entra Agent ID. If you're new to them, start with Microsoft Entra Agent ID key concepts before continuing with this article.

Identity constructs

The following identity constructs are used throughout the patterns described in this article:

- **Agent identity blueprint**: The template and authentication foundation for one or more agent identities. It holds credentials and policies that apply to all agent identities created from it.
- **Agent identity blueprint principal**: The Microsoft Entra object created when a blueprint is added to a tenant. It's what actually acquires tokens, creates agent identities, and appears in audit logs on behalf of the blueprint.
- **Agent identity**: The runtime identity for a specific AI agent, with its own permissions on downstream resources.
- **Agent's user account**: An optional 1:1 account paired with an agent identity, needed only when the agent must access systems that require a user object.

Permissions models

Blueprint-level permissions and agent identity-level permissions serve different purposes:

- **Blueprint permissions** represent the minimum permissions shared across all agent identities created from the blueprint. Use inheritable permissions on the blueprint when you want all agent identities to start with a common baseline.
- **Agent identity permissions** represent differentiated permissions for a specific agent. Use these permissions when different agents in the same system need different access to downstream resources.

For more information, see Configure inheritable permissions for blueprints and Authorization in Microsoft Entra Agent ID.

Trust boundary

A **trust boundary** refers to the shared risk surface where a single compromise is assumed to affect the entire perimeter. Agents running on separate platforms with separate service accounts, secrets, and network segments don't share a trust boundary.

A trust boundary is an application threat-modeling decision, not one that Microsoft Entra Agent ID defines for you.

Deployment patterns

The following patterns are based on real-world agent deployments. Each describes the agent architecture, its identity structure, and relevant permission and governance considerations.

Low-code singleton agent

A single agent that assists with a specific task, commonly built on a low-code or no-code platform like Microsoft Copilot Studio. The agent either always acts on behalf of a signed-in user (interactive) or always acts as itself (autonomous).

Structure: One blueprint → one agent identity

Although a blueprint with a single agent identity might seem redundant, the blueprint gives the agent consistent Conditional Access policies, monitoring, governance, and audit entries — the same infrastructure you need for multi-agent systems, with minimal setup.

Permissions: Grant permissions directly on the agent identity. The blueprint's inheritable permissions aren't typically needed for singleton cases.

Agent's user account: Not required unless the agent needs access to Exchange, Teams, or another system that requires a user object.

Domain worker (sequential multi-agent)

Multiple agents work together in a tightly coupled, sequential workflow to serve a common domain goal. The agents typically share a codebase, run in the same runtime environment (for example, the same Kubernetes namespace or container), and have the same security posture. Each agent has distinct responsibilities and different access to downstream resources. This pattern maps to sequential orchestration in multi-agent design.

Structure: One blueprint → multiple agent identities (one per agent role)

Using a single blueprint is appropriate here because all agents share the same trust boundary. Each agent gets its own agent identity so that actions can be attributed to a specific agent in audit and sign-in logs, and each can hold different permissions on downstream resources.

Example: A retail product management system has three agents — store inventory, product comparison, and supplier inventory. All three run in the same Kubernetes namespace and are built by the same team. They use one blueprint with three agent identities, each with permissions scoped to its own resources.

Permissions: Set shared baseline permissions as inheritable on the blueprint. Assign role-specific permissions directly to each agent identity.

Agent's user account: Not typically required for domain worker agents.

Concurrent orchestrator with domain workers

An orchestrator agent dynamically activates different domain workers based on the incoming task. The domain workers might run on different platforms, be operated by different teams, and cross trust boundaries. This pattern maps to concurrent orchestration in multi-agent design.

Structure:

- Blueprint A → orchestrator agent identity
- Blueprint B → domain worker agent identities (one per role, cross-trust-boundary group)
- Blueprint C → another domain worker group, if operated by a separate team or platform

Because the domain workers cross trust boundaries — separate runtimes, secrets, or teams — they require separate blueprints. The blueprint's credentials are scoped to its trust domain, so a compromise in one domain doesn't affect peer agents.

Ephemeral agent identities: A variant of this pattern uses ephemeral agent identities. The orchestrator creates a temporary agent identity at runtime to facilitate a specific interaction (for example, coordinating with a maintenance subsystem), grants it permissions inherited from the blueprint, and deletes the identity when the session ends. This limits the blast radius to the duration of the task.

> ⓘ **Note**
>
> Ephemeral agent identity creation happens at runtime, which introduces nondeterministic latency. Evaluate this trade-off for latency-sensitive scenarios.

Permissions: Orchestrator and each domain worker group have their own permission sets, scoped to their respective blueprints and agent identities.

Agent's user account: Not typically required at the orchestrator or domain worker level, unless a specific domain worker must access a user-object-dependent resource.

Per-user agents (small-n)

A separate agent identity is created for each user or organizational unit. For example, a SOC analyst agent might have one instance per cloud environment, or an audit agent might have one instance per department. The number of agent identities is moderate — tens to low hundreds — not one per directory user.

Structure: One blueprint → one agent identity per user, department, or environment

This pattern is appropriate when each agent instance needs different permissions, different auditing boundaries, or an independent lifecycle (for example, the agent for a department can be disabled without affecting others).

Permissions: Each agent identity holds permissions scoped to its user or organizational unit. Inheritable permissions from the blueprint set a minimum baseline, and each agent identity is granted additional permissions as needed.

Agent's user account: Consider pairing each agent identity with an agent's user account when each agent acts as a named representative for its user or department — for example, a dedicated sales agent that receives email on behalf of a territory.

Digital worker (fully autonomous agent)

A fully autonomous agent acts as a digital employee, provisioned with resources typically reserved for human employees: an Exchange mailbox, OneDrive share, and Teams presence.

This is the highest level of agent autonomy.

Structure: One blueprint → one agent identity → one agent's user account

Each digital worker requires its own agent's user account. The 1:1 relationship between agent identity and agent's user account is fixed — you can't share an agent's user account across multiple agent identities.

Example: An AI sales representative with a real mailbox, listed in the Global Address List, that responds to email and is assigned a human manager in the org chart.

Permissions: Grant the agent's user account the specific Exchange, Teams, and OneDrive permissions it needs. Grant the agent identity application-level permissions for systems that don't require a user object. For more information, see Grant agent access to Microsoft 365.

Agent's user account: Required. Create one agent's user account per digital worker agent identity.

Patterns to avoid

Scale-out replicas don't need separate agent identities

Running multiple instances of the same agent code (scale-out) doesn't require separate agent identities. Scale-out is a runtime concern: the blueprint acquires tokens as the agent identity, and multiple instances of the agent can all run under the same identity simultaneously. Creating a separate agent identity per replica adds directory objects and management overhead without any audit, access control, or accountability benefit.

Memory and context management don't require separate agent identities

Agent memory is typically a shared data store (for example, Azure Cache for Redis or Azure AI Search) where data is filtered by session ID at retrieval time. Access to that data store is controlled by the agent identity's permissions, not by separate identities per session. Separate agent identities for memory isolation add complexity without a security benefit.

Don't use scaled per-object agent identities in directory

Creating one agent identity per meeting, per document, or per ephemeral object at high volume isn't practical with directory-level identities today. For high-flux scenarios, use shared

agent identities and rely on session or context identifiers at the application layer to distinguish interactions.

Related content

- Agent identity blueprints
- Agent identities
- Agent's user accounts
- Plan your agent identity architecture
- Configure inheritable permissions for blueprints

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 04/03/2026

Plan your agent identity architecture

Before you integrate your AI agent with Microsoft Entra Agent ID, you need to make a series of design decisions. This guide walks you through each decision in order:

1. **Which identity type** your agent needs.
2. **Which operation pattern** your agent uses, including how it acquires tokens and what context it acts in.
3. **How many blueprints** your system requires.
4. **How many agent identities** to create per blueprint.

Work through these decisions in order, because earlier choices shape later ones. Some single-agent deployments might only need the first two steps. For examples of how these decisions map to real-world agent architectures, see Agent ID design patterns.

Step 1: Choose an identity type

Microsoft Entra ID provides several identity types that an AI agent might use. For most AI agents, **agent identity** is the right choice. Configure the **agent's user account** in addition to an agent identity when the agent must access systems that require a user object. Service principals and regular user accounts aren't recommended for AI agents.

Expand table

Identity type	Use when...	Key characteristics
Agent identity	...the agent acts on its own behalf or on behalf of users	Enforced sponsorship, distinct audit log entries, and blueprint-managed credentials
Agent's user account	...the agent needs access to a resource that requires a user object, such as an Exchange mailbox or Teams channel	User account paired 1:1 with an agent identity, with a UPN, manager, and other user properties
Service principal (not recommended for agent workloads)	...the workload runs scripted, predictable operations without autonomous decision-making	Classic application identity without agent-specific governance, transparency controls, or lifecycle management

Why not a service principal?

Service principals are designed for deterministic, static workloads. Microsoft Entra Agent ID provides agent-specific capabilities that aren't available for service principals, such as:

- A dedicated identity type with explicit entries in sign-in and audit logs, providing traceability and transparency that's hard to achieve with generic service principals.
- Platform-level restrictions on certain high-privilege authorizations, which reduce the blast radius of a compromised agent.
- Enforced sponsorship, so every agent identity has an accountable business owner assigned at creation.
- Blueprint-managed credentials and lifecycle: you create, rotate, and delete agent identities through their parent blueprint, not individually.
- Support for ephemeral agent identities that are created at runtime with inheritable permissions already granted through the blueprint, and deleted when the task completes.

For a detailed comparison, see Agent identities, service principals, and applications.

Why not a regular user account?

Don't use a regular Microsoft Entra user account for an AI agent. User accounts are designed for human sign-in patterns, and assigning them to agents causes problems across every Zero Trust enforcement layer:

- **Conditional Access** policies such as compliant device requirements, multifactor authentication, and terms of use conditions fail for agents because these controls are designed for human interactions.
- **Microsoft Entra ID Protection** uses machine learning tuned for human sign-in behavior. Routing AI agent traffic through user accounts degrades detections for both humans and agents.
- **Identity governance** processes such as joiner-mover-leaver workflows, access packages, and access reviews aren't designed for agent lifecycle patterns and might incorrectly remove agent access.
- Your agents would appear in the Global Address List, Teams, and SharePoint alongside human employees, making it harder to distinguish AI agents from people.

Don't register an app for your agent

If you're used to creating workload identities, your instinct might be to create an app registration or service principal for your agent — for example, by running `az ad app create`, `New-MgApplication`, `New-AzADApplication`, or a `POST /applications` Microsoft Graph request. Don't do

this for AI agents. An identity created this way is a standard application: it has no sponsor, no agent-specific audit entries, and no blueprint-managed lifecycle. Microsoft Entra doesn't govern it as an agent, and labeling it an "agent identity" doesn't make it one.

Instead, create an agent identity blueprint and then create agent identities from it. Agent identities are a distinct Microsoft Entra object type (`#Microsoft.Graph.AgentIdentity`) created through the blueprint, not through the application registration APIs.

Expand table

Instead of this (standard app registration)	Do this (agent identity)
`az ad app create`	Create an agent identity blueprint, then create agent identities from it
`New-MgApplication` or `New-AzADApplication`	Use a supported creation channel with the **Agent ID Developer** or **Agent ID Administrator** role
`POST https://graph.microsoft.com/v1.0/applications`	Grant `AgentIdentityBlueprint.Create`, then create the blueprint and agent identities

In .NET, use the `Microsoft.Identity.Web.AgentIdentities` package: call `builder.Services.AddAgentIdentities()`, then acquire tokens with `WithAgentIdentity(...)`. See Call custom APIs from an agent.

For a full comparison, see Agent identities, service principals, and applications.

Step 2: Choose an operation pattern

An agent's operation pattern determines how it acquires tokens and what context it acts in. Microsoft Entra Agent ID supports two primary patterns: autonomous and interactive.

Expand table

Characteristic	Autonomous	Interactive
User context	No user present	User is signed in
Permission type	Application permissions	Delegated permissions
Consent model	Admin consent required	User or admin consent
Token subject	Agent identity	User (with agent as actor)
Common	Background processing, scheduled tasks,	Chat assistants, user-facing copilots, agents

Characteristic	Autonomous	Interactive
scenarios	system-to-system	acting on user data
Access scope	Broad, tenant-wide access	Scoped to the signed-in user's data

Choose **autonomous** when your agent:

- Runs background or scheduled tasks.
- Processes data across multiple users.
- Performs system-to-system operations without a user present.

Choose **interactive** when your agent:

- Operates on behalf of a signed-in user.
- Needs access to that user's data, such as mail, calendar, or files.
- Must respect the user's own permission boundaries.

Some agents need both. For example, an agent might run a nightly background sync using the autonomous pattern and also respond to user chat messages using the interactive pattern. In this case, implement both OAuth flows and select the appropriate token based on the operation.

- For autonomous agents, see Request agent tokens for autonomous agents.
- For interactive agents, see Authenticate users in interactive agents.

Step 3: Decide how many agent identity blueprints

An agent identity blueprint manages authentication for all the agent identities created from it. The blueprint holds the credentials used to acquire tokens on behalf of those agent identities. Because of this capability, a blueprint compromise can affect every agent identity under it. Choose your agent identity blueprint count based on your security boundaries.

Default: use one blueprint per trust boundary.

A trust boundary is the shared risk surface where a single compromise is assumed to affect the entire perimeter. Agents that share a runtime, secrets, file system, and network share a trust boundary and can share a blueprint.

Expand table

Decision factor	Same trust boundary → one blueprint	Different trust boundaries → multiple blueprints
Authentication material	The same credentials can be shared across all agents; credential rotation or compromise affects all agents together	Credentials must be cryptographically isolated; a credential compromise must not spread to peer agents
Security boundary	Agents run in the same trust boundary: same runtime, secrets, file system, and network	Agents cross trust domain boundaries: separate environments, runtimes, or isolation domains

The following factors are *not* reasons to add more blueprints:

- **Blocking authentication**: You can disable a single agent identity or target it with a Conditional Access policy without adding a blueprint.
- **Audit separation**: Each agent identity produces its own sign-in and audit log entries under the parent blueprint.
- **Scale-out or replicas**: Running multiple instances of the same agent doesn't require multiple blueprints.
- **Memory or context separation**: Agent memory is typically a shared data store filtered by session ID at retrieval time, which doesn't require separate blueprints.

For more information about blueprints, see Agent identity blueprints.

Step 4: Decide how many agent identities per blueprint

Default: use one agent identity per logical agent. A distinct identity for each agent gives you the highest fidelity of audit trails, tracing, and access control.

Expand table

Decision factor	Multiple agent identities	Single agent identity
Audit and attribution	Actions must be attributable to a specific agent for investigation, compliance, or accountability	"The system" acting is sufficient; no need to distinguish which agent acted
Lifecycle independence	Agents are created, deleted, or revoked independently	Agents are created, deleted, and governed as one unit
Role separation	Agents have distinct responsibilities. For example, inventory lookup, product comparison, and supplier data	Agents are interchangeable

The following factors are *not* reasons to add more agent identities:

- **Horizontal scale-out**: Running replicas or instances of the same agent code doesn't require separate identities. Scale-out is a runtime concern, not an identity concern.
- **Memory pools and context management**: Agent memory is typically a shared data store filtered by session ID at retrieval time. Separate identities aren't needed for memory or context isolation.

Multi-agent architectures where multiple agent identities are appropriate can include sequential pipelines with distinct agent roles and concurrent orchestrations with specialized domain workers. For more information, see AI agent orchestration patterns.

Related content

- Agent ID design patterns
- Agent identity blueprints
- Agent identities, service principals, and applications

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 08/14/2026

Best practices for Microsoft Entra Agent ID

This article provides operational best practices for designing, securing, and governing AI agent identities with Microsoft Entra Agent ID. These recommendations help you make informed decisions when planning agent deployments, managing credentials, enforcing access policies, and monitoring agent activity.

For foundational concepts, see What is Microsoft Entra Agent ID? and Key concepts.

Design agent identity blueprints

Agent identity blueprints are templates that define the security posture for all agent instances of a common kind. Thoughtful blueprint design is the foundation of well-governed agent deployments.

- **Plan blueprints before deploying agents.** Define required settings, permissions, and metadata in the blueprint up front rather than creating ad-hoc service principals. This ensures a consistent, governed rollout across all instances. For guidance on structuring your identity model, see Plan your agent identity architecture.

- **Provision a unique identity per agent instance.** Avoid sharing identities between different agents. Distinct identities improve traceability and let you disable or update one agent without impacting others. The blueprint model makes scaling with unique identities manageable because credentials live on the blueprint, not each instance. For instructions, see Create and delete agent identities.

- **Assign a sponsor and an owner at creation time.** Every blueprint and agent identity must have a sponsor; the person or group accountable for the agent's purpose. Assign an owner (technical admin) as well. Periodically verify that these assignments are current, especially when personnel changes occur.

- **Provide descriptive metadata.** Fill in the description, tags, and verified publisher fields on each blueprint to clearly document the agent's purpose, scope, and owning team. Good metadata improves discovery and helps colleagues understand the agent's role at a glance. For blueprint setup details, see Create an agent identity blueprint.

- **Apply policies at the blueprint level.** Associate Conditional Access rules, API permissions, and governance controls with the blueprint so that all current and future agent instances

inherit them automatically. Disabling a blueprint instantly blocks all its agent identities. This option provides a rapid kill-switch when needed. For details, see Conditional Access for agents.

- **Use the Agent ID framework for all agents.** Don't create AI agents as plain app registrations or service principals outside the Agent ID framework. Always use the supported creation channels so agents are tracked as agent identities with built-in sponsor accountability and lifecycle controls.

- **Create agent's user accounts only when necessary.** Agent's user accounts should only be created for scenarios that truly require a user object, such as an agent needing a mailbox or Teams presence. If your agent can operate with app credentials alone, avoid agent's user accounts because they add complexity with licenses, group memberships, and user-level policies.

Guardrails for AI coding assistants

AI coding assistants often default to creating a standard app registration when asked to build an agent, because that's the traditional workload identity pattern. Add the following to your repository's `.github/copilot-instructions.md` or `AGENTS.md` file so AI coding assistants use agent identities correctly:

markdown

```
## Microsoft Entra Agent ID
- To give an AI agent its own identity in Microsoft Entra, create an agent identity
blueprint, then create agent identities from it.
- Don't create or reuse a standard app registration or service principal to represent
an agent. Never use `az ad app create`, `New-MgApplication`, `New-AzADApplication`,
or `POST /applications` to create an agent identity.
- In .NET, use the Microsoft.Identity.Web.AgentIdentities package: call
builder.Services.AddAgentIdentities(), then acquire tokens with
WithAgentIdentity(agentIdentity) (app-only) or WithAgentUserIdentity(agentIdentity,
user) (on behalf of a user).
- Requires the Agent ID Developer or Agent ID Administrator role and the
AgentIdentityBlueprint.Create permission.
- Reference: /entra/agent-id/how-to-plan-agent-identity-architecture
```

Manage credentials securely

Credential management is critical to preventing unauthorized access through agent identities. For detailed steps, see Create an agent identity blueprint.

- **Use managed identities or certificates in production.** For production agents, prefer federated identity credentials (managed identities) or certificates over client secrets. Managed identities eliminate stored secrets entirely. Use client secrets only for initial development or testing, and rotate them out before going live.
- **Isolate credentials per blueprint.** Don't reuse the same credential across unrelated blueprints. If you have separate environments (dev, test, prod), use separate blueprints or environment-specific federated credentials so that a compromise in one environment doesn't affect others. For setup instructions, see Create an agent identity blueprint.
- **Store credentials securely.** Store certificate private keys in Azure Key Vault or an HSM. If using federated credentials tied to a managed identity, limit the managed identity's scope. Establish a rotation schedule to rotate certificates at least annually even though blueprints allow long-lived credentials.
- **Align OAuth flows with agent scenarios.** Use the appropriate OAuth flow for your agent's operating model:
  - For autonomous agents with no user context, use the client credentials flow with only the required app permissions.
  - For interactive agents acting on behalf of a user, use the on-behalf-of (OBO) flow so user access policies and consent apply.
  - Avoid granting app permissions when delegated permissions would suffice.
- **Monitor token usage after deployment.** Review sign-in logs to confirm agents are using the intended authentication method and credential type. Periodically audit the API permissions consented on each blueprint to prevent privilege creep.

Enforce access controls

Apply the same Zero Trust principles to agent identities as you do to user identities. For detailed configuration, see Conditional Access for agents and Identity Protection for agents.

- **Segment agents with custom security attributes.** Define organization-wide attributes such as `Environment`, `Department`, or `DataSensitivity` and assign them to agent identities. Use these attributes in Conditional Access policy conditions to apply fine-grained controls at scale, for example, blocking non-production agents from accessing production resources. For details, see Assign custom security attributes.
- **Block high-risk agents automatically.** Deploy a Conditional Access policy that blocks agent identities flagged with high risk levels by Identity Protection. This is analogous to blocking

risky users and ensures compromised agents are cut off immediately.

- **Create agent-specific Conditional Access policies.** Don't rely on user-targeted policies for agents. Agents can't satisfy interactive controls like MFA, so create separate policies that use identity filters, risk signals, and named locations as control points. Use report-only mode to test policies before enforcing them. For step-by-step guidance, see Conditional Access for agents.

- **Review existing policies for agent impact.** Audit broad policies (like "All users must use MFA") to ensure they don't unintentionally block agent flows. Refactor to exclude agent identities and create dedicated agent policies with appropriate controls.

- **Implement least-privilege permissions.** Use the authorization guidance to grant only the permissions each agent needs. Don't grant broad permissions as a convenience. Limit permissions to specific scopes, API resources, or sites. Review and right-size permissions periodically.

Govern the agent lifecycle

Effective governance prevents agent sprawl and ensures agents remain accountable throughout their lifecycle. For governance tools, see Identity governance for agents and Access packages for agent identities.

- **Register all agents in Microsoft Entra.** Register every agent, whether built in Copilot Studio, Azure, or external platforms, in Microsoft Entra through the Agent ID framework. Centralized registration eliminates shadow AI and gives IT full visibility. For supported methods, see Agent creation channels.

- **Standardize naming conventions.** Define and enforce a naming convention for agent identities, such as prefixing display names with the department or function (for example, `Agent-HROnboardingBot`). Consistent naming makes agents recognizable in logs and the admin center.

- **Include agents in access reviews.** Configure periodic access reviews that include agent identities. Have sponsors attest every 6-12 months that each agent is still needed and properly configured. If a sponsor doesn't confirm, evaluate the agent for decommissioning.

- **Monitor for orphaned agents.** Develop a quarterly review process to identify agents with missing sponsors, outdated metadata, or no recent activity. Reassign sponsorship or decommission unused agents. For a centralized view, see View and filter agent identities.

- **Use access packages for standardized access.** For agents with common access patterns (for example, a fleet of customer support agents), use access packages to grant time-bound, auditable access through approval workflows rather than direct permission assignments.

Monitor and audit agent activity

Continuous monitoring ensures agents operate within expected boundaries. For log details, see Sign-in and audit logs for agents.

- **Monitor sign-in logs for anomalies.** Set up alerts for unusual patterns such as sudden spikes in token requests, access to unexpected APIs, or sign-ins from unfamiliar IP ranges. Agent sign-in logs show each token acquisition with details about the resource, credential type, and outcome.
- **Track configuration changes in audit logs.** Monitor audit logs for changes to agent blueprints, credential additions, permission grants, and role assignments. Alert on changes that happen outside your normal deployment pipeline.
- **Include agents in incident response.** When analyzing security incidents, check whether any agents had access to affected resources and review their activity during the incident window. Fold agent identity checks into your existing post-mortem processes.
- **Set up proactive alerts for:**
  - Credential expiration (certificates or secrets approaching their end date)
  - Agent blocked by Conditional Access or Identity Protection
  - Excessive failed token acquisition attempts
  - Unexpected permission or role changes
- **Retain logs for compliance.** Ensure your log retention policy covers agent activity for the duration required by your organization's compliance framework. Export high-volume agent logs to a secure archive if needed. For configuration options, see Configure diagnostic settings.

Coordinate development and IT workflows

Smooth agent deployments require alignment between developers building agents and IT administrators governing them.

- **Use supported creation channels.** Build agents through Copilot Studio, Graph APIs, or the Agent 365 CLI rather than manual Graph calls that might miss required properties. These

tools handle blueprint creation, credential binding, and instance setup automatically.

- **Establish a production handshake process.** When a new agent moves to production, have an identity admin verify its Microsoft Entra Agent ID settings: confirm the blueprint and sponsor are correct, required permissions are consented, Conditional Access policies apply, and the agent is in appropriate groups or administrative units.
- **Test in nonproduction environments.** Use a separate development tenant or sandbox to validate agent authentication flows, Conditional Access policies, and permission configurations before deploying to production.
- **Treat agent configurations as code.** Check blueprint definitions, permission configurations, and setup scripts into source control. This prevents configuration drift, enables peer review, and provides institutional memory for how agents are integrated with Microsoft Entra ID.

Related content

- Manage agent identities in your organization
- What is Microsoft Entra Agent ID?
- Microsoft Entra security for AI overview

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 08/13/2026

AI-guided setup for Microsoft Entra Agent ID

Onboarding to Microsoft Entra Agent ID involves multiple steps: creating an agent identity blueprint, configuring credentials, setting up identifier URIs and scopes, creating blueprint principals, and provisioning agent identities. Each step has its own prerequisites, validation checks, and decision points.

This AI-guided setup automates this entire workflow by using an AI coding agent (such as GitHub Copilot in VS Code) to execute the steps on your behalf. Instead of navigating between multiple documentation pages and running commands manually, you provide the AI agent with a single instruction file and it walks you through the process interactively. This instruction file is available as a Skill and can be accessed in more than one way.

Benefits

The AI-guided setup offers several advantages over the manual workflow:

- **Single entry point**: One instruction file replaces the need to navigate between multiple documentation pages. The AI agent follows the steps in order and handles transitions automatically.
- **Automated prerequisite validation**: The AI agent validates that you have the correct Microsoft Entra roles, that the required tools and Graph modules are installed, and that required permissions are configured with admin consent before making any API calls.
- **Smart defaults and auto-detection**: The AI agent queries your tenant for existing user information and resource details, then uses those values as suggestions when collecting configuration inputs.
- **Derived naming conventions**: You provide a single display name for your agent, and the AI agent derives all related resource names (blueprint, blueprint principal, agent identities, identifier URIs) using consistent patterns.
- **Inline error handling**: When a command fails, the AI agent analyzes the error, suggests a fix, and retries rather than requiring you to search through troubleshooting documentation. This error handling is especially valuable for Agent ID-specific pitfalls like permission propagation delays and OData header requirements.
- **Idempotent operations**: The AI agent checks whether resources already exist before creating them, making it safe to rerun the setup if a previous attempt was interrupted.

Prerequisites

Before you begin, ensure you have the following prerequisites.

Required tools

The AI-guided setup requires an AI coding agent with terminal access:

- Visual Studio Code with GitHub Copilot and GitHub Copilot Chat extensions installed.
- The GitHub Copilot for Azure extension, which provides the Microsoft Entra Agent ID skill directly in VS Code.

The skill supports two provisioning paths. Install one or both depending on your preference:

- **PowerShell path**: PowerShell 7 or later with the Microsoft Graph PowerShell SDK. Install using `Install-Module Microsoft.Graph.Applications -Scope CurrentUser -Force`.
- **Python path**: Python 3.8 or later with `azure-identity` and `requests`. Install using `pip install azure-identity requests`.

Required accounts and permissions

- Access to a **Microsoft Entra tenant** with one of the following roles:
  - Agent ID Developer to create agent identity blueprints and agent identities. Any owner of an agent identity blueprint can create an agent identity for that blueprint without an Agent ID role.
  - Agent ID Administrator for full administrative access to Agent ID resources.

> **Note**
>
> Owners of an agent identity blueprint or agent identity blueprint principal can create agent identities for that blueprint without a Microsoft Entra Agent ID role. Agent identity blueprint creators are automatically set as owners of both the blueprint and the associated agent identity blueprint principal.

- **Additional roles for permission grants:**
  - Privileged Role Administrator to grant Microsoft Graph application permissions.
  - Cloud Application Administrator or Application Administrator to grant Microsoft Graph delegated permissions.

Required Microsoft Graph permissions

The client you use (PowerShell or a custom app registration) must be authorized with the following delegated permissions:

Expand table

Permission	Purpose
`AgentIdentityBlueprint.Create`	Create new agent identity blueprints
`AgentIdentityBlueprint.ReadWrite.All`	Read and update blueprint properties (identifier URI, scopes, credentials)
`AgentIdentityBlueprintPrincipal.Create`	Create the blueprint's service principal
`AgentIdentity.Create.All`	Create agent identities under a blueprint
`AgentIdentity.ReadWrite.All`	Read and update agent identities
`Application.ReadWrite.All`	Blueprint CRUD on application objects
`AppRoleAssignment.ReadWrite.All`	Grant application permissions to agent identities
`DelegatedPermissionGrant.ReadWrite.All`	Grant delegated permissions to agent identities
`User.Read`	Read the signed-in user's profile (for sponsor assignment)

> **Important**
>
> **`DefaultAzureCredential` and Azure CLI tokens don't work with Agent Identity APIs.** Azure CLI tokens include `Directory.AccessAsUser.All`, which Agent Identity APIs reject with a 403 error. Use `Connect-MgGraph` with explicit delegated scopes (PowerShell) or a dedicated app registration with `client_credentials` (Python). Don't use `az login` tokens for Agent ID provisioning.

Required agent code (optional)

If you already have a working agent project (Python, Node.js, or .NET) that needs an agent identity, have the project directory available. If you don't have one yet, you can still complete the blueprint setup independently.

Get started

The AI-guided setup uses a skill, which is a single instruction file that contains all the steps and validation checks. You can access the skill in one of two ways:

- **Through the GitHub Copilot for Azure extension** (recommended): Install the GitHub Copilot for Azure VS Code extension. The Microsoft Entra Agent ID skill activates automatically when you ask about Agent ID setup in Copilot Chat.
- **Directly from GitHub**: Use the standalone Microsoft Entra Agent ID skill by referencing it in your Copilot Chat prompt.

Step 1: Open your project in VS Code

Open your agent project directory (or any working directory) in Visual Studio Code.

Step 2: Open GitHub Copilot Chat in agent mode

Open the GitHub Copilot Chat panel and switch to **Agent mode**. Agent mode gives GitHub Copilot the ability to run terminal commands, read files, and interact with your environment, which the AI-guided setup requires.

> **Important**
>
> You must use **Agent mode** (not Ask or Edit mode). The AI-guided setup requires the ability to execute terminal commands and interact with your environment.

Step 3: Start the guided setup

If you have the GitHub Copilot for Azure extension installed, ask Copilot to set up Agent ID. For example:

```text
@azure Use the Agent ID Skill to set up an agent identity blueprint and create agent identities for my project using Microsoft Entra Agent ID.
```

If you don't have the extension, reference the skill directly from GitHub:

```text
Follow the steps in https://github.com/microsoft/GitHub-Copilot-for-Azure/blob/main/plugin/skills/entra-agent-id/SKILL.md
```

The AI agent reads the skill and begins the guided setup. It works through the steps sequentially:

1. **Validate prerequisites**: Checks Microsoft Entra roles and validates that the required tools (PowerShell with Microsoft Graph module, or Python with `azure-identity`) are installed.
2. **Authenticate**: Connects to Microsoft Graph with the required scopes. For PowerShell, the skill uses `Connect-MgGraph` with explicit delegated scopes. For Python, it uses a dedicated app registration with client credentials.
3. **Create the agent identity blueprint**: Collects a display name, identifies the sponsor (you), creates the blueprint using the typed endpoint (`/applications/microsoft.graph.agentIdentityBlueprint`), and records the `appId`.
4. **Configure credentials**: Adds a federated identity credential with managed identity (for production) or a client secret (for local development/testing) to the blueprint.
5. **Configure identifier URI and scope**: Sets `identifierUris` to `api://{appId}` and creates an OAuth2 permission scope for agent-to-agent and user-to-agent communication.
6. **Create the blueprint principal**: Creates the service principal for the blueprint using the typed endpoint (the principal is **not** autocreated and must be done explicitly).
7. **Create agent identities**: Creates one or more agent identity service principals under the blueprint.

As the agent reads the skill, you may be prompted to install extensions or other tools. Follow the prompts to ensure your environment is ready.

Step 4: Respond to prompts

The AI agent pauses at specific points to collect input from you:

- **Display name**: The display name for your agent identity blueprint (for example, "Contoso Budget Agent").
- **Sponsor**: The user or group who is accountable for the agent. Defaults to the currently signed-in user.
- **Owner**: The user or service principal who can make technical changes to the blueprint. Optional but recommended.
- **Credential type**: Whether to use a managed identity (recommended for production) or a certificate or client secret (for local development).
- **Agent identity count**: How many agent identities to create under this blueprint.
- **Derived value confirmation**: Review autogenerated names and URIs before resources are created.

> **Tip**

The AI agent shows real values from your Microsoft Entra tenant as examples when it asks for configuration inputs. You can accept the suggestions or provide your own values.

Step 5: Verify in the Microsoft Entra admin center

After the setup completes, the AI agent provides instructions on how to verify the resources:

1. Sign in to the Microsoft Entra admin center as at least an Agent ID Developer.
2. Browse to **Entra ID** > **Agents** > **Agent identities** to see your new agent identity blueprint and any agent identities created under it.
3. Verify the blueprint has the correct credentials, identifier URI, and scope configured.

What the AI-guided setup covers

The AI-guided setup automates the following stages of the Agent ID integration:

Expand table

Stage	What happens	Related documentation
Prerequisites	Validates Microsoft Entra roles, PowerShell module, and Graph permissions	Create a blueprint: Prerequisites
Environment setup	Connects to Microsoft Graph with correct scopes	Create a blueprint: Prepare your environment
Blueprint creation	Creates the agent identity blueprint with sponsor and owner	Create a blueprint
Credential config	Adds managed identity FIC or client secret to the blueprint	Configure credentials
Scope config	Sets identifier URI and OAuth2 permission scope	Configure identifier URI and scope
Principal creation	Creates the agent identity blueprint principal (service principal)	Create an agent blueprint principal
Agent identities	Creates agent identity service principals under the blueprint	Create agent identities

Note

The AI-guided setup doesn't replace the need to integrate Agent ID into your agent's code. You should understand how your agent acquires tokens and perform operations using its

agent identity. The guided setup creates the identity infrastructure your agent code uses.

Common pitfalls the AI-guided setup handles

The Agent ID APIs have several requirements that the AI-guided setup detects and resolves automatically, but they're not obvious requirements. Understanding these pitfalls can be helpful if you need to debug issues or extend the setup.

OData-Version header is required

All Agent ID API calls require the `OData-Version: 4.0` header. If you omit this header, the API might silently create a standard application instead of an agent identity blueprint. The AI-guided setup always includes this header. The skill also uses typed endpoints (such as `/applications/microsoft.graph.agentIdentityBlueprint`) instead of raw `/applications` with `@odata.type` properties to reduce the risk of this issue.

Blueprint principal isn't autocreated

Creating an agent identity blueprint (`POST /applications`) does **not** automatically create its blueprint principal (service principal). Without the blueprint principal, all subsequent agent identity creation fails with:

```
400: The Agent Blueprint Principal for the Agent Blueprint does not exist.
```

The AI-guided setup always creates the blueprint principal immediately after the blueprint. It also handles the idempotent case. If a previous run created the blueprint but crashed before creating the principal, the setup detects this event and creates the missing principal.

Sponsors are required

Sponsors are required and can be users, groups with dynamic membership, or unified groups. Both blueprint and agent identity creation require a `sponsors@odata.bind` field. Without it, you receive:

```
400: No sponsor specified. Please provide at least one sponsor.
```

The AI-guided setup only accepts **User** objects for sponsor assignment and uses the `/users/{objectId}` URL format (not `/directoryObjects/` or `/servicePrincipals/`). The setup resolves the current user's object ID and uses it as the default sponsor. To assign a supported group as sponsor for a blueprint, use the Microsoft Graph API directly.

Permission propagation takes 30–120+ seconds

After you grant admin consent for Agent ID permissions, newly granted permissions don't appear in tokens immediately. The token endpoint serves cached claims, and propagation can take 30-120 seconds or more.

The AI-guided setup handles recent permission changes by retrying operations with exponential backoff when a 403 is received. If you're scripting this manually, implement retry logic:

PowerShell

```powershell
# Example: Retry with backoff after admin consent
$maxRetries = 5
for ($i = 0; $i -lt $maxRetries; $i++) {
    try {
        # Attempt the operation
        $result = Invoke-MgGraphRequest -Method POST -Uri $uri -Body $body
        break
    } catch {
        if ($_.Exception.Response.StatusCode -eq 403 -and $i -lt $maxRetries - 1) {
            $wait = 20 * ($i + 1)
            Write-Host "Permission not yet propagated. Retrying in $wait seconds..."
            Start-Sleep -Seconds $wait
            # Disconnect and reconnect to force a fresh token
            Disconnect-MgGraph
            Connect-MgGraph -Scopes $scopes
        } else {
            throw
        }
    }
}
```

Agent identities can't have password credentials

Agent identities are service principals without backing application objects. Attempting to add a `passwordCredential` directly to an agent identity results in:

```
PropertyNotCompatibleWithAgentIdentity
```

Credentials must be configured on the **blueprint**, not on individual agent identities. Use managed identity federation (recommended) or add secrets/certificates to the blueprint, and agent identities inherit the credential through impersonation.

Identifier URI must be set explicitly

The blueprint's `identifierUris` field isn't set by default. Without it, the OAuth2 scope `api://{appId}/.default` won't resolve, and token acquisition for the agent will fail. The AI-guided setup always configures this value as part of the scope setup step.

Federated identity credential path for blueprints

When adding federated identity credentials (FIC) for managed identity federation, you must use the agent-specific API path:

```
POST /applications/{blueprint-obj-id}/microsoft.graph.agentIdentityBlueprint/federatedIdentityCredentials
```

Using the `/applications/{id}/federatedIdentityCredentials` path might work for agent identity blueprints, but it's not supported and isn't recommended.

Token issuer varies by endpoint version

When validating tokens in your agent backend, be aware of the following variations:

- v1.0 tokens use issuer `https://sts.windows.net/{tenant-id}/`
- v2.0 tokens use issuer `https://login.microsoftonline.com/{tenant-id}/v2.0`

Accept both formats in your token validation logic.

Troubleshooting

AI agent doesn't run terminal commands

If the AI agent describes commands but doesn't execute them, make sure you're using **Agent mode** in GitHub Copilot Chat. Ask and Edit modes don't have terminal access.

AI agent skips validation steps

The instruction file enforces strict step ordering. If the AI agent appears to skip a step, remind it to follow the instructions from the beginning. For example:

```text
Please start from Step 1 in the setup instructions and work through each step in order.
```

Graph commands fail with 403-Forbidden

The most common causes of 403 errors:

- **Using Azure CLI or `DefaultAzureCredential` tokens**: Azure CLI tokens include `Directory.AccessAsUser.All`, which Agent Identity APIs reject outright. Use `Connect-MgGraph` with explicit delegated scopes, or a dedicated app registration with `client_credentials`. See the authentication warning in the prerequisites.
- **Permission propagation delay**: Wait 1–2 minutes after admin consent and retry. The AI-guided setup handles this automatically with retry logic.
- **Missing admin consent**: Verify that the required permissions have admin consent granted in the Microsoft Entra admin center under **App registrations** > your client app > **API permissions**.

Blueprint creation succeeds but returns a standard application

This result happens when the `OData-Version: 4.0` header is missing. Use the typed endpoint (`/applications/microsoft.graph.agentIdentityBlueprint`) instead of raw `/applications` with `@odata.type` to avoid this issue.

Agent identity creation fails with "Blueprint Principal does not exist"

The blueprint principal must be created as a separate step after the blueprint. Run:

```HTTP
POST https://graph.microsoft.com/v1.0/servicePrincipals/microsoft.graph.agentIdentityBlueprintPrincipal
OData-Version: 4.0
Content-Type: application/json

{
```

```
  "appId": "<your-blueprint-app-id>"
}
```

Credential lifetime policy errors

Your tenant might have credential lifecycle policies that restrict the maximum lifetime for client secrets. If you receive an error about credential lifetime when adding a password, reduce the `endDateTime` value to align with your organization's policy.

Configuration values need to change

If you need to change configuration values after the setup, you can:

- Rerun the AI-guided setup with updated values. The idempotent checks skip resources that already exist correctly.
- Use Microsoft Graph PowerShell to update specific properties with `PATCH` requests.

Next steps

- Create and delete agent identities
- Administrative relationships in Agent ID (Owners, sponsors, and managers)
- Agent identities, service principals, and applications
- Agent identity blueprint concepts

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/13/2026

Create an agent identity blueprint

An agent identity blueprint is used to create agent identities and request tokens using those agent identities. During the process for creating an agent identity blueprint, you set the owner and sponsor of that blueprint, to establish accountability and administrative relationships. You also configure an identifier URI and define a scope for agents created from this blueprint if the agent is designed to receive incoming requests from other agents and users.

You can create an agent identity blueprint in two ways:

- **Microsoft Entra admin center** — Use the wizard for a quick setup that creates the blueprint and its principal.
- **Microsoft Graph API or PowerShell** — Create and fully configure the blueprint programmatically, including credentials, identifier URIs, scopes, and the blueprint principal in a single workflow.

Prerequisites

To create an agent identity blueprint, you need:

- Privileged Role Administrator role is the least privileged role required to grant Microsoft Graph Application permissions.
- Cloud Application Administrator or Application Administrator is required to grant Microsoft Graph delegated permissions.
- Both Agent ID Developer and Agent ID Administrator roles can create agent identity blueprints and agent identity blueprint principals.
  - Agent ID Developer can configure federated identity credentials on an agent identity blueprint.
  - Agent ID Administrator can configure federated identity credentials on an agent identity blueprint and is required to add a secret or certificate credential.
- If using PowerShell, version 7 is required.

> **Note**
>
> Owners of an agent identity blueprint or agent identity blueprint principal can create agent identities for that blueprint without a Microsoft Entra Agent ID role. Agent identity blueprint

creators are automatically set as owners of both the blueprint and the associated agent identity blueprint principal.

Prepare your environment

To streamline the process, take a few moments to get your environment set up for the right permissions.

Authorize a client to create agent identity blueprints

In this article, you use Microsoft Graph PowerShell or another client to create your agent identity blueprint. You must authorize this client to create and configure an agent identity blueprint and create an agent identity blueprint principal. The client requires the following Microsoft Graph permissions:

- AgentIdentityBlueprint.Create delegated permission
- AgentIdentityBlueprint.AddRemoveCreds.All delegated permission
- AgentIdentityBlueprint.UpdateAuthProperties.All delegated permission
- AgentIdentityBlueprintPrincipal.Create delegated permission

The steps in this guide use all delegated permissions, but you can use application permissions for those scenarios that require them.

To connect to all of the required scopes for Microsoft Graph PowerShell, run the following command:

PowerShell

```
Connect-MgGraph -Scopes "AgentIdentityBlueprint.Create",
"AgentIdentityBlueprint.AddRemoveCreds.All",
"AgentIdentityBlueprint.UpdateAuthProperties.All",
"AgentIdentityBlueprintPrincipal.Create", "User.Read" -TenantId <your-tenant-id>
```

Create an agent identity blueprint

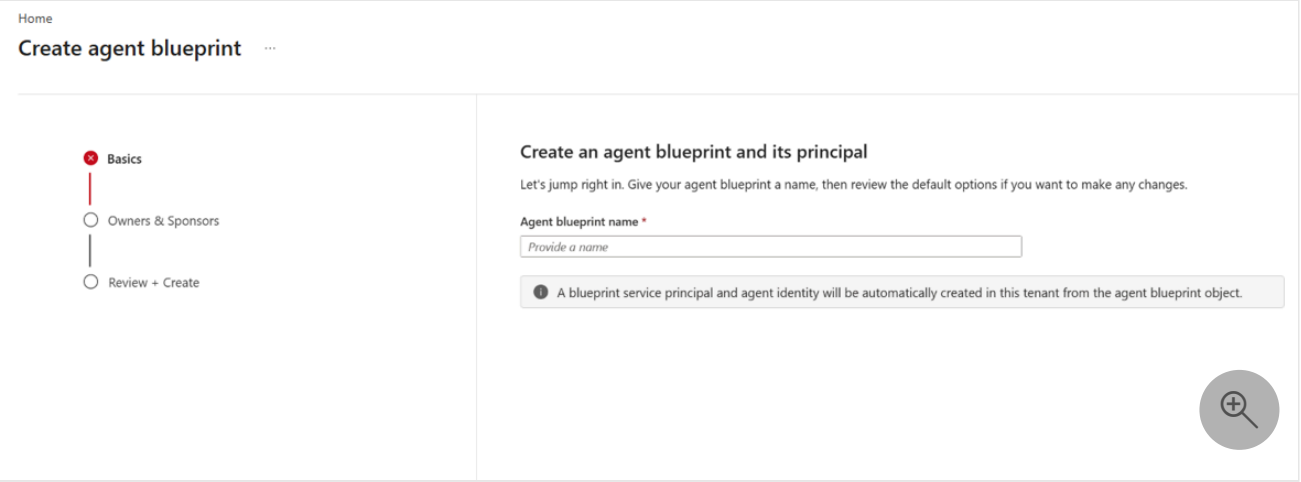
Agent identity blueprints must have a sponsor, which is the user or supported group that's accountable for the agent. An owner is recommended, which is the user or service principal that can make changes to the agent identity blueprint. For information, see Administrative relationships in Microsoft Entra Agent ID.

Use the Microsoft Entra admin center

You can create an agent identity blueprint directly in the Microsoft Entra admin center. The admin center wizard creates both the agent identity blueprint and its blueprint principal automatically.

> **Note**
>
> The admin center wizard sets the blueprint name and assigns owners and sponsors. To configure credentials, identifier URIs, scopes, or permissions, use the Microsoft Graph API or PowerShell, or configure them after creation through the blueprint's detail pages in the admin center.

1. Sign in to the Microsoft Entra admin center.
2. Browse to **Entra ID** > **Agents** > **Agent blueprints**.
3. Select **New agent blueprint (Preview)**.
4. On the **Basics** tab, enter a name in the **Agent blueprint name** field and select **Next**.



5. On the **Owners & Sponsors** tab, optionally change or add owners and sponsors for the blueprint:
   - Select the pencil icon next to the **Owners** field to change or add users who can manage the blueprint.
   - Select the pencil icon next to the **Sponsors** field to change or add users who can sponsor the blueprint.

> **Note**

> Sponsors can be users, dynamic membership groups, or Microsoft 365 groups. Security groups and role-assignable groups are not supported as sponsors.

6. Select **Next**.
7. Review your settings, and then select **Create**.
8. Select **Done** to exit the wizard or **Go to agent blueprint** to view the blueprint's detail page or configure more settings.

For more information on managing agent identity blueprints, see Manage agent identity blueprints.

Create programmatically

To create an agent identity blueprint using code, use the Microsoft Graph API or PowerShell.

Microsoft Graph API

This step creates the agent identity blueprint, assigns an owner and sponsor, and requires the following details:

- The `AgentIdentityBlueprint.Create` permission.
- The OData-Version header must be set to 4.0.
- A user ID for the owner and sponsor fields in the example request body. A sponsor is required, but an owner is optional.

HTTP

```
POST https://graph.microsoft.com/v1.0/applications/
OData-Version: 4.0
Content-Type: application/json
Authorization: Bearer <token>

{
  "@odata.type": "Microsoft.Graph.AgentIdentityBlueprint",
  "displayName": "My Agent Identity Blueprint",
  "sponsors@odata.bind": [
    "https://graph.microsoft.com/v1.0/users/<id>"
  ],
  "owners@odata.bind": [
    "https://graph.microsoft.com/v1.0/users/<id>"
  ]
}
```

After creating the agent identity blueprint, record the value of the `appId` for the next step.

Configure credentials for the agent identity blueprint

To request access tokens using the agent identity blueprint, you must add a client credential. We recommend using a managed identity as a federated identity credential (FIC) for production deployments. Managed identities allow you to obtain Microsoft Entra tokens without having to manage any credentials. For more information, see Managed identities for Azure resources.

Other kinds of app credentials including `keyCredentials` and `passwordCredentials` are supported, but not recommended for production. They can be convenient for local development and testing or where managed identities won't work, but these options don't align with security best practices. For more information, see Security best practices for application properties.

Keep in mind that to use a managed identity you must run your code on an Azure service, such as a virtual machine or Azure App Service. For local development and testing, use a client secret or certificate.

Microsoft Graph API

To send this request:

- You need the `AgentIdentityBlueprint.AddRemoveCreds.All` permission.
- Replace the `<agent-blueprint-id>` placeholder with the `appId` of the agent identity blueprint.
- Replace the `<managed-identity-principal-id>` placeholder with the ID of your managed identity.

Add a managed identity as a credential using the following request:

HTTP

```
POST https://graph.microsoft.com/v1.0/applications/<agent-blueprint-
id>/federatedIdentityCredentials
OData-Version: 4.0
Content-Type: application/json
Authorization: Bearer <token>

{
    "name": "my-managed-identity",
    "issuer": "https://login.microsoftonline.com/<your-tenant-id>/v2.0",
    "subject": "<managed-identity-principal-id>",
    "audiences": [
```

```
        "api://AzureADTokenExchange"
    ]
}
```

Other app credentials

For scenarios where managed identities won't work or if you're creating a blueprint locally for testing, use the following steps to add the credentials.

Microsoft Graph API

To send this request, you first need to obtain an access token with the delegated permission `AgentIdentityBlueprint.AddRemoveCreds.All`

HTTP

```
POST https://graph.microsoft.com/v1.0/applications/<agent-blueprint-id>/addPassword
Content-Type: application/json
Authorization: Bearer <token>

{
  "passwordCredential": {
    "displayName": "My Secret",
    "endDateTime": "2026-08-05T23:59:59Z"
  }
}
```

> **Note**
>
> Your tenant might have credential lifecycle policies that restrict the maximum lifetime for client secrets. If you receive an error about credential lifetime, reduce the `endDateTime` value to align with your organization's policy.

Be sure to securely store the `passwordCredential` values generated. It can't be viewed after initial creation. You can also use client certificates as credentials; see Add a certificate credential.

If the agents created with the blueprint will support interactive agents, where the agent acts on behalf of a user, your blueprint must expose a scope so that the agent front end can pass an access token to the agent backend. This token can then be used by the agent backend to get an access token to act on behalf of the user.

Configure identifier URI and scope

To receive incoming requests from users and other agents, like for any web API, you need to define an identifier URI and OAuth scope for your agent identity blueprint:

Microsoft Graph API

To send this request:

- You need the permission `AgentIdentityBlueprint.UpdateAuthProperties.All`.
- Replace the `<agent-blueprint-id>` placeholder with the `appId` of the agent identity blueprint.
- You need a Globally Unique Identifier (GUID). In PowerShell, run `[guid]::NewGuid()` or use an online GUID generator. Copy the generated GUID and use it to replace the `<generate-a-guid>` placeholder.

HTTP

```
PATCH https://graph.microsoft.com/v1.0/applications/<agent-blueprint-id>
OData-Version: 4.0
Content-Type: application/json
Authorization: Bearer <token>

{
    "identifierUris": ["api://<agent-blueprint-id>"],
    "api": {
      "oauth2PermissionScopes": [
        {
          "adminConsentDescription": "Allow the application to access the agent
on behalf of the signed-in user.",
          "adminConsentDisplayName": "Access agent",
          "id": "<generate-a-guid>",
          "isEnabled": true,
          "type": "User",
          "value": "access_agent"
        }
      ]
  }
}
```

A successful call generates a 204 response.

Create an agent blueprint principal

In this step, you create a principal for the agent identity blueprint. For more information, see Agent identities, service principals, and applications.

Microsoft Graph API

Replace the `<agent-blueprint-app-id>` placeholder with the `appId` you copied from the results of the previous step.

HTTP

```
POST
https://graph.microsoft.com/v1.0/serviceprincipals/microsoft.graph.agentIdentityB
lueprintPrincipal
OData-Version: 4.0
Content-Type: application/json
Authorization: Bearer <token>

{
  "appId": "<agent-blueprint-app-id>"
}
```

Your agent blueprint is now ready and visible in the Microsoft Entra admin center. In the next step, you'll use this blueprint to create agent identities.

Register agents in the Agent 365 registry

After you create an agent identity blueprint, register it in the Agent 365 registry so administrators can discover, govern, and manage the agent from the Microsoft 365 admin center. This section also provides instructions to add existing agent identity blueprints that might not currently appear in the Agent 365 registry.

Use the Microsoft 365 Agents SDK (Recommended)

The Microsoft 365 Agents SDK is now generally available and is the recommended way to build and provision agents. The SDK The SDK handles agent identity creation and registration in the Agent 365 registry for you, so your agent identities will appear automatically with no extra code. If you are starting a new agent project, or have flexibility to migrate existing code, use the SDK. It's the simplest, most durable path and avoids the need to coordinate multiple API calls yourself.

Use the Agent 365 CLI

The Agent 365 CLI is another option that handles setup for you, including agent registration. Follow the setup instructions using the recommended execution order. Use the following command:

```
HTTP
a365 setup all
```

If registration fails, you can rerun just the registration step without having to go through the entire process. Use the following command:

```
HTTP
a365 setup all --agent-registration-only
```

Call the Agent Registry API directly

If you must create agent identity blueprints programmatically with the Microsoft Graph API, for example because you have existing identity-issuance workflows you can't change immediately, you need to add an explicit call to the Agent Registry API *after* creating the agent identity blueprint to post the corresponding agent card. This step registers the agent card in the Agent 365 registry so it appears for administrators.

1. Create the agent identity blueprint using the Microsoft Graph API (as shown in the previous sections).
2. Immediately follow with a call to the Agent Registry API to post the corresponding agent card, including the metadata your administrators need to govern it.
3. Handle the two-call pattern in a retry-safe way so that a transient failure on either call leaves your environment in a recoverable state.

For request and response schemas, required permissions, and code samples, see the Agent Registry API reference.

> **Tip**
>
> If you have existing agent identity blueprints that don't appear in the Agent 365 registry, register them using the Agent Registry API. For agent identity blueprints in bulk, use the batch endpoint. For more information, see **Agent Registry convergence with Microsoft Agent 365**.

Existing agent identity blueprints that aren't in the Agent 365 registry

For agent identity blueprints that were previously created using the Microsoft Entra Agent ID Graph API but are not currently visible in the Agent 365 registry, you can register them using the Agent Registry API. This step ensures they appear in the Agent 365 registry.

Delete an agent identity blueprint

When an agent is decommissioned, delete the associated agent identity blueprint. Deleting the blueprint triggers automatic cleanup of all child agent identities and agents' user accounts. For step-by-step deletion and restore instructions, see Delete and restore agent identity objects.

Next step

Create and delete agent identities

Last updated on 05/09/2026

Create agent identities in agent identity platform

After you create an agent identity blueprint, the next step is to create one or more agent identities that represent AI agents in your tenant. Agent identity creation is typically performed when provisioning a new AI agent.

You can create agent identities in two ways:

- **Microsoft Entra admin center** — Use the admin center wizard for quick, individual identity creation.
- **Microsoft Graph API** — Build a web service that creates agent identities programmatically, which is useful for automated provisioning at scale.

If you want to quickly create agent identities for testing purposes, consider using this Microsoft Entra PowerShell module for creating and using agent identities.

Prerequisites

To create agent identities, you need:

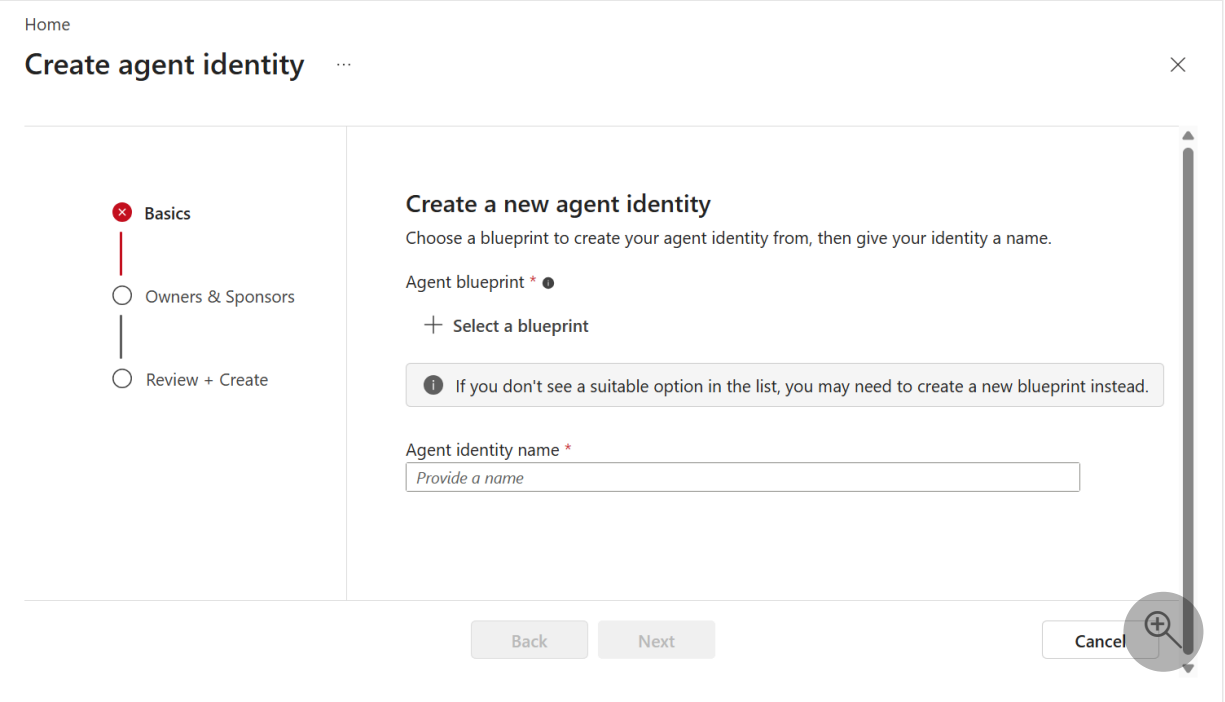
- An agent identity blueprint. Record the agent identity blueprint app ID from the creation process.
- A web service or application (running locally or deployed to Azure) that hosts the agent identity creation logic. This prerequisite applies only if you're creating agent identities programmatically.

Use the Microsoft Entra admin center

You can create an agent identity directly in the Microsoft Entra admin center by selecting an existing blueprint and assigning owners and sponsors.

1. Sign in to the Microsoft Entra admin center.
2. Browse to **Entra ID** > **Agents** > **Agent identities**.
3. Select **New agent identity (Preview)**.
4. On the **Basics** tab:

- Under **Agent blueprint**, select a blueprint to create your agent identity from.
- Enter a name in the **Agent identity name** field and select **Next**.



5. On the **Owners & Sponsors** tab, optionally add owners and sponsors for the identity:
   - Select the pencil icon next to the **Owners** field to change or add users who can manage this agent identity.
   - Select the pencil icon next to the **Sponsors** field to change or add users who can sponsor this agent identity.

> **Note**
>
> Sponsors can be users, dynamic membership groups, or Microsoft 365 groups. Security groups and role-assignable groups are not supported as sponsors.

6. Select **Next**.
7. Review your settings, and then select **Create**.
8. Select **Done** to exit the wizard or **Go to agent identity** to view the identity's detail page or configure more settings.

In the following steps, you'll learn how to create agent identities programmatically using Microsoft Graph API and Microsoft.Identity.Web. Get an access token first, then call the creation API.

Microsoft Graph API

Get an access token using agent identity blueprint

You use the agent identity blueprint to create each agent identity. Request an access token from Microsoft Entra using your agent identity blueprint:

When using a managed identity as a credential, you must first obtain an access token using your managed identity. Managed identity tokens can be requested from an IP address locally exposed in the compute environment. Refer to the managed identity documentation for details.

```
GET http://169.254.169.254/metadata/identity/oauth2/token?api-version=2019-08-01&resource=api://AzureADTokenExchange/.default
Metadata: True
```

After you obtain a token for the managed identity, request a token for the agent identity blueprint:

```
POST https://login.microsoftonline.com/<my-test-tenant>/oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=<agent-blueprint-id>
scope=https://graph.microsoft.com/.default
client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
client_assertion=<msi-token>
grant_type=client_credentials
```

A `client_secret` parameter can also be used instead of `client_assertion` and `client_assertion_type`, when a client secret is being used in local development.

Create an agent identity

Using the access token acquired in the previous step, you can now create agent identities in your tenant. Agent identity creation might occur in response to many different events or triggers, such as a user selecting a button to create a new agent. We recommend you create one agent identity for each agent, but you might choose a different approach based on your needs.

Microsoft Graph API

Always include the OData-Version header when using @odata.type.

```
POST
https://graph.microsoft.com/beta/serviceprincipals/Microsoft.Graph.AgentIdentity
OData-Version: 4.0
Content-Type: application/json
Authorization: Bearer <token>
{
    "displayName": "My Agent Identity",
    "agentIdentityBlueprintId": "<my-agent-blueprint-id>",
    "sponsors@odata.bind": [
        "https://graph.microsoft.com/v1.0/users/<id>",
        "https://graph.microsoft.com/v1.0/groups/<group-id>"
    ],
}
```

> **Note**
>
> When assigning a group as sponsor, only supported group types are accepted. Groups aren't supported as owners.

Delete an agent identity

When an agent is deallocated or destroyed, your service should also delete the associated agent identity:

Microsoft Graph API

HTTP

```
DELETE https://graph.microsoft.com/beta/serviceprincipals/<agent-identity-id>
OData-Version: 4.0
Content-Type: application/json
Authorization: Bearer <token>
```

Related content

- Configure inheritable permissions for blueprints
- Authenticate and acquire tokens for autonomous agents
- Authenticate users and acquire tokens for interactive agents
- Manage owners and sponsors
- Agent identity blueprints

Last updated on 05/01/2026

Agent identity deletion

When you delete an agent identity blueprint or its principal, Microsoft Entra automatically cleans up all child agent identities and agents' user accounts with a *cascade cleanup* process. You don't need to manually query and delete each one. Understanding when and how this cascade cleanup happens helps you plan for restoration if needed.

Agent identity blueprints and their associated objects follow the same soft deletion and hard deletion behavior as other app registrations and service principals in Microsoft Entra. For a full overview of that process, see Deleting and recovering applications FAQ. This article focuses on what's unique to agent identity deletion.

Object relationships

The following objects are involved in the agent identity deletion lifecycle. The cascade cleanup process is based on these relationships.

Expand table

Object	Directory object type	Relationship
Agent identity blueprint	Application	Parent of the blueprint principal
Agent identity blueprint principal	Service principal	The principal for the blueprint
Agent identity	Service principal	Child of the blueprint principal
Agent's user account	User	Paired 1:1 with an agent identity

Disable versus delete

Before deleting a blueprint, consider whether disabling is the right action instead.

- **Disable**: Prevents the blueprint principal or its agent identities from authenticating, but leaves all objects in place. Use this when you want to temporarily stop agent activity, investigate an issue, or decommission agents gradually. Objects remain in the directory and count toward quota.
- **Delete**: Removes the agent identity blueprint or its agent identity blueprint principal from the directory and triggers cascade cleanup of child agent identities. Use this when you're

permanently retiring a blueprint and all agents created from it. Deletion can't be undone after the 30-day soft-deletion window expires.

For information on disabling agent identities, see Disable agent identities.

Cascade cleanup

When you delete an agent identity blueprint or its agent identity blueprint principal, Microsoft Entra automatically soft deletes all associated child agent identities and agents' user accounts. This cleanup is asynchronous.

The cascade process works as follows:

1. **You delete the agent identity blueprint or agent identity blueprint principal**: The object is soft deleted and moves to the recycle bin.
2. **Microsoft Entra triggers automatic cleanup**: A background task soft deletes all child agent identities and agents' user accounts associated with the deleted blueprint.
3. **Objects are restorable for 30 days**: Soft-deleted objects can be restored within 30 days. After that, they're permanently deleted.

> **Important**
>
> If you restore the agent identity blueprint principal before the background cleanup runs, child agent identities aren't affected. After the cleanup runs, each child identity must be restored individually. Restoring the agent identity blueprint principal doesn't reverse cascade deletions that already occurred.

Cascade cleanup in audit logs

When the background cleanup task deletes agent identities, the deletions appear in your tenant's Microsoft Entra audit logs. These entries have the following characteristics:

Expand table

Audit log field	Value
Activity	Delete service principal
Initiated by (actor)	Application: *Delete Agent Identities Task*
Actor App ID	(blank)

The actor is shown as an application named **Delete Agent Identities Task** with no app ID. This is expected because the cleanup is performed by an internal Microsoft Entra background process, not by an external application or user.

> **Note**
>
> The cascade cleanup runs asynchronously after the blueprint is deleted. It might not run immediately — there can be a delay of hours or days between the blueprint deletion and the cleanup of child agent identities. Each child identity deletion appears as a separate audit log entry.

Orphaned objects and quota considerations

When an agent identity blueprint principal is permanently deleted, any associated agent identities and agents' user accounts that weren't deleted become **orphaned objects** and become soft-deleted. Orphaned objects can't authenticate but continue to count toward directory quota until they're permanently deleted after the 30-day retention period expires.

Agent identity deletion follows the same quota rules as other Microsoft Entra objects. Soft-deleted objects continue to count toward quota limits until permanently deleted. For general quota information, see Microsoft Entra service limits and restrictions.

One consideration specific to agent identities: if you're using app-only permissions and are at the 250 agent identity limit for a blueprint, deleting an agent identity doesn't free up space until it's permanently deleted. By default, permanent deletion happens automatically after the 30-day retention period expires. If you need to free up quota immediately, you can force permanent deletion of soft-deleted agent identities. For steps, see Permanently delete agent identity objects. Agent identity blueprints also follow this 250 limit when using app-only permissions.

Related content

- Create and delete agent identities
- Deleting and recovering applications FAQ
- Microsoft Entra service limits and restrictions

Note: The author created this article with assistance from AI. Learn more

Last updated on 05/14/2026

How to delete and restore agent identity objects

When you delete an agent identity blueprint, Microsoft Entra automatically cleans up all associated child agent identities and agents' user accounts. All deletions are soft deletions — deleted objects move to the recycle bin and can be restored within 30 days.

For a conceptual overview of how cascade cleanup works and quota considerations, see Agent identity deletion works.

Prerequisites

To delete and restore agent identity objects, you need:

- Agent ID Administrator to view and manage agent identity objects.
- Cloud Application Administrator to delete the blueprint application and its service principal.
- The `Application.ReadWrite.All` permission for Microsoft Graph API or Microsoft Entra PowerShell operations.
- The `AgentIdentity.ReadWrite.All` permission to permanently delete soft-deleted agent identities.
- Owners of an agent identity blueprint can delete agent identity objects associated with that blueprint without these roles.

Delete a blueprint

Deleting agent identity objects isn't supported in the Microsoft Entra admin center. Use Microsoft Graph API or Microsoft Entra PowerShell to delete blueprints and agent identities.

You can delete the agent identity blueprint and blueprint principal separately. Deleting the app registration also deletes the principal. Deleting only the principal leaves the app registration in place.

> **Note**
>
> Standard deletion is always soft deletion. Objects are moved to the recycle bin, but not immediately removed. Permanent deletion happens automatically after 30 days, or you can

force it using the standard hard-delete process described in Deleting and recovering applications FAQ.

Microsoft Graph API

To delete the blueprint application (which also deletes the principal):

HTTP

```
DELETE https://graph.microsoft.com/v1.0/applications/{blueprint-app-object-id}
```

To delete only the blueprint principal:

HTTP

```
DELETE https://graph.microsoft.com/v1.0/servicePrincipals/{blueprint-principal-object-id}
```

> **Note**
>
> Explicit hard deletion of a blueprint principal (`permanentDelete`) is blocked. Use the standard delete endpoint shown above.

After you delete the blueprint or its principal, Microsoft Entra automatically soft deletes all associated child agent identities and agents' user accounts. For more information on this process, see Agent identity deletion.

> **Important**
>
> If you restore the blueprint principal before the cascade cleanup runs, child agent identities aren't affected. After the cleanup runs, each child identity must be restored individually. Restoring the blueprint principal doesn't reverse cascade deletions that already occurred.

Restore a blueprint principal

You can restore a soft-deleted blueprint principal within 30 days. Restoring agent identity objects isn't supported in the Microsoft Entra admin center. Use Microsoft Graph API or Microsoft Entra PowerShell.

Microsoft Graph API

HTTP

```
POST https://graph.microsoft.com/v1.0/directory/deletedItems/{blueprint-principal-object-id}/restore
```

Restore child agent identities

If the cascade cleanup already ran and child agent identities were soft deleted, restore each one individually.

> **Note**
>
> The Microsoft Graph `/directory/deletedItems` endpoint doesn't support filtering by agent identity type. Query for deleted service principals and filter results on the client side using known object IDs, app IDs, or display names to identify the correct agent identities.

Microsoft Graph API

1. List soft-deleted service principals to find affected agent identities:

   HTTP

   ```
   GET https://graph.microsoft.com/v1.0/directory/deletedItems/microsoft.graph.servicePrincipal
   ```

2. Filter the results on the client side to identify agent identities linked to your blueprint, then restore each one:

   HTTP

   ```
   POST https://graph.microsoft.com/v1.0/directory/deletedItems/{agent-identity-object-id}/restore
   ```

Permanently delete agent identity objects

Soft-deleted objects continue to count toward directory quota until they're permanently deleted. If you're at the 250 agent identity limit for a blueprint using app-only permissions, you might need to force permanent deletion to free up quota immediately rather than waiting for the 30-day retention period to expire. Permanent deletion of an agent identity blueprint principal is blocked. To permanently free quota used by a blueprint principal, wait for the 30-day retention period to expire.

> ⊗ **Caution**
>
> Permanently deleted objects can't be restored. Only permanently delete objects when you're certain they're no longer needed.

Microsoft Graph API

Permanently delete a soft-deleted agent identity:

HTTP

```
DELETE https://graph.microsoft.com/v1.0/directory/deletedItems/{agent-identity-object-id}
```

Permanently delete a soft-deleted blueprint application:

HTTP

```
DELETE https://graph.microsoft.com/v1.0/directory/deletedItems/{blueprint-app-object-id}
```

Agents' user accounts

Agents' user accounts are paired 1:1 with agent identities. If agents' user accounts aren't automatically cleaned up as part of cascade deletion, delete them manually.

Microsoft Graph API

HTTP

```
DELETE https://graph.microsoft.com/v1.0/users/{agent-user-object-id}
```

Related content

- Understand agent identity deletion
- Create and delete agent identities
- Deleting and recovering applications FAQ

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/01/2026

View and manage agent identity blueprints in your tenant

The Microsoft Entra admin center allows you to view all agent identity blueprint principals in your tenant. You can search, filter, sort, and manage blueprint principals including their credentials, permissions, and owners.

Navigate to the agent identity blueprint list

1. Sign in to the Microsoft Entra admin center.
2. Browse to **Entra ID** > **Agents** > **Agent blueprints**.
3. Select a blueprint principal to open its management page.

Search and filter blueprints

1. Enter the **name** or **object ID** of the blueprint principal in the search box.
2. To look up a blueprint by its **Blueprint Application ID**, select **Add filters** and add the **Blueprint App ID** filter.
3. You can further refine the list using filters based on various criteria.

Select viewing options

To customize your view, select **Choose columns** to configure which columns are shown. The available columns are:

Expand table

Column Name	Description	Sortable	Filterable	Special notes
Name	Display name of the agent identity blueprint principal	✓	✓	Primary search field; clickable to view details of the agent identity blueprint principal
Agent identities	The number of child agent identities created by the agent blueprint principal	✗	✗	Select this to see a list of linked child agent identities for that agent identity blueprint principal

Column Name	Description	Sortable	Filterable	Special notes
Status	Current operational state (Active, or Disabled)	✓	✓	
Blueprint Application ID	Unique identifier for the agent identity blueprint of this agent identity blueprint principal	✗	✓	
Object ID	Unique identifier for agent blueprint principal	✗	✓	

View linked agent identities

View the agent identities that were created from a blueprint.

1. From the blueprint's management page, select **Linked agent identities** from the left menu.
2. The list shows each linked identity with its **Name**, **Status**, **View Access** link, and **Owners and Sponsors**.
3. Select an identity name to navigate to its detail page.
4. Use **Add filters** to refine the list.

View granted permissions

Review the permissions assigned to a blueprint principal, organized by consent type.

1. From the blueprint's management page, select **Granted permissions** under **Access**.
2. Select the **Admin consent** tab to view permissions granted through administrator consent, or select the **User consent** tab to view permissions granted through user consent.
3. The list shows the **API name**, **Claim value**, **Permission**, **Type**, **Granted through**, and **Granted by** for each permission entry.

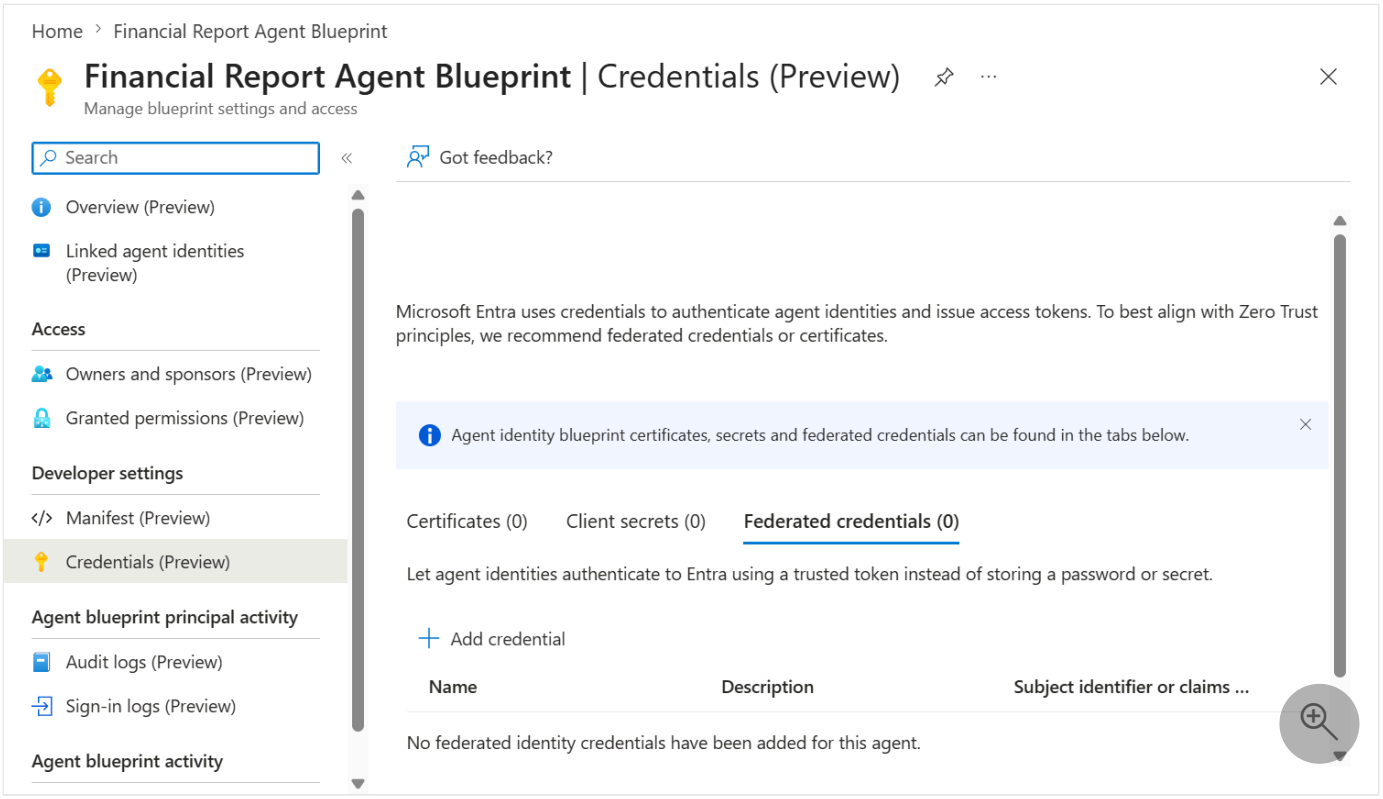
View the manifest

To view or edit the raw JSON manifest for the agent identity blueprint, select **Manifest** under **Developer settings** from the blueprint's management page.

> ⓘ **Note**
>
> The manifest editor is currently in preview.

Manage credentials

Configure the credentials that agent identities use to authenticate. The credentials page has three tabs for different credential types.



> **Important**
>
> To best align with Zero Trust principles, use federated credentials or certificates instead of client secrets.

Upload a certificate

1. From the blueprint's management page, select **Credentials** under **Developer settings**.
2. Select the **Certificates** tab.
3. Select **Upload certificate**.
4. Browse to and select the certificate file, and optionally add a description.
5. Select **Add**.

Create a client secret

1. From the blueprint's management page, select **Credentials** under **Developer settings**.
2. Select the **Client secrets** tab.
3. Select **New client secret**.

4. Enter a **Description** for the secret.
5. Select an expiration period under **Expires**. For custom expiration, set the **Start** and **End** dates.
6. Select **Add**.
7. Copy the secret **Value** immediately. The value isn't displayed again after you leave the page.

> ⓘ **Note**
>
> Your tenant policy might limit the maximum lifetime for client secrets.

Add a federated credential

Federated credentials use workload identity federation to establish trust between Microsoft Entra and external identity providers without storing secrets. For more information about these scenarios and how workload identity federation works, see Workload identity federation.

Take the following steps to add a federated credential for an agent identity blueprint principal.

1. From the blueprint's management page, select **Credentials** under **Developer settings**.
2. Select the **Federated credentials** tab.
3. Select **Add credential**.
4. Under **Federated credential scenario**, select a scenario:

    - **Managed Identity** — Configure a managed identity to get tokens and access resources across tenants.
    - **GitHub Actions deploying Azure resources** — Configure a GitHub workflow to get tokens and deploy to Azure.
    - **Kubernetes accessing Azure resources** — Configure a Kubernetes service account to get tokens and access Azure resources.
    - **Other issuer** — Configure an identity managed by an external OpenID Connect provider.

5. Complete the required fields for the selected scenario.
6. Select **Add**.

Manage owners and sponsors

Owners and sponsors help establish governance for your blueprint. From the **Owners and sponsors** page in the left menu, you can manage ownership for both the agent blueprint and the

agent blueprint principal.

For detailed steps on adding, removing, and managing owners and sponsors, see Add and manage owners and sponsors for agent identities and blueprints.

View audit and sign-in logs

You can view logs for both the agent blueprint principal and the blueprint itself from the blueprint's management page.

Agent blueprint principal activity

The agent blueprint principal supports both audit and sign-in logs:

- To view administrative actions taken on the blueprint principal, select **Audit logs** from the left menu under **Agent blueprint principal activity**.
- To view sign-in activity for the blueprint principal, select **Sign-in logs** from the left menu under **Agent blueprint principal activity**. For more information, see view sign-in logs for agents.

Agent blueprint activity

The agent blueprint supports only audit logs:

- To view administrative actions related to the blueprint configuration, select **Audit logs** from the left menu under **Agent blueprint activity**.

Disable an agent identity blueprint principal

To disable an agent identity blueprint principal, select the **Disable** button in the command bar at the top of the blueprint's overview page. A confirmation dialog appears warning that existing agent identities created from this blueprint **will no longer be able to authenticate**. Confirm the action to proceed.

Last updated on 05/01/2026

Add and manage owners and sponsors for agent identities and blueprints

Owners and sponsors play distinct governance roles for agent identity blueprints and agent identities in Microsoft Entra ID. Owners are technical administrators who can manage the configuration and operations of an agent. Sponsors are business owners who are accountable for the agent's purpose, lifecycle decisions, and access reviews.

This article walks you through adding and removing owners and sponsors using the Microsoft Entra admin center. For more information about the roles and responsibilities of owners and sponsors, see owners, sponsors, and managers.

Prerequisites

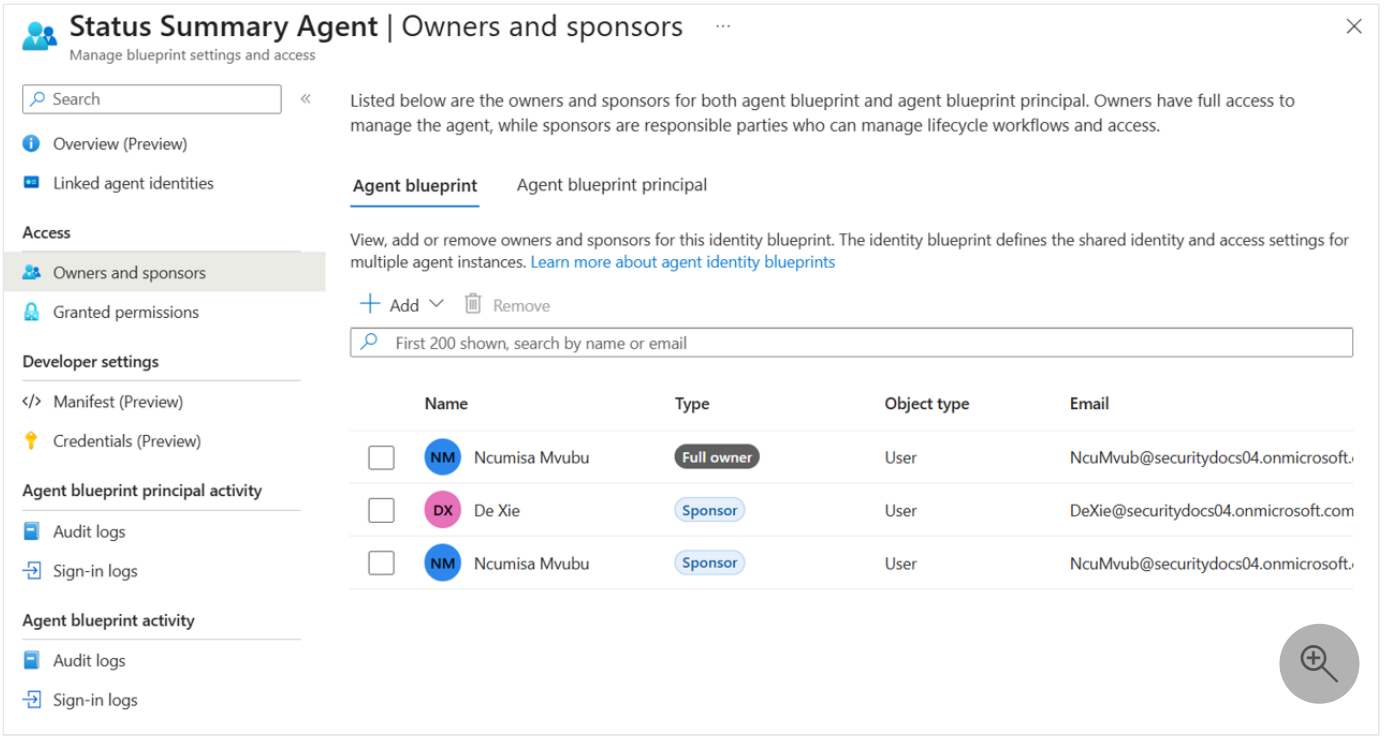
To manage owners and sponsors, you must:

- Have the Agent ID Administrator role.
- Be an existing owner of the agent identity blueprint or agent identity you want to manage.

Add an owner or sponsor to an agent blueprint

When managing agent identity blueprint owners and sponsors, you can assign them to either the agent identity blueprint or the agent blueprint principal using the respective tabs.

> **Note**
>
> When you use a dynamic membership group as a sponsor, it can take up to 24 hours after a membership rule change or a user property change before the authorization check on sponsorship succeeds. Plan accordingly when assigning dynamic groups as sponsors for agent identities or blueprints.



1. Sign in to the Microsoft Entra admin center as an Agent ID Administrator or owner of the agent identity blueprint.
2. Browse to **Entra ID** > **Agents** > **Agent blueprints**.
3. Select the blueprint you want to manage.
4. Under **Access** select **Owners and sponsors**.
5. Select either the **Agent blueprint** or **Agent blueprint principal** tab, depending on which you want to manage.
6. Select **Add** > **Add owner** or **Add sponsor**, depending on which you want to add.
7. Search for and select the users and groups (for sponsors only) you want to add.
8. Select **Add**.

Add an owner or sponsor to an agent identity

The process for adding owners and sponsors to individual agent identities is similar to blueprints.

1. Sign in to the Microsoft Entra admin center as an Agent ID Administrator or owner of the agent identity.
2. Browse to **Entra ID** > **Agents** > **Agent identities**.
3. Select the agent identity you want to manage.
4. Under **Access** select **Owners and sponsors**.
5. Select **Add** > **Add owner** or **Add sponsor** depending on which you want to add.
6. Search for and select the users and groups (for sponsors only) you want to add.
7. Select **Add**.

Remove an owner or sponsor

Remove from blueprints

1. Sign in to the Microsoft Entra admin center as an Agent ID Administrator or owner of the agent identity blueprint.
2. Browse to **Entra ID** > **Agents** > **Agent blueprints**.
3. Select the blueprint you want to manage.
4. Select **Owners and sponsors** from the left menu.
5. Select either the **Agent blueprint** or **Agent blueprint principal** tab, depending on which you want to manage.
6. Select the checkbox next to the owner or sponsor you want to remove.
7. Select **Remove**.

Remove from agent identities

1. Sign in to the Microsoft Entra admin center as an Agent ID Administrator or owner of the agent identity.
2. Browse to **Entra ID** > **Agents** > **Agent identities**.
3. Select the agent identity you want to manage.
4. Under **Access** select **Owners and sponsors**.
5. Select the checkbox next to the owner or sponsor you want to remove.
6. Select **Remove**.

Related content

- Owners, sponsors, and managers
- View and manage agent identity blueprints
- Create an agent identity blueprint

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/28/2026

Agent autonomous app OAuth flow - App-only protocol

App-only operations enable agent identities to act autonomously without user context, using client credentials flows. The agent identity (actor) is used to obtain a token for itself (subject). To obtain this token, the agent identity blueprint impersonates the agent identity. Subjects use app-only access but are supposed to be only assigned the permissions necessary. Tenant administrators grant all permissions.

Agent identity blueprints can only impersonate their child agent identities. Only a single agent identity blueprint can impersonate an agent identity. An agent identity blueprint can impersonate many agent identities, but no agent identity can be owned by multiple blueprints. Agent identities are always single-tenant regardless of their parent Agent identity blueprint's tenancy model. Each agent identity operates within one tenant's security and policy boundaries.

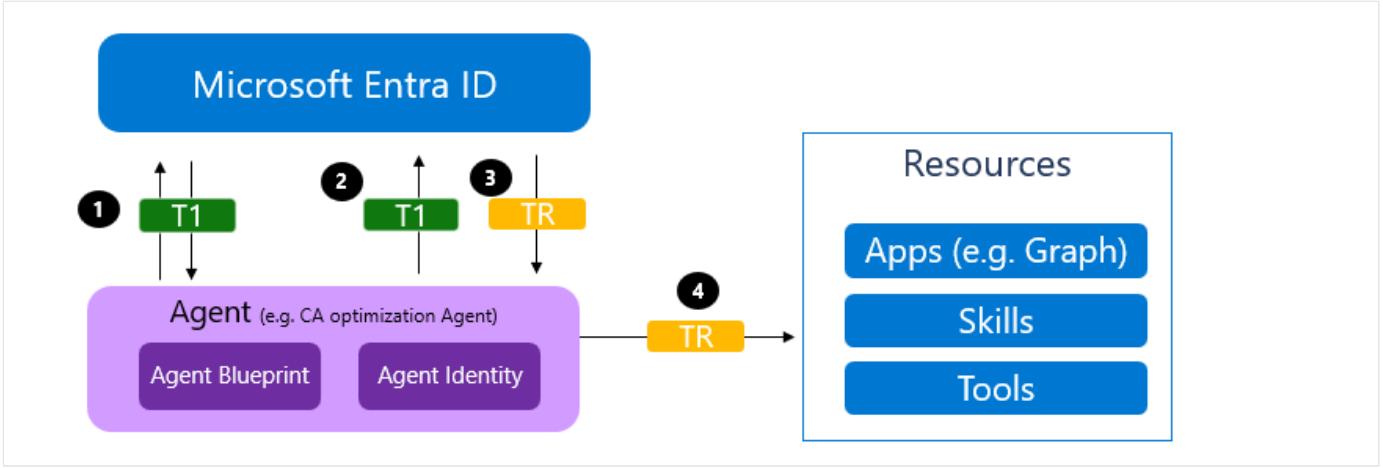
> ⚠ **Warning**
>
> Microsoft recommends using the approved SDKs like Microsoft.Identity.Web and Microsoft Entra ID Auth SDK (sidecar) libraries to implement these protocols. Manual implementation of these protocols is complex and error-prone, and using the SDKs helps ensure security and compliance with best practices.

Managed identities integration

Managed identities are the preferred credential type. In this configuration, the managed identity token serves as the credential for the parent agent identity blueprint, while standard MSI protocols apply for credential acquisition. This integration allows the agent ID to receive the full benefits of MSI security and management, including automatic credential rotation and secure storage.

Protocol steps

The following are the protocol steps.



1. Agent identity blueprint requests an exchange token T1. The agent identity blueprint presents its credentials that could be a secret, a certificate, or a managed identity token. Microsoft Entra ID returns the T1 to the agent identity blueprint. In this example we use a managed identity as Federated Identity Credential (FIC).

   > ⚠ **Warning**
   >
   > Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

   ```
   POST /oauth2/v2.0/token
   Content-Type: application/x-www-form-urlencoded

   client_id=AgentBlueprint
   &scope=api://AzureADTokenExchange/.default
   &fmi_path=AgentIdentity
   &client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
   &client_assertion=TUAMI
   &grant_type=client_credentials
   ```

   - `fmi_path`: The client ID (app ID) of the agent identity. This parameter tells Microsoft Entra ID which child agent identity the blueprint is impersonating during the token exchange.

   Where TUAMI is the managed identity token for user assigned managed identity (UAMI). This step returns T1. Where T1 is the token-exchange token for FIC.

2. Agent identity sends a token exchange request to Microsoft Entra ID. The request includes the token T1.

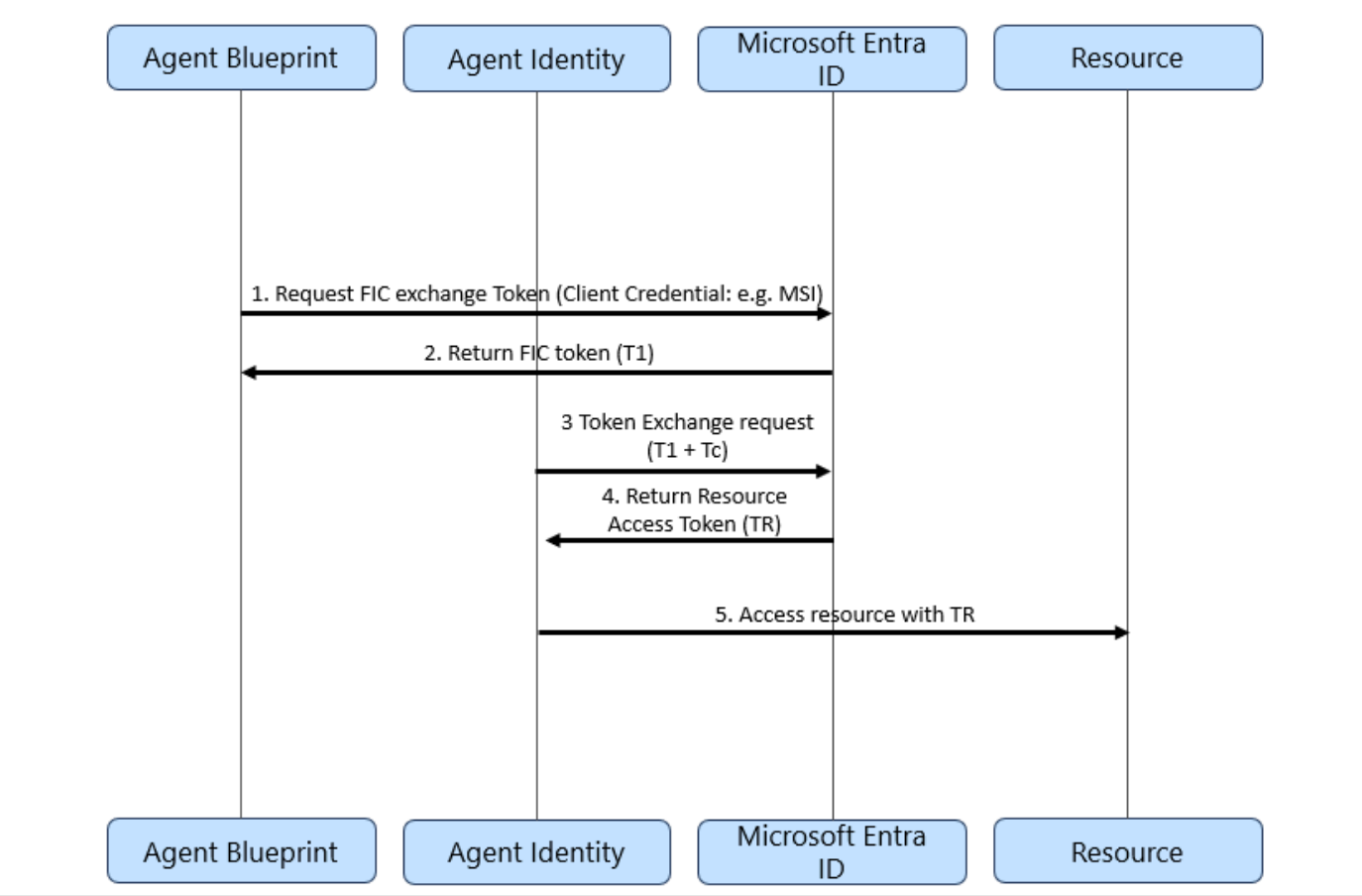
```
POST /oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=AgentIdentity
&scope=https://resource.example.com/.default
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion={T1}
&grant_type=client_credentials
```

3. Microsoft Entra ID issues an app-only resource access token (TR) to the agent identity after validating T1. Microsoft Entra ID validates that T1 (aud) == Agent identity parent app == Agent identity blueprint

Sequence diagram

The following is a sequence diagram for the app-only flow:



Related content

- Oauth2.0 flows for agents
- On-behalf-of flow in agents
- Agent's user account flow in agents

Last updated on 06/11/2026

Agent OAuth flows: On behalf of flow

Agents (agent identity blueprints) operating on behalf of regular, signed-in users use the standard OAuth 2.0 protocol with all its capabilities. User delegation enables agent identities to operate on behalf of signed-in users using standard OAuth 2.0 On-Behalf-Of flows with agent-specific impersonation. The agent identity is assigned the necessary delegated permissions needed for OBO access. It requires consent from users to access their data.

Agents have the capabilities of Microsoft Entra ID resource (API) applications and support the API attributes required for the (OAuth2Permissions, AppURI). Agent identity blueprints can't initiate interactive authorization (`/authorize`) flows directly. They must receive a user token from a client application and then perform an OBO token exchange. A web redirect URI can be configured on a blueprint for consent flows only (`response_type=none`), but it has limited functionality compared to a redirect URI on an app registration.

> **ⓘ Important**
>
> Like their parent blueprints, child agent identities can't initiate interactive `/authorize` flows. Consequently, users can't grant them consent interactively (attempting this on the child identity returns the error `AADSTS82014`). Instead, you must preauthorize the required delegated permissions by configuring inheritable permissions on the parent agent identity blueprint. Ensure an administrator has actually granted consent for these permissions on the blueprint. The child agent identities will then inherit these scopes without triggering an interactive consent prompt. For step-by-step guidance, see Configure inheritable permissions for agent identity blueprints.

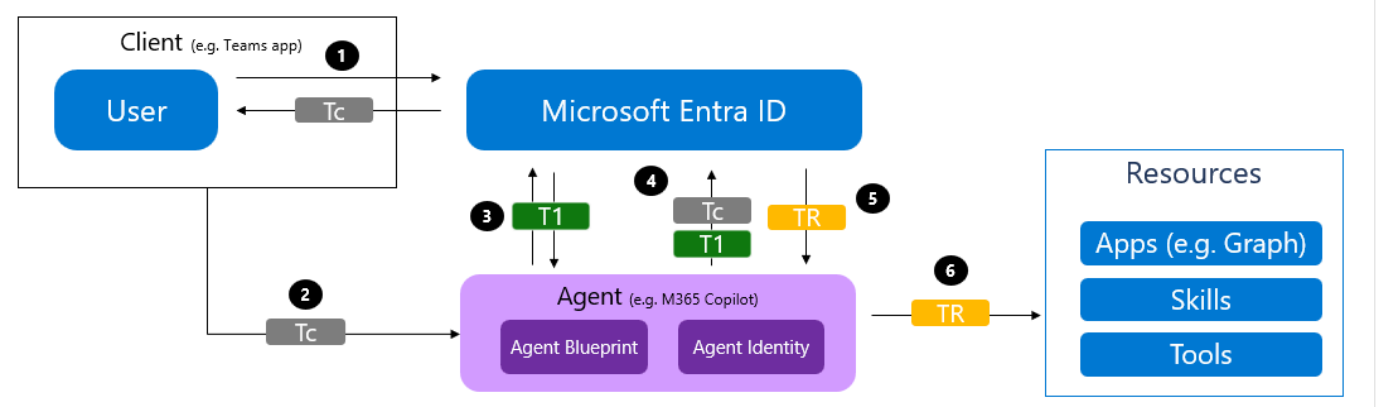
> **⚠ Warning**
>
> Microsoft recommends using the approved SDKs like Microsoft.Identity.Web and Microsoft Entra ID Auth SDK (sidecar) libraries to implement these protocols. Manual implementation of these protocols is complex and error-prone, and using the SDKs helps ensure security and compliance with best practices.

Managed identities integration

Managed identities are the preferred credential type. In this configuration, the managed identity token serves as the credential for the parent agent identity blueprint, while standard MSI protocols apply for credential acquisition. This integration allows the agent ID to receive the full benefits of MSI security and management, including automatic credential rotation and secure storage.

Protocol steps

Agents aren't supported for interactive (`/authorize`) flows. Supported grant types are `client_credential`, `jwt-bearer`, and `refresh_token`. The flow involves the agent identity blueprint, agent identity, and a client credential. The client credential can be a client secret, a client certificate, or a Federated Identity Credential (FIC). When possible, use a managed identity to obtain the FIC.



1. The user authenticates with the client and obtains a user access token (client token, Tc).

2. Client sends the user access token (Tc) to the agent identity blueprint to act on behalf of the user. It's the token that is used for the OBO exchange for the agent identity blueprint.

3. The agent identity blueprint requests an exchange token by presenting its client credential (secret, certificate, or managed identity token (TUAMI)). In this example, we use a managed identity as FIC. Microsoft Entra ID returns token T1 to the agent.

> **⚠ Warning**
>
> Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

```
POST /oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=AgentBlueprint
&scope=api://AzureADTokenExchange/.default
&fmi_path=AgentIdentity
&client_assertion=TUAMI
&grant_type=client_credentials
```

- `fmi_path`: The client ID (app ID) of the agent identity. This parameter tells Microsoft Entra ID which child agent identity the blueprint is impersonating during the token exchange.

Where TUAMI is the managed identity token for user assigned managed identity (UAMI). This step returns T1.

4. The agent identity, a child of the agent identity blueprint, sends an OBO token exchange request. This request includes both T1 and the user access token Tc. Note that `client_id` switches from the blueprint (in the previous step) to the **agent identity** here, because the agent identity is the entity performing the OBO exchange.

```
POST /oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=AgentIdentity
&scope=https://resource.example.com/scope1
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion={T1}
&grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
&assertion={Tc(aud=AgentIdentity Blueprint, oid=User)}
&requested_token_use=on_behalf_of
```

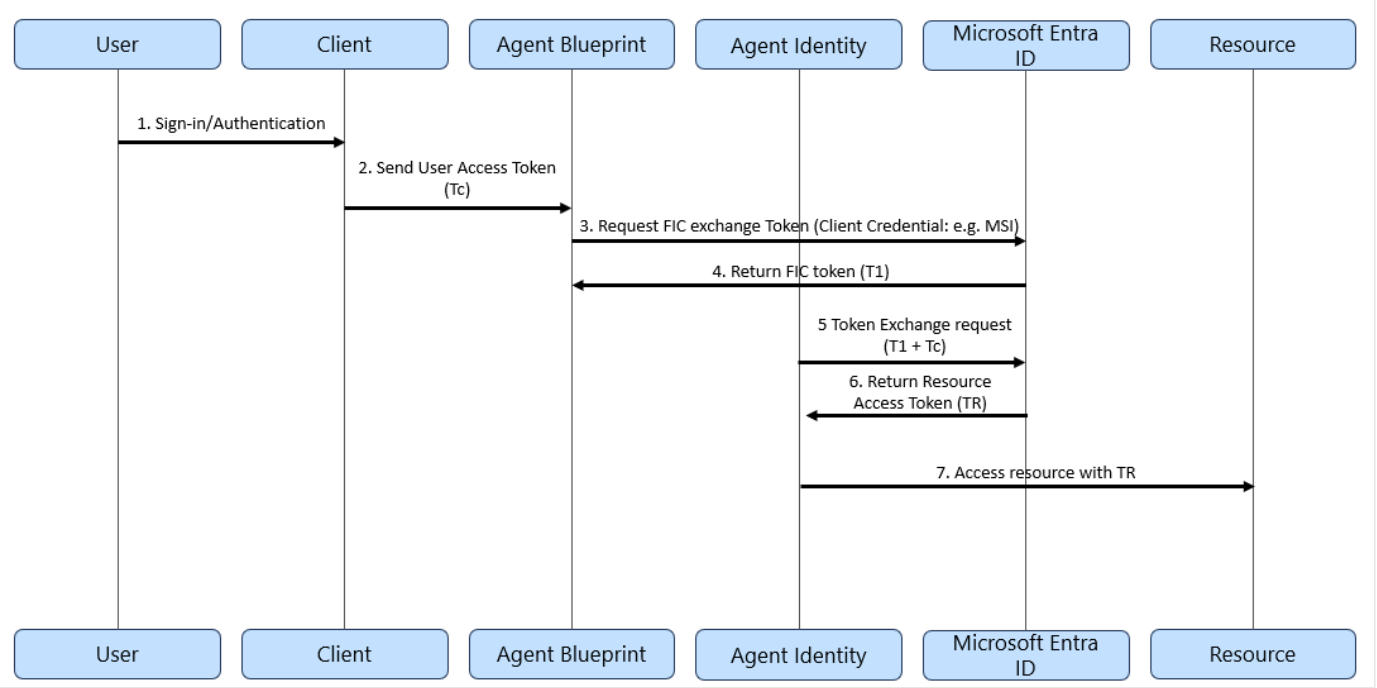
5. Microsoft Entra ID returns the resource token after validating both T1 and Tc. The following audience and linkage requirements apply:

- Tc (aud) == agent identity blueprint client ID. The user assertion must be audienced to the blueprint; a token audienced to another resource (for example, Microsoft Graph) is rejected with `AADSTS50013`.
- T1 is obtained with `scope=api://AzureADTokenExchange/.default`, so its `aud` is the token-exchange resource rather than the blueprint. Microsoft Entra ID validates that

T1 is bound to the blueprint (its `azp` is the blueprint) and that T1's `sub` (the FMI path) resolves to the child agent identity performing the exchange.

Sequence diagram

The following sequence diagram shows the OBO flow



Refresh token support

Refresh token can be used for asynchronous scenarios and background processes:

```
POST /oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=AgentIdentity
&scope=https://resource.example.com/scope1
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion={T1}
&grant_type=refresh_token
&refresh_token={AgentIdentityRefreshToken}
```

Permission inheritance

Agent identities can inherit delegated permissions from their parent agent identity blueprint when the `InheritDelegatedPermissions` property is enabled. This inheritance mechanism reduces consent complexity for multi-instance scenarios by allowing agent identities to use permissions

already granted to their parent application. The inheritance functionality applies specifically when FIC impersonation is used and enables efficient permission management across multiple instances. However, inheritance works only within tenant boundaries, ensuring that permission scope remains contained within appropriate organizational limits.

Related content

- Oauth2.0 flows for agents
- Autonomous app flow in agents
- Agent's user account flow in agents
- Configure inheritable permissions for agent identity blueprints

---

Last updated on 08/10/2026

Agent's user account impersonation protocol

Agent's user account impersonation enables agent identities to operate with user context through an agent's user account, combining user permissions with autonomous operation. In this scenario, an agent identity blueprint (actor 1) impersonates an agent identity (actor 2) that impersonates an agent's user account (subject) using FIC. Access is scoped to delegations assigned to the agent identity. The agent's user account can be impersonated only by a single agent identity.

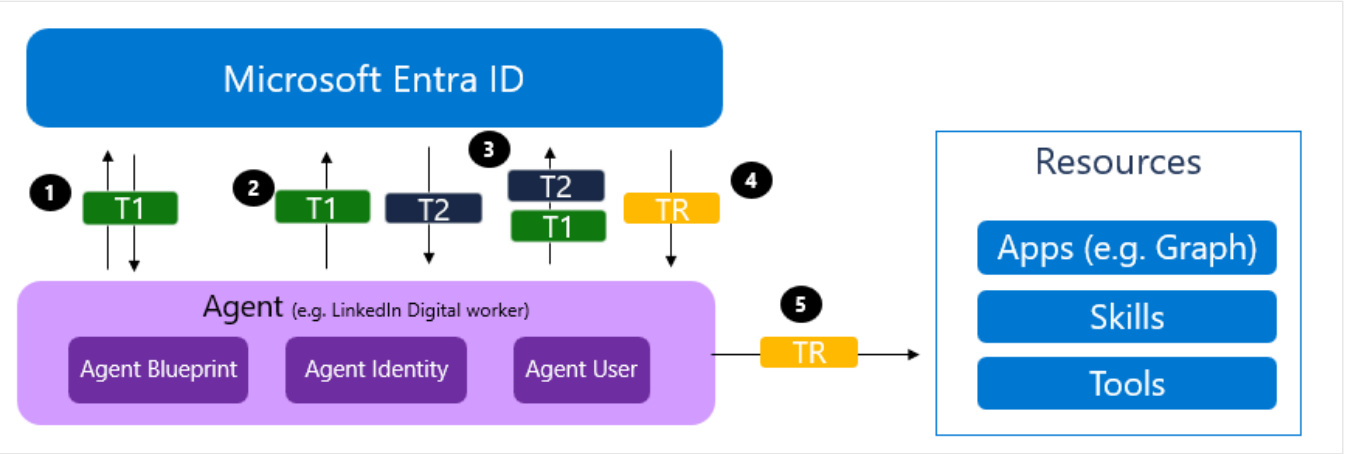
> ⚠ **Warning**
>
> Microsoft recommends using the approved SDKs like Microsoft.Identity.Web and Microsoft Agent ID SDK libraries to implement these protocols. Manual implementation of these protocols is complex and error-prone, and using the SDKs helps ensure security and compliance with best practices.

Managed identities integration

Managed identities are the preferred credential type. In this configuration, the managed identity token serves as the credential for the parent agent identity blueprint, while standard MSI protocols apply for credential acquisition. This integration allows the agent ID to receive the full benefits of MSI security and management, including automatic credential rotation and secure storage.

Protocol steps

Then following are the protocol steps.



1. The agent identity blueprint requests an exchange token (T1) that it uses for agent identity impersonation. The agent identity blueprint presents client credentials that could be a secret, a certificate, or a managed identity token used as an FIC.

   > ⚠ **Warning**
   >
   > Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

   ```
   POST /oauth2/v2.0/token
   Content-Type: application/x-www-form-urlencoded

   client_id=AgentBlueprint
   &scope=api://AzureADTokenExchange/.default
   &fmi_path=AgentIdentity
   &client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
   &client_assertion=TUAMI
   &grant_type=client_credentials
   ```

   Where TUAMI is the MSI token for user assigned managed identity (UAMI). This returns token T1.

2. The agent identity requests a token (T2) that it uses for agent's user account impersonation. The agent identity presents T1 as its client assertion. Microsoft Entra ID returns T2 to the agent identity after validating that T1 (aud) == Agent identity parent app == Agent identity blueprint.

```
POST /oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=AgentIdentity
&scope=api://AzureADTokenExchange/.default
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion={T1}
&grant_type=client_credentials
```

This returns token T2.

3. The agent identity then sends an OBO token exchange request to Microsoft Entra ID, including both T1 and T2. Microsoft Entra ID validates that T2 (aud) == agent identity.

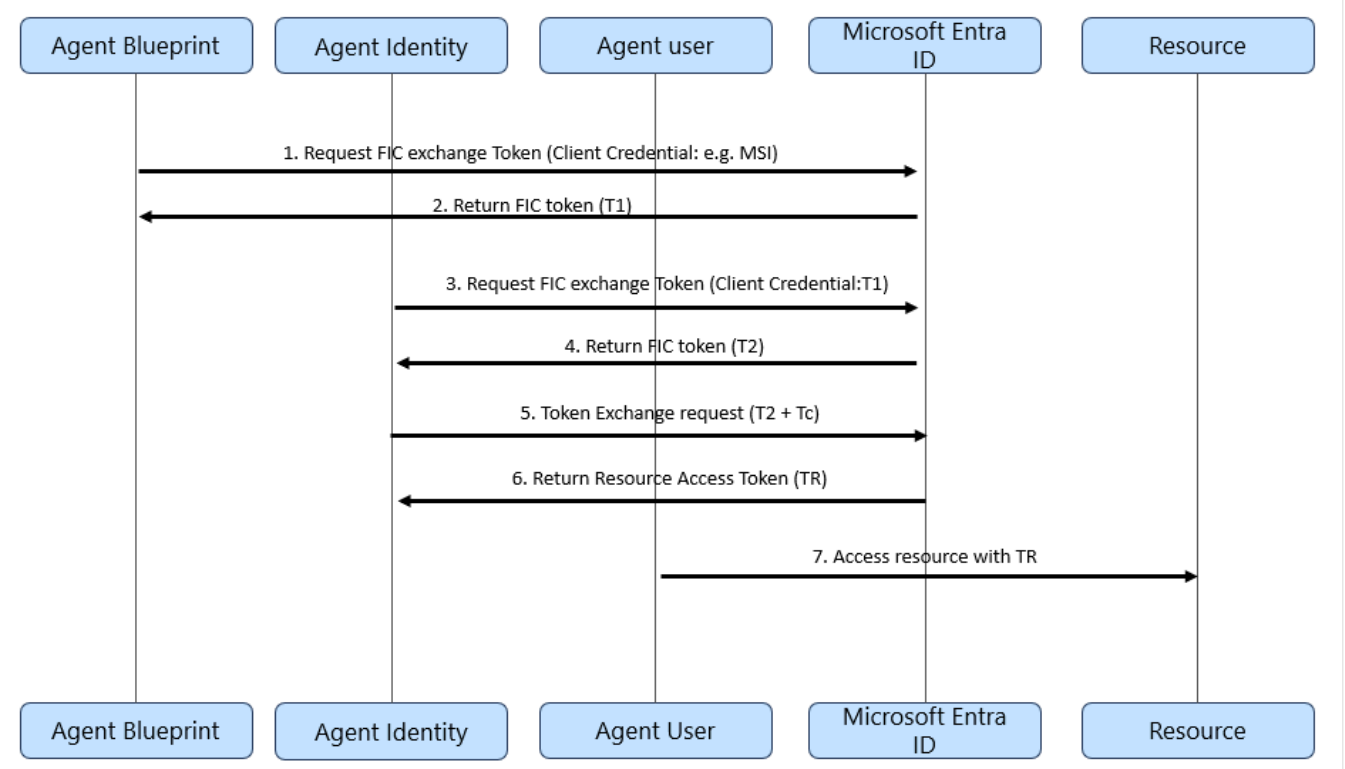
```
POST /oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=AgentIdentity
&scope=https://resource.example.com/scope1
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion={T1}
&user_federated_identity_credential={T2}
&username=agentuser@contoso.com
&grant_type=user_fic
&requested_token_use=on_behalf_of
```

4. Microsoft Entra ID then issues the resource token.

Sequence diagram

The following sequence diagram shows the agent's user account impersonation flow



Agent's user account impersonation requires credential chaining that follows the pattern agent identity blueprint → Agent identity → Agent's user account. Each step in this chain uses the token from the previous step as a credential, creating a secure delegation pathway. The same client ID must be used for both phases to prevent privilege escalation attacks.

Last updated on 04/08/2026

Configure inheritable permissions for agent identity blueprints

Configure inheritable permissions on an agent identity blueprint to preauthorize a base set of delegated scopes and application roles. Agent identities created from the blueprint automatically inherit those permissions without interactive consent prompts.

For conceptual background on how inheritable permissions relate to required resource access and direct permission grants, see Inheritable permissions and required resource access.

Prerequisites

- An existing agent identity blueprint already created and configured
- Either of the following permissions:
  - Agent ID Developer role for managing agent identity blueprints owned by the user
  - Agent ID Administrator role for managing agent identity blueprints

How inheritable permissions work

During token issuance for an agent identity, the platform merges eligible inherited scopes with the agent's requested delegated scopes. Inherited scopes appear in the access token's **scp** claim and inherited roles appear in the **roles** claim. For details on inheritance conditions and the relationship between declarations, grants, and effective permissions, see Inheritable permissions and required resource access.

Inheritance patterns

The concept article describes inheritance patterns at a high level. The following table shows the specific `kind` values used in the API when you configure inheritance per resource app:

Expand table

Inheritance	Kind	Description
All Allowed	`allAllowed`	Inherit all available delegated scopes or application roles for the specified resource app. Newly granted scopes or roles on the agent identity blueprint principal are automatically included.

Inheritance	Kind	Description
None	`none`	Inherit no scopes or roles for the specified resource app. Use this to explicitly disable inheritance for scopes (`noScopes`) or roles (`noRoles`) independently.

You can configure scopes and roles independently on the same resource. For example, you can inherit all scopes while inheriting no roles, or vice versa.

Inheritable permissions limitations

- Maximum of 50 resource apps per agent identity blueprint (for example, up to 50 entries in the *inheritablePermissions* collection). If you exceed this limit, reduce the number of resource apps to stay within the supported boundary.

Regularly review and monitor your inheritable permissions configuration. Reevaluate inherited scopes and roles to ensure they remain appropriate for your use case. Audit which inherited scopes and roles are being used by agents and remove any unused permissions from both the agent identity blueprint principal and the inheritable permissions list to maintain security hygiene.

Configure inheritable permissions (using Microsoft Graph)

To configure inheritable permissions, use the inheritablePermissions navigation property on the `agentIdentityBlueprint` application resource. Each entry specifies the scopes and roles inheritance configuration for a single resource app. Document your configuration decisions by tracking why each scope or role is inheritable and who approved it for audit purposes.

When specifying the `resourceAppId` in your requests, ensure you provide a valid GUID format. Invalid GUIDs result in 400 Bad Request errors.

Add all scopes and roles inheritance for Microsoft Graph

Request

HTTP

```
POST
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint/bc057821-f236-49d6-9f2c-1ebf43e9437a/inheritablePermissions
Content-Type: application/json
OData-Version: 4.0
```

```
{
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "#microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
  "inheritableRoles": {
    "@odata.type": "#microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
  }
}
```

Response

HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "@odata.context":
"https://graph.microsoft.com/v1.0/$metadata#applications('bc057821-f236-49d6-9f2c-
1ebf43e9437a')/inheritablePermissions/$entity",
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
  "inheritableRoles": {
    "@odata.type": "microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
  }
}
```

Add all scopes and roles inheritance for multiple resources

You can configure inheritable permissions for multiple resource apps on the same blueprint. Each resource requires a separate POST request. The following example adds inheritance for both Microsoft Graph and SharePoint Online.

Request (Microsoft Graph)

HTTP

```
POST
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint/b
c057821-f236-49d6-9f2c-1ebf43e9437a/inheritablePermissions
Content-Type: application/json
```

```
OData-Version: 4.0

{
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "#microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
  "inheritableRoles": {
    "@odata.type": "#microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
  }
}
```

Response

HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "@odata.context":
"https://graph.microsoft.com/v1.0/$metadata#applications('bc057821-f236-49d6-9f2c-
1ebf43e9437a')/inheritablePermissions/$entity",
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
  "inheritableRoles": {
    "@odata.type": "microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
  }
}
```

Request (SharePoint Online)

HTTP

```
POST
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint/b
c057821-f236-49d6-9f2c-1ebf43e9437a/inheritablePermissions
Content-Type: application/json
OData-Version: 4.0

{
  "resourceAppId": "00000003-0000-0ff1-ce00-000000000000",
  "inheritableScopes": {
    "@odata.type": "#microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
```

```
  "inheritableRoles": {
    "@odata.type": "#microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
  }
}
```

Response

HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "@odata.context":
"https://graph.microsoft.com/v1.0/$metadata#applications('bc057821-f236-49d6-9f2c-
1ebf43e9437a')/inheritablePermissions/$entity",
  "resourceAppId": "00000003-0000-0ff1-ce00-000000000000",
  "inheritableScopes": {
    "@odata.type": "microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
  "inheritableRoles": {
    "@odata.type": "microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
  }
}
```

Add scopes inheritance only (no roles)

To inherit delegated scopes but not application roles, set `inheritableRoles` to `noRoles`.

Request

HTTP

```
POST
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint/b
c057821-f236-49d6-9f2c-1ebf43e9437a/inheritablePermissions
Content-Type: application/json
OData-Version: 4.0

{
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "#microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
  "inheritableRoles": {
    "@odata.type": "#microsoft.graph.noRoles",
```

```
    "kind": "none"
  }
}
```

Response

HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "@odata.context":
"https://graph.microsoft.com/v1.0/$metadata#applications('bc057821-f236-49d6-9f2c-
1ebf43e9437a')/inheritablePermissions/$entity",
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
  "inheritableRoles": {
    "@odata.type": "microsoft.graph.noRoles",
    "kind": "none"
  }
}
```

Add roles inheritance only (no scopes)

To inherit application roles but not delegated scopes, set `inheritableScopes` to `noScopes`.

Request

HTTP

```
POST
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint/b
c057821-f236-49d6-9f2c-1ebf43e9437a/inheritablePermissions
Content-Type: application/json
OData-Version: 4.0

{
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "#microsoft.graph.noScopes",
    "kind": "none"
  },
  "inheritableRoles": {
    "@odata.type": "#microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
```

```
  }
}
```

Response

HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "@odata.context":
"https://graph.microsoft.com/v1.0/$metadata#applications('bc057821-f236-49d6-9f2c-1ebf43e9437a')/inheritablePermissions/$entity",
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "microsoft.graph.noScopes",
    "kind": "none"
  },
  "inheritableRoles": {
    "@odata.type": "microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
  }
}
```

Update to disable roles inheritance

If an entry already exists for a resourceAppId, use PATCH to update it rather than attempting to create a duplicate entry, which would result in a 409 Conflict error. The following example disables role inheritance while keeping scope inheritance enabled.

Request

HTTP

```
PATCH
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint/bc057821-f236-49d6-9f2c-1ebf43e9437a/inheritablePermissions/00000003-0000-0000-c000-000000000000
Content-Type: application/json
OData-Version: 4.0

{
  "inheritableRoles": {
    "@odata.type": "#microsoft.graph.noRoles",
    "kind": "none"
  }
}
```

Response

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "@odata.context":
"https://graph.microsoft.com/v1.0/$metadata#applications('bc057821-f236-49d6-9f2c-
1ebf43e9437a')/inheritablePermissions/$entity",
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "microsoft.graph.allAllowedScopes",
    "kind": "allAllowed"
  },
  "inheritableRoles": {
    "@odata.type": "microsoft.graph.noRoles",
    "kind": "none"
  }
}
```

Update to disable scopes inheritance

The following example disables scope inheritance while keeping role inheritance enabled.

Request

HTTP

```
PATCH
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint/b
c057821-f236-49d6-9f2c-1ebf43e9437a/inheritablePermissions/00000003-0000-0000-c000-
000000000000
Content-Type: application/json
OData-Version: 4.0

{
  "inheritableScopes": {
    "@odata.type": "#microsoft.graph.noScopes",
    "kind": "none"
  }
}
```

Response

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "@odata.context":
"https://graph.microsoft.com/v1.0/$metadata#applications('bc057821-f236-49d6-9f2c-
1ebf43e9437a')/inheritablePermissions/$entity",
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "microsoft.graph.noScopes",
    "kind": "none"
  },
  "inheritableRoles": {
    "@odata.type": "microsoft.graph.allAllowedRoles",
    "kind": "allAllowed"
  }
}
```

Delete existing inheritable permissions

Request

HTTP

```
DELETE
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint/b
c057821-f236-49d6-9f2c-1ebf43e9437a/inheritablePermissions/00000003-0000-0000-c000-
000000000000
OData-Version: 4.0
```

Response

HTTP

```
HTTP/1.1 204 No Content
```

Related content

- Inheritable permissions and required resource access
- Create an agent identity from your blueprint
- Microsoft Entra roles and permissions for agent identities
- OAuth 2.0 and OpenID Connect protocols on the Microsoft identity platform

---

Last updated on 05/22/2026

Grant agents access to Microsoft 365 resources

This article provides guidance on how to grant access to agents through consent, manual authorization, and other authorization systems. Learn about the different methods available for authorizing agents to access Microsoft 365 resources and when to use each approach.

Prerequisites

- An agent identity blueprint and at least one agent identity created from it.
- An agent identity blueprint with a valid redirect URI.

How to request consent (delegated or application permissions)

Users or administrators can grant agents access to data by consenting to API permissions during the OAuth flow. This section explains how to request consent for delegated and application permissions for your agents.

When to use delegated or application permissions

The type of permissions you request depends on how your agent operates and what resources it needs to access.

Use delegated permissions when your interactive agent needs to act on behalf of a signed-in user. For example, read that user's mail, calendar, or files. Delegated access is carried in the token's scp claim.

Use application permissions when your autonomous agent runs without a user present and requires app-only access. For example, to read all users' profiles. App permissions appear in the token's roles claim.

For more information, see permissions and consent overview

How consent works

For delegated permissions, when you redirect the user to the Microsoft identity platform `/authorize` endpoint, they review scopes such as `User.Read` and `Mail.Read`. If consent is granted, Microsoft Entra ID records an OAuth2PermissionGrant from your agent (client) to the resource such as Microsoft Graph. Future delegated tokens for that resource include the approved scopes in `scp` without reprompting unless consent changes. If your app requests consent for admin restricted permissions, your users get an error. An admin requests these permissions directly. For more information, see admin-restricted permissions.

Building the consent URL

When building your consent URL, the client ID should be your agent identity's ID.

For more information, see requesting permissions through consent. Some permissions require consent from an administrator before they can be granted within a tenant. For more information, see admin consent on the Microsoft identity platform

How to manually grant authorization (delegated or application)

You can create the underlying authorization objects directly. It's useful for situations where you want to automate authorization or when you want to avoid interactive prompts.

- Manually grant delegated permissions. Set your client ID as the agent identity ID.
- Manually grant application permissions (app role assignment). Set your principal ID as the agent identity ID.

How to manage access through access packages

Using access packages, you can enable standardized access for many AI Agents with the same access needs. The access package can include Entra roles, OAuth2 delegated and application permission grants and security group memberships. Agents can then request an access package, or a sponsor or admin can request for them, and once approved, the agent identity or agent's user account receives the access rights until the assignment is revoked or expires. For more information, see access packages for agent identities.

Other authorization systems

Agents can be authorized in several ways beyond app-role assignments, group memberships and OAuth2 permission grants. These alternative systems offer flexibility for assigning access tailored

to different platforms, services, and security requirements. The following sections summarize some of the many methods for agent authorization.

Azure role-based access control (Azure RBAC)

Assign Azure roles to the agent identity at the narrowest scope (resource, resource group, subscription). For example, you can grant the Key Vault Reader role on a single vault so the agent can read secrets without needing broad directory or tenant permissions. For more information and step-by-step guidance, see Azure RBAC documentation.

Microsoft Entra roles

Some low-privilege directory roles might be assignable to agents for metadata or read scenarios. High-privilege roles are blocked for agents by platform policy. To learn more about Microsoft Entra roles and PIM, see Microsoft Entra roles.

Exchange RBAC

Exchange RBAC (Role-Based Access Control) lets administrators delegate permissions to Exchange resources. It ensures agents can get granular authorization, such as autonomous access to one mailbox or a few mailboxes. For information, see Role Based Access Control for Applications in Exchange Online

Teams Resource-Specific Consent (RSC)

Teams Resource-Specific Consent (RSC) enables granular permission assignments for agents within Microsoft Teams. RSC allows you to grant permissions to an app or agent on a per-team basis, rather than at the tenant level. This approach is especially useful when you want to limit access to only the resources and data within specific teams. Such data includes channels, messages, or roster information, without granting broader organizational permissions. For more information, see Resource-specific consent for your Teams app.

Permissions for agent communication across Microsoft 365 channels

An agent with its own identity (Agent ID) can communicate across Microsoft 365 surfaces, including sending and receiving Outlook email, receiving and responding to comments in OneDrive and SharePoint files, and sending and receiving messages from Teams chats and channels. Each surface the agent communicates in requires specific permissions, which you

declare in the agent identity blueprint's required resource access, and which a tenant admin must consent to before the agent can send or receive anything there.

Expand table

Channel	Displayed as	Inbound (channel → agent) Receiving events	Outbound (agent → channel) Sending responses
`email`	Outlook	`Mail.Read` or `Mail.ReadWrite`	`Mail.Send` or `Mail.ReadWrite`
`spo.files.comments`	OneDrive and SharePoint	`Files.Read.All` or `Files.ReadWrite.All`	`Files.ReadWrite.All`
`teams-chat`	Teams chats	`Chat.Read` or `Chat.ReadWrite`	`Chat.ReadWrite` or `ChatMessage.Send`
`teams-channel`	Teams channels	`ChannelMessage.Read.All`	`ChannelMessage.Send`

Custom (third-party) APIs

Agents can call other OAuth-protected APIs. Ensure the resource application and its service principal exist in the tenant, define required scopes/app roles, and follow the same delegated or app-permission flows. For more information, see Microsoft's guide to permissions and consent in the Microsoft identity platform.

Related content

- Authentication protocols in agents
- Permissions and consent in the Microsoft identity platform
- Microsoft Entra built-in roles

Last updated on 08/12/2026

Control user access to agents

This article describes how to control user and agent access to applications using app roles and the `assignmentRequired` property. It covers defining app roles, enforcing explicit role assignments, assigning roles using Microsoft Graph or the Microsoft Entra admin center, and best practices for secure access management.

Prerequisites

- An existing agent identity blueprint already created and configured
- Agent ID Administrator role for managing agent identities and assignments
- Application Administrator or Cloud Application Administrator role for managing app roles and assignmentRequired properties
- *AppRoleAssignment.ReadWrite.All* permission for Microsoft Graph API operations if using programmatic approach.

What are app roles?

App roles define logical permissions or access levels for an application. They're declared in the agent identity blueprint and can be assigned to users, groups, service principals, or agent identities. The following section shows an example app role definition:

JSON

```
{
  "id": "b1a2c3d4-e5f6-7890-abcd-ef1234567890",
  "allowedMemberTypes": ["User", "Application"],
  "description": "Grants ability to invoke agent actions",
  "displayName": "AgentInvoker",
  "isEnabled": true,
  "value": "AgentInvoker"
}
```

- `allowedMemberTypes` controls who can be assigned (users, apps, or both).
- `value` is the string that appears in the token's roles claim when assigned.

If an agent identity blueprint exposes no app-roles, the default app-role `<guid of all zeros>` can be used to assign principals to the agent identity.

When designing app roles for your agent applications, define granular roles that separate duties and responsibilities. For example, create specific roles like "AgentInvoker" for users who can call agent actions and "AgentAdmin" for users who can manage agent configurations. This approach provides better security control and makes it easier to audit access patterns.

What is assignmentRequired?

The assignmentRequired property on an application or agent identity determines whether role assignment is mandatory for access.

- `assignmentRequired = true` → Only principals explicitly assigned an app role can sign in or call the agent identity.
- `assignmentRequired = false` → Any authenticated principal can access the agent identity (subject to other conditions).

Why use assignmentRequired?

You might use the assignmentRequired property for the following reasons:

- Enforces least privilege by requiring explicit assignment.
- Prevents accidental access by users or agents who weren't intended to use the agent.

Enable `assignmentRequired` for sensitive agent identity blueprints to enforce explicit role assignment and ensure only authorized principals can access critical agent functionality. If users can access an application without explicit assignment, it indicates that `assignmentRequired` is set to `false`. Change this setting to true and assign roles explicitly to maintain security controls.

Configure assignmentRequired

The following example uses the Update application API to set the `assignmentRequired` property.

HTTP

```
PATCH https://graph.microsoft.com/v1.0/applications/<agent-app-id>
Content-Type: application/json
Authorization: Bearer <token>

{
  "appRoleAssignmentRequired": true
}
```

The `assignmentRequired` property ensures that only principals explicitly assigned an app role can access the agent identity blueprint, enforcing least privilege and preventing accidental access.

Assign app roles to control access

App roles are granted through app role assignments. They link a principal (user, group, or agent identity) to a specific app role on the target agent identity. If an agent identity fails to access an application after configuration, verify that an `appRoleAssignment` is created for the agent identity using the APIs shown in the following example.

Assign an app role to a user

The following example uses the Grant an `appRoleAssignment` to a user API to assign an app role to a user.

HTTP

```
POST https://graph.microsoft.com/v1.0/users/<user-id>/appRoleAssignments
Authorization: Bearer <token with AppRoleAssignment.ReadWrite.All>
Content-Type: application/json

{
  "principalId": "<user-id>",
  "resourceId": "<agent-identity-blueprint-principal-id>",
  "appRoleId": "<app-role-id>"
}
```

Assign an app role to an agent identity

The following example uses the Grant an `appRoleAssignment` to a service principal API to assign an app role to an agent identity.

HTTP

```
POST https://graph.microsoft.com/v1.0/servicePrincipals/<agent-identity-
id>/appRoleAssignments
Authorization: Bearer <token with AppRoleAssignment.ReadWrite.All>
Content-Type: application/json

{
  "principalId": "<agent-identity-id>",
  "resourceId": "<agent-identity-blueprint-principal-id>",
  "appRoleId": "<app-role-id>"
}
```

Verify role assignment

To verify your role assignment, be sure to take the following actions:

- List `appRoleAssignments` for the user or agent identity.
- Check the roles claim in the token after sign-in.

If roles claims are missing from tokens, verify that the assignment exists with the correct appRoleId, then request a new token. Regularly audit these assignments using Microsoft Graph or the Microsoft Entra admin center to ensure they remain appropriate and remove any unnecessary access.

Related content

- Application manifest reference
- Microsoft Entra roles and permissions for agent identities
- conditional access for enhanced security controls

Last updated on 04/21/2026

Authenticate and acquire tokens for autonomous agents

Autonomous agents perform operations using their own identity rather than acting as a delegate of a user. To operate securely, an autonomous agent must authenticate with Microsoft Entra ID, obtain access tokens, and be granted the appropriate permissions. This article walks you through the end-to-end flow:

1. Configure client credentials.
2. Request a token for the agent identity blueprint.
3. Request an agent identity token.
4. Grant application permissions (admin consent).
5. (Optional) Create and authenticate as an agent's user account for resources that require a user identity.

> **Note**
>
> This article covers autonomous agents that operate with their own identity. If your agent needs to act on behalf of a signed-in user, see Authenticate users and acquire tokens for interactive agents.

Prerequisites

Before implementing agent token authentication, ensure you have:

- An agent identity blueprint.
- An agent identity. You need the agent identity client ID.
- Understanding of OAuth protocols in Microsoft Entra Agent ID.

For admin authorization, you also need:

- Administrator privileges in your Microsoft Entra ID tenant.
- Understanding of the specific permissions your agent requires.

For agent's user account authentication, you also need:

- Understanding of agents' user accounts in Microsoft Entra Agent ID.

Configure your client credentials

Get your client credential details. This could be your client secret, a certificate, or a managed identity that you're using as a federated identity credential.

> ⚠ **Warning**
>
> Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

Microsoft Graph API

Gather the credential you configured on your agent identity blueprint. You need one of the following:

- **Managed identity** (recommended): The managed identity client ID and a token from the Azure Instance Metadata Service (IMDS).
- **Certificate**: The certificate thumbprint or PFX file.
- **Client secret** (development only): The secret value.

Request a token for the agent identity blueprint

When requesting the token for the agent identity blueprint, provide the agent identity's client ID in the `fmi_path` (Federated Managed Identity path) parameter. This parameter tells Microsoft Entra ID which agent identity the blueprint is acting on behalf of.

When using a client secret during local development, provide the `client_secret` parameter. For certificates and managed identities, use `client_assertion` and `client_assertion_type` instead.

Microsoft Graph API

Use the following token request to obtain a token for the agent identity blueprint:

```
HTTP
```

```
POST https://login.microsoftonline.com/<my-test-tenant>/oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=<agent-blueprint-client-id>
&scope=api://AzureADTokenExchange/.default
&grant_type=client_credentials
&client_secret=<client-secret>
&fmi_path=<agent-identity-client-id>
```

Request an agent identity token

Microsoft Graph API

Once you have the agent identity blueprint token (T1), use it to request the agent identity token.

HTTP

```
POST https://login.microsoftonline.com/<my-test-tenant>/oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=<agent-identity-client-id>
&scope=https://graph.microsoft.com/.default
&grant_type=client_credentials
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=<agent-blueprint-token-T1>
```

Grant application permissions

Agents often need to take actions in Microsoft Graph and other web services that require a Microsoft Entra ID application permission (represented as app roles). Autonomous agents need to request these permissions from a Microsoft Entra ID administrator.

There are two ways to grant application permissions to an autonomous agent:

- An admin can create an *appRoleAssignment* by using Microsoft Graph APIs or PowerShell.
- The agent can direct the admin to a consent page using an admin consent URL.

Create an app role assignment via APIs

Use the following steps to get an app role assignment.

1. Request an agent identity token with the application permissions `Application.Read.All` and `AppRoleAssignment.ReadWrite.All`.

2. Get the object ID of the resource service principal that you're trying to access. For example, to find the Microsoft Graph service principal object ID:
   a. Go to the Microsoft Entra admin center.
   b. Navigate to **Entra ID** --> **Enterprise Applications**
   c. Filter by Application type == Microsoft Applications
   d. Search for **Microsoft Graph**.

3. Get the unique ID of the app role you want to assign from the Microsoft Graph permissions reference.

4. Create the app role assignment:

Microsoft Graph API

HTTP

```
POST https://graph.microsoft.com/v1.0/servicePrincipals/<agent-identity-id>/appRoleAssignments
Authorization: Bearer <token>
Content-Type: application/json

{
  "principalId": "<agent-identity-id>",
  "resourceId": "<microsoft-graph-sp-object-id>",
  "appRoleId": "<app-role-id>"
}
```

Request authorization from a tenant administrator

To grant application permissions, construct the authorization URL that is used to prompt the administrator. The role parameter is used to specify the requested application permissions.

Be sure to use the agent identity client ID in the following request.

Bash

```
https://login.microsoftonline.com/contoso.onmicrosoft.com/v2.0/adminconsent
?client_id=<agent-identity-client-id>
&role=https://graph.microsoft.com/User.Read.All
```

```
&redirect_uri=https://entra.microsoft.com/TokenAuthorize
&state=xyz123
```

Agent implementations might redirect the admin to this URL in various ways, such as including it in a message sent to the admin in a chat window. When the admin is redirected to this URL, they're asked to sign in and grant consent to the permissions specified in the scope parameter. At the moment you must use the redirect URI listed, which directs the admin to a blank page after granting consent.

> **Note**
>
> Configure a redirect URI on your blueprint and include a `state` parameter in the consent request. When consent is granted, the user is sent to the redirect URI where you can display confirmation. Your endpoint can use the `state` parameter to track that permission was granted. For single-tenant agents, you can alternatively retry token requests until consent is granted because the tenant ID is already known.

After you grant your agent identity blueprint the required permissions, request a new agent access token for the permissions to take effect.

Authenticate as an agent's user account

In addition to operating with app-only tokens, autonomous agents can authenticate as an agent's user account. Agents' user accounts are a special type of user account in Microsoft Entra purpose-built for use by agents. They're most commonly used when an agent needs to connect to systems that require the existence of a user account, for instance, a mailbox, Teams channel, or other user-specific resources.

Each agent identity can only have a single associated agent's user account, and each agent's user account can only be associated with a single agent identity.

Get authorization to create agents' user accounts

To create agents' user accounts, your agent identity blueprint must be granted the application permission `AgentIdUser.ReadWrite.IdentityParentedBy` in the tenant. You can obtain authorization in one of two ways:

- Request authorization from a tenant administrator. Be sure to use your agent identity blueprint as the `client_id`, not the agent identity.

- Create an app role assignment via APIs in the tenant. Use the service principal object ID of the agent identity blueprint as the `principalId` value, not its application (client) ID.

If you wish to use a different client, not the agent identity blueprint, to create agents' user accounts, that client needs to obtain the `AgentIdUser.ReadWrite.All` delegated or application permission instead.

Create an agent's user account

Create an agent's user account using your agent identity blueprint or other approved client. The recommended way to create an agent's user account is using your agent identity blueprint. You need to get an access token using the agent identity blueprint to create an agent's user account.

REST

Once you have an access token with the necessary permission, make the following request:

HTTP

```
POST https://graph.microsoft.com/beta/users
OData-Version: 4.0
Content-Type: application/json
Authorization: Bearer <token>

{
  "@odata.type": "microsoft.graph.agentUser",
  "displayName": "New Agent User",
  "userPrincipalName": "agentuserupn@tenant.onmicrosoft.com",
  "identityParentId": "{agent-identity-id}",
  "mailNickname": "agentuserupn",
  "accountEnabled": true
}
```

Once you have your agent's user account created, you don't need to configure anything else. These accounts don't have any credentials and can only be authenticated using the protocol described in the next sections.

Grant consent to agent identity

Agents' user accounts behave like any other user account. Before you can request tokens using your agent's user account, you need to authorize the agent identity to act on its behalf. You can authorize the agent identity by requesting authorization from a tenant administrator or manually create an `oAuth2PermissionGrant` using Microsoft Graph or Microsoft Graph PowerShell.

For Microsoft Graph, your request is as shown in the following snippet:

HTTP

```
POST https://graph.microsoft.com/v1.0/oauth2PermissionGrants
Authorization: Bearer {token}
Content-Type: application/json

{
  "clientId": "{agent-identity-id}",
  "consentType": "Principal",
  "principalId": "{agent-id-user-object-id}",
  "resourceId": "{ms-graph-service-principal-object-id}",
  "scope": "Mail.Read"
}
```

For Microsoft Graph PowerShell, use the following script:

PowerShell

```
Connect-MgGraph -Scopes "DelegatedPermissionGrant.ReadWrite.All" -TenantId <your-test-tenant>

# Get the service principal for Microsoft Graph
$graphSp = Get-MgServicePrincipal -Filter "appId eq '00000003-0000-0000-c000-000000000000'"

# Get the service principal for your client app
$clientSp = Get-MgServicePrincipal -Filter "appId eq '{agent-identity-id}'"

# Create the delegated permission grant
New-MgOauth2PermissionGrant -BodyParameter @{
    clientId    = $clientSp.Id
    consentType = "Principal"
    principalId = "{agent-id-user-object-id}"
    resourceId  = $graphSp.Id
    scope       = "Mail.Read"
}
```

Request an agent's user account token

To authenticate an agent's user account, you need to follow a three-step process:

1. Get a token as the agent identity blueprint.
2. Use that token to get another token as the agent identity.
3. Use both previous tokens to get another token as the agent's user account.

REST

First, request a token as the agent identity blueprint, as described in Request a token for the agent identity blueprint. Once you have the agent identity blueprint token, use the agent identity blueprint token to request a Federated Identity Credential (FIC) for your agent identity:

HTTP

```
POST https://login.microsoftonline.com/<my-test-tenant>/oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=<agent-identity-id>
&scope=api://AzureADTokenExchange/.default
&grant_type=client_credentials
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=<agent-blueprint-token>
```

This returns an exchange token (T2) for the agent identity. Use it in the next request to obtain a delegated token for the agent's user account:

HTTP

```
POST https://login.microsoftonline.com/<my-test-tenant>/oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

client_id=<agent-identity-id>
&scope=https://graph.microsoft.com/.default
&grant_type=user_fic
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=<agent-blueprint-token>
&user_id=<agent-user-object-id>
&user_federated_identity_credential=<agent-identity-token>
```

This gives you a delegated access token you can use to call Microsoft Graph as your agent's user account. You can use `user_id=<user-object-id>` instead of `username=<UPN>` for the user identifier.

Related content

- Agent identity access token claims
- Call Microsoft Graph API
- Call custom APIs
- Call Azure services
- Acquire token using Microsoft Entra ID Auth SDK (sidecar)

- Microsoft Graph permissions reference
- Permissions and consent in the Microsoft identity platform
- Agent's user account flow in agents
- Agents' user accounts

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/15/2026

Authenticate users and acquire tokens for interactive agents

Interactive agents take actions on behalf of users. To act on behalf of users securely, the agent authenticates the user, gets consent for required permissions, and acquires access tokens for downstream APIs. This article walks you through the end-to-end authentication and token acquisition flow for your interactive agent:

1. Grant permissions through inheritable permissions or consent.
2. Authenticate the user and obtain an access token.
3. Validate the token and extract user claims.
4. Acquire tokens for downstream APIs using the On-Behalf-Of (OBO) flow.

> **Note**
>
> This article covers interactive agents that act **on behalf of** signed-in users using the OBO flow. If your agent needs its own user-like identity (a digital worker scenario), see Agent's user accounts and Agent's user account OAuth flow.

Prerequisites

Before you begin, ensure you have:

- An agent identity blueprint. Record the agent identity blueprint app ID (client ID).
- An agent identity.
- A client application registered in Microsoft Entra to handle user authentication.
- Familiarity with the OAuth 2.0 authorization code flow.
- The ability to run an ASP.NET Core web API if you plan to use the token validation and OBO samples in this article.

For admin authorization, you also need:

- Administrator access to grant consent for application permissions.

Permissions and consent

Before the agent can act on behalf of a user, the user or an administrator must consent to the required permissions. There are two approaches to granting permissions:

- **Inheritable permissions**: Preauthorize permissions on the blueprint so agent identities inherit them automatically.
- **Request consent**: Register a redirect URI and prompt users or administrators to grant consent through an OAuth request or use the admin consent endpoint.

Use inheritable permissions

Configure inheritable permissions on the agent identity blueprint to preauthorize a base set of delegated scopes and application roles. Agent identities created from the blueprint automatically inherit those permissions without interactive consent prompts. For more information, see Configure inheritable permissions for agent identity blueprints.

Request consent

To request consent through an OAuth flow, your agent identity blueprint must first be configured with a redirect URI. For blueprints, the redirect URI must be a **web application** type. Unlike redirect URIs on app registrations, a redirect URI on a blueprint can't be used to obtain delegated permission tokens. Only `response_type=none` is supported in the OAuth2 request, which means the request records consent only and no tokens are returned.

Register a redirect URI

Microsoft Graph API

To update the redirect URI on the agent identity blueprint, you first need to obtain an access token with the delegated permission `AgentIdentityBlueprint.ReadWrite.All`. Then send a PATCH request to the application object for the agent identity blueprint:

HTTP

```
PATCH https://graph.microsoft.com/beta/applications/<agent-blueprint-id>
OData-Version: 4.0
Content-Type: application/json
Authorization: Bearer <token>

{
  "web": {
    "redirectUris": [
      "https://myagentapp.com/authorize"
    ]
```

```
  }
}
```

Request user consent

Before the agent can act on behalf of a user, the user must consent to the required permissions. The user consent request doesn't return a token. Instead, it records that the user granted the agent permission to act on their behalf. Token acquisition happens in Authenticate the user and request a token.

> **Important**
>
> Use the **agent identity** client ID in the `client_id` parameter, not the agent identity blueprint ID.

To prompt a user for consent, construct an authorization URL and redirect the user to it. The agent can present this URL in different ways, for example, as a link in a chat message.

text

```
https://login.microsoftonline.com/contoso.onmicrosoft.com/oauth2/v2.0/authorize?
  client_id=<agent-identity-id>
  &response_type=none
  &redirect_uri=https%3A%2F%2Fmyagentapp.com%2Fauthorize
  &response_mode=query
  &scope=User.Read
  &state=xyz123
```

When the user opens this URL, Microsoft Entra ID prompts them to sign in and grant consent. After consent, the user is sent back to the redirect URI.

The key parameters in the user consent authorization URL are:

- `client_id`: The agent identity client ID (not the agent identity blueprint client ID).
- `response_type`: Set to `none` because this request records consent only. Token acquisition uses `response_type=code` in Authenticate the user and request a token.
- `redirect_uri`: Must match exactly the redirect URI configured on the agent identity blueprint.
- `scope`: Specify the delegated permissions you need (for example, `User.Read`).
- `state`: Optional parameter for maintaining state between the request and callback.

For more information on OAuth authorization concepts, see Permissions and consent in the Microsoft identity platform.

Request admin consent for all users

Agents can also request authorization from a Microsoft Entra ID administrator, who can grant consent to the agent for all users in their tenant. Admin consent might be required depending on the consent settings configured in the tenant.

To grant tenant-wide admin consent, direct an administrator to the following URL. Use the agent identity ID in the `client_id` parameter.

text

```
https://login.microsoftonline.com/contoso.onmicrosoft.com/v2.0/adminconsent
?client_id=<agent-identity-id>
&scope=User.Read
&redirect_uri=<redirect-uri>
&state=xyz123
```

After the administrator grants consent, the permissions apply to the whole tenant. Users don't need to consent again.

> **Note**
>
> Configure a redirect URI on your blueprint and include a `state` parameter in the consent request. When consent is granted, the user is sent to the redirect URI where you can display confirmation. Your endpoint can use the `state` parameter to track that permission was granted. For single-tenant agents, you can alternatively retry token requests until consent is granted because the tenant ID is already known.

Authenticate the user and request a token

After consent is granted, the client app (such as a frontend or mobile app) initiates an OAuth 2.0 authorization code request to obtain a token where the audience is the agent identity blueprint. In this step, `client_id` refers to the client app's own registered application ID, not the agent identity or agent identity blueprint.

> **Note**

The `redirect_uri` in this request belongs to the **client app's** registration, not the blueprint's redirect URI configured in the previous consent step.

1. Redirect the user to the Microsoft Entra ID authorization endpoint with the following parameters:

   HTTP

   ```
   GET https://login.microsoftonline.com/<my-test-tenant>/oauth2/v2.0/authorize?
   client_id=<client-app-id>
   &response_type=code
   &redirect_uri=<redirect_uri>
   &response_mode=query
   &scope=api://<agent-blueprint-id>/access_agent
   &state=abc123
   ```

2. After the user signs in, your app receives an authorization code at the redirect URI. Exchange the authorization code for an access token:

   HTTP

   ```
   POST https://login.microsoftonline.com/<my-test-tenant>/oauth2/v2.0/token
   Content-Type: application/x-www-form-urlencoded

   client_id=<client-app-id>
   &grant_type=authorization_code
   &code=<authorization_code>
   &redirect_uri=<redirect_uri>
   &scope=api://<agent-blueprint-id>/access_agent
   &client_secret=<client-secret>
   ```

   Include the `client_secret` parameter only if using a confidential client.

   The JSON response contains an access token that can be used to access the agent's API.

Validate the access token

The web API must validate the incoming access token before the agent can act. Always use an approved library to validate tokens. Don't write your own token validation code.

1. Install the `Microsoft.Identity.Web` NuGet package:

   Bash

   ```
   dotnet add package Microsoft.Identity.Web
   ```

2. In your ASP.NET Core web API project, implement Microsoft Entra ID authentication:

C#

```
// Program.cs
using Microsoft.Identity.Web;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApi(builder.Configuration.GetSection("AzureAd"));

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();
```

3. Configure authentication credentials in the `appsettings.json` file:

> **⚠ Warning**
>
> Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

JSON

```
"AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "<my-test-tenant>",
    "ClientId": "<agent-blueprint-id>",
    "Audience": "<agent-blueprint-id>",
    "ClientCredentials": [
        {
            "SourceType": "ClientSecret",
            "ClientSecret": "your-client-secret"
        }
    ]
}
```

For more information about Microsoft.Identity.Web, see Microsoft.Identity.Web documentation.

Validate user claims

After access token validation, the agent can identify the user and perform authorization checks. The following example API route extracts user claims from the access token and returns them in the API response:

C#

```
app.MapGet("/hello-agent", (HttpContext httpContext) =>
{
    var claims = httpContext.User.Claims.Select(c => new
    {
        Type = c.Type,
        Value = c.Value
    });

    return Results.Ok(claims);
})
.RequireAuthorization();
```

Acquire tokens for downstream APIs

After an interactive agent validates the user's token, it can request access tokens to call downstream APIs on behalf of the user. The On-Behalf-Of (OBO) flow allows the agent to:

- Receive an access token from a client.
- Exchange it for a new access token for a downstream API like Microsoft Graph.
- Use that new token to access protected resources on behalf of the original user.

Use Microsoft Identity Web

The `Microsoft.Identity.Web` library simplifies the OBO implementation by handling token exchange automatically, so you don't have to manually implement the flow by following the protocol.

1. Install the required NuGet packages:

   Bash

   ```
   dotnet add package Microsoft.Identity.Web
   dotnet add package Microsoft.Identity.Web.AgentIdentities
   ```

2. In your ASP.NET Core web API project, update the Microsoft Entra ID authentication implementation:

   C#

```
// Program.cs
using Microsoft.AspNetCore.Authorization;
using Microsoft.Identity.Abstractions;
using Microsoft.Identity.Web;
using Microsoft.Identity.Web.Resource;
using Microsoft.Identity.Web.TokenCacheProviders.InMemory;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddMicrosoftIdentityWebApiAuthentication(builder.Configurat
ion)
    .EnableTokenAcquisitionToCallDownstreamApi();
builder.Services.AddAgentIdentities();
builder.Services.AddInMemoryTokenCaches();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.Run();
```

3. In the agent API, exchange the incoming user access token for a new access token for the agent identity. `Microsoft.Identity.Web` validates the incoming access token and handles the on-behalf-of token exchange:

C#

```
app.MapGet("/agent-obo-user", async (HttpContext httpContext) =>
{
    string agentIdentity = "<your-agent-identity>";
    IAuthorizationHeaderProvider authorizationHeaderProvider =
httpContext.RequestServices.GetService<IAuthorizationHeaderProvider>()!;
    AuthorizationHeaderProviderOptions options = new
AuthorizationHeaderProviderOptions().WithAgentIdentity(agentIdentity);

    string authorizationHeaderWithUserToken = await
authorizationHeaderProvider.CreateAuthorizationHeaderForUserAsync(["https://
graph.microsoft.com/.default"], options);

    var response = new { header = authorizationHeaderWithUserToken };
    return Results.Json(response);
})
.RequireAuthorization();
```

Under the hood, the OBO flow involves two token exchanges: first, the agent identity blueprint obtains an exchange token using its client credential, and then the agent identity exchanges that token along with the user's access token for a downstream API token. For the full protocol

walkthrough, including HTTP request formats and token validation details, see On-behalf-of flow in agents.

Related content

Learn more about agent tokens and related APIs:

- Token claims reference
- On-behalf-of flow in agents
- Call Microsoft Graph API
- Call custom APIs
- Call Azure services
- Agent users
- Authenticate and acquire tokens for autonomous agents
- Permissions and consent in the Microsoft identity platform
- Microsoft identity platform and OAuth 2.0 On-Behalf-Of flow

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/15/2026

Authentication with Microsoft Entra ID Auth SDK (sidecar)

The Microsoft Entra ID Auth SDK (sidecar) handles authentication and token operations for your AI agent. It runs as a second container next to your agent that handles client-credentials exchange, on-behalf-of flows, and token lifecycle management. This article explains the Microsoft Entra ID Auth SDK (sidecar) design pattern, how it works, and the identity objects involved.

Why use the Microsoft Entra ID Auth SDK (sidecar)?

AI agents need credentials to call downstream APIs, but the common approaches to agent authentication fall short:

- **Hard-coded secrets in agent code:** Every agent image holds a copy of your app's `client_secret`. Any compromise, log leak, or forgotten `.env` file committed to Git exposes the full tenant.
- **Delegated user tokens for everything:** The agent can only act when a human is present. You also lose individual auditability because every call looks like the same service principal.

Microsoft Entra Agent ID gives each agent its own identity. The sidecar pattern makes that identity easy to use by keeping all credential handling outside your agent code.

How the Microsoft Entra ID Auth SDK (sidecar) works

The Microsoft Entra ID Auth SDK (sidecar) runs as a container that exposes HTTP endpoints on the pod-local network. It handles the following responsibilities:

- Exchanges client credentials with `login.microsoftonline.com`.
- Acquires tokens through client credentials or federated identity credentials (FIC) for the agent identity in autonomous flows.
- Handles on-behalf-of (OBO) flows for user-context calls.
- Caches tokens and manages refresh and expiry.
- Abstracts the credential source - uses `ClientSecret` for development and `SignedAssertionFromManagedIdentity` for Azure deployments through the same API.

The following table summarizes the flow of auth actions between your agent and the sidecar:

Expand table

Agent (your code)	Microsoft Entra ID Auth SDK (sidecar)
Decide when to call the API	Acquire and cache the right token
Build the HTTP request	Perform client-credentials and OBO exchange
Pass through user token for OBO	Validate and forward user assertion
Handle business logic	Communicate with `login.microsoftonline.com`

The security boundary is explicit: the sidecar has no host port. Only services inside the same network, such as your agent container, can request tokens.

Identity objects in the sidecar pattern

The following table describes the Microsoft Entra objects in the sidecar pattern:

Expand table

Object	Role	Location
Blueprint application	Template that creates and issues agent identities. Holds the client credential (secret or federated).	Your Microsoft Entra tenant
Agent identity	The individual AI agent. Has a unique app ID, permission grants, and audit trail.	Your Microsoft Entra tenant
Client single-page application (SPA) (OBO only)	Web UI that signs the user in and exchanges the user's token for an agent token on their behalf.	Your Microsoft Entra tenant
Sidecar container	Runs client-credentials and OBO flows. Holds the blueprint credential.	Next to your agent
Agent container	Your application code. Requests authorization headers from the sidecar.	Your pod, compose service, or App Service

For more information about blueprints and agent identities, see Agent identity blueprints and Agent identities.

Credential source abstraction

The sidecar abstracts the credential source from your agent code. During development, you can use a `ClientSecret` for convenience. In production Azure deployments, you can switch to

`SignedAssertionFromManagedIdentity`, a federated identity credential backed by managed identity, without changing your agent code.

The sidecar's configuration determines which credential source to use. Your agent continues to call the same `/AuthorizationHeader` endpoint regardless of the authentication mechanism.

Sidecar sample scenarios

The Microsoft Entra Agent ID sidecar samples demonstrate the following concepts:

- How a blueprint differs from an agent identity and why agents need their own identity.
- How the sidecar exposes `/AuthorizationHeader` (get token) and `/DownstreamApi` (token + proxied call) endpoints.
- How the sidecar forwards a signed-in user's token and has the Microsoft Entra ID Auth SDK (sidecar) mint an agent-on-behalf-of-user token via OBO.
- How the downstream API validates agent tokens including the signature, issuer, `xms_par_app_azp`, and audience.
- How to swap from `ClientSecret` (development) to `SignedAssertionFromManagedIdentity` (Azure deployments) without changing agent code.

Related content

- Microsoft Entra ID Auth SDK (sidecar) documentation
- Acquire tokens and call downstream APIs with Microsoft Entra ID Auth SDK (sidecar)
- Microsoft Entra Agent ID design patterns
- Agent identity blueprints
- Autonomous agent app OAuth flow
- On-behalf-of OAuth flow

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 08/13/2026

Acquire tokens and call downstream APIs with Microsoft Entra ID Auth SDK (sidecar)

The Microsoft Entra ID Auth SDK (sidecar) is a containerized web service that handles token acquisition, validation, and downstream API calls for agents. This SDK communicates with your application through HTTP APIs, providing consistent integration patterns regardless of your technology stack. Instead of embedding identity logic directly in your application code, the Microsoft Entra ID Auth SDK (sidecar) manages token acquisition, validation, and API calls through standard HTTP requests.

Prerequisites

Before you begin, ensure you have:

- Set up the Microsoft Entra ID Auth SDK (sidecar)
- An agent identity. Record the agent identity client ID.
- An agent identity blueprint. Record the agent identity blueprint client ID.
- Necessary permissions configured in Microsoft Entra ID

Deploy your containerized service

Deploy the Microsoft Entra ID Auth SDK (sidecar) as a containerized service in your environment. Follow the instructions in the Set up the Microsoft Entra ID Auth SDK (sidecar) guide to configure the service with your agent identity details.

Configure your Microsoft Entra ID Auth SDK (sidecar) settings

Follow these steps to configure your Microsoft Entra ID Auth SDK (sidecar) settings:

> ⚠ **Warning**
>
> Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates.

These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

1. Set up the necessary components in Microsoft Entra ID. Ensure you register your application in the Microsoft Entra ID tenant.

2. Configure your client credentials. This credential can be a client secret, a certificate, or a managed identity that you're using as a federated identity credential.

3. If you're calling a downstream API, ensure that the necessary permissions are granted. Calling a custom web API requires you to provide the API registration details in the SDK configuration.

For more information, see Configure your Microsoft Entra ID Auth SDK (sidecar) settings.

Acquire tokens using the Microsoft Entra ID Auth SDK (sidecar)

These are the steps to acquire tokens using the Microsoft Entra ID Auth SDK (sidecar):

1. Acquire token using the Microsoft Entra ID Auth SDK (sidecar). This varies based on whether the agent is operating autonomously or on behalf of a user. There are three scenarios to consider:

   - Autonomous agents: Agents operating on their own behalf using service principals created for agents (autonomous).
   - Autonomous agent's user account: Agents operating on their own behalf using user principals created specifically for agents (for instance agents having their own mailbox).
   - Interactive agents: Agents operating on behalf of human users.

   Specify the downstream API by including its name in the request URL based on your Microsoft Entra ID Auth SDK (sidecar) configuration. The authorization header endpoint takes the format `/AuthorizationHeader/{serviceName}` where `serviceName` is the name of the downstream API configured in the SDK settings.

2. To acquire an app-only (client credentials) token for an autonomous agent, you provide the agent identity client ID in the request.

   Bash

```
GET /AuthorizationHeader/Graph?AgentIdentity=<agent-identity-client-id>
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGc...
```

3. To acquire token for an autonomous agent's user account, provide either the user object ID or User Principal Name but not both. This means providing either `AgentUsername` or `AgentUserId`. Providing both causes a validation error. You must also provide the `AgentIdentity` to specify which agent identity to use for token acquisition. If the agent identity parameter is missing, the request fails with a validation error.

   Bash

   ```
   GET /AuthorizationHeader/Graph?AgentIdentity=<agent-identity-client-
   id>&AgentUserId=<agent-user-object-id>
   Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGc...
   ```

   Bash

   ```
   GET /AuthorizationHeader/Graph?AgentIdentity=<agent-identity-client-
   id>&AgentUsername=<agent-user-principal-name>
   Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGc...
   ```

4. For interactive agents, use the on-behalf-of (OBO) flow. The agent first validates the user token granted to it before acquiring the resource token to call the downstream API.

   Agent web API receives user token from the calling application and validates the token via the Microsoft Entra ID Auth SDK (sidecar) `/Validate` endpoint Acquire token for downstream APIs by calling `/AuthorizationHeader` with only the `AgentIdentity` and the incoming authorization header

   Bash

   ```
   # Step 1: Validate incoming user token
   GET /Validate
   Authorization: Bearer <user-token>

   # Step 2: Get authorization header on behalf of the user
   GET /AuthorizationHeader/Graph?AgentIdentity=<agent-identity-client-id>
   Authorization: Bearer <user-token>
   ```

Call an API

When obtaining the authorization header for calling a downstream API, the Microsoft Entra ID Auth SDK (sidecar) returns the `Authorization` header value that can be used directly in your API

calls.

You can use this header to call the downstream API. The web API should validate the token by calling the `/Validate` endpoint of the Microsoft Entra ID Auth SDK (sidecar). This endpoint returns token claims for further authorization decisions.

Related content

- Python implementation of Microsoft Entra ID Auth SDK (sidecar)
- TypeScript implementation of Microsoft Entra ID Auth SDK (sidecar)
- Call a downstream API - Microsoft Entra ID Auth SDK (sidecar) docs

---

Last updated on 08/13/2026

Run the sidecar for local development

This article shows how to run the Microsoft Entra Auth SDK sidecar in your local environment by using Docker Compose. You start a four-container stack - chat agent, sidecar, downstream weather API, and a local large language model (LLM) such as (Ollama). Then you send a query through the chat UI and observe the full token flow from agent to API. Before you begin, review the Prerequisites for required tools, Microsoft Entra tenant objects, and local environment setup.

The sample demonstrates two execution modes and two identity flows:

Expand table

	Autonomous (app-only)	**OBO (on behalf of user)**
Direct (no LLM)	Agent fetches a token and calls the weather API directly.	Same, but the sidecar exchanges the signed-in user's token.
Ollama + LangChain	LangGraph ReAct agent decides when to call the `get_weather` tool.	Same, but the agent passes the user token through.

Prerequisites

This sample works on macOS, Linux, and Windows 10/11.

Expand table

Requirement	**macOS**	**Linux**	**Windows**
Docker	Docker Desktop	Docker Engine + Compose v2	Docker Desktop (WSL 2 backend recommended)
PowerShell 7+	`brew install --cask powershell`	Install PowerShell on Linux	Built-in (or install PowerShell 7+)
Azure CLI	`brew install azure-cli`	Install the Azure CLI	`winget install -e Microsoft.AzureCLI`

You also need a **Microsoft Entra tenant** with these objects:

- An agent identity blueprint with a client secret. Record the values for `BLUEPRINT_APP_ID` and `BLUEPRINT_CLIENT_SECRET`.
- An agent identity created from that blueprint. Record the `AGENT_CLIENT_ID`.

- (OBO flow only) A SPA app registration. Record the `CLIENT_SPA_APP_ID`.

To create these objects, follow the PowerShell workflow in the Microsoft Entra Agent ID samples repository. The workflow creates the blueprint app, the agent identity, and optionally the SPA app for OBO sign-in.

Ollama is **not** a prerequisite on your host - it runs inside the compose stack and pulls `qwen2.5:1.5b` automatically.

Clone the sample repository

Run the following commands to download the sample project and change into the sidecar directory, which contains the Docker Compose configuration and agent source code for this walkthrough:

Bash

```
git clone https://github.com/microsoft/entra-agentid-samples.git
cd entra-agentid-samples/sidecar/dev
```

Sidecar local development architecture

The stack runs four containers on an internal Docker network: `llm-agent-dev` (Flask chat UI, exposed on port 3003), `agent-id-sidecar-dev` (Microsoft Entra SDK auth sidecar), `weather-api-dev` (downstream API that validates agent tokens), and `ollama-dev` (local LLM). Only the chat UI is exposed to your host; the sidecar and weather API are reachable only from within the Docker network.



All four containers run on a shared Docker bridge network (`agent-network-dev`). Only the chat UI (port 3003) is exposed to your host. The sidecar and weather API have no host port, which keeps the token endpoint inside a trust boundary.

The request path works like this:

1. You open `http://localhost:3003` in your browser and send a query.
2. The agent (`llm-agent-dev`) receives the query and decides to call the `get_weather` tool.

3. The tool asks the sidecar for an authorization header at `GET /AuthorizationHeader...?AgentIdentity={agentId}`.
4. The sidecar (`agent-id-sidecar-dev`) performs an OAuth 2.0 exchange with Microsoft Entra ID and receives a token (TR).
5. The sidecar returns the `Authorization: Bearer TR` header to the agent.
6. The agent calls the weather API (`weather-api-dev`) with that header.
7. The weather API validates TR (JSON Web Key Set (JWKS), RS256 signature algorithm, issuer, expiry, audience) and returns weather data.

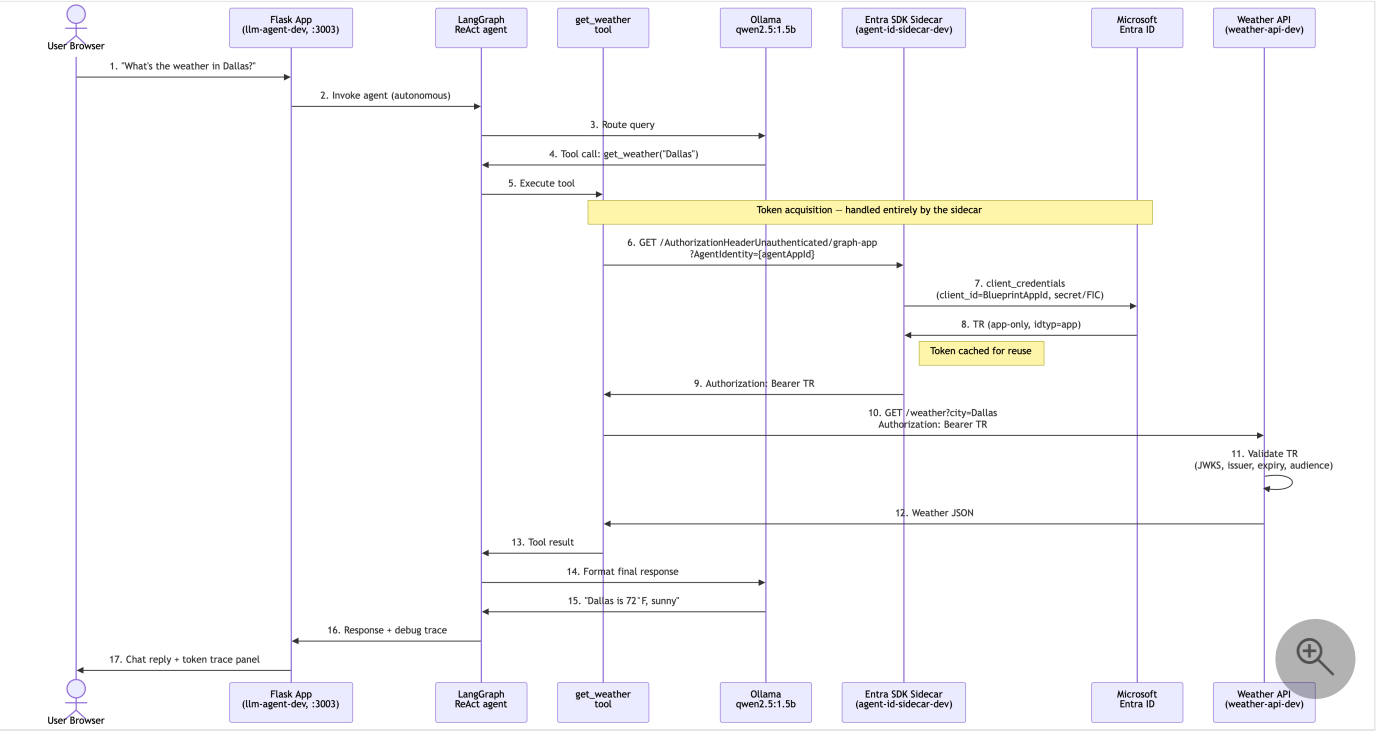
The agent never contacts Microsoft Entra ID directly and never sees a credential. It asks the sidecar for an `Authorization` header, receives a `Bearer` token, and passes that token to the weather API. Only the sidecar communicates with `login.microsoftonline.com`.

Understand the token flow

The autonomous flow uses two tokens: **T1** (blueprint app token from client credentials) and **TR** (agent token for the downstream API). The OBO flow adds a third: **Tc** (user access token from MSAL.js browser sign-in). The sidecar handles all token acquisition and caching so your agent code never manages credentials directly.

Understand the autonomous token flow

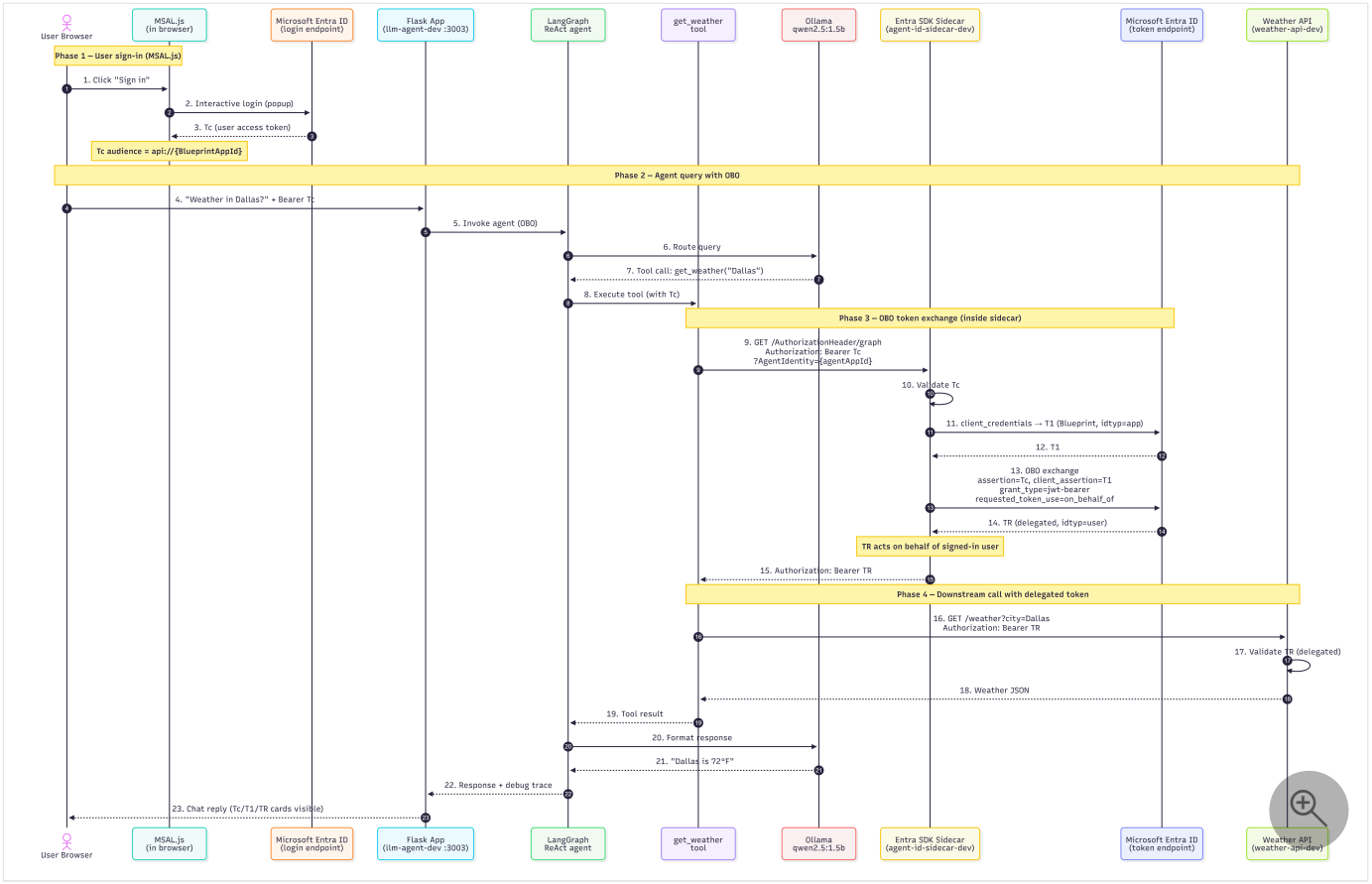
No user sign-in is required. The agent authenticates as itself by using the blueprint's client credentials.



1. The user sends a query through the chat UI.
2. The agent (or LangGraph ReAct agent) decides to call the `get_weather` tool.
3. The tool requests an authorization header from the sidecar at `GET /AuthorizationHeaderUnauthenticated/graph-app?AgentIdentity={agentAppId}`.
4. The sidecar performs a client credentials exchange with Microsoft Entra ID and receives TR (app-only, `idtyp=app`).
5. The tool calls the weather API with `Authorization: Bearer TR`.
6. The weather API validates TR (signature, issuer, expiry, audience) and returns weather data.

Understand the on-behalf-of (OBO) token flow

The agent acts on behalf of a signed-in user. The sidecar performs a three-step token exchange.



1. The user signs in through MSAL.js in the browser and receives Tc (user access token, audience = `api://{BlueprintAppId}`).
2. The user sends a query. The agent receives Tc with the request.
3. The tool requests an authorization header from the sidecar at `GET /AuthorizationHeader/graph`, passing `Authorization: Bearer Tc` and `?AgentIdentity={agentAppId}`.
4. The sidecar validates Tc, performs a client credentials exchange to get T1, then performs an OBO exchange to get TR (delegated, `idtyp=user`). The OBO exchange uses `assertion=Tc`,

`client_assertion=T1`, and `grant_type=jwt-bearer`.

5. The tool calls the weather API with `Authorization: Bearer TR`.
6. The weather API validates TR and returns weather data. TR acts on behalf of the signed-in user.

Configure environment variables

> **Tip**
>
> If you already have a `.env` file from a previous run with `TENANT_ID`, `BLUEPRINT_APP_ID`, `BLUEPRINT_CLIENT_SECRET`, and `AGENT_CLIENT_ID` populated, skip to Start the stack. The Microsoft Entra objects survive container restarts and `docker compose down`.

Copy the example environment file and add your Microsoft Entra values:

Bash

Create a local environment file from the sample template so you can fill in your tenant and app registration values:

Bash

```
cp .env.example .env
```

Open `.env` in your editor and set the following values:

Expand table

Variable	Description
`TENANT_ID`	Your Microsoft Entra tenant ID.
`BLUEPRINT_APP_ID`	The blueprint app registration client ID. The sidecar authenticates as this app.
`BLUEPRINT_CLIENT_SECRET`	The blueprint client secret. Used for local development only.
`AGENT_CLIENT_ID`	Your agent identity client ID. Passed as the `AgentIdentity` query parameter to the sidecar.
`CLIENT_SPA_APP_ID`	The SPA app registration client ID. Required only for the OBO flow.
`OLLAMA_MODEL`	The Ollama model to use. Defaults to `qwen2.5:1.5b`.

The autonomous flow requires `TENANT_ID`, `BLUEPRINT_APP_ID`, `BLUEPRINT_CLIENT_SECRET`, and `AGENT_CLIENT_ID`. The OBO flow also requires `CLIENT_SPA_APP_ID`.

This sidecar sample uses `ClientSecret` as the credential source type. The sidecar supports the following credential types through the `AzureAd__ClientCredentials__0__SourceType` setting in `docker-compose.yml`:

- **`ClientSecret`**: Local development only. This type is the default for this sample.
- **`SignedAssertionFromManagedIdentity`**: Deployed on Azure. Zero secrets, recommended for production.
- **`KeyVault`**: Certificate from Azure Key Vault.
- **`StoreWithThumbprint`**: Certificate from local machine store.

Set up OBO sign-in (optional)

To test the on-behalf-of flow, create the SPA app and configure OBO consent. Run one of the following script pairs from the repository root:

Bash

Bash

```bash
# Create the SPA app registration for MSAL.js browser sign-in
bash ../../scripts/setup-obo-client-app.sh
# → prints CLIENT_SPA_APP_ID

# Wire up the OBO scope + admin consent on the Blueprint
bash ../../scripts/setup-obo-blueprint.sh
```

Add the `CLIENT_SPA_APP_ID` value to your `.env` file after running the scripts.

Start the stack

Build the container images and start all four services in detached mode by running the following command:

Bash

```bash
docker compose up --build -d
```

The first run takes about 30 seconds while Ollama pulls the `qwen2.5:1.5b` model. To verify that the local sample stack is running and that components such as the sidecar and Ollama are ready, query the status endpoint:

Bash

Bash

```
curl http://localhost:3003/api/status
```

The response shows `ollama_available: true` when the stack is ready.

Send a query through the chat UI

Perform the following steps to send a test query through the chat UI and observe the token flow:

1. Open `http://localhost:3003` in your browser.
2. In the header bar, verify that your **Tenant ID** and **Agent ID** appear.
3. Use the two toggles to select your demo configuration:
    - **Execution Mode**: Select **Direct** to skip the LLM and call the weather API directly, or select **Ollama** to use a LangChain ReAct agent.
    - **Identity Flow**: Select **Autonomous** for an app-only token, or **OBO** to act on behalf of a signed-in user. For OBO, choose **Sign in** to authenticate through an MSAL.js popup.
4. Send the prepopulated query "Weather in Dallas?" and review the result.
5. In the right panel, expand **Identity Trace** to inspect each step of the token flow:
    - The token request to the sidecar, including the `AgentIdentity` parameter.
    - Decoded JWT claims for each token (**Tc**, **T1**, **TR** for OBO; **T1** and **TR** for autonomous).
    - Downstream API validation results, including signature (JWKS, RS256), issuer, expiry, and audience checks.

Troubleshoot common issues

Expand table

Symptom	Likely cause	Fix
`/api/status` returns `ollama_available: false`	Model is still downloading.	Wait about 30 seconds. Check logs by using `docker logs ollama-dev`.
Weather API returns `401 Unauthorized`	Token tenant mismatch, expired secret, or signature check failed.	Verify `TENANT_ID` matches the blueprint's tenant. Check sidecar logs by using `docker logs agent-id-sidecar-dev`.
LLM returns weather without calling the tool	The `qwen2.5:1.5b` model is too small for reliable tool calling.	Change `OLLAMA_MODEL` to `qwen2.5:7b` or `llama3.1:8b` in your `.env` file.
OBO sign-in popup is blocked	Browser popup blocker is active.	Allow popups for `localhost:3003`.
`4xx` error from sidecar during OBO	`CLIENT_SPA_APP_ID` is missing or the SPA redirect URI doesn't match.	Rerun the OBO setup scripts. Verify `http://localhost:3003` is listed in the SPA's redirect URIs.

To diagnose startup, authentication, or downstream API issues, display the logs from each container by running the following commands:

Bash

```
docker logs llm-agent-dev
docker logs agent-id-sidecar-dev
docker logs weather-api-dev
```

Clean up resources

Stop the containers when you're done. Choose the cleanup level that fits your needs. The first command stops the demo containers while preserving volumes and images for a faster restart later:

Bash

```
# Stop containers, keep volumes and images
docker compose down

# Stop containers and remove the Ollama model cache
docker compose down -v

# Remove containers, volumes, and images
docker compose down -v --rmi all
```

Related content

- Authentication with Auth SDK (sidecar)
- Microsoft Entra ID Auth SDK (sidecar) documentation
- Acquire tokens and call downstream APIs with Microsoft Entra ID Auth SDK (sidecar)
- Validate agent identity tokens in a downstream API
- Microsoft Entra Agent ID samples repository

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/16/2026

Validate agent identity tokens in a downstream API

When an AI agent calls your API through the Microsoft Entra SDK auth sidecar, the request includes a `Bearer` token. Your API validates this token to confirm the request comes from an authenticated agent with the correct permissions. If validation fails, your API returns HTTP 401 with a reason.

This article explains the validation checks and shows how to configure and run a sample weather API that validates agent identity tokens end-to-end.

Prerequisites

- **Docker Desktop** (macOS / Windows) or **Docker Engine** (Linux).
- A **Microsoft Entra tenant** with an agent identity blueprint and an agent identity. For setup steps, see Create an agent blueprint and Create and delete agent identities.
- An agent identity token for testing. You can get one from the sidecar local development sample or from the scripts in the Microsoft Entra Agent ID samples repository.

Token validation checks

Your downstream API should perform four checks on every incoming agent identity token:

Expand table

Check	What it verifies	Details
Signature	The token isn't tampered with.	Verify the RS256 signature against the JSON Web Key Set (JWKS) at `https://login.microsoftonline.com/<tenant>/discovery/v2.0/keys`.
Issuer	Microsoft Entra ID issued the token.	The `iss` claim matches `https://sts.windows.net/<tenant>/` or `https://login.microsoftonline.com/<tenant>/v2.0`.
Audience	The token is intended for your API.	The `aud` claim matches your API's expected audience value.

Check	What it verifies	Details
Agent identity marker	The token was issued to an agent, not a regular app.	The `xms_par_app_azp` claim is present in agent identity tokens and absent in standard app-only tokens. This claim identifies the blueprint that created the agent.

When all four checks pass, your API can trust and process the request.

How the sample weather API works

The Microsoft Entra Agent ID samples repository includes a sample weather API that demonstrates these validation checks. The sample API is a minimal Flask app that acts as the downstream API your agent calls. It validates incoming agent identity tokens and returns real weather data from Open-Meteo.

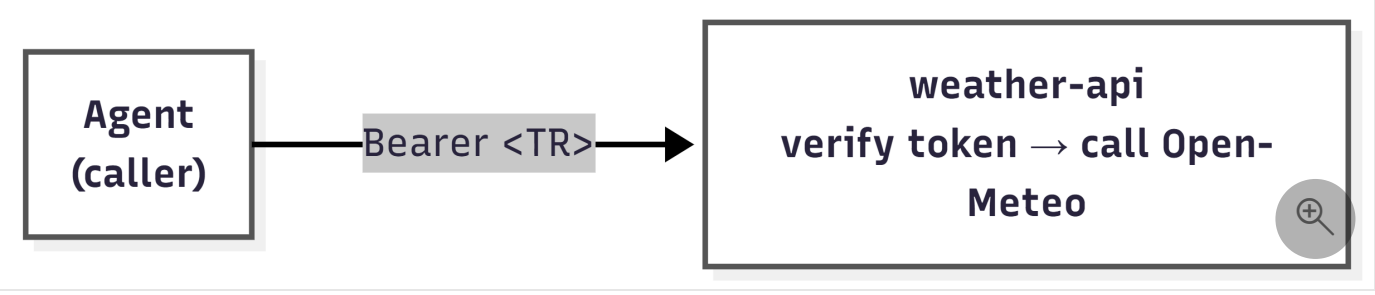
Both the local development (Ollama) and AWS (Bedrock) sidecar samples call the same weather API container. The sample consists of three files:

- **`app.py`**: Flask app with route handlers, token validation logic, and Open-Meteo client.
- **`Dockerfile`**: Uses `python:3.13-slim` as the base image. Runs `gunicorn app:app` on port 8080.
- **`requirements.txt`**: Dependencies: `flask`, `pyjwt[crypto]`, `cryptography`, `requests`, `gunicorn`.

The API exposes two endpoints:

- **`GET /weather?city=<name>`**: Validates the `Authorization: Bearer <token>` header and returns weather data for the specified city.
- **`GET /healthz`**: Returns a health status without requiring token validation.

The following diagram shows how tokens flow from the agent through the sidecar to the weather API. The agent never contacts Microsoft Entra ID directly. Instead, the sidecar acquires a token (TR) on behalf of the agent identity, and the agent passes that token to the weather API in the `Authorization: Bearer` header.



TR is the agent identity token issued by Microsoft Entra ID through the sidecar. It contains the claims your API validates, including the `xms_par_app_azp` agent identity marker. For a detailed breakdown of all tokens in the flow, see Run the sidecar for local development.

Token validation libraries differ across ecosystems (Python, Node.js, .NET). This sample uses PyJWT with the `cryptography` backend for RS256 signature verification. When you build your own downstream API, choose the equivalent JWT validation library for your technology stack.

Configure the sample weather API

The sample weather API accepts the following environment variables:

Expand table

Variable	Required	Default	Purpose
`TENANT_ID`	Yes	—	Your Microsoft Entra tenant ID, used to build the JWKS URL and verify the issuer claim.
`EXPECTED_AUDIENCE`	No	`https://graph.microsoft.com`	The expected `aud` claim value. Defaults to Microsoft Graph so that the same agent tokens work for local testing.
`PORT`	No	`8080`	The HTTP port the API listens on.

Run the sample weather API

To run the weather API as a standalone container and test token validation:

1. Clone the repository and navigate to the weather API directory:

```bash
git clone https://github.com/microsoft/entra-agentid-samples.git
cd entra-agentid-samples/sidecar/weather-api
```

2. Build and run the container:

```bash
docker build -t weather-api:local .
docker run --rm -p 8080:8080 \
  -e TENANT_ID=<your-tenant-id> \
```

```
  -e EXPECTED_AUDIENCE=https://graph.microsoft.com \
  weather-api:local
```

3. Send a request with an agent identity token:

Bash

```
curl -H "Authorization: Bearer $TOKEN" \
     "http://localhost:8080/weather?city=Dallas"
```

The API returns a JSON response that includes both the weather data and the token validation results:

JSON

```
{
  "city": "Dallas",
  "temperature": 61,
  "temperature_unit": "F",
  "condition": "Overcast",
  "humidity": 93,
  "wind_speed": 8,
  "is_agent_identity": true,
  "agent_app_id": "<agent-app-id from xms_par_app_azp>",
  "validated_by": "Agent Identity Token",
  "data_source": "Open-Meteo API (Real-time)"
}
```

The `is_agent_identity`, `agent_app_id`, and `validated_by` fields confirm token validation:

- **`is_agent_identity`**: Set to `true` when the `xms_par_app_azp` claim is present, which confirms the token was issued to an agent identity rather than a standard app registration.
- **`agent_app_id`**: The value of the `xms_par_app_azp` claim, which identifies the blueprint application that created the agent identity.
- **`validated_by`**: The validation method applied to the token. Displays `Agent Identity Token` when the agent marker claim is present.

Related content

- Token claims reference
- Authentication with Auth SDK (sidecar)
- Run the sidecar for local development
- Microsoft Entra Agent ID samples repository

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/01/2026

Call a Microsoft Graph API from an agent using .NET

This article explains how to call a Microsoft Graph API from an agent using agent identities or an agent's user account.

To call an API from an agent, you need to obtain an access token that the agent can use to authenticate itself to the API. We recommend using the *Microsoft.Identity.Web* SDK for .NET to call your web APIs. This SDK simplifies the process of acquiring and validating tokens. For other languages, use the Microsoft Entra ID Auth SDK (sidecar).

Prerequisites

- An agent identity with appropriate permissions to call the target API. You need a user for the on-behalf-of flow.
- An agent's user account with appropriate permissions to call the target API.

Call a Microsoft Graph API

1. Install the *Microsoft.Identity.Web.GraphServiceClient* that handles authentication for the Graph SDK and the *Microsoft.Identity.Web.AgentIdentities* package to add support for agent identities.

   Bash

   ```
   dotnet add package Microsoft.Identity.Web.GraphServiceClient
   dotnet add package Microsoft.Identity.Web.AgentIdentities
   ```

2. Add the support for Microsoft Graph and agent identities in your service collection.

   C#

   ```
   using Microsoft.AspNetCore.Authentication.OpenIdConnect;
   using Microsoft.Identity.Web;

   var builder = WebApplication.CreateBuilder(args);

   // Add authentication (web app or web API)
   builder.Services.AddAuthentication(OpenIdConnectDefaults.AuthenticationScheme)
       .AddMicrosoftIdentityWebApp(builder.Configuration.GetSection("AzureAd"))
       .EnableTokenAcquisitionToCallDownstreamApi()
   ```

```
    .AddInMemoryTokenCaches();

// Add Microsoft Graph support
builder.Services.AddMicrosoftGraph();

// Add Agent Identities support
builder.Services.AddAgentIdentities();

var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.Run();
```

3. Configure Graph and agent identity options in *appsettings.json*.

> **⚠ Warning**
>
> Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

JSON

```
{
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "<your-tenant-id>",
    "ClientId": "<agent-blueprint-client-id>",
    "ClientCredentials": [
      {
        "SourceType": "ClientSecret",
        "ClientSecret": "your-client-secret"
      }
    ]
  },
  "DownstreamApis": {
    "MicrosoftGraph": {
      "BaseUrl": "https://graph.microsoft.com/v1.0",
      "Scopes": ["User.Read", "User.ReadBasic.All"]
    }
  }
}
```

> **ⓘ Note**

> Configure only the Microsoft Graph permissions your agent needs, and make sure the `Scopes` you set match the Graph resources your code calls. These examples use `User.Read` and `User.ReadBasic.All`; calling other resources requires their corresponding permissions.

4. You can now get the `GraphServiceClient` injecting it in your service or from the service provider and call Microsoft Graph.

- For agent identities, you can acquire either an app only token (autonomous agents) or an on-behalf of user token (interactive agents) by using the `WithAgentIdentity` method. For app only tokens, set the `RequestAppToken` property to `true`. For delegated on-behalf of user tokens, don't set the `RequestAppToken` property or explicitly set it to `false`.

C#

```
using Microsoft.Graph;
using Microsoft.Identity.Web;

// Get the GraphServiceClient
GraphServiceClient graphServiceClient =
serviceProvider.GetRequiredService<GraphServiceClient>();

string agentIdentity = "agent-identity-guid";

// Call Microsoft Graph APIs with the agent identity for app only scenario
var usersAppOnly = await graphServiceClient.Users
    .GetAsync(r => r.Options.WithAuthenticationOptions(options =>
    {
        options.WithAgentIdentity(agentIdentity);
        options.RequestAppToken = true; // Set to true for app only
    }));

// Call Microsoft Graph APIs with the agent identity for on-behalf of user
scenario
var usersOnBehalfOfUser = await graphServiceClient.Users
    .GetAsync(r => r.Options.WithAuthenticationOptions(options =>
    {
        options.WithAgentIdentity(agentIdentity);
        options.RequestAppToken = false; // False to show it's on-behalf of user
    }));
```

  - For agent's user account identities, you can specify either User Principal Name (UPN) or Object Identity (OID) to identify the agent's user account by using the `WithAgentUserIdentity` method.

    C#

```
using Microsoft.Graph;
using Microsoft.Identity.Web;

// Get the GraphServiceClient
GraphServiceClient graphServiceClient =
serviceProvider.GetRequiredService<GraphServiceClient>();

string agentIdentity = "agent-identity-guid";

// Call Microsoft Graph APIs with the agent's user account identity using UPN
string userUpn = "user-upn";
var me = await graphServiceClient.Me
    .GetAsync(r => r.Options.WithAuthenticationOptions(options =>
        options.WithAgentUserIdentity(agentIdentity, userUpn)));

// Or using OID
string userOid = "user-object-id";
var meByOid = await graphServiceClient.Me
    .GetAsync(r => r.Options.WithAuthenticationOptions(options =>
        options.WithAgentUserIdentity(agentIdentity, userOid)));
```

Related content

- Call custom APIs
- Call Azure SDKs

Last updated on 08/14/2026

Call custom APIs from an agent using .NET

There are multiple ways to call custom APIs from an agent. Depending on your scenario, you can use either `IDownstreamApi`, `MicrosoftIdentityMessageHandler`, or `IAuthorizationHeaderProvider`. This guide explains the different approaches for calling your own protected APIs all the three ways.

To call an API from an agent, you need to obtain an access token that the agent can use to authenticate itself to the API. We recommend using the *Microsoft.Identity.Web* SDK for .NET to call your web APIs. This SDK simplifies the process of acquiring and validating tokens. For other languages, use the Microsoft Entra ID Auth SDK (sidecar).

Prerequisites

- An agent identity with appropriate permissions to call the target API. You need a user for the on-behalf-of flow.
- An agent's user account with appropriate permissions to call the target API.

Decide which approach to use based on your scenario

The following table helps you decide which approach to use. For most scenarios, we recommend using `IDownstreamApi`.

Expand table

Approach	Complexity	Flexibility	Use Case
`IDownstreamApi`	Low	Medium	Standard REST APIs with configuration
`MicrosoftIdentityMessageHandler`	Medium	High	HttpClient with Direct Injection (DI) and composable pipeline
`IAuthorizationHeaderProvider`	High	Very High	Complete control over HTTP requests

Use IDownstreamApi

`IDownstreamApi` is the preferred way to call a protected API among the three options. It's highly configurable and requires minimal code changes. It also offers automatic token acquisition.

Use `IDownstreamApi` when you need the following listed items:

- You're calling standard REST APIs
- You want a configuration-driven approach
- You need automatic serialization/deserialization
- You want to write minimal code

Call your API

After determining what works for you, proceed to call your custom web API.

> **⚠ Warning**
>
> Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

Use IDownstreamApi

1. Install the required NuGet package:

   Bash

   ```
   dotnet add package Microsoft.Identity.Web.DownstreamApi
   dotnet add package Microsoft.Identity.Web.AgentIdentities
   ```

2. Configure token credential options and your APIs in *appsettings.json*.

   JSON

   ```
   {
     "AzureAd": {
       "Instance": "https://login.microsoftonline.com/",
       "TenantId": "your-tenant-id",
       "ClientId": "your-blueprint-id",
       "ClientCredentials": [
         {
           "SourceType": "ClientSecret",
           "ClientSecret": "your-client-secret"
   ```

```
      }
    ]
  },
  "DownstreamApis": {
    "MyApi": {
      "BaseUrl": "https://api.example.com",
      "Scopes": ["api://my-api-client-id/read", "api://my-api-client-
id/write"],
      "RelativePath": "/api/v1",
      "RequestAppToken": false
    }
  }
}
```

3. Configure your services to add downstream API support:

C#

```
using Microsoft.AspNetCore.Authentication.OpenIdConnect;
using Microsoft.Identity.Web;

var builder = WebApplication.CreateBuilder(args);

// Add authentication
builder.Services.AddAuthentication(OpenIdConnectDefaults.AuthenticationSchem
e)
    .AddMicrosoftIdentityWebApp(builder.Configuration.GetSection("AzureAd"))
    .EnableTokenAcquisitionToCallDownstreamApi()
    .AddInMemoryTokenCaches();

// Register downstream APIs
builder.Services.AddDownstreamApis(
    builder.Configuration.GetSection("DownstreamApis"));

// Add Agent Identities support
builder.Services.AddAgentIdentities();

builder.Services.AddControllersWithViews();

var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

4. Call your protected API using `IDownstreamApi`. When calling the API, you can specify the agent identity or agent's user account identity using the `WithAgentIdentity` or `WithAgentUserIdentity` methods. `IDownstreamApi` automatically handles token acquisition and attaches the access token to the request.

- For `WithAgentIdentity`, you either call the API using an app only token (autonomous agent) or on-behalf of a user (interactive agent).

C#

```
using Microsoft.Identity.Abstractions;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[Authorize]
public class ProductsController : Controller
{
    private readonly IDownstreamApi _api;

    public ProductsController(IDownstreamApi api)
    {
        _api = api;
    }

    // GET request for app only token scenario for agent identity
    public async Task<IActionResult> Index()
    {

        string agentIdentity = "<your-agent-identity>";
        var products = await _api.GetForAppAsync<List<Product>>(
            "MyApi",
            "products",
            options => options.WithAgentIdentity(agentIdentity));

        return View(products);
    }

    // GET request for on-behalf of user token scenario for agent
identity
    public async Task<IActionResult> UserProducts()
    {

        string agentIdentity = "<your-agent-identity>";
        var products = await _api.GetForUserAsync<List<Product>>(
            "MyApi",
            "products",
            options => options.WithAgentIdentity(agentIdentity));

        return View(products);
    }
}
```

- For `WithAgentUserIdentity`, you can specify either User Principal Name (UPN) or Object Identity (OID) to identify the agent's user account.

C#

```
using Microsoft.Identity.Abstractions;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[Authorize]
public class ProductsController : Controller
{
    private readonly IDownstreamApi _api;

    public ProductsController(IDownstreamApi api)
    {
        _api = api;
    }

    // GET request for agent's user account identity using UPN
    public async Task<IActionResult> Index()
    {

        string agentIdentity = "<your-agent-identity>";
        string userUpn = "user@contoso.com";

        var products = await _api.GetForUserAsync<List<Product>>(
            "MyApi",
            "products",
            options => options.WithAgentUserIdentity(agentIdentity,
userUpn));
        return View(products);
    }

    // GET request for agent's user account identity using OID
    public async Task<IActionResult> UserProducts()
    {

        string agentIdentity = "<your-agent-identity>";
        string userOid = "user-object-id";

        var products = await _api.GetForUserAsync<List<Product>>(
            "MyApi",
            "products",
            options => options.WithAgentUserIdentity(agentIdentity,
userOid));

        return View(products);
    }

}
```

Related content

- Microsoft identity web

- Call Microsoft Graph APIs
- Call Azure SDKs

Last updated on 08/14/2026

Call Azure services from your agent using .NET Azure SDK

This article guides you on how to call Azure services from your agent. To authenticate to Azure services such as Azure Storage or Azure Key Vault using agent identities, use the `MicrosoftIdentityTokenCredential` class from *Microsoft.Identity.Web.Azure*. The `MicrosoftIdentityTokenCredential` class implements Azure SDK's `TokenCredential` interface, enabling seamless integration between *Microsoft.Identity.Web* and Azure SDK clients.

To call an API from an agent, you need to obtain an access token that the agent can use to authenticate itself to the API. We recommend using the *Microsoft.Identity.Web* SDK for .NET to call your web APIs. This SDK simplifies the process of acquiring and validating tokens. For other languages, use the Microsoft Entra agent SDK for agent ID.

Prerequisites

- An agent identity with appropriate permissions to call the target API. You need a user for the on-behalf-of flow.
- An agent's user account with appropriate permissions to call the target API.

Implementation steps

1. Install the Azure integration package and the *Microsoft.Identity.Web.AgentIdentities* package to add support for agent identities.

   Bash

   ```
   dotnet add package Microsoft.Identity.Web.Azure
   dotnet add package Microsoft.Identity.Web.AgentIdentities
   ```

2. Install the Azure SDK package you want to use, for example, Azure Storage:

   Bash

   ```
   dotnet add package Azure.Storage.Blobs
   ```

3. Configure your services to add Azure token credential support:

C#

```
using Microsoft.Identity.Web;

var builder = WebApplication.CreateBuilder(args);

// Add authentication
builder.Services.AddAuthentication(OpenIdConnectDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApp(builder.Configuration.GetSection("AzureAd"))
    .EnableTokenAcquisitionToCallDownstreamApi()
    .AddInMemoryTokenCaches();

// Add Azure token credential support
builder.Services.AddMicrosoftIdentityAzureTokenCredential();

builder.Services.AddControllersWithViews();
var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

4. Configure Azure token credential options in *appsettings.json*

> ⚠ **Warning**
>
> Client secrets shouldn't be used as client credentials in production environments for agent identity blueprints due to security risks. Instead, use more secure authentication methods such as federated identity credentials (FIC) with managed identities or client certificates. These methods provide enhanced security by eliminating the need to store sensitive secrets directly within your application configuration.

JSON

```
{
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "<your-tenant>",
    "ClientId": "<agent-blueprint-id>",

   // Other client creedentials available. See <https://aka.ms/ms-id-web/client-
credentials>
    "ClientCredentials": [
      {
        "SourceType": "ClientSecret",
        "ClientSecret": "your-client-secret"
      }
    ]
```

```
    }
}
```

5. Acquire a token credential from the service provider and use it with Azure SDK clients.

    a. For agent identities, you can acquire either an app only token (autonomous agents) or an on-behalf of user token (interactive agents) by using the `WithAgentIdentity` method. For app only tokens, set the `RequestAppToken` property to `true`. For delegated on-behalf of user tokens, don't set the `RequestAppToken` property or explicitly set it to `false`.

C#

```
using Microsoft.Identity.Web;

public class AgentService
{
    private readonly MicrosoftIdentityTokenCredential _credential;

    public AgentService(MicrosoftIdentityTokenCredential credential)
    {
        _credential = credential;
    }

    // Call Azure service with the agent identity for app only scenario
    public async Task<List<string>> ListBlobsForAgentAsync(string
agentIdentity)
    {
        // Configure for agent identity
        string agentIdentity = "agent-identity-guid";
        _credential.Options.WithAgentIdentity(agentIdentity);
        _credential.Options.RequestAppToken = true;

        var blobClient = new BlobServiceClient(
            new Uri("https://myaccount.blob.core.windows.net"),
            _credential);

        var container = blobClient.GetBlobContainerClient("agent-data");
        var blobs = new List<string>();

        await foreach (var blob in container.GetBlobsAsync())
        {
            blobs.Add(blob.Name);
        }

        return blobs;
    }

    // Call Azure service with the agent identity for on-behalf of user
scenario
    public async Task<List<string>> ListBlobsForAgentAsync(string
agentIdentity)
```

```
    {
        // Configure for agent identity
        var blobClient = new BlobServiceClient(
            new Uri("https://myaccount.blob.core.windows.net"));

        var container = blobClient.GetBlobContainerClient("agent-data");
        var blobs = new List<string>();

        await foreach (var blob in container.GetBlobsAsync())
        {
            blobs.Add(blob.Name);
        }

        return blobs;
    }
}
```

b. You can also acquire a token for an agent's user account. To do this, you can use either User Principal Name (UPN) or Object Identity (OID) to identify the agent's user account.

For object ID:

C#

```
using Microsoft.Identity.Web;

public class AgentService
{
    private readonly MicrosoftIdentityTokenCredential _credential;

    public AgentService(MicrosoftIdentityTokenCredential credential)
    {
        _credential = credential;
    }

    // Use object ID to identify the agent's user account
    public async Task<List<string>> ListBlobsForAgentAsync(string
agentIdentity)
    {
        // Configure for agent identity
        string agentIdentity = "agent-identity-guid";
        string userOid = "user-object-id";
        _credential.Options.WithAgentUserIdentity(agentIdentity, userOid);

        var blobClient = new BlobServiceClient(
            new Uri("https://myaccount.blob.core.windows.net"),
            _credential);

        var container = blobClient.GetBlobContainerClient("agent-data");
        var blobs = new List<string>();

        await foreach (var blob in container.GetBlobsAsync())
```

```
        {
            blobs.Add(blob.Name);
        }

        return blobs;
    }

    // Use UPN to identify the agent's user account\
    public async Task<List<string>> ListBlobsForAgentAsync(string
agentIdentity)
    {
        // Configure for agent identity
        string agentIdentity = "agent-identity-guid";
        string userUpn = "user@contoso.com";

        _credential.Options.WithAgentUserIdentity(agentIdentity, userUpn);

        var blobClient = new BlobServiceClient(
            new Uri("https://myaccount.blob.core.windows.net"),
            _credential);

        var container = blobClient.GetBlobContainerClient("agent-data");
        var blobs = new List<string>();

        await foreach (var blob in container.GetBlobsAsync())
        {
            blobs.Add(blob.Name);
        }

        return blobs;
    }
}
```

Related content

- Call custom APIs
- Call Microsoft Graph

Last updated on 05/01/2026

Secure a Model Context Protocol (MCP) server with Microsoft Entra ID

The Model Context Protocol (MCP) lets AI agents and other clients call tools and data sources that you expose from an MCP server. Because an MCP server can act on behalf of a user or an autonomous agent, you should protect it like any other API: require an OAuth 2.0 access token on every request, and validate that token before you run a tool.

This article explains how to configure Microsoft Entra ID as the authorization server for your MCP server, register the server so that MCP clients can request tokens for it, and connect an MCP client by using Microsoft Entra Agent ID.

> **Important**
>
> Don't write your own token-validation logic from scratch. Token validation bugs can silently expose your MCP server to unauthorized callers. Use a well-tested authentication library or middleware for your platform, and follow the source-of-truth guidance linked in **Validate access tokens in your MCP server**.

Prerequisites

- A **Microsoft Entra tenant** where you can register applications. To register apps, you need at least the Application Developer role.
- An **MCP server** that you host at an HTTPS URL, for example `https://mcp.contoso.com`.
- Familiarity with the MCP authorization specification, which builds on OAuth 2.1, Resource Indicators for OAuth 2.0 (RFC 8707), and OAuth 2.0 Protected Resource Metadata (RFC 9728).

How MCP authorization works with Microsoft Entra ID

In the MCP authorization model:

- Your **MCP server** is an OAuth 2.0 *protected resource*.
- **Microsoft Entra ID** is the *authorization server* that issues access tokens.
- The **MCP client**, such as an agent, is the *client* that requests a token and presents it to your server.

The MCP specification requires the client to identify the protected resource it wants a token for by sending the `resource` parameter (from RFC 8707) in its token request. The value of `resource` is the canonical URL of your MCP server. Microsoft Entra ID compares that `resource` value against the **Application ID URI** (also called the identifier URI) configured on your MCP server's app registration. If they match, Microsoft Entra ID issues a token whose audience (`aud`) claim is your MCP server. If they don't match, the token request fails.

For this matching to work, your MCP server must accept **v2 access tokens**. See Register your MCP server in Microsoft Entra ID for the exact steps to configure the access token version.

The end-to-end flow looks like this:

1. The MCP client discovers your server's authorization requirements. When an unauthenticated request arrives, your server returns `HTTP 401` with a `WWW-Authenticate` header that points to your protected resource metadata (RFC 9728), which in turn identifies Microsoft Entra ID as the authorization server.
2. The MCP client requests an access token from Microsoft Entra ID, sending `resource=<your MCP server URL>`.
3. Microsoft Entra ID validates the request, matches `resource` to your app registration's Application ID URI, and issues a v2 access token scoped to your server.
4. The MCP client calls your MCP server with the token in the `Authorization: Bearer` header.
5. Your MCP server validates the token and, if it's valid, runs the requested tool.

Register your MCP server in Microsoft Entra ID

Register your MCP server as an application so that Microsoft Entra ID can issue tokens for it.

If your MCP server is served by an an existing web service or REST API that is already secured by Entra - you may re-use its existing app registration and its OAuth scopes.

The order of the following steps matters: you must enable v2 access tokens *before* you can set an HTTPS Application ID URI that matches your server URL.

Step 1: Create the app registration

1. Sign in to the Microsoft Entra admin center.
2. Browse to **Entra ID** > **App registrations** > **New registration**.
3. Enter a name for your MCP server, such as `Contoso MCP server`, and select **Register**.

Step 2: Set the access token version to v2

This is the step that most people miss. Your app registration must request v2 access tokens by setting `requestedAccessTokenVersion` to `2` in the Microsoft Graph app manifest.

1. In your app registration, select **Manifest**.
2. In the `api` object, set `requestedAccessTokenVersion` to `2`:

   JSON

   ```
   "api": {
       "requestedAccessTokenVersion": 2
   }
   ```

3. Select **Save**.

> **Note**
>
> If you skip this step and leave the app on v1 access tokens, you can't set an HTTPS Application ID URI that matches your MCP server's URL, and MCP clients that send `resource=<your MCP server URL>` receive an error. See FAQ and troubleshooting.

Step 3: Set the Application ID URI to your MCP server URL

After the app issues v2 access tokens, set the Application ID URI to the exact canonical URL of your MCP server. This value is what MCP clients send in the `resource` parameter.

1. In your app registration, select **Expose an API**.
2. Next to **Application ID URI**, select **Add** (or **Edit**), and enter your MCP server's URL, for example `https://mcp.contoso.com`.
3. Select **Save**.

Alternatively, set `identifierUris` directly in the manifest. The `identifierUris` property is a list, so if your MCP server is reachable at more than one URL, add every URL that clients might use. A client typically sends the URL it connected to as its `resource` value, and Microsoft Entra ID only issues a token when that value exactly matches one of the registered Application ID URIs.

JSON

```
"identifierUris": [
    "https://mcp.contoso.com",
    "https://mcp.contoso.com/mcp",
```

```
    "https://contoso-mcp.azurewebsites.net"
]
```

Step 4: Define scopes

Delegated scopes let an MCP client that acts on behalf of a signed-in user request only the level of access it needs. If your MCP server serves interactive clients, define at least one OAuth scope so that clients can request a token for your server and users or administrators can consent to it.

1. In your app registration, select **Expose an API**.
2. Select **Add a scope**.
3. Enter a scope name that reflects an operation your server exposes, such as `tool.read` or `tool.execute`.
4. Choose who can consent (**Admins only**, or **Admins and users**), enter the consent display names and descriptions, set the state to **Enabled**, and select **Add scope**.

An MCP client then requests the fully qualified scope, which combines your Application ID URI and the scope name — for example, `https://mcp.contoso.com/tool.execute`.

Define as many OAuth scopes as needed to provide granular access to your MCP server.

Step 5 (Optional): Define app roles

In addition to acting on behalf of users, MCP clients can also authenticate as an agent identity. To request an access token as an agent identity, the MCP client sends an OAuth client credentials grant request and uses a scope value such as `scope=https://mcp.contoso.com/.default`.

Entra **app roles** can be used to limit the access an MCP client has against your MCP server. You assign app roles to the MCP client's agent identity. Assigned roles appear in the `roles` claim of the access token, and your server checks that claim before it runs a tool.

1. In your app registration, select **App roles** > **Create app role**.
2. Enter a display name and a value such as `Tools.Execute.All`, set **Allowed member types** to **Applications**, and enable the role.
3. Assign the role to the agent identity (or other client application) that calls your server.

For more information, see Add app roles and receive them in the token.

Publish protected resource metadata

MCP clients discover how to authenticate to your server by reading its OAuth 2.0 Protected Resource Metadata (PRM) document. The MCP authorization specification requires every HTTP MCP server to serve a PRM document and to reference it from the `WWW-Authenticate` header of its `401` responses. The document tells clients which authorization server to use (Microsoft Entra ID) and which resource identifier to request a token for.

Serve the document as JSON from the well-known path derived from your server's URL, for example `https://mcp.contoso.com/.well-known/oauth-protected-resource`. For a server secured by Microsoft Entra ID, it should look like this:

JSON

```
{
  "resource": "https://mcp.contoso.com",
  "authorization_servers": [
    "https://login.microsoftonline.com/<tenant-id>/v2.0"
  ],
  "scopes_supported": [
    "tools.read",
    "tools.execute"
  ],
  "bearer_methods_supported": ["header"]
}
```

Set the fields as follows:

- **`resource`** — the canonical URL of your MCP server. This value must be identical to the Application ID URI you registered in Step 3 and to the `resource` parameter that clients send. If your server exposes multiple URLs, each PRM document describes one specific URL, and clients request metadata for the URL they connected to.
- **`authorization_servers`** — the Microsoft Entra ID issuer for your tenant, `https://login.microsoftonline.com/<tenant-id>/v2.0`. Clients perform authorization server metadata discovery against this issuer (for example, at `https://login.microsoftonline.com/<tenant-id>/v2.0/.well-known/openid-configuration`) to find Entra's authorization and token endpoints. Use your tenant ID or a verified domain; use `organizations` or `common` in place of the tenant ID only for multitenant servers.
- **`scopes_supported`** — optional. List the delegated scopes you defined in Step 4. Clients use these to decide what to request when the `WWW-Authenticate` challenge doesn't specify a `scope`.
- **`bearer_methods_supported`** — set to `["header"]` because your server expects the token in the `Authorization: Bearer` request header.

> **Important**
>
> The `authorization_servers` value must be the **v2.0** issuer (`.../v2.0`), which matches the `iss` claim of the v2 access tokens your server accepts (see Step 2). The v1 issuer (`https://sts.windows.net/<tenant-id>/`) doesn't line up with the v2.0 authorization server metadata that clients discover.

On an unauthenticated request, return a `401` response whose `WWW-Authenticate` header points to this document. Optionally include a `scope` parameter to tell the client which scopes the current operation requires:

```http
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer resource_metadata="https://mcp.contoso.com/.well-known/oauth-protected-resource", scope="tools.execute"
```

If you use an official MCP SDK or Azure App Service authentication, the platform can generate and serve this document and the `401` challenge for you. See Validate access tokens in your MCP server.

Validate access tokens in your MCP server

On every request, your MCP server must validate the access token before it runs a tool. At a minimum, verify the token's **signature**, **issuer** (`iss`), **tenant ID** (`tid`), **audience** (`aud`, which must equal the application ID of your MCP server), and expiry. Then check that the **subject** (`sub` or `oid`) of the token is authorized for the requested operation using the **role** (`roles`) claim or another authorization check. For delegated access tokens, also be sure to check the **scope** (`scp`) claim to confirm the caller / actor is authorized for the requested operation.

To avoid introducing a security vulnerability, don't implement these checks by hand. Use the guidance and libraries in the following source-of-truth articles:

- Protected web API: overview and Protected web API: code configuration — the canonical pattern for protecting an API with Microsoft Entra ID, including recommended middleware such as Microsoft.Identity.Web.
- Secure access tokens and Validate tokens — how to validate signature, issuer, and audience correctly.

- Secure applications and APIs by validating claims — how to authorize requests by validating the claims in a token, beyond signature and audience checks.
- Validate agent identity tokens in a downstream API — a worked example for Microsoft Entra Agent ID tokens, including how to detect that a caller is an agent identity.

If you host your MCP server on **Azure App Service**, you can offload authentication to the platform's built-in authentication (Easy Auth) instead of validating tokens in your app code. For step-by-step guidance, see Secure MCP servers with Microsoft Entra authentication on Azure App Service.

Connect an MCP client using Microsoft Entra Agent ID

When the MCP client is an AI agent, use Microsoft Entra Agent ID so the agent authenticates with its own identity instead of an embedded secret. The agent acquires a token for your MCP server and presents it on each call.

1. Set up the agent's identity. See Create and delete agent identities and Authentication protocols in agents.
2. Grant the agent access to your MCP server, either by consenting to a delegated scope that you defined in Step 4 or by assigning an app role that you defined in Step 5.
3. Have the agent acquire a token for your MCP server. The client sends `resource=<your MCP server URL>` (per the MCP specification) and requests the appropriate OAuth scope.
4. The agent calls your MCP server with the token in the `Authorization: Bearer` header. Your server validates it as described in Validate access tokens in your MCP server.

For a broader view of how agents call protected resources, see Configure third-party agents with Agent ID and Call custom APIs from an agent.

FAQ and troubleshooting

Should my MCP server use v1 or v2 access tokens?

Use **v2 access tokens**. Set `requestedAccessTokenVersion` to `2` on your MCP server's app registration, as described in Step 2. MCP authorization relies on matching the client's `resource` parameter to an HTTPS Application ID URI, and that configuration requires v2 tokens.

I get error AADSTS9010010 when a client requests a token for my MCP server

`AADSTS9010010` means the **Application ID URI configured on your app registration doesn't exactly match the value of the `resource` parameter** in the client's OAuth request. To fix it:

1. **Check for a trailing slash or other exact-match difference.** The `resource` value the client sends must match the Application ID URI character for character, including the scheme (`https://`), casing of the host, and path. A trailing slash is a common culprit: because an Application ID URI can't end with a slash, a client that sends `resource=https://mcp.contoso.com/` never matches the registered `https://mcp.contoso.com`. Remove the trailing slash (and any other difference) so the client's `resource` value and the Application ID URI are identical.
2. **Confirm your app uses v2 access tokens.** If `requestedAccessTokenVersion` isn't set to `2`, you can't register an HTTPS Application ID URI that matches your MCP server URL, so the `resource` value can never match. Complete Step 2, then re-add the Application ID URI in Step 3.
3. **Make sure the requested scope belongs to the same app registration as the `resource`.** This error also occurs when the client sends a scope that's defined on a *different* app registration than the one referenced by the `resource` parameter. Microsoft Entra ID resolves the `resource` and the scope to the same resource app; if they point at different app registrations, the request fails. Confirm that every scope the client requests is exposed by the app registration whose Application ID URI you send in `resource`, and remove any scope that belongs to another app.

What if I can't use v2 access tokens?

Some MCP servers reuse an existing app registration that already issues v1 access tokens to other clients, and can't switch to v2 tokens without breaking those clients. In that case, you can keep v1 tokens and instead make the `aud` claim deterministic by configuring the aud optional claim with the `use_guid` additional property.

By default, the `aud` claim in a v1 access token is nondeterministic — Microsoft Entra ID can emit any of the app's Application ID URIs (with or without a trailing slash) or the resource's client ID. Setting `use_guid` forces `aud` to *always* be the resource's client ID in GUID format. Because the audience is no longer tied to a specific Application ID URI, Microsoft Entra ID relaxes the Application ID URI restrictions for the app, so you can add your MCP server's HTTPS URL as an Application ID URI (Step 3) even though the app still issues v1 tokens. Clients can then send `resource=<your MCP server URL>` and receive a token.

Add the optional claim to your app registration's manifest:

JSON

```
"optionalClaims": {
    "accessToken": [
        {
            "name": "aud",
            "additionalProperties": ["use_guid"]
        }
    ]
}
```

With `use_guid` set, the token's `aud` claim is your resource's **client ID** (a GUID), not your MCP server URL. Configure your MCP server (or the middleware or MCP SDK that validates tokens) to expect that client-ID GUID as the audience. Whenever you can move to v2 tokens, prefer that approach (Step 2), because the v2 `aud` claim is consistently the resource's client ID and aligns with the authorization server metadata that MCP clients discover.

Related content

- What is Microsoft Entra Agent ID?
- Authentication protocols in agents
- Validate agent identity tokens in a downstream API
- Protected web API: overview
- Secure MCP servers with Microsoft Entra authentication on Azure App Service
- Understand the app manifest (Microsoft Graph format)

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/03/2026

Integrate third-party agents with Microsoft Entra Agent ID

Microsoft Entra Agent ID enables AI agents from third-party platforms to authenticate and access your APIs securely without handling credentials directly. This article covers two integration patterns - the Microsoft Entra Auth SDK (sidecar) and federation - for platforms such as Amazon Web Service (AWS) Bedrock and n8n.

Prerequisites

Before you start, make sure you have:

- **Microsoft Entra tenant** with agent identity capabilities enabled.
- **Azure subscription**, required for some deployment options.
- **Docker** and **Docker Compose** for the sidecar pattern.
- **Credentials or federation setup**, depending on the pattern you choose.
- **PowerShell 7.5 or later** with the Microsoft Graph PowerShell module.
- **Global Administrator** role, required only for initial setup. Use Privileged Identity Management (PIM) to activate this role just-in-time.
- **Cloud Application Administrator** or **Application Administrator** role to grant Microsoft Graph delegated permissions for agent management operations.

To verify your environment is ready:

1. Confirm you have the permissions to create applications and service principals in your Microsoft Entra tenant.
2. If you're deploying to Azure, confirm your subscription and resource group.
3. Review the documentation for your agent platform, such as AWS Bedrock or n8n.

Why you need third-party agent integration

Organizations use AI agents from multiple platforms, such as AWS Bedrock, n8n, and others. These agents often need to:

- Call Microsoft APIs such as Microsoft Graph and Azure services.
- Access your internal APIs and resources.
- Authenticate securely without storing secrets in code or configuration.

Microsoft Entra Agent ID provides a centralized, secure identity service that third-party agents can use to acquire tokens on demand, without managing secrets or certificates directly. By using Microsoft Entra Agent ID, you can:

- Remove the need for agents to handle credentials directly.
- Use workload identity federation for agents running outside Azure.
- Support multiple authentication patterns, including client credentials, federated identity, and on-behalf-of.
- Integrate with third-party agent platforms by using the Microsoft Entra Auth SDK (sidecar).

Integration patterns for third-party agents

To integrate third-party agents with Microsoft Entra Agent ID, choose from the following patterns:

Use the Microsoft Entra Auth SDK (sidecar)

The **sidecar pattern** runs the Microsoft Entra Auth SDK as a companion container alongside your agent. The agent calls the sidecar to request tokens for API calls. The agent never handles credentials directly; instead, it delegates token acquisition to the sidecar.

Best for:

- Containerized agents on Docker or Kubernetes.
- AWS Bedrock agents running in your own orchestration.
- Local development with Docker Compose.
- Organizations already using container infrastructure.

Supported platforms:

- AWS Bedrock, including Claude and other foundation models.
- Local Large Language Models (LLMs) such as Ollama with LangChain.
- Any containerized agent.

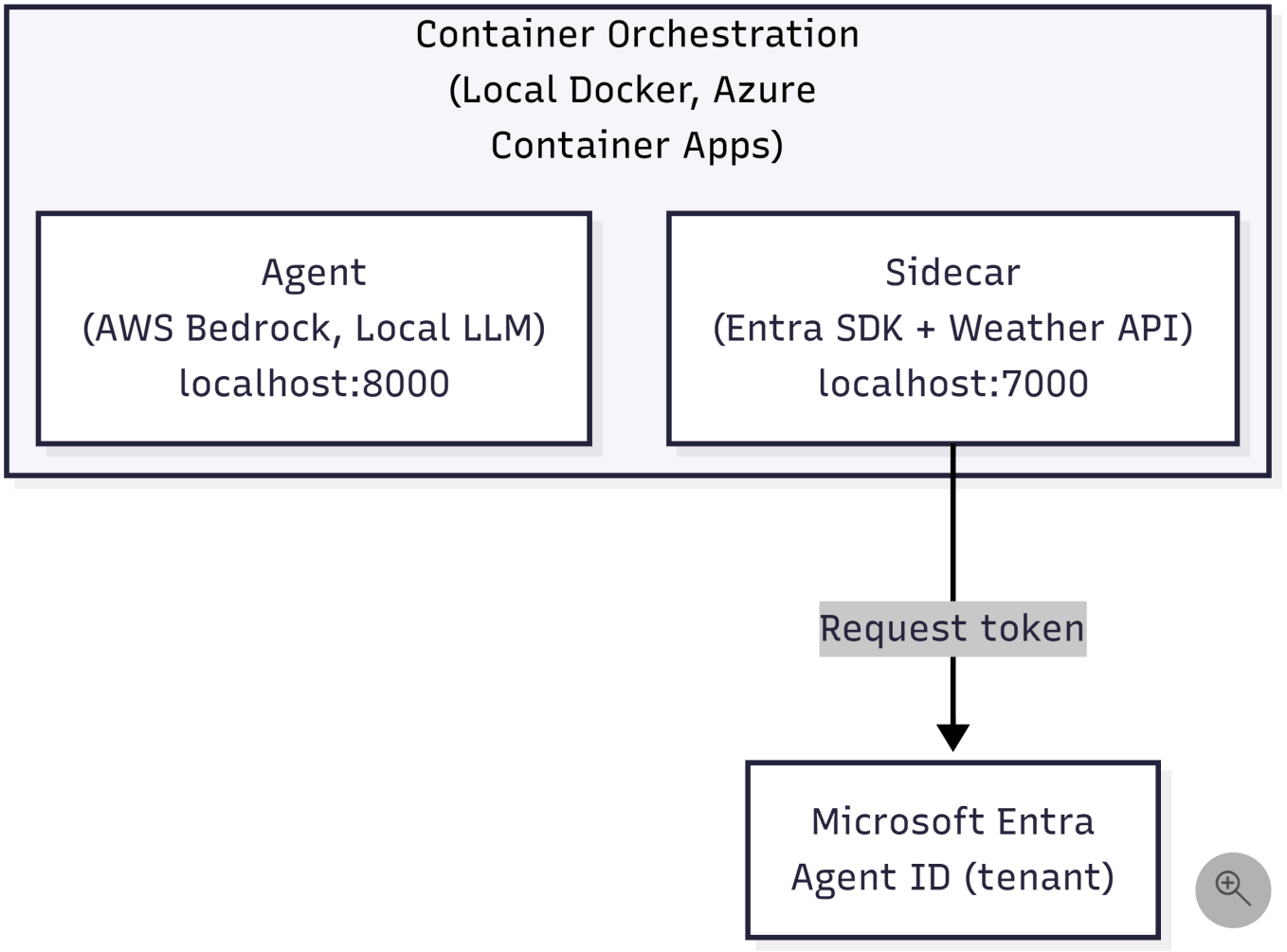
Advantages:

- Credential-free agent code.
- Works with any containerized agent.
- Easy local development with Docker Compose.
- Can be deployed to Azure Container Apps, Kubernetes, or on-premises.

Consideration:

- Requires managing a second container.

The following diagram shows the sidecar architecture. An agent container and a sidecar container run together in the same orchestration environment. The agent requests tokens from the sidecar, which communicates with Microsoft Entra Agent ID to acquire access tokens.



Use Workload Identity Federation (direct identity exchange)

The **federation pattern** uses Workload Identity Federation to exchange credentials from external identity providers such as AWS Security Token Service (STS) directly for Microsoft Entra tokens. This pattern doesn't need a sidecar.

Best for:

- AWS agents that use STS and OIDC.
- Organizations that already have federation infrastructure.
- Agents that can't run containers.

Supported platforms:

- GCP Workload Identity → Microsoft Entra Agent ID.
- AWS STS → Microsoft Entra Agent ID.

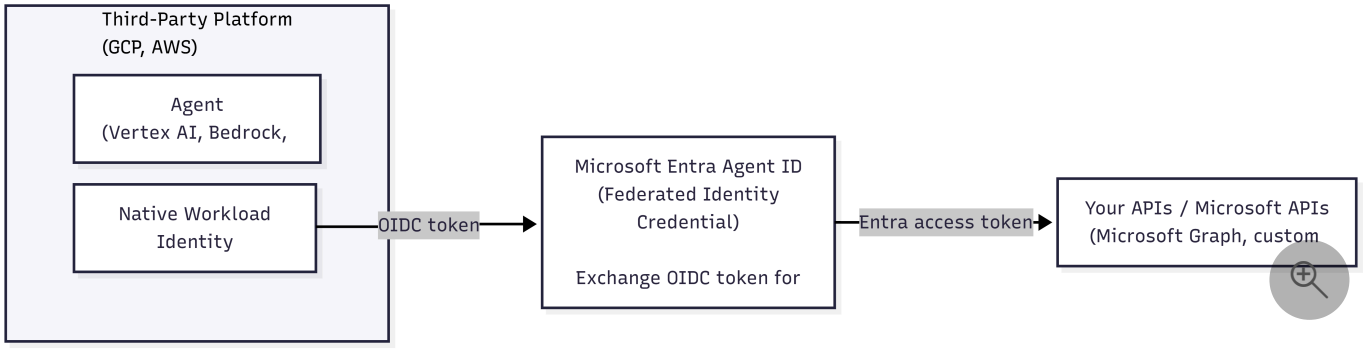
Advantages:

- No sidecar required.
- Uses existing infrastructure in AWS, and other platforms.
- Direct token exchange at the identity layer.

Requirements:

- A preconfigured Federated Identity Credential in Microsoft Entra.
- An agent platform that supports OIDC or STS.

The following diagram shows the federation flow. An agent on a third-party platform authenticates through its native workload identity provider, exchanges the resulting OIDC token for a Microsoft Entra token, and then calls your APIs.



Understand the token flow

Both patterns follow the same core token flow:

1. **Agent requests a token.** The agent, or the sidecar on the agent's behalf, calls Microsoft Entra Agent ID with authentication credentials.
2. **Microsoft Entra validates the identity.** Microsoft Entra verifies the agent's identity through client credentials, a federated credential, or another supported method.
3. **Microsoft Entra returns a token.** The agent receives a Microsoft Entra access token.
4. **Agent calls the API.** The agent uses the token to authenticate to Microsoft or custom APIs.
5. **API validates the token.** The API checks the token signature and claims, then grants access.

Common integration scenarios

AWS Bedrock agent calling Microsoft Graph

An AWS Bedrock agent, such as Claude, needs to query Microsoft 365 data or manage resources through Microsoft Graph. The sidecar pattern is ideal for this scenario. To integrate an AWS Bedrock agent with the sidecar pattern:

1. Deploy the agent and sidecar to AWS or your own infrastructure.
2. Configure an Agent Identity in Microsoft Entra with permissions to Microsoft Graph.
3. The agent calls the sidecar for a token.
4. The sidecar acquires a token from Microsoft Entra Agent ID.
5. The agent uses the token to call Microsoft Graph.

For step-by-step instructions, see Secure an Amazon Bedrock agent with Microsoft Entra Agent ID.

n8n agent calling Microsoft Graph and MCP Server for Enterprise

An n8n agent needs to access Microsoft 365 data through Microsoft Graph or the Microsoft Graph MCP Server for Enterprise. This scenario uses the n8n-nodes-entraagentid community node to manage token acquisition directly within n8n workflows. To integrate an n8n agent:

1. Deploy n8n to Azure Container Apps by using the Azure Developer CLI (`azd`).
2. Configure an Agent Identity in Microsoft Entra with permissions to Microsoft Graph.
3. The n8n workflow uses the community node to acquire a token from Microsoft Entra Agent ID.
4. The agent uses the token to call Microsoft Graph or the MCP Server for Enterprise.

For step-by-step instructions, see Secure an n8n agent with Microsoft Entra Agent ID.

Local development with Ollama

You're developing with a local LLM like Ollama and want to test authentication before deploying. Use the sidecar pattern with Docker Compose. To test locally:

1. Run the agent and sidecar in Docker Compose.
2. The agent calls `localhost:7000/token` to request a token from the sidecar.
3. The sidecar acquires a token from Microsoft Entra Agent ID.
4. Test agent behavior locally before deploying.

For step-by-step instructions, see Run the sidecar for local development.

High-level roadmap

Use the following table to identify the steps for your chosen pattern:

Expand table

Step	Pattern	Task
1	Both	Set up Agent Identity and permissions in Microsoft Entra.
2	Both	Choose an integration pattern: sidecar or federation.
3	Sidecar	Deploy agent and sidecar containers, then test locally.
4	Sidecar	Deploy to production on Azure Container Apps, Kubernetes, or another platform.
5	Federation	Configure federated identity credentials in Microsoft Entra.
6	Federation	Deploy the agent to the target platform.

Security best practices

When you integrate third-party agents, follow these security principles:

- **Never embed credentials in agent code.** Use Microsoft Entra Agent ID to acquire tokens dynamically.
- **Use least privilege.** Grant Agent Identities only the permissions they need through roles or scopes.
- **Validate token audience and issuer.** Always verify that tokens come from your Microsoft Entra tenant.
- **Rotate credentials regularly.** If you use client secrets, rotate them on a schedule. Consider federated credentials instead.
- **Monitor token usage.** Use Microsoft Entra logs to track which agents access which APIs.
- **Keep the Microsoft Entra SDK updated.** Security and compatibility updates are released regularly.

Troubleshoot common issues

If you run into problems during integration, use the following guidance to identify the cause and resolution:

Expand table

Issue	Cause	Solution
Agent can't reach sidecar	Network configuration issue or sidecar not running	Verify the sidecar is running, check DNS and networking, and confirm port binding. The default port is 7000.
Sidecar fails to acquire token	Microsoft Entra authentication failed	Verify Agent Identity credentials, check Microsoft Entra permissions, and review the tenant ID and client ID.
Token request returns 401	Invalid Microsoft Entra credentials or federated credential not configured	Confirm credentials are correct and verify the federated identity credential is set up if you're using the federation pattern.
API rejects token	Token lacks the required scope or permission	Add required API permissions to the Agent Identity and request a token with the correct scope.

Related content

- Configure SDK for agent identities
- Call Microsoft Graph API
- Call custom APIs
- Call Azure services
- Best practices for Agent ID

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/01/2026

Secure an Amazon Bedrock agent with Microsoft Entra Agent ID

This guide shows you how to secure an Amazon Bedrock agent by using the Microsoft Entra ID Auth SDK (sidecar) to authenticate to downstream APIs. The sidecar runs as a separate container and handles all credential management and token exchange with Microsoft Entra ID. Your agent requests an authorization header from the sidecar, and the sidecar handles the OAuth 2.0 exchange with Microsoft Entra ID.

Prerequisites

Before you begin, make sure you have:

- A Microsoft Entra tenant.
- An Azure subscription.
- Docker Desktop (macOS/Windows) or Docker Engine with Compose v2 (Linux).
- PowerShell 7+.
- Azure CLI.
- AWS CLI v2.
- An AWS account with Bedrock model access enabled for Anthropic Claude 3 Haiku (or your preferred model). Enable access in the AWS Bedrock console under **Model access** > **Manage model access**.
- **Global Administrator** role for first-time Microsoft Entra setup. Use Privileged Identity Management (PIM) to activate this role just-in-time.

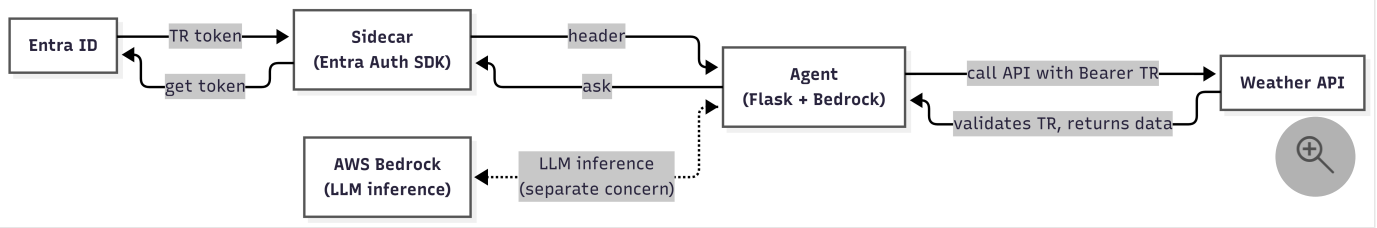
Clone the sample repository

1. Clone the repository and go to the AWS sample directory:

```bash
git clone https://github.com/microsoft/entra-agentid-samples.git
cd entra-agentid-samples/sidecar/aws
```

Architecture

The Microsoft Entra ID Auth SDK (sidecar) sits between your agent and Microsoft Entra ID. The agent never talks to Microsoft Entra ID directly and never manages credentials. It asks the sidecar for an `Authorization` header to call a downstream API. Amazon Bedrock handles LLM inference separately, without having to worry about identity.



The sample runs three containers on a Docker bridge network:

- **`llm-agent-aws`**: Flask app with a chat UI and a LangGraph ReAct agent that calls Amazon Bedrock (Claude) for reasoning. Exposed on port 3001.
- **`agent-id-sidecar-aws`**: The official Microsoft Entra ID Auth SDK (sidecar) container. Acquires and caches tokens. No host port, reachable only from within the Docker network.
- **`weather-api-aws`**: A downstream API that validates the agent's JWT (signature, issuer, expiry, audience) on every request and returns weather data.

The request flows through these steps:

1. You type a query in the chat UI at `http://localhost:3001`.
2. The Flask app sends the query to AWS Bedrock (Claude) via the LangGraph ReAct agent.
3. When Claude decides it needs weather data, it calls the `get_weather` tool.
4. The tool asks the sidecar for an authorization header by calling `GET /AuthorizationHeader?AgentIdentity={agentId}`.
5. The sidecar authenticates to Microsoft Entra ID using OAuth 2.0 (client credentials or on-behalf-of (OBO) exchange).
6. Microsoft Entra ID returns the requested token (TR) to the sidecar.
7. The agent calls the weather API with `Authorization: Bearer TR`.
8. The weather API validates TR and returns the weather JSON response.

Understand the token flow

Three tokens are involved in the identity exchange:

Expand table

Token	Issued to	When	How
Tc	Signed-in user	OBO flow only	MSAL.js in the browser
T1	Blueprint app	Both flows	Sidecar (client credentials)
TR	Agent (downstream API)	Both flows	Sidecar-app-only (autonomous) or OBO exchange

In the autonomous flow, the sidecar uses client credentials to get T1, then exchanges it for TR scoped to the downstream API. In the OBO flow, the sidecar also receives Tc (the user's token) and performs an OBO exchange to get TR that acts on behalf of the signed-in user.

In this setup, only the chat UI (port 3001) is exposed to your host. The sidecar and weather API are only reachable within the Docker network, which establishes a clear security boundary.

Choose an execution mode and identity flow

The sample supports two execution modes and two identity flows that you can combine:

Expand table

	Autonomous **(app-only)**	OBO **(on behalf of user)**
Direct (no LLM)	Fast demo path. Token fetched and weather API called directly.	Same, but uses the authenticated sidecar endpoint with the user token.
Bedrock + LangChain	LangGraph ReAct agent decides when to call `get_weather`.	Same, but the agent passes the user token through when the tool runs.

Use **Direct** mode to verify your token flow end-to-end without needing AWS Bedrock access. Switch to **Bedrock** mode for the full agent experience.

Choose an AWS authentication tier

The sample supports three ways to authenticate to Amazon Bedrock. Choose the tier that matches your environment:

- **Temporary STS credentials:** Best for local development with AWS SSO. Set `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN` in your `.env` file. These credentials expire after approximately one hour.
- **Bedrock API key:** Best for demos and workshops. Set `AWS_BEARER_TOKEN_BEDROCK` in your `.env` file. Scoped to Bedrock only with a configurable lifetime.

- **OIDC federation:** Best for production deployments on Azure App Service. Uses `AWS_ROLE_ARN` and `AWS_WEB_IDENTITY_TOKEN_FILE` set by the platform. No secrets stored anywhere.

> **Tip**
>
> For production deployments, see the Azure App Service deployment guide for step-by-step instructions on setting up OIDC federation between Azure and AWS with zero stored secrets.

Select a Bedrock model

The sample defaults to `us.anthropic.claude-3-haiku-20240307-v1:0` because it's the cheapest Anthropic model on Bedrock and supports tool calling. The `us.` prefix indicates a cross-region inference profile that routes between US regions for higher availability.

Other supported models:

Expand table

Model ID	Cost per 1K input tokens	Notes
`us.anthropic.claude-3-haiku-20240307-v1:0`	$0.00025	Default. Fast, cheapest, supports tool calling.
`us.anthropic.claude-3-5-haiku-20241022-v1:0`	$0.0008	Newer, smarter, still affordable.
`us.anthropic.claude-3-5-sonnet-20241022-v2:0`	$0.003	Best quality/cost ratio.

Override the default by setting `BEDROCK_MODEL_ID` in your `.env` file. You must enable each model in the **AWS Bedrock console** > **Model access** before it can be invoked.

Create the Microsoft Entra objects (first-time setup)

If you already have a `.env` file from a previous run with `BLUEPRINT_APP_ID` populated, skip to Configure environment variables.

Run the following commands once per tenant to create the Blueprint app, Agent ID, and the SPA app used for OBO sign-in.

1. Create the Blueprint app and Agent ID for the autonomous flow by following the PowerShell workflow in Create an agent identity blueprint and Create agent identities. At the end you have:

   - `TENANT_ID`: Your Microsoft Entra tenant.
   - `BLUEPRINT_APP_ID`: Blueprint app registration.
   - `BLUEPRINT_CLIENT_SECRET`: Client secret for the Blueprint.
   - `AGENT_CLIENT_ID`: The Agent ID created from the Blueprint.

2. (Optional) Create the SPA app and configure OBO. This step is required only if you want to use the OBO identity flow:

   Run the following scripts to create the SPA app registration and configure OBO permissions on the Blueprint. The scripts register the SPA redirect URI and grant the required delegated permissions.

   Bash:

   ```Bash
   bash ../../scripts/setup-obo-client-app.sh
   bash ../../scripts/setup-obo-blueprint.sh
   ```

   PowerShell:

   ```PowerShell
   pwsh ../../scripts/setup-obo-client-app.ps1
   pwsh ../../scripts/setup-obo-blueprint.ps1 `
       -TenantId        '<TENANT_ID>' `
       -BlueprintAppId  '<BLUEPRINT_APP_ID>' `
       -AgentAppId      '<AGENT_CLIENT_ID>' `
       -ClientSpaAppId  '<CLIENT_SPA_APP_ID>'
   ```

The SPA redirect URI for this sample is `http://localhost:3001` (port 3001, not 3003). Make sure this URI is registered.

Configure environment variables

The sidecar supports multiple credential types through the `AzureAd__ClientCredentials__0__SourceType` setting in `docker-compose.yml`:

- `ClientSecret`: Local development only. The sample includes this type.

- `SignedAssertionFromManagedIdentity`: Deployed on Azure. Zero secrets, recommended for production.
- `KeyVault`: Certificate from Azure Key Vault.
- `StoreWithThumbprint`: Certificate from local machine store.

1. Create a local `.env` configuration file from the included template. This file stores your tenant, app, and AWS credentials:

Bash:

```
Bash
cp .env.example .env
```

PowerShell:

```
PowerShell
Copy-Item .env.example .env
```

1. Set the following variables in your `.env` file:

    - `TENANT_ID`: Your Microsoft Entra tenant ID.
    - `BLUEPRINT_APP_ID`: Blueprint app registration. The sidecar authenticates as this app.
    - `BLUEPRINT_CLIENT_SECRET`: Blueprint client secret (local development only).
    - `AGENT_CLIENT_ID`: Your Agent ID. Appears as the `AgentIdentity` query parameter.
    - `CLIENT_SPA_APP_ID`: SPA app ID used by MSAL.js for browser sign-in (OBO only).
    - `AWS_REGION`: AWS region for Bedrock, such as `us-east-2`.
    - `BEDROCK_MODEL_ID`: Model ID. Default: `us.anthropic.claude-3-haiku-20240307-v1:0`.
    - `VALIDATE_TOKEN_SIGNATURE`: Default `true`. Set to `false` to skip JWKS signature validation in the weather API (debugging only).

2. Add your AWS credentials based on the tier you chose:

    - **Tier A (STS):** Set `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN`.
    - **Tier B (API key):** Set `AWS_BEARER_TOKEN_BEDROCK`.
    - **Tier C (OIDC):** Configured through platform App Settings, not in `.env`.

> **Tip**

If you already have a `.env` from a previous session, you only need to refresh your AWS credentials (STS tokens expire after about one hour). Skip directly to **Start the stack**.

Start the stack

1. Verify that Docker Desktop (or Docker Engine) is running on your machine.
2. Build the container images and start all three services (agent, sidecar, and weather API) in detached mode:

   ```bash
   docker compose up --build -d
   ```

3. Check that all containers started successfully by querying the status endpoint. The response reports whether the agent can reach AWS Bedrock:

   Bash:

   ```bash
   curl http://localhost:3001/api/status
   ```

   PowerShell:

   ```powershell
   Invoke-RestMethod http://localhost:3001/api/status
   ```

   You see a response indicating `bedrock_available: true` (or `false` if you're using Direct mode without AWS credentials).

> **ⓘ Important**
>
> When you update `.env` (for example, to refresh expired STS credentials), `docker compose restart` doesn't reload environment variables. Use `docker compose up -d --force-recreate llm-agent-aws` instead.

Send a query through the chat UI

1. Open `http://localhost:3001` in your browser.
2. Use the header bar to configure your demo:
   - **Execution Mode:** `Direct` (skip LLM) or `Bedrock` (LangChain ReAct agent on Claude).
   - **Identity Flow:** `Autonomous` (app-only token) or `OBO` (acts for a signed-in user).
3. If you select **OBO**, select **Sign in** to authenticate through the MSAL.js popup.
4. Type a query like "*Weather in Dallas?*" and select **Send**.
5. Watch the **Identity Trace** panel on the right side for a step-by-step breakdown of every token exchange and API call. The panel shows color-coded JWT cards for each token (Tc, T1, TR) with decoded claims.

Troubleshoot common issues

If something isn't working as expected, check the following table for common issues and fixes:

Expand table

Symptom	Likely cause	Fix
`/api/status` shows `bedrock_available: false`	AWS credentials missing or expired, or model access not granted.	Check `docker logs llm-agent-aws`. Refresh STS credentials with `aws sso login`. Enable the model in the Bedrock console.
`ExpiredTokenException` from Bedrock	STS session token (Tier A) expired.	Paste fresh credentials in `.env`, then run `docker compose up -d --force-recreate llm-agent-aws`.
`AccessDeniedException` on `InvokeModel`	IAM principal lacks `bedrock:InvokeModel` permission, or model access isn't enabled.	Grant `bedrock:InvokeModel` on the model and inference profile ARNs.
`ValidationException: invalid model identifier`	Region doesn't host the model, or you used a bare model ID instead of the `us.` inference profile.	Use the `us.`-prefixed inference profile ID (for example, `us.anthropic.claude-3-haiku-20240307-v1:0`).
Weather API returns `401 Unauthorized`	Token tenant mismatch, expired secret, or signature check failed.	Verify `TENANT_ID` matches the Blueprint's tenant. Check sidecar logs.
LLM responds without calling the tool	Query wasn't clearly tool-shaped, or model doesn't support tool calling.	Use Claude 3 Haiku or newer. Phrase the request as "*What's the weather in <city>?*".

Symptom	Likely cause	Fix
OBO sign-in popup blocked	Browser popup blocker.	Allow popups for `localhost:3001`.
`4xx` from sidecar during OBO	`CLIENT_SPA_APP_ID` missing or SPA redirect URI mismatch.	Rerun `setup-obo-client-app`. Ensure `http://localhost:3001` is on the SPA's redirect URIs.

If an issue persists even after following the troubleshooting steps, inspect the container logs directly. Each service logs to its own container:

Bash

```
docker logs llm-agent-aws          # Agent app: Bedrock calls, tool invocations
docker logs agent-id-sidecar-aws   # Sidecar: token acquisition, credential errors
docker logs weather-api-aws        # Weather API: JWT validation, request handling
```

Clean up resources

When you finish testing, stop the local containers to free system resources. Choose one of the following cleanup options based on whether you want to keep images for faster restarts.

> **ⓘ Important**
>
> `docker compose down` removes only the local Docker containers. Microsoft Entra objects (agent blueprint, Agent ID, SPA app registration) are tenant-side state and persist. Delete them manually in the Microsoft Entra admin center if you no longer need them.

Bash

```
# Stop containers but keep volumes and images for faster restarts
docker compose down

# Remove everything including volumes and images
docker compose down -v --rmi all
```

Related content

- Authentication with Microsoft Entra ID Auth SDK (sidecar)
- Integrate third-party agents with Microsoft Entra Agent ID
- Microsoft Entra ID Auth SDK (sidecar) container image

Last updated on 08/14/2026

Secure an n8n agent with Microsoft Entra Agent ID

This guide shows how to deploy n8n on Azure Container Apps with Microsoft Entra Agent ID integration. The deployment uses the Azure Developer CLI (`azd`) to provision infrastructure, create Microsoft Entra identity objects, and configure n8n workflows automatically.

Unlike the Authentication with Microsoft Entra Auth SDK sidecar pattern used for custom agents, the n8n integration uses the n8n-nodes-entraagentid community node to manage token acquisition directly within n8n workflows. The deployed workflows demonstrate both autonomous (app-only) and on-behalf-of (OBO) token flows, with access to Microsoft Graph and the Microsoft Graph MCP Server for Enterprise, `https://mcp.svc.cloud.microsoft/enterprise`.

> **Note**
>
> This sample demonstrates the use of the `n8n-nodes-entraagentid` community node within n8n. It isn't guidance for deploying n8n on Azure in production.

Prerequisites

Before you begin, make sure you have:

- An Azure subscription with quota for Azure OpenAI (GPT-4o or similar), PostgreSQL Flexible Server, and Azure Container Apps.
- **Global Administrator** role in your Microsoft Entra tenant. This role is required because the automation creates multiple Microsoft Entra objects and grants admin consent to permissions. Use Privileged Identity Management (PIM) to activate this role just-in-time.

Azure Cloud Shell (recommended) comes with everything preinstalled: Azure CLI, Azure Developer CLI (`azd`), PowerShell 7, and Git.

If you're running locally instead of Cloud Shell, install these tools before proceeding:

- Azure Developer CLI (azd) v1.9 or later.
- Azure CLI (az) v2.60 or later.
- PowerShell 7.4+.
- Microsoft.Entra PowerShell module v1.2 or later.

- Git.

Authenticate both the Azure CLI and Azure Developer CLI so that the deployment commands can create and manage resources in your subscription:

```
Bash

az login
azd auth login
```

Clone and deploy

The entire deployment runs through a single `azd up` command that provisions Azure infrastructure and configures n8n automatically. Follow these steps to deploy n8n:

1. Open Azure Cloud Shell and select **PowerShell**.

2. Clone the repository and start the deployment:

   ```
   Bash

   git clone https://github.com/astaykov/n8n-aca.git && cd n8n-aca && azd auth login
   && azd up
   ```

   In Azure Cloud Shell, `azd auth login` displays a device code. Open the URL shown and enter the code to authenticate, then `azd up` continues automatically.

3. When prompted, provide the following values:

   - **Environment name:** Any name (for example, `my-n8n`). Used to isolate this deployment.
   - **Azure subscription:** Select the subscription to deploy into.
   - **Azure location:** Select a region (for example, `northeurope`).
   - **n8n admin email:** Email for the n8n owner account.
   - **n8n admin password:** Password for the n8n owner account (minimum 8 characters, mixed case, number).

4. During the postprovision phase, the automation performs a second sign-in. A device code is displayed. Open the URL and enter the code. This step requires the Global Administrator or Application Administrator role. The `azd up` postprovision hook then:

   - Creates Microsoft Entra Agent ID objects (Blueprint, Agent Identity, Agent User).
   - Enables the Microsoft Graph MCP Server for Enterprise.

- Waits for n8n to become ready.
- Creates the owner account.
- Installs the `@astaykov/n8n-nodes-entraagentid` community node.
- Generates an API key for n8n automation.
- Creates all five credentials with real values.
- Imports and activates the three demo workflows.

When the deployment completes, the script prints your n8n URL and a summary of what was configured. The tenant ID is autodetected from your Azure sign-in, so no manual configuration is needed.

Explore the deployed resources

The deployment creates Azure resources, Microsoft Entra identity objects, and n8n configuration assets that work together to support the sample workflows.

Review the Azure infrastructure resources

The deployment creates the following Azure resources:

- **Container Apps Environment:** Hosts n8n and the test SPA.
- **n8n Container App:** Runs the official `n8nio/n8n` image with HTTPS ingress.
- **Static Web App:** Test SPA for the OBO webhook flow.
- **PostgreSQL Flexible Server:** Persistent store for workflows, credentials, and execution history (Burstable B1ms).
- **Storage Account and File Share:** Persistent `/home/node/.n8n` directory. Community nodes and configuration survive restarts.
- **Azure OpenAI:** GPT model deployment used by the AI agent workflows.
- **Log Analytics Workspace:** Diagnostics and monitoring.

Review the Microsoft Entra identity objects

The automation creates these objects once and reuses them on subsequent runs:

- **Agent identity blueprint:** App registration that issues tokens on behalf of Agent Identities via Federated Identity Credentials.
- **Agent identity service principal:** The AI agent's service principal. Acquires Microsoft Graph and MCP tokens autonomously.
- **Agent user account:** A cloud-only user identity that enables delegated (OBO) token flows.

- **Single page app (SPA) app registration:** Client app for the webhook demo, preconfigured with redirect URIs and Blueprint API permissions.

Review the n8n credentials and workflows

The postprovision hook automatically configures n8n:

Credentials created:

- **EntraAgentID - Autonomous:** App-only Microsoft Graph API token (no user context).
- **EntraAgentID - Agent User OBO:** Delegated token on behalf of the Agent User.
- **Azure OpenAI:** Connection to the deployed GPT model for AI agent workflows.
- **AgentID Auth Manager - Access Token:** Token forwarding from the Auth Manager to downstream nodes.
- **Bearer from AuthManager:** Bearer token forwarding for MCP calls.

Workflows imported:

- **Agent ID Auth Manager - Agent User with MCP Enterprise:** Acquires a delegated MCP token for the Agent User and forwards it to a subworkflow.
- **HTTP Request with autonomous agent token:** Demonstrates an autonomous agent calling Microsoft Graph directly with an app-only token.
- **Webhook - assistive agent (on-behalf-of):** Webhook entry point that receives a bearer token from the SPA, calls the Auth Manager, and responds via the Graph MCP Server on behalf of the signed-in user.

Understand token flow

The n8n deployment supports two token flow patterns:

- **Autonomous (app-only):** The n8n workflow uses the Agent Identity Blueprint credentials with Federated Identity Credentials to acquire an app-only token for the Agent Identity service principal. The workflow then calls Microsoft Graph directly with this token. No user context is involved.

- **On-behalf-of (OBO) with MCP:** A browser-based SPA sends a bearer token to an n8n webhook. The webhook calls the Auth Manager workflow, which uses the Blueprint credentials to acquire a delegated token on behalf of the Agent User. The Auth Manager forwards the token to a subworkflow that calls the Microsoft Graph MCP Server for Enterprise, which translates MCP tool calls into Microsoft Graph API requests using the delegated token.

In both patterns, the Agent Identity Blueprint acts as a token factory. The Agent Identity Blueprint issues tokens for Agent Identities without storing credentials on the agent itself. The Auth Manager community node handles token acquisition and AES-256-GCM caching within each workflow run.

Deploy the test SPA (optional)

The test SPA is a static JavaScript app that demonstrates the OBO webhook flow from a browser.

1. Deploy it after the initial provisioning completes:

Bash

```
azd deploy spa
```

Understand the MCP Server scopes

The setup grants the following delegated `MCP.*` scopes to the Agent Identity service principal. These scopes mirror their Microsoft Graph counterparts (for example, `MCP.User.Read.All` corresponds to `User.Read.All`):

- `MCP.User.Read.All`: Read all users.
- `MCP.Organization.Read.All`: Read tenant organization info.
- `MCP.Group.Read.All`: Read all groups.
- `MCP.GroupMember.Read.All`: Read group memberships.
- `MCP.Application.Read.All`: Read app registrations and service principals.
- `MCP.AuditLog.Read.All`: Read sign-in and audit logs.
- `MCP.Reports.Read.All`: Read Microsoft 365 usage reports.
- `MCP.Policy.Read.All`: Read conditional access policies.
- `MCP.Domain.Read.All`: Read verified domains.
- `MCP.Device.Read.All`: Read Microsoft Entra-registered devices.

To add more scopes, edit the `$MCP_SCOPES` array in `scripts/Setup-EntraAgentId.ps1` and rerun `azd provision`.

> ⓘ **Note**
>
> The MCP Server only supports delegated permission flows. Use the autonomous credential for app-only Microsoft Graph calls.

Rerun and update the deployment

The deployment is fully idempotent:

- Bicep skips Azure resources that already exist.
- The `azd` environment saves Microsoft Entra object IDs (Blueprint, Agent Identity, Agent User, Blueprint secret) after the first run and reuses them on subsequent runs.
- n8n configuration (credentials, workflows) is applied fresh each run, which allows repairing a broken state.

To rerun just the postprovision scripts without modifying infrastructure, run provisioning again. The Bicep templates detect no infrastructure changes and only execute the deployment hooks:

Bash

```
azd provision   # Bicep detects no changes, runs hooks only
```

Run the scripts manually (optional)

You can run the configuration scripts independently if needed:

- **Full end-to-end (Microsoft Entra and n8n):**

  PowerShell

  ```
  .\scripts\Run-All.ps1 `
      -TenantId  "<your-tenant-id>" `
      -N8nUrl    "https://ca-n8n-<token>.<region>.azurecontainerapps.io"
  ```

- **n8n configuration only (skip Entra setup):**

  PowerShell

  ```
  .\scripts\Configure-N8n.ps1 `
      -N8nUrl          "https://ca-n8n-<token>.<region>.azurecontainerapps.io" `
      -OwnerEmail      "admin@contoso.com" `
      -OwnerPassword   "MyStr0ngPassword!"
  ```

- **Entra setup only:**

  PowerShell

  ```
  .\scripts\Setup-EntraAgentId.ps1 `
      -TenantId  "<your-tenant-id>" `
  ```

```
    -N8nUrl    "https://ca-n8n-<token>.<region>.azurecontainerapps.io"
```

Clean up resources

Remove all Azure resources that the deployment created and purge any retained deployment state:

Bash

```
azd down --purge
```

> **Note**
>
> The `azd down` command removes Azure resources but doesn't delete Microsoft Entra objects such as blueprints, agent identities, or agent user accounts. Remove these objects manually in the Microsoft Entra admin center if they're no longer needed.

Related content

- Integrate third-party agents with Microsoft Entra Agent ID
- Authentication with Microsoft Entra Auth SDK (sidecar)
- n8n-nodes-entraagentid community node
- Azure Developer CLI overview

Note: The author created this article with assistance from AI. Learn more

---

Last updated on 06/15/2026

Manage assignment of agent identities to an application (Preview)

In this article, you learn how to assign agent identities to an enterprise application in Microsoft Entra ID for agents that plan to interact with an application. The Microsoft Entra configuration steps depend on the capabilities of the agent and the enterprise application.

Organizations can control which of their employee and business guest users in their Microsoft Entra tenant can interact with the organization's enterprise applications, by configuring each application to rely upon Microsoft Entra for single-sign on or provisioning, and requiring those users to be assigned to the application. If an application exposes multiple app roles, you can also assign a specific app role to each user. For more information, see Manage users and groups assignment to an application.

When an AI agent has a tool (or might be referred to in its platform as a skill or connector) to communicate with an enterprise application in your tenant, you can assign that tool's identity to the application. The tool might use a service principal, agent identity, or an agent user for authentication. By assigning these identities to the enterprise application's roles and permissions, you allow agent-supporting applications to recognize the agent identity and enable the agent to obtain appropriate tokens from Microsoft Entra ID for application access.

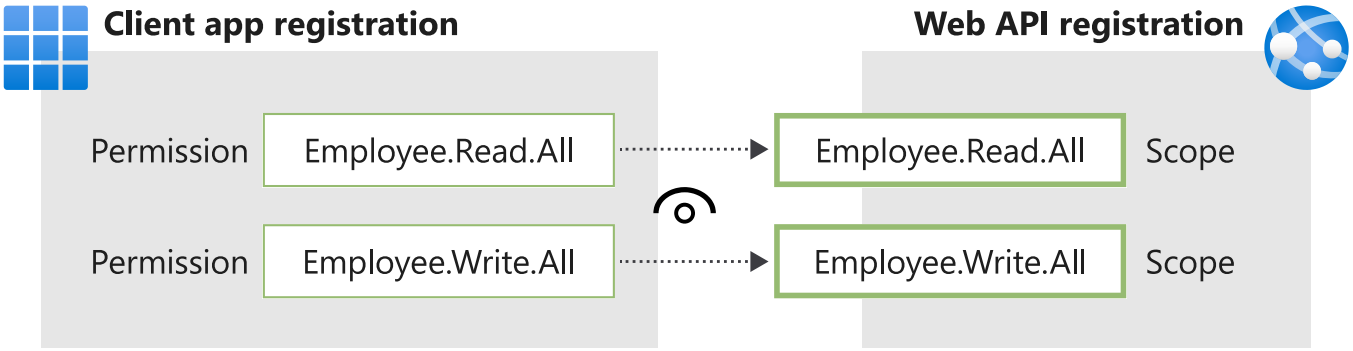
This article outlines how to select across three options, based on the application's capabilities.

Expand table

Application capabilities	Section
Application exposes APIs with OAuth2 permission scopes	consent the agent identity or service principal to an application permission scope
Application recognizes role claims for service principals	assign an agent identity or service principal to an application role
Application only supports SAML for users	assign an agent user to an application role for use with SCIM

Consenting to an application permission scope

Applications using the Microsoft Entra identity platform can expose APIs for other client applications to call. The application with the API can expose OAuth scopes for those API calls. The tool's service principal can be consented permission to those scopes, allowing it to call the APIs.



For more information on consenting an agent identity or service principal, see admin consent for application permissions.

Assigning an agent identity to an application role

For applications that recognize role claims on service principals, you can assign an agent's service principal or agent identity to an application's app role. When you assign to app roles, this results in application permissions. Application permissions are typically used by daemon apps, back-end services or autonomous agents, that need to authenticate and make authorized API call as themselves, without the interaction of a user. For more information, see Add app roles to your application and receive them in the token.

> **Note**
>
> Microsoft Entra SCIM outbound provisioning supports users and groups only; Service principals and agent identities assigned to app roles directly or as members of groups aren't provisioned to applications.

First, confirm that app role in the application's manifest has an `allowedMemberTypes` of `Application`, indicating that the role is available for other applications. If the role only allows users, then agent identities and service principals aren't supported by the application for that role. Then, you can assign an app role to an agent identity or service principal, by using the Microsoft Entra admin center, Microsoft Graph or Microsoft Graph PowerShell.

- If you're using the Microsoft Entra admin center, follow the instructions in how to assign to application app roles.
- If you're using Microsoft Graph or Microsoft Graph PowerShell, then assigning an agent identity or service principal to an application role is similar to assigning users and groups to an application. You can use list app role assignment to check if the agent identity or service principal already has a role assignment, and use add app role assignment to create a new app role assignment. For example, to create an application role assignment for an agent identity using PowerShell, set `$PrincipalId` to the `id` of the agent identity,

`$ResourceId` to the `id` of the target application service principal, and `$AppRoleId` to the ID of the app role of the target application app role. Then, construct a payload for adding an application role assignment:

PowerShell

```
$Body = @{
  principalId = $PrincipalId
  resourceId = $ResourceId
  appRoleId = $AppRoleId
} | ConvertTo-Json

Invoke-MgGraphRequest -Method POST -Uri
"https://graph.microsoft.com/v1.0/servicePrincipals/$ResourceId/appRoleAssigne
dTo" -Body $Body
```

Assigning to SAML-based applications

If you have applications that only support SAML for a user to obtain a token, and you want your agent to be able to autonomously interact with its agent identity with that application, then you can augment the agent's identity with agent users. Then, once you add support for SAML and agent users to your agent, your agent can obtain a token for the application. Unlike agent identities, agent users can optionally also be provisioned into those applications via SCIM, if required by the application.

> **Note**
>
> Confirm with the application developer whether the application supports the agent interaction pattern.

Enabling an agent to interact with an application that accepts SAML assertions requires there to be an extra application registration to participate in the flow for Microsoft Entra ID to provide a SAML assertion in response to an on-behalf-flow. In addition to the agent itself and the application, you need to create a SAML helper application registration for any SAML-based applications in the tenant. If you're developing the agent in-house, this helper could be part of the agent.

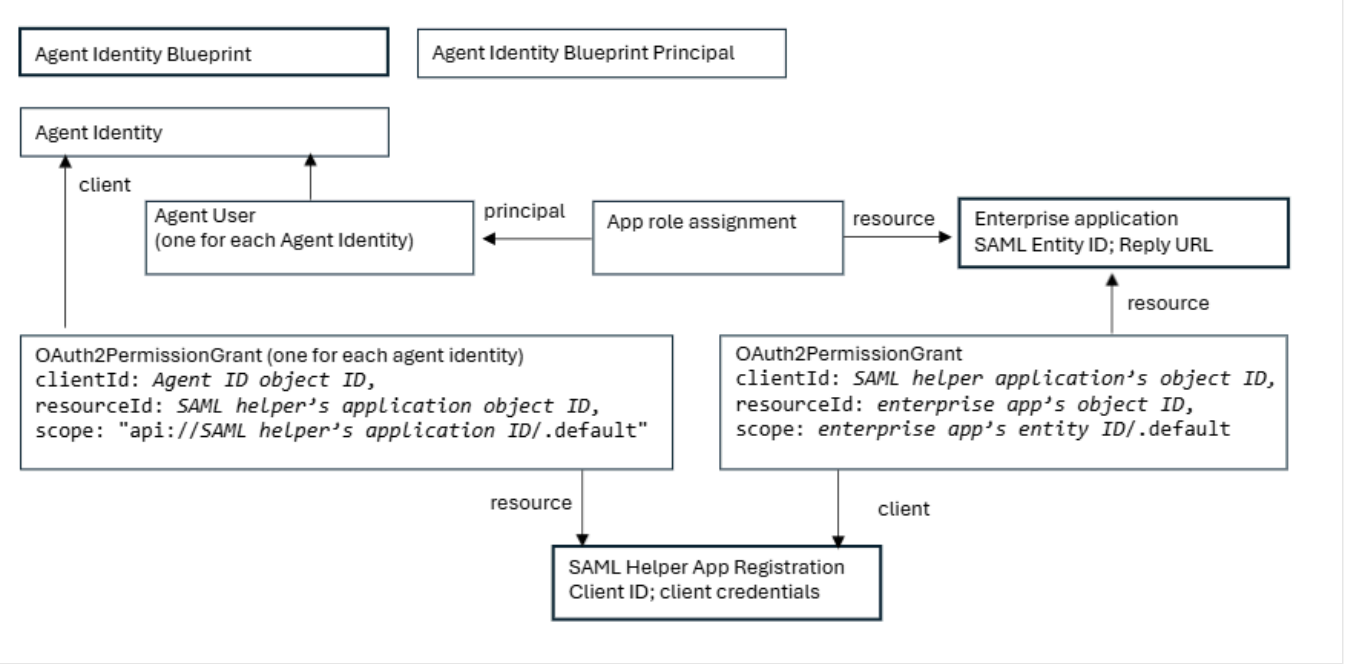
In the Microsoft Entra tenant, the following artifacts need to exist:

- the agent identity blueprint and agent identity blueprint principal
- the SAML helper application registration
- the enterprise application as the target resource

- an `OAuth2PermissionGrant`, with
  - the SAML helper application registration as the client
  - the enterprise application as the resource
  - the scope value of the concatenation of the enterprise application's entity ID with `/.default` appended

Then, for each agent identity of the agent, you create:

- an agent user
- an application role assignment, for the agent user, to one of the roles of the enterprise application
- an `OAuth2PermissionGrant`, for the agent identity, with
  - the agent identity as the client
  - the SAML helper application registration as the resource
  - the scope value of the concatenation of `api://'`, the SAML helper application's application ID, and `/.default`



If the agent has multiple agent identities, then permission inheritance can be used to grant consent once at the agent identity blueprint and inherit it for the agent identities.

Once these applications, users, role assignments, and grants are in place in the tenant, then an agent that needs a SAML assertion for authenticating to the enterprise application can:

- Get a token as the agent identity blueprint.
- Use that token to make a token request to `https://login.microsoftonline.com/<tenantid>/oauth2/v2.0/token` endpoint, and get a federated identity credential (FIC) token as the agent identity.

In that request, the `client_id` is the agent identity ID, the `scope` is `api://AzureADTokenExchange/.default`, the `grant_type` is `client_credentials`, the `client_assertion_type` is `urn:ietf:params:oauth:client-assertion-type:jwt-bearer`, and the `client_assertion` is the agent identity blueprint token from step 1.

- Use those two tokens to make a token request for a token as the agent user with the scope of the helper application.

  In this request, the `client_id` is the agent identity ID, the `scope` is the concatenation of `api://'`, the SAML helper application's application ID, and `/.default`, the `grant_type` is `user_fic`, the `client_assertion_type` is `urn:ietf:params:oauth:client-assertion-type:jwt-bearer`, the `client_assertion` is the agent identity blueprint token, the `user_id` is the agent user object ID, and the `user_federated_identity_credential` is the agent identity token.

- Make the on-behalf-of-call token request to `https://login.microsoftonline.com/<tenantid>/oauth2/v2.0/token` to obtain a SAML token.

  In this request, provide the credentials of the SAML helper application registration, the token returned by `user_fic` as the assertion, the grant type `urn:ietf:params:oauth:grant-type:jwt-bearer`, scope of the enterprise application's app ID with `/.default` appended, the `requested_token_use` of `on_behalf_of` and the `requested_token_type` of `urn:ietf:params:oauth:token-type:saml2`. The response contains a SAML assertion encoded in Base64URL.

Related content

- Assign users and groups to application roles

Last updated on 11/20/2025

Agent identity sponsor tasks in Lifecycle Workflows

Governing agent identities sponsors is a critical aspect of maintaining lifecycle governance and access control in your organization. Agent identity sponsors are responsible for overseeing the lifecycle and access decisions of agent identities. Keeping sponsor information up to date helps with effective governance and compliance. For an overview of agent identity governance including access packages and sponsor responsibilities, see Governing Agent Identities.

Lifecycle Workflows currently contain the following tasks that involve the governing of sponsors of agent identities:

- Send email to manager about sponsorship changes
- Send email to cosponsors about sponsor changes
- Transfer agent identity sponsorships to manager

These tasks ensure continuity of sponsorship when an agent's sponsor changes roles or leaves the organization. All three tasks are classified as **mover and leaver** tasks and are available only under mover or leaver workflow templates.

This article explains how to configure Lifecycle Workflows to streamline agent identity sponsor governance.

License Requirements

Using Microsoft Entra ID Governance for agent identities requires one of the following license plans:

- **Microsoft 365 E7**, which includes Agent 365 and Microsoft Entra Suite, to provide governance of user and agent identities.
- **Microsoft Agent 365** license paired with at least Microsoft Entra P1 or Microsoft 365 E3.

For more information, see Microsoft Agent 365 plans and pricing. For the full list of agent-specific capabilities, refer to the **Microsoft Agent 365** column in the Microsoft Entra ID Governance licensing table.

Create a sponsor workflow using the Microsoft Entra Admin Center

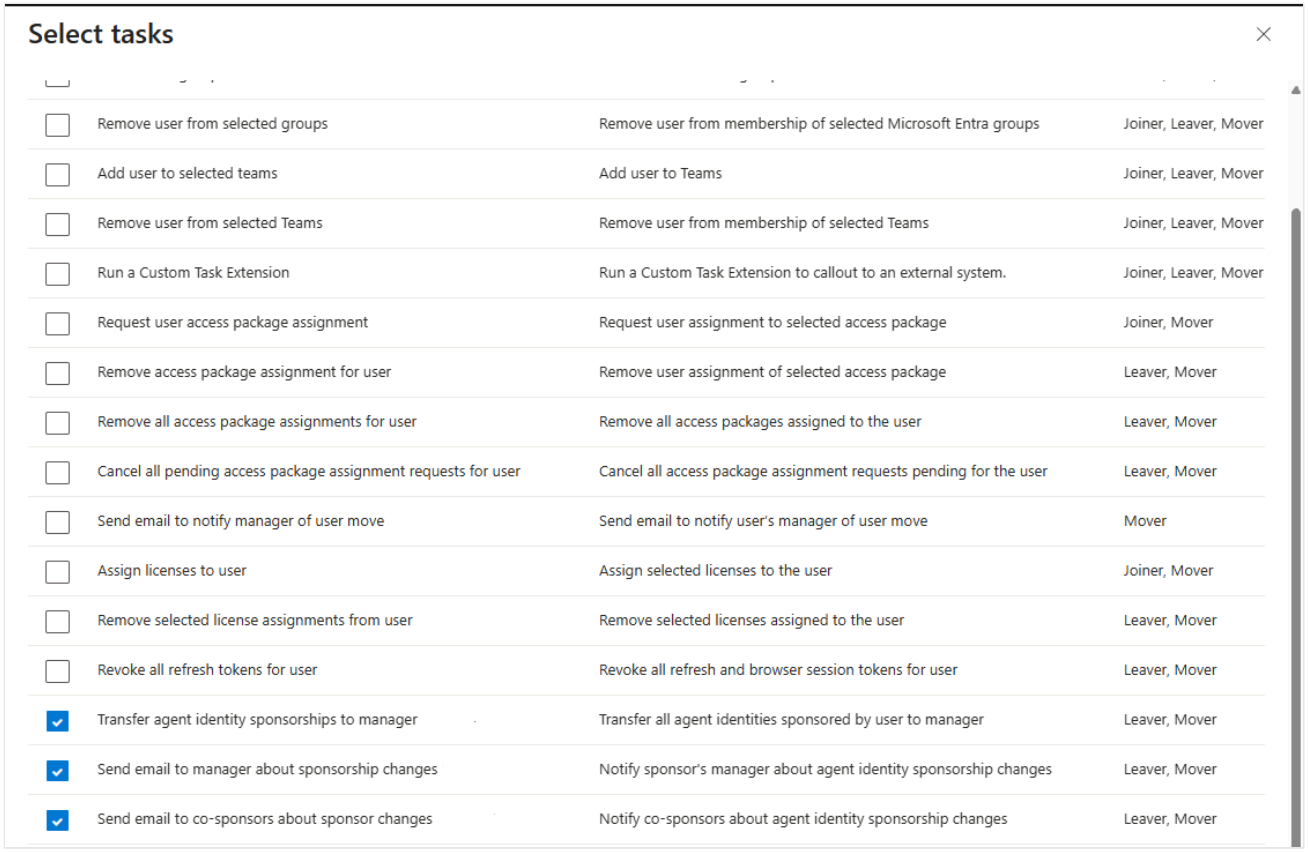
To create a workflow that notifies the manager or cosponsors of an existing agent identity sponsor's move, follow these steps:

1. Sign in to the Microsoft Entra admin center as at least a Lifecycle Workflows Administrator.
2. Browse to **ID Governance** > **Lifecycle workflows** > **workflows**.
3. On the workflow screen, select the specific mover or leaver workflow template you want to add the sponsorship email tasks to, or create a new workflow based on a template.

   > **Note**
   >
   > The **Send email to manager about sponsorship changes**, **Send email to co-sponsors about sponsor changes**, and **Transfer agent identity sponsorships to manager** are mover and leaver tasks, and are only available as selectable tasks under workflow templates of the same category.

4. On the **Basics** tab, after entering a unique display name and description for the workflow, select your trigger and select **Next**.
5. On the **Configure scope** screen, select the scope of the workflow and select **Next**.

6. On the **Tasks** page, select which sponsor related tasks you want to include and select **Next**.



7. Review the created workflow, and then select **Create**.

Related content

- Manage agent identities in your organization - See how sponsor governance fits into overall agent management.
- Governing Agent Identities - Overview of agent identity governance including access packages and sponsor responsibilities.
- Write concepts
- Manage workflow properties

Last updated on 06/24/2026

Manage agent identities in your organization

Microsoft Entra Agent ID provides a centralized set of tools for managing agent identities across your organization. Agent identities are a distinct identity type in Microsoft Entra ID, designed for AI agents with classification, metadata, and security controls tailored to agentic workloads.

This article covers key agent management tasks: from viewing and disabling agents, to governing access, monitoring activity, and responding to security risks. Whether you're an administrator responsible for tenant-wide agent oversight or a sponsor managing specific agents, this guide provides the information you need to effectively manage agent identities in your organization.

Prerequisites

Different management tasks require different roles and licenses. The following table summarizes the roles you need for each area of agent identity management.

Expand table

Task	Required role	Notes
View agent identities	Microsoft Entra user account	No admin role needed for viewing.
Manage agent identities	Agent ID Administrator or Cloud Application Administrator	Agent identity owners can manage their own agents without these roles.
Create agent blueprints	Agent ID Developer	User is added as owner of the blueprint and its service principal.
Configure Conditional Access policies	Conditional Access Administrator	Requires Microsoft Entra ID P1 license.
View ID Protection risk reports	Security Administrator, Security Operator, or Security Reader	Requires Microsoft Entra ID P2 license during preview.
Configure Lifecycle Workflows	Lifecycle Workflows Administrator	

View agent identities

The Microsoft Entra admin center provides a centralized interface to view all agent identities in your tenant. You can search, filter, sort, and customize columns to find specific agents.

1. Sign in to the Microsoft Entra admin center.
2. Browse to **Entra ID** > **Agents** > **Agent identities**.
3. Select any agent identity to view its details, including name, description, status, owners, sponsors, granted permissions, and sign-in logs.

To search for a specific agent, enter the **name** or **object ID** in the search box, or add the **Blueprint App ID** filter. You can customize which columns are shown by selecting the **Choose columns** button. Available columns include **Name**, **Created On**, **Status**, **Object ID**, **View Access**, **Blueprint App ID**, **Owners and Sponsors**, and **Uses agent identity**.

For detailed instructions on filtering, column customization, and viewing agents from this view, see View and filter agent identities in your tenant.

Manage agent identity blueprints

Agent identity blueprints are the parent definitions from which individual agent identities are created. The admin center lets you view all blueprint principals, manage their permissions, and monitor their activity.

1. Sign in to the Microsoft Entra admin center.
2. Browse to **Entra ID** > **Agents** > **Agent blueprints**.
3. Select any agent identity blueprint principal to manage it.

From a blueprint's management page, you can:

- **View linked agent identities**: See all child agent identities created from this blueprint.
- **Manage blueprint access**: View, manage, and revoke permissions assigned to the blueprint.
- **Manage owners and sponsors**: Owners handle technical administration, while sponsors are accountable for the agent's purpose and lifecycle decisions.
- **View audit logs**: Track administrative changes for security and compliance.
- **View sign-in logs**: Monitor authentication events.
- **Disable the blueprint**: Select **Disable** in the command bar.

For detailed instructions, see View and manage agent identity blueprints in your tenant.

Configure inheritable permissions for blueprints

Inheritable permissions let agent identities automatically inherit OAuth 2.0 delegated permission scopes from their parent blueprint. When you configure inheritable permissions on a blueprint, newly created agent identities receive a base set of scopes without requiring interactive user or admin consent prompts.

Two inheritance patterns are supported per resource app:

Expand table

Pattern	Description
Enumerated scopes	Inherit only explicitly listed scopes. Use for fine-grained control.
All allowed scopes	Inherit all available delegated scopes for the resource app. Newly granted scopes on the blueprint are automatically included.

Start with enumerated scopes using only essential permissions, then expand as needed. This approach follows the principle of least privilege and makes it easier to audit which permissions agents actually use.

Key limits:

- Maximum of 10 resource apps per blueprint.
- For enumerated scopes, maximum of 40 scopes per resource app.
- Some high-privilege scopes are blocked by platform policy and can't be inherited.

Inheritable permissions are configured via the `inheritablePermissions` navigation property on the `agentIdentityBlueprint` application resource using Microsoft Graph. For step-by-step API examples (add, update, delete), see Configure inheritable permissions for agent identity blueprints.

Control agent access to resources

Conditional Access policies provide tenant-wide controls for agent identity authentication. You can use these policies to block all agent identities, allow only specific agents, or block risky agents based on ID Protection signals.

Key points about Conditional Access for agent identities:

- Policies can scope to **All agent identities** or **All agent users** with a **Block** grant control.
- Policies can target **All resources** to prevent agent access across your organization.

- **Agent risk conditions** (high, medium, low) allow you to block agents based on risk signals from ID Protection.
- Policies support **Report-only mode** for safe evaluation before enforcement.

> **Important**
>
> Conditional Access enforcement applies when an agent identity or agent's user account requests a token for any resource. It does **not** apply when an agent identity blueprint acquires a token to create agent identities or agent's user accounts.

For detailed policy configuration, step-by-step walkthroughs, and business scenario examples, see Conditional Access for Agent ID.

Monitor agent activity

Sign-in and audit logs

Agent identity activities are captured in Microsoft Entra audit and sign-in logs:

- **Audit logs** record agent-related events under the base identity type from which they originate. For example, creating an agent identity user appears as a "Create user" audit activity, and creating an agent identity appears as a "Create service principal."
- **Sign-in logs** include the `agentSignIn` resource type, which provides properties about the agent and its sign-in behavior.

To view agent sign-in logs:

1. Sign in to the Microsoft Entra admin center as at least a Reports Reader.
2. Browse to **Entra ID** > **Monitoring & health** > **Sign-in logs**.
3. Use the following filters:
    - **Agent type**: Choose from **Agent ID user**, **Agent Identity**, **Agent Identity Blueprint**, or **Not Agentic**.
    - **Is Agent**: Choose from **No** or **Yes**.

You can also retrieve agent sign-in events using Microsoft Graph:

HTTP

```
GET https://graph.microsoft.com/beta/auditLogs/signIns?$filter=signInEventTypes/any(t:
```

```
t eq 'servicePrincipal') and agent/agentType eq 'AgentIdentity'
```

For full details on log types and access methods, see Microsoft Entra Agent ID logs.

Detect and remediate agent risk

Identity Protection for agents

Microsoft Entra ID Protection monitors agent identities for anomalous behavior. It detects six offline risk types, including unfamiliar resource access, sign-in spikes, and failed access attempts. Administrators can review the **Risky Agents report** and take response actions: confirm compromise, confirm safe, dismiss risk, or disable the agent.

> **Note**
>
> ID Protection for agents requires a Microsoft Entra ID P2 license during preview.

When you confirm an agent as compromised, the risk level is set to **High**. If you have a Conditional Access policy configured to block on High Agent Risk, the agent is automatically blocked from accessing resources.

For the full risk detection table, response actions, Graph API details, and report walkthrough, see Identity Protection for agents.

Respond to agent security incidents

When agent activity triggers a risk detection or security concern, follow this sequence:

1. **Detect**: Review the **Risky Agents report** in the Microsoft Entra admin center. Risk detections are viewable for up to 90 days. Details include the agent's display name, risk state, risk level, agent type, and sponsors.
2. **Respond**: Take an immediate action:
    - **Confirm compromise**: Sets the risk level to High and triggers risk-based Conditional Access policies configured to block on High Agent Risk.
    - **Disable**: Prevents all sign-ins across Microsoft Entra ID and connected apps.
3. **Investigate**: Review risk detection details, sign-in logs, and audit logs to understand the scope and impact.
4. **Recover**: Based on your investigation:

- If false positive: dismiss the risk and re-enable the agent.
- If true compromise: rotate credentials before re-enabling, or retire the agent identity.

For detailed information on risk types, detection mechanisms, and response actions, see Identity Protection for agents.

Govern agent identities and sponsor oversight

Sponsor responsibilities

Each agent identity should have an accountable human sponsor responsible for lifecycle and access decisions. Key governance behaviors include:

- **Automatic sponsor transfer**: If a sponsor leaves the organization, Microsoft Entra ID automatically reassigns sponsorship to the sponsor's manager.
- **Expiration notifications**: Sponsors receive notifications when access package assignments approach expiration. Sponsors can request an extension (which triggers a new approval cycle) or let assignments expire.
- **Access request pathways**: Access packages can be requested through three pathways: the agent's own programmatic request, a sponsor on behalf of the agent, or an admin direct assignment.

Access packages can grant Security Group memberships, Application OAuth API permissions (including Microsoft Graph application permissions), and Microsoft Entra roles.

For the full governance overview including access package configuration and sponsor policies, see Governing Agent Identities.

Automate sponsor notifications with Lifecycle Workflows

Lifecycle Workflows provides two automated tasks for agent identity sponsorship:

- Send email to manager about sponsorship changes
- Send email to cosponsors about sponsor changes

Both tasks are **mover and leaver** category tasks: they trigger only under mover or leaver workflow templates, not joiner templates. This ensures continuity of sponsorship when an agent's sponsor changes roles or leaves the organization.

For step-by-step workflow configuration, see Agent identity sponsor tasks in Lifecycle Workflows.

Automate agent management at scale

For organizations managing large numbers of agent identities, the following options are available:

- **Multi-select disable**: The admin center supports selecting multiple agent identities at once and disabling them in batch from the **All agent identities** page.
- **Microsoft Graph API**: Agent identity endpoints support programmatic management. For example, ID Protection exposes `riskyAgents` and `agentRiskDetections` collections for programmatic risk monitoring.

Disable or restrict agent identities

Organizations can control agent identity usage at three levels, depending on the scope needed:

Expand table

Scope	What it does	Details
Individual agent	Disable a specific agent identity to block its access and token issuance. Administrators use the admin center; owners and sponsors use the My Account portal.	View and filter agent identities in your tenant · Manage agents in end user experience
Blueprint-level	Disable an agent identity blueprint from its management page. This prevents new agent identities from being created from that blueprint and blocks existing ones.	View and manage agent identity blueprints in your tenant
Tenant-wide	Block all agent identity authentication using Conditional Access policies, and optionally block creation of new agent identities through product-specific controls (Microsoft Entra ID, Security Copilot, Copilot Studio, Azure AI Foundry, Microsoft Teams).	Disable agent identities in your tenant

Re-enabling a disabled agent identity at any scope restores access and token issuance.

> ⊗ **Caution**
>
> Globally disabling agent identities can cause existing agents to fail, degrade Microsoft product experiences, and push teams to use less transparent application or service principal identities. Evaluate the impact before enforcing. For a partial approach, use Conditional Access policies to block specific agents rather than all agent identities.

Related content

- Best practices for Microsoft Entra Agent ID
- Manage agents in end user experience

Last updated on 05/01/2026

Disable agent identities in your tenant

As an IT administrator overseeing agent identities in your tenant, you might need to temporarily stop agent activity to investigate an issue or review agent usage. You can disable agent identities and agent identity blueprints in your tenant depending on your need. Both actions have implications that you should understand before proceeding.

You can also configure Conditional Access policies and remove agent creation permissions, if your organization needs to restrict agent identities from being created or used in your tenant. These processes can serve as a different approach to disabling agent identities. This article covers scenarios related to disabling agent identities.

Types of identities used by AI agents

Your Microsoft Entra tenant might contain AI agents with or without a Microsoft Entra agent identity.

- **Agents with agent identities**: Agents created in Microsoft Entra Agent ID or using the latest iterations of systems like Microsoft Copilot Studio, Azure AI Foundry, and Security Copilot are created with agent identities. Agent identities have clear classification, richer metadata, and features designed to address the unique security challenges of AI agents.
- **Agents without agent identities**: Agents created in earlier versions of Copilot Studio and Azure AI Foundry might have been created as classic applications / service principals in your tenant. These applications / service principals might have tag values that denote them as AI agents, but they don't have Microsoft Entra agent identities. They're subject to the same policies, governance, and processes as all other applications / service principals in your tenant.

Disabling agent identity considerations

Disabling an agent identity in your tenant might have broader consequences than simply stopping new AI agents from being created or used. Before you proceed, evaluate the following considerations:

- Existing agents running in your organization might begin to fail.
- Microsoft product experiences that assume agent identity availability (for example, Copilot Studio agents, Security Copilot scenario agents, Microsoft Entra Conditional Access

Optimization Agent) might fail or degrade to less transparent application or service principal patterns.

- Support or troubleshooting: Helpdesk and SOC teams might receive increased tickets when features fail quietly due to missing agent identities.
- Blocking agent identities might push teams to build agents using generic application or service principal identities, reducing visibility and making it harder to distinguish agents from other software projects.

Monitor for agent identity creation and activity

Before disabling an agent identity or agent identity blueprint, you should be aware of what kind of activity is associated with those identities. Agent identity activities are logged under the base identity types they originate from. For example, creating an agent identity shows up as *add service principal" and adding an agent identity blueprint shows up as *add application* in the audit logs.

To identify whether an audit event involves an agent identity, check the `agentType` property on the `initiatedBy` and `targetResources` fields. A value other than `notAgentic` indicates agent involvement.

The `agentSignIn` resource type provides descriptive information that identifies and classifies sign-in events as agentic. With this value you can determine when an agent identity was the subtype of the identity involved in an authentication event.

For more information, see agent sign-in logs

Approaches to disabling agent identities

After reviewing agent activity, choose the approach that matches your scenario. Disabling agents in the Microsoft Entra admin center is object-scoped — it targets a specific blueprint or agent identity without affecting others. Conditional Access policies are tenant-wide enforcement that block token issuance for broad categories of identities, without modifying any agent identities or blueprints. The two approaches can be combined. Applying Conditional Access policies in your tenant requires the Microsoft Entra ID P1 license.

Conditional Access policies can block authentication and token issuance of agent identities. Applying the policies prevents existing and new agent identities from authenticating, but it doesn't prevent the creation of agent identities in your tenant.

We recommend running these policies in report-only mode to understand their impact before enforcing them.

Disable agents in the Microsoft Entra admin center when:

- You want to prevent an agent identity from receiving tokens and authenticating, but you need to keep the agent identity and its metadata in your tenant.

Use Conditional Access policies when:

- You want to prevent all agent identities across your tenant from authenticating, including those you didn't create, without modifying individual objects.
  - Policy 1: Block agent identity authentication.
- You want to prevent agents that act on behalf of users from receiving tokens (for example, agents performing delegated actions), without disabling the agent identity objects themselves
  - Policy 2: Block agent's user account authentication.
- You want to prevent human users from signing into agents or triggering agent actions on their behalf, while leaving agent-to-agent and autonomous flows unaffected
  - Policy 3: Block users signing into agents.
- You need to enforce a temporary, reversible, tenant-wide hold on all agent authentication for compliance or incident response purposes, apply all three policies in report-only mode first, then enforce them.

Disable agent identities and agent identity blueprints in the Microsoft Entra admin center

To disable an agent identity:

1. Sign in to the Microsoft Entra admin center as at least an Agent ID Administrator.
2. Browse to **Entra ID** > **Agents** > **Agent identities**.
3. Select the agent identity you want to disable then select **Disable**.

To disable an agent identity blueprint:

1. Sign in to the Microsoft Entra admin center as at least an Agent ID Administrator.
2. Browse to **Entra ID** > **Agents** > **Agent blueprints**.
3. Select the agent identity blueprint you want to disable then select **Disable**.

Create Conditional Access policies to disable agent activities

There are three policy templates you can use to block token issuance or prevent users from signing into agents. These policies can be created in the Microsoft Entra admin center or through the Microsoft Graph API. We recommend applying these policies in report-only mode first to understand their impact before enforcing them.

- Block token issuance to agent identities using Conditional Access > Policy 1: Block agent identity authentication
- Block token issuance to agents' user accounts using Conditional Access > Policy 2: Block agent's user account authentication
- Block users signing into agents using Conditional Access > Policy 3: Block users signing into agents

Policy 1: Block agent identity authentication

The following steps help create a Conditional Access policy to block issuance of access tokens requested using agent identities.

Microsoft Entra admin center

1. Sign in to the Microsoft Entra admin center as at least a Conditional Access Administrator.
2. Browse to **Entra ID** > **Conditional Access** > **Policies**.
3. Select **New policy**.
4. Give your policy a name. We recommend that organizations create a meaningful standard for the names of their policies.
5. Under **Assignments**, select **Users, agents or workload identities**.
   a. Under **Include**, select **All agent identities**
   b. Under **Exclude**, select **None**
6. Under **Target resources** > **Resources (formerly cloud apps)** > **Include**, select **All resources (formerly 'All cloud apps')**.
7. Under **Access controls** > **Grant**.
   a. Select **Block**.
   b. Select **Select**.
8. Confirm your settings and set **Enable policy** to **Report-only**.

9. Select **Create** to create to enable your policy.

Policy 2: Block agent's user account authentication

The following steps help create a Conditional Access policy to block issuance of access tokens requested using agents' user accounts.

Microsoft Entra admin center

1. Sign in to the Microsoft Entra admin center as at least a Conditional Access Administrator.
2. Browse to **Entra ID** > **Conditional Access** > **Policies**.
3. Select **New policy**.
4. Give your policy a name. We recommend that organizations create a meaningful standard for the names of their policies.
5. Under **Assignments**, select **Users, agents or workload identities**.
   a. Under **Include** > **Select agents** > **agents acting as users** > select **All agents acting as users**
6. Under **Target resources** > **Resources (formerly cloud apps)** > **Include**, select **All resources (formerly 'All cloud apps')**.
7. Under **Access controls** > **Grant**.
   a. Select **Block access**.
   b. Select **Select**.
8. Confirm your settings and set **Enable policy** to **Report-only**.
9. Select **Create** to create to enable your policy.

Policy 3: Block users signing into agents

The following steps help create a Conditional Access policy to block issuance of access tokens to agent resources when requested by human users. This blocks human users from signing into agents and agents performing actions on their behalf.

Microsoft Entra admin center

1. Sign in to the Microsoft Entra admin center as at least a Conditional Access Administrator.

2. Browse to **Entra ID** > **Conditional Access** > **Policies**.
3. Select **New policy**.
4. Give your policy a name. We recommend that organizations create a meaningful standard for the names of their policies.
5. Under **Assignments**, select **Users, agents or workload identities**.
    a. Under **Include**, select **All users**
    b. Under **Exclude**:
        i. Select **None**
6. Under **Target resources** > **Resources (formerly cloud apps)** > **Include**, select **All agent resources**.
7. Under **Access controls** > **Grant**.
    a. Select **Block**.
    b. Select **Select**.
8. Confirm your settings and set **Enable policy** to **Report-only**.
9. Select **Create** to create to enable your policy.

(Optional) Block creation of agent identities

The Conditional Access policies are sufficient to prevent all usage of agent identities in your tenant, including newly created agent identities. Should you wish to prevent creation of agent identities in your tenant, follow the steps in this section.

Agent identities can enter your tenant through various channels. For more information, see Agent ID creation channels. You can block creation of agent identities through the following methods.

- Block creation of agent identities in the Microsoft Entra admin center and other Microsoft Entra experiences.
- Block acquisition of agent identities from Independent Software Vendors (ISVs).
- Block creation of agent identities by Microsoft products and services.

Block creation of agent identities in Microsoft Entra ID

To prevent users from creating agent identities in the Microsoft Entra admin center and other Microsoft Entra experiences:

1. Remove any eligible or active assignments to the **Agent ID Administrator** or **Agent ID Developer** built-in roles.

2. Remove any *oauth2PermissionGrants* or *appRoleAssignments* granted to service principals that allow creation of agent identities. Refer to the following table for the specific permissions.

Expand table

Permission	Type
`AgentIdentity.Create.All`	Application permission
`AgentIdentityBlueprint.Create`	Delegated and application permission
`AgentIdentityBlueprint.ReadWrite.All`	Delegated and application permission
`AgentIdentityBlueprintPrincipal.Create`	Delegated and application permission
`AgentIdentityBlueprintPrincipal.ReadWrite.All`	Delegated and application permission
`AgentIdUser.ReadWrite.IdentityParentedBy`	Delegated and application permission
`AgentIdUser.ReadWrite.All`	Delegated and application permission
`User.ReadWrite.All`	Delegated and application permission. These permissions can also be used to manage human user accounts. Removing this permission causes systems to lose access to manage human users.

See the Microsoft Graph permissions reference for full details of these permissions.

Block acquisition of agent identities from ISVs

To prevent users from creating agent identities by granting consent to an ISVs agent identity blueprint, use Microsoft Entra settings to disable user ability to consent to applications. There's no method to prevent users from granting consent to agent identities without also affecting ability to grant consent to applications. Disabling user consent is broad and also blocks onboarding of legitimate non-agent SaaS apps that depend on user consent flows and granting permissions to existing non-agent apps.

If this impact is too high, keep user consent enabled and instead rely on the Conditional Access block policies to prevent tokens for unapproved ISV agent identities.

Block creation of agent identities by Microsoft products and services

To block Microsoft products and services from creating agent identities in your tenant, you must use the settings available in each Microsoft product:

Security Copilot

To disable agent identity creation by Security Copilot, shut off Security Copilot by deleting all Security Compute Unit (SCU) capacity. This blocks both agents and Security Copilot itself. For more information, see Security Copilot documentation. Users with the following roles can turn Security Copilot back on by creating SCU capacity:

- Billing Administrator
- Microsoft Entra Compliance Administrator
- Global Administrator
- Intune Administrator
- Security Administrator
- Purview Compliance Administrator
- Purview Data Governance Administrator
- Purview Organization Management.

To enable use of Security Copilot but block agent creation, you can use the following methods depending on the type of agents you want to block:

- To block Microsoft agents (For example, Microsoft Entra conditional access Agent), request users with relevant roles not to enable each agent. Roles that can enable agent currently include:
  - Security Administrator
  - Identity Governance Administrator
  - Lifecycle Workflows Administrator
  - Security Copilot Contributor
- To block third-party agents (agents not owned by Microsoft), remove all users from the Owner / Contributor role in any Security Copilot workspaces.

Copilot Studio

To disable use of Copilot Studio and creation of agent identities, you can restrict agent creation via licensing, RBAC, or data policies.

- Licensing:
  - Prevent users from signing up for free trials of Copilot Studio.
  - Don't assign Copilot Studio licenses to users.

- RBAC:
  - Prevent users from signing up for free trials, which prevents creation of trial environments.
  - Creation of Copilot Studio environments requires the Power Platform Administrator role. Remove access to environments and ability to create new environments.
- Data policies:
  - Apply policies that prevent agents from being published, so nobody can chat with an agent. This does **not** block agent creation.

See Copilot Studio documentation for details, which recommends using data policies.

Azure AI Foundry

To disable creation of agent identities, prevent users from creating projects and agents in Azure AI Foundry, you can turn off user ability to create Azure subscriptions via free trials or pay-as-you-go. This enforces the following settings:

- Only Billing Administrator or Account Administrator roles can create subscriptions.
- Within a subscription, only the Azure AI Account Owner role can create Foundry projects.
- Within a project, users must have the Azure AI User role to create agents.

By not assigning these roles to users, users can't create agents or agent identities.

For more information, see Azure AI Foundry documentation here.

Microsoft Teams

To disable creation of agent identities via Microsoft Teams, use settings in the Teams admin center:

- Prevent users from adding apps/agents to Teams.
- Pivot by Microsoft apps, third-party apps, or custom apps.
- Enable specific apps/agents for specific users/groups as needed.

For more information, see Teams admin center documentation.

Related content

- Agent identity deletion
- Create and delete agent identities
- Agent sign-in and audit logs

- Agent identity creation channels

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/01/2026

View and filter agent identities in your tenant

Microsoft Entra admin center provides you with a centralized interface to view and filter your agent identities. This comes with the ability to search, filter, sort, and customize columns to find specific agent identities in your tenant.

- To view and manage your agent identity blueprint principals, see View and manage agent identity blueprints using Microsoft Entra admin center.
- To view and manage agents registered in the Agent Registry without an identity, see manage agent identity blueprints with no identities.

Prerequisites

To view agent identities in your Microsoft Entra tenant, you need:

- A Microsoft Entra user account. If you don't already have one, you can Create an account for free. No admin role is required for viewing.

To manage agent identities in your Microsoft Entra tenant, you need:

- Agent ID Administrator or Cloud Application Administrator role.
- You can also manage your agent identity if you're the owner of that agent identity, with or without the above roles.

View a list of agent identities

To view agent identities in your tenant:

1. Sign in to the Microsoft Entra admin center
2. Browse to **Entra ID** > **Agents** > **Agent identities**.
3. Select any agent identity you'd like to manage.

This page contains a list of all agent identities in your organization. This includes both agent identity objects and agents using a service principal.

Search for an agent identity

- To search for an agent identity, enter either the **name** or **object ID** of the agent identity you want to find in the search box.
- To look up an agent identity by its **Blueprint App ID**, add the **Blueprint App ID** filter. You can further refine the list using filters based on various criteria.

You can select an agent identity from this list to see information like:

- An overview of the agent identity, including:
  - The name, description, and logo for your agent identity
  - The status of that agent identity, and the ability to enable/disable a given agent identity
  - The link to the parent agent identity blueprint
- The list of owners and sponsors for that agent identity
- The agent's access via this agent identity's granted permissions and Microsoft Entra roles
- Audit logs and sign-in logs for that agent identity

Select viewing options

To customize your view of agent identities, you can change filters or select which columns are shown for each agent identity. Not all columns are shown by default. To see all available columns and edit shown columns, select the **Choose columns** button. The table columns and their filter options are as follows:

Expand table

Column Name	Description	Sortable	Filterable	Special notes
Name	Display name of the agent identity	✓	✓	Primary search field; clickable to view details of the agent identity
Created On	Date when the agent was created	✓	✓	Filter by "Last N days"
Status	Current operational state (Active, or Disabled)	✓	✓	
Object ID	Unique identifier for agent identity	✗	✓	
View Access	Direct link to agent identity's permissions	✗	✗	Navigates to the Agent's Access pane, on Permissions tab
Blueprint App ID	Unique identifier for the agent identity blueprint of this agent	✗	✓	Will be blank for agents using service principals

Column Name	Description	Sortable	Filterable	Special notes
	identity			
Owners and Sponsors	Direct link to the owners and sponsors for a given agent identity	X	X	
Uses agent identity	Represents whether or not this agent has an agent identity object, or utilizes a service principal	X	X	If the answer is "yes," then it uses an agent identity object. If "no" this agent utilizes a service principal

Related content

- Manage agent identities in your organization - Overview of agent identity management including roles, lifecycle, and governance.
- Manage agents in end user experience - Owners and sponsors can manage their agents from the My Account portal without admin roles.
- Conditional Access for Agent ID - Enforce Conditional Access policies across all agent identities or specific groups.
- View and manage agent identity blueprints in your tenant

Last updated on 05/01/2026

Manage Agents in end user experience

The Manage Agents feature in Microsoft Entra lets you view and control, agent identities you own or sponsor. Agents identities are special identities, such as bots or automated processes, that act on behalf of users or teams. With the manage agents feature, you can easily see which agents you're responsible for, review their details, and take action to enable, disable, or request access for them.

> ⓘ Note
>
> This article is for agent identity owners and sponsors. The **Manage agents** menu only appears for users who own or sponsor at least one agent identity. For tenant-wide management by administrators, see Manage agent identities in your organization.

Manage agents as an agent identity owner or sponsor

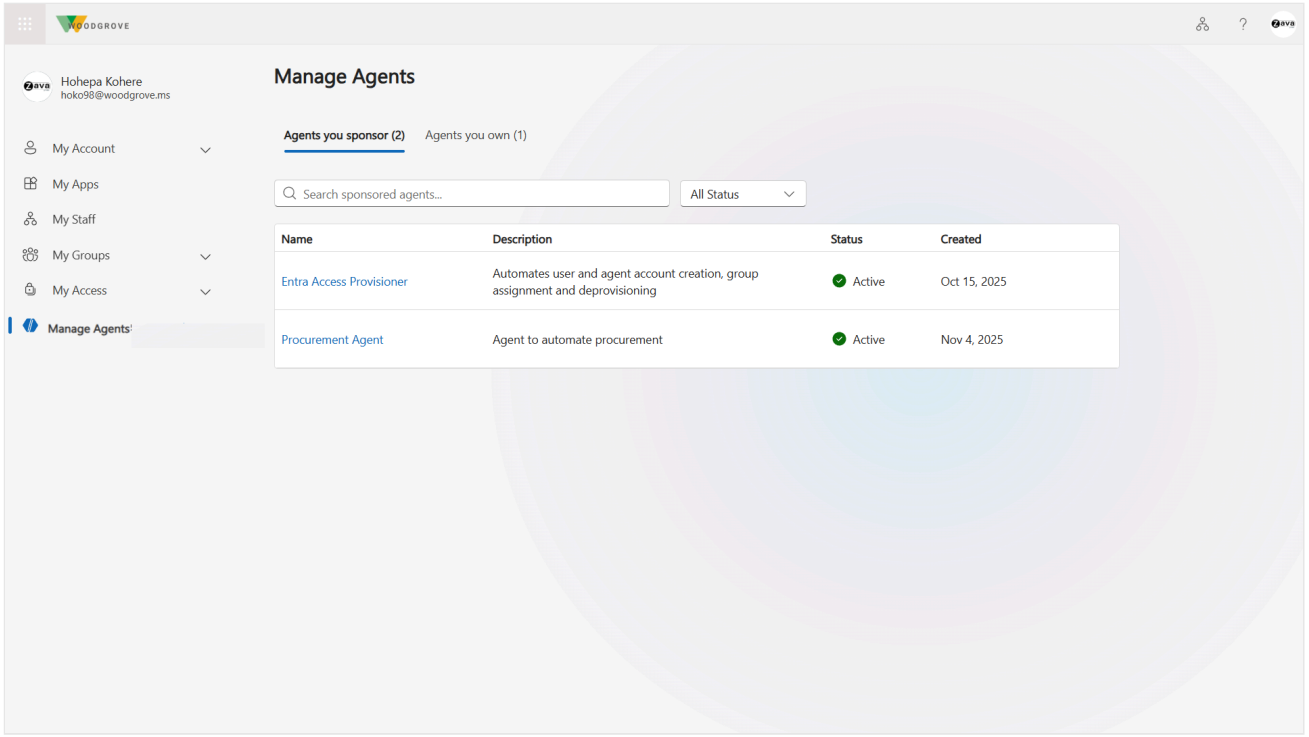
1. Sign in to the My Account end user portal as either an owner or sponsor of at least one agent identity.
2. If you haven't opted in to the new homepage yet, select **Use new version** in the banner.

   > ⓘ Note
   >
   > If you're already using the new homepage, the banner still appears with the message "You're using the new version of the account homepage" and a **Use previous version** button.

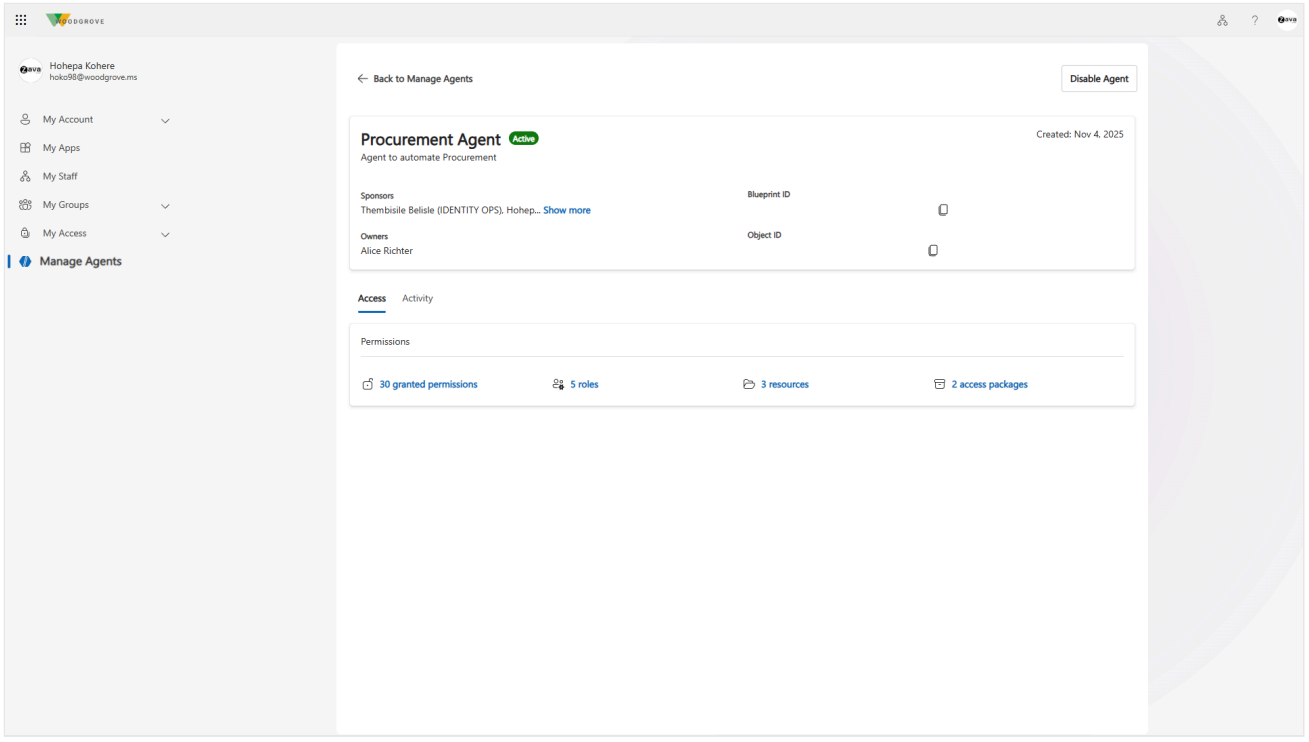
3. In the left menu, select **Manage agents**.

   > ⓘ Note
   >
   > This menu item will only appear if you're an owner or sponsor of at least one agent identity.

4. Choose either the **Agents you sponsor** or **Agents you own** tab to view your agents.



5. Select an agent to view details about it.



Enable or disable an agent

1. To disable an agent, select it from the list and choose **Disable agent**. This blocks users from being able to access it and prevents it from being issued tokens. This has the same effect as disabling the agent from the admin center.

2. To re-enable, select the agent that is disabled and choose **Enable agent**. This allows users to access it, and allows it to be issued tokens. Note that sponsors can't re-enable agents. They will need an owner or admin's help if one of their agents needs to be re-enabled.

Request an access package on behalf of an agent identity

As the owner or sponsor of an agent identity, you can request an access package for that agent identity by doing the following steps:

1. Sign in to the My Access portal at https://myaccess.microsoft.com .
2. On the My Access Portal page, select **Access packages**.
3. On the Access packages page, locate the access package you want to request for an agent identity to have, and select **Request**.
4. On the Request pane under **Request details**, select **Requesting for Sponsored agent** or **Requesting for Owned agent**.
5. Select the agent identity and then select **Continue**.

Next steps

- Manage agent identities in your organization - Full agent management overview including roles, security, and governance.
- View and filter agent identities in your tenant - For organization-wide agent viewing, filtering, and search.
- If an agent needs other access packages, Request an access package on behalf of an agent identity
- Governing Agent Identities - Understand sponsor responsibilities and access package governance.

Last updated on 09/03/2026

Agent Registry convergence with Microsoft Agent 365

Organizations are rapidly adopting AI agents across Microsoft platforms, partner ecosystems, and custom applications. As agents grow in number and capability, organizations need a centralized way to observe, govern, and secure them.

Microsoft Agent 365 is Microsoft's control plane for AI agents. It helps organizations:

- Observe agents across the enterprise.
- Govern how agents access systems, data, and tools.
- Secure agents using Microsoft identity and security capabilities.

A key capability within Microsoft Agent 365 is the agent registry, which provides a unified inventory of all agents operating in the organization, including both Microsoft and non-Microsoft agents.

Registry convergence

Previously, agent visibility appeared across multiple portals, including Microsoft Entra and the Microsoft 365 admin center. Based on customer feedback, Microsoft is converging these registry experiences under Agent 365 to provide a simpler and more consistent management experience.

With this change:

- **Agent 365** becomes the unified registry and control plane for agents.
- **Microsoft Entra** continues to provide the identity foundation through Agent ID.

This approach gives customers one place to discover and manage agents, while continuing to use Microsoft Entra for agent identity and access control.

Frequently asked questions

Which registry should I use today?

Use Agent 365 to discover and manage all agents in your organization and monitor operational activity. Use Microsoft Entra to manage agent identities (Agent ID), apply identity governance and Conditional Access policies, and monitor identity-related security signals.

What happens to the features announced as part of the Microsoft Entra Agent Registry?

The capabilities introduced with the Microsoft Entra Agent Registry continue to exist. Specifically:

- Agent identity capabilities remain part of Microsoft Entra Agent ID.
- Agent registration APIs remain supported.
- Identity governance and security controls for agents remain unchanged.

The change simplifies where customers see and manage all agents in their organization, while Microsoft Entra continues to provide identity and access management.

Will I see the same agent inventory in the Microsoft Entra admin center?

The Microsoft Entra admin center focuses on identity and access management for agents. Identity administrators can see agents that have a Microsoft Entra Agent ID for viewing and managing them. The comprehensive agent inventory, including agents without a Microsoft Entra agent identity, is available in Agent 365.

What can I manage in the Microsoft Entra admin center?

In the Microsoft Entra admin center, administrators can:

- View agents with Microsoft Entra agent identities.
- Manage agent identities, blueprints, and permissions.
- Apply Conditional Access, identity governance, and network security controls.
- Monitor identity-related security signals.

Why should I go to Microsoft Agent 365 if I'm an identity administrator?

Identity administrators can optionally use Agent 365, via the Microsoft 365 admin center, to view all agents in the organization, while continuing to use the Microsoft Entra admin center to manage agent identities, access policies, and identity governance controls.

Do I need a different role to view agents in Agent 365?

To see all agents in Agent 365 and Microsoft Entra Agent ID, users need the AI Reader role. This role is intended for users who need visibility for monitoring or reporting access.

The Agent ID Administrator role is needed to manage or change agent identities.

Is a license required?

Viewing all agents in the Microsoft 365 admin center doesn't require a specific license. Administrators only need the appropriate role, such as AI Reader (recommended least-privilege role) or AI Administrator, to access the inventory view.

Applying security and governance controls for agents, such as Conditional Access or identity governance policies, requires the appropriate licensing for Microsoft Entra Agent ID.

How do I get agent identity blueprints to appear in the Agent 365 registry?

After you create an agent identity blueprint, register it in the Agent 365 registry so administrators can discover, govern, and manage the agent from the Microsoft 365 admin center. In some scenarios, previously created agent identity blueprints may not appear in the registry. To ensure that all agent identity blueprints are registered, see Register agents in the Agent 365 registry.

How to view the complete agent inventory

To view the complete inventory of agents in your organization:

1. Go to the Microsoft 365 admin center as at least an AI Reader.
2. Select **Agents** from the navigation menu.
3. Select **All agents** to view the comprehensive list of agents in your tenant.

Related content

- What is Microsoft Entra Agent ID?
- What is the Microsoft Entra Agent Registry?
- Microsoft Agent 365 overview

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/09/2026

Migrate custom app registrations to Agent ID

If you built AI agents by using standard Microsoft Entra app registrations or service principals, your agents authenticate like any other application. They don't take advantage of agent-specific governance capabilities like Conditional Access policies, centralized audit logging, and lifecycle management that Agent ID provides.

This article shows you how to migrate agents where you own the code and the identity configuration to Agent ID. You create Agent ID resources (a blueprint and agent identity), update your application code, and decommission the old identity. For agents created through Microsoft Copilot Studio, see Migrate Copilot Studio agents to Agent ID.

Prerequisites

- An existing AI agent that uses an app registration or service principal for authentication.
- **Agent ID Developer** or **Agent ID Administrator** role to create blueprints and agent identities.
- **Privileged Role Administrator** role to grant Microsoft Graph application permissions.
- Access to the Microsoft Graph v1.0 API.
- Familiarity with Agent ID key concepts. For more information, see Agent identity concepts.

Licensing requirements

Microsoft Entra Agent ID is a product within Microsoft Entra that provides the platform for creating and managing agent identities and agent identity blueprints. Agent ID is available for all Microsoft Entra customers.

Microsoft Agent 365 enables agents to operate across Microsoft 365 services and enterprise workflows, which requires a **Microsoft Agent 365** license for each user. For pricing details, see Microsoft Agent 365 plans and pricing.

Extending Microsoft Entra security features to agents requires **Microsoft 365 E7** (includes Agent 365 and Microsoft Entra Suite) or **Microsoft 365 E5** paired with a **Microsoft Agent 365** license. Customers without E5 or E7 can use the following standalone licensing options with a **Microsoft Agent 365** license:

- **Conditional Access for agents**: Microsoft Entra ID P1
- **ID Protection for agents**: Microsoft Entra ID P2
- **ID Governance for agents**: Microsoft Entra ID P1
- **Network controls for agents**: Microsoft Entra Internet Access, included in Microsoft Entra Suite or licensed separately. For more information, see What is Global Secure Access.

Review migration benefits

Microsoft Entra Agent ID provides specialized identity constructs for AI agent authentication and authorization. Agent identity blueprints serve as governance templates, and agent identities are the runtime principals your agents use to authenticate, acquire tokens, and access resources. By migrating to Agent ID, you get:

Agent ID brings your agents under the same governance model as users and workloads. Moving to Agent ID gives you:

Access control

- **Conditional Access enforcement.** Apply Conditional Access policies and controls directly to agents and the resources they access, just as you would for users.

Agent governance

- **Lifecycle management.** Microsoft Entra ID Governance can manage agent access lifecycle policies, so agents don't accumulate stale permissions.
- **Multi-cloud visibility.** Agents operating across Azure, on-premises, and other cloud environments are governed through a single identity plane.

Visibility and audit

- **Agent-specific audit trails.** Audit and sign-in logs attribute agent actions and the resources they access to the specific agent identity, not a generic app registration. This approach makes investigation and attribution straightforward.
- **Agent registry visibility.** Agents appear in your organizational agent registry, giving you a complete inventory instead of blind spots.

Risk and protection

- **ID Protection coverage.** ID Protection monitors for anomalous agent behavior, giving you risk-based detection and automated remediation.

Review the migration phases

Migrating to Agent ID is a phased process, not a single-step switch. This guide organizes the process into four phases, each building on the outputs of the previous one. The phased model prevents premature deletion of service principals that might support active workloads. Proper discovery and classification occur before any migration or decommission action.

Expand table

Phase	Goal	Key output
Discover	Inventory all agent-related identities in your tenant: service principals, app registrations, and their usage signals.	A structured report or dashboard with identity metadata, sign-in activity, permissions, ownership, and builder origin for each agent.
Classify	Categorize each identity by usage level and origin.	A prioritized action plan: which identities to clean up, which to migrate, and which to leave as-is.
Migrate	Create Agent ID resources (blueprint + agent identity) and update your agents to use them.	New agent identities running on Agent ID with matching permissions and credentials.
Validate and decommission	Confirm the new identity works end-to-end, parallel-run if needed, then retire the legacy identity with safeguards.	Legacy identities removed; agents running fully on Agent ID.

Phase 1: Discover existing app registrations

Before migrating anything, build a complete inventory of the agent-related identities in your tenant. Start with an organization-wide sweep to find all migration candidates, then narrow down to individual agents to capture the configuration details you need during migration.

Create an organization-level inventory

Scan your tenant for all service principals that might represent AI agents. The goal is to create a structured report that surfaces usage signals and helps you classify each identity in the next phase.

For each candidate service principal, capture:

- **Identity metadata:** display name, application ID, object ID, creation date, tenant.
- **Ownership:** assigned owners, owning team or department (where available).

- **Sign-in activity:** last interactive and non-interactive sign-in, sign-in count over 30, 90, and 180 days.
- **Audit log signals:** source system that created the service principal, creation event, recent modifications, or permission changes.
- **Tag analysis:** tags applied to the service principal; check whether it carries any builder-specific tag patterns.
- **API permissions:** delegated and application permissions granted, admin consent status, permission sensitivity level.
- **Other identity configurations:** credentials used, OAuth flows, custom attributes, RBAC role assignments, redirect URIs, and any downstream dependencies. You need these configuration details during migration to correctly recreate the identity as an Agent ID.

> **Tip**
>
> You can pull most of the configuration details programmatically by using Microsoft Graph. Use the `GET /applications` and `GET /servicePrincipals` endpoints to pull permissions, credentials, ownership, and more. For OAuth flows your application uses and any downstream dependencies, check your application code.

Identify agents through log and permission analysis

For service principals that don't have tags, use behavioral and configuration signals to identify likely agent identities. No single signal is definitive. Weight them in combination to produce a confidence score.

Expand table

Signal category	What to look for	Why it indicates an agent
API permissions	Application permissions targeting Bot Framework, Azure OpenAI, Azure AI Services, or Cognitive Services APIs.	Agents require these APIs to function; traditional applications rarely request them together.
Redirect URIs	URIs pointing to `token.botframework.com`, `botframework.com`, or Azure Bot Service endpoints.	These URIs are Bot Framework callback URLs used exclusively by conversational agents.
Sign-in patterns	Non-interactive or service principal sign-ins with high frequency and no associated user context.	Agents authenticate autonomously; human-facing apps typically have interactive sign-in activity.

Signal category	What to look for	Why it indicates an agent
Token audience	Tokens requested for `https://api.botframework.com`, Azure OpenAI endpoints, or AI Services endpoints.	The token audience reveals what resource the service principal is calling.
Resource group association	Service principal linked to a resource group containing Bot Service, Azure OpenAI, or AI Search resources.	Co-location with AI infrastructure suggests the service principal serves an agent workload.
Naming conventions	Display names containing terms like `agent`, `bot`, `copilot`, `assistant`, or `orchestrator`.	While not deterministic, naming patterns correlate strongly with agent workloads when combined with other signals.

To operationalize heuristic discovery, query service principal sign-in logs and permission grants programmatically with Microsoft Graph. For tenants with extended log retention (through Microsoft Sentinel or another SIEM), expand the analysis window beyond Microsoft Entra's default 30-day sign-in retention to improve signal accuracy.

Heuristic discovery produces candidates, not confirmed agents. Review every match before proceeding to classification. Expect false positives, such as backend microservices that call Azure OpenAI but aren't autonomous agents.

Discover agents through organizational knowledge sources

Automated methods might not find every agent, especially custom-built agents with generic API permissions and no naming convention overlap are invisible to tag-based and heuristic scanning. You should therefore consider:

CMDB and asset inventory reconciliation

Cross-reference your discovered service principals against your organization's Configuration Management Database or application portfolio registry. Applications registered as "AI agent," "chatbot," or "virtual assistant" workload types in the CMDB should be matched to their corresponding service principals and added to the migration inventory.

Developer self-attestation

For tenants with large numbers of app registrations, publish the migration inventory to application owners and ask them to attest whether their service principal represents an AI agent. This approach scales discovery beyond what any automated scan can achieve, because the

application owner has definitive knowledge of the workload type. Combine self-attestation with a deadline and escalation path to drive completion.

Recommended discovery sequence

Run these methods in order:

1. **Tag-based scan:** Automatically identify all tagged agents. These agents are confirmed candidates.
2. **Heuristic analysis:** Scan remaining untagged service principals for behavioral signals. Flag high-confidence matches as probable candidates.
3. **CMDB reconciliation:** Match remaining service principals against your asset inventory to catch agents registered under non-obvious names.
4. **Developer attestation:** Publish the residual unmatched list to application owners for final classification.

Phase 2: Classify agent identities for migration

Using the discovery report, classify each service principal by usage level, which determines how you proceed with the migration.

Expand table

Usage	Signals	Recommended action
Low	No sign-in activity in 30+ days, no owner assigned, no API permissions in active use.	Candidate for cleanup. Skip migration; decommission directly (Phase 4).
Medium	Some recent sign-in activity, non-production permissions, identifiable owner.	Candidate for migration. Proceed through Phases 3–4 with standard validation.
High	Frequent sign-ins, production API permissions, active workflows dependent on the SP.	Migrate with extreme caution. Requires extended parallel run (Phase 4), rollback plan, and stakeholder sign-off.

The 30-day sign-in threshold is based on Microsoft Entra's default retention period. If your organization has audit log retention through Microsoft Sentinel or another SIEM, adjust thresholds accordingly.

Phase 3: Migrate app registrations to Agent ID

This phase covers the technical migration steps for agents whose identity was created as a standard app registration or service principal. You create new Agent ID resources and update your application code to use the two-stage token acquisition model. Steps 1 through 3 target the Microsoft Entra tenant. Step 4 targets changes to your application code.

> **Important**
>
> There's no in-place conversion that transforms an existing app registration or service principal into an agent identity. Migration requires creating a new agent identity alongside your existing identity, then transitioning your agent to use it.

Step 1: Create an agent identity blueprint

To complete this step, you need the **Agent ID Developer** or **Agent ID Administrator** role to create blueprints and agent identities, and the **Privileged Role Administrator** role to grant Microsoft Graph application permissions.

The blueprint is the governance template for your agent. It defines credentials and permission policies that all agent identities created under it inherit.

Create the blueprint using Microsoft Graph:

HTTP

```
POST
https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint
Content-Type: application/json

{
    "@odata.type": "Microsoft.Graph.AgentIdentityBlueprint",
    "displayName": "My Agent Blueprint",
    "sponsors@odata.bind": [
        "https://graph.microsoft.com/v1.0/users/<sponsor-user-id>"
    ],
    "owners@odata.bind": [
        "https://graph.microsoft.com/v1.0/users/<owner-user-id>"
    ]
}
```

Or use PowerShell:

PowerShell

```
$body = @{
    "@odata.type" = "Microsoft.Graph.AgentIdentityBlueprint"
    "displayName" = "My Agent Blueprint"
    "sponsors@odata.bind" = @("https://graph.microsoft.com/v1.0/users/<sponsor-user-id>")
    "owners@odata.bind" = @("https://graph.microsoft.com/v1.0/users/<owner-user-id>")
} | ConvertTo-Json -Depth 5

Invoke-MgGraphRequest `
    -Method POST `
    -Uri "https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint" `
    -Headers @{ "OData-Version" = "4.0" } `
    -Body $body `
    -ContentType "application/json"
```

> **Important**
>
> You create credentials on your blueprint, not on individual agent identities. Configure the federated identity credential on the blueprint before creating agent identities.

Configure credentials on the blueprint

Agent ID uses federated identity credentials (FIC) as the recommended credential type. Managed identities are the preferred option for production deployments because they eliminate secret management. Client secrets and certificates are also supported for production use. To reduce migration effort, consider reusing the same credential type your existing app registration uses.

The recommended credentials depend on your hosting environment:

Expand table

Hosting environment	Recommended credential	Notes
Azure (App Service, AKS, Container Apps, VMs)	User-assigned managed identity	Most secure for production. No secret management required.
Non-Azure cloud (AWS, GCP)	Federated identity credential with external IDP	Configure workload identity federation with your cloud provider.
On-premises or local development	Client secret or certificate	Certificates are recommended over client secrets for production on-premises deployments. Use client secrets for local testing only.

After you configure the credential, create a service principal for the blueprint. The service principal is required before you can create agent identities under the blueprint:

HTTP

```
POST https://graph.microsoft.com/v1.0/servicePrincipals
Content-Type: application/json

{
    "appId": "<blueprint-app-id>"
}
```

Step 2: Create the agent identity

To complete this step, you need the **Agent ID Developer** or **Agent ID Administrator** role.

The agent identity is the runtime principal your agent uses. Create it under a blueprint and assign a **sponsor** (required), a human user, or group accountable for the agent. Assigning an **owner** is recommended for management purposes.

HTTP

```
POST https://graph.microsoft.com/beta/serviceprincipals/Microsoft.Graph.AgentIdentity
OData-Version: 4.0
Content-Type: application/json

{
    "displayName": "Customer Service Agent - Production",
    "agentIdentityBlueprintId": "<blueprint-app-id>",
    "sponsors@odata.bind": [
        "https://graph.microsoft.com/v1.0/users/<sponsor-user-id>"
    ]
}
```

Record the agent identity's client ID from the response. You need this value when updating your application code.

Step 3: Configure permissions

Replicate the API permissions from your original app registration onto the agent identity. To complete this step, you need the **Privileged Role Administrator** role to grant application permissions that require admin consent. You have two options:

- **Direct assignment:** Assign permissions directly to the agent identity. Use this option when each agent identity needs different permissions.

- **Inherited permissions:** Configure permissions on the blueprint and enable inheritance. Use this option when all agent identities under a blueprint share the same permissions.

To replicate Microsoft Graph application permissions, grant the same `appRoles` to the agent identity's service principal by using the Grant an appRoleAssignment to a service principal API. For delegated permissions, add the required `oauth2PermissionGrants` by using the Create oAuth2PermissionGrant API. Ensure admin consent is granted for application permissions that require it.

Configure OBO support (interactive agents only)

If your agent uses the on-behalf-of (OBO) flow to act on behalf of a user, you need to add a custom scope to the blueprint so that front-end applications can request tokens for it. After adding the scope, update your front-end application to request a token for the blueprint's resource instead of the previous resource. For the complete OBO configuration walkthrough, see Interactive agent authentication and authorization flow.

If your original app registration had Azure RBAC role assignments, assign the same roles to the new agent identity's service principal so it can access the required Azure resources at runtime:

PowerShell

```
New-AzRoleAssignment `
    -ObjectId "<agent-identity-service-principal-id>" `
    -RoleDefinitionName "Contributor" `
    -Scope "/subscriptions/<sub-id>/resourceGroups/<rg-name>"
```

Step 4: Update application code

Agent ID uses a two-stage token acquisition model. Your agent first acquires a bootstrap token by using the blueprint's credential, then exchanges it for an agent-specific token. For more information on updating your code, see these scenarios:

- Autonomous agent authentication and authorization flow for the complete token acquisition walkthrough.
- Interactive agent authentication and authorization flow for interactive agents guidance, code samples, and token configuration. The token flow in these agents adds an OBO exchange where a user token is exchanged for an agent token with user context.

Phase 4: Validate and decommission

After migrating an agent identity, validate that the new Agent ID works correctly before decommissioning the old identity.

Validate the migrated agent identity

Expand table

Check	How to verify	Expected result
Token acquisition	Run your agent and inspect the token response.	Agent successfully acquires tokens using the two-stage flow. No authentication errors.
API access	Exercise all API calls the agent makes.	All downstream APIs return expected results. No 401/403 errors.
Sign-in logs	**Microsoft Entra admin center** > **Sign-in logs**, filter by the new agent identity.	Sign-in events appear for the new agent identity with successful status.
Audit logs	**Microsoft Entra admin center** > **Audit logs**.	Agent ID creation and permission assignment events are logged.
Conditional Access	Review CA policies that target the resource.	CA policies evaluate correctly for the new agent identity. No unexpected blocks.

Run old and new identities in parallel

For agents with high usage, run the old and new identities side by side before switching over:

1. Deploy a parallel instance of your agent configured with the new Agent ID.
2. Route a percentage of traffic to the new instance (start with 5–10%).
3. Monitor both instances for errors, latency differences, and permission issues.
4. Gradually increase traffic to the new instance over one to two weeks.
5. Once 100% of traffic runs on the new identity, proceed to decommission.

Use feature flags or traffic-splitting infrastructure to control the rollout. This approach lets you revert to the old identity instantly if issues arise.

Decommission safeguards

After you validate the new Agent ID and it carries production traffic, decommission the old identity. Follow these safeguards to minimize risk:

- **Pre-deletion snapshot:** Export the service principal's metadata, permissions, and audit history before deletion. This export provides a rollback reference if anything was missed

during validation.

- **Phased batches:** If you're decommissioning multiple identities, delete in small batches (20-50 service principals per wave) rather than bulk-deleting all at once.
- **Soft-delete first:** Use Microsoft Entra's soft-delete capability (30-day recycle bin for applications) before hard deletion. This step gives you a recovery window.

Decommission the original app registration

To decommission the old identity, follow these steps:

1. Remove API permissions from the old app registration.
2. Revoke Azure RBAC role assignments for the old service principal.
3. Delete or rotate client secrets and certificates on the old app registration.
4. Delete the app registration (enters 30-day soft-delete).
5. After 30 days, confirm the app registration is permanently deleted or hard-delete if needed.

Troubleshoot common issues

The following table lists common issues you might encounter during migration and how to resolve them.

Expand table

Symptom	Likely cause	Resolution
Token exchange fails with `invalid_grant`	The federated identity credential on the blueprint is misconfigured or the managed identity isn't correctly linked.	Verify the FIC subject, issuer, and audience match your hosting environment. Check that the managed identity has the correct assignment.
Audience mismatch error	The token request specifies the wrong audience or the blueprint audience doesn't match the target resource.	Ensure the token request audience matches the resource you're calling. For Microsoft Graph, use `https://graph.microsoft.com/.default`.
`403 Forbidden` on API calls	API permissions weren't correctly replicated from the old app registration to the agent identity.	Compare the permissions on both identities. Ensure admin consent was granted for application permissions.
OBO flow fails	The agent identity isn't configured for delegated permissions, or the user	Verify delegated permissions are assigned. Check that the user token includes the scopes your agent needs for the OBO exchange.

Symptom	Likely cause	Resolution
	token doesn't include the required scopes.	
`400 Bad Request: Object not found` (after creating blueprint)	Read-after-write consistency delay. The blueprint or agent identity isn't yet available for subsequent API calls.	Wait 30–60 seconds after creating the blueprint before creating agent identities or service principals. Implement retry logic with exponential backoff.
Agent ID not visible in **Microsoft Entra admin center**	Agent identities created through the API might take a few minutes to appear in the portal.	Wait and refresh. If the identity still doesn't appear, verify it was created successfully through the Graph API.

Related content

- Migrate Copilot Studio agents to Agent ID
- Agent identity concepts
- Create an agent identity blueprint
- Create and delete agent identities
- Best practices for Agent ID

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/15/2026

Recreate Copilot Studio agents in Microsoft Entra Agent ID

Some agents created in Microsoft Copilot Studio authenticate by using platform-managed service principals. These agents could have been created before Copilot Studio began automatically creating agent identities for all new agents, which happened on March 18, 2026. These agents could have also been created before your organization opted in to the Microsoft Entra Agent ID integration with Copilot Studio or if your organization opted out of creating agent identities.

These service principals let agents communicate with Azure Bot Service, Microsoft Teams, and Bot Framework skills, but Microsoft Entra treats these service principals as standard applications, not as AI agents. Adopting Microsoft Entra Agent ID gives you agent-specific governance, including Conditional Access policies, centralized audit logging, and lifecycle management.

This article describes the manual process of recreating Copilot Studio agents with Microsoft Entra Agent ID and decommissioning the legacy service principals. Because Copilot Studio manages the agent code, credentials, and deployment lifecycle, this process differs from custom-built agents. For agents where you own the code and identity configuration, see Migrate custom app registrations to Agent ID.

> **Important**
>
> At this time, there is no automated or in-place migration path to convert an existing Copilot Studio agent's service principal to an agent identity. To use Microsoft Entra Agent ID with Copilot Studio, you must **create a new agent** with the Agent ID integration enabled, manually reconfigure the agent, and then decommission the legacy agent. This article describes that recreate-and-deprecate process.

Prerequisites

Before you begin, make sure you meet the following requirements:

- One or more Copilot Studio agents that use legacy service principals, created before March 18, 2026 or before your tenant opted in to Microsoft Entra Agent ID.
- Access to the **Copilot Studio admin center** and the **Microsoft Entra admin center**.

- **Agent ID Developer** or **Agent ID Administrator** role in Microsoft Entra to manage agent identities.
- Microsoft Entra Agent ID integration enabled for your tenant in Copilot Studio.
- Familiarity with Microsoft Entra Agent ID key concepts.

Licensing requirements

Microsoft Entra Agent ID is a product within Microsoft Entra that provides the platform for creating and managing agent identities and agent identity blueprints. Agent ID is available for all Microsoft Entra customers.

Microsoft Agent 365 enables agents to operate across Microsoft 365 services and enterprise workflows, which requires a **Microsoft Agent 365** license for each user. For pricing details, see Microsoft Agent 365 plans and pricing.

Extending Microsoft Entra security features to agents requires **Microsoft 365 E7** (includes Agent 365 and Microsoft Entra Suite) or **Microsoft 365 E5** paired with a **Microsoft Agent 365** license. Customers without E5 or E7 can use the following standalone licensing options with a **Microsoft Agent 365** license:

- **Conditional Access for agents**: Microsoft Entra ID P1
- **ID Protection for agents**: Microsoft Entra ID P2
- **ID Governance for agents**: Microsoft Entra ID P1
- **Network controls for agents**: Microsoft Entra Internet Access, included in Microsoft Entra Suite or licensed separately. For more information, see What is Global Secure Access.

Why adopt Microsoft Entra Agent ID?

Copilot Studio agents that use legacy service principals don't take advantage of the governance capabilities that Microsoft Entra Agent ID provides. By adopting Microsoft Entra Agent ID, your organization gets:

Agent ID brings your agents under the same governance model as users and workloads. Moving to Agent ID gives you:

Access control

- **Conditional Access enforcement.** Apply Conditional Access policies and controls directly to agents and the resources they access, just as you would for users.

Agent governance

- **Lifecycle management.** Microsoft Entra ID Governance can manage agent access lifecycle policies, so agents don't accumulate stale permissions.
- **Multi-cloud visibility.** Agents operating across Azure, on-premises, and other cloud environments are governed through a single identity plane.

Visibility and audit

- **Agent-specific audit trails.** Audit and sign-in logs attribute agent actions and the resources they access to the specific agent identity, not a generic app registration. This approach makes investigation and attribution straightforward.
- **Agent registry visibility.** Agents appear in your organizational agent registry, giving you a complete inventory instead of blind spots.

Risk and protection

- **ID Protection coverage.** ID Protection monitors for anomalous agent behavior, giving you risk-based detection and automated remediation.

How Copilot Studio agents differ from custom agents

Copilot Studio agents require a different approach than custom agents:

Expand table

Aspect	Custom app registration	Copilot Studio
Who controls the code	You (the developer)	The Copilot Studio platform
Who created the SP	You, through the Azure portal or Graph API	Copilot Studio, automatically during agent publishing
Credential management	You configure FIC, managed identity, or client secrets	Copilot Studio manages credentials on your behalf
Microsoft Entra Agent ID action	Create blueprint + agent identity through Graph API, update your code	Create a new agent with Microsoft Entra Agent ID integration enabled, then manually reconfigure
Rollback strategy	Revert code to use old app registration	Keep the legacy agent active until the new agent is fully validated

Because Copilot Studio manages the identity lifecycle, you can't create a blueprint and agent identity directly as you would for a custom agent. There's no in-place conversion that transforms an existing service principal into an agent identity, and republishing an existing agent doesn't

create an agent identity. Instead, you must create a new agent in Copilot Studio with Microsoft Entra Agent ID integration enabled, manually reconfigure all channels and connections, and then decommission the legacy agent.

Review the process phases

The recreate-and-deprecate process follows four phases, each building on the previous:

Expand table

Phase	Goal	Key output
Discover	Inventory all Copilot Studio agent-related service principals in your tenant.	A report or dashboard with identity metadata, agent mapping, sign-in activity, and channel deployments.
Classify	Categorize each agent by usage level.	A prioritized action plan: which agents to clean up, which to recreate with Microsoft Entra Agent ID, and which to leave as-is.
Recreate	Create new agents with Microsoft Entra Agent ID integration enabled and manually reconfigure them.	New agents running on Microsoft Entra Agent ID-native agent identities.
Validate and decommission	Confirm the new agent works end-to-end, then retire the legacy agent and service principal with safeguards.	Legacy service principals removed; agents running fully on Microsoft Entra Agent ID.

Phase 1: Discover Copilot Studio agents

Copilot Studio agents leave specific fingerprints on their service principals. Use the following signals to identify them:

- **Tag patterns:** Copilot Studio applies specific tag patterns to service principals during provisioning. The most reliable discovery signal is the `AgentCreatedBy:CopilotStudio` tag. Other consistent tags include `AgenticApp`, `AIAgentBuilder`, `AgenticInstance`, and a `power-virtual-agents-{agent-id}` tag that links the service principal to the specific agent in Copilot Studio.
- **Copilot Studio admin center:** Cross-reference discovered service principals with the **Copilot Studio admin center** to find the associated agent name and ID, last published date, and agent status.

Copilot Studio configures platform-managed credentials (federated identity credentials) on the agent's app registration to authenticate on behalf of the agent. Copilot Studio manages these credentials. You don't need to inspect or manage them directly.

For each Copilot Studio service principal, capture:

- **Identity metadata:** display name, application ID, object ID, creation date.
- **Agent mapping:** the corresponding agent name and ID in Copilot Studio.
- **Last published date:** when the agent was last published or updated.
- **Agent status:** active, inactive, or draft.
- **API permissions:** delegated and application permissions granted.
- **Connection references:** connectors and data sources the agent uses.
- **Channel deployments:** Teams, web chat, or other channels the agent is published to.
- **Downstream dependencies:** any workflows, Power Automate flows, or other resources that depend on the agent's service principal.

You can enrich your discovery with more data sources: Copilot Studio telemetry (agent session counts, last conversation date), Power Platform admin center (environment-level deployment status), and Microsoft 365 usage analytics.

Phase 2: Classify Copilot Studio agents

Classify each Copilot Studio agent by its usage level to determine the migration approach:

Expand table

Usage	Signals	Recommended action
Low	No sign-in activity in 30+ days, no active conversations, agent in draft or unpublished state.	Candidate for cleanup. Decommission the agent and its service principal directly; no migration needed.
Medium	Some recent conversation activity, noncritical business function, identifiable owner.	Candidate for recreation with Microsoft Entra Agent ID. Proceed through Phase 3 with standard validation.
High	Active daily conversations, production business workflows dependent on the agent, integrated with Teams or customer-facing channels.	**Don't recreate at this time.** See Production-critical agents.

The 30-day sign-in threshold is based on Microsoft Entra's default retention period. If your organization has extended audit log retention (through Microsoft Sentinel or another SIEM), adjust thresholds accordingly.

Production-critical agents

For Copilot Studio agents with high usage, **don't recreate them until Microsoft provides an official automated migration path**. There's no automated migration path, and the manual recreation process carries significant risk for production agents. Maintain these service principals as-is until an official automated migration path is available.

The risks specific to high-usage Copilot Studio agents include:

- **Channel disruption:** Teams channel configurations, web chat embeddings, and other deployments reference the existing service principal's application ID. Creating a new agent generates a new identity, which requires reconfiguring all channel deployments.
- **Connector re-authorization:** Connection references tied to the old service principal need to be reauthorized with the new Microsoft Entra Agent ID.
- **Flow dependencies:** Power Automate flows and other integrations that reference the agent's service principal break during the transition.
- **No configuration portability:** Agent topics, knowledge sources, and settings must be manually recreated in the new agent.

Phase 3: Recreate the agent with Microsoft Entra Agent ID

> ⚠ **Warning**
>
> In-place migration of an existing Copilot Studio service principal to a Microsoft Entra Agent ID-native agent identity isn't supported. You must create a new agent in Copilot Studio with Microsoft Entra Agent ID integration enabled. Configuration, channels, and internal IDs don't carry over automatically.

For **medium-usage** Copilot Studio agents, complete the following seven-step recreation procedure:

Step 1: Identify the agent

Find the corresponding agent in Copilot Studio for the service principal (SP) that you're replacing. Match the SP's display name and application ID with the agent listing in the **Copilot Studio admin center**.

Step 2: Document the current configuration

Before making any changes, document:

- All API permissions on the existing SP.
- Connection references and their authorization status.
- Channel deployments (Teams, web chat, and others).
- Power Automate flows or other downstream integrations.
- Any custom connector configurations.

Use this documentation as your checklist to re-establish every item on the new agent.

Step 3: Enable Microsoft Entra Agent ID integration

Ensure your tenant opts in to Microsoft Entra Agent ID integration for Copilot Studio. Verify this setting in the **Copilot Studio admin center** or through your tenant's feature settings.

Step 4: Create a new agent with Microsoft Entra Agent ID

In Copilot Studio, create a new agent with the Microsoft Entra Agent ID integration enabled. This new agent receives a native agent identity during provisioning. Recreate the agent's topics, knowledge sources, and configuration to match the legacy agent.

> ⓘ **Note**
>
> Republishing or re-registering an existing agent does **not** create a Microsoft Entra agent identity. You must create a new agent to get a Microsoft Entra Agent ID-native agent identity.

Step 5: Validate the new agent identity

Confirm the new agent identity appears correctly under the **Agents** area in the **Microsoft Entra admin center**. Verify:

- The agent identity is linked to the correct blueprint.
- The display name and metadata match your agent.
- The sponsor (owner) is correctly assigned.

Step 6: Reconfigure permissions, connections, and channels

Manually configure the new agent to match the legacy agent:

- Assign API permissions.
- Authorize connection references.
- Configure connector access.
- Set up channel deployments (Teams, web chat, and others).
- **Assign the correct owner and sponsor:** creating a new agent defaults the creator as the owner. Review and update ownership to ensure the appropriate individual or group is designated as the agent's sponsor for governance purposes.

> **Important**
>
> Channel configurations (Teams app manifest, web chat embed codes) reference the agent's application ID. You must update all channel integrations to point to the new agent's identity.

Step 7: Monitor the recreated agent in production

Monitor the new agent for **10–14 days** before proceeding to decommission the legacy agent. During this period, verify:

- Agent conversations function correctly.
- All connected data sources respond as expected.
- No authentication errors appear in Microsoft Entra sign-in logs.
- Channel integrations (Teams, web chat) work as expected.

Phase 4: Validate and decommission

After recreating the agent, validate the new identity end to end and then decommission the legacy service principal using the safeguards in this phase.

Validate the recreated agent before decommissioning

Use the following checklist to confirm the new agent is functioning correctly before decommissioning the legacy identity.

Expand table

Check	How to verify	Expected result
Agent conversations	Send test messages through all configured channels.	Agent responds correctly with no authentication errors.
Connection references	Trigger actions that use each connected data source.	All data sources return expected results.
Sign-in logs	**Microsoft Entra admin center** > **Sign-in logs**, filter by the new agent identity.	Sign-in events appear for the new agent identity with successful status.
Audit logs	**Microsoft Entra admin center** > **Audit logs**.	Agent ID creation events are logged.
Channel deployments	Test each channel (Teams, web chat, and others).	Agent is reachable and functional on all channels.
Power Automate flows	Trigger any flows that reference the agent.	Flows execute successfully with the new identity.

Parallel run (recommended for medium-usage agents)

Where possible, run the old and new agents side by side before fully cutting over. To run a parallel validation, follow these steps:

1. Keep the old agent published and functional while the new agent with Agent ID is being validated.
2. Direct a subset of users or test channels to the new agent.
3. Monitor both for errors, conversation failures, and permission issues over 10–14 days.
4. Once the new agent is fully validated, proceed to decommission.

Decommission safeguards

Before decommissioning the legacy SP:

- **Pre-deletion snapshot:** Export the SP's metadata, permissions, and audit history. This export provides a rollback reference if anything was missed.
- **Phased batches:** If you're decommissioning multiple Copilot Studio SPs, delete in small batches (20-50 SPs per wave).
- **Soft-delete first:** Use Microsoft Entra's soft-delete capability (30-day recycle bin) before hard deletion.

Decommission the legacy agent and service principal

> ⚠ **Warning**
>
> If you delete the app registration without first unpublishing or deleting the agent in Copilot Studio, the agent's authentication breaks. Always decommission the agent in Copilot Studio before touching the Microsoft Entra identity.

To decommission the legacy service principal, follow these steps:

1. **Unpublish or delete the agent in Copilot Studio.** This step ensures the platform stops authenticating on behalf of the agent.
2. Remove API permissions from the old app registration.
3. Delete or rotate client secrets and certificates on the old app registration.
4. Delete the app registration (enters 30-day soft-delete).
5. After 30 days, confirm permanent deletion or hard-delete if needed.

Troubleshoot common issues

The following table lists common issues you might encounter during the recreate-and-deprecate process and how to resolve them.

Expand table

Symptom	Likely cause	Resolution
New Agent ID doesn't appear in **Microsoft Entra admin center** after creating the agent	Agent ID integration might not be enabled for your tenant.	Verify tenant opt-in for Agent ID integration. Try creating the agent again. If the issue persists, contact Copilot Studio support.
Agent stops responding after switchover	Channel configurations still reference the old SP's application ID.	Update channel configurations to reference the new agent's identity. Check Teams app manifest and web chat embed codes.
Connection reference errors	Connectors were authorized against the old SP and need reauthorization with the new identity.	Reauthorize each connection reference in Copilot Studio using the new agent's identity.
Power Automate flows fail	Flows reference the old SP's application ID or service principal.	Update flow connections and triggers to use the new agent's identity.

Related content

- Migrate custom app registrations to Agent ID
- Agent identity concepts
- Create an agent identity blueprint
- Best practices for Agent ID

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/15/2026

Token claims reference for agents

Agents use specialized token claims to identify different entity types and their relationships during authentication and authorization flows. These claims enable proper attribution, policy evaluation, and audit trails for agent operations. This article outlines the token claims for agent applications, detailing how tokens identify agent entities and their roles in authentication flows.

Clients using agent identities are expected to treat the access tokens issued to them to use at resource servers as opaque, and not try to parse them. However, resource servers that receive access tokens issued to agents need to parse the tokens to validate them and extract claims for authorization purposes.

Core token claim types

Tokens issued for identities used for resource access include claims that you'd normally expect to see in access tokens that Microsoft Entra issues. For more information, see access token claim reference. The following example shows a sample access token issued to an agent acting autonomously.

JSON


```
{
  "aud": "f2510d34-8dca-4ab8-a0bc-aaec4d3a3e36",
  "iss": "https://sts.windows.net/00000001-0000-0ff1-ce00-000000000000/",
  "iat": 1753392285,
  "nbf": 1753392285,
  "exp": 1753421385,
  "aio": "Y2JgYGhn1nzmErKqi0vc4Fr6H22/C5/4FP+xZbZYpik8nRkp+gEA",
  "appid": "aaaaaaaa-1111-2222-3333-444444444444",
  "appidacr": "2",
  "idp": "https://sts.windows.net/00000001-0000-0ff1-ce00-000000000000/",
  "idtyp": "app",
  "oid": "bbbbbbbb-1111-2222-3333-444444444444",
  "rh": "1.AAAAAQAAAAAA8Q_OAAAAAAAAADQNUfLKjbhKoLyq7E06PjYAAAAAAA.",
  "sub": "cccccccc-1111-2222-3333-444444444444",
  "tid": "00000001-0000-0ff1-ce00-000000000000",
  "uti": "m5RaaRnoFUyp2TbSCAAAAA",
  "ver": "1.0",
  "xms_act_fct": "3 9 11",
  "xms_ftd": "Z5DrW4HFOkR_Lz0M5qETa260d2-fO6seMZJ_tOwRNuc",
  "xms_idrel": "7 10",
  "xms_sub_fct": "9 3 11",
  "xms_tnt_fct": "3 9",
```

```
  "xms_par_app_azp": "30cf4c22-9985-4ef7-8756-91cc888176bd"
}
```

In v2 tokens, you see `azp` instead of `appid`. They both refer to the application ID of the agent identity.

You'd notice that the token includes a few claims that aren't previously seen in access tokens issued to applications. The following optional claims are also supported to identify that the tokens are for agent identities. They also provide more context in which the agent identity is acting.

- `xms_tnt_fct`
- `xms_sub_fct`
- `xms_act_fct`
- `xms_par_app_azp`

Expand table

Claim Name	Description
`tid`	Tenant ID of the customer tenant where the agent identity is registered. It's the tenant where the token is valid.
`sub`	Subject (the user, service principal, or agent identity being authenticated)
`oid`	Object ID of the subject. User object ID for user delegation scenarios. Agent ID service principal OID for app-only scenarios. Agent's user account OID for user impersonation scenarios.
`idtyp`	Type of entity the subject is. Values are `user`, `app`.
`tid`	Tenant ID of the customer tenant where the agent identity is registered.
`xms_idrel`	Relationship between the subject and the resource tenant. Learn more.
`aud`	Audience (the API that the agent is trying to access)
`azp` or `appid`	Authorized party / actor. The application ID of the agent identity. Enables proper client attribution in audit logs.
`scp`	Scope. Delegated permissions for user-context tokens. Only present in user delegation and agent's user account scenarios. Empty or `/` for app-only scenarios
`xms_act_fct`	Actor facets claim. Learn more.
`xms_sub_fct`	Subject facets claim. Learn more .
`xms_tnt_fct`	Tenant facets claim. Learn more .

Claim Name	Description
`xms_par_app_azp`	Parent application of the authorized party. Learn more .

idtyp claim values by scenario

The `idtyp` claim identifies the type of entity the token subject represents. The value depends on which agent flow issued the token:

Expand table

Agent scenario	`idtyp` value	Subject (`sub` / `oid`)
On-behalf-of flow (interactive agent)	`user`	The human user the agent acts on behalf of
Autonomous app flow (app-only)	`app`	The agent identity service principal
Agent's user account flow	`user`	The agent's own user account

> **Note**
>
> The `idtyp` value alone doesn't distinguish a human user from an agent's user account. Use the `xms_sub_fct` claim (value `13` = agent's user account) to differentiate these scenarios.

xms_idrel

The `xms_idrel` claim indicates the identity relationship between the entity for which the token is issued and the resource tenant.

Here are the possible values for the `xms_idrel` claim. It's a multivalued claim, meaning it can have multiple values, separated by spaces. The values are represented as integers. The valid values are always odd numbers starting from 1.

Expand table

Claim Value	Description
`1`	Member user
`3`	MSA member user
`5`	Guest user

Claim Value	Description
`7`	Service principal
`9`	Device principal
`11`	GDAP user
`13`	SPLess application
`15`	Passthrough
`17`	Native identity user with profile
`19`	Native identity user
`21`	Native identity Teams meeting participant
`23`	Passthrough authenticated Teams meeting participant
`25`	Native identity content sharing user
`27`	Fully synced MTO member
`29`	Weak MTO user
`31`	DAP user
`33`	Federated managed identity

xms_tnt_fct, xms_sub_fct, and xms_act_fct claims

The `xms_tnt_fct` claim describes the tenant (identified by the `tid` claim). The `xms_sub_fct` and `xms_act_fct` claims are used to describe facts about the subject (`sub`) and the actor (`azp` or `appid`) of the token, respectively. These claims provide more context about the agent's identity and the actions it's performing.

Here are the relevant values for these claims. These claims are multivalued, meaning they can have multiple values, separated by spaces. The valid values are always odd numbers starting from 1.

Expand table

Claim Value	Description
`11`	AgentIdentity

Claim Value	Description
`13`	AgentIDUser

You should ignore any values that aren't relevant to your scenario or validation logic. Ignore the values that aren't relevant to your application. Don't assume any order of the values in these claims.

xms_par_app_azp

The `xms_par_app_azp` claim is used to identify the parent application of the authorized party (`azp` or `appid`). It's a GUID, when included. You can use the claim to determine the parent

Log the parent application ID for auditing purposes. Microsoft Entra ID sign-in logs always includes the parent ID if available, so resource server should do the same. It isn't recommended to use the parent application ID for authorization decisions, as it would result in widespread access by many agents.

Scenario-wise examples

The following section outlines some auth scenarios and the relevant claims for each one of them.

Agent identity acting on behalf of a human user

In this scenario, the agent identity is acting on behalf of a human user. The access token includes the following claims:

Expand table

Claim Name	Description
`tid`	Tenant ID of the customer tenant
`idtyp`	`user` (indicating the subject is a user)
`xms_idrel`	`1` (indicating a member user; others possible too)
`azp` / `appid`	Application ID of the agent identity
`scp`	Delegated permissions granted to the agent identity
`oid`	Object ID of the user
`aud`	Resource audience for the token

Claim Name	Description
`xms_act_fct`	`11` (AgentIdentity)

Agent identity acting autonomously

In this scenario, the agent identity acts using its own identity. The access token includes the following claims:

Expand table

Claim Name	Description
`tid`	Tenant ID of the customer tenant
`idtyp`	`app` (indicating the subject is an application)
`xms_idrel`	`7` (indicating a service principal)
`azp` / `appid`	Application ID of the agent identity
`roles`	Permissions granted to the agent identity
`oid`	Object ID of the agent identity
`xms_act_fct`	`11` (AgentIdentity)
`xms_sub_fct`	`11` (AgentIdentity)
`aud`	Resource audience for the token
`scp`	Empty or `/` (unscoped).

Agent identity acts autonomously via agent's user account

In this scenario, the agent obtains a token using the agent's user account associated with its agent identity. The access token includes the following claims:

Expand table

Claim Name	Description
`tid`	Tenant ID of the customer tenant
`idtyp`	`user` (indicating the subject is a user)
`xms_idrel`	`1` (indicating a member user; others possible too)

Claim Name	Description
`azp` / `appid`	Application ID of the agent identity
`scp`	Delegated permissions granted to the agent identity
`oid`	Object ID of the agent's user account
`xms_act_fct`	`11` (Agent identity)
`xms_sub_fct`	`13` (Agent's user account)
`aud`	Resource audience for the token

Last updated on 06/11/2026

Frequently asked questions about Microsoft Entra Agent ID

Microsoft Entra Agent ID is an identity and security framework that extends Microsoft Entra capabilities to AI agents. As organizations deploy assistive, autonomous, and user-like agents, they need purpose-built identity constructs to authenticate, authorize, govern, and protect these nonhuman identities. Microsoft Entra Agent ID addresses these needs by providing a unified platform for managing agent identities at enterprise scale.

Agent identities and blueprints

How do I filter Microsoft Graph API queries to return only agent identities?

Microsoft Graph APIs that support relationships involving agent identities such as `/ownedObjects`, `/deletedItems`, and `/owners` don't support filtering by entity type. Use the existing APIs and filter results on the client side using the `odata.type` property to identify agent identity objects in the response.

What happens to an agent's user account when an agent identity or blueprint is deleted?

When an agent identity blueprint or agent identity is deleted, any associated agents' user accounts remain in the tenant. They aren't shown as disabled or deleted, though they can't authenticate. Delete orphaned agents' user accounts manually using Microsoft Graph API or Microsoft Entra PowerShell.

Why do sequential Microsoft Graph API requests sometimes fail when creating agent identity objects?

When creating agent identity objects in quick succession using Microsoft Graph APIs, requests might fail with errors like `400 Bad Request: Object with id {id} not found`. Common sequences that trigger this behavior include:

- Creating an agent identity blueprint, then immediately creating a blueprint principal.

- Creating a blueprint principal, then immediately using the blueprint to create an agent identity.
- Creating an agent identity, then immediately creating an agent's user account.

These failures are more common when using app-only permissions. Use delegated permissions where possible, and add retry logic with exponential backoff to your requests.

Are there limits on the number of agent identity blueprints per tenant?

The following limits apply to agent identity blueprints:

- Non-Microsoft management platforms using app-only permissions are limited to 250 agent identities per blueprint. Delegated calls and Microsoft-owned platforms (Foundry, Copilot Studio) aren't subject to this cap.
- Non-admin users are subject to the preexisting 250-owned-objects limit in Microsoft Entra ID, which applies across all Microsoft Entra resource types.
- Blueprints can take up no more than 95% of the tenant's overall resource quota. For more information, see the **Resources** row in Microsoft Entra service limits and restrictions.

How do I know when an admin approves my agent identity blueprint in their tenant?

There are no built-in notification mechanisms for agent identity blueprint approval. When a tenant admin creates or approves an agent identity blueprint for your agent, you aren't notified through Microsoft Entra or Microsoft Graph.

To check whether your blueprint has been approved in a specific tenant, query the Microsoft Graph API for blueprint principal objects associated with your application. If an admin hasn't approved the blueprint yet, the query returns no results for that tenant.

Can I delete or restore agent identity objects in the Microsoft Entra admin center?

No. Deleting and restoring agent identity objects isn't supported in the Microsoft Entra admin center. Use the Microsoft Graph API or Microsoft Entra PowerShell instead.

Can I recover a permanently deleted agent identity?

No. Permanently deleted objects can't be restored. Soft-deleted objects can be restored within the 30-day retention period; after that window, permanent deletion happens automatically and is irreversible.

Can I permanently delete an agent identity blueprint principal to free quota immediately?

No. Permanent deletion of an agent identity blueprint principal is blocked. To free the quota it uses, wait for the 30-day retention period to expire.

Roles, permissions, and groups

Can I add agent identities to administrative units?

Agent identities, agent identity blueprints, and agent identity blueprint principals can't be added to administrative units. Use the `owners` property of agent identities to limit which users can manage specific objects.

Can I update the photo for an agent's user account?

The *Agent ID Administrator* role doesn't have permission to update photos for an agent's user account. Use the *User Administrator* role for this task.

Can I use dynamic groups to manage an agent's user account?

An agent's user account can be added to Microsoft Entra groups, including dynamic groups, enabling it to inherit access granted to those group.

Which Microsoft Entra roles and groups can't be assigned to an agent identity?

Highly privileged directory roles are blocked for agent identities, including Global Administrator, Privileged Role Administrator, and User Administrator. Only lower-privileged roles, such as reader roles, can be assigned. Agent identities can't be members of role-assignable groups.

Authentication and consent

Can agent identities sign in to web apps using single sign-on (SSO)?

Agent identities can't sign in to Microsoft Entra ID sign-in pages, which means they can't use single sign-on with OpenID Connect or SAML protocols. Use available web APIs to integrate agents with workplace apps and services.

Does the admin consent workflow work for Microsoft Entra Agent ID permission requests?

The Microsoft Entra ID admin consent workflow doesn't work correctly for permissions requested by agent identities. Users should contact their Microsoft Entra tenant admin to request that permissions be granted directly to the agent identity.

What should I do if a user consent is blocked by risk-based step-up?

Risk-based step-up is enforced for agent identity consent flows. If a user's consent is blocked, there's no workaround. The user must resolve the flagged risk before consent can proceed.

Can an administrator consent to high-privilege Microsoft Graph permissions for an agent?

No. A set of high-risk Microsoft Graph permissions — including `Application.ReadWrite.All`, `RoleManagement.ReadWrite.All`, `User.ReadWrite.All`, and `Directory.AccessAsUser.All` — is blocked for agents and can't be granted through Microsoft Graph or the Microsoft Entra admin center. Requests for these permissions are rejected. Agents can still receive lower-privilege, scoped permissions that a user or administrator consents to.

Monitoring and logs

How can I identify agent identity activities in audit logs?

Audit logs don't distinguish agent identities from other Microsoft Entra identity types by default:

- Operations on agent identities, blueprints, and blueprint principals are logged in the *ApplicationManagement* category.
- Operations on agents' user accounts are logged in the *User Management* category.
- Operations initiated by agent identities appear as service principals.
- Operations initiated by agents' user accounts appear as users.

To identify Agent ID-related activity, use object IDs from audit logs to query Microsoft Graph and determine the entity type. You can also use the sign-in logs correlation ID to locate the identity of the actor or subject involved in the activity.

How do I identify agent identities in Microsoft Graph activity logs?

Microsoft Graph activity logs don't currently separate agent identities from other identity types:

- Requests from agent identities are logged as applications, with the agent identity included in the *appID* column.
- Requests from agents' user accounts are logged as users, with the agent user ID in the *UserID* column.

Join with Microsoft Entra sign-in logs to determine the entity type.

Development resources

Are there SDKs or libraries available for Microsoft Entra Agent ID scenarios?

The SDK you use depends on your scenario:

Microsoft Agent 365 CLI and SDK are the recommended starting point for most developers. The CLI handles agent identity provisioning, blueprint creation, and permission wiring in a single command. The SDK handles token acquisition at runtime. For more information, see Microsoft Entra Agent 365 SDK documentation.

Microsoft.Identity.Web provides higher-level APIs for acquiring tokens for agent identities in .NET applications. Use the Microsoft.Identity.Web.AgentIdentities package to simplify managing agent identities.

The Microsoft Entra ID Auth SDK (sidecar) provides Microsoft Entra authentication and token-management capabilities as a containerized web service that runs alongside your application. Use it when your agent runs in a containerized environment (such as Kubernetes, Docker, or Azure container services), or when the agent isn't built in .NET and you want a language-agnostic HTTP interface. For .NET applications, Microsoft.Identity.Web provides the equivalent in-process integration. For more information, see Microsoft Entra ID Auth SDK (sidecar).

Microsoft Graph APIs provide agent identity management when the other options don't fit your scenario. For more information, see Microsoft Graph API for agent identity blueprints.

Last updated on 10/01/2026

Microsoft Graph permissions reference

For an app to access data in Microsoft Graph, the user or administrator must grant it the necessary permissions. This article lists the delegated and application permissions exposed by Microsoft Graph. For guidance about how to use the permissions, see the Overview of Microsoft Graph permissions.

To read information about all Microsoft Graph permissions programmatically, sign in to an API client such as Graph Explorer using an account that has at least the *Application.Read.All* permission and run the following request.

msgraph

```
GET https://graph.microsoft.com/v1.0/servicePrincipals(appId='00000003-0000-0000-c000-000000000000')?
$select=id,appId,displayName,appRoles,oauth2PermissionScopes,resourceSpecificApplicationPermissions
```

> **Note**
>
> As a best practice, request the least privileged permissions that your app needs in order to access data and function correctly. Requesting permissions with more than the necessary privileges is poor security practice, which may cause users to refrain from consenting and affect your app's usage.

All permissions

AccessReview.Read.All

Expand table

Category	Application	Delegated
Identifier	d07a8cc0-3d51-4b77-b3b0-32704d1f69fa	ebfcd32b-babb-40f4-a14b-42706e83bd28
DisplayText	Read all access reviews	Read all access reviews that user can access

Category	Application	Delegated
Description	Allows the app to read access reviews, reviewers, decisions and settings in the organization, without a signed-in user.	Allows the app to read access reviews, reviewers, decisions and settings that the signed-in user has access to in the organization.
AdminConsentRequired	Yes	Yes

AccessReview.ReadWrite.All

Expand table

Category	Application	Delegated
Identifier	ef5f7d5c-338f-44b0-86c3-351f46c8bb5f	e4aa47b9-9a69-4109-82ed-36ec70d85ff1
DisplayText	Manage all access reviews	Manage all access reviews that user can access
Description	Allows the app to read, update, delete and perform actions on access reviews, reviewers, decisions and settings in the organization, without a signed-in user.	Allows the app to read, update, delete and perform actions on access reviews, reviewers, decisions and settings that the signed-in user has access to in the organization.
AdminConsentRequired	Yes	Yes

AccessReview.ReadWrite.Membership

Expand table

Category	Application	Delegated
Identifier	18228521-a591-40f1-b215-5fad4488c117	5af8c3f5-baca-439a-97b0-ea58a435e269
DisplayText	Manage access reviews for group and app memberships	Manage access reviews for group and app memberships
Description	Allows the app to read, update, delete and perform actions on access reviews, reviewers, decisions and settings in the organization for group	Allows the app to read, update, delete and perform actions on access reviews, reviewers, decisions and settings for group and app memberships that the

Category	Application	Delegated
	and app memberships, without a signed-in user.	signed-in user has access to in the organization.
AdminConsentRequired	Yes	Yes

Acronym.Read.All

Expand table

Category	Application	Delegated
Identifier	8c0aed2c-0c61-433d-b63c-6370ddc73248	9084c10f-a2d6-4713-8732-348def50fe02
DisplayText	Read all acronyms	Read all acronyms that the user can access
Description	Allows an app to read all acronyms without a signed-in user.	Allows an app to read all acronyms that the signed-in user can access.
AdminConsentRequired	Yes	No

AdministrativeUnit.Read.All

Expand table

Category	Application	Delegated
Identifier	134fd756-38ce-4afd-ba33-e9623dbe66c2	3361d15d-be43-4de6-b441-3c746d05163d
DisplayText	Read all administrative units	Read administrative units
Description	Allows the app to read administrative units and administrative unit membership without a signed-in user.	Allows the app to read administrative units and administrative unit membership on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AdministrativeUnit.ReadWrite.All

Expand table

Category	Application	Delegated
Identifier	5eb59dd3-1da2-4329-8733-9dabdc435916	7b8a2d34-6b3f-4542-a343-54651608ad81
DisplayText	Read and write all administrative units	Read and write administrative units
Description	Allows the app to create, read, update, and delete administrative units and manage administrative unit membership without a signed-in user.	Allows the app to create, read, update, and delete administrative units and manage administrative unit membership on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentCard.Read.All

Expand table

Category	Application	Delegated
Identifier	aec9e0a0-6f46-4150-a9f7-05e9e3e87399	73ea6732-992c-4292-98f7-9feff18d3ade
DisplayText	Read all agent cards in Agent Registry	Read agent cards in Agent Registry
Description	Allows the app to read all agent cards and their skills in your organization's Agent Registry without a signed-in user.	Allows the app to read agent cards and their skills in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentCard.ReadWrite.All

Expand table

Category	Application	Delegated
Identifier	ef566853-42d6-45a5-bed9-5ccb82c98b4f	b0f726a8-0fa2-4ce2-937b-fd17a446261f
DisplayText	Read and write all agent cards in Agent	Read and write agent cards in Agent

Category	Application	Delegated
	Registry	Registry
Description	Allows the app to create, read, update, and delete all agent cards and manage their skills in your organization's Agent Registry without a signed-in user.	Allows the app to create, read, update, and delete agent cards and manage their skills in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentCard.ReadWrite.ManagedBy

Expand table

Category	Application	Delegated
Identifier	9c4a07db-e0c1-4fb0-8e85-dfd8ae3b8201	-
DisplayText	Read and write managed-by agent cards in Agent Registry	-
Description	Allows the app to read and update agent cards that designate the calling app as their manager and manage their skills in your organization's Agent Registry without a signed-in user.	-
AdminConsentRequired	Yes	-

AgentCardManifest.Read.All

Expand table

Category	Application	Delegated
Identifier	3ee18438-e6e5-4858-8f1c-d7b723b45213	ada96a26-9579-4c29-a578-c3482a765716
DisplayText	Read all agent card manifests in Agent Registry	Read all agent card manifests in Agent Registry
Description	Allows the app to read all agent card manifests in your organization's Agent Registry without a signed-in user.	Allows the app to read agent card manifests in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentCardManifest.ReadWrite.All

Expand table

Category	Application	Delegated
Identifier	228b1a03-f7ca-4348-b50d-e8a547ab61af	80151b1a-1c31-4846-ae0d-c79939ee13d1
DisplayText	Read and write all agent card manifests in Agent Registry	Read and write agent card manifests in Agent Registry
Description	Allows the app to read and write to all agent card manifests in your organization's Agent Registry without a signed-in user.	Allows the app to read and write agent card manifests in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentCardManifest.ReadWrite.ManagedBy

Expand table

Category	Application	Delegated
Identifier	77f6034c-52f5-4526-9fa1-d55a67e72cc4	-
DisplayText	Read and write managed-by agent card manifests in Agent Registry	-
Description	Allows the app to read and write agent card manifests that name it as manager in your organization's Agent Registry without a signed-in user.	-
AdminConsentRequired	Yes	-

AgentCollection.Read.All

Expand table

Category	Application	Delegated
Identifier	e65ee1da-d1d5-467b-bdd0-3e9bb94e6e0c	fa50be38-fdff-469c-96dc-ef5fce3c64bf

Category	Application	Delegated
DisplayText	Read all collections in Agent Registry, except quarantined and global	Read all collections in Agent Registry, except quarantined and global
Description	Allows the app to read all collections and their membership in your organization's Agent Registry without a signed-in user.	Allows the app to read collections and their membership in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentCollection.Read.Global

Expand table

Category	Application	Delegated
Identifier	-	b14924c8-87f1-438a-81f2-dc370ba2f45d
DisplayText	-	Read global collection in Agent Registry
Description	-	Allows the app to read global collection and its membership in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AgentCollection.Read.Quarantined

Expand table

Category	Application	Delegated
Identifier	-	43acfda3-daf3-4aa4-955d-b051d0024e82
DisplayText	-	Read quarantined collection in Agent Registry
Description	-	Allows the app to read quarantined collection and its membership in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AgentCollection.ReadWrite.All

Category	Application	Delegated
Identifier	feb31d7d-a227-4487-898c-e014840d07b3	6d8a7002-a05e-4b95-a768-0e6f0badc6c8
DisplayText	Read and write all collections in Agent Registry, except quarantined and global	Read and write collections in Agent Registry, except quarantined and global
Description	Allows the app to create, read, update, and delete all collections and manage their membership in your organization's Agent Registry without a signed-in user.	Allows the app to create, read, update, and delete collections and manage their membership in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentCollection.ReadWrite.Global

Category	Application	Delegated
Identifier	-	c001dd65-8a6b-4349-ab0c-4e8a410d28d2
DisplayText	-	Read and write global collection in Agent Registry
Description	-	Allows the app to read and update global collection and manage its membership in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AgentCollection.ReadWrite.ManagedBy

Category	Application	Delegated
Identifier	2e0fb698-9996-479f-926b-ce63f4397829	-
DisplayText	Read and write managed-by collections in Agent Registry	-
Description	Allows the app to create, read, update, and delete collections that designate the calling app as their manager and manage their membership in your organization's Agent Registry without a signed-in user.	-
AdminConsentRequired	Yes	-

AgentCollection.ReadWrite.Quarantined

Category	Application	Delegated
Identifier	-	ae331cc9-9f51-484b-a90b-124f2e4a6398
DisplayText	-	Read and write quarantined collection in Agent Registry
Description	-	Allows the app to read and update quarantined collection and manage its membership in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AgentCommunicationConfiguration.Read

Category	Application	Delegated
Identifier	-	57ff8075-2fb3-4879-8693-5d51ee8d5e9e
DisplayText	-	Read agent communication configuration
Description	-	Allows the app to read the communication configuration of agent blueprints on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AgentCommunicationConfiguration.Read.All

Category	Application	Delegated
Identifier	eccf3f2d-f81a-4718-95d7-ef4a0c42ac43	-
DisplayText	Read all agent communication configurations	-
Description	Allows the app to read the communication configuration of agent blueprints without a signed-in user.	-
AdminConsentRequired	Yes	-

AgentCommunicationConfiguration.ReadWrite

Category	Application	Delegated
Identifier	-	15e0db35-0641-4175-b014-c2cb39286338
DisplayText	-	Read and write agent communication configuration
Description	-	Allows the app to read and update the communication configuration of agent blueprints on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AgentCommunicationConfiguration.ReadWrite.All

Category	Application	Delegated
Identifier	9c72696d-c77b-4d8d-b59e-dfca4792c9ec	-
DisplayText	Read and write all agent communication configurations	-
Description	Allows the app to read and update the communication configuration of agent blueprints without a signed-in user.	-
AdminConsentRequired	Yes	-

AgentIdentity.Create.All

Category	Application	Delegated
Identifier	ad25cc1d-84d8-47df-a08e-b34c2e800819	e75eeac6-d759-4ba3-ae5c-773a27efafba
DisplayText	Create agent identities without an agent blueprint parent	Create agent identities without an agent blueprint parent
Description	Allows the client to create agent identities without a signed-in user, even if the client is not the parent agent identity blueprint.	Allows the client to create agent identities on behalf of the signed-in user, even if the client is not the parent agent identity blueprint.
AdminConsentRequired	Yes	Yes

AgentIdentity.CreateAsManager

Category	Application	Delegated
Identifier	4c390976-b2b7-42e0-9187-c6be3bead001	-
DisplayText	Create and manage agent identities as the parent agent identity blueprint	-
Description	Allows the app to create agent identities as the parent agent identity blueprint and fully manage them, including reading, updating, and deleting, without a signed-in user.	-
AdminConsentRequired	Yes	-

AgentIdentity.DeleteRestore.All

Category	Application	Delegated
Identifier	5b016f9b-18eb-41d4-869a-66931914d1c8	c8ee41e5-35e7-4fe9-8ecb-93493adcac5b
DisplayText	Delete and restore agent identities	Delete and restore agent identities
Description	Allows the client to delete and restore agent identities without a signed-in user.	Allows the client to delete and restore agent identities on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentity.EnableDisable.All

Category	Application	Delegated
Identifier	69ee0943-4fa4-4ec8-8e52-d12e4ea661a3	a501206a-e364-4a3f-be6e-765806d0e323
DisplayText	Enable or disable agent identities	Enable or disable agent identities
Description	Allows the client to enable or disable agent identities without a signed-in user.	Allows the client to enable or disable agent identities on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentity.Read.All

Category	Application	Delegated
Identifier	b2b8f011-2898-4234-9092-5059f6c1ebfa	5e850691-d86a-4b24-bfa6-8a52fb37a0c1
DisplayText	Read all agent identities	Read all agent identities
Description	Allows the app to read all agent identities without a signed-in user.	Allows the client to read all agent identities on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentity.ReadWrite.All

Category	Application	Delegated
Identifier	dcf7150a-88d4-4fe6-9be1-c2744c455397	4a4facd5-0ee1-49b7-a5b2-fdcc2491685e
DisplayText	Read and write all agent identities	Read and write all agent identities
Description	Allows the client to read, update, create, and delete agent identities without a signed-in user.	Allows the client to read, update, create, and delete agent identities on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprint.AddRemoveCreds.All

Category	Application	Delegated
Identifier	0510736e-bdfb-4b37-9a1f-89b4a074763a	75b5feb2-bfe7-423f-907d-cc505186f246
DisplayText	Update agent identity blueprint credentials	Update agent identity blueprint credentials
Description	Allows updating agent identity blueprint credentials without a signed-in user.	Allows updating agent identity blueprint credentials on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprint.Create

Category	Application	Delegated
Identifier	ea4b2453-ad2d-4d94-9155-10d5d9493ce9	8fc15edd-ba24-494e-9bf6-d38e1b7ba8fd
DisplayText	Create agent identity blueprints.	Create agent identity blueprints.
Description	Allows creating new agent identity blueprints without a signed-in user.	Allows creating new agent identity blueprints on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprint.DeleteRestore.All

Category	Application	Delegated
Identifier	3f80b699-6405-4e36-a4df-4f19950ff91e	f12ba1f6-afb7-4685-9a30-21e8c3f551d8
DisplayText	Delete and restore agent identity blueprints.	Delete and restore agent identity blueprints.
Description	Allows deleting or restoring agent identity blueprints without a signed-in user.	Allows deleting or restoring agent identity blueprints on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprint.Read.All

Category	Application	Delegated
Identifier	7547a7d1-36fa-4479-9c31-559a600eaa4f	26512dc8-1364-4e9f-867c-6d8b22a9e162
DisplayText	Read all agent identity blueprints	Read all agent identity blueprints
Description	Allows the client to read all agent identity blueprints without a signed-in user.	Allows the client to read all agent identity blueprints on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprint.ReadWrite.All

Category	Application	Delegated
Identifier	7fddd33b-d884-4ec0-8696-72cff90ff825	4fd490fc-1467-48eb-8a4c-421597ab0402
DisplayText	Read and write all agent identity blueprints.	Read and write all agent identity blueprints.
Description	Allows the client to read, update, create, and delete agent identity blueprints without a signed-in user.	Allows the client to read, update, create, and delete agent identity blueprints on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprint.UpdateAuthProperties.All

Category	Application	Delegated
Identifier	19202363-278e-49c2-bf00-391e2ba00881	6f677aa9-25af-49a5-8a1d-628dc7f0d009
DisplayText	Update agent identity blueprint authorization and authentication properties	Update agent identity blueprint authorization and authentication properties
Description	Allows updating agent identity blueprint authorization and authentication properties without a signed-in user.	Allows updating agent identity blueprint authorization and authentication properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprint.UpdateBranding.All

Category	Application	Delegated
Identifier	76232daa-a1e4-4544-b664-495a006513bf	60960e31-67cb-4d25-9d36-4922109923a2
DisplayText	Update agent identity blueprint branding	Update agent identity blueprint branding

Category	Application	Delegated
Description	Allows updating agent identity blueprint branding without a signed-in user.	Allows updating agent identity blueprint branding on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprintPrincipal.Create

Category	Application	Delegated
Identifier	8959696d-d07e-4916-9b1e-3ba9ce459161	00dcd896-6b23-42ce-b5de-c58493c05e22
DisplayText	Create agent identity blueprint principals.	Create agent identity blueprint principals.
Description	Allows creating new agent identity blueprint principals without a signed-in user.	Allows creating new agent identity blueprint principals on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprintPrincipal.DeleteRestore.All

Category	Application	Delegated
Identifier	f86a2dd8-9298-4675-bd78-f5a3572da2d7	2c70023e-a482-4af2-9ff1-51ded53e6bad
DisplayText	Delete and restore agent identity blueprint principals.	Delete and restore agent identity blueprint principals.
Description	Allows deleting or restoring agent identity blueprint principals without a signed-in user.	Allows deleting or restoring agent identity blueprint principals on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprintPrincipal.EnableDisable.All

Category	Application	Delegated
Identifier	a0bdd23d-8b19-4682-b428-574d96527c6f	e7475e0a-9f02-43e2-a250-5c2ea74ccd0e
DisplayText	Enable or disable agent identity blueprint principals.	Enable or disable agent identity blueprint principals.
Description	Allows enabling or disabling agent identity blueprint principals without a signed-in user.	Allows enabling or disabling agent identity blueprint principals on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprintPrincipal.Read.All

Category	Application	Delegated
Identifier	9361dea9-4524-493d-941d-f1b65aaf6c7c	88c856a2-de61-4632-b2d4-ac503cbc8dd2
DisplayText	Read agent identity blueprint principals.	Read agent identity blueprint principals.
Description	Allows reading agent identity blueprint principals without a signed-in user.	Allows reading agent identity blueprint principals on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdentityBlueprintPrincipal.ReadWrite.All

Category	Application	Delegated
Identifier	3bc933bc-8b4d-4cb6-ac49-b73774299250	bf2cad6a-9082-438a-9a63-95fa2687af65
DisplayText	Read and write all agent identity blueprint principals.	Read and write all agent identity blueprint principals.
Description	Allows the app to read, update, create, and delete agent identity blueprint principals without a signed-in user.	Allows the app to read, update, create, and delete agent identity blueprint principals on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdUser.ReadWrite.All

Category	Application	Delegated
Identifier	b782c9ad-6f2b-4894-a21b-72bf22417f0a	ad57fb88-4658-4fd6-ab7d-e43184b08e4e
DisplayText	Read and write all agent ID users' full profiles	Read and write all agent ID users' full profiles
Description	Allows the app to create agent users, read and update agent ID user profiles and read basic company properties, delete and restore agent users without a signed in user.	Allows the app to create agent users, read and write the full set of profile properties, reports, and managers of agent ID users, delete and restore agent users in your organization, and read basic company properties, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentIdUser.ReadWrite.IdentityParentedBy

Category	Application	Delegated
Identifier	4aa6e624-eee0-40ab-bdd8-f9639038a614	52a417d9-0b3c-4466-9a3b-66960de73d74
DisplayText	Read and write full profiles of agent ID users under an agent blueprint	Read and write full profiles of agent ID users under an agent blueprint
Description	Allows the app to create agent users, read and update agent ID user profiles, delete and restore agent users under an agent blueprint, and read basic company properties without a signed-in user.	Allows the app to create agent users, read and write the full set of profile properties, reports, and managers of agent ID users in your organization, delete and restore agent users under an agent blueprint, and read basic company properties, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentInstance.Read.All

Category	Application	Delegated
Identifier	799a4732-85b8-4c67-b048-75f0e88a232b	4c3c738a-2df0-4877-bf4a-f796950ff34c
DisplayText	Read all agent instances in Agent Registry	Read all agent instances in Agent Registry
Description	Allows the app to read all agent instances and their related collections in your organization's Agent Registry without a signed-in user.	Allows the app to read agent instances and their related collections in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentInstance.ReadWrite.All

Category	Application	Delegated
Identifier	07abdd95-78dc-4353-bd32-09f880ea43d0	fc79e324-da24-497a-b5ec-e7de08320375
DisplayText	Read and write all agent instances in Agent Registry	Read and write agent instances in Agent Registry
Description	Allows the app to create, read, update, and delete all agent instances in your organization's Agent Registry without a signed-in user.	Allows the app to create, read, update, and delete agent instances in your organization's Agent Registry on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgentInstance.ReadWrite.ManagedBy

Category	Application	Delegated
Identifier	782ab1bf-24f1-4c27-8bbc-2006d42792a6	-
DisplayText	Read and write managed-by agent instances in Agent Registry	-
Description	Allows the app to create, read, update, and delete agent instances that designate the calling app as their manager in your organization's Agent Registry without a signed-in user.	-
AdminConsentRequired	Yes	-

AgentRegistration.Read.All

Category	Application	Delegated
Identifier	d3acceb6-4673-47c0-aeac-582f2c7cf72c	ef96ce0b-b2ea-4ae4-a783-108212d8ecee
DisplayText	Read all agent registrations	Read all agent registrations
Description	Allows the app to read agent registration information without a signed-in user.	Allows the user to read all agent registration information
AdminConsentRequired	Yes	No

AgentRegistration.ReadWrite.All

Category	Application	Delegated
Identifier	39fb8c64-7bd3-4107-8515-14d6e55ddda4	20f263bf-7d50-4e66-912c-16b4b4194fd4
DisplayText	Read and write all agent registrations	Read and write all agent registrations
Description	Allows the app to read and write agent registration information without a signed-in user.	Allows the user to read and write all agent registration information
AdminConsentRequired	Yes	Yes

Agreement.Read.All

Category	Application	Delegated
Identifier	2f3e6f8c-093b-4c57-a58b-ba5ce494a169	af2819c9-df71-4dd3-ade7-4d7c9dc653b7
DisplayText	Read all terms of use agreements	Read all terms of use agreements
Description	Allows the app to read terms of use agreements, without a signed in user.	Allows the app to read terms of use agreements on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Agreement.ReadWrite.All

Category	Application	Delegated
Identifier	c9090d00-6101-42f0-a729-c41074260d47	ef4b5d93-3104-4664-9053-a5c49ab44218
DisplayText	Read and write all terms of use agreements	Read and write all terms of use agreements
Description	Allows the app to read and write terms of use agreements, without a signed in user.	Allows the app to read and write terms of use agreements on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AgreementAcceptance.Read

Category	Application	Delegated
Identifier	-	0b7643bb-5336-476f-80b5-18fbfbc91806
DisplayText	-	Read user terms of use acceptance statuses
Description	-	Allows the app to read terms of use acceptance statuses on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AgreementAcceptance.Read.All

Category	Application	Delegated
Identifier	d8e4ec18-f6c0-4620-8122-c8b1f2bf400e	a66a5341-e66e-4897-9d52-c2df58c2bfb9
DisplayText	Read all terms of use acceptance statuses	Read terms of use acceptance statuses that user can access
Description	Allows the app to read terms of use acceptance statuses, without a signed in user.	Allows the app to read terms of use acceptance statuses on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AiEnterpriseInteraction.Read

Category	Application	Delegated
Identifier	-	859cceb9-2ec2-4e48-bcd7-b8490b5248a5
DisplayText	-	Read user AI enterprise interactions.
Description	-	Allows the app to read user AI enterprise interactions, on behalf of the signed-in user.
AdminConsentRequired	-	No

AiEnterpriseInteraction.Read.All

Category	Application	Delegated
Identifier	839c90ab-5771-41ee-aef8-a562e8487c1e	-
DisplayText	Read all AI enterprise interactions.	-
Description	Allows the app to read all AI enterprise interactions.	-
AdminConsentRequired	Yes	-

Analytics.Read

Category	Application	Delegated
Identifier	-	e03cf23f-8056-446a-8994-7d93dfc8b50e
DisplayText	-	Read user activity statistics
Description	-	Allows the app to read the signed-in user's activity statistics, such as how much time the user has spent on emails, in meetings, or in chat sessions.
AdminConsentRequired	-	No

APIConnectors.Read.All

Category	Application	Delegated
Identifier	b86848a7-d5b1-41eb-a9b4-54a4e6306e97	1b6ff35f-31df-4332-8571-d31ea5a4893f
DisplayText	Read API connectors for authentication flows	Read API connectors for authentication flows
Description	Allows the app to read the API connectors used in user authentication flows, without a signed-in user.	Allows the app to read the API connectors used in user authentication flows, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

APIConnectors.ReadWrite.All

Category	Application	Delegated
Identifier	1dfe531a-24a6-4f1b-80f4-7a0dc5a0a171	c67b52c5-7c69-48b6-9d48-7b3af3ded914
DisplayText	Read and write API connectors for authentication flows	Read and write API connectors for authentication flows
Description	Allows the app to read, create and manage the API connectors used in user authentication flows, without a signed-in user.	Allows the app to read, create and manage the API connectors used in user authentication flows, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AppCatalog.Read.All

Category	Application	Delegated
Identifier	e12dae10-5a57-4817-b79d-dfbec5348930	88e58d74-d3df-44f3-ad47-e89edf4472e4
DisplayText	Read all app catalogs	Read all app catalogs
Description	Allows the app to read apps in the app catalogs without a signed-in user.	Allows the app to read the apps in the app catalogs.
AdminConsentRequired	Yes	No

AppCatalog.ReadWrite.All

Category	Application	Delegated
Identifier	dc149144-f292-421e-b185-5953f2e98d7f	1ca167d5-1655-44a1-8adf-1414072e1ef9
DisplayText	Read and write to all app catalogs	Read and write to all app catalogs
Description	Allows the app to create, read, update, and delete apps in the app catalogs without a signed-in user.	Allows the app to create, read, update, and delete apps in the app catalogs.
AdminConsentRequired	Yes	Yes

AppCatalog.Submit

Category	Application	Delegated
Identifier	-	3db89e36-7fa6-4012-b281-85f3d9d9fd2e
DisplayText	-	Submit application packages to the catalog and cancel pending submissions
Description	-	Allows the app to submit application packages to the catalog and cancel submissions that are pending review on behalf of the signed-in user.
AdminConsentRequired	-	No

AppCertTrustConfiguration.Read.All

Category	Application	Delegated
Identifier	-	af281d3a-030d-4122-886e-146fb30a0413
DisplayText	-	Read the trusted certificate authority configuration for applications
Description	-	Allows the app to read the trusted certificate authority configuration which can be used to restrict application certificates based on their issuing authority, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AppCertTrustConfiguration.ReadWrite.All

Category	Application	Delegated
Identifier	-	4bae2ed4-473e-4841-a493-9829cfd51d48
DisplayText	-	Read and write the trusted certificate authority configuration for applications
Description	-	Allows the app to create, read, update and delete the trusted certificate authority configuration which can be used to restrict application certificates based on their issuing authority, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

Application-RemoteDesktopConfig.ReadWrite.All

Category	Application	Delegated
Identifier	3be0012a-cc4e-426b-895b-f9c836bf6381	ffa91d43-2ad8-45cc-b592-09caddeb24bb
DisplayText	Read and write the remote desktop security configuration for all apps	Read and write the remote desktop security configuration for apps
Description	Allows the app to read and write the remote desktop security configuration for all apps in your organization, without a signed-in user.	Allows the app to read and write other apps' remote desktop security configuration, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Application.Read.All

Category	Application	Delegated
Identifier	9a5d68dd-52b0-4cc2-bd40-abcf44ac3a30	c79f8feb-a9db-4090-85f9-90d820caa0eb
DisplayText	Read all applications	Read applications
Description	Allows the app to read all applications and service principals without a signed-in user.	Allows the app to read applications and service principals on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

The *Application.Read.All* delegated permission is available for consent in personal Microsoft accounts.

Application.ReadUpdate.All

Category	Application	Delegated
Identifier	fc023787-fd04-4e44-9bc7-d454f00c0f0a	0586a906-4d89-4de8-b3c8-1aacdcc0c679
DisplayText	Read and update all apps	Read and update all apps
Description	Allows the app to read and update all apps in your organization, without a signed-in user.	Allows the app to read and update all apps in your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Application.ReadWrite.All

Category	Application	Delegated
Identifier	1bfefb4e-e0b5-418b-a88f-73c46d2cc8e9	bdfbf15f-ee85-4955-8675-146e8e5296b5
DisplayText	Read and write all applications	Read and write applications
Description	Allows the app to create, read, update and delete applications and service principals without a signed-in user. Allows management of app role assignments, except those exposed by Microsoft Graph. Does not allow management of delegated permission grants.	Allows the app to create, read, update and delete applications and service principals on behalf of the signed-in user. Allows management of app role assignments, except those exposed by Microsoft Graph. Does not allow management of delegated permission grants.
AdminConsentRequired	Yes	Yes

The *Application.ReadWrite.All* delegated permission is available for consent in personal Microsoft accounts.

Permissions that allow managing credentials, such as *Application.ReadWrite.All*, allow an application to act as other entities, and use the privileges they were granted. Use caution when granting any of these permissions.

Application.ReadWrite.OwnedBy

Category	Application	Delegated
Identifier	18a4783c-866b-4cc7-a460-3d5e5662c884	-
DisplayText	Manage apps that this app creates or owns	-
Description	Allows the app to create other applications, and fully manage those applications (read, update, update application secrets and delete), without a signed-in user. It cannot update any apps that it is not an owner of.	-
AdminConsentRequired	Yes	-

The *Application.ReadWrite.OwnedBy* permission allows the same operations as *Application.ReadWrite.All* but only on applications and service principals that the calling app is an owner of.

The *Application.ReadWrite.OwnedBy* permission allows an app to call `GET /applications` and `GET /servicePrincipals` endpoints to list all applications and service principals in the tenant. This scope of access has been allowed for the permission.

AppRoleAssignment.ReadWrite.All

Category	Application	Delegated
Identifier	06b708a9-e830-4db3-a914-8e69da51d44f	84bccea3-f856-4a8a-967b-dbe0a3d53a64
DisplayText	Manage app permission grants and app role assignments	Manage app permission grants and app role assignments
Description	Allows the app to manage permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, without a signed-in user.	Allows the app to manage permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

> ⊗ Caution
>
> Permissions that allow granting authorization, such as *AppRoleAssignment.ReadWrite.All*, allow an application to grant additional privileges to itself, other applications, or any user. Use caution when granting any of these permissions.

ApprovalSolution.Read

Category	Application	Delegated
Identifier	-	b0df437d-d341-4df0-aa3e-89ca81a1207f
DisplayText	-	Read approvals
Description	-	Allows the app to read approvals on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ApprovalSolution.Read.All

Category	Application	Delegated
Identifier	9f265de7-8d5e-4e9a-a805-5e8bbc49656f	-
DisplayText	Read all approvals	-
Description	Allows the app to read all approvals and approval item subscriptions, without a signed-in user.	-
AdminConsentRequired	Yes	-

ApprovalSolution.ReadWrite

Category	Application	Delegated
Identifier	-	6768d3af-4562-48ff-82d2-c5e19eb21b9c

Category	Application	Delegated
DisplayText	-	Read, create, and respond to approvals
Description	-	Allows the app to provision, read, create, and respond to approvals on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ApprovalSolution.ReadWrite.All

Category	Application	Delegated
Identifier	45583558-1113-4d06-8969-e79a28edc9ad	-
DisplayText	Read all approvals and manage approval subscriptions	-
Description	Allows the app to read all approvals and create, update, or remove approval item subscriptions, without a signed-in user.	-
AdminConsentRequired	Yes	-

ApprovalSolutionResponse.ReadWrite

Category	Application	Delegated
Identifier	-	89d944f2-2011-44ad-830c-aa9bf5ef2319
DisplayText	-	Read and respond to approvals assigned to the current user
Description	-	Allows the app to read and respond to approvals on behalf of the signed-in user.
AdminConsentRequired	-	Yes

AttackSimulation.Read.All

Category	Application	Delegated
Identifier	93283d0a-6322-4fa8-966b-8c121624760d	104a7a4b-ca76-4677-b7e7-2f4bc482f381
DisplayText	Read attack simulation data of an	Read attack simulation data of an

Category	Application	Delegated
	organization	organization
Description	Allows the app to read attack simulation and training data for an organization without a signed-in user.	Allows the app to read attack simulation and training data for an organization for the signed-in user.
AdminConsentRequired	Yes	Yes

AttackSimulation.ReadWrite.All

Category	Application	Delegated
Identifier	e125258e-8c8a-42a8-8f55-ab502afa52f3	27608d7c-2c66-4cad-a657-951d575f5a60
DisplayText	Read, create, and update all attack simulation data of an organization	Read, create, and update attack simulation data of an organization
Description	Allows the app to read, create, and update attack simulation and training data for an organization without a signed-in user.	Allows the app to read, create, and update attack simulation and training data for an organization for the signed-in user.
AdminConsentRequired	Yes	No

AuditActivity.Read

Category	Application	Delegated
Identifier	99bc85fb-e857-4220-9f8c-3a1c83148d2e	16786f81-40d2-4116-bb26-d1a753bf0b20
DisplayText	Read activity audit log from the audit store.	Read activity audit log from the audit store.
Description	Read activity audit log from the audit store.	Read activity audit log from the audit store.
AdminConsentRequired	Yes	Yes

AuditActivity.Write

Category	Application	Delegated
Identifier	f6318678-2713-4bb6-b123-233e7336c1bd	a78fd341-0672-4792-a8ae-a5925b2546eb
DisplayText	Upload activity audit logs to the audit store.	Upload activity audit logs to the audit store.
Description	Allows the application to upload bulk activity audit logs to the audit store.	Allows the application to upload bulk activity audit logs to the audit store.
AdminConsentRequired	Yes	Yes

AuditLog.Read.All

Category	Application	Delegated
Identifier	b0afded3-3588-46d8-8b3d-9842eff778da	e4c9e354-4dc5-45b8-9e7c-e1393b0b1a20
DisplayText	Read all audit log data	Read audit log data
Description	Allows the app to read and query your audit log activities, without a signed-in user.	Allows the app to read and query your audit log activities, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AuditLogsQuery-CRM.Read.All

Category	Application	Delegated
Identifier	20e6f8e4-ffac-4cf7-82f7-70ddb7564318	ba78b16f-1e01-41b6-89ca-73e0a32b304c
DisplayText	Read audit logs data from Dynamics CRM workload	Read audit logs data from Dynamics CRM workload
Description	Allows the app to read and query audit logs from Dynamics CRM workload, without a signed-in user	Allows the app to read and query audit logs from Dynamics CRM workload, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AuditLogsQuery-Endpoint.Read.All

Category	Application	Delegated
Identifier	0bc85aed-7b0b-437a-bac8-3b29a1b84c99	ee3409fe-617f-43cf-bd1e-fc8b38049e69
DisplayText	Read audit logs data from Endpoint Data Loss Prevention workload	Read audit logs data from Endpoint Data Loss Prevention workload
Description	Allows the app to read and query audit logs from Endpoint Data Loss Prevention workload, without a signed-in user	Allows the app to read and query audit logs from Endpoint Data Loss Prevention workload, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AuditLogsQuery-Entra.Read.All

Category	Application	Delegated
Identifier	7276d950-48fc-4269-8348-f22f2bb296d0	5ff2f415-e0f1-4d11-bfd0-6d87c0f667fd
DisplayText	Read audit logs data from Entra (Azure AD) workload	Read audit logs data from Entra (Azure AD) workload
Description	Allows the app to read and query audit logs from Entra (Azure AD) workload, without a signed-in user	Allows the app to read and query audit logs from Entra (Azure AD) workload, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AuditLogsQuery-Exchange.Read.All

Category	Application	Delegated
Identifier	6b0d2622-d34e-4470-935b-b96550e5ca8d	6c8c71d2-c7e1-45b0-ac6d-1d2724fba6ae
DisplayText	Read audit logs data from Exchange workload	Read audit logs data from Exchange workload
Description	Allows the app to read and query audit logs from Exchange workload, without	Allows the app to read and query audit logs from Exchange workload, on behalf

Category	Application	Delegated
	a signed-in user	of a signed-in user.
AdminConsentRequired	Yes	Yes

AuditLogsQuery-OneDrive.Read.All

Category	Application	Delegated
Identifier	8a169a81-841c-45fd-ad43-96aede8801a0	4a72c235-a50d-4870-b598-fd88fd1fa074
DisplayText	Read audit logs data from OneDrive workload	Read audit logs data from OneDrive workload
Description	Allows the app to read and query audit logs from OneDrive workload, without a signed-in user	Allows the app to read and query audit logs from OneDrive workload, on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

AuditLogsQuery-SharePoint.Read.All

Category	Application	Delegated
Identifier	91c64a47-a524-4fce-9bf3-3d569a344ecf	30630b65-ed12-4a81-9130-e3a964109fae
DisplayText	Read audit logs data from SharePoint workload	Read audit logs data from SharePoint workload
Description	Allows the app to read and query audit logs from SharePoint workload, without a signed-in user	Allows the app to read and query audit logs from SharePoint workload, on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

AuditLogsQuery.Read.All

Category	Application	Delegated
Identifier	5e1e9171-754d-478c-812c-	1d9e7ac3-0eca-442c-82f9-e92625af6e6d

Category	Application	Delegated
	f1755a9a4c2d	
DisplayText	Read audit logs data from all services	Read audit logs data from all services
Description	Allows the app to read and query audit logs from all services.	Allows the app to read and query audit logs from all services, on behalf of a signed-in user
AdminConsentRequired	Yes	Yes

AuthenticationContext.Read.All

Category	Application	Delegated
Identifier	381f742f-e1f8-4309-b4ab-e3d91ae4c5c1	57b030f1-8c35-469c-b0d9-e4a077debe70
DisplayText	Read all authentication context information	Read all authentication context information
Description	Allows the app to read the authentication context information in your organization without a signed-in user.	Allows the app to read all authentication context information in your organization on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

AuthenticationContext.ReadWrite.All

Category	Application	Delegated
Identifier	a88eef72-fed0-4bf7-a2a9-f19df33f8b83	ba6d575a-1344-4516-b777-1404f5593057
DisplayText	Read and write all authentication context information	Read and write all authentication context information
Description	Allows the app to read and update the authentication context information in your organization without a signed-in user.	Allows the app to read and update all authentication context information in your organization on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

BackupRestore-Configuration.Read.All

Category	Application	Delegated
Identifier	5fbb5982-3230-4882-93c0-2167523ce0c2	444ed4b6-0554-4dc6-8e9c-3f9a34ee3ff6
DisplayText	Read all backup configuration policies	Read backup configuration policies
Description	Allows the app to read all backup configurations, and lists of Microsoft 365 service resources to be backed-up, without a signed-in user.	Allows the app to read the backup configuration, and list of Microsoft 365 service resources to be backed-up, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

BackupRestore-Configuration.ReadWrite.All

Category	Application	Delegated
Identifier	18133149-5489-40ac-80f0-4b6fa85f6cdc	a0244d16-171c-4496-8ffb-7b9b6954d339
DisplayText	Read and edit all backup configuration policies	Read and edit backup configuration policies
Description	Allows the app to read and update the backup configuration, and list of Microsoft 365 service resources to be backed-up, without a signed-in user.	Allows the app to read and update the backup configuration, and list of Microsoft 365 service resources to be backed-up, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

BackupRestore-Control.Read.All

Category	Application	Delegated
Identifier	6fe20a79-0e15-45a1-b019-834c125993a0	af598c63-4292-4437-b925-e996354d3854
DisplayText	Read the status of the M365 backup service	Read the status of the M365 backup service
Description	Allows the app to read the status of M365 backup service (enable/disable),	Allows the app to read the status of M365 backup service (enable/disable),

Category	Application	Delegated
	without signed in user	on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

BackupRestore-Control.ReadWrite.All

Category	Application	Delegated
Identifier	fb240865-88f8-4a1d-923f-98dbc7920860	96d46335-d92d-41b8-bc9f-273a692381ea
DisplayText	Update or read the status of the M365 backup service	Update or read the status of the M365 backup service
Description	Allows the app to update or read the status of M365 backup service (enable/disable), without signed in user	Allows the app to update or read the status of M365 backup service (enable/disable), on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

BackupRestore-Monitor.Read.All

Category	Application	Delegated
Identifier	ecae8511-f2d7-4be4-bdbf-91f244d45986	b4e98de1-4600-4e90-b5e1-7c1dfef04e5c
DisplayText	Read all monitoring, quota and billing information for the tenant	Read monitoring, quota and billing information for the tenant
Description	Allows the app to monitor all backup and restore jobs, view quota usage and billing details, without a signed-in user.	Allows the app to monitor backup and restore jobs, view quota usage and billing details, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

BackupRestore-Restore.Read.All

Category	Application	Delegated
Identifier	87853aa5-0372-4710-b34b-cef27bb7156e	94b36f78-434f-4904-8c08-421d9a9c1dc2
DisplayText	Read all restore sessions	Read restore sessions
Description	Allows the app to read all restore sessions, without a signed-in user.	Allows the app to read restore sessions, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

BackupRestore-Restore.ReadWrite.All

Category	Application	Delegated
Identifier	bebd0841-a3d8-4313-a51d-731112c8ee41	9f89e109-94b9-4c9b-b4fc-98cdaa54f574
DisplayText	Read restore all sessions and start restore sessions from backups	Read restore sessions and start restore sessions from backups
Description	Allows the app to search all backup snapshots for Microsoft 365 resources, and restore Microsoft 365 resources from a backed-up snapshot, without a signed-in user.	Allows the app to search the backup snapshots for Microsoft 365 resources, and restore Microsoft 365 resources from a backed-up snapshot, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

BackupRestore-Search.Read.All

Category	Application	Delegated
Identifier	f6135c51-c766-4be1-9638-ed90c2ed2443	2b24830f-f435-446f-ab5a-b1e70d9a2eb5
DisplayText	Search for metadata properties in all backup snapshots	Search for metadata properties in backup snapshots
Description	Allows the app to search all backup snapshots for Microsoft 365 resources, without a signed-in user.	Allows the app to search the backup snapshots for Microsoft 365 resources, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

BillingConfiguration.ReadWrite.All

Category	Application	Delegated
Identifier	9e8be751-7eee-4c09-bcfd-d64f6b087fd8	2bf6d319-dfca-4c22-9879-f88dcfaee6be
DisplayText	Read and write application billing configuration	Read and write application billing configuration
Description	Allows the app to read and write the billing configuration on all applications without a signed-in user.	Allows the app to read and write the billing configuration on all applications on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

BitlockerKey.Read.All

Category	Application	Delegated
Identifier	57f1cf28-c0c4-4ec3-9a30-19a2eaaf2f6e	b27a61ec-b99c-4d6a-b126-c4375d08ae30
DisplayText	Read all BitLocker keys	Read BitLocker keys
Description	Allows an app to read BitLocker keys for all devices, without a signed-in user. Allows read of the recovery key.	Allows the app to read BitLocker keys on behalf of the signed-in user, for their owned devices. Allows read of the recovery key.
AdminConsentRequired	Yes	Yes

BitlockerKey.ReadBasic.All

Category	Application	Delegated
Identifier	f690d423-6b29-4d04-98c6-694c42282419	5a107bfc-4f00-4e1a-b67e-66451267bc68
DisplayText	Read all BitLocker keys basic information	Read BitLocker keys basic information
Description	Allows an app to read basic BitLocker key properties for all devices, without	Allows the app to read basic BitLocker key properties on behalf of the signed-in

Category	Application	Delegated
	a signed-in user. Does not allow read of the recovery key.	user, for their owned devices. Does not allow read of the recovery key itself.
AdminConsentRequired	Yes	Yes

Bookings.Manage.All

Category	Application	Delegated
Identifier	6b22000a-1228-42ec-88db-b8c00399aecb	7f36b48e-542f-4d3b-9bcb-8406f0ab9fdb
DisplayText	Manage bookings information	Manage bookings information
Description	Allows an app to read, write and manage bookings appointments, businesses, customers, services, and staff on behalf of the signed-in user.	Allows an app to read, write and manage bookings appointments, businesses, customers, services, and staff on behalf of the signed-in user.
AdminConsentRequired	Yes	No

Bookings.Read.All

Category	Application	Delegated
Identifier	6e98f277-b046-4193-a4f2-6bf6a78cd491	33b1df99-4b29-4548-9339-7a7b83eaeebc
DisplayText	Read all Bookings related resources.	Read bookings information
Description	Allows an app to read Bookings appointments, businesses, customers, services, and staff without a signed-in user.	Allows an app to read bookings appointments, businesses, customers, services, and staff on behalf of the signed-in user.
AdminConsentRequired	Yes	No

Bookings.ReadWrite.All

Category	Application	Delegated
Identifier	0c4b2d20-7919-468d-8668-c54b09d4dee8	948eb538-f19d-4ec5-9ccc-f059e1ea4c72
DisplayText	Read and write bookings information	Read and write bookings information
Description	Allows an app to read and write bookings appointments, businesses, customers, services, and staff on behalf of the signed-in user. Does not allow create, delete and publish of booking businesses.	Allows an app to read and write bookings appointments, businesses, customers, services, and staff on behalf of the signed-in user. Does not allow create, delete and publish of booking businesses.
AdminConsentRequired	Yes	No

BookingsAppointment.ReadWrite.All

Category	Application	Delegated
Identifier	9769393e-5a9f-4302-9e3d-7e018ecb64a7	02a5a114-36a6-46ff-a102-954d89d9ab02
DisplayText	Read and write all Bookings related resources.	Read and write booking appointments
Description	Allows an app to read and write Bookings appointments and customers, and additionally allows reading businesses, services, and staff without a signed-in user.	Allows an app to read and write bookings appointments and customers, and additionally allows read businesses information, services, and staff on behalf of the signed-in user.
AdminConsentRequired	Yes	No

Bookmark.Read.All

Category	Application	Delegated
Identifier	be95e614-8ef3-49eb-8464-1c9503433b86	98b17b35-f3b1-4849-a85f-9f13733002f0
DisplayText	Read all bookmarks	Read all bookmarks that the user can access
Description	Allows an app to read all bookmarks without a signed-in user.	Allows an app to read all bookmarks that the signed-in user can access.

Category	Application	Delegated
AdminConsentRequired	Yes	No

BrowserSiteLists.Read.All

Category	Application	Delegated
Identifier	c5ee1f21-fc7f-4937-9af0-c91648ff9597	fb9be2b7-a7fc-4182-aec1-eda4597c43d5
DisplayText	Read all browser site lists for your organization	Read browser site lists for your organization
Description	Allows an app to read all browser site lists configured for your organization, without a signed-in user.	Allows an app to read the browser site lists configured for your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	No

BrowserSiteLists.ReadWrite.All

Category	Application	Delegated
Identifier	8349ca94-3061-44d5-9bfb-33774ea5e4f9	83b34c85-95bf-497b-a04e-b58eca9d49d0
DisplayText	Read and write all browser site lists for your organization	Read and write browser site lists for your organization
Description	Allows an app to read and write all browser site lists configured for your organization, without a signed-in user.	Allows an app to read and write the browser site lists configured for your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	No

BusinessScenarioConfig.Read.All

Category	Application	Delegated
Identifier	-	d16480b2-e469-4118-846b-d3d177327bee

Category	Application	Delegated
DisplayText	-	Read business scenario configurations
Description	-	Allows the app to read the configurations of your organization's business scenarios, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

BusinessScenarioConfig.Read.OwnedBy

Category	Application	Delegated
Identifier	acc0fc4d-2cd6-4194-8700-1768d8423d86	c47e7b6e-d6f1-4be9-9ffd-1e00f3e32892
DisplayText	Read all business scenario configurations this app creates or owns	Read business scenario configurations this app creates or owns
Description	Allows the app to read the configurations of business scenarios it owns, without a signed-in user.	Allows the app to read the configurations of business scenarios it owns, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

BusinessScenarioConfig.ReadWrite.All

Category	Application	Delegated
Identifier	-	755e785b-b658-446f-bb22-5a46abd029ea
DisplayText	-	Read and write business scenario configurations
Description	-	Allows the app to read and write the configurations of your organization's business scenarios, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

BusinessScenarioConfig.ReadWrite.OwnedBy

Category	Application	Delegated
Identifier	bbea195a-4c47-4a4f-bff2-cba399e11698	b3b7fcff-b4d4-4230-bf6f-90bd91285395
DisplayText	Read and write all business scenario configurations this app creates or owns	Read and write business scenario configurations this app creates or owns
Description	Allows the app to create new business scenarios and fully manage the configurations of scenarios it owns, without a signed-in user.	Allows the app to create new business scenarios and fully manage the configurations of scenarios it owns, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

BusinessScenarioData.Read.OwnedBy

Category	Application	Delegated
Identifier	6c0257fd-cffe-415b-8239-2d0d70fdaa9c	25b265c4-5d34-4e44-952d-b567f6d3b96d
DisplayText	Read data for all business scenarios this app creates or owns	Read all data for business scenarios this app creates or owns
Description	Allows the app to read the data associated with the business scenarios it owns, without a signed-in user.	Allows the app to read all data associated with the business scenarios it owns. Data access will be attributed to the signed-in user.
AdminConsentRequired	Yes	Yes

BusinessScenarioData.ReadWrite.OwnedBy

Category	Application	Delegated
Identifier	f2d21f22-5d80-499e-91cc-0a8a4ce16f54	19932d57-2952-4c60-8634-3655c79fc527
DisplayText	Read and write data for all business scenarios this app creates or owns	Read and write all data for business scenarios this app creates or owns
Description	Allows the app to fully manage the data associated with the business	Allows the app to fully manage all data associated with the business scenarios it

Category	Application	Delegated
	scenarios it owns, without a signed-in user.	owns. Data access and changes will be attributed to the signed-in user.
AdminConsentRequired	Yes	Yes

Calendars.Read

Category	Application	Delegated
Identifier	798ee544-9d2d-430c-a058-570e29e34338	465a38f9-76ea-45b9-9f34-9e8b0d4b0b42
DisplayText	Read calendars in all mailboxes	Read user calendars
Description	Allows the app to read events of all calendars without a signed-in user.	Allows the app to read events in user calendars.
AdminConsentRequired	Yes	No

The *Calendars.Read* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to *specific* mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *Calendars.Read* application permission.

Calendars.Read.All

Category	Application	Delegated
Identifier	470229df-a15a-4b08-9d95-8c534862b362	-
DisplayText	Read all users' work hours and locations	-
Description	Allows the app to read work hours and locations settings, recurrences, and occurrences for all users in the organization, without a signed-in user.	-
AdminConsentRequired	Yes	-

Calendars.Read.Shared

Category	Application	Delegated
Identifier	-	2b9c4092-424d-4249-948d-b43879977640
DisplayText	-	Read user and shared calendars
Description	-	Allows the app to read events in all calendars that the user can access, including delegate and shared calendars.
AdminConsentRequired	-	No

The *Calendars.Read.Shared* delegated permission is available for consent in personal Microsoft accounts.

Calendars.ReadBasic

Category	Application	Delegated
Identifier	-	662d75ba-a364-42ad-adee-f5f880ea4878
DisplayText	-	Read basic details of user calendars
Description	-	Allows the app to read events in user calendars, except for properties such as body, attachments, and extensions.
AdminConsentRequired	-	No

The *Calendars.ReadBasic* delegated permission is available for consent in personal Microsoft accounts.

Calendars.ReadBasic.All

Category	Application	Delegated
Identifier	8ba4a692-bc31-4128-9094-475872af8a53	-
DisplayText	Read basic details of calendars in all mailboxes	-
Description	Allows the app to read events of all calendars, except for properties such as body, attachments, and extensions, without a signed-in user.	-
AdminConsentRequired	Yes	-

Calendars.ReadWrite

Category	Application	Delegated
Identifier	ef54d2bf-783f-4e0f-bca1-3210c0444d99	1ec239c2-d7c9-4623-a91a-a9775856bb36
DisplayText	Read and write calendars in all mailboxes	Have full access to user calendars
Description	Allows the app to create, read, update, and delete events of all calendars without a signed-in user.	Allows the app to create, read, update, and delete events in user calendars.
AdminConsentRequired	Yes	No

The *Calendars.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to specific mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *Calendars.ReadWrite* application permission.

Calendars.ReadWrite.All

Category	Application	Delegated
Identifier	d91697b0-a708-4c11-a712-8fb2dc081aa2	-
DisplayText	Read and write all users' work hours and locations	-
Description	Allows the app to read, create, update, and delete work hours and locations settings, recurrences, and occurrences for all users in the organization, without a signed-in user.	-
AdminConsentRequired	Yes	-

Calendars.ReadWrite.Shared

Category	Application	Delegated
Identifier	-	12466101-c9b8-439a-8589-dd09ee67e8e9
DisplayText	-	Read and write user and shared calendars

Category	Application	Delegated
Description	-	Allows the app to create, read, update and delete events in all calendars in the organization user has permissions to access. This includes delegate and shared calendars.
AdminConsentRequired	-	No

CallAiInsights.Read.All

Category	Application	Delegated
Identifier	792b782b-7822-4b92-8103-77e44f2f706c	e24bdaf9-83f8-468b-a144-c681ccb6caf4
DisplayText	Read all AI Insights for calls.	Read all AI Insights for calls.
Description	Allows the app to read all AI Insights for all calls, without a signed-in user.	Allows the app to read all AI Insights for calls, on behalf of the signed-in user.
AdminConsentRequired	Yes	No

CallDelegation.Read

Category	Application	Delegated
Identifier	-	305b375b-00fe-48bf-81bc-e8d78954c1b6
DisplayText	-	Read delegation settings
Description	-	Allows the app to read delegation settings of you
AdminConsentRequired	-	Yes

CallDelegation.Read.All

Category	Application	Delegated
Identifier	5aa33e77-b893-495e-bdc5-4bf6f27d42a0	-
DisplayText	Read delegation settings	-
Description	Allows the app to read delegation settings of you	-

Category	Application	Delegated
AdminConsentRequired	Yes	-

CallDelegation.ReadWrite

Category	Application	Delegated
Identifier	-	599abf67-f72b-4b5f-98a3-cb38fe646118
DisplayText	-	Read and write delegation settings
Description	-	Allows the app to read and write delegation settings of you
AdminConsentRequired	-	Yes

CallDelegation.ReadWrite.All

Category	Application	Delegated
Identifier	8d06abce-e69b-4122-ba60-4f901bb1db2f	-
DisplayText	Read and write delegation settings	-
Description	Allows the app to read and write delegation settings of you	-
AdminConsentRequired	Yes	-

CallEvents-Emergency.Read.All

Category	Application	Delegated
Identifier	f0a35f91-2aa6-4a99-9d5a-5b6bcb66204e	-
DisplayText	Read all emergency call events	-
Description	Allows the app to read emergency call event information for all users in your organization without a signed-in user.	-
AdminConsentRequired	Yes	-

CallEvents.Read

Category	Application	Delegated
Identifier	-	43431c03-960e-400f-87c6-8f910321dca3
DisplayText	-	Read call event data
Description	-	Allows the app to read call event information for an organization for the signed-in user.
AdminConsentRequired	-	Yes

CallEvents.Read.All

Category	Application	Delegated
Identifier	1abb026f-7572-49f6-9ddd-ad61cbba181e	-
DisplayText	Read all call events	-
Description	Allows the app to read call event information for all users in your organization, without a signed-in user.	-
AdminConsentRequired	Yes	-

CallRecord-PstnCalls.Read.All

Category	Application	Delegated
Identifier	a2611786-80b3-417e-adaa-707d4261a5f0	-
DisplayText	Read PSTN and direct routing call log data	-
Description	Allows the app to read all PSTN and direct routing call log data without a signed-in user.	-
AdminConsentRequired	Yes	-

The *CallRecord-PstnCalls.Read.All* permission grants an application access to PSTN (calling plans) and direct routing call logs. This includes potentially sensitive information about users as well as calls to and from external phone numbers.

> **Important**

- Discretion should be used when granting these permissions to applications. Call records can provide insights into the operation of your business, and so can be a target for malicious actors. Only grant these permissions to applications you trust to meet your data protection requirements.
- Make sure that you are compliant with the laws and regulations in your area regarding data protection and confidentiality of communications. Please see the Terms of Use and consult with your legal counsel for more information.

CallRecordings.Read.All

Category	Application	Delegated
Identifier	ce8fb1f1-5e1f-44a0-b102-4ec28454d0dc	63d31bd6-bcf5-40ca-8283-ba4130a66405
DisplayText	Read all call recordings	Read all recordings of calls.
Description	Allows the app to read call recordings for all calls without a signed-in user.	Allows the app to read all recordings of calls, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CallRecords.Read.All

Category	Application	Delegated
Identifier	45bbb07e-7321-4fd7-a8f6-3ff27e6a81c8	-
DisplayText	Read all call records	-
Description	Allows the app to read call records for all calls and online meetings without a signed-in user.	-
AdminConsentRequired	Yes	-

The *CallRecords.Read.All* permission grants an application privileged access to callRecords for every call and online meeting within your organization, including calls to and from external phone numbers. This includes potentially sensitive details about who participated in the call, as well as technical information pertaining to these calls and meetings that can be used for network troubleshooting, such as IP addresses, device details, and other network information.

> **Important**
>
> - Discretion should be used when granting these permissions to applications. Call records can provide insights into the operation of your business, and so can be a target for malicious actors. Only grant these permissions to applications you trust to meet your data protection requirements.
> - Make sure that you are compliant with the laws and regulations in your area regarding data protection and confidentiality of communications. Please see the Terms of Use and consult with your legal counsel for more information.

Calls.AccessMedia.All

Category	Application	Delegated
Identifier	a7a681dc-756e-4909-b988-f160edc6655f	-
DisplayText	Access media streams in a call as an app	-
Description	Allows the app to get direct access to media streams in a call, without a signed-in user.	-
AdminConsentRequired	Yes	-

Calls.Initiate.All

Category	Application	Delegated
Identifier	284383ee-7f6e-4e40-a2a8-e85dcb029101	-
DisplayText	Initiate outgoing 1 to 1 calls from the app	-
Description	Allows the app to place outbound calls to a single user and transfer calls to users in your organization's directory, without a signed-in user.	-
AdminConsentRequired	Yes	-

Calls.InitiateGroupCall.All

Category	Application	Delegated
Identifier	4c277553-8a09-487b-8023-29ee378d8324	-
DisplayText	Initiate outgoing group calls from the app	-
Description	Allows the app to place outbound calls to multiple users and add participants to meetings in your organization, without a signed-in user.	-
AdminConsentRequired	Yes	-

Calls.JoinGroupCall.All

Category	Application	Delegated
Identifier	f6b49018-60ab-4f81-83bd-22caeabfed2d	-
DisplayText	Join group calls and meetings as an app	-
Description	Allows the app to join group calls and scheduled meetings in your organization, without a signed-in user. The app will be joined with the privileges of a directory user to meetings in your organization.	-
AdminConsentRequired	Yes	-

Calls.JoinGroupCallAsGuest.All

Category	Application	Delegated
Identifier	fd7ccf6b-3d28-418b-9701-cd10f5cd2fd4	-
DisplayText	Join group calls and meetings as a guest	-
Description	Allows the app to anonymously join group calls and scheduled meetings in your organization, without a signed-in user. The app will be joined as a guest to meetings in your organization.	-
AdminConsentRequired	Yes	-

Calls.ReportSyntheticMedia.All

Category	Application	Delegated
Identifier	050b0d28-840a-4ff6-bdf9-cc6221acbc1f	-
DisplayText	Report synthetic media detections in calls	-
Description	Allows the app to report synthetic media detections for participants in calls, without a signed-in user.	-
AdminConsentRequired	Yes	-

CallTranscripts.Read.All

Category	Application	Delegated
Identifier	4cd61b6d-8692-40bf-9d90-7f38db5e5fce	fbace248-5d8e-441c-85ca-cc19221a69a2
DisplayText	Read all call transcripts	Read all transcripts of calls.
Description	Allows the app to read call transcripts for all calls without a signed-in user.	Allows the app to read all transcripts of calls, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CaseManagement.Read.All

Category	Application	Delegated
Identifier	b328f35f-712c-40d6-b6b4-95449d77b352	7bdc421c-99cd-4b67-a749-aa8e92775f7e
DisplayText	Read all cases, relations, tasks, attachments and activities	Read all cases, relations, tasks, attachments and activities
Description	Allows the app to read all cases, relations, tasks, attachments and activities, without a signed-in user.	Allows the app to read all cases, relations, tasks, attachments and activities, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CaseManagement.ReadWrite.All

Category	Application	Delegated
Identifier	57ac77a0-bb98-4ba8-bf9a-7af06dcdcc1f	363a0763-d7eb-40bc-9457-6be55acd81e2
DisplayText	Read and write to all cases, relations, tasks, attachments and activities	Read and write to all cases, relations, tasks, attachments and activities
Description	Allows the app to read and write to all cases, relations, tasks, attachments and activities, without a signed-in user.	Allows the app to read and write to all cases, relations, tasks, attachments and activities, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ChangeManagement.Read.All

Category	Application	Delegated
Identifier	418dae40-2b65-4819-900c-519a04e4d278	4628dff5-c33e-4fde-b17a-b64e7acb1bed
DisplayText	Read Change Management items	Read Change Management items
Description	Allows to read all Change Management items.	Allows to read all Change Management items.
AdminConsentRequired	Yes	No

Channel.Create

Category	Application	Delegated
Identifier	f3a65bd4-b703-46df-8f7e-0174fea562aa	101147cf-4178-4455-9d58-02b5c164e759
DisplayText	Create channels	Create channels
Description	Create channels in any team, without a signed-in user.	Create channels in any team, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Channel.Delete.All

Category	Application	Delegated
Identifier	6a118a39-1227-45d4-af0c-ea7b40d210bc	cc83893a-e232-4723-b5af-bd0b01bcfe65
DisplayText	Delete channels	Delete channels
Description	Delete channels in any team, without a signed-in user.	Delete channels in any team, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Channel.ReadBasic.All

Category	Application	Delegated
Identifier	59a6b24b-4225-4393-8165-ebaec5f55d7a	9d8982ae-4365-4f57-95e9-d6032a4c0b87
DisplayText	Read the names and descriptions of all channels	Read the names and descriptions of channels
Description	Read all channel names and channel descriptions, without a signed-in user.	Read channel names and channel descriptions, on behalf of the signed-in user.
AdminConsentRequired	Yes	No

ChannelMember.Read.All

Category	Application	Delegated
Identifier	3b55498e-47ec-484f-8136-9013221c06a9	2eadaff8-0bce-4198-a6b9-2cfc35a30075
DisplayText	Read the members of all channels	Read the members of channels
Description	Read the members of all channels, without a signed-in user.	Read the members of channels, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ChannelMember.ReadWrite.All

Category	Application	Delegated
Identifier	35930dcf-aceb-4bd1-b99a-8ffed403c974	0c3e411a-ce45-4cd1-8f30-f99a3efa7b11
DisplayText	Add and remove members from all channels	Add and remove members from channels
Description	Add and remove members from all channels, without a signed-in user. Also allows changing a member's role, for example from owner to non-owner.	Add and remove members from channels, on behalf of the signed-in user. Also allows changing a member's role, for example from owner to non-owner.
AdminConsentRequired	Yes	Yes

ChannelMessage.Edit

Category	Application	Delegated
Identifier	-	2b61aa8a-6d36-4b2f-ac7b-f29867937c53
DisplayText	-	Edit user's channel messages
Description	-	Allows an app to edit channel messages in Microsoft Teams, on behalf of the signed-in user.
AdminConsentRequired	-	No

ChannelMessage.Read.All

Category	Application	Delegated
Identifier	7b2449af-6ccd-4f4d-9f78-e550c193f0d1	767156cb-16ae-4d10-8f8b-41b657c8c8c8
DisplayText	Read all channel messages	Read user channel messages
Description	Allows the app to read all channel messages in Microsoft Teams	Allows an app to read a channel's messages in Microsoft Teams, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ChannelMessage.ReadWrite

Category	Application	Delegated
Identifier	-	5922d31f-46c8-4404-9eaf-2117e390a8a4
DisplayText	-	Read and write user channel messages
Description	-	Allows the app to read and write channel messages, on behalf of the signed-in user. This doesn't allow the app to edit the policyViolation of a channel message.
AdminConsentRequired	-	Yes

ChannelMessage.Send

Category	Application	Delegated
Identifier	-	ebf0f66e-9fb1-49e4-a278-222f76911cf4
DisplayText	-	Send channel messages
Description	-	Allows an app to send channel messages in Microsoft Teams, on behalf of the signed-in user.
AdminConsentRequired	-	No

ChannelMessage.UpdatePolicyViolation.All

Category	Application	Delegated
Identifier	4d02b0cc-d90b-441f-8d82-4fb55c34d6bb	-
DisplayText	Flag channel messages for violating policy	-
Description	Allows the app to update Microsoft Teams channel messages by patching a set of Data Loss Prevention (DLP) policy violation properties to handle the output of DLP processing.	-
AdminConsentRequired	Yes	-

ChannelSettings.Read.All

Category	Application	Delegated
Identifier	c97b873f-f59f-49aa-8a0e-52b32d762124	233e0cf1-dd62-48bc-b65b-b38fe87fcf8e
DisplayText	Read the names, descriptions, and settings of all channels	Read the names, descriptions, and settings of channels
Description	Read all channel names, channel descriptions, and channel settings, without a signed-in user.	Read all channel names, channel descriptions, and channel settings, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ChannelSettings.ReadWrite.All

Category	Application	Delegated
Identifier	243cded2-bd16-4fd6-a953-ff8177894c3d	d649fb7c-72b4-4eec-b2b4-b15acf79e378
DisplayText	Read and write the names, descriptions, and settings of all channels	Read and write the names, descriptions, and settings of channels
Description	Read and write the names, descriptions, and settings of all channels, without a signed-in user.	Read and write the names, descriptions, and settings of all channels, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Chat.Create

Category	Application	Delegated
Identifier	d9c48af6-9ad9-47ad-82c3-63757137b9af	38826093-1258-4dea-98f0-00003be2b8d0
DisplayText	Create chats	Create chats
Description	Allows the app to create chats without a signed-in user.	Allows the app to create chats on behalf of the signed-in user.
AdminConsentRequired	Yes	No

Chat.ManageDeletion.All

Category	Application	Delegated
Identifier	9c7abde0-eacd-4319-bf9e-35994b1a1717	bb64e6fc-6b6d-4752-aea0-dd922dbba588
DisplayText	Delete and recover deleted chats	Delete and recover deleted chats
Description	Allows the app to delete and recover deleted chats, without a signed-in user.	Allows the app to delete and recover deleted chats, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Chat.Read

Category	Application	Delegated
Identifier	-	f501c180-9344-439a-bca0-6cbf209fd270
DisplayText	-	Read user chat messages
Description	-	Allows an app to read 1 on 1 or group chats threads, on behalf of the signed-in user.
AdminConsentRequired	-	No

Chat.Read.All

Category	Application	Delegated
Identifier	6b7d71aa-70aa-4810-a8d9-5d9fb2830017	-
DisplayText	Read all chat messages	-
Description	Allows the app to read all 1-to-1 or group chat messages in Microsoft Teams.	-
AdminConsentRequired	Yes	-

Chat.Read.WhereInstalled

Category	Application	Delegated
Identifier	1c1b4c8e-3cc7-4c58-8470-9b92c9d5848b	-
DisplayText	Read all chat messages for chats where the associated Teams application is installed.	-
Description	Allows the app to read all one-to-one or group chat messages in Microsoft Teams for chats where the associated Teams application is installed, without a signed-in user.	-
AdminConsentRequired	Yes	-

Chat.ReadBasic

Category	Application	Delegated
Identifier	-	9547fcb5-d03f-419d-9948-5928bbf71b0f
DisplayText	-	Read names and members of user chat threads
Description	-	Allows an app to read the members and descriptions of one-to-one and group chat threads, on behalf of the signed-in user.
AdminConsentRequired	-	No

Chat.ReadBasic.All

Category	Application	Delegated
Identifier	b2e060da-3baf-4687-9611-f4ebc0f0cbde	-
DisplayText	Read names and members of all chat threads	-
Description	Read names and members of all one-to-one and group chats in Microsoft Teams, without a signed-in user.	-
AdminConsentRequired	Yes	-

Chat.ReadBasic.WhereInstalled

Category	Application	Delegated
Identifier	818ba5bd-5b3e-4fe0-bbe6-aa4686669073	-
DisplayText	Read names and members of all chat threads where the associated Teams application is installed.	-
Description	Allows the app to read names and members of all one-to-one and group chats in Microsoft Teams where the associated Teams application is installed, without a signed-in user.	-
AdminConsentRequired	Yes	-

Chat.ReadWrite

Category	Application	Delegated
Identifier	-	9ff7295e-131b-4d94-90e1-69fde507ac11
DisplayText	-	Read and write user chat messages
Description	-	Allows an app to read and write 1 on 1 or group chats threads, on behalf of the signed-in user.
AdminConsentRequired	-	No

Chat.ReadWrite.All

Category	Application	Delegated
Identifier	294ce7c9-31ba-490a-ad7d-97a7d075e4ed	7e9a077b-3711-42b9-b7cb-5fa5f3f7fea7
DisplayText	Read and write all chat messages	Read and write all chat messages
Description	Allows an app to read and write all chat messages in Microsoft Teams, without a signed-in user.	Allows an app to read and write all one-to-one and group chats in Microsoft Teams, without a signed-in user. Does not allow sending messages.
AdminConsentRequired	Yes	Yes

Chat.ReadWrite.WhereInstalled

Category	Application	Delegated
Identifier	ad73ce80-f3cd-40ce-b325-df12c33df713	-
DisplayText	Read and write all chat messages for chats where the associated Teams application is installed.	-
Description	Allows the app to read and write all chat messages in Microsoft Teams for chats where the associated Teams application is installed, without a signed-in user.	-
AdminConsentRequired	Yes	-

Chat.UpdatePolicyViolation.All

Category	Application	Delegated
Identifier	7e847308-e030-4183-9899-5235d7270f58	-
DisplayText	Flag chat messages for violating policy	-
Description	Allows the app to update Microsoft Teams 1-to-1 or group chat messages by patching a set of Data Loss Prevention (DLP) policy violation properties to handle the output of DLP processing.	-
AdminConsentRequired	Yes	-

ChatMember.Read

Category	Application	Delegated
Identifier	-	c5a9e2b1-faf6-41d4-8875-d381aa549b24
DisplayText	-	Read the members of chats
Description	-	Read the members of chats, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ChatMember.Read.All

Category	Application	Delegated
Identifier	a3410be2-8e48-4f32-8454-c29a7465209d	-
DisplayText	Read the members of all chats	-
Description	Read the members of all chats, without a signed-in user.	-
AdminConsentRequired	Yes	-

ChatMember.Read.WhereInstalled

Category	Application	Delegated
Identifier	93e7c9e4-54c5-4a41-b796-f2a5adaacda7	-
DisplayText	Read the members of all chats where the associated Teams application is installed.	-
Description	Allows the app to read the members of all chats where the associated Teams application is installed, without a signed-in user.	-
AdminConsentRequired	Yes	-

ChatMember.ReadWrite

Category	Application	Delegated
Identifier	-	dea13482-7ea6-488f-8b98-eb5bbecf033d
DisplayText	-	Add and remove members from chats
Description	-	Add and remove members from chats, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ChatMember.ReadWrite.All

Category	Application	Delegated
Identifier	57257249-34ce-4810-a8a2-a03adf0c5693	-

Category	Application	Delegated
DisplayText	Add and remove members from all chats	-
Description	Add and remove members from all chats, without a signed-in user.	-
AdminConsentRequired	Yes	-

ChatMember.ReadWrite.WhereInstalled

Category	Application	Delegated
Identifier	e32c2cd9-0124-4e44-88fc-772cd98afbdb	-
DisplayText	Add and remove members from all chats where the associated Teams application is installed.	-
Description	Allows the app to add and remove members from all chats where the associated Teams application is installed, without a signed-in user.	-
AdminConsentRequired	Yes	-

ChatMessage.Read

Category	Application	Delegated
Identifier	-	cdcdac3a-fd45-410d-83ef-554db620e5c7
DisplayText	-	Read user chat messages
Description	-	Allows an app to read one-to-one and group chat messages, on behalf of the signed-in user.
AdminConsentRequired	-	No

ChatMessage.Read.All

Category	Application	Delegated
Identifier	b9bb2381-47a4-46cd-aafb-00cb12f68504	-
DisplayText	Read all chat messages	-

Category	Application	Delegated
Description	Allows the app to read all one-to-one and group chats messages in Microsoft Teams, without a signed-in user.	-
AdminConsentRequired	Yes	-

ChatMessage.Send

Category	Application	Delegated
Identifier	-	116b7235-7cc6-461e-b163-8e55691d839e
DisplayText	-	Send user chat messages
Description	-	Allows an app to send one-to-one and group chat messages in Microsoft Teams, on behalf of the signed-in user.
AdminConsentRequired	-	No

CloudApp-Discovery.Read.All

Category	Application	Delegated
Identifier	64a59178-dad3-4673-89db-84fdcd622fec	ad46d60e-1027-4b75-af88-7c14ccf43a19
DisplayText	Read all discovered cloud applications data	Read discovered cloud applications data
Description	Allows the app to read all details of discovered cloud apps in the organization, without a signed-in user.	Allows the app to read details of discovered cloud apps in the organization, on behalf of the signed in user.
AdminConsentRequired	Yes	No

CloudPC.Read.All

Category	Application	Delegated
Identifier	a9e09520-8ed4-4cde-838e-4fdea192c227	5252ec4e-fd40-4d92-8c68-89dd1d3c6110

Category	Application	Delegated
DisplayText	Read Cloud PCs	Read Cloud PCs
Description	Allows the app to read the properties of Cloud PCs, without a signed-in user.	Allows the app to read the properties of Cloud PCs on behalf of the signed-in user.
AdminConsentRequired	Yes	No

CloudPC.ReadWrite.All

Category	Application	Delegated
Identifier	3b4349e1-8cf5-45a3-95b7-69d1751d3e6a	9d77138f-f0e2-47ba-ab33-cd246c8b79d1
DisplayText	Read and write Cloud PCs	Read and write Cloud PCs
Description	Allows the app to read and write the properties of Cloud PCs, without a signed-in user.	Allows the app to read and write the properties of Cloud PCs on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Community.Read.All

Category	Application	Delegated
Identifier	407f0cce-3212-441f-9f55-3bc91342cf86	12ae2e92-14b5-47b2-babb-4e890bbedc0a
DisplayText	Read all Viva Engage communities	Read all Viva Engage communities
Description	Allows the app to list Viva Engage communities, and to read their properties without a signed-in user.	Allows the app to list Viva Engage communities, and to read their properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Community.ReadWrite.All

Category	Application	Delegated
Identifier	35d59e32-eab5-4553-9345-abb62b4c703c	9e69467d-e0e2-402b-a926-3d796990197f
DisplayText	Read and write all Viva Engage communities	Read and write all Viva Engage communities
Description	Allows the app to create Viva Engage communities, read all community properties, update community properties, and delete communities without a signed-in user.	Allows the app to create Viva Engage communities and read all community properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ConfigurationMonitoring.Read.All

Category	Application	Delegated
Identifier	aca929ec-9830-44dc-bda1-85cf938aaa95	c645bb69-adc4-4242-b620-02e635f03bf6
DisplayText	Read all Configuration Monitoring entities	Read all Configuration Monitoring entities
Description	Allows the app to read all Configuration Monitoring entities, without a signed-in user.	Allows the app to read all Configuration Monitoring entities on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ConfigurationMonitoring.ReadWrite.All

Category	Application	Delegated
Identifier	cfa85bfb-2ee8-4e13-8e7f-489e57a015a1	54505ce9-e719-41f7-a7cc-dbe114e1d811
DisplayText	Read and write all Configuration Monitoring entities	Read and write all Configuration Monitoring entities
Description	Allows the app to read and write all Configuration Monitoring entities, without a signed-in user.	Allows the app to read and write all Configuration Monitoring entities on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

ConsentRequest.Create

Category	Application	Delegated
Identifier	-	f2143d35-9b4b-480d-951c-d083e69eeb2c
DisplayText	-	Create consent requests
Description	-	Allows the app to read create consent requests on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ConsentRequest.Read

Category	Application	Delegated
Identifier	-	5942b2f6-5a7b-40af-aa37-4b6ea5447506
DisplayText	-	Read consent requests created by the user
Description	-	Allows the app to read consent requests and approvals created by the signed-in user, on behalf of the signed-in user.
AdminConsentRequired	-	No

ConsentRequest.Read.All

Category	Application	Delegated
Identifier	1260ad83-98fb-4785-abbb-d6cc1806fd41	f3bfad56-966e-4590-a536-82ecf548ac1e
DisplayText	Read all consent requests	Read consent requests
Description	Allows the app to read consent requests and approvals without a signed-in user.	Allows the app to read consent requests and approvals on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

ConsentRequest.ReadApprove.All

Category	Application	Delegated
Identifier	-	e694a3a1-7878-46d8-8c29-3d195f6589f4
DisplayText	-	Read and approve consent requests
Description	-	Allows the app to read and approve consent requests on behalf of the signed in user.
AdminConsentRequired	-	Yes

ConsentRequest.ReadWrite.All

Category	Application	Delegated
Identifier	9f1b81a7-0223-4428-bfa4-0bcb5535f27d	497d9dfa-3bd1-481a-baab-90895e54568c
DisplayText	Read and write all consent requests	Read and write consent requests
Description	Allows the app to read app consent requests and approvals, and deny or approve those requests without a signed-in user.	Allows the app to read app consent requests and approvals, and deny or approve those requests on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Contacts-OnPremisesSyncBehavior.ReadWrite.All

Category	Application	Delegated
Identifier	c8948c23-e66b-42db-83fd-770b71ab78d2	1e4c6c41-0803-4f52-85ef-0a5d63ad8670
DisplayText	Read and update the on-premises sync behavior of contacts	Read and update the on-premises sync behavior of contacts

Category	Application	Delegated
Description	Allows the app to update the on-premises sync behavior of all contacts in all mailboxes without a signed-in user.	Allows the app to read and update the on-premises sync behavior of contacts a user has permissions to, including their own and shared contacts.
AdminConsentRequired	Yes	Yes

Contacts.Read

Category	Application	Delegated
Identifier	089fe4d0-434a-44c5-8827-41ba8a0b17f5	ff74d97f-43af-4b68-9f2a-b77ee6968c5d
DisplayText	Read contacts in all mailboxes	Read user contacts
Description	Allows the app to read all contacts in all mailboxes without a signed-in user.	Allows the app to read user contacts.
AdminConsentRequired	Yes	No

The *Contacts.Read* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to *specific* mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *Contacts.Read* application permission.

Contacts.Read.Shared

Category	Application	Delegated
Identifier	-	242b9d9e-ed24-4d09-9a52-f43769beb9d4
DisplayText	-	Read user and shared contacts
Description	-	Allows the app to read contacts a user has permissions to access, including their own and shared contacts.
AdminConsentRequired	-	No

Contacts.ReadWrite

Category	Application	Delegated
Identifier	6918b873-d17a-4dc1-b314-35f528134491	d56682ec-c09e-4743-aaf4-1a3aac4caa21
DisplayText	Read and write contacts in all mailboxes	Have full access to user contacts
Description	Allows the app to create, read, update, and delete all contacts in all mailboxes without a signed-in user.	Allows the app to create, read, update, and delete user contacts.
AdminConsentRequired	Yes	No

The *Contacts.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to specific mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *Contacts.ReadWrite* application permission.

Contacts.ReadWrite.Shared

Category	Application	Delegated
Identifier	-	afb6c84b-06be-49af-80bb-8f3f77004eab
DisplayText	-	Read and write user and shared contacts
Description	-	Allows the app to create, read, update, and delete contacts a user has permissions to, including their own and shared contacts.
AdminConsentRequired	-	No

Content.Process.All

Category	Application	Delegated
Identifier	5ad511bf-571c-4ef6-8c3c-85b94b85df98	7e2467d1-f874-46bb-828e-24cb06b29d3f
DisplayText	Process content for data security, governance and compliance	Process content for data security, governance and compliance

Category	Application	Delegated
Description	Allows the app to process and evaluate content for data security, governance and compliance outcomes at tenant scope.	Allows the app to process and evaluate content for data security, governance and compliance outcomes at tenant scope.
AdminConsentRequired	Yes	Yes

Content.Process.User

Category	Application	Delegated
Identifier	24ceb246-ad29-4680-90b4-3e91ffad15eb	1d787a13-f750-4ad6-875a-fcbd2725596b
DisplayText	Process content for data security, governance and compliance	Process content for data security, governance and compliance
Description	Allows the app to process and evaluate content for data security, governance and compliance outcomes for a user.	Allows the app to process and evaluate content for data security, governance and compliance outcomes for a user.
AdminConsentRequired	Yes	Yes

ContentActivity.Read

Category	Application	Delegated
Identifier	368425e7-6954-4f5a-9d92-90b75bd580c9	62c55b2f-a2b1-4312-8385-be57afd901b4
DisplayText	Read contents activity audit log from the audit store.	Read contents activity audit log from the audit store.
Description	Read contents activity audit log from the audit store.	Read contents activity audit log from the audit store.
AdminConsentRequired	Yes	Yes

ContentActivity.Write

Category	Application	Delegated
Identifier	2932e07a-3c29-44e4-bb36-6d0fc176387f	948caae6-152a-48cd-a746-4844af30e8e9
DisplayText	Upload content activity audit logs to the audit store.	Upload contents activity audit logs to the audit store.
Description	Allows the application to upload bulk contents activity audit logs to the audit store.	Allows the application to upload bulk contents activity audit logs to the audit store.
AdminConsentRequired	Yes	Yes

Contracts.Read.All

Category	Application	Delegated
Identifier	f9af4646-98b0-4e9d-a53e-40d4f6452fc4	9df4d5b0-7921-4437-9ea8-adf0c9e276dc
DisplayText	Read contracts	Read contracts
Description	Allows the app to read available properties on contracts, without a signed-in user.	Allows the app to read available properties of contracts, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CopilotConversation.Delete

Category	Application	Delegated
Identifier	-	ed510a02-ac32-45f9-93e6-04864f7f7e47
DisplayText	-	Delete Microsoft 365 Copilot conversations
Description	-	Allows the app to delete Microsoft 365 Copilot conversations on behalf of the signed-in user.
AdminConsentRequired	-	No

CopilotPackages.Read.All

Category	Application	Delegated
Identifier	72f0655d-6228-4ddc-8e1b-164973b9213b	a2dcfcb9-cbe8-4d42-812d-952e55cf7f3f
DisplayText	Read all packages information	Read all packages information
Description	Allows the app to read packages information without a signed-in user.	Allows the user to read the packages information
AdminConsentRequired	Yes	No

CopilotPackages.ReadWrite.All

Category	Application	Delegated
Identifier	ed31732f-9495-47ed-ba3b-4ed0948c1c64	e9c5fd18-ac15-43dd-9f5c-6f9611dd5604
DisplayText	Read and update all packages information	Read and update all packages information
Description	Allows the app to read and update packages information without a signed-in user.	Allows the user to read and update the packages information
AdminConsentRequired	Yes	Yes

CopilotPolicySettings.Read

Category	Application	Delegated
Identifier	556d5e2e-1081-4452-8147-26c3a1b06f58	b7281c63-cd4d-40c3-b721-73aa8ee7c3a8
DisplayText	Read Copilot policy settings	Read Copilot policy settings
Description	Allows the app to read Copilot policy settings for the organization, without a signed-in user.	Allows the app to read Copilot policy settings for the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CopilotPolicySettings.ReadWrite

Category	Application	Delegated
Identifier	cc147c17-b8e8-4d3f-9f94-aa9e279a079a	e2edbde8-4448-4e49-8ebb-d53ba72df0f3
DisplayText	Read and write Copilot policy settings	Read and write Copilot policy settings
Description	Allows the app to read and write Copilot policy settings for the organization, without a signed-in user.	Allows the app to read and write Copilot policy settings for the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CopilotSettings-LimitedMode.Read

Category	Application	Delegated
Identifier	-	aeb2982d-632d-4155-b533-18756ab6fdd8
DisplayText	-	Read organization-wide copilot limited mode setting
Description	-	Allows the app to read organization-wide copilot limited mode setting on behalf of the signed-in user.
AdminConsentRequired	-	Yes

CopilotSettings-LimitedMode.ReadWrite

Category	Application	Delegated
Identifier	-	4704e5b2-0ada-4aa0-b18c-00ad7525bc06
DisplayText	-	Read and write organization-wide copilot limited mode setting
Description	-	Allows the app to read and write organization-wide copilot limited mode setting on behalf of the signed-in user.
AdminConsentRequired	-	Yes

CrossTenantInformation.ReadBasic.All

Category	Application	Delegated
Identifier	cac88765-0581-4025-9725-5ebc13f729ee	81594d25-e88e-49cf-ac8c-fecbff49f994
DisplayText	Read cross-tenant basic information	Read cross-tenant basic information
Description	Allows the application to obtain basic tenant information about another target tenant within the Azure AD ecosystem without a signed-in user.	Allows the application to obtain basic tenant information about another target tenant within the Azure AD ecosystem on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CrossTenantUserProfileSharing.Read

Category	Application	Delegated
Identifier	-	cb1ba48f-d22b-4325-a07f-74135a62ee41
DisplayText	-	Read shared cross-tenant user profile and export data
Description	-	Allows the application to list and query user profile information associated with the current tenant on behalf of the signed-in user. It also permits the application to export external user data (e.g. customer content or system-generated logs), associated with the current tenant on behalf of the signed-in user.
AdminConsentRequired	-	Yes

The *CrossTenantUserProfileSharing.Read* delegated permission is available for consent in personal Microsoft accounts.

CrossTenantUserProfileSharing.Read.All

Category	Application	Delegated
Identifier	8b919d44-6192-4f3d-8a3b-f86f8069ae3c	759dcd16-3c90-463c-937e-abf89f991c18
DisplayText	Read all shared cross-tenant user profiles and export their data	Read all shared cross-tenant user profiles and export their data
Description	Allows the application to list and query any shared user profile information	Allows the application to list and query any shared user profile information

Category	Application	Delegated
	associated with the current tenant without a signed-in user. It also permits the application to export external user data (e.g. customer content or system-generated logs), for any user associated with the current tenant without a signed-in user.	associated with the current tenant on behalf of the signed-in user. It also permits the application to export external user data (e.g. customer content or system-generated logs), for any user associated with the current tenant on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

The *CrossTenantUserProfileSharing.Read.All* delegated permission is available for consent in personal Microsoft accounts.

CrossTenantUserProfileSharing.ReadWrite

Category	Application	Delegated
Identifier	-	eed0129d-dc60-4f30-8641-daf337a39ffd
DisplayText	-	Read shared cross-tenant user profile and export or delete data
Description	-	Allows the application to list and query user profile information associated with the current tenant on behalf of the signed-in user. It also permits the application to export and remove external user data (e.g. customer content or system-generated logs), associated with the current tenant on behalf of the signed-in user.
AdminConsentRequired	-	Yes

CrossTenantUserProfileSharing.ReadWrite.All

Category	Application	Delegated
Identifier	306785c5-c09b-4ba0-a4ee-023f3da165cb	64dfa325-cbf8-48e3-938d-51224a0cac01
DisplayText	Read all shared cross-tenant user profiles and export or delete their data	Read all shared cross-tenant user profiles and export or delete their data
Description	Allows the application to list and query any shared user profile information associated with the current tenant without a signed-in user. It also permits the application to export and	Allows the application to list and query any shared user profile information associated with the current tenant on behalf of the signed-in user. It also permits the application to export and

Category	Application	Delegated
	remove external user data (e.g. customer content or system-generated logs), for any user associated with the current tenant without a signed-in user.	remove external user data (e.g. customer content or system-generated logs), for any user associated with the current tenant on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CustomAuthenticationExtension.Read.All

Category	Application	Delegated
Identifier	88bb2658-5d9e-454f-aacd-a3933e079526	b2052569-c98c-4f36-a5fb-43e5c111e6d0
DisplayText	Read all custom authentication extensions	Read your organization's custom authentication extensions
Description	Allows the app to read your organization's custom authentication extensions without a signed-in user.	Allows the app to read your organization's custom authentication extensions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CustomAuthenticationExtension.ReadWrite.All

Category	Application	Delegated
Identifier	c2667967-7050-4e7e-b059-4cbbb3811d03	8dfcf82f-15d0-43b3-bc78-a958a13a5792
DisplayText	Read and write all custom authentication extensions	Read and write your organization's custom authentication extensions
Description	Allows the app to read or write your organization's custom authentication extensions without a signed-in user.	Allows the app to read or write your organization's custom authentication extensions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CustomAuthenticationExtension.Receive.Payload

Category	Application	Delegated
Identifier	214e810f-fda8-4fd7-a475-29461495eb00	-
DisplayText	Receive custom authentication extension HTTP requests	-
Description	Allows custom authentication extensions associated with the app to receive HTTP requests triggered by an authentication event. The request can include information about a user, client and resource service principals, and other information about the authentication.	-
AdminConsentRequired	Yes	-

CustomDetection.Read.All

Category	Application	Delegated
Identifier	673a007a-9e0f-4c97-b066-3c0164486909	b13ff42e-f321-4d7d-a462-141c46a1b832
DisplayText	Read all custom detection rules	Read custom detection rules
Description	Allows the app to read custom detection rules without a signed-in user.	Allows the app to read custom detection rules on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CustomDetection.ReadWrite.All

Category	Application	Delegated
Identifier	e0fd9c8d-a12e-4cc9-9827-20c8c3cd6fb8	c34088fb-0649-4714-af0b-bcbfec155897
DisplayText	Read and write all custom detection rules	Read and write custom detection rules
Description	Allows the app to read and write custom detection rules without a signed-in user.	Allows the app to read and write custom detection rules on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CustomSecAttributeAssignment.Read.All

Category	Application	Delegated
Identifier	3b37c5a4-1226-493d-bec3-5d6c6b866f3f	b46ffa80-fe3d-4822-9a1a-c200932d54d0
DisplayText	Read custom security attribute assignments	Read custom security attribute assignments
Description	Allows the app to read custom security attribute assignments for all principals in the tenant without a signed in user.	Allows the app to read custom security attribute assignments for all principals in the tenant on behalf of a signed in user.
AdminConsentRequired	Yes	Yes

CustomSecAttributeAssignment.ReadWrite.All

Category	Application	Delegated
Identifier	de89b5e4-5b8f-48eb-8925-29c2b33bd8bd	ca46335e-8453-47cd-a001-8459884efeae
DisplayText	Read and write custom security attribute assignments	Read and write custom security attribute assignments
Description	Allows the app to read and write custom security attribute assignments for all principals in the tenant without a signed in user.	Allows the app to read and write custom security attribute assignments for all principals in the tenant on behalf of a signed in user.
AdminConsentRequired	Yes	Yes

CustomSecAttributeAuditLogs.Read.All

Category	Application	Delegated
Identifier	2a4f026d-e829-4e84-bdbf-d981a2703059	1fcdeaab-b519-44dd-bffc-ed1fd15a24e0
DisplayText	Read all custom security attribute audit logs	Read custom security attribute audit logs

Category	Application	Delegated
Description	Allows the app to read all audit logs for events that contain information about custom security attributes, without a signed-in user.	Allows the app to read audit logs for events that contain information about custom security attributes, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

CustomSecAttributeDefinition.Read.All

Category	Application	Delegated
Identifier	b185aa14-d8d2-42c1-a685-0f5596613624	ce026878-a0ff-4745-a728-d4fedd086c07
DisplayText	Read custom security attribute definitions	Read custom security attribute definitions
Description	Allows the app to read custom security attribute definitions for the tenant without a signed in user.	Allows the app to read custom security attribute definitions for the tenant on behalf of a signed in user.
AdminConsentRequired	Yes	Yes

CustomSecAttributeDefinition.ReadWrite.All

Category	Application	Delegated
Identifier	12338004-21f4-4896-bf5e-b75dfaf1016d	8b0160d4-5743-482b-bb27-efc0a485ca4a
DisplayText	Read and write custom security attribute definitions	Read and write custom security attribute definitions
Description	Allows the app to read and write custom security attribute definitions for the tenant without a signed in user.	Allows the app to read and write custom security attribute definitions for the tenant on behalf of a signed in user.
AdminConsentRequired	Yes	Yes

CustomSecAttributeProvisioning.Read.All

Category	Application	Delegated
Identifier	9fd1f8bf-a443-4df6-bc2a-5d00c5ec7828	9ddd870d-077c-49e7-b3e3-6b3012a8a880
DisplayText	Read the provisioning configuration of all active custom security attributes	Read the provisioning configuration of all active custom security attributes
Description	Allows the app to read the provisioning configuration of all active custom security attributes without a signed-in user.	Allows the app to read the provisioning configuration of all active custom security attributes on behalf of a signed in user.
AdminConsentRequired	Yes	Yes

CustomSecAttributeProvisioning.ReadWrite.All

Category	Application	Delegated
Identifier	1db69e9c-8d0a-498d-a5df-11fd0b68ceab	1140d9e4-6776-433e-a9e4-b9831adbb2e0
DisplayText	Read and edit the provisioning configuration of all active custom security attributes	Read and edit the provisioning configuration of all active custom security attributes
Description	Allows the app to read and edit the provisioning configuration of all active custom security attributes without a signed-in user.	Allows the app to read and edit the provisioning configuration of all active custom security attributes on behalf of a signed in user.
AdminConsentRequired	Yes	Yes

CustomTags.Read.All

Category	Application	Delegated
Identifier	ab8a5872-7c88-47a6-8141-7becce939190	de6ea87d-10bd-467c-8682-d525a0c61b89
DisplayText	Read all custom tags data	Read all custom tags data
Description	Read custom tags data, without a signed-in user	Read custom tags data on behalf of the signed-in user
AdminConsentRequired	Yes	Yes

CustomTags.ReadWrite.All

Category	Application	Delegated
Identifier	2f503208-e509-4e39-974c-8cc16e5785c9	2f1bbe0a-f34b-4efb-9edb-8db8dcb50eca
DisplayText	Read and write custom tags data	Read and write custom tags data
Description	Read and write custom tags data, without a signed-in user	Read and write custom tags data on behalf of the signed-in user
AdminConsentRequired	Yes	Yes

DelegatedAdminRelationship.Read.All

Category	Application	Delegated
Identifier	f6e9e124-4586-492f-adc0-c6f96e4823fd	0c0064ea-477b-4130-82a5-4c2cc4ff68aa
DisplayText	Read Delegated Admin relationships with customers	Read Delegated Admin relationships with customers
Description	Allows the app to read details of delegated admin relationships with customers like access details (that includes roles) and the duration as well as specific role assignments to security groups without a signed-in user.	Allows the app to read details of delegated admin relationships with customers like access details (that includes roles) and the duration as well as specific role assignments to security groups on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

DelegatedAdminRelationship.ReadWrite.All

Category	Application	Delegated
Identifier	cc13eba4-8cd8-44c6-b4d4-f93237adce58	885f682f-a990-4bad-a642-36736a74b0c7
DisplayText	Manage Delegated Admin relationships with customers	Manage Delegated Admin relationships with customers

Category	Application	Delegated
Description	Allows the app to manage (create-update-terminate) Delegated Admin relationships with customers and role assignments to security groups for active Delegated Admin relationships without a signed-in user.	Allows the app to manage (create-update-terminate) Delegated Admin relationships with customers as well as role assignments to security groups for active Delegated Admin relationships on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

DelegatedPermissionGrant.Read.All

Category	Application	Delegated
Identifier	81b4724a-58aa-41c1-8a55-84ef97466587	a197cdc4-a8e8-4d49-9d35-4ca7c83887b4
DisplayText	Read all delegated permission grants	Read delegated permission grants
Description	Allows the app to read all delegated permission grants, without a signed-in user.	Allows the app to read delegated permission grants, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

DelegatedPermissionGrant.ReadWrite.All

Category	Application	Delegated
Identifier	8e8e4742-1d95-4f68-9d56-6ee75648c72a	41ce6ca6-6826-4807-84f1-1c82854f7ee5
DisplayText	Manage all delegated permission grants	Manage all delegated permission grants
Description	Allows the app to manage permission grants for delegated permissions exposed by any API (including Microsoft Graph), without a signed-in user.	Allows the app to manage permission grants for delegated permissions exposed by any API (including Microsoft Graph), on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

Device.Command

Category	Application	Delegated
Identifier	-	bac3b9c2-b516-4ef4-bd3b-c2ef73d8d804
DisplayText	-	Communicate with user devices
Description	-	Allows the app to launch another app or communicate with another app on a user's device on behalf of the signed-in user.
AdminConsentRequired	-	No

The *Device.Command* delegated permission is available for consent in personal Microsoft accounts.

Device.CreateFromOwnedTemplate

Category	Application	Delegated
Identifier	-	edc92e89-a987-48a9-911a-a7b1967dd7b1
DisplayText	-	Create devices based on owned device templates
Description	-	Allows the app to create device objects based on device templates owned by the signed-in user, on behalf of the signed in user.
AdminConsentRequired	-	Yes

Device.Read

Category	Application	Delegated
Identifier	-	11d4cd79-5ba5-460f-803f-e22c8ab85ccd
DisplayText	-	Read user devices
Description	-	Allows the app to read a user's list of devices on behalf of the signed-in user.
AdminConsentRequired	-	No

The *Device.Read* delegated permission is available for consent in personal Microsoft accounts.

Device.Read.All

Category	Application	Delegated
Identifier	7438b122-aefc-4978-80ed-43db9fcc7715	951183d1-1a61-466f-a6d1-1fde911bfd95
DisplayText	Read all devices	Read all devices
Description	Allows the app to read your organization's devices' configuration information without a signed-in user.	Allows the app to read your organization's devices' configuration information on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

The *Device.Read.All* delegated permission is available for consent in personal Microsoft accounts.

Device.ReadWrite.All

Category	Application	Delegated
Identifier	1138cb37-bd11-4084-a2b7-9f71582aeddb	-
DisplayText	Read and write devices	-
Description	Allows the app to read and write all device properties without a signed in user. Does not allow device creation, device deletion or update of device alternative security identifiers.	-
AdminConsentRequired	Yes	-

Before December 3rd, 2020, when the application permission *Device.ReadWrite.All* was granted, the Device Managers directory role was also assigned to the app's service principal. This directory role assignment is not removed automatically when the associated application permissions is revoked. To ensure that an application's access to read or write to devices is removed, customers must also remove any related directory roles that were granted to the application.

A service update disabling this behavior began rolling out on December 3rd, 2020. Deployment to all customers completed on January 11th, 2021. Directory roles are no longer automatically assigned when application permissions are granted.

DeviceLocalCredential.Read.All

Category	Application	Delegated
Identifier	884b599e-4d48-43a5-ba94-15c414d00588	280b3b69-0437-44b1-bc20-3b2fca1ee3e9
DisplayText	Read device local credential passwords	Read device local credential passwords
Description	Allows the app to read device local credential properties including passwords, without a signed-in user.	Allows the app to read device local credential properties including passwords, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

DeviceLocalCredential.ReadBasic.All

Category	Application	Delegated
Identifier	db51be59-e728-414b-b800-e0f010df1a79	9917900e-410b-4d15-846e-42a357488545
DisplayText	Read device local credential properties	Read device local credential properties
Description	Allows the app to read device local credential properties excluding passwords, without a signed-in user.	Allows the app to read device local credential properties excluding passwords, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

DeviceManagementApps.Read.All

Category	Application	Delegated
Identifier	7a6ee1e7-141e-4cec-ae74-d9db155731ff	4edf5f54-4666-44af-9de9-0144fb4b6e8c
DisplayText	Read Microsoft Intune apps	Read Microsoft Intune apps
Description	Allows the app to read the properties, group assignments and status of apps, app configurations and app protection	Allows the app to read the properties, group assignments and status of apps, app configurations

Category	Application	Delegated
	policies managed by Microsoft Intune, without a signed-in user.	and app protection policies managed by Microsoft Intune.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementApps.ReadWrite.All

Category	Application	Delegated
Identifier	78145de6-330d-4800-a6ce-494ff2d33d07	7b3f05d5-f68c-4b8d-8c59-a2ecd12f24af
DisplayText	Read and write Microsoft Intune apps	Read and write Microsoft Intune apps
Description	Allows the app to read and write the properties, group assignments and status of apps, app configurations and app protection policies managed by Microsoft Intune, without a signed-in user.	Allows the app to read and write the properties, group assignments and status of apps, app configurations and app protection policies managed by Microsoft Intune.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementCloudCA.Read.All

Category	Application	Delegated
Identifier	315b6e8c-d92a-4691-919d-00ce76d1344a	ac5c8443-d999-471f-9247-ce92cf5c5560
DisplayText	Read Microsoft Cloud PKI objects	Read Microsoft Cloud PKI objects
Description	Allows the app to read certification authority information without a	Allows the app to read certification authority information on behalf of the

Category	Application	Delegated
	signed-in user.	signed-in user.
AdminConsentRequired	Yes	Yes

DeviceManagementCloudCA.ReadWrite.All

Category	Application	Delegated
Identifier	f15eb2ba-ef8a-4f70-991d-da5d045154e2	93028c58-65aa-48db-a706-1fe4ada325ec
DisplayText	Read and write Microsoft Cloud PKI objects	Read and write Microsoft Cloud PKI objects
Description	Allows the app to read and write certification authority information without a signed-in user.	Allows the app to read and write certification authority information on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

DeviceManagementConfiguration.Read.All

Category	Application	Delegated
Identifier	dc377aa6-52d8-4e23-b271-2a7ae04cedf3	f1493658-876a-4c87-8fa7-edb559b3476a
DisplayText	Read Microsoft Intune device configuration and policies	Read Microsoft Intune Device Configuration and Policies
Description	Allows the app to read properties of Microsoft Intune-managed device configuration and device compliance policies and their assignment to groups, without a signed-in user.	Allows the app to read properties of Microsoft Intune-managed device configuration and device compliance policies and their assignment to groups.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementConfiguration.ReadWrite.All

Category	Application	Delegated
Identifier	9241abd9-d0e6-425a-bd4f-47ba86e767a4	0883f392-0a7a-443d-8c76-16a6d39c7b63
DisplayText	Read and write Microsoft Intune device configuration and policies	Read and write Microsoft Intune Device Configuration and Policies
Description	Allows the app to read and write properties of Microsoft Intune-managed device configuration and device compliance policies and their assignment to groups, without a signed-in user.	Allows the app to read and write properties of Microsoft Intune-managed device configuration and device compliance policies and their assignment to groups.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementDeploymentPlans.Read.All

Category	Application	Delegated
Identifier	c5825671-0390-4bf2-b99b-80496fd4b673	700bfe0b-b3bd-4fa4-bec2-6849edd7fb7b
DisplayText	Read Microsoft Intune Deployment Plans	Read Microsoft Intune Deployment Plans
Description	Allows the app to read properties of Microsoft Intune-managed deployment plans and their ring configurations, without a signed-in user.	Allows the app to read properties of Microsoft Intune-managed deployment plans and their ring configurations.
AdminConsentRequired	Yes	Yes

DeviceManagementDeploymentPlans.ReadWrite.All

Category	Application	Delegated
Identifier	68356fd1-028d-4ce3-b724-241dec11127a	9d95843d-67b9-48b2-8e20-e42372db8549
DisplayText	Read and write Microsoft Intune Deployment Plans	Read and write Microsoft Intune Deployment Plans
Description	Allows the app to read and write properties of Microsoft Intune-managed deployment plans and their ring configurations, without a signed-in user.	Allows the app to read and write properties of Microsoft Intune-managed deployment plans and their ring configurations.
AdminConsentRequired	Yes	Yes

DeviceManagementManagedDevices.PrivilegedOperations.All

Category	Application	Delegated
Identifier	5b07b0dd-2377-4e44-a38d-703f09a0dc3c	3404d2bf-2b13-457e-a330-c24615765193
DisplayText	Perform user-impacting remote actions on Microsoft Intune devices	Perform user-impacting remote actions on Microsoft Intune devices
Description	Allows the app to perform remote high impact actions such as wiping the device or resetting the passcode on devices managed by Microsoft Intune, without a signed-in user.	Allows the app to perform remote high impact actions such as wiping the device or resetting the passcode on devices managed by Microsoft Intune.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementManagedDevices.Read.All

Category	Application	Delegated
Identifier	2f51be20-0bb4-4fed-bf7b-db946066c75e	314874da-47d6-4978-88dc-cf0d37f0bb82

Category	Application	Delegated
DisplayText	Read Microsoft Intune devices	Read Microsoft Intune devices
Description	Allows the app to read the properties of devices managed by Microsoft Intune, without a signed-in user.	Allows the app to read the properties of devices managed by Microsoft Intune.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementManagedDevices.ReadWrite.All

Category	Application	Delegated
Identifier	243333ab-4d21-40cb-a475-36241daa0842	44642bfe-8385-4adc-8fc6-fe3cb2c375c3
DisplayText	Read and write Microsoft Intune devices	Read and write Microsoft Intune devices
Description	Allows the app to read and write the properties of devices managed by Microsoft Intune, without a signed-in user. Does not allow high impact operations such as remote wipe and password reset on the device's owner	Allows the app to read and write the properties of devices managed by Microsoft Intune. Does not allow high impact operations such as remote wipe and password reset on the device's owner.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementRBAC.Read.All

Category	Application	Delegated
Identifier	58ca0d9a-1575-47e1-a3cb-007ef2e4583b	49f0cc30-024c-4dfd-ab3e-82e137ee5431
DisplayText	Read Microsoft Intune RBAC settings	Read Microsoft Intune RBAC settings
Description	Allows the app to read the properties relating to the Microsoft Intune Role-Based Access Control (RBAC) settings, without a signed-in user.	Allows the app to read the properties relating to the Microsoft Intune Role-Based Access Control (RBAC) settings.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementRBAC.ReadWrite.All

Category	Application	Delegated
Identifier	e330c4f0-4170-414e-a55a-2f022ec2b57b	0c5e8a55-87a6-4556-93ab-adc52c4d862d
DisplayText	Read and write Microsoft Intune RBAC settings	Read and write Microsoft Intune RBAC settings
Description	Allows the app to read and write the properties relating to the Microsoft Intune Role-Based Access Control (RBAC) settings, without a signed-in user.	Allows the app to read and write the properties relating to the Microsoft Intune Role-Based Access Control (RBAC) settings.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementScripts.Read.All

Category	Application	Delegated
Identifier	c7a5be92-2b3d-4540-8a67-c96dcaae8b43	d32381d8-ee89-4220-9c83-b672aa68d404
DisplayText	Read Microsoft Intune Scripts	Read Microsoft Intune Scripts
Description	Allows the app to read Microsoft Intune device compliance scripts, device management scripts, device shell scripts, device custom attribute shell scripts and device health scripts, without a signed-in user.	Allows the app to read Microsoft Intune device compliance scripts, device management scripts, device shell scripts, device custom attribute shell scripts and device health scripts on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

DeviceManagementScripts.ReadWrite.All

Category	Application	Delegated
Identifier	9255e99d-faf5-445e-bbf7-cb71482737c4	8b9d79d0-ad75-4566-8619-f7500ecfcebe
DisplayText	Read and write Microsoft Intune Scripts	Read and write Microsoft Intune Scripts
Description	Allows the app to read and write Microsoft Intune device compliance scripts, device management scripts, device shell scripts, device custom attribute shell scripts and device health scripts, without a signed-in user.	Allows the app to read and write Microsoft Intune device compliance scripts, device management scripts, device shell scripts, device custom attribute shell scripts and device health scripts on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

DeviceManagementServiceConfig.Read.All

Category	Application	Delegated
Identifier	06a5fe6d-c49d-46a7-b082-56b1b14103c7	8696daa5-bce5-4b2e-83f9-51b6defc4e1e
DisplayText	Read Microsoft Intune configuration	Read Microsoft Intune configuration
Description	Allows the app to read Microsoft Intune service properties including device enrollment and third party service	Allows the app to read Microsoft Intune service properties including

Category	Application	Delegated
	connection configuration, without a signed-in user.	device enrollment and third party service connection configuration.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceManagementServiceConfig.ReadWrite.All

Category	Application	Delegated
Identifier	5ac13192-7ace-4fcf-b828-1a26f28068ee	662ed50a-ac44-4eef-ad86-62eed9be2a29
DisplayText	Read and write Microsoft Intune configuration	Read and write Microsoft Intune configuration
Description	Allows the app to read and write Microsoft Intune service properties including device enrollment and third party service connection configuration, without a signed-in user.	Allows the app to read and write Microsoft Intune service properties including device enrollment and third party service connection configuration.
AdminConsentRequired	Yes	Yes

Using the Microsoft Graph APIs to configure Intune controls and policies still requires that the Intune service is correctly licensed by the customer.

These permissions aren't supported for personal Microsoft accounts.

DeviceTemplate.Create

Category	Application	Delegated
Identifier	abf6441f-0772-4932-96e7-0191478dd73a	0b1717ff-3e42-4a73-8c29-e6b2e1093960
DisplayText	Create device template	Create device templates

Category	Application	Delegated
Description	Allows the app to create device templates. The app is marked as owner of the created device template. As a member of owners, the app will be allowed to manage devices created from the template.	Allows the app to create device templates on behalf of the signed in user. The user is marked as owners of the created device template. As a member of owners, the user will be allowed to manage devices created from the template.
AdminConsentRequired	Yes	Yes

DeviceTemplate.Read.All

Category	Application	Delegated
Identifier	dd9febb5-0c6d-419f-b256-3afe12c6adeb	2bcae0b0-aa93-48e4-a9e4-855482dffdcd
DisplayText	Read all device templates	Read all device templates
Description	Allows the app to read all device templates, without a signed-in user.	Allows the app to read all device templates, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

DeviceTemplate.ReadWrite.All

Category	Application	Delegated
Identifier	9fadb66e-6421-4744-aede-4ab6fb98a884	2d372e98-f1ae-406c-a157-2ea83f6f5e4a
DisplayText	Read and write all device templates	Read and write all device templates
Description	Allows the app to create, read, update and delete any device template, without a signed-in user. It also allows the app to add or remove owners on any device template.	Allows the app to create, read, update and delete the device template, on behalf of the signed in user. It also allows the app to add or remove owners on any device template.
AdminConsentRequired	Yes	Yes

Directory.AccessAsUser.All

Category	Application	Delegated
Identifier	-	0e263e50-5827-48a4-b97c-d940288653c7
DisplayText	-	Access directory as the signed in user
Description	-	Allows the app to have the same access to information in the directory as the signed-in user.
AdminConsentRequired	-	Yes

> ⊗ Caution
>
> Directory permissions grant broad access to directory (Microsoft Entra ID) resources such as user, group, and device in an organization. Whenever possible, choose permissions specific to these resources and avoid using directory permissions.
>
> Directory permissions might be deprecated in the future.

Directory.Read.All

Category	Application	Delegated
Identifier	7ab1d382-f21e-4acd-a863-ba3e13f7da61	06da0dbc-49e2-44d2-8312-53f166ab848a
DisplayText	Read directory data	Read directory data
Description	Allows the app to read data in your organization's directory, such as users, groups and apps, without a signed-in user.	Allows the app to read data in your organization's directory, such as users, groups and apps.
AdminConsentRequired	Yes	Yes

> ⊗ Caution
>
> Directory permissions grant broad access to directory (Microsoft Entra ID) resources such as user, group, and device in an organization. Whenever possible, choose permissions specific to these resources and avoid using directory permissions.
>
> Directory permissions might be deprecated in the future.

Before December 3rd, 2020, when the application permission *Directory.Read.All* was granted, the Directory Readers directory role was also assigned to the app's service principal. This directory role isn't removed automatically when the associated application permissions are revoked. To remove an application's access to read or write to the directory, customers must also remove any directory roles that were granted to the application.

A service update disabling this behavior began rolling out on December 3rd, 2020. Deployment to all customers completed on January 11th, 2021. Directory roles are no longer automatically assigned when application permissions are granted.

Directory.ReadWrite.All

Category	Application	Delegated
Identifier	19dbc75e-c2e2-444c-a770-ec69d8559fc7	c5366453-9fb0-48a5-a156-24f0c49a4b84
DisplayText	Read and write directory data	Read and write directory data
Description	Allows the app to read and write data in your organization's directory, such as users, and groups, without a signed-in user. Does not allow user or group deletion.	Allows the app to read and write data in your organization's directory, such as users, and groups. It does not allow the app to delete users or groups, or reset user passwords.
AdminConsentRequired	Yes	Yes

> **⊗ Caution**
>
> Directory permissions grant broad access to directory (Microsoft Entra ID) resources such as **user**, **group**, and **device** in an organization. Whenever possible, choose permissions specific to these resources and avoid using directory permissions.
>
> Directory permissions might be deprecated in the future.

Before December 3rd, 2020, when the application permission *Directory.ReadWrite.All* was granted, the Directory Writers directory role was also assigned. This directory role isn't removed automatically when the associated application permissions are revoked. To remove an application's access to read or write to the directory, customers must also remove any directory roles that were granted to the application.

A service update disabling this behavior began rolling out on December 3rd, 2020. Deployment to all customers completed on January 11, 2021. Directory roles are no longer automatically assigned when application permissions are granted.

DirectoryRecommendations.Read.All

Category	Application	Delegated
Identifier	ae73097b-cb2a-4447-b064-5d80f6093921	34d3bd24-f6a6-468c-b67c-0c365c1d6410
DisplayText	Read all Azure AD recommendations	Read Azure AD recommendations
Description	Allows the app to read all Azure AD recommendations, without a signed-in user.	Allows the app to read Azure AD recommendations, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

DirectoryRecommendations.ReadWrite.All

Category	Application	Delegated
Identifier	0e9eea12-4f01-45f6-9b8d-3ea4c8144158	f37235e8-90a0-4189-93e2-e55b53867ccd
DisplayText	Read and update all Azure AD recommendations	Read and update Azure AD recommendations
Description	Allows the app to read and update all Azure AD recommendations, without a signed-in user.	Allows the app to read and update Azure AD recommendations, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Domain-InternalFederation.Read.All

Category	Application	Delegated
Identifier	c0e5a7b0-e8b7-40a7-b8e0-8249e6ea81d5	33203a2a-a761-40f0-8a7c-a7e74a9f8ac6

Category	Application	Delegated
DisplayText	Read internal federation configuration for a domain.	Read internal federation configuration for a domain.
Description	Allows the app to read internal federation configuration for a domain.	Allows the app to read internal federation configuration for a domain.
AdminConsentRequired	Yes	Yes

Domain-InternalFederation.ReadWrite.All

Category	Application	Delegated
Identifier	64d40371-8d58-4270-bc8a-b4a66de36b9a	857bd3ea-490e-4284-88a7-a7de1893b6ee
DisplayText	Create, read, update and delete internal federation configuration for a domain.	Create, read, update and delete internal federation configuration for a domain.
Description	Allows the app to create, read, update and delete internal federation configuration for a domain.	Allows the app to create, read, update and delete internal federation configuration for a domain.
AdminConsentRequired	Yes	Yes

Domain.Read.All

Category	Application	Delegated
Identifier	dbb9058a-0e50-45d7-ae91-66909b5d4664	2f9ee017-59c1-4f1d-9472-bd5529a7b311
DisplayText	Read domains	Read domains.
Description	Allows the app to read all domain properties without a signed-in user.	Allows the app to read all domain properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Domain.ReadWrite.All

Category	Application	Delegated
Identifier	7e05723c-0bb0-42da-be95-ae9f08a6e53c	0b5d694c-a244-4bde-86e6-eb5cd07730fe
DisplayText	Read and write domains	Read and write domains
Description	Allows the app to read and write all domain properties without a signed in user. Also allows the app to add, verify and remove domains.	Allows the app to read and write all domain properties on behalf of the signed-in user. Also allows the app to add, verify and remove domains.
AdminConsentRequired	Yes	Yes

EAS.AccessAsUser.All

Category	Application	Delegated
Identifier	-	ff91d191-45a0-43fd-b837-bd682c4a0b0f
DisplayText	-	Access mailboxes via Exchange ActiveSync
Description	-	Allows the app to have the same access to mailboxes as the signed-in user via Exchange ActiveSync.
AdminConsentRequired	-	No

eDiscovery.Read.All

Category	Application	Delegated
Identifier	50180013-6191-4d1e-a373-e590ff4e66af	99201db3-7652-4d5a-809a-bdb94f85fe3c
DisplayText	Read all eDiscovery objects	Read all eDiscovery objects
Description	Allows the app to read eDiscovery objects such as cases, custodians, review sets and other related objects without a signed-in user.	Allows the app to read eDiscovery objects such as cases, custodians, review sets and other related objects on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

eDiscovery.ReadWrite.All

Category	Application	Delegated
Identifier	b2620db1-3bf7-4c5b-9cb9-576d29eac736	acb8f680-0834-4146-b69e-4ab1b39745ad
DisplayText	Read and write all eDiscovery objects	Read and write all eDiscovery objects
Description	Allows the app to read and write eDiscovery objects such as cases, custodians, review sets and other related objects without a signed-in user.	Allows the app to read and write eDiscovery objects such as cases, custodians, review sets and other related objects on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EduAdministration.Read

Category	Application	Delegated
Identifier	-	8523895c-6081-45bf-8a5d-f062a2f12c9f
DisplayText	-	Read education app settings
Description	-	Read the state and settings of all Microsoft education apps on behalf of the user.
AdminConsentRequired	-	Yes

EduAdministration.Read.All

Category	Application	Delegated
Identifier	7c9db06a-ec2d-4e7b-a592-5a1e30992566	-
DisplayText	Read Education app settings	-
Description	Read the state and settings of all Microsoft education apps.	-
AdminConsentRequired	Yes	-

EduAdministration.ReadWrite

Category	Application	Delegated
Identifier	-	63589852-04e3-46b4-bae9-15d5b1050748
DisplayText	-	Manage education app settings
Description	-	Manage the state and settings of all Microsoft education apps on behalf of the user.
AdminConsentRequired	-	Yes

EduAdministration.ReadWrite.All

Category	Application	Delegated
Identifier	9bc431c3-b8bc-4a8d-a219-40f10f92eff6	-
DisplayText	Manage education app settings	-
Description	Manage the state and settings of all Microsoft education apps.	-
AdminConsentRequired	Yes	-

EduAssignments.Read

Category	Application	Delegated
Identifier	-	091460c9-9c4a-49b2-81ef-1f3d852acce2
DisplayText	-	Read users' class assignments and their grades
Description	-	Allows the app to read assignments and their grades on behalf of the user.
AdminConsentRequired	-	Yes

EduAssignments.Read.All

Category	Application	Delegated
Identifier	4c37e1b6-35a1-43bf-926a-6f30f2cdf585	-
DisplayText	Read all class assignments with grades	-

Category	Application	Delegated
Description	Allows the app to read all class assignments with grades for all users without a signed-in user.	-
AdminConsentRequired	Yes	-

EduAssignments.ReadBasic

Category	Application	Delegated
Identifier	-	c0b0103b-c053-4b2e-9973-9f3a544ec9b8
DisplayText	-	Read users' class assignments without grades
Description	-	Allows the app to read assignments without grades on behalf of the user.
AdminConsentRequired	-	Yes

EduAssignments.ReadBasic.All

Category	Application	Delegated
Identifier	6e0a958b-b7fc-4348-b7c4-a6ab9fd3dd0e	-
DisplayText	Read all class assignments without grades	-
Description	Allows the app to read all class assignments without grades for all users without a signed-in user.	-
AdminConsentRequired	Yes	-

EduAssignments.ReadWrite

Category	Application	Delegated
Identifier	-	2f233e90-164b-4501-8bce-31af2559a2d3
DisplayText	-	Read and write users' class assignments and their grades
Description	-	Allows the app to read and write assignments and their grades on behalf of the user.

Category	Application	Delegated
AdminConsentRequired	-	Yes

EduAssignments.ReadWrite.All

Category	Application	Delegated
Identifier	0d22204b-6cad-4dd0-8362-3e3f2ae699d9	-
DisplayText	Create, read, update and delete all class assignments with grades	-
Description	Allows the app to create, read, update and delete all class assignments with grades for all users without a signed-in user.	-
AdminConsentRequired	Yes	-

EduAssignments.ReadWriteBasic

Category	Application	Delegated
Identifier	-	2ef770a1-622a-47c4-93ee-28d6adbed3a0
DisplayText	-	Read and write users' class assignments without grades
Description	-	Allows the app to read and write assignments without grades on behalf of the user.
AdminConsentRequired	-	Yes

EduAssignments.ReadWriteBasic.All

Category	Application	Delegated
Identifier	f431cc63-a2de-48c4-8054-a34bc093af84	-
DisplayText	Create, read, update and delete all class assignments without grades	-
Description	Allows the app to create, read, update and delete all class assignments without grades for all users without a signed-in user.	-
AdminConsentRequired	Yes	-

EduCurricula.Read

Category	Application	Delegated
Identifier	-	484859e8-b9e2-4e92-b910-84db35dadd29
DisplayText	-	Read the user's class modules and resources
Description	-	Allows the app to read the user's modules and resources on behalf of the signed-in user.
AdminConsentRequired	-	Yes

EduCurricula.Read.All

Category	Application	Delegated
Identifier	6cdb464c-3a03-40f8-900b-4cb7ea1da9c0	-
DisplayText	Read all class modules and resources	-
Description	Allows the app to read all modules and resources, without a signed-in user.	-
AdminConsentRequired	Yes	-

EduCurricula.ReadWrite

Category	Application	Delegated
Identifier	-	4793c53b-df34-44fd-8d26-d15c517732f5
DisplayText	-	Read and write the user's class modules and resources
Description	-	Allows the app to read and write user's modules and resources on behalf of the signed-in user.
AdminConsentRequired	-	Yes

EduCurricula.ReadWrite.All

Category	Application	Delegated
Identifier	6a0c2318-d59d-4c7d-bf2e-5f3902dc2593	-
DisplayText	Read and write all class modules and resources	-
Description	Allows the app to read and write all modules and resources, without a signed-in user.	-
AdminConsentRequired	Yes	-

EduReports-Reading.Read.All

Category	Application	Delegated
Identifier	ad248c30-1919-40c8-b3d2-304481894e88	-
DisplayText	Read all tenant reading assignments submissions data	-
Description	Allows the app to read all tenant users reading assignments submissions data without a signed-in user.	-
AdminConsentRequired	Yes	-

EduReports-Reading.ReadAnonymous.All

Category	Application	Delegated
Identifier	040330d7-be7e-4130-b349-a6eb3a56e2f8	-
DisplayText	Read all tenant reading assignments submissions data	-
Description	Allows the app to read all tenant users reading assignments submissions data (excludes student-identifying information) without a signed-in user.	-
AdminConsentRequired	Yes	-

EduReports-Reflect.Read.All

Category	Application	Delegated
Identifier	c5debf73-bdc8-473d-bf07-f4074ad05f71	-

Category	Application	Delegated
DisplayText	Read all tenant reflect check-ins submissions data	-
Description	Allows the app to read all tenant users reflect check-ins submissions data without a signed-in user.	-
AdminConsentRequired	Yes	-

EduReports-Reflect.ReadAnonymous.All

Category	Application	Delegated
Identifier	f5d05dba-7ef0-46fc-b62c-a7282555f428	-
DisplayText	Read all tenant reflect check-ins submissions data	-
Description	Allows the app to read all tenant users reflect check-ins submissions data (excludes responder-identifying information) without a signed-in user.	-
AdminConsentRequired	Yes	-

EduRoster.Read

Category	Application	Delegated
Identifier	-	a4389601-22d9-4096-ac18-36a927199112
DisplayText	-	Read users' view of the roster
Description	-	Allows the app to read the structure of schools and classes in an organization's roster and education-specific information about users to be read on behalf of the user.
AdminConsentRequired	-	Yes

EduRoster.Read.All

Category	Application	Delegated
Identifier	e0ac9e1b-cb65-4fc5-87c5-1a8bc181f648	-

Category	Application	Delegated
DisplayText	Read the organization's roster	-
Description	Allows the app to read the structure of schools and classes in the organization's roster and education-specific information about all users to be read.	-
AdminConsentRequired	Yes	-

EduRoster.ReadBasic

Category	Application	Delegated
Identifier	-	5d186531-d1bf-4f07-8cea-7c42119e1bd9
DisplayText	-	Read a limited subset of users' view of the roster
Description	-	Allows the app to read a limited subset of the properties from the structure of schools and classes in an organization's roster and a limited subset of properties about users to be read on behalf of the user. Includes name, status, education role, email address and photo.
AdminConsentRequired	-	Yes

EduRoster.ReadBasic.All

Category	Application	Delegated
Identifier	0d412a8c-a06c-439f-b3ec-8abcf54d2f96	-
DisplayText	Read a limited subset of the organization's roster	-
Description	Allows the app to read a limited subset of properties from both the structure of schools and classes in the organization's roster and education-specific information about all users. Includes name, status, role, email address and photo.	-
AdminConsentRequired	Yes	-

EduRoster.ReadWrite

Category	Application	Delegated
Identifier	-	359e19a6-e3fa-4d7f-bcab-d28ec592b51e
DisplayText	-	Read and write users' view of the roster
Description	-	Allows the app to read and write the structure of schools and classes in an organization's roster and education-specific information about users to be read and written on behalf of the user.
AdminConsentRequired	-	Yes

EduRoster.ReadWrite.All

Category	Application	Delegated
Identifier	d1808e82-ce13-47af-ae0d-f9b254e6d58a	-
DisplayText	Read and write the organization's roster	-
Description	Allows the app to read and write the structure of schools and classes in the organization's roster and education-specific information about all users to be read and written.	-
AdminConsentRequired	Yes	-

email

Category	Application	Delegated
Identifier	-	64a6cdd6-aab1-4aaf-94b8-3cc8405e90d0
DisplayText	-	View users' email address
Description	-	Allows the app to read your users' primary email address
AdminConsentRequired	-	No

email is an OpenID Connect (OIDC) scope.

You can use the OIDC scopes to specify artifacts that you want returned in Microsoft identity platform authorization and token requests. The Microsoft identity platform v1.0 and v2.0 endpoints support OIDC scopes differently.

With the Microsoft identity platform v1.0 endpoint, only the *openid* scope is used. You specify it in the *scope* parameter in an authorization request to return an ID token when you use the OpenID Connect protocol to sign in a user to your app. For more information, see Microsoft identity platform and OAuth 2.0 authorization code flow. To successfully return an ID token, you must also make sure that the *User.Read* permission is configured when you register your app.

With the Microsoft identity platform v2.0 endpoint, you specify the *offline_access* scope in the **scope** parameter to explicitly request a refresh token when using the OAuth 2.0 or OpenID Connect protocols. With OpenID Connect, you specify the *openid* scope to request an ID token. You can also specify the *email* scope, *profile* scope, or both to return additional claims in the ID token. You don't need to specify the *User.Read* permission to return an ID token with the v2.0 endpoint. For more information, see OpenID Connect scopes.

The Microsoft Authentication Library (MSAL) currently specifies *offline_access*, *openid*, *profile*, and *email* by default in authorization and token requests. Therefore, for the default case, if you specify these scopes explicitly, the Microsoft identity platform might return an error.

EngagementConversation.Migration.All

Category	Application	Delegated
Identifier	e1d2136d-eaaf-427a-a7db-f97dbe847c27	-
DisplayText	Read and write all Viva Engage conversations	-
Description	Allows the app to create Viva Engage conversations without a signed-in user.	-
AdminConsentRequired	Yes	-

EngagementConversation.Read.All

Category	Application	Delegated
Identifier	2c495153-cd0e-41b4-9980-3bcecf1ca22f	c55541d9-2cdd-4fad-8ead-0c08fae5b0c8
DisplayText	Read all Viva Engage conversations	Read all Viva Engage conversations

Category	Application	Delegated
Description	Allows the app to list Viva Engage conversations, and to read their properties without a signed-in user.	Allows the app to read Viva Engage conversations, and to read their properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EngagementConversation.ReadWrite.All

Category	Application	Delegated
Identifier	bfbd4840-fba0-43a7-93a9-465b687e47d0	ebbfd079-1634-4640-8618-68b19ebbed1d
DisplayText	Read and write all Viva Engage conversations	Read and write all Viva Engage conversations
Description	Allows the app to create Viva Engage conversations, read all conversation properties, update conversation properties, and delete conversations without a signed-in user.	Allows the app to create Viva Engage conversations and read all conversation properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EngagementExport.Read.All

Category	Application	Delegated
Identifier	eda8c187-a7d5-42cb-b2e1-c9142f63899f	-
DisplayText	Export Viva Engage data	-
Description	Allows the app to export Viva Engage data for compliance, GDPR, and admin scenarios without a signed-in user.	-
AdminConsentRequired	Yes	-

EngagementMeetingConversation.Read.All

Category	Application	Delegated
Identifier	d746beae-b46e-446e-924a-5b805a5c4467	58c5819e-29bd-4400-ad52-82cd82a63fbd
DisplayText	Read all Viva Engage Teams QA conversations	Read all Viva Engage Teams QA conversations
Description	Allows the app to list Viva Engage Teams QA conversations, and to read their properties without a signed-in user.	Allows the app to read Viva Engage Teams QA conversations, and to read their properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EngagementRole.Read

Category	Application	Delegated
Identifier	-	9f1da0fc-345c-4dfb-bab5-5215a073a417
DisplayText	-	Read a user's Viva Engage roles
Description	-	Allows the app to list a user's Viva Engage roles, on behalf of the signed-in user.
AdminConsentRequired	-	No

EngagementRole.Read.All

Category	Application	Delegated
Identifier	30614864-4114-45ef-bdd9-0dd7894a1cc4	3cad91a5-8413-4c4a-acfe-dfeb83d1366d
DisplayText	Read all Viva Engage roles and role memberships	Read all Viva Engage roles and role memberships
Description	Allows the app to list all Viva Engage roles and role memberships without a signed-in user.	Allows the app to list all Viva Engage roles and role memberships on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EngagementRole.ReadWrite.All

Category	Application	Delegated
Identifier	3ede5358-7366-4da8-a2f7-472bf9c7cc34	4905982d-6459-4ccd-949c-949fefc0a8f2
DisplayText	Modify Viva Engage role membership	Modify Viva Engage role membership
Description	Allows the app to assign Viva Engage role to a user, and remove a Viva Engage role from a user without a signed-in user.	Allows the app to assign Viva Engage role to a user, and remove a Viva Engage role from a user behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EntitlementManagement.Read.All

Category	Application	Delegated
Identifier	c74fd47d-ed3c-45c3-9a9e-b8676de685d2	5449aa12-1393-4ea2-a7c7-d0e06c1a56b2
DisplayText	Read all entitlement management resources	Read all entitlement management resources
Description	Allows the app to read access packages and related entitlement management resources without a signed-in user.	Allows the app to read access packages and related entitlement management resources on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EntitlementManagement.ReadWrite.All

Category	Application	Delegated
Identifier	9acd699f-1e81-4958-b001-93b1d2506e19	ae7a573d-81d7-432b-ad44-4ed5c9d89038
DisplayText	Read and write all entitlement management resources	Read and write entitlement management resources
Description	Allows the app to read and write access packages and related	Allows the app to request access to and management of access packages and

Category	Application	Delegated
	entitlement management resources without a signed-in user.	related entitlement management resources on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

> **⊗ Caution**
>
> Permissions that allow granting authorization, such as *EntitlementManagement.ReadWrite.All*, allow an application to grant additional privileges to itself, other applications, or any user. Use caution when granting any of these permissions.
>
> With the *EntitlementManagement.ReadWrite.All* permission, an application can manage authorizations for all resources that have been **onboarded by resource owners to entitlement management catalogs**. This includes Microsoft Entra role assignments, app role assignments and API permissions (including Microsoft Graph) of Microsoft Entra integrated apps, Azure resource roles, SharePoint site roles, group memberships, and more. For details, see **Add a resource to a catalog**.

EntitlementMgmt-SubjectAccess.ReadWrite

Category	Application	Delegated
Identifier	-	e9fdcbbb-8807-410f-b9ec-8d5468c7c2ac
DisplayText	-	Read and write entitlement management resources related to self-service operations
Description	-	Allows the app to manage self-service entitlement management resources on behalf of the signed-in user. This includes operations such as requesting access and approving access of others.
AdminConsentRequired	-	No

EntraBackup.Read.All

Category	Application	Delegated
Identifier	56eda3c5-3834-4815-bd41-6f8fa1295247	a6ea9dd7-4dd9-4484-a80a-ac9ad981dcf1

Category	Application	Delegated
DisplayText	Read Preview jobs and snapshots	Read Preview jobs and snapshots
Description	Allows the app to list the all the snapshots, jobs and enumerate the changes of a specific preview job, on behalf of the signed-in user.	Allows the app to list the all the snapshots, jobs and enumerate the changes of a specific preview job, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EntraBackup.ReadWrite.Preview

Category	Application	Delegated
Identifier	-	cef123a8-c18c-4eba-852e-d90cfbf67c91
DisplayText	-	Create a preview job, read preview job and snapshots
Description	-	Allows the app to list the all the snapshots, create a preview job and enumerate the changes of a specific preview job, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

EntraBackup.ReadWrite.Recovery

Category	Application	Delegated
Identifier	-	8269c6ff-41d7-4172-a783-b2ce38322e42
DisplayText	-	Create preview and recovery job, read recovery job and snapshots
Description	-	Allows the app to list the all the snapshots, create a recovery job and enumerate the changes of a specific recovery job, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

EventListener.Read.All

Category	Application	Delegated
Identifier	b7f6385c-6ce6-4639-a480-e23c42ed9784	f7dd3bed-5eec-48da-bc73-1c0ef50bc9a1
DisplayText	Read all authentication event listeners	Read your organization's authentication event listeners
Description	Allows the app to read your organization's authentication event listeners without a signed-in user.	Allows the app to read your organization's authentication event listeners on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EventListener.ReadWrite.All

Category	Application	Delegated
Identifier	0edf5e9e-4ce8-468a-8432-d08631d18c43	d11625a6-fe21-4fc6-8d3d-063eba5525ad
DisplayText	Read and write all authentication event listeners	Read and write your organization's authentication event listeners
Description	Allows the app to read or write your organization's authentication event listeners without a signed-in user.	Allows the app to read or write your organization's authentication event listeners on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

EWS.AccessAsUser.All

Category	Application	Delegated
Identifier	-	9769c687-087d-48ac-9cb3-c37dde652038
DisplayText	-	Access mailboxes as the signed-in user via Exchange Web Services
Description	-	Allows the app to have the same access to mailboxes as the signed-in user via Exchange Web Services.
AdminConsentRequired	-	No

ExchangeMessageTrace.Read.All

Category	Application	Delegated
Identifier	89b20d8a-76e2-4057-867b-9961f800b9a4	b2e7d27e-14e7-41ad-bb15-a88ceb9c3e90
DisplayText	Search the email message trace	Search the email message trace
Description	Allows the app to search the email message trace, without a signed-in user.	Allows the app to search the email message trace on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ExternalConnection.Read.All

Category	Application	Delegated
Identifier	1914711b-a1cb-4793-b019-c2ce0ed21b8c	a38267a5-26b6-4d76-9493-935b7599116b
DisplayText	Read all external connections	Read all external connections
Description	Allows the app to read all external connections without a signed-in user.	Allows the app to read all external connections on behalf of a signed-in user. The signed-in user must be an administrator.
AdminConsentRequired	Yes	Yes

ExternalConnection.ReadWrite.All

Category	Application	Delegated
Identifier	34c37bc0-2b40-4d5e-85e1-2365cd256d79	bbbbd9b3-3566-4931-ac37-2b2180d9e334
DisplayText	Read and write all external connections	Read and write all external connections
Description	Allows the app to read and write all external connections without a signed-in user.	Allows the app to read and write all external connections on behalf of a signed-in user. The signed-in user must be an administrator.
AdminConsentRequired	Yes	Yes

ExternalConnection.ReadWrite.OwnedBy

Category	Application	Delegated
Identifier	f431331c-49a6-499f-be1c-62af19c34a9d	4082ad95-c812-4f02-be92-780c4c4f1830
DisplayText	Read and write external connections	Read and write external connections
Description	Allows the app to read and write external connections without a signed-in user. The app can only read and write external connections that it is authorized to, or it can create new external connections.	Allows the app to read and write settings of external connections on behalf of a signed-in user. The signed-in user must be an administrator. The app can only read and write settings of connections that it is authorized to.
AdminConsentRequired	Yes	Yes

ExternalItem.Read.All

Category	Application	Delegated
Identifier	7a7cffad-37d2-4f48-afa4-c6ab129adcc2	922f9392-b1b7-483c-a4be-0089be7704fb
DisplayText	Read all external items	Read items in external datasets
Description	Allows the app to read all external items without a signed-in user.	Allow the app to read external datasets and content, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ExternalItem.ReadWrite.All

Category	Application	Delegated
Identifier	38c3d6ee-69ee-422f-b954-e17819665354	b02c54f8-eb48-4c50-a9f0-a149e5a2012f
DisplayText	Read and write items in external datasets	Read and write all external items
Description	Allow the app to read or write items in all external datasets that the app is authorized to access	Allows the app to read and write all external items on behalf of a signed-in

Category	Application	Delegated
		user. The signed-in user must be an administrator.
AdminConsentRequired	Yes	Yes

ExternalItem.ReadWrite.OwnedBy

Category	Application	Delegated
Identifier	8116ae0f-55c2-452d-9944-d18420f5b2c8	4367b9d7-cee7-4995-853c-a0bdfe95c1f9
DisplayText	Read and write external items	Read and write external items
Description	Allows the app to read and write external items without a signed-in user. The app can only read external items of the connection that it is authorized to.	Allows the app to read and write external items on behalf of a signed-in user. The signed-in user must be an administrator. The app can only read external items of the connection that it is authorized to.
AdminConsentRequired	Yes	Yes

ExternalUserProfile.Read.All

Category	Application	Delegated
Identifier	1987d7a0-d602-4262-ab90-cfdd43b37545	47167bec-55a7-4caf-9ecc-8d4566e3cfb1
DisplayText	Read all external user profiles	Read external user profiles
Description	Allows the app to read available properties of external user profiles, without a signed-in user.	Allows the app to read available properties of external user profiles, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ExternalUserProfile.ReadWrite.All

Category	Application	Delegated
Identifier	761327c9-d819-4c08-9a5f-874cd2826608	c6068dc7-a791-46a4-a811-b8228e6649ab
DisplayText	Read and write all external user profiles	Read and write external user profiles
Description	Allows the app to read and write available properties of external user profiles, without a signed-in user.	Allows the app to read and write available properties of external user profiles, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Family.Read

Category	Application	Delegated
Identifier	-	3a1e4806-a744-4c70-80fc-223bf8582c46
DisplayText	-	Read your family info
Description	-	Allows the app to read your family information, members and their basic profile.
AdminConsentRequired	-	No

FileIngestion.Ingest

Category	Application	Delegated
Identifier	65891b00-2fd9-4e33-be27-04a53132e3df	-
DisplayText	Ingest SharePoint and OneDrive content to make it available in the search index	-
Description	Allows the app to ingest SharePoint and OneDrive content to make it available in the search index, without a signed-in user.	-
AdminConsentRequired	Yes	-

FileIngestionHybridOnboarding.Manage

Category	Application	Delegated
Identifier	766c601b-c009-4438-8290-c8b05fa00c4b	-
DisplayText	Manage onboarding for a Hybrid Cloud tenant	-
Description	Allows the app to manage onboarding for a Hybrid Cloud tenant, without a signed-in user.	-
AdminConsentRequired	Yes	-

Files.Read

Category	Application	Delegated
Identifier	-	10465720-29dd-4523-a11a-6a75c743c9d9
DisplayText	-	Read user files
Description	-	Allows the app to read the signed-in user's files.
AdminConsentRequired	-	No

The *Files.Read* delegated permission is available for consent in personal Microsoft accounts.

For personal accounts, *Files.Read* also grant access to files shared with the signed-in user.

Files.Read.All

Category	Application	Delegated
Identifier	01d4889c-1287-42c6-ac1f-5d1e02578ef6	df85f4d6-205c-4ac5-a5ea-6bf408dba283
DisplayText	Read files in all site collections	Read all files that user can access
Description	Allows the app to read all files in all site collections without a signed in user.	Allows the app to read all files the signed-in user can access.
AdminConsentRequired	Yes	No

The *Files.Read.All* delegated permission is available for consent in personal Microsoft accounts.

Files.Read.Selected

Category	Application	Delegated
Identifier	-	5447fe39-cb82-4c1a-b977-520e67e724eb
DisplayText	-	Read files that the user selects (preview)
Description	-	(Preview) Allows the app to read files that the user selects. The app has access for several hours after the user selects a file.
AdminConsentRequired	-	No

The *Files.Read.Selected* delegated permission is only valid on work or school accounts and is only exposed for working with Office 365 file handlers (v1.0). It should not be used for directly calling Microsoft Graph APIs.

Files.ReadWrite

Category	Application	Delegated
Identifier	-	5c28f0bf-8a70-41f1-8ab2-9032436ddb65
DisplayText	-	Have full access to user files
Description	-	Allows the app to read, create, update and delete the signed-in user's files.
AdminConsentRequired	-	No

The *Files.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

For personal accounts, *Files.ReadWrite* also grant access to files shared with the signed-in user.

Files.ReadWrite.All

Category	Application	Delegated
Identifier	75359482-378d-4052-8f01-80520e7db3cd	863451e7-0667-486c-a5d6-d135439485f0
DisplayText	Read and write files in all site collections	Have full access to all files user can access
Description	Allows the app to read, create, update and delete all files in all site collections	Allows the app to read, create, update and delete all files the

Category	Application	Delegated
	without a signed in user.	signed-in user can access.
AdminConsentRequired	Yes	No

The *Files.ReadWrite.All* delegated permission is available for consent in personal Microsoft accounts.

Files.ReadWrite.AppFolder

Category	Application	Delegated
Identifier	b47b160b-1054-4efd-9ca0-e2f614696086	8019c312-3263-48e6-825e-2b833497195b
DisplayText	Have full access to the application's folder without a signed in user.	Have full access to the application's folder (preview)
Description	Allows the app to read, create, update and delete files in the application's folder without a signed in user.	(Preview) Allows the app to read, create, update and delete files in the application's folder.
AdminConsentRequired	Yes	No

The *Files.ReadWrite.AppFolder* delegated permission is available for consent in personal Microsoft accounts.

Files.ReadWrite.Selected

Category	Application	Delegated
Identifier	-	17dde5bd-8c17-420f-a486-969730c1b827
DisplayText	-	Read and write files that the user selects (preview)
Description	-	(Preview) Allows the app to read and write files that the user selects. The app has access for several hours after the user selects a file.
AdminConsentRequired	-	No

The *Files.ReadWrite.Selected* delegated permission is only valid on work or school accounts and is only exposed for working with Office 365 file handlers (v1.0). It should not be used for directly

calling Microsoft Graph APIs.

Files.SelectedOperations.Selected

Category	Application	Delegated
Identifier	bd61925e-3bf4-4d62-bc0b-06b06c96d95c	ef2779dc-ef1b-4211-8310-8a0ac2450081
DisplayText	Access selected Files without a signed in user.	Access selected Files, on behalf of the signed-in user
Description	Allow the application to access a subset of files without a signed in user. The specific files and the permissions granted will be configured in SharePoint Online or OneDrive.	Allow the application to access files explicitly permissioned to the application on behalf of the signed in user. The specific files and the permissions granted will be configured in SharePoint Online or OneDrive.
AdminConsentRequired	Yes	Yes

FileStorageContainer.Manage.All

Category	Application	Delegated
Identifier	-	527b6d64-cdf5-4b8b-b336-4aa0b8ca2ce5
DisplayText	-	Manage all file storage containers
Description	-	Allows the application to utilize the file storage container administration capabilities on behalf of an administrator user.
AdminConsentRequired	-	Yes

FileStorageContainer.Selected

Category	Application	Delegated
Identifier	40dc41bc-0f7e-42ff-89bd-d9516947e474	085ca537-6565-41c2-aca7-db852babc212
DisplayText	Access selected file storage containers	Access selected file storage containers

Category	Application	Delegated
Description	Allows the application to utilize the file storage container platform to manage containers, without a signed-in user. The specific file storage containers and the permissions granted to them will be configured in Microsoft 365 by the developer of each container type.	Allows the application to utilize the file storage container platform to manage containers on behalf of the signed in user. The specific file storage containers and the permissions granted to them will be configured in Microsoft 365 by the developer of each container type.
AdminConsentRequired	Yes	No

FileStorageContainerType.Manage.All

Category	Application	Delegated
Identifier	-	8e6ec84c-5fcd-4cc7-ac8a-2296efc0ed9b
DisplayText	-	Manage file storage container types on behalf of the signed in user
Description	-	Allows the application to manage file storage container types on behalf of the signed in user. The user must be a SharePoint Embedded Admin or Global Admin.
AdminConsentRequired	-	No

FileStorageContainerTypeReg.Manage.All

Category	Application	Delegated
Identifier	-	c319a7df-930e-44c0-a43b-7e5e9c7f4f24
DisplayText	-	Manage file storage container type registrations on behalf of the signed in user
Description	-	Allows the application to manage file storage container type registrations on behalf of the signed in user. The user must be a SharePoint Embedded Admin or Global Admin.
AdminConsentRequired	-	No

FileStorageContainerTypeReg.Selected

Category	Application	Delegated
Identifier	2dcc6599-bd30-442b-8f11-90f88ad441dc	d1e4f63a-1569-475c-b9b2-bdc140405e38
DisplayText	Access selected file storage container type registrations	Access selected file storage container type registrations.
Description	Allows the application to manage file storage container type registrations without a signed-in user.	Allows the application to manage selected file storage container type registrations on behalf of the signed in user. The user must be a SharePoint Embedded Admin or Global Admin.
AdminConsentRequired	Yes	No

Financials.ReadWrite.All

Category	Application	Delegated
Identifier	-	f534bf13-55d4-45a9-8f3c-c92fe64d6131
DisplayText	-	Read and write financials data
Description	-	Allows the app to read and write financials data on behalf of the signed-in user.
AdminConsentRequired	-	No

Goals-Export.Read.All

Category	Application	Delegated
Identifier	-	092211d9-ca1a-427b-813e-b79c7653fe71
DisplayText	-	Read all goals and export jobs that a user can access
Description	-	Allows the app to read all goals and export jobs that the signed-in user can access.
AdminConsentRequired	-	Yes

Goals-Export.ReadWrite.All

Category	Application	Delegated
Identifier	-	2edeb9fd-4228-480c-a26d-2ed52011cf3d
DisplayText	-	Have full access to all goals and export jobs a user can access
Description	-	Allows the app to read goals, create and read export jobs that the signed-in user can access.
AdminConsentRequired	-	Yes

Group-Conversation.Read.All

Category	Application	Delegated
Identifier	4f0a8235-6f6f-4ec7-9500-34b452a4a0c3	c92fbbc2-50e0-4842-93ef-385c3293ea3d
DisplayText	Read all group conversations	Read group conversations
Description	Allows the app to read conversations of the groups this app has access to without a signed-in user.	Allows the app to read group conversations that the signed-in user has access to.
AdminConsentRequired	Yes	Yes

Group-Conversation.ReadWrite.All

Category	Application	Delegated
Identifier	6679c91b-820a-4900-ab47-e97b197a89c4	302bcbb5-855a-4e49-ae20-94a331b0281e
DisplayText	Read and write all group conversations	Read and write group conversations
Description	Allows the app to read and write conversations of the groups this app has access to without a signed-in user.	Allows the app to read and write group conversations that the signed-in user has access to.
AdminConsentRequired	Yes	Yes

Group-NestingSupport.ReadWrite.All

Category	Application	Delegated
Identifier	2d53948b-d2c5-4008-9c4e-6361bf192555	afc507db-8793-4d2f-999d-6e34cff02b7c
DisplayText	Read and write groups' disableNesting property	Read and write groups' disableNesting property
Description	Allows the app to read and write groups' disableNesting property without a signed-in user.	Allows the app to read and write groups' disableNesting property on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Group-OnPremisesSyncBehavior.ReadWrite.All

Category	Application	Delegated
Identifier	2d9bd318-b883-40be-9df7-63ec4fcdc424	37e00479-5776-4659-aecf-4841ec5d590a
DisplayText	Read and update the on-premises sync behavior of groups	Read and update the on-premises sync behavior of groups
Description	Allows the app to update the on-premises sync behavior of all groups without a signed-in user.	Allows the app to read and update the on-premises sync behavior of groups on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Group-XTenantIdentitySync.Read.All

Category	Application	Delegated
Identifier	35b96aac-d839-4362-abd4-7381f2b27ccd	f7c0661b-4247-48ac-a371-05ff047614c6
DisplayText	Read all Group Cross-Tenant Identity Synchronization properties	Read all Group Cross-Tenant Identity Synchronization properties
Description	Allows the app to read all Cross-Tenant Identity Synchronization properties on Groups, without a signed-in user.	Allows the app to read all Cross-Tenant Identity Synchronization properties on Groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Group.Create

Category	Application	Delegated
Identifier	bf7b1a76-6e77-406b-b258-bf5c7720e98f	-
DisplayText	Create groups	-
Description	Allows the app to create groups without a signed-in user.	-
AdminConsentRequired	Yes	-

Group.ManageProtection.All

Category	Application	Delegated
Identifier	60f8cea0-2476-45c9-ab18-70e79e60ad14	36263ed6-285e-4f84-b25a-62ec2ba17d29
DisplayText	Manage the Microsoft Information Protection (MIP) label for M365 and security groups.	Manage the Microsoft Information Protection (MIP) label for M365 and security groups.
Description	Allows the app to list groups, and to read their basic properties and manage the MIP label for all label enabled groups without a signed-in user.	Allows the app to list groups, and to read their basic properties and manage the MIP label for all label enabled groups on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Group.Read.All

Category	Application	Delegated
Identifier	5b567255-7703-4780-807c-7be8301ae99b	5f8c59db-677d-491f-a6b8-5f174b11ec1d
DisplayText	Read all groups	Read all groups
Description	Allows the app to read group properties and memberships, and read conversations for all	Allows the app to list groups, and to read their properties and all group memberships on behalf of the signed-in user. Also allows the app to read calendar, conversations, files, and other

Category	Application	Delegated
	groups, without a signed-in user.	group content for all groups the signed-in user can access.
AdminConsentRequired	Yes	Yes

For Microsoft 365 groups, *Group.** permissions grant the app access to the contents of the group; for example, conversations, files, notes, and so on.

In some cases, an app might need extra permissions to read some group properties like `member` and `memberOf`. For example, if a group has one or more service principals as members, the app also needs permissions to read service principals, otherwise Microsoft Graph returns an error or limited information. To read the full information, the app also needs permissions in the organization to read service principals. For more information, see Limited information returned for inaccessible member objects.

*Group.** permissions are used to control access to Microsoft Teams resources and APIs. Personal Microsoft accounts are not supported.

*Group.** permissions are also used to control access to Microsoft Planner resources and APIs. Only delegated permissions are supported for Microsoft Planner APIs; application permissions are not supported. Personal Microsoft accounts are not supported.

Group.ReadWrite.All

Category	Application	Delegated
Identifier	62a82d76-70ea-41e2-9197-370581804d09	4e46008b-f24c-477d-8fff-7bb4ec7aafe0
DisplayText	Read and write all groups	Read and write all groups
Description	Allows the app to create groups, read all group properties and memberships, update group properties and memberships, and delete groups. Also allows the app to read and write conversations. All of these operations can be performed by the app without a signed-in user.	Allows the app to create groups and read all group properties and memberships on behalf of the signed-in user. Additionally allows group owners to manage their groups and allows group members to update group content.
AdminConsentRequired	Yes	Yes

For Microsoft 365 groups, *Group.** permissions grant the app access to the contents of the group; for example, conversations, files, notes, and so on.

In some cases, an app may need extra properties to update some group properties and relationships like `member` and `memberOf`. For example, to add a servicePrincipal object as a member, the app also needs permissions to write the service principal, otherwise Microsoft Graph returns an error. For more information, see Limited information returned for inaccessible member objects.

*Group.** permissions are used to control access to Microsoft Teams resources and APIs. Personal Microsoft accounts are not supported.

*Group.** permissions are also used to control access to Microsoft Planner resources and APIs. Only delegated permissions are supported for Microsoft Planner APIs; application permissions are not supported. Personal Microsoft accounts are not supported.

GroupMember.Read.All

Category	Application	Delegated
Identifier	98830695-27a2-44f7-8c18-0c3ebc9698f6	bc024368-1153-4739-b217-4326f2e966d0
DisplayText	Read all group memberships	Read group memberships
Description	Allows the app to read memberships and basic group properties for all groups without a signed-in user.	Allows the app to list groups, read basic group properties and read membership of all groups the signed-in user has access to.
AdminConsentRequired	Yes	Yes

GroupMember.ReadWrite.All

Category	Application	Delegated
Identifier	dbaae8cf-10b5-4b86-a4a1-f871c94c6695	f81125ac-d3b7-4573-a3b2-7099cc39df9e
DisplayText	Read and write all group memberships	Read and write group memberships
Description	Allows the app to list groups, read basic properties, read and update the	Allows the app to list groups, read basic properties, read and update the

Category	Application	Delegated
	membership of the groups this app has access to without a signed-in user. Group properties and owners cannot be updated and groups cannot be deleted.	membership of the groups the signed-in user has access to. Group properties and owners cannot be updated and groups cannot be deleted.
AdminConsentRequired	Yes	Yes

GroupSettings.Read.All

Category	Application	Delegated
Identifier	f3c4f514-c65a-43f5-bfce-1735872258dd	2eb2bc92-94ef-4c6b-b4ab-2a09bc975e0e
DisplayText	Read all group settings	Read all group settings that user can access
Description	Allows the app to read a list of tenant-level or group-specific group settings objects, without a signed-in user.	Allows the app to read a list of tenant-level or group-specific group settings objects, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

GroupSettings.ReadWrite.All

Category	Application	Delegated
Identifier	546168c3-1183-4281-9491-fafb24dea37e	c1691a6d-99e2-4cfa-b4b5-9e4d67dc0f36
DisplayText	Read and write all group settings	Read and write all group settings that user can access
Description	Allows the app to create, read, update, and delete on the list of tenant-level or group-specific group settings objects, without a signed-in user.	Allows the app to create, read, update, and delete on the list of tenant-level or group-specific group settings objects that you have access to in the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

HealthMonitoringAlert.Read.All

Category	Application	Delegated
Identifier	5183ed5d-b7f8-4e9a-915e-dafb46b9cb62	74b4ff32-4917-4536-a66d-38a4861e6220
DisplayText	Read all scenario health monitoring alert	Read all scenario health monitoring alerts
Description	Allows the app to read all scenario health monitoring alerts, without a signed-in user.	Allows the app to read all scenario health monitoring alerts
AdminConsentRequired	Yes	Yes

HealthMonitoringAlert.ReadWrite.All

Category	Application	Delegated
Identifier	ac29eb50-f2f9-4518-a117-4bef18e84c7d	b7c60f27-2195-4d5f-96a7-6b98bdfd9664
DisplayText	Read and write all scenario monitoring alerts	Read and write all scenario monitoring alerts
Description	Allows the app to read and write all scenario monitoring alerts, without a signed-in user.	Allows the app to read and write all scenario monitoring alerts, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

HealthMonitoringAlertConfig.Read.All

Category	Application	Delegated
Identifier	bb424d73-e898-4c97-9d42-688c32810003	fb873030-8626-47e6-96ff-8a5bff3b725f
DisplayText	Read all scenario health monitoring alert configurations	Read all scenario health monitoring alert configurations
Description	Allows the app to read all scenario health monitoring alert configurations, without a signed-in user.	Allows the app to read all scenario health monitoring alert configurations
AdminConsentRequired	Yes	Yes

HealthMonitoringAlertConfig.ReadWrite.All

Category	Application	Delegated
Identifier	432e76f0-8af6-4315-a853-66ab9538f480	b3e5ebc6-1c23-4337-8286-3f27165addb4
DisplayText	Read and write all scenario monitoring alerts	Read and write all scenario monitoring alert configurations.
Description	Allows the app to read and write all scenario monitoring alerts, without a signed-in user.	Allows the app to read and write all scenario monitoring alert configurations, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityDiagnostic.Read

Category	Application	Delegated
Identifier	-	3839e465-e636-4c8e-b959-340182fb0567
DisplayText	-	Read your identity diagnostics
Description	-	Allows the app to read own identity diagnostics information, including symptoms, runs, statuses, and results for the signed-in user
AdminConsentRequired	-	No

IdentityDiagnostic.Read.All

Category	Application	Delegated
Identifier	bb1e8ab4-fe40-4b26-82ec-65dfaf4d367b	9181fb3f-4b8e-45ef-98ae-41774b813b80
DisplayText	Read all identity diagnostics	Read all identity diagnostics
Description	Allows the app to read all identity diagnostics information, including symptoms, runs, statuses, and results for all users in the organization, without a signed-in user.	Allows the app to read all identity diagnostics information, including symptoms, runs, statuses, and results for all users in the organization, on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

IdentityDiagnostic.StartDiagnosis

Category	Application	Delegated
Identifier	-	1ad58246-a11b-4ea3-9b75-8b6c315cbe21
DisplayText	-	Start identity diagnostics
Description	-	Allows the app to start identity diagnostic processes for the signed-in user.
AdminConsentRequired	-	No

IdentityDiagnostic.StartDiagnosis.All

Category	Application	Delegated
Identifier	2607bb8b-0a9b-4f53-9d2f-6f81b99ac145	51470ff5-62b6-4aef-9a3a-d8c8a1e66d09
DisplayText	Start identity diagnostics for all users	Start identity diagnostics for all users
Description	Allows the app to start identity diagnostic processes for all users in the organization, without a signed-in user.	Allows the app to start identity diagnostic processes for all users in the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityNotifications.Read.All

Category	Application	Delegated
Identifier	52ced3dd-dbb6-41a0-9ce5-61a056be97b8	59cd3e28-aa9c-4f72-a734-1b592eb06853
DisplayText	Read all identity notification settings and templates	Read identity notification settings and templates

Category	Application	Delegated
Description	Allows the app to read identity notification settings, email templates, and prerequisites without a signed-in user.	Allows the app to read identity notification settings, email templates, and prerequisites on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityNotifications.ReadWrite.All

Category	Application	Delegated
Identifier	d9fe7b9f-cb27-4289-9cb4-54debd9d3c25	c9c9fdea-4ecc-4d82-a2ea-3feff3489275
DisplayText	Read and write all identity notification settings and templates	Read and write identity notification settings and templates
Description	Allows the app to read and write identity notification settings, customize email templates, and send test emails without a signed-in user.	Allows the app to read and write identity notification settings, customize email templates, and send test emails on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityProvider.Read.All

Category	Application	Delegated
Identifier	e321f0bb-e7f7-481e-bb28-e3b0b32d4bd0	43781733-b5a7-4d1b-98f4-e8edff23e1a9
DisplayText	Read identity providers	Read identity providers
Description	Allows the app to read your organization's identity (authentication) providers' properties without a signed in user.	Allows the app to read your organization's identity (authentication) providers' properties on behalf of the user.
AdminConsentRequired	Yes	Yes

IdentityProvider.ReadWrite.All

Category	Application	Delegated
Identifier	90db2b9a-d928-4d33-a4dd-8442ae3d41e4	f13ce604-1677-429f-90bd-8a10b9f01325
DisplayText	Read and write identity providers	Read and write identity providers
Description	Allows the app to read and write your organization's identity (authentication) providers' properties without a signed in user.	Allows the app to read and write your organization's identity (authentication) providers' properties on behalf of the user.
AdminConsentRequired	Yes	Yes

IdentityRiskEvent.Read.All

Category	Application	Delegated
Identifier	6e472fd1-ad78-48da-a0f0-97ab2c6b769e	8f6a01e7-0391-4ee5-aa22-a3af122cef27
DisplayText	Read all identity risk event information	Read identity risk event information
Description	Allows the app to read the identity risk event information for your organization without a signed in user.	Allows the app to read identity risk event information for all users in your organization on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityRiskEvent.ReadWrite.All

Category	Application	Delegated
Identifier	db06fb33-1953-4b7b-a2ac-f1e2c854f7ae	9e4862a5-b68f-479e-848a-4e07e25c9916
DisplayText	Read and write all risk detection information	Read and write risk event information
Description	Allows the app to read and update identity risk detection information for your organization without a signed-in user. Update operations include confirming risk event detections.	Allows the app to read and update identity risk event information for all users in your organization on behalf of the signed-in user. Update operations include confirming risk event detections.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

IdentityRiskyAgent.Read.All

Category	Application	Delegated
Identifier	4aadfb66-d49a-414a-a883-d8c240b6fa33	3215c57f-3faa-4295-95c2-6f14a5bc6124
DisplayText	Read all risky agents information	Read risky agents information
Description	Allows the app to read the risky agents information in your organization without a signed-in user.	Allows the app to read risky agents information in your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityRiskyAgent.ReadWrite.All

Category	Application	Delegated
Identifier	dca4e4fd-a7cf-4e6f-86d1-d1ec094d766e	d343bdeb-db6a-4e06-97da-9dafc2d61c60
DisplayText	Read and write risky agents information	Read and write risky agents information
Description	Allows the app to read and update risky agents information in your organization without a signed-in user.	Allows the app to read and update identity risky agents information for all agents in your organization on behalf of the signed-in user. Update operations include dismissing risky agents.
AdminConsentRequired	Yes	Yes

IdentityRiskyServicePrincipal.Read.All

Category	Application	Delegated
Identifier	607c7344-0eed-41e5-823a-9695ebe1b7b0	ea5c4ab0-5a73-4f35-8272-5d5337884e5d

Category	Application	Delegated
DisplayText	Read all identity risky service principal information	Read all identity risky service principal information
Description	Allows the app to read all risky service principal information for your organization, without a signed-in user.	Allows the app to read all identity risky service principal information for your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityRiskyServicePrincipal.ReadWrite.All

Category	Application	Delegated
Identifier	cb8d6980-6bcb-4507-afec-ed6de3a2d798	bb6f654c-d7fd-4ae3-85c3-fc380934f515
DisplayText	Read and write all identity risky service principal information	Read and write all identity risky service principal information
Description	Allows the app to read and update identity risky service principal for your organization, without a signed-in user.	Allows the app to read and update identity risky service principal information for all service principals in your organization, on behalf of the signed-in user. Update operations include dismissing risky service principals.
AdminConsentRequired	Yes	Yes

IdentityRiskyUser.Read.All

Category	Application	Delegated
Identifier	dc5007c0-2d7d-4c42-879c-2dab87571379	d04bb851-cb7c-4146-97c7-ca3e71baf56c
DisplayText	Read all identity risky user information	Read identity risky user information
Description	Allows the app to read the identity risky user information for your organization without a signed in user.	Allows the app to read identity risky user information for all users in your organization on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityRiskyUser.ReadWrite.All

Category	Application	Delegated
Identifier	656f6061-f9fe-4807-9708-6a2e0934df76	e0a7cdbb-08b0-4697-8264-0069786e9674
DisplayText	Read and write all risky user information	Read and write risky user information
Description	Allows the app to read and update identity risky user information for your organization without a signed-in user. Update operations include dismissing risky users.	Allows the app to read and update identity risky user information for all users in your organization on behalf of the signed-in user. Update operations include dismissing risky users.
AdminConsentRequired	Yes	Yes

IdentityUserFlow.Read.All

Category	Application	Delegated
Identifier	1b0c317f-dd31-4305-9932-259a8b6e8099	2903d63d-4611-4d43-99ce-a33f3f52e343
DisplayText	Read all identity user flows	Read all identity user flows
Description	Allows the app to read your organization's user flows, without a signed-in user.	Allows the app to read your organization's user flows, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IdentityUserFlow.ReadWrite.All

Category	Application	Delegated
Identifier	65319a09-a2be-469d-8782-f6b07debf789	281892cc-4dbf-4e3a-b6cc-b21029bb4e82
DisplayText	Read and write all identity user flows	Read and write all identity user flows
Description	Allows the app to read or write your organization's user flows, without a	Allows the app to read or write your organization's user flows, on behalf of

Category	Application	Delegated
	signed-in user.	the signed-in user.
AdminConsentRequired	Yes	Yes

IMAP.AccessAsUser.All

Category	Application	Delegated
Identifier	-	652390e4-393a-48de-9484-05f9b1212954
DisplayText	-	Read and write access to mailboxes via IMAP.
Description	-	Allows the app to have the same access to mailboxes as the signed-in user via IMAP protocol.
AdminConsentRequired	-	No

The *IMAP.AccessAsUser.All* delegated permission is available for consent in personal Microsoft accounts.

IndustryData-DataConnector.Read.All

Category	Application	Delegated
Identifier	7ab52c2f-a2ee-4d98-9ebc-725e3934aae2	d19c0de5-7ecb-4aba-b090-da35ebcd5425
DisplayText	View data connector definitions	View data connector definitions
Description	Allows the app to read data connectors without a signed-in user.	Allows the app to read data connectors on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-DataConnector.ReadWrite.All

Category	Application	Delegated
Identifier	eda0971c-482e-4345-b28f-69c309cb8a34	5ce933ac-3997-4280-aed0-cc072e5c062a

Category	Application	Delegated
DisplayText	Manage data connector definitions	Manage data connector definitions
Description	Allows the app to read and write data connectors without a signed-in user.	Allows the app to read and write data connectors on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-DataConnector.Upload

Category	Application	Delegated
Identifier	9334c44b-a7c6-4350-8036-6bf8e02b4c1f	fc47391d-ab2c-410f-9059-5600f7af660d
DisplayText	Upload files to a data connector	Upload files to a data connector
Description	Allows the app to upload data files to a data connector without a signed-in user.	Allows the app to upload data files to a data connector on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-InboundFlow.Read.All

Category	Application	Delegated
Identifier	305f6ba2-049a-4b1b-88bb-fe7e08758a00	cb0774da-a605-42af-959c-32f438fb38f4
DisplayText	View inbound flow definitions	View inbound flow definitions
Description	Allows the app to read inbound data flows without a signed-in user.	Allows the app to read inbound data flows on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-InboundFlow.ReadWrite.All

Category	Application	Delegated
Identifier	e688c61f-d4c6-4d64-a197-3bcf6ba1d6ad	97044676-2cec-40ee-bd70-38df444c9e70
DisplayText	Manage inbound flow definitions	Manage inbound flow definitions
Description	Allows the app to read and write inbound data flows without a signed-in user.	Allows the app to read and write inbound data flows on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-OutboundFlow.Read.All

Category	Application	Delegated
Identifier	61d0354c-5d88-483c-b974-a37ec3395a2c	4741a003-8952-4be4-9217-33a0ac327122
DisplayText	View outbound flow definitions	View outbound flow definitions
Description	Allows the app to read outbound data flows without a signed-in user.	Allows the app to read outbound data flows on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-OutboundFlow.ReadWrite.All

Category	Application	Delegated
Identifier	24a65b4a-e501-47e2-8849-d679517887f0	aeb68e0b-e562-4a1f-b6dd-3484ad0cbb4b
DisplayText	Manage outbound flow definitions	Manage outbound flow definitions
Description	Allows the app to read and write outbound data flows without a signed-in user.	Allows the app to read and write outbound data flows on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-ReferenceDefinition.Read.All

Category	Application	Delegated
Identifier	6ee891c3-74a4-4148-8463-0c834375dfaf	a3f96ffe-cb84-40a8-ac85-582d7ef97c2a
DisplayText	View reference definitions	View reference definitions
Description	Allows the app to read reference definitions without a signed-in user.	Allows the app to read reference definitions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-ReferenceDefinition.ReadWrite.All

Category	Application	Delegated
Identifier	bda16293-63d3-45b7-b16b-833841d27d56	a757d430-be6d-430f-af57-28aabe79d247
DisplayText	Manage reference definitions	Manage reference definitions
Description	Allows the app to read and write reference definitions without a signed-in user.	Allows the app to read and write reference definitions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-Run.Read.All

Category	Application	Delegated
Identifier	f6f5d10b-3024-4d1d-b674-aae4df4a1a73	92685235-50c4-4702-b2c8-36043db6fa79
DisplayText	View current and previous runs	View current and previous runs
Description	Allows the app to read current and previous IndustryData runs without a signed-in user.	Allows the app to read current and previous IndustryData runs on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-Run.Start

Category	Application	Delegated
Identifier	7e429772-5b5e-47c0-8fd6-7279294c8033	f03a6d0e-0989-460f-80b2-e57c8561763e
DisplayText	View and start runs	View and start runs
Description	Allows the app to view and start IndustryData runs without a signed-in user.	Allows the app to view and start IndustryData runs on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-SourceSystem.Read.All

Category	Application	Delegated
Identifier	bc167a60-39fe-4865-8b44-78400fc6ed03	49b7016c-89ae-41e7-bd6f-b7170c5490bf
DisplayText	View source system definitions	View source system definitions
Description	Allows the app to read source system definitions without a signed-in user.	Allows the app to read source system definitions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-SourceSystem.ReadWrite.All

Category	Application	Delegated
Identifier	7d866958-e06e-4dd6-91c6-a086b3f5cfeb	9599f005-05d6-4ea7-b1b1-4929768af5d0
DisplayText	Manage source system definitions	Manage source system definitions
Description	Allows the app to read and write source system definitions without a signed-in user.	Allows the app to read and write source system definitions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-TimePeriod.Read.All

Category	Application	Delegated
Identifier	7c55c952-b095-4c23-a522-022bce4cc1e3	c9d51f28-8ccd-42b2-a836-fd8fe9ebf2ae
DisplayText	Read time period definitions	Read time period definitions
Description	Allows the app to read time period definitions without a signed-in user.	Allows the app to read time period definitions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData-TimePeriod.ReadWrite.All

Category	Application	Delegated
Identifier	7afa7744-a782-4a32-b8c2-e3db637e8de7	b6d56528-3032-4f9d-830f-5a24a25e6661
DisplayText	Manage time period definitions	Manage time period definitions
Description	Allows the app to read and write time period definitions without a signed-in user.	Allows the app to read and write time period definitions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

IndustryData.ReadBasic.All

Category	Application	Delegated
Identifier	4f5ac95f-62fd-472c-b60f-125d24ca0bc5	60382b96-1f5e-46ea-a544-0407e489e588
DisplayText	View basic service and resource information	Read basic Industry Data service and resource definitions
Description	Allows the app to read basic service and resource information without a signed-in user.	Allows the app to read basic Industry Data service and resource information on behalf of the signed-in user.
AdminConsentRequired	Yes	No

InformationProtectionConfig.Read

Category	Application	Delegated
Identifier	-	12f4bffb-b598-413c-984b-db99728f8b54
DisplayText	-	Read configurations for protecting organizational data applicable to the user
Description	-	Allows the app to read the configurations applicable to the signed-in user for protecting organizational data, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

InformationProtectionConfig.Read.All

Category	Application	Delegated
Identifier	14f49b9f-4bf2-4d24-b80e-b27ec58409bd	-
DisplayText	Read all configurations for protecting organizational data applicable to users	-
Description	Allows the app to read all configurations applicable to users for protecting organizational data, without a signed-in user.	-
AdminConsentRequired	Yes	-

InformationProtectionContent.Sign.All

Category	Application	Delegated
Identifier	cbe6c7e4-09aa-4b8d-b3c3-2dbb59af4b54	-
DisplayText	Sign digests for data	-
Description	Allows an app to sign digests for data without a signed-in user.	-
AdminConsentRequired	Yes	-

InformationProtectionContent.Write.All

Category	Application	Delegated
Identifier	287bd98c-e865-4e8c-bade-1a85523195b9	-
DisplayText	Create protected content	-
Description	Allows the app to create protected content without a signed-in user.	-
AdminConsentRequired	Yes	-

InformationProtectionPolicy.Read

Category	Application	Delegated
Identifier	-	4ad84827-5578-4e18-ad7a-86530b12f884
DisplayText	-	Read user sensitivity labels and label policies.
Description	-	Allows an app to read information protection sensitivity labels and label policy settings, on behalf of the signed-in user.
AdminConsentRequired	-	No

InformationProtectionPolicy.Read.All

Category	Application	Delegated
Identifier	19da66cb-0fb0-4390-b071-ebc76a349482	-
DisplayText	Read all published labels and label policies for an organization.	-
Description	Allows an app to read published sensitivity labels and label policy settings for the entire organization or a specific user, without a signed in user.	-
AdminConsentRequired	Yes	-

Insights-UserMetric.Read.All

Category	Application	Delegated
Identifier	34cbd96c-d824-4755-90d3-1008ef47efc1	7d249730-51a3-4180-8ec1-214f144f1bff

Category	Application	Delegated
DisplayText	Read all user metrics insights	Read user metrics insights
Description	Allows an app to read all user metrics insights, such as daily and monthly active users, without a signed-in user.	Allows an app to read user metrics insights, such as daily and monthly active users, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

LearningAssignedCourse.Read

Category	Application	Delegated
Identifier	-	ac08cdae-e845-41db-adf9-5899a0ec9ef6
DisplayText	-	Read user's assignments
Description	-	Allows the app to read data for the learner's assignments in the organization's directory, on behalf of the signed-in user.
AdminConsentRequired	-	No

LearningAssignedCourse.Read.All

Category	Application	Delegated
Identifier	535e6066-2894-49ef-ab33-e2c6d064bb81	-
DisplayText	Read all assignments	-
Description	Allows the app to read data for all assignments in the organization's directory, without a signed-in user.	-
AdminConsentRequired	Yes	-

LearningAssignedCourse.ReadWrite.All

Category	Application	Delegated
Identifier	236c1cbd-1187-427f-b0f5-b1852454973b	-

Category	Application	Delegated
DisplayText	Read and write all assignments	-
Description	Allows the app to create, update, read and delete all assignments in the organization's directory, without a signed-in user.	-
AdminConsentRequired	Yes	-

LearningContent.Read.All

Category	Application	Delegated
Identifier	8740813e-d8aa-4204-860e-2a0f8f84dbc8	ea4c1fd9-6a9f-4432-8e5d-86e06cc0da77
DisplayText	Read all learning content	Read learning content
Description	Allows the app to read all learning content in the organization's directory, without a signed-in user.	Allows the app to read learning content in the organization's directory, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

LearningContent.ReadWrite.All

Category	Application	Delegated
Identifier	444d6fcb-b738-41e5-b103-ac4f2a2628a3	53cec1c4-a65f-4981-9dc1-ad75dbf1c077
DisplayText	Manage all learning content	Manage learning content
Description	Allows the app to manage all learning content in the organization's directory, without a signed-in user.	Allows the app to manage learning content in the organization's directory, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

LearningProvider.Read

Category	Application	Delegated
Identifier	-	dd8ce36f-9245-45ea-a99e-8ac398c22861
DisplayText	-	Read learning provider
Description	-	Allows the app to read data for the learning provider in the organization's directory, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

LearningProvider.ReadWrite

Category	Application	Delegated
Identifier	-	40c2eb57-abaf-49f5-9331-e90fd01f7130
DisplayText	-	Manage learning provider
Description	-	Allows the app to create, update, read, and delete data for the learning provider in the organization's directory, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

LearningSelfInitiatedCourse.Read

Category	Application	Delegated
Identifier	-	f6403ef7-4a96-47be-a190-69ba274c3f11
DisplayText	-	Read user's self-initiated courses
Description	-	Allows the app to read data for the learner's self-initiated courses in the organization's directory, on behalf of the signed-in user.
AdminConsentRequired	-	No

LearningSelfInitiatedCourse.Read.All

Category	Application	Delegated
Identifier	467524fc-ed22-4356-a910-af61191e3503	-

Category	Application	Delegated
DisplayText	Read all self-initiated courses	-
Description	Allows the app to read data for all self-initiated courses in the organization's directory, without a signed-in user.	-
AdminConsentRequired	Yes	-

LearningSelfInitiatedCourse.ReadWrite.All

Category	Application	Delegated
Identifier	7654ed61-8965-4025-846a-0856ec02b5b0	-
DisplayText	Read and write all self-initiated courses	-
Description	Allows the app to create, update, read and delete all self-initiated courses in the organization's directory, without a signed-in user.	-
AdminConsentRequired	Yes	-

LicenseAssignment.Read.All

Category	Application	Delegated
Identifier	e2f98668-2877-4f38-a2f4-8202e0717aa1	f395577a-0960-456b-979f-7228de0c5996
DisplayText	Read all license assignments.	Read all license assignments.
Description	Allows an app to read license assignments for users and groups, without a signed-in user.	Allows an app to read license assignments for users and groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	No

LicenseAssignment.ReadWrite.All

Category	Application	Delegated
Identifier	5facf0c1-8979-4e95-abcf-ff3d079771c0	f55016cc-149c-447e-8f21-7cf3ec1d6350

Category	Application	Delegated
DisplayText	Manage all license assignments	Manage all license assignments
Description	Allows an app to manage license assignments for users and groups, without a signed-in user.	Allows an app to manage license assignments for users and groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

LifecyclePolicies-AgentId.Read.All

Category	Application	Delegated
Identifier	6343d63f-034f-45b5-832d-9f9d7632e182	65857db0-62ac-4279-aa73-c2b5dab186f5
DisplayText	Read identity lifecycle policies for agent identities	Read identity lifecycle policies for agent identities
Description	Allows the app to read identity lifecycle policies for agent identities in the organization, without a signed-in user.	Allows the app to read identity lifecycle policies for agent identities that the signed-in user has access to in the organization.
AdminConsentRequired	Yes	Yes

LifecyclePolicies-AgentId.ReadWrite.All

Category	Application	Delegated
Identifier	00d1c504-8dc7-461b-8a0b-dc15c8f1bd5a	f2292ca5-46fc-4195-9b4d-16491bf9bf7f
DisplayText	Read and write identity lifecycle policies for agent identities	Read and write identity lifecycle policies for agent identities
Description	Allows the app to read, create, update and delete identity lifecycle policies for agent identities in the organization, without a signed-in user.	Allows the app to read, create, update and delete identity lifecycle policies for agent identities that the signed-in user has access to in the organization.
AdminConsentRequired	Yes	Yes

LifecyclePolicies-Guests.Read.All

Category	Application	Delegated
Identifier	-	1bbb7916-b98a-449f-8ee4-c68bfcba5724
DisplayText	-	Read identity lifecycle policies for external guests
Description	-	Allows the app to read identity lifecycle policies for external guests on behalf of the signed-in user.
AdminConsentRequired	-	Yes

LifecyclePolicies-Guests.ReadWrite.All

Category	Application	Delegated
Identifier	-	d9ec82ed-63db-4905-b1b3-859b74d2bbf5
DisplayText	-	Read and write identity lifecycle policies for external guests
Description	-	Allows the app to create, update, and delete identity lifecycle policies for external guests on behalf of the signed-in user.
AdminConsentRequired	-	Yes

LifecycleWorkflows-CustomExt.Read.All

Category	Application	Delegated
Identifier	2cb19e7d-9012-40bf-9a22-69fc776af8b0	2973a298-1d69-4f87-8d30-7025f0ec19d7
DisplayText	Read all Lifecycle workflows custom task extensionss	Read all Lifecycle workflows custom task extensions
Description	Allows the app to read all Lifecycle workflows custom task extensions without a signed-in user.	Allows the app to read all Lifecycle workflows custom task extensions on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

LifecycleWorkflows-CustomExt.ReadWrite.All

Category	Application	Delegated
Identifier	3351c766-bacc-4d93-94fa-f2c8b1986ee7	ef6bafb1-3019-4a22-a332-103aff92225f
DisplayText	Read and write all Lifecycle workflows custom task extensions	Read and write all Lifecycle workflows custom task extensions
Description	Allows the app to create, update, list, read and delete all Lifecycle workflows custom task extensions without a signed-in user.	Allows the app to create, update, list, read and delete all Lifecycle workflows custom task extensions on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

LifecycleWorkflows-Reports.Read.All

Category	Application	Delegated
Identifier	fe615156-48b5-4c83-b613-e6e31a43c446	4d3d7f81-163f-426a-8432-5638d2e82083
DisplayText	Read all Lifecycle workflows reports	Read all Lifecycle workflows reports
Description	Allows the app to read all Lifecycle workflows reports without a signed-in user.	Allows the app to read all Lifecycle workflows reports on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

LifecycleWorkflows-Workflow.Activate

Category	Application	Delegated
Identifier	3a87a643-13d2-47aa-8d6a-b0a8377cb03b	df1c25b3-072c-45cd-8403-c63441e4cca1
DisplayText	Run workflows on-demand in Lifecycle workflows	Run workflows on-demand in Lifecycle workflows
Description	Allows the app run workflows on-demand without a signed-in user.	Allows the app to run workflows on-demand on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

LifecycleWorkflows-Workflow.Read.All

Category	Application	Delegated
Identifier	03b0ad3e-fc2b-4ef1-b0ff-252e865cb608	7fabe5bd-2e47-4e61-b924-327117024e18
DisplayText	Read all workflows in Lifecycle workflows	Read all workflows in Lifecycle workflows
Description	Allows the app to list and read all workflows and tasks without a signed-in user.	Allows the app to list and read all workflows and tasks on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

LifecycleWorkflows-Workflow.ReadBasic.All

Category	Application	Delegated
Identifier	021ea6db-c06b-45c6-8c9c-c1cd9a37a483	789c445d-433c-4575-a1fc-367a58a1bd4a
DisplayText	List all workflows in Lifecycle workflows	List all workflows in Lifecycle workflows
Description	Allows the app to list all workflows without a signed-in user.	Allows the app to list all workflows on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

LifecycleWorkflows-Workflow.ReadWrite.All

Category	Application	Delegated
Identifier	94c88098-1d9d-4c42-a356-4d5a95312554	29e49f0c-a053-4cc5-a4b1-7da0c8c1e643
DisplayText	Read and write all workflows in Lifecycle workflows	Read and write all workflows in Lifecycle workflows
Description	Allows the app to create, update, list, read and delete all workflows and tasks in lifecycle workflows without a signed-in user.	Allows the app to create, update, list, read and delete all workflows and tasks in lifecycle workflows on behalf of a signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

LifecycleWorkflows.Read.All

Category	Application	Delegated
Identifier	7c67316a-232a-4b84-be22-cea2c0906404	9bcb9916-765a-42af-bf77-02282e26b01a
DisplayText	Read all lifecycle workflows resources	Read all lifecycle workflows resources
Description	Allows the app to list and read all workflows, tasks and related lifecycle workflows resources without a signed-in user.	Allows the app to list and read all workflows, tasks and related lifecycle workflows resources on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

LifecycleWorkflows.ReadWrite.All

Category	Application	Delegated
Identifier	5c505cf4-8424-4b8e-aa14-ee06e3bb23e3	84b9d731-7db8-4454-8c90-fd9e95350179
DisplayText	Read and write all lifecycle workflows resources	Read and write all lifecycle workflows resources
Description	Allows the app to create, update, list, read and delete all workflows, tasks and related lifecycle workflows resources without a signed-in user.	Allows the app to create, update, list, read and delete all workflows, tasks and related lifecycle workflows resources on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ListItems.SelectedOperations.Selected

Category	Application	Delegated
Identifier	de4e4161-a10a-4dfd-809c-e328d89aefeb	d6d361b3-211a-4191-9fa7-15f72de4aac4

Category	Application	Delegated
DisplayText	Access selected ListItems without a signed in user.	Access selected ListItems, on behalf of the signed-in user
Description	Allow the application to access a subset of listitems without a signed in user. The specific listitems and the permissions granted will be configured in SharePoint Online.	Allow the application to access a subset of listitems on behalf of the signed in user. The specific listitems and the permissions granted will be configured in SharePoint Online.
AdminConsentRequired	Yes	Yes

Lists.SelectedOperations.Selected

Category	Application	Delegated
Identifier	23c5a9bd-d900-4ecf-be26-a0689755d9e5	033b51ee-d6fa-4add-b627-ee680c7212b5
DisplayText	Access selected Lists without a signed in user.	Access selected Lists, on behalf of the signed-in user
Description	Allow the application to access a subset of lists without a signed in user. The specific lists and the permissions granted will be configured in SharePoint Online.	Allow the application to access a subset of lists on behalf of the signed in user. The specific lists and the permissions granted will be configured in SharePoint Online.
AdminConsentRequired	Yes	Yes

LockboxRequest.Read.All

Category	Application	Delegated
Identifier	-	a368a3ce-713b-46d9-9c93-bb559acf38ca
DisplayText	-	Read lockbox requests
Description	-	Allows the app to read lockbox requests on behalf of the signed-in user
AdminConsentRequired	-	Yes

LockboxRequest.ReadWrite.All

Category	Application	Delegated
Identifier	-	524d6259-f823-43c4-964a-97e3b8ddb56c
DisplayText	-	Read and manage lockbox requests
Description	-	Allows the app to read and manage lockbox requests on behalf of the signed-in user
AdminConsentRequired	-	Yes

LockboxSettings.Read.All

Category	Application	Delegated
Identifier	-	2ea429a7-d1fa-4f96-b451-e91ecc5e5fe2
DisplayText	-	Read lockbox settings
Description	-	Allows the app to read lockbox configuration settings on behalf of the signed-in user
AdminConsentRequired	-	Yes

LockboxSettings.ReadWrite.All

Category	Application	Delegated
Identifier	-	bdc52289-9ed7-4339-83ab-772ae618713c
DisplayText	-	Read and write lockbox settings
Description	-	Allows the app to read and modify lockbox configuration settings on behalf of the signed-in user
AdminConsentRequired	-	Yes

Mail-Advanced.ReadWrite

Category	Application	Delegated
Identifier	-	f3af82f6-18e0-4a41-8dc8-a03c11854a8d
DisplayText	-	Read and write the user's mail, including modifying existing non-draft mails
Description	-	Allows the app to create, read, update, and delete email, including contents of non-draft emails in user mailboxes, on behalf of the signed-in user. Does not include permission to send mail.
AdminConsentRequired	-	Yes

Mail-Advanced.ReadWrite.All

Category	Application	Delegated
Identifier	e118f1da-5c1c-46cf-bff6-8858d786f46f	-
DisplayText	Read and write mail in all mailboxes, including modifying existing non-draft mails	-
Description	Allows the app to create, read, update, and delete all email, including contents of non-draft emails in user mailboxes, without a signed-in user. Does not include permission to send mail.	-
AdminConsentRequired	Yes	-

Mail-Advanced.ReadWrite.Shared

Category	Application	Delegated
Identifier	-	bebf0bb6-2ff3-4295-a17d-f3561da294fb
DisplayText	-	Read and write all mail the user can access, including modifying existing non-draft mails
Description	-	Allows the app to create, read, update, and delete mail including contents of non-draft emails for all mails a user has permission to access, on behalf of the signed-in user. This includes their own and shared mail. Does not include permission to send mail.
AdminConsentRequired	-	Yes

Mail.Read

Category	Application	Delegated
Identifier	810c84a8-4a9e-49e6-bf7d-12d183f40d01	570282fd-fa5c-430d-a7fd-fc8dc98a9dca
DisplayText	Read mail in all mailboxes	Read user mail
Description	Allows the app to read mail in all mailboxes without a signed-in user.	Allows the app to read the signed-in user's mailbox.
AdminConsentRequired	Yes	No

The *Mail.Read* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to *specific* mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *Mail.Read* application permission.

Mail.Read is valid valid for both Microsoft accounts and work or school accounts.

Mail.Read.Shared

Category	Application	Delegated
Identifier	-	7b9103a5-4610-446b-9670-80643382c1fa
DisplayText	-	Read user and shared mail
Description	-	Allows the app to read mail a user can access, including their own and shared mail.
AdminConsentRequired	-	No

Mail.Read.Shared is only valid for work or school accounts.

Mail.ReadBasic

Category	Application	Delegated
Identifier	6be147d2-ea4f-4b5a-a3fa-3eab6f3c140a	a4b8392a-d8d1-4954-a029-8e668a39a170

Category	Application	Delegated
DisplayText	Read basic mail in all mailboxes	Read user basic mail
Description	Allows the app to read basic mail properties in all mailboxes without a signed-in user. Includes all properties except body, previewBody, attachments and any extended properties.	Allows the app to read email in the signed-in user's mailbox except body, previewBody, attachments and any extended properties.
AdminConsentRequired	Yes	No

The *Mail.ReadBasic* delegated permission is available for consent in personal Microsoft accounts.

Mail.ReadBasic.All

Category	Application	Delegated
Identifier	693c5e45-0940-467d-9b8a-1022fb9d42ef	-
DisplayText	Read basic mail in all mailboxes	-
Description	Allows the app to read basic mail properties in all mailboxes without a signed-in user. Includes all properties except body, previewBody, attachments and any extended properties.	-
AdminConsentRequired	Yes	-

Mail.ReadBasic.Shared

Category	Application	Delegated
Identifier	-	b11fa0e7-fdb7-4dc9-b1f1-59facd463480
DisplayText	-	Read user and shared basic mail
Description	-	Allows the app to read mail the signed-in user can access, including their own and shared mail, except for body, bodyPreview, uniqueBody, attachments, extensions, and any extended properties.
AdminConsentRequired	-	No

Mail.ReadWrite

Category	Application	Delegated
Identifier	e2a3a72e-5f79-4c64-b1b1-878b674786c9	024d486e-b451-40bb-833d-3e66d98c5c73
DisplayText	Read and write mail in all mailboxes	Read and write access to user mail
Description	Allows the app to create, read, update, and delete mail in all mailboxes without a signed-in user. Does not include permission to send mail.	Allows the app to create, read, update, and delete email in user mailboxes. Does not include permission to send mail.
AdminConsentRequired	Yes	No

The *Mail.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to *specific* mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *Mail.ReadWrite* application permission.

Mail.ReadWrite.Shared

Category	Application	Delegated
Identifier	-	5df07973-7d5d-46ed-9847-1271055cbd51
DisplayText	-	Read and write user and shared mail
Description	-	Allows the app to create, read, update, and delete mail a user has permission to access, including their own and shared mail. Does not include permission to send mail.
AdminConsentRequired	-	No

Mail.ReadWrite.Shared is only valid for work or school accounts.

Mail.Send

Category	Application	Delegated
Identifier	b633e1c5-b582-4048-a93e-9f11b44c7e96	e383f46e-2787-4529-855e-0e479a3ffac0
DisplayText	Send mail as any user	Send mail as a user
Description	Allows the app to send mail as any user without a signed-in user.	Allows the app to send mail as users in the organization.
AdminConsentRequired	Yes	No

The *Mail.Send* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to *specific* mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *Mail.Send* application permission.

Mail.Send is valid valid for both Microsoft accounts and work or school accounts.

With the *Mail.Send* permission, an app can send mail and save a copy to the user's Sent Items folder, even if the app isn't granted the *Mail.ReadWrite* or *Mail.ReadWrite.Shared* permission.

Mail.Send.Shared

Category	Application	Delegated
Identifier	-	a367ab51-6b49-43bf-a716-a1fb06d2a174
DisplayText	-	Send mail on behalf of others
Description	-	Allows the app to send mail as the signed-in user, including sending on-behalf of others.
AdminConsentRequired	-	No

Mail.Send.Shared is only valid for work or school accounts.

With the *Mail.Send.Shared* permission, an app can send mail and save a copy to the user's Sent Items folder, even if the app isn't granted the *Mail.ReadWrite* or *Mail.ReadWrite.Shared* permission.

MailboxConfigItem.Read

Category	Application	Delegated
Identifier	27d9d776-f4d2-426d-80ad-5f22f2b01b0a	dce2e6fc-0f4b-40da-94e2-14b4477f3d92
DisplayText	Read all users' UserConfiguration objects	Read user's UserConfiguration objects
Description	Allows the app to read all users' UserConfiguration objects.	Allows the app to read user's UserConfiguration objects, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

MailboxConfigItem.ReadWrite

Category	Application	Delegated
Identifier	aa6d92d4-b25a-4640-aefe-3e3231e5e736	7d461784-7715-4b09-9f90-91a6d8722652
DisplayText	Read and write all users' UserConfiguration objects	Read and write user's UserConfiguration objects
Description	Allows the app to create, read, update and delete all users' UserConfiguration objects.	Allows the app to create, read, update and delete user's UserConfiguration objects, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

MailboxFolder.Read

Category	Application	Delegated
Identifier	-	52dc2051-4958-4636-8f2a-281d39c6981c
DisplayText	-	Read a user's mailbox folders
Description	-	Allows the app to read the user's mailbox folders, on behalf of the signed-in user.
AdminConsentRequired	-	No

MailboxFolder.Read.All

Category	Application	Delegated
Identifier	99280d24-a782-4793-93cc-0888549957f6	-
DisplayText	Read all the users' mailbox folders	-
Description	Allows the app to read all the users' mailbox folders, without signed-in user.	-
AdminConsentRequired	Yes	-

MailboxFolder.ReadWrite

Category	Application	Delegated
Identifier	-	077fde41-7e0b-4c5b-bcd1-e9d743a30c80
DisplayText	-	Read and write a user's mailbox folders
Description	-	Allows the app to read and write the user's mailbox folders, on behalf of the signed-in user.
AdminConsentRequired	-	No

MailboxFolder.ReadWrite.All

Category	Application	Delegated
Identifier	fef87b92-8391-4589-9da7-eb93dab7dc8a	-
DisplayText	Read and write all the users' mailbox folders	-
Description	Allows the app to read and write all the users' mailbox folders, without signed-in user.	-
AdminConsentRequired	Yes	-

MailboxItem.Export

Category	Application	Delegated
Identifier	-	58d3e7fa-3ce9-4a0c-9baa-0971f64709d9

Category	Application	Delegated
DisplayText	-	Export a user's mailbox items
Description	-	Allows the app to export the user's mailbox items, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

MailboxItem.Export.All

Category	Application	Delegated
Identifier	937550e9-33a3-494b-88ae-d9cd394b1fbb	-
DisplayText	Export all the users' mailbox items	-
Description	Allows the app to export all the users' mailbox items, without signed-in user.	-
AdminConsentRequired	Yes	-

MailboxItem.ImportExport

Category	Application	Delegated
Identifier	-	df96e8a0-f4e1-4ecf-8d83-a429f822cbd6
DisplayText	-	Export and import a user's mailbox items
Description	-	Allows the app to export and import the user's mailbox items, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

MailboxItem.ImportExport.All

Category	Application	Delegated
Identifier	76577085-e73d-4f1d-b26a-85fb33892327	-
DisplayText	Export and import all the users' mailbox items	-

Category	Application	Delegated
Description	Allows the app to export and import all the users' mailbox items, without signed-in user.	-
AdminConsentRequired	Yes	-

MailboxItem.Read

Category	Application	Delegated
Identifier	-	82305458-296d-4edd-8b0b-74dd74c34526
DisplayText	-	Read a user's mailbox items
Description	-	Allows the app to read the user's mailbox items, on behalf of the signed-in user.
AdminConsentRequired	-	No

MailboxItem.Read.All

Category	Application	Delegated
Identifier	7d9f353d-a7bd-4fbb-822a-26d5dd39a3ce	-
DisplayText	Read all the users' mailbox items	-
Description	Allows the app to read all the users' mailbox items, without signed-in user.	-
AdminConsentRequired	Yes	-

MailboxItem.ReadWrite

Category	Application	Delegated
Identifier	-	ec1ade38-5268-4bc8-87fa-b230e42f7a88
DisplayText	-	Read and write your mailbox items
Description	-	Allows the app to read, write, and delete the user's mailbox items, on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	-	Yes

MailboxItem.ReadWrite.All

Category	Application	Delegated
Identifier	1583d471-fede-4e7c-b062-57e9d60cfb49	-
DisplayText	Read and write all users' mailbox items	-
Description	Allows the app to read, write, and delete all users' mailbox items, without a signed-in user.	-
AdminConsentRequired	Yes	-

MailboxSettings.Read

Category	Application	Delegated
Identifier	40f97065-369a-49f4-947c-6a255697ae91	87f447af-9fa4-4c32-9dfa-4a57a73d18ce
DisplayText	Read all user mailbox settings	Read user mailbox settings
Description	Allows the app to read user's mailbox settings without a signed-in user. Does not include permission to send mail.	Allows the app to the read user's mailbox settings. Does not include permission to send mail.
AdminConsentRequired	Yes	No

The *MailboxSettings.Read* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to *specific* mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *MailboxSettings.Read* application permission.

MailboxSettings.Read is valid valid for both Microsoft accounts and work or school accounts.

MailboxSettings.ReadWrite

Category	Application	Delegated
Identifier	6931bccd-447a-43d1-b442-00a195474933	818c620a-27a9-40bd-a6a5-d96f7d610b4b
DisplayText	Read and write all user mailbox settings	Read and write user mailbox settings
Description	Allows the app to create, read, update, and delete user's mailbox settings without a signed-in user. Does not include permission to send mail.	Allows the app to create, read, update, and delete user's mailbox settings. Does not include permission to send mail.
AdminConsentRequired	Yes	No

The *MailboxSettings.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

Administrators can configure application access policy to limit app access to *specific* mailboxes and not to all the mailboxes in the organization, even if the app has been granted the *MailboxSettings.ReadWrite* application permission.

MailboxSettings.ReadWrite is valid valid for both Microsoft accounts and work or school accounts.

MailTips.ReadBasic.All

Category	Application	Delegated
Identifier	a2c9652d-4d7f-4e4e-9d75-ac32fdc6f413	-
DisplayText	Read mail tips for all users	-
Description	Allows the app to read mail tips for all users in the organization without a signed-in user. Mail tips include automatic replies, mailbox status, custom tips, and delivery information.	-
AdminConsentRequired	Yes	-

MailTips.ReadBasic.Shared

Category	Application	Delegated
Identifier	-	4776cae1-54bd-4dfd-823c-e5861ed49a98
DisplayText	-	Read mail tips for mailboxes you can access

Category	Application	Delegated
Description	-	Allows the app to read mail tips on behalf of the signed-in user for mailboxes they have access to. Mail tips include automatic replies, mailbox status, custom tips, and delivery information.
AdminConsentRequired	-	No

ManagedTenants.Read.All

Category	Application	Delegated
Identifier	-	dc34164e-6c4a-41a0-be89-3ae2fbad7cd3
DisplayText	-	Read all managed tenant information
Description	-	Allows the app to read all managed tenant information on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ManagedTenants.ReadWrite.All

Category	Application	Delegated
Identifier	-	b31fa710-c9b3-4d9e-8f5e-8036eecddab9
DisplayText	-	Read and write all managed tenant information
Description	-	Allows the app to read and write all managed tenant information on behalf of the signed-in user.
AdminConsentRequired	-	Yes

Member.Read.Hidden

Category	Application	Delegated
Identifier	658aa5d8-239f-45c4-aa12-864f4fc7e490	f6a3db3e-f7e8-4ed2-a414-557c8c9830be
DisplayText	Read all hidden memberships	Read hidden memberships

Category	Application	Delegated
Description	Allows the app to read the memberships of hidden groups and administrative units without a signed-in user.	Allows the app to read the memberships of hidden groups and administrative units on behalf of the signed-in user, for those hidden groups and administrative units that the signed-in user has access to.
AdminConsentRequired	Yes	Yes

MultiTenantOrganization.Read.All

Category	Application	Delegated
Identifier	4f994bc0-31bb-44bb-b480-7a7c1be8c02e	526aa72a-5878-49fe-bf4e-357973af9b06
DisplayText	Read all multi-tenant organization details and tenants	Read multi-tenant organization details and tenants
Description	Allows the app to read all multi-tenant organization details and tenants, without a signed-in user.	Allows the app to read multi-tenant organization details and tenants on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

MultiTenantOrganization.ReadBasic.All

Category	Application	Delegated
Identifier	f9c2b2a7-3895-4b2e-80f6-c924b456e50b	225db56b-15b2-4daa-acb3-0eec2bbe4849
DisplayText	Read multi-tenant organization basic details and active tenants	Read multi-tenant organization basic details and active tenants
Description	Allows the app to read multi-tenant organization basic details and active tenants, without a signed-in user.	Allows the app to read multi-tenant organization basic details and active tenants on behalf of the signed-in user.
AdminConsentRequired	Yes	No

MultiTenantOrganization.ReadWrite.All

Category	Application	Delegated
Identifier	920def01-ca61-4d2d-b3df-105b46046a70	77af1528-84f3-4023-8d90-d219cd433108
DisplayText	Read and write all multi-tenant organization details and tenants	Read and write multi-tenant organization details and tenants
Description	Allows the app to read and write all multi-tenant organization details and tenants, without a signed-in user.	Allows the app to read and write multi-tenant organization details and tenants on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

MutualTlsOauthConfiguration.Read.All

Category	Application	Delegated
Identifier	6daaff82-2880-496d-9d80-57e8e31195e2	51ae584e-e736-4718-897b-10af70f8e3cc
DisplayText	Read all configurations used for mutual-TLS client authentication.	Read all configurations used for mutual-TLS client authentication.
Description	Allows the app to read configuration used for OAuth 2.0 mutual-TLS client authentication, without a signed-in user. This includes reading trusted certificate authorities.	Allows the app to read configuration used for OAuth 2.0 mutual-TLS client authentication, on behalf of the signed-in user. This includes reading trusted certificate authorities.
AdminConsentRequired	Yes	Yes

MutualTlsOauthConfiguration.ReadWrite.All

Category	Application	Delegated
Identifier	78bbf8cf-07d8-45ba-b0eb-1a7b48efbcf1	a51115bc-f64f-498f-bcee-00dcd28f4a03
DisplayText	Read and write all configurations used for mutual-TLS client authentication.	Read and write all configurations used for mutual-TLS client authentication.
Description	Allows the app to read and update configuration used for OAuth 2.0 mutual-TLS client authentication, without a signed-in user. This includes	Allows the app to read and update configuration used for OAuth 2.0 mutual-TLS client authentication, on behalf of the signed-in user. This

Category	Application	Delegated
	reading and updating trusted certificate authorities.	includes adding and updating trusted certificate authorities.
AdminConsentRequired	Yes	Yes

NetworkAccess-Reports.Read.All

Category	Application	Delegated
Identifier	40049381-3cc1-42af-94ec-5ce755db4b0d	b0c61509-cfc3-42bd-9bd4-66d81785fee4
DisplayText	Read all network access reports	Read all network access reports
Description	Allows the app to read all network access reports without a signed-in user.	Allows the app to read all network access reports on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

NetworkAccess.Read.All

Category	Application	Delegated
Identifier	e30060de-caa5-4331-99d3-6ac6c966a9a4	2f7013e0-ab4e-447f-a5e1-5d419950692d
DisplayText	Read all network access information	Read all network access information
Description	Allows the app to read all network access information and configuration settings without a signed-in user.	Allows the app to read all network access information on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

NetworkAccess.ReadWrite.All

Category	Application	Delegated
Identifier	b10642fc-a6cf-4c46-87f9-e1f96c2a18aa	ae2df9c5-f18d-4ec4-a51b-bdeb807f177b

Category	Application	Delegated
DisplayText	Read and write all network access information	Read and write all network access information
Description	Allows the app to read and write all network access information and configuration settings without a signed-in user.	Allows the app to read and write all network access information and configuration settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

NetworkAccessBranch.Read.All

Category	Application	Delegated
Identifier	39ae4a24-1ef0-49e8-9d63-2a66f5c39edd	4051c7fc-b429-4804-8d80-8f1f8c24a6f7
DisplayText	Read properties of all branches for network access	Read properties of branches for network access
Description	Allows the app to read your organization's network access branches, without a signed-in user.	Allows the app to read your organization's branches for network access on behalf of the signed-in user.
AdminConsentRequired	Yes	No

NetworkAccessBranch.ReadWrite.All

Category	Application	Delegated
Identifier	8137102d-ec16-4191-aaf8-7aeda8026183	b8a36cc2-b810-461a-baa4-a7281e50bd5c
DisplayText	Read and write properties of all branches for network access	Read and write properties of branches for network access
Description	Allows the app to read and write your organization's network access branches, without a signed-in user.	Allows the app to read and write your organization's branches for network access on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

NetworkAccessPolicy.Read.All

Category	Application	Delegated
Identifier	8a3d36bf-cb46-4bcc-bec9-8d92829dab84	ba22922b-752c-446f-89d7-a2d92398fceb
DisplayText	Read all security and routing policies for network access	Read security and routing policies for network access
Description	Allows the app to read your organization's network access policies, without a signed-in user.	Allows the app to read your organization's security and routing network access policies on behalf of the signed-in user.
AdminConsentRequired	Yes	No

NetworkAccessPolicy.ReadWrite.All

Category	Application	Delegated
Identifier	f0c341be-8348-4989-8e43-660324294538	b1fbad0f-ef6e-42ed-8676-bca7fa3e7291
DisplayText	Read and write all security and routing policies for network access	Read and write security and routing policies for network access
Description	Allows the app to read and write your organization's network access policies, without a signed-in user.	Allows the app to read and write your organization's security and routing network access policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Notes.Create

Category	Application	Delegated
Identifier	-	9d822255-d64d-4b7a-afdb-833b9a97ed02
DisplayText	-	Create user OneNote notebooks
Description	-	Allows the app to read the titles of OneNote notebooks and sections and to create new pages, notebooks, and sections on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	-	No

The *Notes.Create* delegated permission is available for consent in personal Microsoft accounts.

Notes.Read

Category	Application	Delegated
Identifier	-	371361e4-b9e2-4a3f-8315-2a301a3b0a3d
DisplayText	-	Read user OneNote notebooks
Description	-	Allows the app to read OneNote notebooks on behalf of the signed-in user.
AdminConsentRequired	-	No

The *Notes.Read* delegated permission is available for consent in personal Microsoft accounts.

Notes.Read.All

Category	Application	Delegated
Identifier	3aeca27b-ee3a-4c2b-8ded-80376e2134a4	dfabfca6-ee36-4db2-8208-7a28381419b3
DisplayText	Read all OneNote notebooks	Read all OneNote notebooks that user can access
Description	Allows the app to read all the OneNote notebooks in your organization, without a signed-in user.	Allows the app to read OneNote notebooks that the signed-in user has access to in the organization.
AdminConsentRequired	Yes	No

Notes.ReadWrite

Category	Application	Delegated
Identifier	-	615e26af-c38a-4150-ae3e-c3b0d4cb1d6a

Category	Application	Delegated
DisplayText	-	Read and write user OneNote notebooks
Description	-	Allows the app to read, share, and modify OneNote notebooks on behalf of the signed-in user.
AdminConsentRequired	-	No

The *Notes.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

Notes.ReadWrite.All

Category	Application	Delegated
Identifier	0c458cef-11f3-48c2-a568-c66751c238c0	64ac0503-b4fa-45d9-b544-71a463f05da0
DisplayText	Read and write all OneNote notebooks	Read and write all OneNote notebooks that user can access
Description	Allows the app to read all the OneNote notebooks in your organization, without a signed-in user.	Allows the app to read, share, and modify OneNote notebooks that the signed-in user has access to in the organization.
AdminConsentRequired	Yes	No

Notes.ReadWrite.CreatedByApp

Category	Application	Delegated
Identifier	-	ed68249d-017c-4df5-9113-e684c7f8760b
DisplayText	-	Limited notebook access (deprecated)
Description	-	This is deprecated! Do not use! This permission no longer has any effect. You can safely consent to it. No additional privileges will be granted to the app.
AdminConsentRequired	-	No

Notifications.ReadWrite.CreatedByApp

Category	Application	Delegated
Identifier	-	89497502-6e42-46a2-8cb2-427fd3df970a
DisplayText	-	Deliver and manage user notifications for this app
Description	-	Allows the app to deliver its notifications on behalf of signed-in users. Also allows the app to read, update, and delete the user's notification items for this app.
AdminConsentRequired	-	No

The *Notifications.ReadWrite.CreatedByApp* delegated permission is available for consent in personal Microsoft accounts.

---

offline_access

Category	Application	Delegated
Identifier	-	7427e0e9-2fba-42fe-b0c0-848c9e6a8182
DisplayText	-	Maintain access to data you have given it access to
Description	-	Allows the app to see and update the data you gave it access to, even when users are not currently using the app. This does not give the app any additional permissions.
AdminConsentRequired	-	No

offline_access is an OpenID Connect (OIDC) scope.

You can use the OIDC scopes to specify artifacts that you want returned in Microsoft identity platform authorization and token requests. The Microsoft identity platform v1.0 and v2.0 endpoints support OIDC scopes differently.

With the Microsoft identity platform v1.0 endpoint, only the *openid* scope is used. You specify it in the *scope* parameter in an authorization request to return an ID token when you use the OpenID Connect protocol to sign in a user to your app. For more information, see Microsoft identity platform and OAuth 2.0 authorization code flow. To successfully return an ID token, you must also make sure that the *User.Read* permission is configured when you register your app.

With the Microsoft identity platform v2.0 endpoint, you specify the *offline_access* scope in the **scope** parameter to explicitly request a refresh token when using the OAuth 2.0 or OpenID Connect protocols. With OpenID Connect, you specify the *openid* scope to request an ID token. You can also specify the *email* scope, *profile* scope, or both to return additional claims in the ID token. You don't need to specify the *User.Read* permission to return an ID token with the v2.0 endpoint. For more information, see OpenID Connect scopes.

The Microsoft Authentication Library (MSAL) currently specifies *offline_access*, *openid*, *profile*, and *email* by default in authorization and token requests. Therefore, for the default case, if you specify these scopes explicitly, the Microsoft identity platform might return an error.

OnlineMeetingAiInsight.Read.All

Category	Application	Delegated
Identifier	c0cf7895-985f-42d4-a693-b618f36674ad	166741d6-eeb8-46fe-91f4-817d2af7bc88
DisplayText	Read all AI Insights for online meetings.	Read all AI Insights for online meetings.
Description	Allows the app to read all AI Insights for all online meetings, without a signed-in user.	Allows the app to read all AI Insights for online meetings, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OnlineMeetingAiInsight.Read.Chat

Category	Application	Delegated
Identifier	01892c31-3b66-4bcf-b5f5-bf0a03d5ed9f	-
DisplayText	Read all AI Insights for online meetings where the Teams application is installed.	-
Description	Allows the teams-app to read all aiInsights for online meetings where the Teams-app is installed, without a signed-in user.	-
AdminConsentRequired	Yes	-

OnlineMeetingArtifact.Read.All

Category	Application	Delegated
Identifier	df01ed3b-eb61-4eca-9965-6b3d789751b2	110e5abb-a10c-4b59-8b55-9b4daa4ef743
DisplayText	Read online meeting artifacts	Read user's online meeting artifacts
Description	Allows the app to read online meeting artifacts in your organization, without a signed-in user.	Allows the app to read online meeting artifacts on behalf of the signed-in user.
AdminConsentRequired	Yes	No

Administrators can configure application access policy to allow apps to access online meetings on behalf of a user.

OnlineMeetingRecording.Read.All

Category	Application	Delegated
Identifier	a4a08342-c95d-476b-b943-97e100569c8d	190c2bb6-1fdd-4fec-9aa2-7d571b5e1fe3
DisplayText	Read all recordings of online meetings.	Read all recordings of online meetings.
Description	Allows the app to read all recordings of all online meetings, without a signed-in user.	Allows the app to read all recordings of online meetings, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Administrators can configure application access policy to allow apps to access online meetings on behalf of a user.

OnlineMeetings.Read

Category	Application	Delegated
Identifier	-	9be106e1-f4e3-4df5-bdff-e4bc531cbe43
DisplayText	-	Read user's online meetings

Category	Application	Delegated
Description	-	Allows the app to read online meeting details on behalf of the signed-in user.
AdminConsentRequired	-	No

Administrators can configure application access policy to allow apps to access online meetings on behalf of a user.

OnlineMeetings.Read.All

Category	Application	Delegated
Identifier	c1684f21-1984-47fa-9d61-2dc8c296bb70	-
DisplayText	Read online meeting details	-
Description	Allows the app to read online meeting details in your organization, without a signed-in user.	-
AdminConsentRequired	Yes	-

OnlineMeetings.ReadWrite

Category	Application	Delegated
Identifier	-	a65f2972-a4f8-4f5e-afd7-69ccb046d5dc
DisplayText	-	Read and create user's online meetings
Description	-	Allows the app to read and create online meetings on behalf of the signed-in user.
AdminConsentRequired	-	No

Administrators can configure application access policy to allow apps to access online meetings on behalf of a user.

OnlineMeetings.ReadWrite.All

Category	Application	Delegated
Identifier	b8bb2037-6e08-44ac-a4ea-4674e010e2a4	-
DisplayText	Read and create online meetings	-
Description	Allows the app to read and create online meetings as an application in your organization.	-
AdminConsentRequired	Yes	-

OnlineMeetingTranscript.Read.All

Category	Application	Delegated
Identifier	a4a80d8d-d283-4bd8-8504-555ec3870630	30b87d18-ebb1-45db-97f8-82ccb1f0190c
DisplayText	Read all transcripts of online meetings.	Read all transcripts of online meetings.
Description	Allows the app to read all transcripts of all online meetings, without a signed-in user.	Allows the app to read all transcripts of online meetings, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Administrators can configure application access policy to allow apps to access online meetings on behalf of a user.

OnPremDirectorySynchronization.Read.All

Category	Application	Delegated
Identifier	bb70e231-92dc-4729-aff5-697b3f04be95	f6609722-4100-44eb-b747-e6ca0536989d
DisplayText	Read all on-premises directory synchronization information	Read all on-premises directory synchronization information
Description	Allows the app to read all on-premises directory synchronization information for the organization, without a signed-in user.	Allows the app to read all on-premises directory synchronization information for the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OnPremDirectorySynchronization.ReadWrite.All

Category	Application	Delegated
Identifier	c22a92cc-79bf-4bb1-8b6c-e0a05d3d80ce	c2d95988-7604-4ba1-aaed-38a5f82a51c7
DisplayText	Read and write all on-premises directory synchronization information	Read and write all on-premises directory synchronization information
Description	Allows the app to read and write all on-premises directory synchronization information for the organization, without a signed-in user.	Allows the app to read and write all on-premises directory synchronization information for the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OnPremisesPublishingProfiles.ReadWrite.All

Category	Application	Delegated
Identifier	0b57845e-aa49-4e6f-8109-ce654fffa618	8c4d5184-71c2-4bf8-bb9d-bc3378c9ad42
DisplayText	Manage on-premises published resources	Manage on-premises published resources
Description	Allows the app to create, view, update and delete on-premises published resources, on-premises agents and agent groups, as part of a hybrid identity configuration, without a signed in user.	Allows the app to manage hybrid identity service configuration by creating, viewing, updating and deleting on-premises published resources, on-premises agents and agent groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

openid

Category	Application	Delegated
Identifier	-	37f7f235-527c-4136-accd-4a02d197296e
DisplayText	-	Sign users in

Category	Application	Delegated
Description	-	Allows users to sign in to the app with their work or school accounts and allows the app to see basic user profile information.
AdminConsentRequired	-	No

openid is an OpenID Connect (OIDC) scope.

You can use the OIDC scopes to specify artifacts that you want returned in Microsoft identity platform authorization and token requests. The Microsoft identity platform v1.0 and v2.0 endpoints support OIDC scopes differently.

With the Microsoft identity platform v1.0 endpoint, only the *openid* scope is used. You specify it in the *scope* parameter in an authorization request to return an ID token when you use the OpenID Connect protocol to sign in a user to your app. For more information, see Microsoft identity platform and OAuth 2.0 authorization code flow. To successfully return an ID token, you must also make sure that the *User.Read* permission is configured when you register your app.

With the Microsoft identity platform v2.0 endpoint, you specify the *offline_access* scope in the **scope** parameter to explicitly request a refresh token when using the OAuth 2.0 or OpenID Connect protocols. With OpenID Connect, you specify the *openid* scope to request an ID token. You can also specify the *email* scope, *profile* scope, or both to return additional claims in the ID token. You don't need to specify the *User.Read* permission to return an ID token with the v2.0 endpoint. For more information, see OpenID Connect scopes.

The Microsoft Authentication Library (MSAL) currently specifies *offline_access*, *openid*, *profile*, and *email* by default in authorization and token requests. Therefore, for the default case, if you specify these scopes explicitly, the Microsoft identity platform might return an error.

Organization.Read.All

Category	Application	Delegated
Identifier	498476ce-e0fe-48b0-b801-37ba7e2685c6	4908d5b9-3fb2-4b1e-9336-1888b7937185
DisplayText	Read organization information	Read organization information
Description	Allows the app to read the organization and related resources, without a signed-in user. Related resources include things like	Allows the app to read the organization and related resources, on behalf of the signed-in user. Related resources

Category	Application	Delegated
	subscribed skus and tenant branding information.	include things like subscribed skus and tenant branding information.
AdminConsentRequired	Yes	Yes

Organization.ReadWrite.All

Category	Application	Delegated
Identifier	292d869f-3427-49a8-9dab-8c70152b74e9	46ca0847-7e6b-426e-9775-ea810a948356
DisplayText	Read and write organization information	Read and write organization information
Description	Allows the app to read and write the organization and related resources, without a signed-in user. Related resources include things like subscribed skus and tenant branding information.	Allows the app to read and write the organization and related resources, on behalf of the signed-in user. Related resources include things like subscribed skus and tenant branding information.
AdminConsentRequired	Yes	Yes

OrganizationalBranding.Read.All

Category	Application	Delegated
Identifier	eb76ac34-0d62-4454-b97c-185e4250dc20	9082f138-6f02-4f3a-9f4d-5f3c2ce5c688
DisplayText	Read organizational branding information	Read organizational branding information
Description	Allows the app to read the organizational branding information, without a signed-in user.	Allows the app to read the organizational branding information, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrganizationalBranding.ReadWrite.All

Category	Application	Delegated
Identifier	d2ebfbc1-a5f8-424b-83a6-56ab5927a73c	15ce63de-b141-4c9a-a9a5-241bf27c6aaf
DisplayText	Read and write organizational branding information	Read and write organizational branding information
Description	Allows the app to read and write the organizational branding information, without a signed-in user.	Allows the app to read and write the organizational branding information, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgContact.Read.All

Category	Application	Delegated
Identifier	e1a88a34-94c4-4418-be12-c87b00e26bea	08432d1b-5911-483c-86df-7980af5cdee0
DisplayText	Read organizational contacts	Read organizational contacts
Description	Allows the app to read all organizational contacts without a signed-in user. These contacts are managed by the organization and are different from a user's personal contacts.	Allows the app to read all organizational contacts on behalf of the signed-in user. These contacts are managed by the organization and are different from a user's personal contacts.
AdminConsentRequired	Yes	Yes

OrgSettings-AppsAndServices.Read.All

Category	Application	Delegated
Identifier	56c84fa9-ea1f-4a15-90f2-90ef41ece2c9	1e9b7a7e-4d64-44ff-acf5-2e9651c1519f
DisplayText	Read organization-wide apps and services settings	Read organization-wide apps and services settings
Description	Allows the app to read organization-wide apps and services settings, without a signed-in user.	Allows the app to read organization-wide apps and services settings on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

OrgSettings-AppsAndServices.ReadWrite.All

Category	Application	Delegated
Identifier	4a8e4191-c1c8-45f8-b801-f9a1a5ee6ad3	c167b0e7-47c0-48e8-9eee-9892f58018fa
DisplayText	Read and write organization-wide apps and services settings	Read and write organization-wide apps and services settings
Description	Allows the app to read and write organization-wide apps and services settings, without a signed-in user.	Allows the app to read and write organization-wide apps and services settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgSettings-DynamicsVoice.Read.All

Category	Application	Delegated
Identifier	c18ae2dc-d9f3-4495-a93f-18980a0e159f	9862d930-5aec-4a98-8d4f-7277a8db9bcb
DisplayText	Read organization-wide Dynamics customer voice settings	Read organization-wide Dynamics customer voice settings
Description	Allows the app to read organization-wide Dynamics customer voice settings, without a signed-in user.	Allows the app to read organization-wide Dynamics customer voice settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgSettings-DynamicsVoice.ReadWrite.All

Category	Application	Delegated
Identifier	c3f1cc32-8bbd-4ab6-bd33-f270e0d9e041	4cea26fb-6967-4234-82c4-c044414743f8

Category	Application	Delegated
DisplayText	Read and write organization-wide Dynamics customer voice settings	Read and write organization-wide Dynamics customer voice settings
Description	Allows the app to read and write organization-wide Dynamics customer voice settings, without a signed-in user.	Allows the app to read and write organization-wide Dynamics customer voice settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgSettings-Forms.Read.All

Category	Application	Delegated
Identifier	434d7c66-07c6-4b1f-ab21-417cf2cdaaca	210051a0-1ffc-435c-ae76-02d226d05752
DisplayText	Read organization-wide Microsoft Forms settings	Read organization-wide Microsoft Forms settings
Description	Allows the app to read organization-wide Microsoft Forms settings, without a signed-in user.	Allows the app to read organization-wide Microsoft Forms settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgSettings-Forms.ReadWrite.All

Category	Application	Delegated
Identifier	2cb92fee-97a3-4034-8702-24a6f5d0d1e9	346c19ff-3fb2-4e81-87a0-bac9e33990c1
DisplayText	Read and write organization-wide Microsoft Forms settings	Read and write organization-wide Microsoft Forms settings
Description	Allows the app to read and write organization-wide Microsoft Forms settings, without a signed-in user.	Allows the app to read and write organization-wide Microsoft Forms settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgSettings-Microsoft365Install.Read.All

Category	Application	Delegated
Identifier	6cdf1fb1-b46f-424f-9493-07247caa22e2	8cbdb9f6-9c2e-451a-814d-ec606e5d0212
DisplayText	Read organization-wide Microsoft 365 apps installation settings	Read organization-wide Microsoft 365 apps installation settings
Description	Allows the app to read organization-wide Microsoft 365 apps installation settings, without a signed-in user.	Allows the app to read organization-wide Microsoft 365 apps installation settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgSettings-Microsoft365Install.ReadWrite.All

Category	Application	Delegated
Identifier	83f7232f-763c-47b2-a097-e35d2cbe1da5	1ff35e91-19eb-42d8-aa2d-cc9891127ae5
DisplayText	Read and write organization-wide Microsoft 365 apps installation settings	Read and write organization-wide Microsoft 365 apps installation settings
Description	Allows the app to read and write organization-wide Microsoft 365 apps installation settings, without a signed-in user.	Allows the app to read and write organization-wide Microsoft 365 apps installation settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgSettings-Todo.Read.All

Category	Application	Delegated
Identifier	e4d9cd09-d858-4363-9410-abb96737f0cf	7ff96f41-f022-45ba-acd8-ef3f03063d6b
DisplayText	Read organization-wide Microsoft To Do settings	Read organization-wide Microsoft To Do settings
Description	Allows the app to read organization-wide Microsoft To Do settings, without	Allows the app to read organization-wide Microsoft To Do settings on behalf

Category	Application	Delegated
	a signed-in user.	of the signed-in user.
AdminConsentRequired	Yes	Yes

OrgSettings-Todo.ReadWrite.All

Category	Application	Delegated
Identifier	5febc9da-e0d0-4576-bd13-ae70b2179a39	087502c2-5263-433e-abe3-8f77231a0627
DisplayText	Read and write organization-wide Microsoft To Do settings	Read and write organization-wide Microsoft To Do settings
Description	Allows the app to read and write organization-wide Microsoft To Do settings, without a signed-in user.	Allows the app to read and write organization-wide Microsoft To Do settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PartnerBilling.Read.All

Category	Application	Delegated
Identifier	7c3e1994-38ff-4412-a99b-9369f6bb7706	8804798e-5934-4e30-8ce3-ef88257cecd4
DisplayText	Read all billing data for your company's tenant	Read all billing data for your company's tenant
Description	Allows the app to read all of billing data from Microsoft for your company's tenant, without a signed-in user. This includes reading billed and unbilled azure usage and invoice reconciliation data.	Allows the app to read all of billing data from Microsoft for your company's tenant, on behalf of the signed-in user. This includes reading billed and unbilled Usage and Invoice reconciliation data.
AdminConsentRequired	Yes	Yes

PartnerSecurity.Read.All

Category	Application	Delegated
Identifier	21ffa320-2e7f-47d3-a466-7ff04d2dd68d	5567b981-0bf1-4796-9038-0648b46e116d
DisplayText	Read security alerts of customer with CSP relationship	Read security alerts of customer with CSP relationship
Description	Allows the app to read security alerts of customer with CSP relationship, without a signed-in user.	Allows the app to read security alerts of customer with CSP relationship on behalf of the partner signed-in user.
AdminConsentRequired	Yes	Yes

PartnerSecurity.ReadWrite.All

Category	Application	Delegated
Identifier	04a2c935-5b4b-474a-be42-11f53111f271	0cd2c1f6-94a1-4075-ab8c-0b1aff2e1ad5
DisplayText	Read security alerts and update status of security alerts of customer with CSP relationship	Read security alerts and update status of security alerts of customer with CSP relationship
Description	Allows the app to read security alerts and update status of alerts of customer with CSP relationship, without a signed-in user.	Allows the app to read security alerts and update status of alerts of customer with CSP relationship on behalf of the partner signed-in user.
AdminConsentRequired	Yes	Yes

PendingExternalUserProfile.Read.All

Category	Application	Delegated
Identifier	bdfb26d9-bb36-49be-9b4c-b8cbf4b05808	d88fd3fb-53d3-4c1c-8c39-787fcac2ed7a
DisplayText	Read all pending external user profiles	Read pending external user profiles
Description	Allows the app to read available properties of pending external user profiles, without a signed-in user.	Allows the app to read available properties of pending external user profiles, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PendingExternalUserProfile.ReadWrite.All

Category	Application	Delegated
Identifier	8363c2b8-6ff7-420b-9966-c5884c2d48bc	93a1fb28-c908-4826-904e-0c74ad352b73
DisplayText	Read and write all pending external user profiles	Read and write pending external user profiles
Description	Allows the app to read and write available properties of pending external user profiles, without a signed-in user.	Allows the app to read and write available properties of pending external user profiles, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

People.Read

Category	Application	Delegated
Identifier	-	ba47897c-39ec-4d83-8086-ee8256fa737d
DisplayText	-	Read users' relevant people lists
Description	-	Allows the app to read a ranked list of relevant people of the signed-in user. The list includes local contacts, contacts from social networking, your organization's directory, and people from recent communications (such as email and Skype).
AdminConsentRequired	-	No

The *People.Read* delegated permission is available for consent in personal Microsoft accounts.

People.Read.All

Category	Application	Delegated
Identifier	b528084d-ad10-4598-8b93-929746b4d7d6	b89f9189-71a5-4e70-b041-9887f0bc7e4a
DisplayText	Read all users' relevant people lists	Read all users' relevant people lists

Category	Application	Delegated
Description	Allows the app to read any user's scored list of relevant people, without a signed-in user. The list can include local contacts, contacts from social networking, your organization's directory, and people from recent communications (such as email and Skype).	Allows the app to read a scored list of relevant people of the signed-in user or other users in the signed-in user's organization. The list can include local contacts, contacts from social networking, your organization's directory, and people from recent communications (such as email and Skype).
AdminConsentRequired	Yes	Yes

PeopleSettings.Read.All

Category	Application	Delegated
Identifier	ef02f2e7-e22d-4c77-8614-8f765683b86e	ec762c5f-388b-4b16-8693-ac1efbc611bc
DisplayText	Read all tenant-wide people settings	Read tenant-wide people settings
Description	Allows the application to read tenant-wide people settings without a signed-in user.	Allows the application to read tenant-wide people settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PeopleSettings.ReadWrite.All

Category	Application	Delegated
Identifier	b6890674-9dd5-4e42-bb15-5af07f541ae1	e67e6727-c080-415e-b521-e3f35d5248e9
DisplayText	Read and write all tenant-wide people settings	Read and write tenant-wide people settings
Description	Allows the application to read and write tenant-wide people settings without a signed-in user.	Allows the application to read and write tenant-wide people settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Place.Read.All

Category	Application	Delegated
Identifier	913b9306-0ce1-42b8-9137-6a7df690a760	cb8f45a0-5c2e-4ea1-b803-84b870a7d7ec
DisplayText	Read all company places	Read all company places
Description	Allows the app to read company places (conference rooms and room lists) for calendar events and other applications, without a signed-in user.	Allows the app to read your company's places (conference rooms and room lists) for calendar events and other applications, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Place.ReadWrite.All

Category	Application	Delegated
Identifier	f1f5e9aa-ad18-4b97-883e-6aa7e95b7a5f	4c06a06a-098a-4063-868e-5dfee3827264
DisplayText	Read and write organization places	Read and write organization places
Description	Allows the app to manage organization places (conference rooms and room lists) for calendar events and other applications, on behalf of the signed-in user.	Allows the app to manage organization places (conference rooms and room lists) for calendar events and other applications, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PlaceDevice.Read.All

Category	Application	Delegated
Identifier	8b724a84-ceac-4fd9-897e-e31ba8f2d7a3	4c7f93d2-6b0b-4e05-91aa-87842f0a2142
DisplayText	Read all workplace devices	Read all workplace devices
Description	Allows the app to read all workplace devices, without a signed-in user.	Allows the app to read all workplace devices, on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

PlaceDevice.ReadWrite.All

Category	Application	Delegated
Identifier	2d510721-5c4e-43cd-bfdb-ac0f8819fb92	eafd6a71-e95a-4f8a-bb6e-fb84ab7fbd9e
DisplayText	Read and write all workplace devices	Read and write all workplace devices
Description	Allows the app to read and write all workplace devices, without a signed-in user.	Allows the app to read and write all workplace devices, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PlaceDeviceTelemetry.ReadWrite.All

Category	Application	Delegated
Identifier	27fc435f-44e2-4b30-bf3c-e0ce74aed618	-
DisplayText	Read and write telemetry for all workplace devices.	-
Description	Allows the app to read and write telemetry for all workplace devices, without a signed-in user.	-
AdminConsentRequired	Yes	-

Policy.Read.All

Category	Application	Delegated
Identifier	246dd0d5-5bd0-4def-940b-0421030a5b68	572fea84-0151-49b2-9301-11cb16974376
DisplayText	Read your organization's policies	Read your organization's policies
Description	Allows the app to read all your organization's policies without a	Allows the app to read your organization's policies on behalf of the

Category	Application	Delegated
	signed in user.	signed-in user.
AdminConsentRequired	Yes	Yes

The *Policy.Read.All* delegated permission is available for consent in personal Microsoft accounts.

Policy.Read.AuthenticationMethod

Category	Application	Delegated
Identifier	8e3bc81b-d2f3-4b7b-838c-32c88218d2f0	a6ff13ac-1851-4993-8ca9-a671d70de2d5
DisplayText	Read authentication method policies	Read authentication method policies
Description	Allows the app to read all authentication method policies for the tenant, without a signed-in user.	Allows the app to read the authentication method policies, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.Read.B2BManagementPolicy

Category	Application	Delegated
Identifier	227900ff-df89-40f8-90e2-8157cf6995d5	4b293250-121d-4cb4-acc7-5280438c18a6
DisplayText	Read your organization's B2BManagement policies	Read your organization's B2BManagement policies
Description	Allows the app to read all your organization's B2BManagement policies without a signed in user.	Allows the app to read your organization's B2BManagement policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.Read.ConditionalAccess

Category	Application	Delegated
Identifier	37730810-e9ba-4e46-b07e-8ca78d182097	633e0fce-8c58-4cfb-9495-12bbd5a24f7c
DisplayText	Read your organization's conditional access policies	Read your organization's conditional access policies
Description	Allows the app to read your organization's conditional access policies, without a signed-in user.	Allows the app to read your organization's conditional access policies on behalf of the signed-in user.
AdminConsentRequired	Yes	No

Policy.Read.CrossTenantAccess

Category	Application	Delegated
Identifier	8fc84b9a-5c21-479b-ba9d-02d252d062aa	b337372a-8b4d-428d-9cd8-3d7363865736
DisplayText	Read your organization's cross tenant access policies	Read your organization's cross tenant access policies
Description	Allows the app to read your organization's cross tenant access policies without a signed-in user.	Allows the app to read your organization's cross tenant access policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.Read.DeviceConfiguration

Category	Application	Delegated
Identifier	bdba4817-6ba1-4a7c-8a01-be9bc7c242dd	3616a4b0-6746-49c4-a678-4c237599074d
DisplayText	Read your organization's device configuration policies	Read your organization's device configuration policies
Description	Allows the application to read your organization's device configuration policies without a signed-in user. For example, device registration policy can limit initial provisioning controls using quota restrictions, additional	Allows the app to read your organization's device configuration policies on behalf of the signed-in user. For example, device registration policy can limit initial provisioning controls using quota restrictions, additional

Category	Application	Delegated
	authentication and authorization checks.	authentication and authorization checks.
AdminConsentRequired	Yes	Yes

Policy.Read.IdentityProtection

Category	Application	Delegated
Identifier	b21b72f6-4e6a-4533-9112-47eea9f97b28	d146432f-b803-4ed4-8d42-ba74193a6ede
DisplayText	Read your organization's identity protection policy	Read your organization's identity protection policy
Description	Allows the app to read your organization's identity protection policy without a signed-in user.	Allows the app to read your organization's identity protection policy on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.Read.PermissionGrant

Category	Application	Delegated
Identifier	9e640839-a198-48fb-8b9a-013fd6f6cbcd	414de6ea-2d92-462f-b120-6e2a809a6d01
DisplayText	Read consent and permission grant policies	Read consent and permission grant policies
Description	Allows the app to read policies related to consent and permission grants for applications, without a signed-in user.	Allows the app to read policies related to consent and permission grants for applications, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.Read.Recovery

Category	Application	Delegated
Identifier	447f996a-7c58-4ce7-9a9e-da80381a45ab	61faa1e9-0931-4f9a-94ba-bc2e3505c685
DisplayText	Read your organization's recovery policy	Read your organization's recovery policy
Description	Allows the application to read and update the organization's recovery policy without a signed-in user.	Allows the application to read the organization's recovery policy on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.AccessReview

Category	Application	Delegated
Identifier	77c863fd-06c0-47ce-a7eb-49773e89d319	4f5bc9c8-ea54-4772-973a-9ca119cb0409
DisplayText	Read and write your organization's directory access review default policy	Read and write your organization's directory access review default policy
Description	Allows the app to read and write your organization's directory access review default policy without a signed-in user.	Allows the app to read and write your organization's directory access review default policy on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.ApplicationConfiguration

Category	Application	Delegated
Identifier	be74164b-cff1-491c-8741-e671cb536e13	b27add92-efb2-4f16-84f5-8108ba77985c
DisplayText	Read and write your organization's application configuration policies	Read and write your organization's application configuration policies
Description	Allows the app to read and write your organization's application configuration policies, without a signed-in user. This includes policies such as activityBasedTimeoutPolicy,	Allows the app to read and write your organization's application configuration policies on behalf of the signed-in user. This includes policies such as activityBasedTimeoutPolicy,

Category	Application	Delegated
	claimsMappingPolicy, homeRealmDiscoveryPolicy, tokenIssuancePolicy and tokenLifetimePolicy.	claimsMappingPolicy, homeRealmDiscoveryPolicy, tokenIssuancePolicy and tokenLifetimePolicy.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.AuthenticationFlows

Category	Application	Delegated
Identifier	25f85f3c-f66c-4205-8cd5-de92dd7f0cec	edb72de9-4252-4d03-a925-451deef99db7
DisplayText	Read and write authentication flow policies	Read and write authentication flow policies
Description	Allows the app to read and write all authentication flow policies for the tenant, without a signed-in user.	Allows the app to read and write the authentication flow policies, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.AuthenticationMethod

Category	Application	Delegated
Identifier	29c18626-4985-4dcd-85c0-193eef327366	7e823077-d88e-468f-a337-e18f1f0e6c7c
DisplayText	Read and write all authentication method policies	Read and write authentication method policies
Description	Allows the app to read and write all authentication method policies for the tenant, without a signed-in user.	Allows the app to read and write the authentication method policies, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

The *Policy.ReadWrite.AuthenticationMethod* delegated permission is available for consent in personal Microsoft accounts.

Policy.ReadWrite.Authorization

Category	Application	Delegated
Identifier	fb221be6-99f2-473f-bd32-01c6a0e9ca3b	edd3c878-b384-41fd-95ad-e7407dd775be
DisplayText	Read and write your organization's authorization policy	Read and write your organization's authorization policy
Description	Allows the app to read and write your organization's authorization policy without a signed in user. For example, authorization policies can control some of the permissions that the out-of-the-box user role has by default.	Allows the app to read and write your organization's authorization policy on behalf of the signed-in user. For example, authorization policies can control some of the permissions that the out-of-the-box user role has by default.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.B2BManagementPolicy

Category	Application	Delegated
Identifier	886bd2d9-5b8b-4b49-adea-ca75fb50d9ef	723c4a0c-85b0-4a02-bb2a-c9eb07959de9
DisplayText	Read and write your organization's B2BManagement policies	Read and write your organization's B2BManagement policies
Description	Allows the app to read and write all your organization's B2BManagement policies without a signed in user.	Allows the app to read and write your organization's B2BManagement policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.ConditionalAccess

Category	Application	Delegated
Identifier	01c0a623-fc9b-48e9-b794-0756f8e8f067	ad902697-1014-4ef5-81ef-2b4301988e8c
DisplayText	Read and write your organization's conditional access policies	Read and write your organization's conditional access policies

Category	Application	Delegated
Description	Allows the app to read and write your organization's conditional access policies, without a signed-in user.	Allows the app to read and write your organization's conditional access policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.ConsentRequest

Category	Application	Delegated
Identifier	999f8c63-0a38-4f1b-91fd-ed1947bdd1a9	4d135e65-66b8-41a8-9f8b-081452c91774
DisplayText	Read and write your organization's consent request policy	Read and write consent request policy
Description	Allows the app to read and write your organization's consent requests policy without a signed-in user.	Allows the app to read and write your organization's consent requests policy on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.CrossTenantAccess

Category	Application	Delegated
Identifier	338163d7-f101-4c92-94ba-ca46fe52447c	014b43d0-6ed4-4fc6-84dc-4b6f7bae7d85
DisplayText	Read and write your organization's cross tenant access policies	Read and write your organization's cross tenant access policies
Description	Allows the app to read and write your organization's cross-tenant access policies and configuration for automatic user consent settings to suppress consent prompts for users of the other tenant on behalf of the signed-in user.	Allows the app to read and write your organization's cross-tenant access policies and configuration for automatic user consent settings to suppress consent prompts for users of the other tenant on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.CrossTenantCapability

Category	Application	Delegated
Identifier	a6325ae7-2b73-4dbd-abed-fbeacfbf8696	9ef7463f-1d39-406f-89ea-3483a4645e1c
DisplayText	Read and write your organization's M365 cross tenant access capabilities	Read and write your organization's M365 cross tenant access capabilities
Description	Allows the app to read and write your organization's M365 cross tenant access capabilities without a signed-in user.	Allows the app to read and write your organization's M365 cross tenant access capabilities on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.DeviceConfiguration

Category	Application	Delegated
Identifier	230fb2d5-aa21-49c1-bfa7-ae1be179d867	40b534c3-9552-4550-901b-23879c90bcf9
DisplayText	Read and write your organization's device configuration policies	Read and write your organization's device configuration policies
Description	Allows the application to read and write your organization's device configuration policies without a signed-in user. For example, device registration policy can limit initial provisioning controls using quota restrictions, additional authentication and authorization checks.	Allows the app to read and write your organization's device configuration policies on behalf of the signed-in user. For example, device registration policy can limit initial provisioning controls using quota restrictions, additional authentication and authorization checks.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.ExternalIdentities

Category	Application	Delegated
Identifier	03cc4f92-788e-4ede-b93f-199424d144a5	b5219784-1215-45b5-b3f1-88fe1081f9c0

Category	Application	Delegated
DisplayText	Read and write your organization's external identities policy	Read and write your organization's external identities policy
Description	Allows the application to read and update the organization's external identities policy without a signed-in user. For example, external identities policy controls if users invited to access resources in your organization via B2B collaboration or B2B direct connect are allowed to self-service leave.	Allows the application to read and update the organization's external identities policy on behalf of the signed-in user. For example, external identities policy controls if users invited to access resources in your organization via B2B collaboration or B2B direct connect are allowed to self-service leave.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.FeatureRollout

Category	Application	Delegated
Identifier	2044e4f1-e56c-435b-925c-44cd8f6ba89a	92a38652-f13b-4875-bc77-6e1dbb63e1b2
DisplayText	Read and write feature rollout policies	Read and write your organization's feature rollout policies
Description	Allows the app to read and write feature rollout policies without a signed-in user. Includes abilities to assign and remove users and groups to rollout of a specific feature.	Allows the app to read and write your organization's feature rollout policies on behalf of the signed-in user. Includes abilities to assign and remove users and groups to rollout of a specific feature.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.FedTokenValidation

Category	Application	Delegated
Identifier	90bbca0b-227c-4cdc-8083-1c6cfb95bac6	be1be369-4540-4ac9-8928-79de99f70d8f
DisplayText	Read and write your organization's federated token validation policy	Read and write your organization's federated token validation policy

Category	Application	Delegated
Description	Allows the application to read and update the organization's federated token validation policy without a signed-in user.	Allows the application to read and update the organization's federated token validation policy on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.IdentityProtection

Category	Application	Delegated
Identifier	2dcf8603-09eb-4078-b1ec-d30a1a76b873	7256e131-3efb-4323-9854-cf41c6021770
DisplayText	Read and write your organization's identity protection policy	Read and write your organization's identity protection policy
Description	Allows the app to read and write your organization's identity protection policy without a signed-in user.	Allows the app to read and write your organization's identity protection policy on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.MobilityManagement

Category	Application	Delegated
Identifier	-	a8ead177-1889-4546-9387-f25e658e2a79
DisplayText	-	Read and write your organization's mobility management policies
Description	-	Allows the app to read and write your organization's mobility management policies on behalf of the signed-in user. For example, a mobility management policy can set the enrollment scope for a given mobility management application.
AdminConsentRequired	-	Yes

Policy.ReadWrite.PermissionGrant

Category	Application	Delegated
Identifier	a402ca1c-2696-4531-972d-6e5ee4aa11ea	2672f8bb-fd5e-42e0-85e1-ec764dd2614e
DisplayText	Manage consent and permission grant policies	Manage consent and permission grant policies
Description	Allows the app to manage policies related to consent and permission grants for applications, without a signed-in user.	Allows the app to manage policies related to consent and permission grants for applications, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.Recovery

Category	Application	Delegated
Identifier	795fc94d-3deb-4632-b1eb-e6d1a5f44918	1e7a2f4c-e602-4b1b-9547-304dd65c4cc2
DisplayText	Read and write your organization's recovery policy	Read and write your organization's recovery policy
Description	Allows the application to read and update the organization's recovery policy without a signed-in user.	Allows the application to read and update the organization's recovery policy on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.SecurityDefaults

Category	Application	Delegated
Identifier	1c6e93a6-28e2-4cbb-9f64-1a46a821124d	0b2a744c-2abf-4f1e-ad7e-17a087e2be99
DisplayText	Read and write your organization's security defaults policy	Read and write your organization's security defaults policy
Description	Allows the app to read and write your organization's security defaults policy, without a signed-in user.	Allows the app to read and write your organization's security defaults policy on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Policy.ReadWrite.TrustFramework

Category	Application	Delegated
Identifier	79a677f7-b79d-40d0-a36a-3e6f8688dd7a	cefba324-1a70-4a6e-9c1d-fd670b7ae392
DisplayText	Read and write your organization's trust framework policies	Read and write your organization's trust framework policies
Description	Allows the app to read and write your organization's trust framework policies without a signed in user.	Allows the app to read and write your organization's trust framework policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

POP.AccessAsUser.All

Category	Application	Delegated
Identifier	-	d7b7f2d9-0f45-4ea1-9d42-e50810c06991
DisplayText	-	Read and write access to mailboxes via POP.
Description	-	Allows the app to have the same access to mailboxes as the signed-in user via POP protocol.
AdminConsentRequired	-	No

The *POP.AccessAsUser.All* delegated permission is available for consent in personal Microsoft accounts.

PreAuthorizationGrant.Read.All

Category	Application	Delegated
Identifier	66ae8ecc-328f-47f6-97ca-4d9e952df081	9c98cbde-410c-4719-9058-166504f17863
DisplayText	Read all preauthorization grants	Read all preauthorization grants
Description	Allows the app to read preauthorization grants for service	Allows the app to read preauthorization grants for service principals on behalf of

Category	Application	Delegated
	principals without a signed-in user.	the signed-in user.
AdminConsentRequired	Yes	Yes

Presence.Read

Category	Application	Delegated
Identifier	-	76bc735e-aecd-4a1d-8b4c-2b915deabb79
DisplayText	-	Read user's presence information
Description	-	Allows the app to read presence information on behalf of the signed-in user. Presence information includes activity, availability, status note, calendar out-of-office message, timezone and location.
AdminConsentRequired	-	No

Presence.Read.All

Category	Application	Delegated
Identifier	a70e0c2d-e793-494c-94c4-118fa0a67f42	9c7a330d-35b3-4aa1-963d-cb2b9f927841
DisplayText	Read presence information for all users	Read presence information of all users in your organization
Description	Allows the app to read presence information of all users in the directory without a signed-in user. Presence information includes activity, availability, status note, calendar out-of-office message, timezone and location.	Allows the app to read presence information of all users in the directory on behalf of the signed-in user. Presence information includes activity, availability, status note, calendar out-of-office message, timezone and location.
AdminConsentRequired	Yes	No

Presence.ReadWrite

Category	Application	Delegated
Identifier	-	8d3c54a7-cf58-4773-bf81-c0cd6ad522bb
DisplayText	-	Read and write a user's presence information
Description	-	Allows the app to read the presence information and write activity and availability on behalf of the signed-in user. Presence information includes activity, availability, status note, calendar out-of-office message, timezone and location.
AdminConsentRequired	-	No

Presence.ReadWrite.All

Category	Application	Delegated
Identifier	83cded22-8297-4ff6-a7fa-e97e9545a259	-
DisplayText	Read and write presence information for all users	-
Description	Allows the app to read all presence information and write activity and availability of all users in the directory without a signed-in user. Presence information includes activity, availability, status note, calendar out-of-office message, time zone and location.	-
AdminConsentRequired	Yes	-

PrintConnector.Read.All

Category	Application	Delegated
Identifier	-	d69c2d6d-4f72-4f99-a6b9-663e32f8cf68
DisplayText	-	Read print connectors
Description	-	Allows the application to read print connectors on behalf of the signed-in user.
AdminConsentRequired	-	Yes

PrintConnector.ReadWrite.All

Category	Application	Delegated
Identifier	-	79ef9967-7d59-4213-9c64-4b10687637d8
DisplayText	-	Read and write print connectors
Description	-	Allows the application to read and write print connectors on behalf of the signed-in user.
AdminConsentRequired	-	Yes

Printer.Create

Category	Application	Delegated
Identifier	-	90c30bed-6fd1-4279-bf39-714069619721
DisplayText	-	Register printers
Description	-	Allows the application to create (register) printers on behalf of the signed-in user.
AdminConsentRequired	-	Yes

Printer.FullControl.All

Category	Application	Delegated
Identifier	-	93dae4bd-43a1-4a23-9a1a-92957e1d9121
DisplayText	-	Register, read, update, and unregister printers
Description	-	Allows the application to create (register), read, update, and delete (unregister) printers on behalf of the signed-in user.
AdminConsentRequired	-	Yes

Printer.Read.All

Category	Application	Delegated
Identifier	9709bb33-4549-49d4-8ed9-a8f65e45bb0f	3a736c8a-018e-460a-b60c-863b2683e8bf

Category	Application	Delegated
DisplayText	Read printers	Read printers
Description	Allows the application to read printers without a signed-in user.	Allows the application to read printers on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Printer.ReadWrite.All

Category	Application	Delegated
Identifier	f5b3f73d-6247-44df-a74c-866173fddab0	89f66824-725f-4b8f-928e-e1c5258dc565
DisplayText	Read and update printers	Read and update printers
Description	Allows the application to read and update printers without a signed-in user. Does not allow creating (registering) or deleting (unregistering) printers.	Allows the application to read and update printers on behalf of the signed-in user. Does not allow creating (registering) or deleting (unregistering) printers.
AdminConsentRequired	Yes	Yes

PrinterShare.Read.All

Category	Application	Delegated
Identifier	-	ed11134d-2f3f-440d-a2e1-411efada2502
DisplayText	-	Read printer shares
Description	-	Allows the application to read printer shares on behalf of the signed-in user.
AdminConsentRequired	-	No

PrinterShare.ReadBasic.All

Category	Application	Delegated
Identifier	-	5fa075e9-b951-4165-947b-c63396ff0a37
DisplayText	-	Read basic information about printer shares
Description	-	Allows the application to read basic information about printer shares on behalf of the signed-in user. Does not allow reading access control information.
AdminConsentRequired	-	No

PrinterShare.ReadWrite.All

Category	Application	Delegated
Identifier	-	06ceea37-85e2-40d7-bec3-91337a46038f
DisplayText	-	Read and write printer shares
Description	-	Allows the application to read and update printer shares on behalf of the signed-in user.
AdminConsentRequired	-	Yes

PrintJob.Create

Category	Application	Delegated
Identifier	-	21f0d9c0-9f13-48b3-94e0-b6b231c7d320
DisplayText	-	Create print jobs
Description	-	Allows the application to create print jobs on behalf of the signed-in user and upload document content to print jobs that the signed-in user created.
AdminConsentRequired	-	No

In this to *PrintJob.Create*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintJob.Manage.All

Category	Application	Delegated
Identifier	58a52f47-9e36-4b17-9ebe-ce4ef7f3e6c8	-
DisplayText	Perform advanced operations on print jobs	-
Description	Allows the application to perform advanced operations like redirecting a print job to another printer without a signed-in user. Also allows the application to read and update the metadata of print jobs.	-
AdminConsentRequired	Yes	-

PrintJob.Read

Category	Application	Delegated
Identifier	-	248f5528-65c0-4c88-8326-876c7236df5e
DisplayText	-	Read user's print jobs
Description	-	Allows the application to read the metadata and document content of print jobs that the signed-in user created.
AdminConsentRequired	-	No

In this to *PrintJob.Read*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintJob.Read.All

Category	Application	Delegated
Identifier	ac6f956c-edea-44e4-bd06-64b1b4b9aec9	afdd6933-a0d8-40f7-bd1a-b5d778e8624b
DisplayText	Read print jobs	Read print jobs
Description	Allows the application to read the metadata and document content of print jobs without a signed-in user.	Allows the application to read the metadata and document content of print jobs on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

In this to *PrintJob.Read.All*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintJob.ReadBasic

Category	Application	Delegated
Identifier	-	6a71a747-280f-4670-9ca0-a9cbf882b274
DisplayText	-	Read basic information of user's print jobs
Description	-	Allows the application to read the metadata of print jobs that the signed-in user created. Does not allow access to print job document content.
AdminConsentRequired	-	No

In this to *PrintJob.ReadBasic*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintJob.ReadBasic.All

Category	Application	Delegated
Identifier	fbf67eee-e074-4ef7-b965-ab5ce1c1f689	04ce8d60-72ce-4867-85cf-6d82f36922f3
DisplayText	Read basic information for print jobs	Read basic information of print jobs
Description	Allows the application to read the metadata of print jobs without a signed-in user. Does not allow access to print job document content.	Allows the application to read the metadata of print jobs on behalf of the signed-in user. Does not allow access to print job document content.
AdminConsentRequired	Yes	Yes

In this to *PrintJob.ReadBasic.All*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintJob.ReadWrite

Category	Application	Delegated
Identifier	-	b81dd597-8abb-4b3f-a07a-820b0316ed04
DisplayText	-	Read and write user's print jobs
Description	-	Allows the application to read and update the metadata and document content of print jobs that the signed-in user created.
AdminConsentRequired	-	No

In this to *PrintJob.ReadWrite*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintJob.ReadWrite.All

Category	Application	Delegated
Identifier	5114b07b-2898-4de7-a541-53b0004e2e13	036b9544-e8c5-46ef-900a-0646cc42b271
DisplayText	Read and write print jobs	Read and write print jobs
Description	Allows the application to read and update the metadata and document content of print jobs without a signed-in user.	Allows the application to read and update the metadata and document content of print jobs on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

In this to *PrintJob.ReadWrite.All*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintJob.ReadWriteBasic

Category	Application	Delegated
Identifier	-	6f2d22f2-1cb6-412c-a17c-3336817eaa82
DisplayText	-	Read and write basic information of user's print jobs
Description	-	Allows the application to read and update the metadata of print jobs that the signed-in user created. Does not allow access to print job document content.

Category	Application	Delegated
AdminConsentRequired	-	No

In this to *PrintJob.ReadWriteBasic*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintJob.ReadWriteBasic.All

Category	Application	Delegated
Identifier	57878358-37f4-4d3a-8c20-4816e0d457b1	3a0db2f6-0d2a-4c19-971b-49109b19ad3d
DisplayText	Read and write basic information for print jobs	Read and write basic information of print jobs
Description	Allows the application to read and update the metadata of print jobs without a signed-in user. Does not allow access to print job document content.	Allows the application to read and update the metadata of print jobs on behalf of the signed-in user. Does not allow access to print job document content.
AdminConsentRequired	Yes	Yes

In this to *PrintJob.ReadWriteBasic.All*, the app requires at least the *Printer.Read.All* (or a more prviliged permission) because print jobs are stored within printers.

PrintSettings.Read.All

Category	Application	Delegated
Identifier	b5991872-94cf-4652-9765-29535087c6d8	490f32fd-d90f-4dd7-a601-ff6cdc1a3f6c
DisplayText	Read tenant-wide print settings	Read tenant-wide print settings
Description	Allows the application to read tenant-wide print settings without a signed-in user.	Allows the application to read tenant-wide print settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrintSettings.ReadWrite.All

Category	Application	Delegated
Identifier	-	9ccc526a-c51c-4e5c-a1fd-74726ef50b8f
DisplayText	-	Read and write tenant-wide print settings
Description	-	Allows the application to read and write tenant-wide print settings on behalf of the signed-in user.
AdminConsentRequired	-	Yes

PrintTaskDefinition.ReadWrite.All

Category	Application	Delegated
Identifier	456b71a7-0ee0-4588-9842-c123fcc8f664	-
DisplayText	Read, write and update print task definitions	-
Description	Allows the application to read and update print task definitions without a signed-in user.	-
AdminConsentRequired	Yes	-

PrivilegedAccess-CustomExt.Read.All

Category	Application	Delegated
Identifier	e7ebe2d9-6e26-487a-8286-191d623a6904	bc04fe80-7e6a-4154-8b8f-d1e3465613bf
DisplayText	Read Privileged Access (PIM) custom extensions	Read Privileged Access (PIM) custom extensions
Description	Allows the app to read Privileged Access (PIM) custom extensions for your organization, without a signed-in user.	Allows the app to read Privileged Access (PIM) custom extensions for your organization, without a signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAccess-CustomExt.ReadWrite.All

Category	Application	Delegated
Identifier	124325f3-0c46-4c57-a050-d6d1a82510f6	157efa76-20fd-4db4-876e-90c049322467
DisplayText	Read and write Privileged Access (PIM) custom extensions	Read and write Privileged Access (PIM) custom extensions
Description	Allows the app to read and write Privileged Access (PIM) custom extensions for your organization, without a signed-in user.	Allows the app to read and write Privileged Access (PIM) custom extensions for your organization, without a signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAccess.Read.AzureAD

Category	Application	Delegated
Identifier	4cdc2547-9148-4295-8d11-be0db1391d6b	b3a539c9-59cb-4ad5-825a-041ddbdc2bdb
DisplayText	Read privileged access to Azure AD roles	Read privileged access to Azure AD
Description	Allows the app to read time-based assignment and just-in-time elevation (including scheduled elevation) of Azure AD built-in and custom administrative roles in your organization, without a signed-in user.	Allows the app to read time-based assignment and just-in-time elevation (including scheduled elevation) of Azure AD built-in and custom administrative roles, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAccess.Read.AzureADGroup

Category	Application	Delegated
Identifier	01e37dc9-c035-40bd-b438-b2879c4870a6	d329c81c-20ad-4772-abf9-3f6fdb7e5988
DisplayText	Read privileged access to Azure AD groups	Read privileged access to Azure AD groups

Category	Application	Delegated
Description	Allows the app to read time-based assignment and just-in-time elevation (including scheduled elevation) of Azure AD groups in your organization, without a signed-in user.	Allows the app to read time-based assignment and just-in-time elevation (including scheduled elevation) of Azure AD groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAccess.Read.AzureResources

Category	Application	Delegated
Identifier	5df6fe86-1be0-44eb-b916-7bd443a71236	1d89d70c-dcac-4248-b214-903c457af83a
DisplayText	Read privileged access to Azure resources	Read privileged access to Azure resources
Description	Allows the app to read time-based assignment and just-in-time elevation of user privileges to audit Azure resources in your organization, without a signed-in user.	Allows the app to read time-based assignment and just-in-time elevation of Azure resources (like your subscriptions, resource groups, storage, compute) on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAccess.ReadWrite.AzureAD

Category	Application	Delegated
Identifier	854d9ab1-6657-4ec8-be45-823027bcd009	3c3c74f5-cdaa-4a97-b7e0-4e788bfcfb37
DisplayText	Read and write privileged access to Azure AD roles	Read and write privileged access to Azure AD
Description	Allows the app to request and manage time-based assignment and just-in-time elevation (including scheduled elevation) of Azure AD built-in and custom administrative roles in your organization, without a signed-in user.	Allows the app to request and manage just in time elevation (including scheduled elevation) of users to Azure AD built-in administrative roles, on behalf of signed-in users.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

PrivilegedAccess.ReadWrite.AzureADGroup

Category	Application	Delegated
Identifier	2f6817f8-7b12-4f0f-bc18-eeaf60705a9e	32531c59-1f32-461f-b8df-6f8a3b89f73b
DisplayText	Read and write privileged access to Azure AD groups	Read and write privileged access to Azure AD groups
Description	Allows the app to request and manage time-based assignment and just-in-time elevation (including scheduled elevation) of Azure AD groups in your organization, without a signed-in user.	Allows the app to request and manage time-based assignment and just-in-time elevation (including scheduled elevation) of Azure AD groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAccess.ReadWrite.AzureResources

Category	Application	Delegated
Identifier	6f9d5abc-2db6-400b-a267-7de22a40fb87	a84a9652-ffd3-496e-a991-22ba5529156a
DisplayText	Read and write privileged access to Azure resources	Read and write privileged access to Azure resources
Description	Allows the app to request and manage time-based assignment and just-in-time elevation of Azure resources (like your subscriptions, resource groups, storage, compute) in your organization, without a signed-in user.	Allows the app to request and manage time-based assignment and just-in-time elevation of user privileges to manage Azure resources (like subscriptions, resource groups, storage, compute) on behalf of the signed-in users.
AdminConsentRequired	Yes	Yes

PrivilegedAssignmentSchedule.Read.AzureADGroup

Category	Application	Delegated
Identifier	cd4161cb-f098-48f8-a884-1eda9a42434c	02a32cc4-7ab5-4b58-879a-0586e0f7c495
DisplayText	Read assignment schedules for access to Azure AD groups	Read assignment schedules for access to Azure AD groups
Description	Allows the app to read time-based assignment schedules for access to Azure AD groups, without a signed-in user.	Allows the app to read time-based assignment schedules for access to Azure AD groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAssignmentSchedule.Read.EntraAppRole

Category	Application	Delegated
Identifier	3a728f2e-df1d-4294-9899-86f601fae70a	d5767d44-e1c1-4fc7-8fb1-7daa58df022a
DisplayText	Read assignment schedules for app permission grants and app role assignments	Read assignment schedules for app permission grants and app role assignments
Description	Allows the app to read time-based assignment schedules for permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, without a signed-in user.	Allows the app to read time-based assignment schedules for permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAssignmentSchedule.ReadWrite.AzureADGroup

Category	Application	Delegated
Identifier	41202f2c-f7ab-45be-b001-85c9728b9d69	06dbc45d-6708-4ef0-a797-f797ee68bf4b
DisplayText	Read, create, and delete assignment schedules for access to Azure AD groups	Read, create, and delete assignment schedules for access to Azure AD groups

Category	Application	Delegated
Description	Allows the app to read, create, and delete time-based assignment schedules for access to Azure AD groups, without a signed-in user.	Allows the app to read, create, and delete time-based assignment schedules for access to Azure AD groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAssignmentSchedule.ReadWrite.EntraAppRole

Category	Application	Delegated
Identifier	81adad77-a25a-489d-ac43-321115620139	e07122a7-d275-4a27-a2f5-eb62349edae0
DisplayText	Read, create, and delete assignment schedules for app permission grants and app role assignments	Read, create, and delete assignment schedules for app permission grants and app role assignments
Description	Allows the app to read, create, and delete time-based assignment schedules for permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, without a signed-in user.	Allows the app to read, create, and delete time-based assignment schedules for permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedAssignmentSchedule.Remove.AzureADGroup

Category	Application	Delegated
Identifier	55d1104b-3821-413d-b3ca-e2393d333cd3	ca5fe595-68ff-4dfd-907d-4509501a0e49
DisplayText	Delete assignment schedules for access to Azure AD groups	Delete assignment schedules for access to Azure AD groups
Description	Delete time-based assignment schedules for access to Azure AD groups, without a signed-in user.	Allows the app to delete time-based assignment schedules for access to Azure AD groups, on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

PrivilegedEligibilitySchedule.Read.AzureADGroup

Category	Application	Delegated
Identifier	edb419d6-7edc-42a3-9345-509bfdf5d87c	8f44f93d-ecef-46ae-a9bf-338508d44d6b
DisplayText	Read eligibility schedules for access to Azure AD groups	Read eligibility schedules for access to Azure AD groups
Description	Allows the app to read time-based eligibility schedules for access to Azure AD groups, without a signed-in user.	Allows the app to read time-based eligibility schedules for access to Azure AD groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedEligibilitySchedule.Read.EntraAppRole

Category	Application	Delegated
Identifier	d2ab45a0-ed46-4f7f-806a-0f1146144d5a	9b9eb231-5483-4f3c-89e9-9d5048dafe9d
DisplayText	Read eligibility schedules for app permission grants and app role assignments	Read eligibility schedules for app permission grants and app role assignments
Description	Allows the app to read time-based eligibility schedules for permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, without a signed-in user.	Allows the app to read time-based eligibility schedules for permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedEligibilitySchedule.ReadWrite.AzureADGroup

Category	Application	Delegated
Identifier	618b6020-bca8-4de6-99f6-ef445fa4d857	ba974594-d163-484e-ba39-c330d5897667
DisplayText	Read, create, and delete eligibility schedules for access to Azure AD groups	Read, create, and delete eligibility schedules for access to Azure AD groups
Description	Allows the app to read, create, and delete time-based eligibility schedules for access to Azure AD groups, without a signed-in user.	Allows the app to read, create, and delete time-based eligibility schedules for access to Azure AD groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedEligibilitySchedule.ReadWrite.EntraAppRole

Category	Application	Delegated
Identifier	7f4c39f1-1aa7-44b7-ab05-38df2609c37a	f7ff1cb0-e255-4bb3-b24a-6708c60c5418
DisplayText	Read, create, and delete eligibility schedules for app permission grants and app role assignments	Read, create, and delete eligibility schedules for app permission grants and app role assignments
Description	Allows the app to read, create, and delete time-based eligibility schedules for permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, without a signed-in user.	Allows the app to read, create, and delete time-based eligibility schedules for permission grants for application permissions to any API (including Microsoft Graph) and application assignments for any app, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PrivilegedEligibilitySchedule.Remove.AzureADGroup

Category	Application	Delegated
Identifier	55745561-7572-4314-a737-a2c2a1b0dd2e	c5ea9ab4-9b41-4c09-a400-53e652fb5096
DisplayText	Delete eligibility schedules for	Delete eligibility schedules for access to

Category	Application	Delegated
	access to Azure AD groups	Azure AD groups
Description	Delete time-based eligibility schedules for access to Azure AD groups, without a signed-in user.	Allows the app to delete time-based eligibility schedules for access to Azure AD groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

profile

Category	Application	Delegated
Identifier	-	14dad69e-099b-42c9-810b-d002981feec1
DisplayText	-	View users' basic profile
Description	-	Allows the app to see your users' basic profile (e.g., name, picture, user name, email address)
AdminConsentRequired	-	No

The *profile* delegated permission is available for consent in personal Microsoft accounts.

profile is an OpenID Connect (OIDC) scope.

You can use the OIDC scopes to specify artifacts that you want returned in Azure AD authorization and token requests. They are supported differently by the Azure AD v1.0 and v2.0 endpoints.

With the Azure AD v1.0 endpoint, only the *openid* scope is used. You specify it in the *scope* parameter in an authorization request to return an ID token when you use the OpenID Connect protocol to sign in a user to your app. For more information, see Authorize access to web applications using OpenID Connect and Azure Active Directory. To successfully return an ID token, you must also make sure that the *User.Read* permission is configured when you register your app.

With the Azure AD v2.0 endpoint, you specify the *offline_access* scope in the *scope* parameter to explicitly request a refresh token when using the OAuth 2.0 or OpenID Connect protocols. With OpenID Connect, you specify the *openid* scope to request an ID token. You can also specify the *email* scope, *profile* scope, or both to return additional claims in the ID token. You do not need to

specify the *User.Read* permission to return an ID token with the v2.0 endpoint. For more information, see OpenID Connect scopes.

The Microsoft Authentication Library (MSAL) currently specifies *offline_access*, *openid*, *profile*, and *email* by default in authorization and token requests. This means that, for the default case, if you specify these scopes explicitly, Azure AD may return an error.

ProfilePhoto.Read.All

Category	Application	Delegated
Identifier	e24d31aa-e1ab-4c80-85fe-23018690335d	469cd065-729e-4dee-b1fa-d92e0fab6310
DisplayText	Read profile photo of a user or group	Read profile photo of a user or group
Description	Allows the app to read all profile photos of users and groups, without a signed-in user	Allows the app to read all profile photos of users and groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ProfilePhoto.ReadWrite.All

Category	Application	Delegated
Identifier	27baa7f6-5dfb-4ba8-b1d3-1e812c143013	f5b24df7-511e-48bb-ae88-643f023b55e1
DisplayText	Read and write profile photo of a user or group	Read and write profile photo of a user or group
Description	Allows the app to read and write all profile photos of users and groups, without a signed-in user	Allows the app to read and write all profile photos of users and groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ProgramControl.Read.All

Category	Application	Delegated
Identifier	eedb7fdd-7539-4345-a38b-4839e4a84cbd	c492a2e1-2f8f-4caa-b076-99bbf6e40fe4
DisplayText	Read all programs	Read all programs that user can access
Description	Allows the app to read programs and program controls in the organization, without a signed-in user.	Allows the app to read programs and program controls that the signed-in user has access to in the organization.
AdminConsentRequired	Yes	Yes

ProgramControl.ReadWrite.All

Category	Application	Delegated
Identifier	60a901ed-09f7-4aa5-a16e-7dd3d6f9de36	50fd364f-9d93-4ae1-b170-300e87cccf84
DisplayText	Manage all programs	Manage all programs that user can access
Description	Allows the app to read, update, delete and perform actions on programs and program controls in the organization, without a signed-in user.	Allows the app to read, update, delete and perform actions on programs and program controls that the signed-in user has access to in the organization.
AdminConsentRequired	Yes	Yes

ProtectionScopes.Compute.All

Category	Application	Delegated
Identifier	e5a76501-dbb0-492c-ab55-5d09e8837263	98f5a27a-539a-48bc-a597-f78e9e1e76bf
DisplayText	Compute Purview policies at tenant scope	Compute Purview policies at tenant scope
Description	Allows the app to identify Purview data protection, compliance and governance policy scopes defined for all users across tenant.	Allows the app to identify Purview data protection, compliance and governance policy scopes defined for all users across tenant.
AdminConsentRequired	Yes	Yes

ProtectionScopes.Compute.User

Category	Application	Delegated
Identifier	fe696d63-5e1f-4515-8232-cccc316903c6	4fc04d16-a9fc-4c5e-8da4-79b6c33638a4
DisplayText	Compute Purview policies for an individual user	Compute Purview policies for an individual user
Description	Allows the app to identify Purview data protection, compliance and governance policy scopes defined for an individual user.	Allows the app to identify Purview data protection, compliance and governance policy scopes defined for an individual user.
AdminConsentRequired	Yes	Yes

ProvisioningLog.Read.All

Category	Application	Delegated
Identifier	091937d3-3e38-47a1-8649-b2f99d3035f1	95aec97b-cf27-4a8d-a67d-42f60b5b38ef
DisplayText	Read all provisioning log data	Read provisioning log data
Description	Allows the app to read and query your provisioning log activities, without a signed-in user.	Allows the app to read and query your provisioning log activities, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PublicKeyInfrastructure.Read.All

Category	Application	Delegated
Identifier	214fda0c-514a-4650-b037-b562b1a66124	04a4b2a2-3f26-4fc8-87ee-9c46e68db175
DisplayText	Read all certificate based authentication configurations	Read certificate based authentication configurations
Description	Allows the application to read certificate-based authentication	Allows the application to read certificate-based authentication

Category	Application	Delegated
	configuration such as all public key infrastructures (PKI) and certificate authorities (CA) configured for the organization, without a signed-in user.	configuration such as all public key infrastructures (PKI) and certificate authorities (CA) configured for the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PublicKeyInfrastructure.ReadWrite.All

Category	Application	Delegated
Identifier	a2b63618-5350-462d-b1b3-ba6eb3684e26	3591b7f3-dba8-4bad-b667-7a64bd4f2b83
DisplayText	Read and write all certificate based authentication configurations	Read and write certificate based authentication configurations
Description	Allows the application to read and write certificate-based authentication configuration such as all public key infrastructures (PKI) and certificate authorities (CA) configured for the organization, without a signed-in user.	Allows the application to read and write certificate-based authentication configuration such as all public key infrastructures (PKI) and certificate authorities (CA) configured for the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PullPrintPrinter.FullControl.All

Category	Application	Delegated
Identifier	-	ef6b83cd-f762-47ff-97d7-6f6f2d0486ea
DisplayText	-	Create, read, update and delete pull-print printers and manage member printers
Description	-	Allows the application to create, read, update and delete pull-print printers and manage member printers on behalf of the signed-in user.
AdminConsentRequired	-	Yes

PullPrintPrinter.Read.All

Category	Application	Delegated
Identifier	f369d3b8-fe98-4772-85e1-b23e7cf41982	deac7994-79bc-44c9-8828-01c1a9a96618
DisplayText	Read pull-print printers	Read pull-print printers
Description	Allows the application to read pull-print printers without a signed-in user.	Allows the application to read pull-print printers on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

PullPrintPrinter.ReadWrite.All

Category	Application	Delegated
Identifier	-	c628c397-5d7f-4c93-ae0f-4680282fd6d5
DisplayText	-	Read and update pull-print printers
Description	-	Allows the application to read and update pull-print printers on behalf of the signed-in user. Does not allow creating or deleting pull-print printers or managing member printers.
AdminConsentRequired	-	Yes

QnA.Read.All

Category	Application	Delegated
Identifier	ee49e170-1dd1-4030-b44c-61ad6e98f743	f73fa04f-b9a5-4df9-8843-993ce928925e
DisplayText	Read all Question and Answers	Read all Questions and Answers that the user can access.
Description	Allows an app to read all question and answers, without a signed-in user.	Allows an app to read all question and answer sets that the signed-in user can access.
AdminConsentRequired	Yes	No

RealTimeActivityFeed.Read.All

Category	Application	Delegated
Identifier	abafe00f-ea87-4c63-b8a8-0e7bb0a88144	db5d5bae-0c9e-444e-9390-8a5fea98c253
DisplayText	Access real-time enriched data in a meeting as an app	Access real-time enriched data in a meeting
Description	Allows the app to get direct access to real-time enriched data in a meeting, without a signed-in user.	Allows the app to get direct access to real-time enriched data in a meeting, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RecordsManagement.Read.All

Category	Application	Delegated
Identifier	ac3a2b8e-03a3-4da9-9ce0-cbe28bf1accd	07f995eb-fc67-4522-ad66-2b8ca8ea3efd
DisplayText	Read Records Management configuration, labels and policies	Read Records Management configuration, labels, and policies
Description	Allows the application to read any data from Records Management, such as configuration, labels, and policies without the signed in user.	Allows the application to read any data from Records Management, such as configuration, labels, and policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RecordsManagement.ReadWrite.All

Category	Application	Delegated
Identifier	eb158f57-df43-4751-8b21-b8932adb3d34	f2833d75-a4e6-40ab-86d4-6dfe73c97605
DisplayText	Read and write Records Management configuration, labels and policies	Read and write Records Management configuration, labels, and policies
Description	Allow the application to create, update and delete any data from Records Management, such as configuration,	Allow the application to create, update and delete any data from Records Management, such as configuration,

Category	Application	Delegated
	labels, and policies without the signed in user.	labels, and policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RemoteTenantGroups.Read.All

Category	Application	Delegated
Identifier	faa08cc0-0dcd-4ea9-9f9c-8b16f842b36e	d207fff0-6e36-4360-a23b-495e23b60385
DisplayText	Read RemoteTenantGroups information	Read RemoteTenantGroups information
Description	Allows the app to read available properties on remoteTenantGroups, without a signed-in user.	Allows the app to read available properties of remoteTenantGroups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Reports.Read.All

Category	Application	Delegated
Identifier	230c1aed-a721-4c5d-9cb4-a90514e508ef	02e97553-ed7b-43d0-ab3c-f8bace0d040c
DisplayText	Read all usage reports	Read all usage reports
Description	Allows an app to read all service usage reports without a signed-in user. Services that provide usage reports include Office 365 and Azure Active Directory.	Allows an app to read all service usage reports on behalf of the signed-in user. Services that provide usage reports include Office 365 and Azure Active Directory.
AdminConsentRequired	Yes	Yes

ReportSettings.Read.All

Category	Application	Delegated
Identifier	ee353f83-55ef-4b78-82da-555bfa2b4b95	84fac5f4-33a9-4100-aa38-a20c6d29e5e7
DisplayText	Read all admin report settings	Read admin report settings
Description	Allows the app to read all admin report settings, such as whether to display concealed information in reports, without a signed-in user.	Allows the app to read admin report settings, such as whether to display concealed information in reports, on behalf of the signed-in user
AdminConsentRequired	Yes	Yes

ReportSettings.ReadWrite.All

Category	Application	Delegated
Identifier	2a60023f-3219-47ad-baa4-40e17cd02a1d	b955410e-7715-4a88-a940-dfd551018df3
DisplayText	Read and write all admin report settings	Read and write admin report settings
Description	Allows the app to read and update all admin report settings, such as whether to display concealed information in reports, without a signed-in user.	Allows the app to read and update admin report settings, such as whether to display concealed information in reports, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ResourceSpecificPermissionGrant.ReadForChat

Category	Application	Delegated
Identifier	-	cb530fca-534b-4e72-aa74-bca7e8bbd06f
DisplayText	-	Read resource specific permissions granted on a chat
Description	-	Allows the app to read the resource specific permissions granted on the chat, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ResourceSpecificPermissionGrant.ReadForChat.All

Category	Application	Delegated
Identifier	2ff643d8-43e4-4a9b-88c1-86cb4a4b4c2f	-
DisplayText	Read resource specific permissions granted on a chat	-
Description	Allows the app to read the resource specific permissions granted on the chat without a signed-in user.	-
AdminConsentRequired	Yes	-

ResourceSpecificPermissionGrant.ReadForTeam

Category	Application	Delegated
Identifier	-	eafad40c-bf7a-415a-b7f8-acdf5706b58f
DisplayText	-	Read resource specific permissions granted on a team
Description	-	Allows the app to read the resource specific permissions granted on the team, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ResourceSpecificPermissionGrant.ReadForTeam.All

Category	Application	Delegated
Identifier	ad4600ae-d900-42cb-a9a2-2415d05593d0	-
DisplayText	Read resource specific permissions granted on a team	-
Description	Allows the app to read the resource specific permissions granted on the team without a signed-in user.	-
AdminConsentRequired	Yes	-

ResourceSpecificPermissionGrant.ReadForUser

Category	Application	Delegated
Identifier	-	f1d91a8f-88e7-4774-8401-b668d5bca0c5
DisplayText	-	Read resource specific permissions granted on a user account
Description	-	Allows the app to read the resource specific permissions granted on a user account, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ResourceSpecificPermissionGrant.ReadForUser.All

Category	Application	Delegated
Identifier	acfca4d5-f49f-40ed-9648-84068b474c73	-
DisplayText	Read all resource specific permissions granted on user accounts	-
Description	Allows the app to read all resource specific permissions granted on user accounts, without a signed-in user.	-
AdminConsentRequired	Yes	-

RiskPreventionProviders.Read.All

Category	Application	Delegated
Identifier	2a6baefd-edea-4ff6-b24e-bebcaa27a50d	e197c06f-ae7b-4398-b0a2-89f76ebca159
DisplayText	Read all identity risk prevention providers	Read all identity risk prevention providers
Description	Allows the app to read your organization's risk prevention providers, without a signed-in user.	Allows the app to read your organization's risk prevention providers, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RiskPreventionProviders.ReadWrite.All

Category	Application	Delegated
Identifier	7fc7225d-eb37-4c39-90f3-a33a57cf1081	2a7babba-9623-4109-bc9c-79728cf3bb4f
DisplayText	Read and write all identity risk prevention providers	Read and write all identity risk prevention providers
Description	Allows the app to read and write your organization's risk prevention providers, without a signed-in user.	Allows the app to read and write your organization's risk prevention providers, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RoleAssignmentSchedule.Read.Directory

Category	Application	Delegated
Identifier	d5fe8ce8-684c-4c83-a52c-46e882ce4be1	344a729c-0285-42c6-9014-f12b9b8d6129
DisplayText	Read all active role assignments and role schedules for your company's directory	Read all active role assignments for your company's directory
Description	Allows the app to read the active role-based access control (RBAC) assignments and schedules for your company's directory, without a signed-in user. This includes reading directory role templates, and directory roles.	Allows the app to read the active role-based access control (RBAC) assignments for your company's directory, on behalf of the signed-in user. This includes reading directory role templates, and directory roles.
AdminConsentRequired	Yes	Yes

RoleAssignmentSchedule.ReadWrite.Directory

Category	Application	Delegated
Identifier	dd199f4a-f148-40a4-a2ec-f0069cc799ec	8c026be3-8e26-4774-9372-8d5d6f21daff
DisplayText	Read, update, and delete all policies for privileged role assignments of your company's directory	Read, update, and delete all active role assignments for your company's directory

Category	Application	Delegated
Description	Allows the app to read, update, and delete policies for privileged role-based access control (RBAC) assignments of your company's directory, without a signed-in user.	Allows the app to read and manage the active role-based access control (RBAC) assignments for your company's directory, on behalf of the signed-in user. This includes managing active directory role membership, and reading directory role templates, directory roles and active memberships.
AdminConsentRequired	Yes	Yes

RoleAssignmentSchedule.Remove.Directory

Category	Application	Delegated
Identifier	d3495511-98b7-4df3-b317-4e35c19f6129	f71cd05c-3fdb-4568-aef2-e1cf62ee20d4
DisplayText	Delete all active role assignments of your company's directory	Delete all active role assignments for your company's directory
Description	Delete all active privileged role-based access control (RBAC) assignments of your company's directory, without a signed-in user.	Allows the app to delete the active role-based access control (RBAC) assignments for your company's directory, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RoleEligibilitySchedule.Read.Directory

Category	Application	Delegated
Identifier	ff278e11-4a33-4d0c-83d2-d01dc58929a5	eb0788c2-6d4e-4658-8c9e-c0fb8053f03d
DisplayText	Read all eligible role assignments and role schedules for your company's directory	Read all eligible role assignments for your company's directory
Description	Allows the app to read the eligible role-based access control (RBAC) assignments and schedules for your company's directory, without a signed-in	Allows the app to read the eligible role-based access control (RBAC) assignments for your company's directory, on behalf of the signed-in

Category	Application	Delegated
	user. This includes reading directory role templates, and directory roles.	user. This includes reading directory role templates, and directory roles.
AdminConsentRequired	Yes	Yes

RoleEligibilitySchedule.ReadWrite.Directory

Category	Application	Delegated
Identifier	fee28b28-e1f3-4841-818e-2704dc62245f	62ade113-f8e0-4bf9-a6ba-5acb31db32fd
DisplayText	Read, update, and delete all eligible role assignments and schedules for your company's directory	Read, update, and delete all eligible role assignments for your company's directory
Description	Allows the app to read and manage the eligible role-based access control (RBAC) assignments and schedules for your company's directory, without a signed-in user. This includes managing eligible directory role membership, and reading directory role templates, directory roles and eligible memberships.	Allows the app to read and manage the eligible role-based access control (RBAC) assignments for your company's directory, on behalf of the signed-in user. This includes managing eligible directory role membership, and reading directory role templates, directory roles and eligible memberships.
AdminConsentRequired	Yes	Yes

RoleEligibilitySchedule.Remove.Directory

Category	Application	Delegated
Identifier	79c7e69c-0d9f-4eff-97a8-49170a5a08ba	58ac4fa2-b484-4d6e-ba97-beee2a574220
DisplayText	Delete all eligible role assignments of your company's directory	Delete all eligible role assignments for your company's directory
Description	Delete all eligible privileged role-based access control (RBAC) assignments of your company's directory, without a signed-in user.	Allows the app to delete the eligible role-based access control (RBAC) assignments for your company's directory, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RoleManagement.Read.All

Category	Application	Delegated
Identifier	c7fbd983-d9aa-4fa7-84b8-17382c103bc4	48fec646-b2ba-4019-8681-8eb31435aded
DisplayText	Read role management data for all RBAC providers	Read role management data for all RBAC providers
Description	Allows the app to read role-based access control (RBAC) settings for all RBAC providers without a signed-in user. This includes reading role definitions and role assignments.	Allows the app to read the role-based access control (RBAC) settings for all RBAC providers, on behalf of the signed-in user. This includes reading role definitions and role assignments.
AdminConsentRequired	Yes	Yes

RoleManagement.Read.CloudPC

Category	Application	Delegated
Identifier	031a549a-bb80-49b6-8032-2068448c6a3c	9619b88a-8a25-48a7-9571-d23be0337a79
DisplayText	Read Cloud PC RBAC settings	Read Cloud PC RBAC settings
Description	Allows the app to read the Cloud PC role-based access control (RBAC) settings, without a signed-in user.	Allows the app to read the Cloud PC role-based access control (RBAC) settings, on behalf of the signed-in user. This includes reading Cloud PC role definitions and role assignments.
AdminConsentRequired	Yes	Yes

RoleManagement.Read.Defender

Category	Application	Delegated
Identifier	4d6e30d1-e64e-4ae7-bf9d-c706cc928cef	dd689728-6eb8-4deb-bd38-2924a935f3de
DisplayText	Read M365 Defender RBAC configuration	Read M365 Defender RBAC configuration

Category	Application	Delegated
Description	Allows the app to read the role-based access control (RBAC) settings for your company's directory, without a signed-in user.	Allows the app to read the role-based access control (RBAC) settings for your company's directory, on behalf of the signed-in user. This includes reading M365 Defender role definitions and role assignments.
AdminConsentRequired	Yes	Yes

RoleManagement.Read.Directory

Category	Application	Delegated
Identifier	483bed4a-2ad3-4361-a73b-c83ccdbdc53c	741c54c3-0c1e-44a1-818b-3f97ab4e8c83
DisplayText	Read all directory RBAC settings	Read directory RBAC settings
Description	Allows the app to read the role-based access control (RBAC) settings for your company's directory, without a signed-in user. This includes reading directory role templates, directory roles and memberships.	Allows the app to read the role-based access control (RBAC) settings for your company's directory, on behalf of the signed-in user. This includes reading directory role templates, directory roles and memberships.
AdminConsentRequired	Yes	Yes

RoleManagement.Read.Exchange

Category	Application	Delegated
Identifier	c769435f-f061-4d0b-8ff1-3d39870e5f85	3bc15058-7858-4141-b24f-ae43b4e80b52
DisplayText	Read Exchange Online RBAC configuration	Read Exchange Online RBAC configuration
Description	Allows the app to read the role-based access control (RBAC) configuration for your organization's Exchange Online service, without a signed-in user. This includes reading Exchange management role definitions, role groups, role group membership, role	Allows the app to read the role-based access control (RBAC) settings for your organization's Exchange Online service, on behalf of the signed-in user. This includes reading Exchange management role definitions, role groups, role group membership, role

Category	Application	Delegated
	assignments, management scopes, and role assignment policies.	assignments, management scopes, and role assignment policies.
AdminConsentRequired	Yes	Yes

RoleManagement.ReadWrite.CloudPC

Category	Application	Delegated
Identifier	274d0592-d1b6-44bd-af1d-26d259bcb43a	501d06f8-07b8-4f18-b5c6-c191a4af7a82
DisplayText	Read and write all Cloud PC RBAC settings	Read and write Cloud PC RBAC settings
Description	Allows the app to read and manage the Cloud PC role-based access control (RBAC) settings, without a signed-in user. This includes reading and managing Cloud PC role definitions and memberships.	Allows the app to read and manage the Cloud PC role-based access control (RBAC) settings, on behalf of the signed-in user. This includes reading and managing Cloud PC role definitions and role assignments.
AdminConsentRequired	Yes	Yes

RoleManagement.ReadWrite.Defender

Category	Application	Delegated
Identifier	8b7e8c0a-7e9d-4049-97ec-04b5e1bcaf05	d8914f8f-9f64-4bd1-b4d3-f5a701ed8457
DisplayText	Read M365 Defender RBAC configuration	Read M365 Defender RBAC configuration
Description	Allows the app to read the role-based access control (RBAC) settings for your company's directory, without a signed-in user.	Allows the app to read the role-based access control (RBAC) settings for your company's directory, on behalf of the signed-in user. This includes reading M365 Defender role definitions and role assignments.
AdminConsentRequired	Yes	Yes

RoleManagement.ReadWrite.Directory

Category	Application	Delegated
Identifier	9e3f62cf-ca93-4989-b6ce-bf83c28f9fe8	d01b97e9-cbc0-49fe-810a-750afd5527a3
DisplayText	Read and write all directory RBAC settings	Read and write directory RBAC settings
Description	Allows the app to read and manage the role-based access control (RBAC) settings for your company's directory, without a signed-in user. This includes instantiating directory roles and managing directory role membership, and reading directory role templates, directory roles and memberships.	Allows the app to read and manage the role-based access control (RBAC) settings for your company's directory, on behalf of the signed-in user. This includes instantiating directory roles and managing directory role membership, and reading directory role templates, directory roles and memberships.
AdminConsentRequired	Yes	Yes

> **⊗ Caution**
>
> Permissions that allow granting authorization, such as *RoleManagement.ReadWrite.Directory*, allow an application to grant additional privileges to itself, other applications, or any user. Use caution when granting any of these permissions.
>
> With the *RoleManagement.ReadWrite.Directory* permission an application can read and write `/directoryRoles` and `/roleManagement/directory/*`. This includes adding and removing members to and from Microsoft Entra roles, and working with PIM for Microsoft Entra roles APIs.

RoleManagement.ReadWrite.Exchange

Category	Application	Delegated
Identifier	025d3225-3f02-4882-b4c0-cd5b541a4e80	c1499fe0-52b1-4b22-bed2-7a244e0e879f
DisplayText	Read and write Exchange Online RBAC configuration	Read and write Exchange Online RBAC configuration

Category	Application	Delegated
Description	Allows the app to read and manage the role-based access control (RBAC) settings for your organization's Exchange Online service, without a signed-in user. This includes reading, creating, updating, and deleting Exchange management role definitions, role groups, role group membership, role assignments, management scopes, and role assignment policies.	Allows the app to read and manage the role-based access control (RBAC) settings for your organization's Exchange Online service, on behalf of the signed-in user. This includes reading, creating, updating, and deleting Exchange management role definitions, role groups, role group membership, role assignments, management scopes, and role assignment policies.
AdminConsentRequired	Yes	Yes

RoleManagementAlert.Read.Directory

Category	Application	Delegated
Identifier	ef31918f-2d50-4755-8943-b8638c0a077e	cce71173-f76d-446e-97ff-efb2d82e11b1
DisplayText	Read all alert data for your company's directory	Read all alert data for your company's directory
Description	Allows the app to read all role-based access control (RBAC) alerts for your company's directory, without a signed-in user. This includes reading alert statuses, alert definitions, alert configurations and incidents that lead to an alert.	Allows the app to read the role-based access control (RBAC) alerts for your company's directory, on behalf of the signed-in user. This includes reading alert statuses, alert definitions, alert configurations and incidents that lead to an alert.
AdminConsentRequired	Yes	Yes

RoleManagementAlert.ReadWrite.Directory

Category	Application	Delegated
Identifier	11059518-d6a6-4851-98ed-509268489c4a	435644c6-a5b1-40bf-8f52-fe8e5b53e19c
DisplayText	Read all alert data, configure alerts, and take actions on all alerts for your	Read all alert data, configure alerts, and take actions on all alerts for your

Category	Application	Delegated
	company's directory	company's directory
Description	Allows the app to read and manage all role-based access control (RBAC) alerts for your company's directory, without a signed-in user. This includes managing alert settings, initiating alert scans, dismissing alerts, remediating alert incidents, and reading alert statuses, alert definitions, alert configurations and incidents that lead to an alert.	Allows the app to read and manage the role-based access control (RBAC) alerts for your company's directory, on behalf of the signed-in user. This includes managing alert settings, initiating alert scans, dismissing alerts, remediating alert incidents, and reading alert statuses, alert definitions, alert configurations and incidents that lead to an alert.
AdminConsentRequired	Yes	Yes

RoleManagementPolicy.Read.AzureADGroup

Category	Application	Delegated
Identifier	69e67828-780e-47fd-b28c-7b27d14864e6	7e26fdff-9cb1-4e56-bede-211fe0e420e8
DisplayText	Read all policies in PIM for Groups	Read all policies in PIM for Groups
Description	Allows the app to read policies in Privileged Identity Management for Groups, without a signed-in user.	Allows the app to read policies in Privileged Identity Management for Groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RoleManagementPolicy.Read.Directory

Category	Application	Delegated
Identifier	fdc4c997-9942-4479-bfcb-75a36d1138df	3de2cdbe-0ff5-47d5-bdee-7f45b4749ead
DisplayText	Read all policies for privileged role assignments of your company's directory	Read all policies for privileged role assignments of your company's directory
Description	Allows the app to read policies for privileged role-based access control	Allows the app to read policies for privileged role-based access control (RBAC) assignments of your company's

Category	Application	Delegated
	(RBAC) assignments of your company's directory, without a signed-in user.	directory, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RoleManagementPolicy.Read.EntraAppRole

Category	Application	Delegated
Identifier	3d201a4e-90f1-420d-bd4f-3beec28a46b9	8b3ffd3b-178e-4aae-be39-ba87585eddb2
DisplayText	Read all policies in PIM for App Roles	Read all policies in PIM for App Roles
Description	Allows the app to read policies in Privileged Identity Management for App Roles, without a signed-in user.	Allows the app to read policies in Privileged Identity Management for App Roles, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RoleManagementPolicy.ReadWrite.AzureADGroup

Category	Application	Delegated
Identifier	b38dcc4d-a239-4ed6-aa84-6c65b284f97c	0da165c7-3f15-4236-b733-c0b0f6abe41d
DisplayText	Read, update, and delete all policies in PIM for Groups	Read, update, and delete all policies in PIM for Groups
Description	Allows the app to read, update, and delete policies in Privileged Identity Management for Groups, without a signed-in user.	Allows the app to read, update, and delete policies in Privileged Identity Management for Groups, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RoleManagementPolicy.ReadWrite.Directory

Category	Application	Delegated
Identifier	31e08e0a-d3f7-4ca2-ac39-7343fb83e8ad	1ff1be21-34eb-448c-9ac9-ce1f506b2a68
DisplayText	Read, update, and delete all policies for privileged role assignments of your company's directory	Read, update, and delete all policies for privileged role assignments of your company's directory
Description	Allows the app to read, update, and delete policies for privileged role-based access control (RBAC) assignments of your company's directory, without a signed-in user.	Allows the app to read, update, and delete policies for privileged role-based access control (RBAC) assignments of your company's directory, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

RoleManagementPolicy.ReadWrite.EntraAppRole

Category	Application	Delegated
Identifier	ec563bdb-80dc-47c0-81d3-bff47cc6ac06	652ec839-e4ac-4eb5-b545-ecc90eeceb2d
DisplayText	Manage all policies in PIM for App Roles	Manage all policies in PIM for App Roles
Description	Allows the app to manage policies in Privileged Identity Management for App Roles, without a signed-in user.	Allows the app to manage policies in Privileged Identity Management for App Roles, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Schedule-WorkingTime.ReadWrite.All

Category	Application	Delegated
Identifier	0b21c159-dbf4-4dbb-a6f6-490e412c716e	-
DisplayText	Trigger working time policies and read the working time status	-
Description	Allows the app to trigger the working time policies and read the working time status for other users in your organization, without a signed-in user.	-
AdminConsentRequired	Yes	-

Schedule.Read.All

Category	Application	Delegated
Identifier	7b2ebf90-d836-437f-b90d-7b62722c4456	fccf6dd8-5706-49fa-811f-69e2e1b585d0
DisplayText	Read all schedule items	Read user schedule items
Description	Allows the app to read all schedules, schedule groups, shifts and associated entities in the Teams or Shifts application without a signed-in user.	Allows the app to read schedule, schedule groups, shifts and associated entities in the Teams or Shifts application on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Schedule.ReadWrite.All

Category	Application	Delegated
Identifier	b7760610-0545-4e8a-9ec3-cce9e63db01c	63f27281-c9d9-4f29-94dd-6942f7f1feb0
DisplayText	Read and write all schedule items	Read and write user schedule items
Description	Allows the app to manage all schedules, schedule groups, shifts and associated entities in the Teams or Shifts application without a signed-in user.	Allows the app to manage schedule, schedule groups, shifts and associated entities in the Teams or Shifts application on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SchedulePermissions.ReadWrite.All

Category	Application	Delegated
Identifier	7239b71d-b402-4150-b13d-78ecfe8df441	07919803-6073-4cd8-bc55-28077db0ee10
DisplayText	Read/Write schedule permissions for a role	Read/Write schedule permissions for a role.

Category	Application	Delegated
Description	Allows the app to read/write schedule permissions for a specific role in Shifts application without a signed-in user.	Allows the app to read/write schedule permissions for a specific role in Shifts application on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SearchConfiguration.Read.All

Category	Application	Delegated
Identifier	ada977a5-b8b1-493b-9a91-66c206d76ecf	7d307522-aa38-4cd0-bd60-90c6f0ac50bd
DisplayText	Read your organization's search configuration	Read your organization's search configuration
Description	Allows the app to read search configurations, without a signed-in user.	Allows the app to read search configuration, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SearchConfiguration.ReadWrite.All

Category	Application	Delegated
Identifier	0e778b85-fefa-466d-9eec-750569d92122	b1a7d408-cab0-47d2-a2a5-a74a3733600d
DisplayText	Read and write your organization's search configuration	Read and write your organization's search configuration
Description	Allows the app to read and write search configurations, without a signed-in user.	Allows the app to read and write search configuration, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityActions.Read.All

Category	Application	Delegated
Identifier	5e0edab9-c148-49d0-b423-ac253e121825	1638cddf-07a4-4de2-8645-69c96cacad73
DisplayText	Read your organization's security actions	Read your organization's security actions
Description	Allows the app to read security actions, without a signed-in user.	Allows the app to read security actions, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityActions.ReadWrite.All

Category	Application	Delegated
Identifier	f2bf083f-0179-402a-bedb-b2784de8a49b	dc38509c-b87d-4da0-bd92-6bec988bac4a
DisplayText	Read and update your organization's security actions	Read and update your organization's security actions
Description	Allows the app to read or update security actions, without a signed-in user.	Allows the app to read or update security actions, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityAlert.Create.All

Category	Application	Delegated
Identifier	06870c4c-7370-4a2a-ad10-239a337af816	7417b8c6-a088-4c4c-99c7-bca9ab3eb9ba
DisplayText	Create security alerts	Create security alerts
Description	Allows the app to create security alerts, without a signed-in user.	Allows the app to create security alerts, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityAlert.Read.All

Category	Application	Delegated
Identifier	472e4a4d-bb4a-4026-98d1-0b0d74cb74a5	bc257fb8-46b4-4b15-8713-01e91bfbe4ea
DisplayText	Read all security alerts	Read all security alerts
Description	Allows the app to read all security alerts, without a signed-in user.	Allows the app to read all security alerts, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityAlert.ReadWrite.All

Category	Application	Delegated
Identifier	ed4fca05-be46-441f-9803-1873825f8fdb	471f2a7f-2a42-4d45-a2bf-594d0838070d
DisplayText	Read and write to all security alerts	Read and write to all security alerts
Description	Allows the app to read and write to all security alerts, without a signed-in user.	Allows the app to read and write to all security alerts, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityAnalyzedMessage.Read.All

Category	Application	Delegated
Identifier	b48f7ac2-044d-4281-b02f-75db744d6f5f	53e6783e-b127-4a35-ab3a-6a52d80a9077
DisplayText	Read metadata and detection details for all emails in your organization	Read metadata and detection details for emails in your organization
Description	Read email metadata and security detection details, without a signed-in user.	Read email metadata and security detection details on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

SecurityAnalyzedMessage.ReadWrite.All

Category	Application	Delegated
Identifier	04c55753-2244-4c25-87fc-704ab82a4f69	48eb8c83-6e58-46e7-a6d3-8805822f5940
DisplayText	Read metadata, detection details, and execute remediation actions on all emails in your organization	Read metadata, detection details, and execute remediation actions on emails in your organization
Description	Read email metadata and security detection details, and execute remediation actions like deleting an email, without a signed-in user.	Read email metadata, security detection details, and execute remediation actions like deleting an email, on behalf of the signed in user.
AdminConsentRequired	Yes	Yes

SecurityCopilotWorkspaces.Read.All

Category	Application	Delegated
Identifier	-	84499c31-ac2e-44d3-a0cf-a6c386d4dfe8
DisplayText	-	Read all Security Copilot resources for the signed-in user
Description	-	Allows the app to read all Security Copilot signed-in user's resources on behalf of the signed-in user
AdminConsentRequired	-	No

SecurityCopilotWorkspaces.ReadWrite.All

Category	Application	Delegated
Identifier	-	206291b0-2167-47a7-a640-6cdc1df710ba
DisplayText	-	Read and write individually owned Security Copilot resources of the signed-in user
Description	-	Allows the app to read and write Security Copilot resources owned by the signed-in user on their behalf.
AdminConsentRequired	-	No

SecurityEvents.Read.All

Category	Application	Delegated
Identifier	bf394140-e372-4bf9-a898-299cfc7564e5	64733abd-851e-478a-bffb-e47a14b18235
DisplayText	Read your organization's security events	Read your organization's security events
Description	Allows the app to read your organization's security events without a signed-in user.	Allows the app to read your organization's security events on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityEvents.ReadWrite.All

Category	Application	Delegated
Identifier	d903a879-88e0-4c09-b0c9-82f6a1333f84	6aedf524-7e1c-45a7-bd76-ded8cab8d0fc
DisplayText	Read and update your organization's security events	Read and update your organization's security events
Description	Allows the app to read your organization's security events without a signed-in user. Also allows the app to update editable properties in security events.	Allows the app to read your organization's security events on behalf of the signed-in user. Also allows the app to update editable properties in security events on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityIdentitiesAccount.Read.All

Category	Application	Delegated
Identifier	c5bc96f5-b4a1-4cfc-8189-d5f0d772278f	3e9ed69a-a48e-473c-8b97-413016703a37
DisplayText	Read all identity security available identity accounts	Read identity security available identity accounts
Description	Allows the app to read all the identity security available identity accounts without a signed-in user.	Allows the app to read all the identity security available identity accounts

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

SecurityIdentitiesActions.ReadWrite.All

Category	Application	Delegated
Identifier	af2bf46f-7bf1-4be3-8bad-e17e279e8462	818229ce-20e4-47bd-92f4-bc94dbb37a56
DisplayText	Read and perform all identity security available actions	Read and perform identity security available actions
Description	Allows the app to read and write identity security available actions without a signed-in user.	Allows the app to read and write identity security available actions on behalf of the signed-in identity.
AdminConsentRequired	Yes	Yes

SecurityIdentitiesAutoConfig.Read.All

Category	Application	Delegated
Identifier	58971758-9844-4fe4-9fba-7e4ce7a659bf	8ff90903-1ecb-4f3a-b8b2-42120374ecd6
DisplayText	Read sensors window auditing configuration	Read sensors window auditing configuration
Description	Allows the app to read sensors window auditing configuration without a signed-in user	Allows the app to read the sensors window auditing configuration of the signed in user
AdminConsentRequired	Yes	Yes

SecurityIdentitiesAutoConfig.ReadWrite.All

Category	Application	Delegated
Identifier	4f1f0deb-08d1-4ffb-8cca-21dfc362b7c0	b810fdb4-8733-43bd-9b37-fddb7215c69f

Category	Application	Delegated
DisplayText	Read and write sensors window auditing configuration	Read and write sensors window auditing configuration
Description	Allows the app to read and write sensors window auditing configuration without a signed-in user	Allows the app to read and write the sensors window auditing configuration of the signed in user
AdminConsentRequired	Yes	Yes

SecurityIdentitiesHealth.Read.All

Category	Application	Delegated
Identifier	f8dcd971-5d83-4e1e-aa95-ef44611ad351	a0d0da43-a6df-4416-b63d-99c79991aae8
DisplayText	Read all identity security health issues	Read identity security health issues
Description	Allows the app to read all the identity security health issues without a signed-in user.	Allows the app to read all the identity security health issues of signed user
AdminConsentRequired	Yes	Yes

SecurityIdentitiesHealth.ReadWrite.All

Category	Application	Delegated
Identifier	ab03ddd5-7ae4-4f2e-8af8-86654f7e0a27	53e51eec-2d9b-4990-97f3-c9aa5d5652c3
DisplayText	Read and write all identity security health issues	Read and write identity security health issues
Description	Allows the app to read and write identity security health issues without a signed-in user.	Allows the app to read and write identity security health issues on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityIdentitiesMigration.Read.All

Category	Application	Delegated
Identifier	b018cc1c-c680-4e91-bb6e-462ee243fdb5	63595162-fcc0-4127-8b1e-bfe90b23a10e
DisplayText	Read all identity security sensor migration	Read identity security sensor migration
Description	Allows the app to read all the identity security sensor migration information without a signed-in user.	Allows the app to read all the identity security sensor migration information of signed user
AdminConsentRequired	Yes	Yes

SecurityIdentitiesMigration.ReadWrite.All

Category	Application	Delegated
Identifier	afd28a5a-707f-4edf-85c2-c446291e63da	741a6ef0-37e6-4b0a-9178-133d94fbc46e
DisplayText	Read and write all identity security sensor migration	Read and write identity security sensor migration
Description	Allows the app to read and write identity security sensor migration without a signed-in user.	Allows the app to read and write identity security sensor migration on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityIdentitiesSensors.Read.All

Category	Application	Delegated
Identifier	5f0ffea2-f474-4cf2-9834-61cda2bcea5c	2c221239-7c5c-4b30-9355-d84663bfcd96
DisplayText	Read all identity security sensors	Read identity security sensors
Description	Allows the app to read all the identity security sensors without a signed-in user.	Allows the app to read all the identity security sensors of signed user
AdminConsentRequired	Yes	Yes

SecurityIdentitiesSensors.ReadWrite.All

Category	Application	Delegated
Identifier	d4dcee6d-0774-412a-b06c-aeabbd99e816	087c3ad9-c2ca-4b82-9885-d5e25ce9e183
DisplayText	Read and write all identity security sensors	Read and write identity security sensors
Description	Allows the app to read and write identity security sensors without a signed-in user.	Allows the app to read and write identity security sensors on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityIdentitiesUserActions.Read.All

Category	Application	Delegated
Identifier	3e5d0bee-973f-4736-a123-4e1ab146f3a8	c7d0a939-da1c-4aca-80fa-d0a6cd924801
DisplayText	Read all identity security available user actions	Read identity security available user actions
Description	Allows the app to read all the identity security available user actions without a signed-in user.	Allows the app to read all the identity security available user actions of signed user
AdminConsentRequired	Yes	Yes

SecurityIdentitiesUserActions.ReadWrite.All

Category	Application	Delegated
Identifier	b4146a3a-dd4f-4af4-8d91-7cc0eef3d041	bf230e97-1957-4df6-b3f6-57f9029eacdf
DisplayText	Read and perform all identity security available user actions	Read and perform identity security available user actions
Description	Allows the app to read and write identity security available user actions without a signed-in user.	Allows the app to read and write identity security available user actions on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

SecurityIncident.Read.All

Category	Application	Delegated
Identifier	45cc0394-e837-488b-a098-1918f48d186c	b9abcc4f-94fc-4457-9141-d20ce80ec952
DisplayText	Read all security incidents	Read incidents
Description	Allows the app to read all security incidents, without a signed-in user.	Allows the app to read security incidents, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SecurityIncident.ReadWrite.All

Category	Application	Delegated
Identifier	34bf0e97-1971-4929-b999-9e2442d941d7	128ca929-1a19-45e6-a3b8-435ec44a36ba
DisplayText	Read and write to all security incidents	Read and write to incidents
Description	Allows the app to read and write to all security incidents, without a signed-in user.	Allows the app to read and write security incidents, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SensitivityLabel.Evaluate

Category	Application	Delegated
Identifier	57f0b71b-a759-45a0-9a0f-cc099fbd9a44	a4633e44-d355-4474-99df-8c2de6b0e39e
DisplayText	Evaluate sensitivity labels	Evaluate sensitivity labels

Category	Application	Delegated
Description	Allow the app to determine if there is any sensitivity label to be applied automatically to the content or recommended to the user for manual application, without a signed-in user.	Allow the app to determine if there is any sensitivity label to be applied automatically to the content or recommended to the user for manual application, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SensitivityLabel.Evaluate.All

Category	Application	Delegated
Identifier	986fa56a-6680-4aac-af09-4d1765376739	a42e3c42-b31e-4919-b699-696dca5dc9e7
DisplayText	Evaluate labels tenant scope.	Evaluate labels tenant scope.
Description	Allows the app to evaluate all sensitivity label.	Allows the app to evaluate all sensitivity label.
AdminConsentRequired	Yes	Yes

SensitivityLabel.Read

Category	Application	Delegated
Identifier	3b8e7aad-f6e3-4299-83f8-6fc6a5777f0b	1aeb73ce-68d7-49b7-913a-eedc80844551
DisplayText	Get labels application scope.	Get labels user scope.
Description	Allows the app to get sensitivity labels.	Allows the app to get sensitivity labels.
AdminConsentRequired	Yes	Yes

SensitivityLabels.Read.All

Category	Application	Delegated
Identifier	e46a01e9-b2cf-4d89-8424-bcdc6dd445ab	8b377c27-ea19-4863-a948-8a8588c8f2c3
DisplayText	Get labels tenant scope.	Get labels app scope.
Description	Allows the app to get sensitivity labels.	Allows the app to get sensitivity labels.
AdminConsentRequired	Yes	Yes

SentimentSurvey.Export.All

Category	Application	Delegated
Identifier	84fa35c1-f997-4c1c-894c-bb52108cfbbf	df9fd94d-51ff-443d-8f31-ae4dc1b5b8d8
DisplayText	Export all Sentiment Survey	Export all Sentiment Survey
Description	Allows the app to read all Sentiment Survey, without a signed-in user.	Allows the app to export all Sentiment Survey, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ServiceActivity-Exchange.Read.All

Category	Application	Delegated
Identifier	2b655018-450a-4845-81e7-d603b1ebffdb	1fe7aa48-9373-4a47-8df3-168335e0f4c9
DisplayText	Read all Exchange service activity	Read all Exchange service activity
Description	Allows the app to read all Exchange service activity, without a signed-in user.	Allows the app to read all Exchange service activity, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ServiceActivity-Microsoft365Web.Read.All

Category	Application	Delegated
Identifier	c766cb16-acc4-4663-ba09-6eedef5876c5	d74c75b1-d5a9-479d-902d-92f8f99182c1
DisplayText	Read all Microsoft 365 Web service activity	Read all Microsoft 365 Web service activity
Description	Allows the app to read all Microsoft 365 Web service activity, without a signed-in user.	Allows the app to read all Microsoft 365 Web service activity, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ServiceActivity-OneDrive.Read.All

Category	Application	Delegated
Identifier	57b4f899-b8c5-47c7-bdd3-c410c55602b7	347e3c16-30f3-4ac7-9b52-fc3c053de9c9
DisplayText	Read all One Drive service activity	Read all One Drive service activity
Description	Allows the app to read all One Drive service activity, without a signed-in user.	Allows the app to read all One Drive service activity, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ServiceActivity-Teams.Read.All

Category	Application	Delegated
Identifier	4dfee10b-fa4a-41b5-b34d-ccf54cc0c394	404d76f0-e10e-460a-92be-ef19600c54d1
DisplayText	Read all Teams service activity	Read all Teams service activity
Description	Allows the app to read all Teams service activity, without a signed-in user.	Allows the app to read all Teams service activity, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ServiceHealth.Read.All

Category	Application	Delegated
Identifier	79c261e0-fe76-4144-aad5-bdc68fbe4037	55896846-df78-47a7-aa94-8d3d4442ca7f
DisplayText	Read service health	Read service health
Description	Allows the app to read your tenant's service health information, without a signed-in user. Health information may include service issues or service health overviews.	Allows the app to read your tenant's service health information on behalf of the signed-in user. Health information may include service issues or service health overviews.
AdminConsentRequired	Yes	Yes

The *ServiceHealth.Read.All* delegated permission is available for consent in personal Microsoft accounts.

ServiceMessage.Read.All

Category	Application	Delegated
Identifier	1b620472-6534-4fe6-9df2-4680e8aa28ec	eda39fa6-f8cf-4c3c-a909-432c683e4c9b
DisplayText	Read service messages	Read service announcement messages
Description	Allows the app to read your tenant's service announcement messages, without a signed-in user. Messages may include information about new or changed features.	Allows the app to read your tenant's service announcement messages on behalf of the signed-in user. Messages may include information about new or changed features.
AdminConsentRequired	Yes	Yes

The *ServiceMessage.Read.All* delegated permission is available for consent in personal Microsoft accounts.

ServiceMessageViewpoint.Write

Category	Application	Delegated
Identifier	-	636e1b0b-1cc2-4b1c-9aa9-4eeed9b9761b
DisplayText	-	Update user status on service announcement messages
Description	-	Allows the app to update service announcement messages' user status on behalf of the signed-in user. The message status can be marked as read, archive, or favorite.
AdminConsentRequired	-	Yes

The *ServiceMessageViewpoint.Write* delegated permission is available for consent in personal Microsoft accounts.

ServicePrincipalEndpoint.Read.All

Category	Application	Delegated
Identifier	5256681e-b7f6-40c0-8447-2d9db68797a0	9f9ce928-e038-4e3b-8faf-7b59049a8ddc
DisplayText	Read service principal endpoints	Read service principal endpoints
Description	Allows the app to read service principal endpoints	Allows the app to read service principal endpoints
AdminConsentRequired	Yes	Yes

ServicePrincipalEndpoint.ReadWrite.All

Category	Application	Delegated
Identifier	89c8469c-83ad-45f7-8ff2-6e3d4285709e	7297d82c-9546-4aed-91df-3d4f0a9b3ff0
DisplayText	Read and update service principal endpoints	Read and update service principal endpoints
Description	Allows the app to update service principal endpoints	Allows the app to update service principal endpoints
AdminConsentRequired	Yes	Yes

SharePointCrossTenantMigration.Manage.All

Category	Application	Delegated
Identifier	a0521574-fcd8-4742-b29c-f796df57ea70	c608c170-08b5-466b-a8fe-0b4074b01613
DisplayText	Read, write and manage SharePoint Cross-Tenant migration settings and tasks	Read, write and manage SharePoint Cross-Tenant migration settings and tasks
Description	Allows the app to read, write and manage your tenant's SharePoint Cross-Tenant migration settings and tasks, without a signed-in user.	Allows the app to read, write and manage your tenant's SharePoint Cross-Tenant migration settings and tasks, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SharePointCrossTenantMigration.Read.All

Category	Application	Delegated
Identifier	f5fa52a5-b9ab-4dc3-885e-9e5b4a67068e	00dcb678-f9af-4e73-acb1-4f1657364629
DisplayText	Read SharePoint Cross-Tenant migration settings and tasks	Read SharePoint Cross-Tenant migration settings and tasks
Description	Allows the app to read your tenant's SharePoint Cross-Tenant migration settings and tasks, without a signed-in user.	Allows the app to read your tenant's SharePoint Cross-Tenant migration settings and tasks, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SharePointTenantSettings.Read.All

Category	Application	Delegated
Identifier	83d4163d-a2d8-4d3b-9695-4ae3ca98f888	2ef70e10-5bfd-4ede-a5f6-67720500b258
DisplayText	Read SharePoint and OneDrive tenant settings	Read SharePoint and OneDrive tenant settings

Category	Application	Delegated
Description	Allows the application to read the tenant-level settings of SharePoint and OneDrive, without a signed-in user.	Allows the application to read the tenant-level settings in SharePoint and OneDrive on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SharePointTenantSettings.ReadWrite.All

Category	Application	Delegated
Identifier	19b94e34-907c-4f43-bde9-38b1909ed408	aa07f155-3612-49b8-a147-6c590df35536
DisplayText	Read and change SharePoint and OneDrive tenant settings	Read and change SharePoint and OneDrive tenant settings
Description	Allows the application to read and change the tenant-level settings of SharePoint and OneDrive, without a signed-in user.	Allows the application to read and change the tenant-level settings of SharePoint and OneDrive on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ShortNotes.Read

Category	Application	Delegated
Identifier	-	50f66e47-eb56-45b7-aaa2-75057d9afe08
DisplayText	-	Read short notes of the signed-in user
Description	-	Allows the app to read all the short notes a sign-in user has access to.
AdminConsentRequired	-	No

The *ShortNotes.Read* delegated permission is available for consent in personal Microsoft accounts.

ShortNotes.Read.All

Category	Application	Delegated
Identifier	0c7d31ec-31ca-4f58-b6ec-9950b6b0de69	-
DisplayText	Read all users' short notes	-
Description	Allows the app to read all the short notes without a signed-in user.	-
AdminConsentRequired	Yes	-

ShortNotes.ReadWrite

Category	Application	Delegated
Identifier	-	328438b7-4c01-4c07-a840-e625a749bb89
DisplayText	-	Read, create, edit, and delete short notes of the signed-in user
Description	-	Allows the app to read, create, edit, and delete short notes of a signed-in user.
AdminConsentRequired	-	No

The *ShortNotes.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

ShortNotes.ReadWrite.All

Category	Application	Delegated
Identifier	842c284c-763d-4a97-838d-79787d129bab	-
DisplayText	Read, create, edit, and delete all users' short notes	-
Description	Allows the app to read, create, edit, and delete all the short notes without a signed-in user.	-
AdminConsentRequired	Yes	-

SignInIdentifier.Read.All

Category	Application	Delegated
Identifier	28e1fe78-598f-4df4-b55e-18bf34218925	458e1edc-1e75-438c-8c7b-c32115c9d373
DisplayText	Read all sign-in identifiers	Read SignInIdentifiers
Description	Allows the app to read your organization's sign-in identifiers, without a signed-in user.	Allows the app to read your organization's sign-in identifiers, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SignInIdentifier.ReadWrite.All

Category	Application	Delegated
Identifier	7fc588a2-ea2d-4d1f-bcf7-33c324b149b8	b4673c3c-7b5a-4012-9826-7c7e3c8db6af
DisplayText	Read and write all sign-in identifiers	Read and write all sign-in identifiers
Description	Allows the app to read and write your organization's sign-in identifiers, without a signed-in user.	Allows the app to read and write your organization's sign-in identifiers, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Sites.Archive.All

Category	Application	Delegated
Identifier	e3530185-4080-478c-a4ab-39322704df58	-
DisplayText	Archive/reactivate Site Collections without a signed in user.	-
Description	Allow the application to archive/reactivate site collections without a signed in user.	-
AdminConsentRequired	Yes	-

Sites.Create.All

Category	Application	Delegated
Identifier	80819dd8-2b3b-4551-a1ad-2700fc44f533	0e2e68e1-3f32-4e10-9281-f749e097fcbe
DisplayText	Create Site Collections without a signed in user.	Create Site Collections, on behalf of the signed-in user
Description	Allow the application to create site collections without a signed in user. Upon creation the application will be granted Sites.Selected(application) + FullControl to the newly created site.	Allow the application to create site collections on behalf of the signed in user. Upon creation the application will be granted Sites.Selected(delegated) + FullControl to the newly created site.
AdminConsentRequired	Yes	Yes

Sites.FullControl.All

Category	Application	Delegated
Identifier	a82116e5-55eb-4c41-a434-62fe8a61c773	5a54b8b3-347c-476d-8f8e-42d5c7424d29
DisplayText	Have full control of all site collections	Have full control of all site collections
Description	Allows the app to have full control of all site collections without a signed in user.	Allows the application to have full control of all site collections on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

The *Sites.FullControl.All* delegated permission is available for consent in personal Microsoft accounts.

Sites.Manage.All

Category	Application	Delegated
Identifier	0c0bf378-bf22-4481-8f81-9e89a9b4960a	65e50fdc-43b7-4915-933e-e8138f11f40a
DisplayText	Create, edit, and delete items and lists in all site collections	Create, edit, and delete items and lists in all site collections

Category	Application	Delegated
Description	Allows the app to create or delete document libraries and lists in all site collections without a signed in user.	Allows the application to create or delete document libraries and lists in all site collections on behalf of the signed-in user.
AdminConsentRequired	Yes	No

Sites.Read.All

Category	Application	Delegated
Identifier	332a536c-c7ef-4017-ab91-336970924f0d	205e70e5-aba6-4c52-a976-6d2d46c48043
DisplayText	Read items in all site collections	Read items in all site collections
Description	Allows the app to read documents and list items in all site collections without a signed in user.	Allows the application to read documents and list items in all site collections on behalf of the signed-in user
AdminConsentRequired	Yes	No

The *Sites.Read.All* delegated permission is available for consent in personal Microsoft accounts.

Sites.ReadWrite.All

Category	Application	Delegated
Identifier	9492366f-7969-46a4-8d15-ed1a20078fff	89fe6a52-be36-487e-b7d8-d061c450a026
DisplayText	Read and write items in all site collections	Edit or delete items in all site collections
Description	Allows the app to create, read, update, and delete documents and list items in all site collections without a signed in user.	Allows the application to edit or delete documents and list items in all site collections on behalf of the signed-in user.
AdminConsentRequired	Yes	No

The *Sites.ReadWrite.All* delegated permission is available for consent in personal Microsoft accounts.

Sites.Selected

Category	Application	Delegated
Identifier	883ea226-0bf2-4a8f-9f9d-92c9162a727d	f89c84ef-20d0-4b54-87e9-02e856d66d53
DisplayText	Access selected site collections	Access selected Sites, on behalf of the signed-in user
Description	Allow the application to access a subset of site collections without a signed in user. The specific site collections and the permissions granted will be configured in SharePoint Online.	Allow the application to access a subset of site collections on behalf of the signed-in user. The specific site collections and the permissions granted will be configured in SharePoint Online.
AdminConsentRequired	Yes	No

SMTP.Send

Category	Application	Delegated
Identifier	-	258f6531-6087-4cc4-bb90-092c5fb3ed3f
DisplayText	-	Send emails from mailboxes using SMTP AUTH.
Description	-	Allows the app to be able to send emails from the user's mailbox using the SMTP AUTH client submission protocol.
AdminConsentRequired	-	No

The *SMTP.Send* delegated permission is available for consent in personal Microsoft accounts.

SpiffeTrustDomain.Read.All

Category	Application	Delegated
Identifier	dcdfc277-41fd-4d68-ad0c-c3057235bd8e	9b4aa4b1-aaf3-41b7-b743-698b27e77ff6
DisplayText	Read SPIFFE trust domains and child resources	Read SPIFFE trust domains and child resources

Category	Application	Delegated
Description	Allows the app to read your organization's SPIFFE trust domains and child resources without a signed in user.	Allows the app to read your organization's SPIFFE trust domains and child resources on behalf of the user.
AdminConsentRequired	Yes	Yes

SpiffeTrustDomain.ReadWrite.All

Category	Application	Delegated
Identifier	17b78cfd-eeff-447d-8bab-2795af00055a	8ba47079-8c47-4bfe-b2ce-13f28ef37247
DisplayText	Read and write SPIFFE trust domains and child resources	Read and write SPIFFE trust domains and child resources
Description	Allows the app to read and write your organization's SPIFFE trust domains and child resources without a signed in user.	Allows the app to read and write your organization's SPIFFE trust domains and child resources on behalf of the user.
AdminConsentRequired	Yes	Yes

Storyline.ReadWrite.All

Category	Application	Delegated
Identifier	6eff534b-699e-44d9-af61-a4182f0ec37e	fd1d61cb-4e4b-4d15-a6d2-161348681d84
DisplayText	Read and write all Viva Engage storylines	Read and write all Viva Engage storylines
Description	Allows the app to modify Viva Engage storylines, read all storylines properties, update storyline properties, and delete storyline properties without a signed-in user.	Allows the app to modify the Viva Engage storyline and read all storyline properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SubjectNameRegistration.ReadWrite

Category	Application	Delegated
Identifier	-	fb0d7592-1943-4141-a7bc-b7ae45b84ecf
DisplayText	-	Read and write a subject name registration.
Description	-	Allows the app to read and write all the subject name registration properties on behalf of the signed-in user.
AdminConsentRequired	-	No

SubjectRightsRequest.Read.All

Category	Application	Delegated
Identifier	ee1460f0-368b-4153-870a-4e1ca7e72c42	9c3af74c-fd0f-4db4-b17a-71939e2a9d77
DisplayText	Read all subject rights requests	Read subject rights requests
Description	Allows the app to read subject rights requests without a signed-in user.	Allows the app to read subject rights requests on behalf of the signed-in user
AdminConsentRequired	Yes	Yes

SubjectRightsRequest.ReadWrite.All

Category	Application	Delegated
Identifier	8387eaa4-1a3c-41f5-b261-f888138e6041	2b8fcc74-bce1-4ae3-a0e8-60c53739299d
DisplayText	Read and write all subject rights requests	Read and write subject rights requests
Description	Allows the app to read and write subject rights requests without a signed in user.	Allows the app to read and write subject rights requests on behalf of the signed-in user
AdminConsentRequired	Yes	Yes

Subscription.Read.All

Category	Application	Delegated
Identifier	-	5f88184c-80bb-4d52-9ff2-757288b2e9b7
DisplayText	-	Read all webhook subscriptions
Description	-	Allows the app to read all webhook subscriptions on behalf of the signed-in user.
AdminConsentRequired	-	Yes

Synchronization.Read.All

Category	Application	Delegated
Identifier	5ba43d2f-fa88-4db2-bd1c-a67c5f0fb1ce	7aa02aeb-824f-4fbe-a3f7-611f751f5b55
DisplayText	Read all Azure AD synchronization data.	Read all Azure AD synchronization data
Description	Allows the application to read Azure AD synchronization information, without a signed-in user.	Allows the app to read Azure AD synchronization information, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Synchronization.ReadWrite.All

Category	Application	Delegated
Identifier	9b50c33d-700f-43b1-b2eb-87e89b703581	7bb27fa3-ea8f-4d67-a916-87715b6188bd
DisplayText	Read and write all Azure AD synchronization data.	Read and write all Azure AD synchronization data
Description	Allows the application to configure the Azure AD synchronization service, without a signed-in user.	Allows the app to configure the Azure AD synchronization service, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SynchronizationData-User.Upload

Category	Application	Delegated
Identifier	db31e92a-b9ea-4d87-bf6a-75a37a9ca35a	1a2e7420-4e92-4d2b-94cb-fb2952e9ddf7
DisplayText	Upload user data to the identity synchronization service	Upload user data to the identity synchronization service
Description	Allows the application to upload bulk user data to the identity synchronization service, without a signed-in user.	Allows the app to upload bulk user data to the identity synchronization service, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

SynchronizationData-User.Upload.OwnedBy

Category	Application	Delegated
Identifier	25c32ff3-849a-494b-b94f-20a8ac4e6774	-
DisplayText	Upload user data to the identity sync service for apps that this application creates or owns	-
Description	Allows the application to upload bulk user data to the identity synchronization service for apps that this application creates or owns, without a signed-in user.	-
AdminConsentRequired	Yes	-

Tasks.Read

Category	Application	Delegated
Identifier	-	f45671fb-e0fe-4b4b-be20-3d3ce43f1bcb
DisplayText	-	Read user's tasks and task lists
Description	-	Allows the app to read the signed-in user's tasks and task lists, including any shared with the user. Doesn't include permission to create, delete, or update anything.
AdminConsentRequired	-	No

The *Tasks.Read* delegated permission is available for consent in personal Microsoft accounts.

Tasks.Read.All

Category	Application	Delegated
Identifier	f10e1f91-74ed-437f-a6fd-d6ae88e26c1f	-
DisplayText	Read all users' tasks and tasklist	-
Description	Allows the app to read all users' tasks and task lists in your organization, without a signed-in user.	-
AdminConsentRequired	Yes	-

Tasks.Read.Shared

Category	Application	Delegated
Identifier	-	88d21fd4-8e5a-4c32-b5e2-4a1c95f34f72
DisplayText	-	Read user and shared tasks
Description	-	Allows the app to read tasks a user has permissions to access, including their own and shared tasks.
AdminConsentRequired	-	No

Tasks.ReadWrite

Category	Application	Delegated
Identifier	-	2219042f-cab5-40cc-b0d2-16b1540b4c5f
DisplayText	-	Create, read, update, and delete user's tasks and task lists
Description	-	Allows the app to create, read, update, and delete the signed-in user's tasks and task lists, including any shared with the user.
AdminConsentRequired	-	No

The *Tasks.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

Tasks.ReadWrite.All

Category	Application	Delegated
Identifier	44e666d1-d276-445b-a5fc-8815eeb81d55	-
DisplayText	Read and write all users' tasks and tasklists	-
Description	Allows the app to create, read, update and delete all users' tasks and task lists in your organization, without a signed-in user	-
AdminConsentRequired	Yes	-

Tasks.ReadWrite.Shared

Category	Application	Delegated
Identifier	-	c5ddf11b-c114-4886-8558-8a4e557cd52b
DisplayText	-	Read and write user and shared tasks
Description	-	Allows the app to create, read, update, and delete tasks a user has permissions to, including their own and shared tasks.
AdminConsentRequired	-	No

Team.Create

Category	Application	Delegated
Identifier	23fc2474-f741-46ce-8465-674744c5c361	7825d5d6-6049-4ce7-bdf6-3b8d53f4bcd0
DisplayText	Create teams	Create teams
Description	Allows the app to create teams without a signed-in user.	Allows the app to create teams on behalf of the signed-in user.
AdminConsentRequired	Yes	No

Team.ReadBasic.All

Category	Application	Delegated
Identifier	2280dda6-0bfd-44ee-a2f4-cb867cfc4c1e	485be79e-c497-4b35-9400-0e3fa7f2a5d4
DisplayText	Get a list of all teams	Read the names and descriptions of teams
Description	Get a list of all teams, without a signed-in user.	Read the names and descriptions of teams, on behalf of the signed-in user.
AdminConsentRequired	Yes	No

TeamMember.Read.All

Category	Application	Delegated
Identifier	660b7406-55f1-41ca-a0ed-0b035e182f3e	2497278c-d82d-46a2-b1ce-39d4cdde5570
DisplayText	Read the members of all teams	Read the members of teams
Description	Read the members of all teams, without a signed-in user.	Read the members of teams, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TeamMember.ReadWrite.All

Category	Application	Delegated
Identifier	0121dc95-1b9f-4aed-8bac-58c5ac466691	4a06efd2-f825-4e34-813e-82a57b03d1ee
DisplayText	Add and remove members from all teams	Add and remove members from teams
Description	Add and remove members from all teams, without a signed-in user. Also allows changing a team member's role, for example from owner to non-owner.	Add and remove members from teams, on behalf of the signed-in user. Also allows changing a member's role, for example from owner to non-owner.
AdminConsentRequired	Yes	Yes

TeamMember.ReadWriteNonOwnerRole.All

Category	Application	Delegated
Identifier	4437522e-9a86-4a41-a7da-e380edd4a97d	2104a4db-3a2f-4ea0-9dba-143d457dc666
DisplayText	Add and remove members with non-owner role for all teams	Add and remove members with non-owner role for all teams
Description	Add and remove members from all teams, without a signed-in user. Does not allow adding or removing a member with the owner role. Additionally, does not allow the app to elevate an existing member to the owner role.	Add and remove members from all teams, on behalf of the signed-in user. Does not allow adding or removing a member with the owner role. Additionally, does not allow the app to elevate an existing member to the owner role.
AdminConsentRequired	Yes	Yes

TeamsActivity.Read

Category	Application	Delegated
Identifier	-	0e755559-83fb-4b44-91d0-4cc721b9323e
DisplayText	-	Read user's teamwork activity feed
Description	-	Allows the app to read the signed-in user's teamwork activity feed.
AdminConsentRequired	-	No

TeamsActivity.Read.All

Category	Application	Delegated
Identifier	70dec828-f620-4914-aa83-a29117306807	-
DisplayText	Read all users' teamwork activity feed	-
Description	Allows the app to read all users' teamwork activity feed, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsActivity.Send

Category	Application	Delegated
Identifier	a267235f-af13-44dc-8385-c1dc93023186	7ab1d787-bae7-4d5d-8db6-37ea32df9186
DisplayText	Send a teamwork activity to any user	Send a teamwork activity as the user
Description	Allows the app to create new notifications in users' teamwork activity feeds without a signed in user. These notifications may not be discoverable or be held or governed by compliance policies.	Allows the app to create new notifications in users' teamwork activity feeds on behalf of the signed in user. These notifications may not be discoverable or be held or governed by compliance policies.
AdminConsentRequired	Yes	No

TeamsAppInstallation.ManageSelectedForChat

Category	Application	Delegated
Identifier	-	d1ba22c6-3f02-4c91-addb-bc3399bcca88
DisplayText	-	Manage installation and permission grants of selected Teams apps in chats
Description	-	Allows the app to read, install, upgrade, and uninstall selected Teams apps in chats the signed-in user can access. Gives the ability to manage permission grants for accessing those specific chats' data.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ManageSelectedForChat.All

Category	Application	Delegated
Identifier	22b74aab-d9e4-46f7-9424-f24b42307227	-
DisplayText	Manage installation and permission grants of selected Teams apps in all chats	-
Description	Allows the app to read, install, upgrade, and uninstall selected Teams apps in any chat, without a signed-in user. Gives the ability to manage permission grants for accessing those specific chats' data.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ManageSelectedForTeam

Category	Application	Delegated
Identifier	-	c67b2d7e-6b80-4218-938a-05e73058e42d
DisplayText	-	Manage installation and permission grants of selected Teams apps in teams
Description	-	Allows the app to read, install, upgrade, and uninstall Teams apps in teams the signed-in user can access. Gives the ability to manage permission grants for accessing those specific teams' data.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ManageSelectedForTeam.All

Category	Application	Delegated
Identifier	b448d252-1f26-4227-b6ff-21ab510975a2	-
DisplayText	Manage installation and permission grants of selected Teams apps in all teams	-
Description	Allows the app to read, install, upgrade, and uninstall selected Teams apps in any team, without a signed-in user. Gives the ability to manage permission grants for accessing those specific teams' data.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ManageSelectedForUser

Category	Application	Delegated
Identifier	-	830c2bd9-c335-4caf-bf83-c07fa8a23ef1
DisplayText	-	Manage installation and permission grants of selected Teams apps in users' personal scope
Description	-	Allows the app to read, install, upgrade, and uninstall seleected Teams apps in user accounts, on behalf of the signed-in user. Gives the ability to manage permission grants for accessing those specific users' data.

Category	Application	Delegated
AdminConsentRequired	-	Yes

TeamsAppInstallation.ManageSelectedForUser.All

Category	Application	Delegated
Identifier	e97a9235-5b3c-43c4-b37d-6786a173fae4	-
DisplayText	Manage installation and permission grants of selected Teams apps for all user accounts	-
Description	Allows the app to read, install, upgrade, and uninstall selected Teams apps in any user account, without a signed-in user. Gives the ability to manage permission grants for accessing those specific users' data.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.Read.All

Category	Application	Delegated
Identifier	0fdf35a5-82f8-41ff-9ded-0b761cc73512	-
DisplayText	Read installed Teams apps for all installation scopes	-
Description	Allows the app to read the Teams apps that are installed in any scope, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadForChat

Category	Application	Delegated
Identifier	-	bf3fbf03-f35f-4e93-963e-47e4d874c37a
DisplayText	-	Read installed Teams apps in chats
Description	-	Allows the app to read the Teams apps that are installed in chats the signed-in user can access. Does not give the ability to read

Category	Application	Delegated
		application-specific settings.
AdminConsentRequired	-	No

TeamsAppInstallation.ReadForChat.All

Category	Application	Delegated
Identifier	cc7e7635-2586-41d6-adaa-a8d3bcad5ee5	-
DisplayText	Read installed Teams apps for all chats	-
Description	Allows the app to read the Teams apps that are installed in any chat, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadForTeam

Category	Application	Delegated
Identifier	-	5248dcb1-f83b-4ec3-9f4d-a4428a961a72
DisplayText	-	Read installed Teams apps in teams
Description	-	Allows the app to read the Teams apps that are installed in teams the signed-in user can access. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadForTeam.All

Category	Application	Delegated
Identifier	1f615aea-6bf9-4b05-84bd-46388e138537	-
DisplayText	Read installed Teams apps for all teams	-
Description	Allows the app to read the Teams apps that are installed in any team, without a signed-in user. Does not give the ability to read	-

Category	Application	Delegated
	application-specific settings.	
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadForUser

Category	Application	Delegated
Identifier	-	c395395c-ff9a-4dba-bc1f-8372ba9dca84
DisplayText	-	Read user's installed Teams apps
Description	-	Allows the app to read the Teams apps that are installed for the signed-in user. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	No

TeamsAppInstallation.ReadForUser.All

Category	Application	Delegated
Identifier	9ce09611-f4f7-4abd-a629-a05450422a97	-
DisplayText	Read installed Teams apps for all users	-
Description	Allows the app to read the Teams apps that are installed for any user, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadSelectedForChat

Category	Application	Delegated
Identifier	-	0f3420c2-c6ec-46de-ab72-fd51267087d5
DisplayText	-	Read selected installed Teams apps in chats
Description	-	Allows the app to read the selected Teams apps that are installed in chats the signed-in user can access. Does not give the ability to

Category	Application	Delegated
		read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadSelectedForChat.All

Category	Application	Delegated
Identifier	53d40ddb-9b27-4c97-b800-985be6041990	-
DisplayText	Read selected installed Teams apps in all chats	-
Description	Allows the app to read the selected Teams apps that are installed in any chat, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadSelectedForTeam

Category	Application	Delegated
Identifier	-	b55df1c0-db20-435b-aef2-afe6ed487e16
DisplayText	-	Read selected installed Teams apps in teams
Description	-	Allows the app to read the selected Teams apps that are installed in teams the signed-in user can access. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadSelectedForTeam.All

Category	Application	Delegated
Identifier	93c6a289-70fd-489e-a053-6cf8f7d772f6	-
DisplayText	Read selected installed Teams apps in all teams	-
Description	Allows the app to read the selected Teams apps that are installed in any team, without a signed-in user. Does not give the ability to read	-

Category	Application	Delegated
	application-specific settings.	
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadSelectedForUser

Category	Application	Delegated
Identifier	-	fe2e4e1d-101f-4fb2-9cb1-9d6659db45d4
DisplayText	-	Read user's selected installed Teams apps
Description	-	Allows the app to read the selected Teams apps that are installed for the signed-in user. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadSelectedForUser.All

Category	Application	Delegated
Identifier	44fb0e7c-1f9a-47f1-bb9e-7f92d48ed288	-
DisplayText	Read selected installed Teams apps for all users	-
Description	Allows an app to read, install, upgrade, and uninstall selected apps to any user, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteAndConsentForChat

Category	Application	Delegated
Identifier	-	e1408a66-8f82-451b-a2f3-3c3e38f7413f
DisplayText	-	Manage installed Teams apps in chats
Description	-	Allows the app to read, install, upgrade, and uninstall Teams apps in chats the signed-in user can access. Gives the ability to manage permission grants for accessing those specific chats' data.

Category	Application	Delegated
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteAndConsentForChat.All

Category	Application	Delegated
Identifier	6e74eff9-4a21-45d6-bc03-3a20f61f8281	-
DisplayText	Manage installation and permission grants of Teams apps for all chats	-
Description	Allows the app to read, install, upgrade, and uninstall Teams apps in any chat, without a signed-in user. Gives the ability to manage permission grants for accessing those specific chats' data.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteAndConsentForTeam

Category	Application	Delegated
Identifier	-	946349d5-2a9d-4535-abc0-7beeacaedd1d
DisplayText	-	Manage installed Teams apps in teams
Description	-	Allows the app to read, install, upgrade, and uninstall Teams apps in teams the signed-in user can access. Gives the ability to manage permission grants for accessing those specific teams' data.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteAndConsentForTeam.All

Category	Application	Delegated
Identifier	b0c13be0-8e20-4bc5-8c55-963c23a39ce9	-
DisplayText	Manage installation and permission grants of Teams apps for all teams	-
Description	Allows the app to read, install, upgrade, and uninstall Teams apps in any team, without a signed-in user. Gives the ability to manage	-

Category	Application	Delegated
	permission grants for accessing those specific teams' data.	
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteAndConsentForUser

Category	Application	Delegated
Identifier	-	2da62c49-dfbd-40df-ba16-fef3529d391c
DisplayText	-	Manage installation and permission grants of Teams apps in users' personal scope
Description	-	Allows the app to read, install, upgrade, and uninstall Teams apps in user accounts, on behalf of the signed-in user. Gives the ability to manage permission grants for accessing those specific users' data.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteAndConsentForUser.All

Category	Application	Delegated
Identifier	32ca478f-f89e-41d0-aaf8-101deb7da510	-
DisplayText	Manage installation and permission grants of Teams apps in a user account	-
Description	Allows the app to read, install, upgrade, and uninstall Teams apps in any user account, without a signed-in user. Gives the ability to manage permission grants for accessing those specific users' data.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteAndConsentSelfForChat

Category	Application	Delegated
Identifier	-	a0e0e18b-8fb2-458f-8130-da2d7cab9c75

Category	Application	Delegated
DisplayText	-	Allow the Teams app to manage itself and its permission grants in chats
Description	-	Allows a Teams app to read, install, upgrade, and uninstall itself in chats the signed-in user can access, and manage its permission grants for accessing those specific chats' data.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteAndConsentSelfForChat.All

Category	Application	Delegated
Identifier	ba1ba90b-2d8f-487e-9f16-80728d85bb5c	-
DisplayText	Allow the Teams app to manage itself and its permission grants for all chats	-
Description	Allows a Teams app to read, install, upgrade, and uninstall itself for any chat, without a signed-in user, and manage its permission grants for accessing those specific chats' data.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteAndConsentSelfForTeam

Category	Application	Delegated
Identifier	-	4a6bbf29-a0e1-4a4d-a7d1-cef17f772975
DisplayText	-	Allow the Teams app to manage itself and its permission grants in teams
Description	-	Allows a Teams app to read, install, upgrade, and uninstall itself in teams the signed-in user can access, and manage its permission grants for accessing those specific teams' data.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteAndConsentSelfForTeam.All

Category	Application	Delegated
Identifier	1e4be56c-312e-42b8-a2c9-009600d732c0	-
DisplayText	Allow the Teams app to manage itself and its permission grants for all teams	-
Description	Allows a Teams app to read, install, upgrade, and uninstall itself for any team, without a signed-in user, and manage its permission grants for accessing those specific teams' data.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteAndConsentSelfForUser

Category	Application	Delegated
Identifier	-	7a349935-c54d-44ab-ab66-1b460d315be7
DisplayText	-	Allow the Teams app to manage itself and its permission grants in user accounts
Description	-	Allows a Teams app to read, install, upgrade, and uninstall itself in user accounts, and manage its permission grants for accessing those specific users' data, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteAndConsentSelfForUser.All

Category	Application	Delegated
Identifier	a87076cf-6abd-4e56-8559-4dbdf41bef96	-
DisplayText	Allow the Teams app to manage itself and its permission grants in all user accounts	-
Description	Allows a Teams app to read, install, upgrade, and uninstall itself for any user account, without a signed-in user, and manage its permission grants for accessing those specific users' data.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteForChat

Category	Application	Delegated
Identifier	-	aa85bf13-d771-4d5d-a9e6-bca04ce44edf
DisplayText	-	Manage installed Teams apps in chats
Description	-	Allows the app to read, install, upgrade, and uninstall Teams apps in chats the signed-in user can access. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteForChat.All

Category	Application	Delegated
Identifier	9e19bae1-2623-4c4f-ab6e-2664615ff9a0	-
DisplayText	Manage Teams apps for all chats	-
Description	Allows the app to read, install, upgrade, and uninstall Teams apps in any chat, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteForTeam

Category	Application	Delegated
Identifier	-	2e25a044-2580-450d-8859-42eeb6e996c0
DisplayText	-	Manage installed Teams apps in teams
Description	-	Allows the app to read, install, upgrade, and uninstall Teams apps in teams the signed-in user can access. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteForTeam.All

Category	Application	Delegated
Identifier	5dad17ba-f6cc-4954-a5a2-a0dcc95154f0	-
DisplayText	Manage Teams apps for all teams	-
Description	Allows the app to read, install, upgrade, and uninstall Teams apps in any team, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteForUser

Category	Application	Delegated
Identifier	-	093f8818-d05f-49b8-95bc-9d2a73e9a43c
DisplayText	-	Manage user's installed Teams apps
Description	-	Allows the app to read, install, upgrade, and uninstall Teams apps installed for the signed-in user. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteForUser.All

Category	Application	Delegated
Identifier	74ef0291-ca83-4d02-8c7e-d2391e6a444f	-
DisplayText	Manage Teams apps for all users	-
Description	Allows the app to read, install, upgrade, and uninstall Teams apps for any user, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteSelectedForChat

Category	Application	Delegated
Identifier	-	690aa3b6-4b71-41c2-a990-77a8c4768d2b
DisplayText	-	Manage selected Teams apps installed in chats
Description	-	Allows the app to read, install, upgrade, and uninstall selected Teams apps in chats the signed-in user can access. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteSelectedForChat.All

Category	Application	Delegated
Identifier	25bbeaad-04be-4207-83ed-a263aae76ddf	-
DisplayText	Manage selected installed Teams apps in all chats	-
Description	Allows the app to read, install, upgrade, and uninstall selected Teams apps in any chat, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteSelectedForTeam

Category	Application	Delegated
Identifier	-	9131c833-9a49-4c54-b38f-615ecfc4fc69
DisplayText	-	Manage selected Teams apps installed in teams
Description	-	Allows the app to read, install, upgrade, and uninstall selected Teams apps in teams the signed-in user can access. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteSelectedForTeam.All

Category	Application	Delegated
Identifier	7b5823ae-d0f2-424d-b90c-d843ffada7d9	-
DisplayText	Manage selected installed Teams apps in all teams	-
Description	Allows the app to read, install, upgrade, and uninstall selected Teams apps in any team, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteSelectedForUser

Category	Application	Delegated
Identifier	-	ea819e27-c92a-4118-b83b-4540b125d744
DisplayText	-	Manage selected Teams apps installed for a user
Description	-	Allows the app to read, install, upgrade, and uninstall selected Teams apps installed for the signed in user. Does not give the ability to read application-specific settings.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteSelectedForUser.All

Category	Application	Delegated
Identifier	650a76ec-4118-4b25-9d3a-1f98048a5ee0	-
DisplayText	Manage selected Teams apps installed for all users	-
Description	Allows the app to read, install, upgrade, and uninstall selected Teams apps for any user, without a signed-in user. Does not give the ability to read application-specific settings.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteSelfForChat

Category	Application	Delegated
Identifier	-	0ce33576-30e8-43b7-99e5-62f8569a4002
DisplayText	-	Allow the Teams app to manage itself in chats
Description	-	Allows a Teams app to read, install, upgrade, and uninstall itself in chats the signed-in user can access.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteSelfForChat.All

Category	Application	Delegated
Identifier	73a45059-f39c-4baf-9182-4954ac0e55cf	-
DisplayText	Allow the Teams app to manage itself for all chats	-
Description	Allows a Teams app to read, install, upgrade, and uninstall itself for any chat, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteSelfForTeam

Category	Application	Delegated
Identifier	-	0f4595f7-64b1-4e13-81bc-11a249df07a9
DisplayText	-	Allow the app to manage itself in teams
Description	-	Allows a Teams app to read, install, upgrade, and uninstall itself to teams the signed-in user can access.
AdminConsentRequired	-	Yes

TeamsAppInstallation.ReadWriteSelfForTeam.All

Category	Application	Delegated
Identifier	9f67436c-5415-4e7f-8ac1-3014a7132630	-

Category	Application	Delegated
DisplayText	Allow the Teams app to manage itself for all teams	-
Description	Allows a Teams app to read, install, upgrade, and uninstall itself in any team, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsAppInstallation.ReadWriteSelfForUser

Category	Application	Delegated
Identifier	-	207e0cb1-3ce7-4922-b991-5a760c346ebc
DisplayText	-	Allow the Teams app to manage itself for a user
Description	-	Allows a Teams app to read, install, upgrade, and uninstall itself for the signed-in user.
AdminConsentRequired	-	No

TeamsAppInstallation.ReadWriteSelfForUser.All

Category	Application	Delegated
Identifier	908de74d-f8b2-4d6b-a9ed-2a17b3b78179	-
DisplayText	Allow the app to manage itself for all users	-
Description	Allows a Teams app to read, install, upgrade, and uninstall itself to any user, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamSettings.Read.All

Category	Application	Delegated
Identifier	242607bd-1d2c-432c-82eb-bdb27baa23ab	48638b3c-ad68-4383-8ac4-e6880ee6ca57
DisplayText	Read all teams' settings	Read teams' settings

Category	Application	Delegated
Description	Read all team's settings, without a signed-in user.	Read all teams' settings, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TeamSettings.ReadWrite.All

Category	Application	Delegated
Identifier	bdd80a03-d9bc-451d-b7c4-ce7c63fe3c8f	39d65650-9d3e-4223-80db-a335590d027e
DisplayText	Read and change all teams' settings	Read and change teams' settings
Description	Read and change all teams' settings, without a signed-in user.	Read and change all teams' settings, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TeamsPolicyUserAssign.ReadWrite.All

Category	Application	Delegated
Identifier	1801e8f4-cf09-4c4e-a1b5-036dfcca6c90	6997c35c-a586-440c-8a0b-4ffe5d118dc0
DisplayText	Read and Write Teams policy user assignment and unassigment for all policy types.	Read and Write Teams policy user assignment and unassigment for all policy types.
Description	Allow the app to read or write/update the policy assignment and unassigment for Teams users for all policy type categories.	Allow the app to read or write/update the policy assignment and unassigment for Teams users for all policy type categories.
AdminConsentRequired	Yes	Yes

TeamsResourceAccount.Read.All

Category	Application	Delegated
Identifier	b55aa226-33a1-4396-bcf4-edce5e7a31c1	ea2cbd09-253c-4f69-a0e6-07383c5f07cc
DisplayText	Read Teams resource accounts	Read Teams resource accounts
Description	Allows the app to read your tenant's resource accounts without a signed-in user.	Allows the app to read your tenant's resource accounts on behalf of the signed-in admin user.
AdminConsentRequired	Yes	Yes

TeamsTab.Create

Category	Application	Delegated
Identifier	49981c42-fd7b-4530-be03-e77b21aed25e	a9ff19c2-f369-4a95-9a25-ba9d460efc8e
DisplayText	Create tabs in Microsoft Teams.	Create tabs in Microsoft Teams.
Description	Allows the app to create tabs in any team in Microsoft Teams, without a signed-in user. This does not grant the ability to read, modify or delete tabs after they are created, or give access to the content inside the tabs.	Allows the app to create tabs in any team in Microsoft Teams, on behalf of the signed-in user. This does not grant the ability to read, modify or delete tabs after they are created, or give access to the content inside the tabs.
AdminConsentRequired	Yes	Yes

TeamsTab.Read.All

Category	Application	Delegated
Identifier	46890524-499a-4bb2-ad64-1476b4f3e1cf	59dacb05-e88d-4c13-a684-59f1afc8cc98
DisplayText	Read tabs in Microsoft Teams.	Read tabs in Microsoft Teams.
Description	Read the names and settings of tabs inside any team in Microsoft Teams, without a signed-in user. This does not give access to the content inside the tabs.	Read the names and settings of tabs inside any team in Microsoft Teams, on behalf of the signed-in user. This does not give access to the content inside the tabs.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

TeamsTab.ReadWrite.All

Category	Application	Delegated
Identifier	a96d855f-016b-47d7-b51c-1218a98d791c	b98bfd41-87c6-45cc-b104-e2de4f0dafb9
DisplayText	Read and write tabs in Microsoft Teams.	Read and write tabs in Microsoft Teams.
Description	Read and write tabs in any team in Microsoft Teams, without a signed-in user. This does not give access to the content inside the tabs.	Read and write tabs in any team in Microsoft Teams, on behalf of the signed-in user. This does not give access to the content inside the tabs.
AdminConsentRequired	Yes	Yes

TeamsTab.ReadWriteForChat

Category	Application	Delegated
Identifier	-	ee928332-e9c2-4747-b4a0-f8c164b68de6
DisplayText	-	Allow the Teams app to manage all tabs in chats
Description	-	Allows a Teams app to read, install, upgrade, and uninstall all tabs in chats the signed-in user can access.
AdminConsentRequired	-	Yes

TeamsTab.ReadWriteForChat.All

Category	Application	Delegated
Identifier	fd9ce730-a250-40dc-bd44-8dc8d20f39ea	-
DisplayText	Allow the Teams app to manage all tabs for all chats	-
Description	Allows a Teams app to read, install, upgrade, and uninstall all tabs	-

Category	Application	Delegated
	for any chat, without a signed-in user.	
AdminConsentRequired	Yes	-

TeamsTab.ReadWriteForTeam

Category	Application	Delegated
Identifier	-	c975dd04-a06e-4fbb-9704-62daad77bb49
DisplayText	-	Allow the Teams app to manage all tabs in teams
Description	-	Allows a Teams app to read, install, upgrade, and uninstall all tabs to teams the signed-in user can access.
AdminConsentRequired	-	Yes

TeamsTab.ReadWriteForTeam.All

Category	Application	Delegated
Identifier	6163d4f4-fbf8-43da-a7b4-060fe85ed148	-
DisplayText	Allow the Teams app to manage all tabs for all teams	-
Description	Allows a Teams app to read, install, upgrade, and uninstall all tabs in any team, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsTab.ReadWriteForUser

Category	Application	Delegated
Identifier	-	c37c9b61-7762-4bff-a156-afc0005847a0
DisplayText	-	Allow the Teams app to manage all tabs for a user
Description	-	Allows a Teams app to read, install, upgrade, and uninstall all tabs for the signed-in user.
AdminConsentRequired	-	No

TeamsTab.ReadWriteForUser.All

Category	Application	Delegated
Identifier	425b4b59-d5af-45c8-832f-bb0b7402348a	-
DisplayText	Allow the app to manage all tabs for all users	-
Description	Allows a Teams app to read, install, upgrade, and uninstall all tabs for any user, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsTab.ReadWriteSelfForChat

Category	Application	Delegated
Identifier	-	0c219d04-3abf-47f7-912d-5cca239e90e6
DisplayText	-	Allow the Teams app to manage only its own tabs in chats
Description	-	Allows a Teams app to read, install, upgrade, and uninstall its own tabs in chats the signed-in user can access.
AdminConsentRequired	-	Yes

TeamsTab.ReadWriteSelfForChat.All

Category	Application	Delegated
Identifier	9f62e4a2-a2d6-4350-b28b-d244728c4f86	-
DisplayText	Allow the Teams app to manage only its own tabs for all chats	-
Description	Allows a Teams app to read, install, upgrade, and uninstall its own tabs for any chat, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsTab.ReadWriteSelfForTeam

Category	Application	Delegated
Identifier	-	f266662f-120a-4314-b26a-99b08617c7ef
DisplayText	-	Allow the Teams app to manage only its own tabs in teams
Description	-	Allows a Teams app to read, install, upgrade, and uninstall its own tabs to teams the signed-in user can access.
AdminConsentRequired	-	Yes

TeamsTab.ReadWriteSelfForTeam.All

Category	Application	Delegated
Identifier	91c32b81-0ef0-453f-a5c7-4ce2e562f449	-
DisplayText	Allow the Teams app to manage only its own tabs for all teams	-
Description	Allows a Teams app to read, install, upgrade, and uninstall its own tabs in any team, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsTab.ReadWriteSelfForUser

Category	Application	Delegated
Identifier	-	395dfec1-a0b9-465f-a783-8250a430cb8c
DisplayText	-	Allow the Teams app to manage only its own tabs for a user
Description	-	Allows a Teams app to read, install, upgrade, and uninstall its own tabs for the signed-in user.
AdminConsentRequired	-	No

TeamsTab.ReadWriteSelfForUser.All

Category	Application	Delegated
Identifier	3c42dec6-49e8-4a0a-b469-36cff0d9da93	-

Category	Application	Delegated
DisplayText	Allow the Teams app to manage only its own tabs for all users	-
Description	Allows a Teams app to read, install, upgrade, and uninstall its own tabs for any user, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamsTelephoneNumber.Read.All

Category	Application	Delegated
Identifier	39b17d18-680c-41f4-b9c2-5f30629e7cb6	1bc6eab1-058d-4557-b011-d4c41cec88b7
DisplayText	Read Tenant-Acquired Telephone Number Details	Read Tenant-Acquired Telephone Number Details
Description	Allows the app to read your tenant's acquired telephone number details, without a signed-in user. Acquired telephone numbers may include attributes related to assigned object, emergency location, network site, etc.	Allows the app to read your tenant's acquired telephone number details on behalf of the signed-in admin user. Acquired telephone numbers may include attributes related to assigned object, emergency location, network site, etc.
AdminConsentRequired	Yes	Yes

TeamsTelephoneNumber.ReadWrite.All

Category	Application	Delegated
Identifier	0a42382f-155c-4eb1-9bdc-21548ccaa387	424b07a8-1209-4d17-9fe4-9018a93a1024
DisplayText	Read and Modify Tenant-Acquired Telephone Number Details	Read and Modify Tenant-Acquired Telephone Number Details
Description	Allows the app to read your tenant's acquired telephone number details, without a signed-in user. Acquired telephone numbers may include attributes related to assigned object, emergency location, network site, etc.	Allows the app to read and modify your tenant's acquired telephone number details on behalf of the signed-in admin user. Acquired telephone numbers may include attributes related to assigned

Category	Application	Delegated
		object, emergency location, network site, etc.
AdminConsentRequired	Yes	Yes

TeamsUserConfiguration.Read.All

Category	Application	Delegated
Identifier	a91eadaf-2c3c-4362-908b-fb172d208fc6	5c469ce4-dab5-4afd-b9de-14f1ba4004a7
DisplayText	Read Teams user configurations	Read Teams user configurations
Description	Allows the app to read your tenant's user configurations, without a signed-in user. User configuration may include attributes related to user, such as telephone number, assigned policies, etc.	Allows the app to read your tenant's user configurations on behalf of the signed-in admin user. User configuration may include attributes related to user, such as telephone number, assigned policies, etc.
AdminConsentRequired	Yes	Yes

TeamTemplates.Read

Category	Application	Delegated
Identifier	-	cd87405c-5792-4f15-92f7-debc0db6d1d6
DisplayText	-	Read available Teams templates
Description	-	Allows the app to read the available Teams templates, on behalf of the signed-in user.
AdminConsentRequired	-	No

TeamTemplates.Read.All

Category	Application	Delegated
Identifier	6323133e-1f6e-46d4-9372-ac33a0870636	-

Category	Application	Delegated
DisplayText	Read all available Teams Templates	-
Description	Allows the app to read all available Teams Templates, without a signed-user.	-
AdminConsentRequired	Yes	-

Teamwork.Migrate.All

Category	Application	Delegated
Identifier	dfb0dd15-61de-45b2-be36-d6a69fba3c79	-
DisplayText	Create chat and channel messages with anyone's identity and with any timestamp	-
Description	Allows the app to create chat and channel messages, without a signed in user. The app specifies which user appears as the sender, and can backdate the message to appear as if it was sent long ago. The messages can be sent to any chat or channel in the organization.	-
AdminConsentRequired	Yes	-

The *Teamwork.Migrate.All* delegated permission is available for consent in personal Microsoft accounts.

Teamwork.Read.All

Category	Application	Delegated
Identifier	75bcfbce-a647-4fba-ad51-b63d73b210f4	594f4bb6-c083-4cf9-8aa8-213823bdf351
DisplayText	Read organizational teamwork settings	Read organizational teamwork settings
Description	Allows the app to read all teamwork settings of the organization without a signed-in user.	Allows the app to read the teamwork settings of the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TeamworkAppSettings.Read.All

Category	Application	Delegated
Identifier	475ebe88-f071-4bd7-af2b-642952bd4986	44e060c4-bbdc-4256-a0b9-dcc0396db368
DisplayText	Read Teams app settings	Read Teams app settings
Description	Allows the app to read the Teams app settings without a signed-in user.	Allows the app to read the Teams app settings on behalf of the signed-in user.
AdminConsentRequired	Yes	No

TeamworkAppSettings.ReadWrite.All

Category	Application	Delegated
Identifier	ab5b445e-8f10-45f4-9c79-dd3f8062cc4e	87c556f0-2bd9-4eed-bd74-5dd8af6eaf7e
DisplayText	Read and write Teams app settings	Read and write Teams app settings
Description	Allows the app to read and write the Teams app settings without a signed-in user.	Allows the app to read and write the Teams app settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TeamworkCustomEmoji.Create

Category	Application	Delegated
Identifier	-	72464cd4-58fd-4116-8a9e-b74757574757
DisplayText	-	Create custom emoji
Description	-	Create custom emoji on behalf of the signed-in user.
AdminConsentRequired	-	No

TeamworkCustomEmoji.Create.All

Category	Application	Delegated
Identifier	85643b08-0e25-4d99-9d68-04ba0fef9740	-
DisplayText	Create custom emoji	-
Description	Create custom emoji without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamworkCustomEmoji.Read

Category	Application	Delegated
Identifier	-	89b231b1-414e-4dd4-bb87-d59906da4e05
DisplayText	-	Read custom emoji
Description	-	Read custom emoji on behalf of the signed-in user.
AdminConsentRequired	-	No

TeamworkCustomEmoji.Read.All

Category	Application	Delegated
Identifier	1efa3d37-1703-4685-9a59-baf6296fb956	-
DisplayText	Read custom emoji	-
Description	Read custom emoji without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamworkDevice.Read.All

Category	Application	Delegated
Identifier	0591bafd-7c1c-4c30-a2a5-2b9aacb1dfe8	b659488b-9d28-4208-b2be-1c6652b3c970
DisplayText	Read Teams devices	Read Teams devices

Category	Application	Delegated
Description	Allow the app to read the management data for Teams devices, without a signed-in user.	Allow the app to read the management data for Teams devices on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TeamworkDevice.ReadWrite.All

Category	Application	Delegated
Identifier	79c02f5b-bd4f-4713-bc2c-a8a4a66e127b	ddd97ecb-5c31-43db-a235-0ee20e635c40
DisplayText	Read and write Teams devices	Read and write Teams devices
Description	Allow the app to read and write the management data for Teams devices, without a signed-in user.	Allow the app to read and write the management data for Teams devices on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TeamworkSection.Read

Category	Application	Delegated
Identifier	-	87a3258d-8c34-49e2-ab91-9b8bdbd79177
DisplayText	-	Read your sections
Description	-	Allows the app to read the signed-in user's sections (folders) for organizing chats and channels in Teams.
AdminConsentRequired	-	No

TeamworkSection.Read.All

Category	Application	Delegated
Identifier	e9e1b87a-726e-4628-8fab-d1fc58d4d9ad	-
DisplayText	Read all users' sections	-

Category	Application	Delegated
Description	Allows the app to read all users' sections (folders) for organizing chats and channels in Teams, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamworkSection.ReadWrite

Category	Application	Delegated
Identifier	-	70dbe5e8-39b9-40f3-8c65-3ec7b00ad804
DisplayText	-	Read and write your sections
Description	-	Allows the app to read and write the signed-in user's sections (folders) for organizing chats and channels in Teams.
AdminConsentRequired	-	No

TeamworkSection.ReadWrite.All

Category	Application	Delegated
Identifier	fd99f9da-42d6-4d00-8a41-4161bea42309	-
DisplayText	Read and write all users' sections	-
Description	Allows the app to read and write all users' sections (folders) for organizing chats and channels in Teams, without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamworkTag.Read

Category	Application	Delegated
Identifier	-	57587d0b-8399-45be-b207-8050cec54575
DisplayText	-	Read tags in Teams
Description	-	Allows the app to read tags in Teams, on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	-	Yes

TeamworkTag.Read.All

Category	Application	Delegated
Identifier	b74fd6c4-4bde-488e-9695-eeb100e4907f	-
DisplayText	Read tags in Teams	-
Description	Allows the app to read tags in Teams without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamworkTag.ReadWrite

Category	Application	Delegated
Identifier	-	539dabd7-b5b6-4117-b164-d60cd15a8671
DisplayText	-	Read and write tags in Teams
Description	-	Allows the app to read and write tags in Teams, on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TeamworkTag.ReadWrite.All

Category	Application	Delegated
Identifier	a3371ca5-911d-46d6-901c-42c8c7a937d8	-
DisplayText	Read and write tags in Teams	-
Description	Allows the app to read and write tags in Teams without a signed-in user.	-
AdminConsentRequired	Yes	-

TeamworkTargetedMessage.Read.All

Category	Application	Delegated
Identifier	b0cfd829-be18-4b31-bb0e-ec1df8197ba3	-
DisplayText	Read all targeted messages of group chat or channel	-
Description	Allows the app to read all group chat or channel targeted messages in Microsoft Teams.	-
AdminConsentRequired	Yes	-

TeamworkTargetedMessage.ReadWrite

Category	Application	Delegated
Identifier	-	162354de-2885-4e5a-94fb-2f03019a65a8
DisplayText	-	Update targeted messages belonging to the user
Description	-	Allows the user to update group chat or channel targeted messages in Microsoft Teams.
AdminConsentRequired	-	Yes

TeamworkUserInteraction.Read.All

Category	Application	Delegated
Identifier	-	b4d26916-07e0-4daf-9096-9f6d9174aa96
DisplayText	-	Read all of the possible Teams interactions between the user and other users
Description	-	Allows the app to read all of the possible Teams interactions between the signed-in user and other users
AdminConsentRequired	-	Yes

TenantGovernance-Invitation.Read.All

Category	Application	Delegated
Identifier	3f4f98e9-6faf-4e5f-814b-ed2ed8a4ec9e	fda068e8-0524-485e-8d7f-b5bc29b0dae9
DisplayText	Read Tenant Governance invitations	Read Tenant Governance invitations
Description	Allows the application to list and read all Tenant Governance invitations without a signed-in user.	Allows the application to list and read all Tenant Governance invitations on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TenantGovernance-Invitation.ReadWrite.All

Category	Application	Delegated
Identifier	-	42b91635-3803-4af2-a2d5-e91127f9c488
DisplayText	-	Read and write Tenant Governance invitations
Description	-	Allows the application to list, read, create, and delete Tenant Governance invitations on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TenantGovernance-PolicyTemplate.Read.All

Category	Application	Delegated
Identifier	eb9465d8-e7c0-4301-8e51-927f34ee3134	ad222a15-813d-46b8-8f8d-1976a69a74f3
DisplayText	Read Tenant Governance policy templates	Read Tenant Governance policy templates
Description	Allows the application to list and read all Tenant Governance policy templates without a signed-in user.	Allows the application to list and read all Tenant Governance policy templates on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TenantGovernance-PolicyTemplate.ReadWrite.All

Category	Application	Delegated
Identifier	-	7cd0bd21-45fe-4c8e-a549-3c95bd27d185
DisplayText	-	Read and write Tenant Governance policy templates
Description	-	Allows the application to list, read, create, update, and delete Tenant Governance policy templates on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TenantGovernance-RelatedTenant.Read.All

Category	Application	Delegated
Identifier	7ced9a83-8e7c-46df-b3e0-6b45a6ecedcd	9caaca93-f090-4b9a-b4bb-17de251354d4
DisplayText	Read related tenants	Read related tenants
Description	Allows the application to list and read related tenants information without a signed-in user.	Allows the application to list and read related tenants information on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TenantGovernance-RelatedTenant.ReadWrite.All

Category	Application	Delegated
Identifier	-	e61db2de-de55-461e-942d-52a028ed1076
DisplayText	-	Read and write related tenants
Description	-	Allows the application to list, read, and refresh related tenants information on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TenantGovernance-Relationship.Read.All

Category	Application	Delegated
Identifier	41c250d0-8793-44e1-a130-5fdbd5bccd0a	0b1c2458-4845-477b-a704-3cce8b06bf28
DisplayText	Read Tenant Governance relationships	Read Tenant Governance relationships
Description	Allows the application to list and read all Tenant Governance relationships without a signed-in user.	Allows the application to list and read all Tenant Governance relationships on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TenantGovernance-Relationship.ReadWrite.All

Category	Application	Delegated
Identifier	-	3fbcd6a3-a9a5-4d69-8a78-acc7d7195180
DisplayText	-	Read and write Tenant Governance relationships
Description	-	Allows the application to list, read, and update Tenant Governance relationships on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TenantGovernance-Request.Read.All

Category	Application	Delegated
Identifier	294294d5-2b81-4cf1-837c-28fc22bc3290	a924b9f1-7af0-4982-aecf-b6e0e10b2830
DisplayText	Read Tenant Governance requests	Read Tenant Governance requests
Description	Allows the application to list and read all Tenant Governance requests without a signed-in user.	Allows the application to list and read all Tenant Governance requests on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TenantGovernance-Request.ReadWrite.All

Category	Application	Delegated
Identifier	-	3c7a434e-4e5d-413f-be82-b77ea4ba5a4d
DisplayText	-	Read and write Tenant Governance requests
Description	-	Allows the application to list, read, create, and update Tenant Governance requests on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TenantGovernance-Setting.Read.All

Category	Application	Delegated
Identifier	e2d1cac5-0317-4ae3-aca9-59737eb75317	4ad3e05f-2467-49d9-baa2-8e4de7bcee9b
DisplayText	Read Tenant Governance settings	Read Tenant Governance settings
Description	Allows the application to read Tenant Governance settings without a signed-in user.	Allows the application to read Tenant Governance settings on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TenantGovernance-Setting.ReadWrite.All

Category	Application	Delegated
Identifier	-	135f3533-12fc-4608-97ac-5c5cea64baf0
DisplayText	-	Read and write Tenant Governance settings
Description	-	Allows the application to read Tenant Governance settings and update them on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TermStore.Read.All

Category	Application	Delegated
Identifier	ea047cc2-df29-4f3e-83a3-205de61501ca	297f747b-0005-475b-8fef-c890f5152b38
DisplayText	Read all term store data	Read term store data
Description	Allows the app to read all term store data, without a signed-in user. This includes all sets, groups and terms in the term store.	Allows the app to read the term store data that the signed-in user has access to. This includes all sets, groups and terms in the term store.
AdminConsentRequired	Yes	Yes

TermStore.ReadWrite.All

Category	Application	Delegated
Identifier	f12eb8d6-28e3-46e6-b2c0-b7e4dc69fc95	6c37c71d-f50f-4bff-8fd3-8a41da390140
DisplayText	Read and write all term store data	Read and write term store data
Description	Allows the app to read, edit or write all term store data, without a signed-in user. This includes all sets, groups and terms in the term store.	Allows the app to read or modify data that the signed-in user has access to. This includes all sets, groups and terms in the term store.
AdminConsentRequired	Yes	Yes

ThreatAssessment.Read.All

Category	Application	Delegated
Identifier	f8f035bb-2cce-47fb-8bf5-7baf3ecbee48	-
DisplayText	Read threat assessment requests	-
Description	Allows an app to read your organization's threat assessment requests, without a signed-in user.	-
AdminConsentRequired	Yes	-

ThreatAssessment.ReadWrite.All

Category	Application	Delegated
Identifier	-	cac97e40-6730-457d-ad8d-4852fddab7ad
DisplayText	-	Read and write threat assessment requests
Description	-	Allows an app to read your organization's threat assessment requests on behalf of the signed-in user. Also allows the app to create new requests to assess threats received by your organization on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ThreatHunting.Read.All

Category	Application	Delegated
Identifier	dd98c7f5-2d42-42d3-a0e4-633161547251	b152eca8-ea73-4a48-8c98-1a6742673d99
DisplayText	Run hunting queries	Run hunting queries
Description	Allows the app to run hunting queries, without a signed-in user.	Allows the app to run hunting queries, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ThreatIndicators.Read.All

Category	Application	Delegated
Identifier	197ee4e9-b993-4066-898f-d6aecc55125b	9cc427b4-2004-41c5-aa22-757b755e9796
DisplayText	Read all threat indicators	Read all threat indicators
Description	Allows the app to read all the indicators for your organization, without a signed-in user.	Allows the app to read all the indicators for your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ThreatIndicators.ReadWrite.OwnedBy

Category	Application	Delegated
Identifier	21792b6c-c986-4ffc-85de-df9da54b52fa	91e7d36d-022a-490f-a748-f8e011357b42
DisplayText	Manage threat indicators this app creates or owns	Manage threat indicators this app creates or owns
Description	Allows the app to create threat indicators, and fully manage those threat indicators (read, update and delete), without a signed-in user. It cannot update any threat indicators it does not own.	Allows the app to create threat indicators, and fully manage those threat indicators (read, update and delete), on behalf of the signed-in user. It cannot update any threat indicators it does not own.
AdminConsentRequired	Yes	Yes

ThreatIntelligence.Read.All

Category	Application	Delegated
Identifier	e0b77adb-e790-44a3-b0a0-257d06303687	f266d9c0-ccb9-4fb8-a228-01ac0d8d6627
DisplayText	Read all Threat Intelligence Information	Read all threat intelligence information
Description	Allows the app to read threat intelligence information, such as indicators, observations, and and articles, without a signed in user.	Allows the app to read threat intelligence information, such as indicators, observations, and articles, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ThreatSubmission.Read

Category	Application	Delegated
Identifier	-	fd5353c6-26dd-449f-a565-c4e16b9fce78
DisplayText	-	Read threat submissions
Description	-	Allows the app to read the threat submissions and threat submission policies owned by the signed-in user.
AdminConsentRequired	-	Yes

ThreatSubmission.Read.All

Category	Application	Delegated
Identifier	86632667-cd15-4845-ad89-48a88e8412e1	7083913a-4966-44b6-9886-c5822a5fd910
DisplayText	Read all of the organization's threat submissions	Read all threat submissions
Description	Allows the app to read your organization's threat submissions and to view threat submission policies without a signed-in user.	Allows the app to read your organization's threat submissions and threat submission policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ThreatSubmission.ReadWrite

Category	Application	Delegated
Identifier	-	68a3156e-46c9-443c-b85c-921397f082b5
DisplayText	-	Read and write threat submissions
Description	-	Allows the app to read the threat submissions and threat submission policies owned by the signed-in user. Also allows the app to create new threat submissions on behalf of the signed-in user.
AdminConsentRequired	-	Yes

ThreatSubmission.ReadWrite.All

Category	Application	Delegated
Identifier	d72bdbf4-a59b-405c-8b04-5995895819ac	8458e264-4eb9-4922-abe9-768d58f13c7f
DisplayText	Read and write all of the organization's threat submissions	Read and write all threat submissions
Description	Allows the app to read your organization's threat submissions and threat submission policies without a signed-in user. Also allows the app to	Allows the app to read your organization's threat submissions and threat submission policies on behalf of the signed-in user. Also allows the app

Category	Application	Delegated
	create new threat submissions without a signed-in user.	to create new threat submissions on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

ThreatSubmissionPolicy.ReadWrite.All

Category	Application	Delegated
Identifier	926a6798-b100-4a20-a22f-a4918f13951d	059e5840-5353-4c68-b1da-666a033fc5e8
DisplayText	Read and write all of the organization's threat submission policies	Read and write all threat submission policies
Description	Allows the app to read your organization's threat submission policies without a signed-in user. Also allows the app to create new threat submission policies without a signed-in user.	Allows the app to read your organization's threat submission policies on behalf of the signed-in user. Also allows the app to create new threat submission policies on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Topic.Read.All

Category	Application	Delegated
Identifier	-	79c4c76f-409a-4f98-884d-e2c09291ec26
DisplayText	-	Read topic items
Description	-	Allows the app to read topics data on behalf of the signed-in user.
AdminConsentRequired	-	Yes

TrustFrameworkKeySet.Read.All

Category	Application	Delegated
Identifier	fff194f1-7dce-4428-8301-1badb5518201	7ad34336-f5b1-44ce-8682-31d7dfcd9ab9
DisplayText	Read trust framework key sets	Read trust framework key sets
Description	Allows the app to read trust framework key set properties without a signed-in user.	Allows the app to read trust framework key set properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

TrustFrameworkKeySet.ReadWrite.All

Category	Application	Delegated
Identifier	4a771c9a-1cf2-4609-b88e-3d3e02d539cd	39244520-1e7d-4b4a-aee0-57c65826e427
DisplayText	Read and write trust framework key sets	Read and write trust framework key sets
Description	Allows the app to read and write trust framework key set properties without a signed-in user.	Allows the app to read and write trust framework key set properties on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

UnifiedGroupMember.Read.AsGuest

Category	Application	Delegated
Identifier	-	73e75199-7c3e-41bb-9357-167164dbb415
DisplayText	-	Read unified group memberships as guest
Description	-	Allows the app to read basic unified group properties, memberships and owners of the group the signed-in guest is a member of.
AdminConsentRequired	-	Yes

User-ConvertToInternal.ReadWrite.All

Category	Application	Delegated
Identifier	9d952b72-f741-4b40-9185-8c53076c2339	550e695c-7511-40f4-ac79-e8fb9c82552d
DisplayText	Convert an external user to internal member user	Convert an external user to internal memeber user
Description	Allow the app to convert an external user to an internal member user, without a signed-in user.	Allow the app to convert an external user to an internal member user, on behalf of signed-in user.
AdminConsentRequired	Yes	Yes

User-LifeCycleInfo.Read.All

Category	Application	Delegated
Identifier	8556a004-db57-4d7a-8b82-97a13428e96f	ed8d2a04-0374-41f1-aefe-da8ac87ccc87
DisplayText	Read all users' lifecycle information	Read all users' lifecycle information
Description	Allows the app to read the lifecycle information like employeeLeaveDateTime of users in your organization, without a signed-in user.	Allows the app to read the lifecycle information like employeeLeaveDateTime of users in your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User-LifeCycleInfo.ReadWrite.All

Category	Application	Delegated
Identifier	925f1248-0f97-47b9-8ec8-538c54e01325	7ee7473e-bd4b-4c9f-987c-bd58481f5fa2
DisplayText	Read and write all users' lifecycle information	Read and write all users' lifecycle information
Description	Allows the app to read and write the lifecycle information like employeeLeaveDateTime of users in your organization, without a signed-in user.	Allows the app to read and write the lifecycle information like employeeLeaveDateTime of users in your organization, on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

User-Mail.ReadWrite.All

Category	Application	Delegated
Identifier	280d0935-0796-47d1-8d26-273470a3f17a	6166886a-9576-433b-8544-658177bdef1d
DisplayText	Read and write all secondary mail addresses for users	Read and write secondary mail addresses for users
Description	Allows the app to read and write secondary mail addresses for all users, without a signed-in user.	Allows the app to read and write secondary mail addresses for all users, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User-OnPremisesSyncBehavior.ReadWrite.All

Category	Application	Delegated
Identifier	a94a502d-0281-4d15-8cd2-682ac9362c4c	7ff9afdd-0cdb-439d-a61c-fea3e9339e89
DisplayText	Read and update the on-premises sync behavior of users	Read and update the on-premises sync behavior of users
Description	Allows the app to update the on-premises sync behavior of all users without a signed-in user.	Allows the app to read and update the on-premises sync behavior of users on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User-PasswordProfile.ReadWrite.All

Category	Application	Delegated
Identifier	cc117bb9-00cf-4eb8-b580-ea2a878fe8f7	56760768-b641-451f-8906-e1b8ab31bca7

Category	Application	Delegated
DisplayText	Read and write all password profiles and reset user passwords	Read and write password profiles and reset user passwords
Description	Allows the app to read and write password profiles and reset passwords for all users, without a signed-in user.	Allows the app to read and write password profiles and reset passwords for all users, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User-Phone.ReadWrite.All

Category	Application	Delegated
Identifier	86ceff06-c822-49ff-989a-d912845ffe69	e29d5979-5b06-4a7f-ae24-6a9348d2e1ff
DisplayText	Read and write all user mobile phone and business phones	Read and write user mobile phone and business phones
Description	Allows the app to read and write the mobile phone and business phones for all users, without a signed-in user.	Allows the app to read and write the mobile phone and business phones for all users, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User.Create

Category	Application	Delegated
Identifier	4240f680-4f73-4082-a766-aa916a2dc9b3	d8ce6a2a-46ff-438e-96bc-020f23870a55
DisplayText	Create users	Create users
Description	Allows the app to create users, without a signed-in user.	Allows the app to create users, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User.DeleteRestore.All

Category	Application	Delegated
Identifier	eccc023d-eccf-4e7b-9683-8813ab36cecc	4bb440cd-2cf2-4f90-8004-aa2acd2537c5
DisplayText	Delete and restore all users	Delete and restore users
Description	Allows the app to delete and restore all users, without a signed-in user.	Allows the app to delete and restore all users, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User.EnableDisableAccount.All

Category	Application	Delegated
Identifier	3011c876-62b7-4ada-afa2-506cbbecc68c	f92e74e7-2563-467f-9dd0-902688cb5863
DisplayText	Enable and disable user accounts	Enable and disable user accounts
Description	Allows the app to enable and disable users' accounts, without a signed-in user.	Allows the app to enable and disable users' accounts, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User.Export.All

Category	Application	Delegated
Identifier	405a51b5-8d8d-430b-9842-8be4b0e9f324	405a51b5-8d8d-430b-9842-8be4b0e9f324
DisplayText	Export user's data	Export user's data
Description	Allows the app to export data (e.g. customer content or system-generated logs), associated with any user in your company, when the app is used by a privileged user (e.g. a Company Administrator).	Allows the app to export data (e.g. customer content or system-generated logs), associated with any user in your company, when the app is used by a privileged user (e.g. a Company Administrator).
AdminConsentRequired	Yes	Yes

User.Invite.All

Category	Application	Delegated
Identifier	09850681-111b-4a89-9bed-3f2cae46d706	63dd7cd9-b489-4adf-a28c-ac38b9a0f962
DisplayText	Invite guest users to the organization	Invite guest users to the organization
Description	Allows the app to invite guest users to the organization, without a signed-in user.	Allows the app to invite guest users to the organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User.ManageIdentities.All

Category	Application	Delegated
Identifier	c529cfca-c91b-489c-af2b-d92990b66ce6	637d7bec-b31e-4deb-acc9-24275642a2c9
DisplayText	Manage all users' identities	Manage user identities
Description	Allows the app to read, update and delete identities that are associated with a user's account, without a signed in user. This controls the identities users can sign-in with.	Allows the app to read, update and delete identities that are associated with a user's account that the signed-in user has access to. This controls the identities users can sign-in with.
AdminConsentRequired	Yes	Yes

User.Read

Category	Application	Delegated
Identifier	-	e1fe6dd8-ba31-4d61-89e7-88639da4683d
DisplayText	-	Sign in and read user profile
Description	-	Allows users to sign-in to the app, and allows the app to read the profile of signed-in users. It also allows the app to read basic company information of signed-in users.
AdminConsentRequired	-	No

The *User.Read* delegated permission is available for consent in personal Microsoft accounts.

The *User.Read* permission also allows an app to read the basic company information of the signed-in user for a work or school account through the organization resource. Information in the following properties is available: **id**, **displayName**, and **verifiedDomains**.

User.Read.All

Category	Application	Delegated
Identifier	df021288-bdef-4463-88db-98f22de89214	a154be20-db9c-4678-8ab7-66f6cc099a59
DisplayText	Read all users' full profiles	Read all users' full profiles
Description	Allows the app to read user profiles without a signed in user.	Allows the app to read the full set of profile properties, reports, and managers of other users in your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

The *User.Read.All* delegated permission is available for consent in personal Microsoft accounts.

User.ReadBasic.All

Category	Application	Delegated
Identifier	97235f07-e226-4f63-ace3-39588e11d3a1	b340eb25-3456-403f-be2f-af7a0d370277
DisplayText	Read all users' basic profiles	Read all users' basic profiles
Description	Allows the app to read a basic set of profile properties of other users in your organization without a signed-in user. Includes display name, first and last name, email address, open extensions, and photo.	Allows the app to read a basic set of profile properties of other users in your organization on behalf of the signed-in user. This includes display name, first and last name, email address and photo.
AdminConsentRequired	Yes	No

The *User.ReadBasic.All* delegated permission is available for consent in personal Microsoft accounts.

The *User.ReadBasic.All* permission constrains app access to reading a limited set of properties for other users' work or school accounts. This basic profile includes only the following properties:

- displayName
- givenName
- id
- mail
- photo
- securityIdentifier
- surname
- userPrincipalName

User.ReadUpdate.All

Category	Application	Delegated
Identifier	5639c449-cfd9-4088-bc48-3e16da108bf8	f8b099f1-a6ce-4e00-8fff-5f783ff4faeb
DisplayText	Read and update users	Read and update users
Description	Allows the app to read and update users, without a signed-in user.	Allows the app to read and update users, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

User.ReadWrite

Category	Application	Delegated
Identifier	-	b4e74841-8e56-480b-be8b-910348b18b4c
DisplayText	-	Read and write access to user profile
Description	-	Allows the app to read your profile. It also allows the app to update your profile information on your behalf.
AdminConsentRequired	-	No

The *User.ReadWrite* delegated permission is available for consent in personal Microsoft accounts.

The *User.ReadWrite* delegated permission allow the app to update the following profile properties for the signed-in user's work or school account:

- aboutMe
- birthday
- hireDate
- interests
- mobilePhone
- mySite
- pastProjects
- photo
- preferredName
- responsibilities
- schools
- skills

User.ReadWrite.All

Category	Application	Delegated
Identifier	741f803b-c850-494e-b5df-cde7c675a1ca	204e0828-b5ca-4ad8-b9f3-f32a958e7cc4
DisplayText	Read and write all users' full profiles	Read and write all users' full profiles
Description	Allows the app to read and update user profiles without a signed in user.	Allows the app to read and write the full set of profile properties, reports, and managers of other users in your organization, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

The *User.ReadWrite.All* delegated permission is available for consent in personal Microsoft accounts.

The *User.ReadWrite.All* delegated and application permissions allow the app to update all the declared properties for a user's work or school account except for their **passwordProfile** and **employeeLeaveDateTime**.

Updating sensitive properties is only allowed on non-admin users and users with lesser-privileged admin roles as indicated in Who can perform sensitive actions.

User.ReadWrite.CrossCloud

Category	Application	Delegated
Identifier	5652f862-b626-407b-a3e6-248aeb95763c	-
DisplayText	Read and write profiles of users that originate from an external cloud.	-
Description	Allows the app to read and update external cloud user profiles without a signed in user.	-
AdminConsentRequired	Yes	-

User.RevokeSessions.All

Category	Application	Delegated
Identifier	77f3a031-c388-4f99-b373-dc68676a979e	fc30e98b-8810-4501-81f5-c20a3196387b
DisplayText	Revoke all sign in sessions for a user	Revoke all sign in sessions for a user
Description	Allow the app to revoke all sign in sessions for a user, without a signed-in user.	Allow the app to revoke all sign in sessions for a user, on behalf of a signed-in user.
AdminConsentRequired	Yes	Yes

UserActivity.ReadWrite.CreatedByApp

Category	Application	Delegated
Identifier	-	47607519-5fb1-47d9-99c7-da4b48f369b1
DisplayText	-	Read and write app activity to users' activity feed
Description	-	Allows the app to read and report the signed-in user's activity in the app.
AdminConsentRequired	-	No

The *UserActivity.ReadWrite.CreatedByApp* delegated permission is available for consent in personal Microsoft accounts.

UserAuthenticationMethod.Delete.All

Category	Application	Delegated
Identifier	712f5e0d-bc8d-4ae5-8242-cfb9a4921ed3	-
DisplayText	Delete all users' authentication methods	-
Description	Allows the app to delete authentication methods of all users in your organization, without a signed-in user. Authentication methods include things like a user's phone numbers and Authenticator app settings. This does not allow the app to read, create, or modify authentication methods.	-
AdminConsentRequired	Yes	-

UserAuthenticationMethod.Read

Category	Application	Delegated
Identifier	-	1f6b61c5-2f65-4135-9c9f-31c0f8d32b52
DisplayText	-	Read user authentication methods.
Description	-	Allows the app to read the signed-in user's authentication methods, including phone numbers and Authenticator app settings. This does not allow the app to see secret information like the signed-in user's passwords, or to sign-in or otherwise use the signed-in user's authentication methods.
AdminConsentRequired	-	Yes

UserAuthenticationMethod.Read.All

Category	Application	Delegated
Identifier	38d9df27-64da-44fd-b7c5-a6fbac20248f	aec28ec7-4d02-4e8c-b864-50163aea77eb
DisplayText	Read all users' authentication methods	Read all users' authentication methods
Description	Allows the app to read authentication methods of all users in your organization, without a signed-in user.	Allows the app to read authentication methods of all users in your organization that the signed-in user has

Category	Application	Delegated
	Authentication methods include things like a user's phone numbers and Authenticator app settings. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	access to. Authentication methods include things like a user's phone numbers and Authenticator app settings. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthenticationMethod.ReadWrite

Category	Application	Delegated
Identifier	-	48971fc1-70d7-4245-af77-0beb29b53ee2
DisplayText	-	Read and write user authentication methods
Description	-	Allows the app to read and write the signed-in user's authentication methods, including phone numbers and Authenticator app settings. This does not allow the app to see secret information like the signed-in user's passwords, or to sign-in or otherwise use the signed-in user's authentication methods.
AdminConsentRequired	-	Yes

UserAuthenticationMethod.ReadWrite.All

Category	Application	Delegated
Identifier	50483e42-d915-4231-9639-7fdb7fd190e5	b7887744-6746-4312-813d-72daeaee7e2d
DisplayText	Read and write all users' authentication methods	Read and write all users' authentication methods
Description	Allows the application to read and write authentication methods of all users in your organization, without a signed-in user. Authentication methods include things like a user's phone numbers and Authenticator app settings. This does not allow the app to see secret information like passwords,	Allows the app to read and write authentication methods of all users in your organization that the signed-in user has access to. Authentication methods include things like a user's phone numbers and Authenticator app settings. This does not allow the app to see secret information like passwords,

Category	Application	Delegated
	or to sign-in or otherwise use the authentication methods	or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-Email.Delete.All

Category	Application	Delegated
Identifier	f0e9adfd-ed6b-45f5-b969-324a75286a39	-
DisplayText	Delete all users' email methods	-
Description	Allows the application to delete email methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify email methods.	-
AdminConsentRequired	Yes	-

UserAuthMethod-Email.Read

Category	Application	Delegated
Identifier	-	12b23cea-90c1-4873-9094-f45c5f290f86
DisplayText	-	Read the signed-in user's email authentication methods
Description	-	Allows the app to read the signed-in user's email authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-Email.Read.All

Category	Application	Delegated
Identifier	a1e58be0-1095-422b-b067-73434bd7d40f	76caaf3a-ebdb-40a3-9299-4196e636f290
DisplayText	Read all users' email methods	Read all users' email methods

Category	Application	Delegated
Description	Allows the app to read email methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read email methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-Email.ReadWrite

Category	Application	Delegated
Identifier	-	696aa421-62dc-4c99-be16-015b23444089
DisplayText	-	Read and write the signed-in user's email authentication methods
Description	-	Allows the app to read and write the signed-in user's email authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-Email.ReadWrite.All

Category	Application	Delegated
Identifier	e8ecb853-1435-4a49-95ba-ec5b31b11672	074f680f-c89e-45be-880e-5d0642860a1c
DisplayText	Read and write all users' email methods	Read and write all users' email methods.
Description	Allows the application to read and write email methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read and write email methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.

Category	Application	Delegated
AdminConsentRequired	Yes	Yes

UserAuthMethod-External.Delete.All

Category	Application	Delegated
Identifier	7fa6d39e-1e4e-44be-bf9c-e8260b12e1f5	-
DisplayText	Delete all users' external authentication methods	-
Description	Allows the application to delete external authentication methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify external authentication methods.	-
AdminConsentRequired	Yes	-

UserAuthMethod-External.Read

Category	Application	Delegated
Identifier	-	d1739827-146b-4f7f-b52c-1c509253aa57
DisplayText	-	Read the signed-in user's external authentication methods
Description	-	Allows the app to read the signed-in user's external authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-External.Read.All

Category	Application	Delegated
Identifier	d2c4289f-9f95-40da-ad43-eeb1506f0db7	cbca9646-4c34-4cea-8e54-9a7088018820
DisplayText	Read all users' external authentication methods	Read all users' external authentication methods

Category	Application	Delegated
Description	Allows the app to read external authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read external authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-External.ReadWrite

Category	Application	Delegated
Identifier	-	28c2e8f9-828a-4691-a090-f2f0b7fc07b3
DisplayText	-	Read and write the signed-in user's external authentication methods
Description	-	Allows the app to read and write the signed-in user's external authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-External.ReadWrite.All

Category	Application	Delegated
Identifier	c7a22c2e-5b01-4129-8159-6c8be2c78f16	9d91805d-0f53-43e3-a0f3-303ad4f3056f
DisplayText	Read and write all users' external authentication methods	Read and write all users' external methods.
Description	Allows the application to read and write external authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or	Allows the app to read and write external authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-

Category	Application	Delegated
	otherwise use the authentication methods.	in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-HardwareOATH.Delete.All

Category	Application	Delegated
Identifier	9d8eb432-7ea3-491a-9ed7-e6361b308f08	-
DisplayText	Delete all users' HardwareOATH authentication methods	-
Description	Allows the application to delete HardwareOATH authentication methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify HardwareOATH authentication methods.	-
AdminConsentRequired	Yes	-

UserAuthMethod-HardwareOATH.Read

Category	Application	Delegated
Identifier	-	ccd2eb40-8874-44e6-8f96-335908b3cfdb
DisplayText	-	Read the signed-in user's HardwareOATH authentication methods
Description	-	Allows the app to read the signed-in user's HardwareOATH authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-HardwareOATH.Read.All

Category	Application	Delegated
Identifier	7b544555-7811-49ff-8223-a56be870e33a	acd68c26-c283-4bf4-8b5c-200fc179bdd5

Category	Application	Delegated
DisplayText	Read all users' HardwareOATH authentication methods	Read all users' HardwareOATH authentication methods
Description	Allows the app to read HardwareOATH authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read HardwareOATH authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-HardwareOATH.ReadWrite

Category	Application	Delegated
Identifier	-	147ca97b-6686-4849-b37e-09d9b5ad45fc
DisplayText	-	Read and write the signed-in user's HardwareOATH authentication methods
Description	-	Allows the app to read and write the signed-in user's HardwareOATH authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-HardwareOATH.ReadWrite.All

Category	Application	Delegated
Identifier	7e9ebcc1-90aa-4471-8051-e68d6b4e9c89	480643f2-a162-43c5-a670-dc1494fc911b
DisplayText	Read and write all users' HardwareOATH authentication methods	Read and write all users' HardwareOATH methods.
Description	Allows the application to read and write HardwareOATH authentication methods of all users in your organization, without a signed-in user.	Allows the app to read and write HardwareOATH authentication methods of all users in your organization that the signed-in user has access to. This does

Category	Application	Delegated
	This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-MicrosoftAuthApp.Delete.All

Category	Application	Delegated
Identifier	ae494ca6-9612-417a-972a-ef52efaf2de3	-
DisplayText	Delete all users' Microsoft Authentication methods	-
Description	Allows the application to delete Microsoft Authentication methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify Microsoft Authentication methods.	-
AdminConsentRequired	Yes	-

UserAuthMethod-MicrosoftAuthApp.Read

Category	Application	Delegated
Identifier	-	f14a567b-3280-4124-95a0-eca86006967e
DisplayText	-	Read the signed-in user's Microsoft Authenticator authentication methods
Description	-	Allows the app to read the signed-in user's Microsoft Authenticator authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-MicrosoftAuthApp.Read.All

Category	Application	Delegated
Identifier	a9c5f16e-e5ca-4e33-89ad-903fcfc01c23	7b627679-e2fd-4bfd-990e-989e2914d4e6
DisplayText	Read all users' Microsoft authentication methods	Read all users' Microsoft authentication methods
Description	Allows the app to read Microsoft authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read Microsoft authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-MicrosoftAuthApp.ReadWrite

Category	Application	Delegated
Identifier	-	9f7dfa0c-eb40-42be-8d45-8af4a9219c6f
DisplayText	-	Read and write the signed-in user's Microsoft Authenticator authentication methods
Description	-	Allows the app to read and write the signed-in user's Microsoft Authenticator authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-MicrosoftAuthApp.ReadWrite.All

Category	Application	Delegated
Identifier	c833c349-a1ab-4b6d-94a2-fa9a8674420c	1b7322b2-5cb3-4f13-928f-d7ca97c5fba9
DisplayText	Read and write all users' Microsoft Authentication methods	Read and write all users' Microsoft Authentication methods.
Description	Allows the application to read and write Microsoft Authentication	Allows the app to read and write Microsoft Authentication methods of all

Category	Application	Delegated
	methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-Passkey.Delete.All

Category	Application	Delegated
Identifier	9563fbd0-03a7-466e-8042-63d668b7d1a3	-
DisplayText	Delete all users' passkey authentication methods	-
Description	Allows the application to delete passkey authentication methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify passkey authentication methods.	-
AdminConsentRequired	Yes	-

UserAuthMethod-Passkey.Read

Category	Application	Delegated
Identifier	-	828fcbda-0d26-431d-8bfb-83f217224621
DisplayText	-	Read the signed-in user's passkey authentication methods
Description	-	Allows the app to read the signed-in user's passkey authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-Passkey.Read.All

Category	Application	Delegated
Identifier	72e00c1d-3e3d-43bb-a0b9-c435611bb1d2	14195339-1fe4-48a7-a0d3-a39eb9fd8958
DisplayText	Read all users' passkey authentication methods	Read all users' passkey authentication methods
Description	Allows the app to read passkey authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read passkey authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-Passkey.ReadWrite

Category	Application	Delegated
Identifier	-	b2de7db9-10f7-4800-b04c-b5b91e4891d6
DisplayText	-	Read and write the signed-in user's passkey authentication methods
Description	-	Allows the app to read and write the signed-in user's passkey authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-Passkey.ReadWrite.All

Category	Application	Delegated
Identifier	0400e371-7db1-4338-a269-96069eb65227	64930478-d0ea-4671-ad72-fe0d9821df09
DisplayText	Read and write all users' passkey authentication methods	Read and write all users' passkey methods.
Description	Allows the application to read and write passkey authentication methods	Allows the app to read and write passkey authentication methods of all

Category	Application	Delegated
	of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods	users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-Password.Read

Category	Application	Delegated
Identifier	-	7f0f82c3-de19-4ddc-810d-a2206d7637fd
DisplayText	-	Read the signed-in user's password authentication methods
Description	-	Allows the app to read the signed-in user's password authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	No

UserAuthMethod-Password.Read.All

Category	Application	Delegated
Identifier	8d2c17ff-b93d-40d5-9def-d843680509cb	4f69a4e2-2aa0-43a7-ad6b-98b4cda1f23f
DisplayText	Read all users' password authentication methods	Read all users' password authentication methods
Description	Allows the app to read password authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read password authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	No

UserAuthMethod-Password.ReadWrite

Category	Application	Delegated
Identifier	-	60cce20d-d41e-4594-b391-84bbf8cc31f3
DisplayText	-	Read and write the signed-in user's password authentication methods
Description	-	Allows the app to read and write the signed-in user's password authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	No

UserAuthMethod-Password.ReadWrite.All

Category	Application	Delegated
Identifier	f6d38dfd-ec08-4995-8f07-23e929df0936	7f5b683d-df96-4690-a88d-6e336ed6dc7c
DisplayText	Read and write all users' password authentication methods	Read and write all users' password methods.
Description	Allows the application to read and write password authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read and write password authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	No

UserAuthMethod-Phone.Delete.All

Category	Application	Delegated
Identifier	59f17651-8b6c-494e-a269-4ac582fbbca0	-
DisplayText	Delete all users' phone methods	-

Category	Application	Delegated
Description	Allows the application to delete phone methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify phone methods.	-
AdminConsentRequired	Yes	-

UserAuthMethod-Phone.Read

Category	Application	Delegated
Identifier	-	43dab3b9-e8b4-424d-8e13-6a2ad2a625fa
DisplayText	-	Read the signed-in user's phone authentication methods
Description	-	Allows the app to read the signed-in user's phone authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	No

UserAuthMethod-Phone.Read.All

Category	Application	Delegated
Identifier	f529a223-ea70-43ec-b268-5012de2fbaa2	20cf4ae1-09b9-4d29-a6f8-43e1820ce60c
DisplayText	Read all users' phone authentication methods	Read all users' phone authentication methods
Description	Allows the app to read phone authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read phone authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	No

UserAuthMethod-Phone.ReadWrite

Category	Application	Delegated
Identifier	-	6c4aad61-f76b-46ad-a22c-57d4d3d962af
DisplayText	-	Read and write the signed-in user's phone authentication methods
Description	-	Allows the app to read and write the signed-in user's phone authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	No

UserAuthMethod-Phone.ReadWrite.All

Category	Application	Delegated
Identifier	6e85d483-7092-4375-babe-0a94a8213a58	48c99302-9a24-4f27-a8a7-acef4debba14
DisplayText	Read and write all users' phone methods	Read and write all users' phone methods.
Description	Allows the application to read and write phone methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read and write Phone methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	No

UserAuthMethod-PlatformCred.Delete.All

Category	Application	Delegated
Identifier	bd760918-651f-4e67-b66f-8f614384dec2	-
DisplayText	Delete all users' platform credentials methods	-
Description	Allows the application to delete platform credentials methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify platform credentials methods.	-

Category	Application	Delegated
AdminConsentRequired	Yes	-

UserAuthMethod-PlatformCred.Read

Category	Application	Delegated
Identifier	-	9c694582-e8f2-40e2-8353-fb43e2e0f12a
DisplayText	-	Read the signed-in user's platform credential authentication methods
Description	-	Allows the app to read the signed-in user's platform credential authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-PlatformCred.Read.All

Category	Application	Delegated
Identifier	07c0b1e4-15bd-442f-834b-30f8291388d1	5936156c-f89b-4850-997d-026c4e6ce529
DisplayText	Read all users' platform credentials methods	Read all users' platform credentials methods
Description	Allows the app to read platform credentials methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read platform credentials methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-PlatformCred.ReadWrite

Category	Application	Delegated
Identifier	-	70327f81-b953-43c9-92d3-131c74e4beb8
DisplayText	-	Read and write the signed-in user's platform credential authentication methods
Description	-	Allows the app to read and write the signed-in user's platform credential authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-PlatformCred.ReadWrite.All

Category	Application	Delegated
Identifier	1a87acf4-a9ca-4576-a974-452ea265d5f6	cb11bf8c-dde1-4504-b6a5-31e1562b0749
DisplayText	Read and write all users' platform credentials methods	Read and write all users' platform credentials methods.
Description	Allows the application to read and write platform credentials methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read and write platform credentials methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-QR.Delete.All

Category	Application	Delegated
Identifier	e1c34213-26ac-400b-9548-a749f1b1a4e0	-
DisplayText	Delete all users' QR authentication methods	-
Description	Allows the application to delete QR authentication methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify QR authentication methods.	-

Category	Application	Delegated
AdminConsentRequired	Yes	-

UserAuthMethod-QR.Read

Category	Application	Delegated
Identifier	-	d6893c31-9187-405c-8dfc-f700c8fc161a
DisplayText	-	Read the signed-in user's QR authentication methods
Description	-	Allows the app to read the signed-in user's QR authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-QR.Read.All

Category	Application	Delegated
Identifier	9a45bc50-cddd-4ebe-bd9c-4f2eacf646ae	e4900dfb-ad17-410d-8ddb-7aebd8a6af1a
DisplayText	Read all users' QR methods	Read all users' QR methods
Description	Allows the app to read QR authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read QR authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-QR.ReadWrite

Category	Application	Delegated
Identifier	-	651210da-18ce-4e42-b7db-302ff88e9326

Category	Application	Delegated
DisplayText	-	Read and write the signed-in user's QR authentication methods
Description	-	Allows the app to read and write the signed-in user's QR authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-QR.ReadWrite.All

Category	Application	Delegated
Identifier	4869299f-18c3-40c8-98f2-222657e67db1	db39086a-da7d-4cbd-9ac0-6816f9a80c95
DisplayText	Read and write all users' QR methods	Read and write all users' QR methods.
Description	Allows the application to read and write QR authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read and write QR authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-ResourceKey.Delete.All

Category	Application	Delegated
Identifier	a71aecaf-82f1-47c5-ad0a-5e63503b928f	-
DisplayText	Delete all users' resource key authentication methods	-
Description	Allows the application to delete resource key authentication methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify resource key authentication methods.	-
AdminConsentRequired	Yes	-

UserAuthMethod-ResourceKey.Read.All

Category	Application	Delegated
Identifier	94ed018c-a499-47e0-beef-803b93873ece	c86e40fc-66fd-4d68-802e-b90e3038f5e8
DisplayText	Read and write all users' external authentication methods	Read the keys associated with the user representing a resource account.
Description	Allows the app to read the keys associated with the user representing a resource account.	Allows the app to read the keys associated with the user representing a resource account.
AdminConsentRequired	Yes	Yes

UserAuthMethod-ResourceKey.ReadWrite.All

Category	Application	Delegated
Identifier	1bf7461f-222a-4525-9760-f7739228d0f4	e71dec0d-02b4-429d-a49a-030950d45faa
DisplayText	Read and delete all users' external authentication methods	Read and delete the keys associated with the user representing a resource account.
Description	Allows the app to read and delete the keys associated with the user representing a resource account.	Allows the app to read and delete the keys associated with the user representing a resource account.
AdminConsentRequired	Yes	Yes

UserAuthMethod-SoftwareOATH.Delete.All

Category	Application	Delegated
Identifier	e5676e10-1a16-452b-ad10-71f54b755852	-
DisplayText	Delete all users' SoftwareOATH authentication methods	-
Description	Allows the application to delete SoftwareOATH authentication methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify SoftwareOATH authentication methods.	-

Category	Application	Delegated
AdminConsentRequired	Yes	-

UserAuthMethod-SoftwareOATH.Read

Category	Application	Delegated
Identifier	-	247f2733-6e3d-46ff-a904-f5fd58eb0d97
DisplayText	-	Read the signed-in user's SoftwareOATH authentication methods
Description	-	Allows the app to read the signed-in user's SoftwareOATH authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-SoftwareOATH.Read.All

Category	Application	Delegated
Identifier	a6b423df-a0c8-411d-a809-a4a5985d2939	3e366fa0-3097-4eb6-8294-3028f77eea6f
DisplayText	Read all users' SoftwareOATH methods	Read all users' SoftwareOATH methods
Description	Allows the app to read SoftwareOATH authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read SoftwareOATH authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-SoftwareOATH.ReadWrite

Category	Application	Delegated
Identifier	-	16721eb3-4493-4ae1-9542-264d9ffe3ce9

Category	Application	Delegated
DisplayText	-	Read and write the signed-in user's SoftwareOATH authentication methods
Description	-	Allows the app to read and write the signed-in user's SoftwareOATH authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-SoftwareOATH.ReadWrite.All

Category	Application	Delegated
Identifier	787442d4-3c6e-4e99-aa95-8ccca20a48ff	5b34c8b5-2396-4b35-b284-83fb6a3e73ce
DisplayText	Read and write all users' SoftwareOATH methods	Read and write all users' SoftwareOATH methods.
Description	Allows the application to read and write SoftwareOATH authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read and write SoftwareOATH authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-TAP.Delete.All

Category	Application	Delegated
Identifier	4f872e9d-d232-4ecd-ab9c-337cbdb184e5	-
DisplayText	Delete all users' Temporary Access Pass authentication methods	-
Description	Allows the application to delete Temporary Access Pass authentication methods of all users in your organization, without a signed-in user. This does not allow the app to read, create, or modify Temporary Access Pass authentication methods.	-

Category	Application	Delegated
AdminConsentRequired	Yes	-

UserAuthMethod-TAP.Read

Category	Application	Delegated
Identifier	-	84ded88f-26ba-49d6-b776-efec398de692
DisplayText	-	Read the signed-in user's Temporary Access Pass authentication methods
Description	-	Allows the app to read the signed-in user's Temporary Access Pass authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-TAP.Read.All

Category	Application	Delegated
Identifier	bf82209c-b22b-4747-ac88-a68be99032cf	6976c635-c9c2-41e6-a21d-e6913a155273
DisplayText	Read all users' Temporary Access Pass methods	Read all users' Temporary Access Pass methods
Description	Allows the app to read Temporary Access Pass authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read Temporary Access Pass authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-TAP.ReadWrite

Category	Application	Delegated
Identifier	-	2424436d-902f-4651-a1c7-b3b93147c960
DisplayText	-	Read and write the signed-in user's Temporary Access Pass authentication methods
Description	-	Allows the app to read and write the signed-in user's Temporary Access Pass authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-TAP.ReadWrite.All

Category	Application	Delegated
Identifier	627169a8-8c15-451c-861a-5b80e383de5c	05de4a66-e51a-4312-842a-30c8094698d2
DisplayText	Read and write all users' Temporary Access Pass methods	Read and write all users' Temporary Access Pass methods.
Description	Allows the application to read and write Temporary Access Pass authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read and write Temporary Access Pass authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-WindowsHello.Delete.All

Category	Application	Delegated
Identifier	f3197110-aa7f-4acd-a0fd-71981ad68d42	-
DisplayText	Delete all users' Windows Hello authentication methods	-
Description	Allows the application to delete Windows Hello authentication methods of all users in your organization, without a signed-in user.	-

Category	Application	Delegated
	This does not allow the app to read, create, or modify Windows Hello authentication methods.	
AdminConsentRequired	Yes	-

UserAuthMethod-WindowsHello.Read

Category	Application	Delegated
Identifier	-	efe2b5aa-3a8e-486c-b0be-cc4d185c1b40
DisplayText	-	Read the signed-in user's Windows Hello methods
Description	-	Allows the app to read the signed-in user's Windows Hello authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-WindowsHello.Read.All

Category	Application	Delegated
Identifier	9b8dd4c7-8cca-4ef5-a34a-9c2c75fcc934	ff37d46d-b88a-4e0c-85ee-7e26c37b18eb
DisplayText	Read all users' Windows Hello methods	Read all users' Windows Hello methods
Description	Allows the app to read Windows Hello authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read Windows Hello authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserAuthMethod-WindowsHello.ReadWrite

Category	Application	Delegated
Identifier	-	f11e1db9-d419-4a24-b677-792723ffd727
DisplayText	-	Read and write the signed-in user's Windows Hello authentication methods
Description	-	Allows the app to read and write the signed-in user's Windows Hello authentication methods. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	-	Yes

UserAuthMethod-WindowsHello.ReadWrite.All

Category	Application	Delegated
Identifier	f14eee8a-713e-45aa-8223-2ab74632db1a	13eae17d-aaa4-47b8-aaee-0eb33c6e2450
DisplayText	Read and write all users' Windows Hello authentication methods	Read and write all users' Windows Hello methods.
Description	Allows the application to read and write Windows Hello authentication methods of all users in your organization, without a signed-in user. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.	Allows the app to read and write Windows Hello authentication methods of all users in your organization that the signed-in user has access to. This does not allow the app to see secret information like passwords, or to sign-in or otherwise use the authentication methods.
AdminConsentRequired	Yes	Yes

UserCloudClipboard.Read

Category	Application	Delegated
Identifier	-	61e8a09a-087f-4e36-8c8c-1c77c5228017
DisplayText	-	Read cloud clipboard items
Description	-	Allows the app to read cloud clipboard data on behalf of the signed-in user.

Category	Application	Delegated
AdminConsentRequired	-	No

UserNotification.ReadWrite.CreatedByApp

Category	Application	Delegated
Identifier	4e774092-a092-48d1-90bd-baad67c7eb47	26e2f3e8-b2a1-47fc-9620-89bb5b042024
DisplayText	Deliver and manage all user's notifications	Deliver and manage user's notifications
Description	Allows the app to send, read, update and delete user's notifications, without a signed-in user.	Allows the app to send, read, update and delete user's notifications.
AdminConsentRequired	Yes	No

UserShiftPreferences.Read.All

Category	Application	Delegated
Identifier	de023814-96df-4f53-9376-1e2891ef5a18	-
DisplayText	Read all user shift preferences	-
Description	Allows the app to read all users' shift schedule preferences without a signed-in user.	-
AdminConsentRequired	Yes	-

UserShiftPreferences.ReadWrite.All

Category	Application	Delegated
Identifier	d1eec298-80f3-49b0-9efb-d90e224798ac	-
DisplayText	Read and write all user shift preferences	-
Description	Allows the app to manage all users' shift schedule preferences without a signed-in user.	-

Category	Application	Delegated
AdminConsentRequired	Yes	-

UserTeamwork.Read

Category	Application	Delegated
Identifier	-	834bcc1c-762f-41b0-bb91-1cdc323ee4bf
DisplayText	-	Read user teamwork settings
Description	-	Allows the app to read the teamwork settings of the signed-in user.
AdminConsentRequired	-	Yes

UserTeamwork.Read.All

Category	Application	Delegated
Identifier	fbcd7ef1-df0d-4e05-bb28-93424a89c6df	-
DisplayText	Read all user teamwork settings	-
Description	Allows the app to read all user teamwork settings without a signed-in user.	-
AdminConsentRequired	Yes	-

UserTimelineActivity.Write.CreatedByApp

Category	Application	Delegated
Identifier	-	367492fc-594d-4972-a9b5-0d58c622c91c
DisplayText	-	Write app activity to users' timeline
Description	-	Allows the app to report the signed-in user's app activity information to Microsoft Timeline.
AdminConsentRequired	-	No

UserWindowsSettings.Read.All

Category	Application	Delegated
Identifier	-	77e07bab-1b34-40a5-bb6c-4b197b3f6027
DisplayText	-	Read windows settings for all devices
Description	-	Allows the app to read a user's windows settings which are stored in cloud and their values on behalf of the signed-in user.
AdminConsentRequired	-	Yes

UserWindowsSettings.ReadWrite.All

Category	Application	Delegated
Identifier	-	dcb1026d-b7e1-4d31-9f61-6724d5140bf9
DisplayText	-	Read and write windows settings for all devices
Description	-	Allows the app to read and write a user's windows settings which are stored in cloud and their values on behalf of the signed-in user.
AdminConsentRequired	-	Yes

VerifiedId-Profile.Read.All

Category	Application	Delegated
Identifier	e227c591-dd64-4a8a-a033-816167f7c938	604b2056-41ed-4c56-aad5-1241d4ef7333
DisplayText	Read Verified Id profiles	Read Verified Id profiles
Description	This role can read Verified Id profiles in a tenant.	This role can read Verified Id profiles in a tenant.
AdminConsentRequired	Yes	Yes

VerifiedId-Profile.ReadWrite.All

Category	Application	Delegated
Identifier	-	e4a9cb5e-4767-48f8-9029-decf26a54456
DisplayText	-	Read and write Verified Id profiles
Description	-	This role can read and write Verified Id profiles in a tenant.
AdminConsentRequired	-	Yes

VirtualAppointment.Read

Category	Application	Delegated
Identifier	-	27470298-d3b8-4b9c-aad4-6334312a3eac
DisplayText	-	Read a user's virtual appointments
Description	-	Allows an application to read virtual appointments for the signed-in user. Only an organizer or participant user can read their virtual appointments.
AdminConsentRequired	-	Yes

VirtualAppointment.Read.All

Category	Application	Delegated
Identifier	d4f67ec2-59b5-4bdc-b4af-d78f6f9c1954	-
DisplayText	Read all virtual appointments for users, as authorized by online meetings application access policy	-
Description	Allows the application to read virtual appointments for all users, without a signed-in user. The app must also be authorized to access an individual user's data by the online meetings application access policy.	-
AdminConsentRequired	Yes	-

VirtualAppointment.ReadWrite

Category	Application	Delegated
Identifier	-	2ccc2926-a528-4b17-b8bb-860eed29d64c
DisplayText	-	Read and write a user's virtual appointments
Description	-	Allows an application to read and write virtual appointments for the signed-in user. Only an organizer or participant user can read and write their virtual appointments.
AdminConsentRequired	-	Yes

VirtualAppointment.ReadWrite.All

Category	Application	Delegated
Identifier	bf46a256-f47d-448f-ab78-f226fff08d40	-
DisplayText	Read-write all virtual appointments for users, as authorized by online meetings app access policy	-
Description	Allows the application to read and write virtual appointments for all users, without a signed-in user. The app must also be authorized to access an individual user's data by the online meetings application access policy.	-
AdminConsentRequired	Yes	-

VirtualAppointmentNotification.Send

Category	Application	Delegated
Identifier	97e45b36-1250-48e4-bd70-2df6dab7e94a	20d02fff-a0ef-49e7-a46e-019d4a6523b7
DisplayText	Send notification regarding virtual appointments as any user	Send notification regarding virtual appointments for the signed-in user
Description	Allows the application to send notification regarding virtual appointments as any user, without a signed-in user. The app must also be authorized to access an individual user's data by the online meetings application access policy.	Allows an application to send notifications for virtual appointments for the signed-in user.
AdminConsentRequired	Yes	Yes

VirtualEvent.Read

Category	Application	Delegated
Identifier	-	6b616635-ae58-433a-a918-8c45e4f304dc
DisplayText	-	Read your virtual events
Description	-	Allows the app to read virtual events created by you
AdminConsentRequired	-	Yes

VirtualEvent.Read.All

Category	Application	Delegated
Identifier	1dccb351-c4e4-4e09-a8d1-7a9ecbf027cc	-
DisplayText	Read all users' virtual events	-
Description	Allows the app to read all virtual events without a signed-in user.	-
AdminConsentRequired	Yes	-

VirtualEvent.ReadWrite

Category	Application	Delegated
Identifier	-	d38d189c-e29b-4344-8b3b-829bfa81380b
DisplayText	-	Read and write your virtual events
Description	-	Allows the app to read and write virtual events for you
AdminConsentRequired	-	Yes

VirtualEventRegistration-Anon.ReadWrite.All

Category	Application	Delegated
Identifier	23211fc1-f9d1-4e8e-8e9e-08a5d0a109bb	-

Category	Application	Delegated
DisplayText	Read and write anonymous users' virtual event registrations	-
Description	Allows the app to read and write anonymous users' virtual event registrations, without a signed-in user	-
AdminConsentRequired	Yes	-

WindowsUpdates.Read.All

Category	Application	Delegated
Identifier	50a8bf5f-b06a-4ac7-881f-3ca0c4be7550	e09fef2d-bf5e-4439-affa-7c48d23bb1c2
DisplayText	Read all Windows update deployment settings	Read all Windows update deployment settings
Description	Allows the app to read all Windows update deployment settings for the organization without a signed-in user.	Allows the app to read all Windows update deployment settings for the organization on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

WindowsUpdates.ReadWrite.All

Category	Application	Delegated
Identifier	7dd1be58-6e76-4401-bf8d-31d1e8180d5b	11776c0c-6138-4db3-a668-ee621bea2555
DisplayText	Read and write all Windows update deployment settings	Read and write all Windows update deployment settings
Description	Allows the app to read and write all Windows update deployment settings for the organization without a signed-in user.	Allows the app to read and write all Windows update deployment settings for the organization on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

WorkforceIntegration.Read.All

Category	Application	Delegated
Identifier	f10b94b9-37d1-4c88-8b7e-bf75a1152d39	f1ccd5a7-6383-466a-8db8-1a656f7d06fa
DisplayText	Read workforce integrations	Read workforce integrations
Description	Allows the app to read workforce integrations without a signed-in user.	Allows the app to read workforce integrations, to synchronize data from Microsoft Teams Shifts, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

WorkforceIntegration.ReadWrite.All

Category	Application	Delegated
Identifier	202bf709-e8e6-478e-bcfd-5d63c50b68e3	08c4b377-0d23-4a8b-be2a-23c1c1d88545
DisplayText	Read and write workforce integrations	Read and write workforce integrations
Description	Allows the app to manage workforce integrations to synchronize data from Microsoft Teams Shifts, without a signed-in user.	Allows the app to manage workforce integrations, to synchronize data from Microsoft Teams Shifts, on behalf of the signed-in user.
AdminConsentRequired	Yes	Yes

Resource-specific consent (RSC) permissions

Learn more about RSC authorization framework and RSC permissions.

Name	ID	Display text	Description
AiEnterpriseInteraction.Read.User	10d712aa-b4cd-4472-b0ba-6196e04c344f	Read user AI enterprise interactions.	Allows the app to read user AI enterprise interactions, without a signed-in user.
CallAiInsights.Read.Chat	ff9d3910-ca91-4e7f-843f-d44ab36a961a	Read all AI Insights for calls where the Teams	Allows the teams-app to read all aiInsights for calls where the Teams-

Name	ID	Display text	Description
		application is installed.	app is installed, without a signed-in user.
CallRecordings.Read.Chat	22748df0-bd8c-4626-aad9-6dab421b33e4	Read all recordings of calls where the Teams application is installed.	Allows the teams-app to read all recordings of calls where the Teams-app is installed, without a signed-in user.
Calls.AccessMedia.Chat	e716890c-c30a-4ac3-a0e3-551e7d9e8deb	Access media streams in calls associated with this chat or meeting	Allows the app to access media streams in calls associated with this chat or meeting, without a signed-in user.
Calls.JoinGroupCalls.Chat	a01e73f1-94da-4f6d-9b73-02e4ea65560b	Join calls associated with this chat or meeting	Allows the app to join calls associated with this chat or meeting, without a signed-in user.
CallTranscripts.Read.Chat	7990a5df-4c51-43ea-939c-3e8b18d6ddad	Read all transcripts of calls where the Teams app is installed.	Allows the Teams app to read all transcripts of calls where the Teams-app is installed, without a signed-in user.
Channel.Create.Group	65af85d7-62bb-4339-a206-7160fd427454	Create channels in this team	Allows the app to create channels in this team, without a signed-in user.
Channel.Delete.Group	4432e57d-0983-4c17-881c-235c529f96dc	Delete this team's channels	Allows the app to delete this team's channels, without a signed-in user.
ChannelMeeting.ReadBasic.Group	6c13459c-facc-4b0a-93cb-63f0dff28046	Read basic properties of the channel meetings in this team	Allows the app to read basic properties, such as name, schedule, organizer, join link, and start or end notifications, of channel meetings in this team, without a signed-in user.
ChannelMeetingNotification.Send.Group	bbb12bdb-71e6-4602-9f5e-b1172c505746	Send notifications in all the channel meetings	Allows the app to send notifications inside all the channel meetings associated with this

Name	ID	Display text	Description
		associated with this team	team, without a signed-in user.
ChannelMeetingParticipant.Read.Group	bd118236-e8f5-4bec-a62d-89a623717e05	Read the participants of this team's channel meetings	Allows the app to read participant information, including name, role, id, joined and left times, of channel meetings associated with this team, without a signed-in user.
ChannelMeetingRecording.Read.Group	30a40618-9b50-4764-b62e-b04023a8f5f3	Read the recordings of all channel meetings associated with this team	Allows the app to read recordings of all the channel meetings associated with this team, without a signed-in user.
ChannelMeetingTranscript.Read.Group	37e59e88-1a46-482b-b623-0a4aa6abdf67	Read the transcripts of all channel meetings associated with this team	Allows the app to read transcripts of all the channel meetings associated with this team, without a signed-in user.
ChannelMember.Read.Group	7e3614f5-3467-419c-9c63-dd0bbd2a88f9	Read the members of channels of a team	Read the members of channels of a team, without a signed-in user
ChannelMember.ReadWrite.Group	1342a0fc-cd33-4c75-ad65-d5defcfc7232	Read and write the members of channels of a team	Read and write the members of channels of a team, without a signed-in user
ChannelMessage.Read.Group	19103a54-c397-4bcd-be5a-ef111e0406fa	Read this team's channel messages	Allows the app to read this team's channel's messages, without a signed-in user.
ChannelMessage.Send.Group	3e38d437-815b-4368-9f19-e39dea9a6c7f	Send messages to this team's channels	Allows the app to send messages to this team's channels, without a signed-in user.
ChannelSettings.Read.Group	0a7b3084-8d18-46f5-8aef-b5b829292c6f	Read the names, descriptions, and	Allows the app to read this team's channel names, channel

Name	ID	Display text	Description
		settings of this team's channels	descriptions, and channel settings, without a signed-in user.
ChannelSettings.ReadWrite.Group	d057ad03-b27b-49f7-8219-e0d4a706da55	Update the names, descriptions, and settings of this team's channels	Allows the app to update and read the names, descriptions, and settings of this team's channels, without a signed-in user.
Chat.Manage.Chat	4a14842e-6bb6-4088-b21a-7d0a24f835a6	Manage this chat	Allows the app to manage the chat, the chat's members and grant access to the chat's data, without a signed-in user.
Chat.ManageDeletion.Chat	b827a2af-24b2-4f61-9eb3-8788e66a0d86	Delete and recover deleted chat	Allows the app to delete and recover deleted chat, without a signed-in user.
ChatMember.Read.Chat	e854bbc6-07e3-45cc-af99-b6e78fab5b80	Read this chat's members	Allows the app to read the members of this chat, without a signed-in user.
ChatMessage.Read.Chat	9398c3de-3f6b-4958-90f3-5098714ff50c	Read this chat's messages	Allows the app to read this chat's messages, without a signed-in user.
ChatMessage.Send.Chat	19cbeeb2-02a0-49d7-95cd-ab0841beed7f	Send messages to this chat	Allows the app to send messages to this chat, without a signed-in user.
ChatMessageReadReceipt.Read.Chat	a236cb34-7076-45a1-9381-22db8111a3d3	Read the ID of the last seen message in this chat	Allows the app to read the ID of the last message seen by the users in this chat.
ChatSettings.Read.Chat	40d35d7c-9cc3-4f2d-912b-464457412a00	Read this chat's settings	Allows the app to read this chat's settings, without a signed-in user.
ChatSettings.ReadWrite.Chat	ed928a9c-7530-496a-a624-4c0a460ab3ed	Read and write this chat's settings	Allows the app to read and write this chat's settings, without a signed-in user.

Name	ID	Display text	Description
Member.Read.Group	0a8ce3c7-89dd-46cf-b2c3-5ef0064437a8	Read this group's members	Allows the app to read the basic profile of this group's members, without a signed-in user.
OnlineMeeting.Read.Chat	f991ed3f-9617-4d8d-b06c-d18d9fcbcf2a	Read this meeting and subscribe to meeting call updates .	Allows the app to read this meeting and subscribe to meeting call updates.
OnlineMeeting.ReadBasic.Chat	eda8d262-4e6e-4ff6-a7ba-a2fb50535165	Read basic properties of meetings associated with this chat	Allows the app to read basic properties, such as name, schedule, organizer, join link, and start or end notifications, of meetings associated with this chat, without a signed-in user.
OnlineMeeting.ReadWrite.Chat	93400bb4-2282-4371-a745-a86d64c966d0	Manage this meeting and subscribe to meeting call updates.	Allows the app to manage this online meeting, and subscribe to meeting call updates.
OnlineMeetingArtifact.Read.Chat	c5d06837-8c0d-42fc-9e49-545e3f941261	Read virtual event artifacts	Read attendance reports & attendance records for this webinar or town hall.
OnlineMeetingNotification.Send.Chat	d9837fe0-9c31-4faa-8acb-b10874560161	Send notifications in the meetings associated with this chat	Allows the app to send notifications inside meetings associated with this chat, without a signed-in user.
OnlineMeetingParticipant.Read.Chat	6324a770-185c-4b4f-be13-2d9a1668e6eb	Read the participants of the meetings associated with this chat	Allows the app to read participant information, including name, role, id, joined and left times, of meetings associated with this chat, without a signed-in user.
OnlineMeetingRecording.Read.Chat	d20f0153-08ff-48a9-b299-96a8d1131d1d	Read the recordings of the meetings	Allows the app to read recordings of the meetings associated with

Name	ID	Display text	Description
		associated with this chat	this chat, without a signed-in user.
OnlineMeetingTranscript.Read.Chat	8c477e19-f0f7-45f9-ae72-604f77a599e3	Read the transcripts of the meetings associated with this chat	Allows the app to read transcripts of the meetings associated with this chat, without a signed-in user.
Owner.Read.Group	70d5316c-9b27-4057-a650-3b0fe49002ab	Read this group's owners	Allows the app to read the basic profile of this group's owners, without a signed-in user.
Team.Read.Group	41027e3b-d156-4913-bb0d-06cbbe931eb7	Read this team's metadata	Allows the app to read this team's metadata, without a signed-in user.
TeamMember.Read.Group	b8731755-de22-4604-be08-93e1e5c2d2d6	Read this team's members	Allows the app to read the members of this team, without a signed-in user.
TeamsActivity.Send.Chat	119b5846-be45-44cd-87d7-bfc566330e11	Send activity feed notifications to users in this chat	Allows the app to create new notifications in the teamwork activity feeds of the users in this chat, without a signed-in user.
TeamsActivity.Send.Group	d4539c25-0937-4095-b844-b97228dd8655	Send activity feed notifications to users in this team	Allows the app to create new notifications in the teamwork activity feeds of the users in this team, without a signed-in user.
TeamsActivity.Send.User	483c432d-7210-44e7-a362-954c0c5e4108	Send activity feed notifications to this user	Allows the app to create new notifications in the teamwork activity feed of this user, without a signed-in user.
TeamsAppInstallation.Read.Chat	b60343cd-f77a-4c4f-8036-41938b1abd8b	Read which apps are installed in this chat	Allows the app to read the Teams apps that are installed in this chat along with the permissions granted to each app, without a signed-in user.

Name	ID	Display text	Description
TeamsAppInstallation.Read.Group	ba4beb29-863b-4f02-8969-37a289cd91c0	Read which apps are installed in this team	Allows the app to read the Teams apps that are installed in this team, without a signed-in user.
TeamsAppInstallation.Read.User	39a4b5e8-1aa6-4da4-877a-d2345944028d	Read installed Teams apps for a user	Allows the app to read the Teams apps that are installed in user's personal scope, without a signed-in user. Does not give the ability to read application-specific settings.
TeamSettings.Edit.Group	33f7a028-d012-4bd9-b40f-3c970d089bc8	Edit this team's settings	Allows the app to edit this team's settings, without a signed-in user.
TeamSettings.Read.Group	87909ea6-7b07-42cf-b3a0-b8bd8e7072a8	Read this team's settings	Allows the app to read this team's settings, without a signed-in user.
TeamSettings.ReadWrite.Group	13451d84-ced2-4d45-9b0d-98688b90e5bf	Read and write this team's settings	Allows the app to read and write this team's settings, without a signed-in user.
TeamsTab.Create.Chat	0029d2bb-fc98-4712-9310-69dd5fcc94d5	Create tabs in this chat	Allows the app to create tabs in this chat, without a signed-in user.
TeamsTab.Create.Group	c4d7203b-1e46-4c4a-95f9-862779aa39e1	Create tabs in this team	Allows the app to create tabs in this team, without a signed-in user.
TeamsTab.Delete.Chat	fa50d890-02fe-4696-b82b-110dc7f7382a	Delete this chat's tabs	Allows the app to delete this chat's tabs, without a signed-in user.
TeamsTab.Delete.Group	cc2e79a6-9a86-45cc-91c1-41c15745287e	Delete this team's tabs	Allows the app to delete this team's tabs, without a signed-in user.
TeamsTab.Read.Chat	aa07ff41-1317-4f07-8edb-a1558e9bfc84	Read this chat's tabs	Allows the app to read this chat's tabs, without a signed-in user.
TeamsTab.Read.Group	60d920d0-44e7-44f4-a811-	Read this team's tabs	Allows the app to read this team's tabs, without

Name	ID	Display text	Description
	1a172a2ea5b3		a signed-in user.
TeamsTab.ReadWrite.Chat	d583f4d7-57da-4b2c-9744-253e9ec3c7be	Manage this chat's tabs	Allows the app to manage this chat's tabs, without a signed-in user.
TeamsTab.ReadWrite.Group	717ca3a4-bc73-47f8-b613-4d43e657fa9c	Manage this team's tabs	Allows the app to manage this team's tabs, without a signed-in user.
VirtualEvent.Read.Chat	298266a0-fbf7-4804-b988-5a54e61566c8	Read virtual event details	Read information for this webinars or town halls, including schedules, speakers, and event settings and webinar registrations.
VirtualEventRegistration-Anon.ReadWrite.Chat	0e646cc8-6b07-4030-9a41-a7db4644b4cc	Manage virtual event registrations	Register attendees and cancel registrations for this webinar.

Related content

- Overview of Microsoft Graph permissions
- Grant or revoke Microsoft Graph permissions programmatically

Last updated on 09/14/2026

agentIdentityBlueprint resource type

Namespace: microsoft.graph

An agent identity blueprint serves as a template for creating agent identities within the Microsoft Entra ID ecosystem.

Inherits from application.

This resource is an open type that allows additional properties beyond those documented here.

Methods

Expand table

Method	Return type	Description
List	agentIdentityBlueprint collection	Get a list of the agentIdentityBlueprint objects and their properties.
Create	agentIdentityBlueprint	Create (register) a new agentIdentityBlueprint.
Get	agentIdentityBlueprint	Read the properties and relationships of agentIdentityBlueprint object.
Update	agentIdentityBlueprint	Update the properties of an agentIdentityBlueprint object.
Upsert	agentIdentityBlueprint	Create a new agent identity blueprint if it doesn't exist, or update the properties of an existing blueprint.
Delete	None	Delete an agentIdentityBlueprint object.
Credentials		
Add password	passwordCredential	Add a strong password or secret to an agent identity blueprint.
Remove password	passwordCredential	Remove a password or secret from an agent identity blueprint.
Add key	keyCredential	Add a key credential to an agent identity blueprint.
Remove key	None	Remove a key credential from an agent identity blueprint.

Method	Return type	Description
List federated identity credential	federatedIdentityCredential collection	Get a list of the federatedIdentityCredential objects and their properties.
Create federated identity credential	federatedIdentityCredential	Create a new federatedIdentityCredential object.
Get federated identity credential	federatedIdentityCredential	Read the properties and relationships of a federatedIdentityCredential object.
Update federated identity credential	None	Update the properties of a federatedIdentityCredential object.
Upsert federated identity credential	federatedIdentityCredential	Create a new federatedIdentityCredential if it doesn't exist, or update the properties of an existing federatedIdentityCredential object.
Delete federated identity credential	None	Delete a federatedIdentityCredential object.
Deleted items		
List	directoryObject collection	Retrieve a list of recently deleted agent identities.
Get	directoryObject	Retrieve the properties of a recently deleted agent identity.
Restore	directoryObject	Restore a recently deleted agent identity.
Permanently delete	None	Permanently delete an agent identity.
Owners		
List owners	directoryObject collection	Get the owners of this agent identity blueprint principal.
Add owners	directoryObject	Assign an owner to this agent identity blueprint principal.
Remove owners	None	Remove an owner from this agent identity blueprint principal.
Sponsors		

Method	Return type	Description
List sponsors	directoryObject collection	Get the sponsors for this agent identity blueprint. Sponsors are users or service principals who can authorize and manage the lifecycle of agent identity instances.
Add sponsors	directoryObject	Add sponsors by posting to the sponsors collection.
Remove sponsors	None	Remove a directoryObject object.
Verified publisher		
Set	None	Set the verified publisher of an application.
Unset	None	Unset the verified publisher of an application.

Properties

> **Important**
>
> While this resource inherits from **application**, some properties are not applicable and return `null` or default values. These properties are excluded from the table below.

Expand table

Property	Type	Description
api	apiApplication	Specifies settings for an agent identity blueprint that implements a web API. Inherited from application.
appId	String	The unique identifier for the agent identity blueprint that is assigned by Microsoft Entra ID. Not nullable. Read-only. Inherited from application.
appRoles	appRole collection	The collection of roles defined for the agent identity blueprint. With app role assignments, these roles can be assigned to users, groups, or service principals associated with other applications. Not nullable. Inherited from application.

Property	Type	Description
certification	certification	Specifies the certification status of the agent identity blueprint. Inherited from application.
createdByAppId	String	The **appId** of the application that created this agent identity blueprint. Set internally by Microsoft Entra ID. Read-only. Inherited from application.
createdDateTime	DateTimeOffset	The date and time the agent identity blueprint was registered. The DateTimeOffset type represents date and time information using ISO 8601 format and is always in UTC time. Read-only. Inherited from application.
description	String	Free text field to provide a description of the agent identity blueprint to end users. The maximum allowed size is 1,024 characters. The least privileged permission to update this property is *AgentIdentityBlueprint.UpdateBranding.All*. Inherited from application.
disabledByMicrosoftStatus	String	Specifies whether Microsoft has disabled the registered agent identity blueprint. The possible values are: `null` (default value), `NotDisabled`, and `DisabledDueToViolationOfServicesAgreement` (reasons may include suspicious, abusive, or malicious activity, or a violation of the Microsoft Services Agreement). Inherited from application.
displayName	String	The display name for the agent identity blueprint. Maximum length is 256 characters. The least privileged permission to update this property is *AgentIdentityBlueprint.UpdateBranding.All*. Inherited from application.
groupMembershipClaims	String	Configures the `groups` claim issued in a user or OAuth 2.0 access token that the agent identity blueprint expects. To set this attribute, use one of the following string values: `None`, `SecurityGroup` (for security groups and Microsoft Entra roles), `All` (this gets all security groups, distribution groups, and Microsoft Entra directory roles that the signed-in user is a member of). Inherited from application.

Property	Type	Description
id	String	Unique identifier for the agent identity blueprint object. This property is referred to as **Object ID** in the Microsoft Entra admin center. Key. Not nullable. Read-only. Inherited from directoryObject.
identifierUris	String collection	Also known as App ID URI, this value is set when an agent identity blueprint is used as a resource app. The identifierUris acts as the prefix for the scopes you reference in your API's code, and it must be globally unique across Microsoft Entra ID. Not nullable. Inherited from application.
info	informationalUrl	Basic profile information of the agent identity blueprint, such as it's marketing, support, terms of service, and privacy statement URLs. The terms of service and privacy statement are surfaced to users through the user consent experience. Inherited from application.
keyCredentials	keyCredential collection	The collection of key credentials associated with the agent identity blueprint. Not nullable. The least privileged permission to update this property is *AgentIdentityBlueprint.AddRemoveCreds.All*. Inherited from application.
managerApplications	Guid collection	A collection of application IDs for Microsoft first-party applications designated as managers of this agent blueprint. Manager applications can create agent blueprint principals, agent identities, and agent users for managed agent blueprints without requiring highly privileged permissions such as `AgentIdentityBlueprintPrincipal.ReadWrite.All`. Limited to a maximum of 10 entries. Not nullable. Only Microsoft first-party applications can be designated as managers. Not returned by default. Supports `$select`.
optionalClaims	optionalClaims	Application developers can configure optional claims in their Microsoft Entra agent identity blueprints to specify the claims that are sent to their application by the Microsoft security token service. Inherited from application.
passwordCredentials	passwordCredential collection	The collection of password credentials associated with the agent identity blueprint. Not

Property	Type	Description
		nullable. The least privileged permission to update this property is *AgentIdentityBlueprint.AddRemoveCreds.All*. Inherited from application. You can also add passwords after creating the agent identity blueprint by calling the Add password API.
publisherDomain	String	The verified publisher domain for the agent identity blueprint. Read-only. Inherited from application.
requiredResourceAccess	requiredResourceAccess collection	Specifies the resources that the agentIdentityBlueprint needs to access. This property also specifies the set of delegated permissions and application roles that it needs for each of those resources. This configuration of access to the required resources drives the consent experience. No more than 50 resource services (APIs) can be configured. The total number of required permissions must not exceed 400. For more information, see Limits on requested permissions per app. Not nullable. Inherited from application. Supports `$filter` (`eq`, `not`, `ge`, `le`).
serviceManagementReference	String	References application or service contact information from a Service or Asset Management database. Nullable. Inherited from application.
signInAudience	String	Specifies the Microsoft accounts that are supported for the current agent identity blueprint. The possible values are: `AzureADMyOrg` (default), `AzureADMultipleOrgs`, `AzureADandPersonalMicrosoftAccount`, and `PersonalMicrosoftAccount`. Inherited from application.
tags	String collection	Custom strings that can be used to categorize and identify the agent identity blueprint. Not nullable. Inherited from application.
tokenEncryptionKeyId	Guid	Specifies the keyId of a public key from the keyCredentials collection. When configured, Microsoft Entra ID encrypts all the tokens it

Property	Type	Description
		emits by using the key this property points to. Inherited from application.
uniqueName	String	The unique identifier that can be assigned to an agent identity blueprint and used as an alternate key. Immutable. Read-only. Inherited from application.
verifiedPublisher	verifiedPublisher	Specifies the verified publisher of the agent identity blueprint. Inherited from application.
web	webApplication	Specifies settings for a web application. Inherited from application.

Limits on requested permissions per app

Microsoft Entra ID limits the number of permissions that can be requested and consented by a client app. These limits depend on the `signInAudience` value for an app, shown in the app's manifest.

Expand table

signInAudience	Allowed users	Maximum permissions the app can request	Maximum Microsoft Graph permissions the app can request	Maximum permissions that can be consented in a single request
AzureADMyOrg	Users from the organization where the app is registered	400	400	About 155 delegated permissions and about 300 application permissions
AzureADMultipleOrgs	Users from any Microsoft Entra organization	400	400	About 155 delegated permissions and about 300 application permissions
PersonalMicrosoftAccount	Consumer users (such as Outlook.com	30	30	30

signInAudience	Allowed users	Maximum permissions the app can request	Maximum Microsoft Graph permissions the app can request	Maximum permissions that can be consented in a single request
	or Live.com accounts)			
AzureADandPersonalMicrosoftAccount	Consumer users and users from any Microsoft Entra organization	30	30	30

> **Note**
>
> For Microsoft Entra Agent ID, some high-risk Microsoft Graph permissions are globally blocked for agents and can't be granted to agent identities.
>
> If you include a blocked Microsoft Graph delegated permission scope or app role in the `resourceAccess` collection of a `requiredResourceAccess` entry, the request is rejected with an HTTP `400 Bad Request` response and an error indicating that the permission is blocked and can't be granted to agent identities.
>
> For the list of blocked Microsoft Graph permissions for agents, see Microsoft Graph permissions blocked for agents.

Relationships

Expand table

Relationship	Type	Description
appManagementPolicies	appManagementPolicy collection	The appManagementPolicy applied to this agent identity blueprint. Inherited from microsoft.graph.application
federatedIdentityCredentials	federatedIdentityCredential collection	Federated identities for agent identity blueprints. Inherited from microsoft.graph.application

Relationship	Type	Description
inheritablePermissions	inheritablePermission collection	Defines scopes of a resource application that may be automatically granted to agent identities without additional consent.
owners	directoryObject collection	Directory objects that are owners of this agent identity blueprint. The owners are a set of nonadmin users or service principals allowed to modify this object. Read-only. Nullable. Inherited from microsoft.graph.application
sponsors	directoryObject collection	The sponsors for this agent identity blueprint. Sponsors are users or groups who can authorize and manage the lifecycle of agent identity instances. Required during the create operation.

JSON representation

The following JSON representation shows the resource type.

JSON


```
{
  "@odata.type": "#microsoft.graph.agentIdentityBlueprint",
  "id": "String (identifier)",
  "appId": "String",
  "identifierUris": ["String"],
  "createdByAppId": "String",
  "createdDateTime": "String (timestamp)",
  "description": "String",
  "disabledByMicrosoftStatus": "String",
  "displayName": "String",
  "groupMembershipClaims": "String",
  "publisherDomain": "String",
  "requiredResourceAccess": [{"@odata.type":
"microsoft.graph.requiredResourceAccess"}],
  "signInAudience": "String",
  "tags": ["String"],
  "tokenEncryptionKeyId": "Guid",
  "uniqueName": "String",
  "serviceManagementReference": "String",
  "certification": {
    "@odata.type": "microsoft.graph.certification"
  },
  "optionalClaims": {
    "@odata.type": "microsoft.graph.optionalClaims"
  },
  "api": {
    "@odata.type": "microsoft.graph.apiApplication"
```

```
  },
  "appRoles": [
    {
      "@odata.type": "microsoft.graph.appRole"
    }
  ],
  "info": {
    "@odata.type": "microsoft.graph.informationalUrl"
  },
  "keyCredentials": [
    {
      "@odata.type": "microsoft.graph.keyCredential"
    }
  ],
  "managerApplications": ["Guid"],
  "passwordCredentials": [
    {
      "@odata.type": "microsoft.graph.passwordCredential"
    }
  ],
  "verifiedPublisher": {
    "@odata.type": "microsoft.graph.verifiedPublisher"
  },
  "web": {
    "@odata.type": "microsoft.graph.webApplication"
  }
}
```

Last updated on 04/25/2026

agentIdentityBlueprintPrincipal resource type

Namespace: microsoft.graph

Represents an agent identity blueprint principal in a tenant. An agent identity blueprint principal is instantiated from an agentIdentityBlueprint object and is used to create agent identities within a Microsoft Entra ID tenant, and perform various identity management operations that affect all agent identities created.

Inherits from servicePrincipal.

This resource is an open type that allows additional properties beyond those documented here.

Methods

Expand table

Method	Return type	Description
List	agentIdentityBlueprintPrincipal collection	Get a list of the agentIdentityBlueprintPrincipal objects and their properties.
Create	agentIdentityBlueprintPrincipal	Create a new agentIdentityBlueprintPrincipal object.
Get	agentIdentityBlueprintPrincipal	Read the properties and relationships of agentIdentityBlueprintPrincipal object.
Update	agentIdentityBlueprintPrincipal	Update the properties of an agentIdentityBlueprintPrincipal object.
Delete	None	Delete an agentIdentityBlueprintPrincipal object.
App role assignments		
List app role assigned to	appRoleAssignment collection	Get the users, groups, and agent identities assigned app roles for this agent identity blueprint principal.
Add app role assigned to	appRoleAssignment	Assign an app role for this agent identity blueprint principal to a user, group, or service principal.
Remove app role assigned to	None	Remove an app role assignment for this agent identity blueprint principal from a user, group, or

Method	Return type	Description
		service principal.
List app role assignments	appRoleAssignment collection	Get the app roles that this agent identity blueprint principal is assigned.
Add app role assignment	appRoleAssignment	Assign an app role to this agent identity blueprint principal.
Remove app role assignment	None	Remove an app role assignment from this agent identity blueprint principal.
Delegated permission grants		
List OAuth 2.0 permission grants	oAuth2PermissionGrant collection	Get the delegated permission grants authorizing this agent identity blueprint principal to access an API on behalf of a signed-in user.
Deleted items		
List	directoryObject collection	Retrieve a list of recently deleted agent identities.
Get	directoryObject	Retrieve the properties of a recently deleted agent identity.
Restore	directoryObject	Restore a recently deleted agent identity.
Permanently delete	None	Permanently delete an agent identity.
Directory objects		
List owned objects	directoryObject collection	Get directory objects owned by this agent identity blueprint principal.
List created objects	directoryObject collection	Get directory objects created by this agent identity blueprint principal.
Memberships		
List member of	directoryObject collection	Get the groups that this agent identity blueprint principal is a direct member of.
Owners		
List owners	directoryObject collection	Get the owners of this agent identity blueprint principal.
Add owners	None	Assign an owner to this agent identity blueprint principal.

Method	Return type	Description
Remove owners	None	Remove an owner from this agent identity blueprint principal.
Sponsors		
List sponsors	directoryObject collection	Get the sponsors for this agent identity blueprint principal.
Add sponsors	directoryObject	Add sponsors by posting to the sponsors collection.
Remove sponsors	None	Remove a directoryObject object.

Properties

> **Important**
>
> While this resource inherits from **servicePrincipal**, some properties are not applicable and return `null` or default values. These properties are excluded from the table below.

Expand table

Property	Type	Description
accountEnabled	Boolean	`true` if the agent identity blueprint principal account is enabled; otherwise, `false`. If set to `false`, then no users are able to sign in to this app, even if they're assigned to it. Inherited from servicePrincipal.
appDescription	String	The description exposed by the associated agent identity blueprint. Inherited from servicePrincipal.
appDisplayName	String	The display name exposed by the associated agent identity blueprint. Maximum length is 256 characters. Inherited from servicePrincipal.
appId	String	The **appId** of the associated agent identity blueprint. Alternate key. Inherited from servicePrincipal.
appOwnerOrganizationId	Guid	Contains the tenant ID where the agent identity blueprint is registered. This is applicable only to agent identity blueprint principals backed by applications. Inherited from servicePrincipal.
appRoleAssignmentRequired	Boolean	Specifies whether users or other service principals need to be granted an app role assignment for this agent

Property	Type	Description
		identity blueprint principal before users can sign in or apps can get tokens. The default value is `false`. Not nullable. Inherited from servicePrincipal.
appRoles	appRole collection	The roles exposed by the agent identity blueprint, which this agent identity blueprint principal represents. For more information, see the **appRoles** property definition on the application entity. Not nullable. Inherited from servicePrincipal.
createdByAppId	String	The **appId** of the application that created this agent identity blueprint principal. Set internally by Microsoft Entra ID. Read-only. Inherited from servicePrincipal.
disabledByMicrosoftStatus	String	Specifies whether Microsoft has disabled the registered agent identity blueprint. The possible values are: `null` (default value), `NotDisabled`, and `DisabledDueToViolationOfServicesAgreement` (reasons may include suspicious, abusive, or malicious activity, or a violation of the Microsoft Services Agreement). Inherited from servicePrincipal.
displayName	String	The display name for the agent identity blueprint principal. Inherited from servicePrincipal.
id	String	The unique identifier for the agent identity blueprint principal. Inherited from entity. Key. Not nullable. Read-only.
info	informationalUrl	Basic profile information of the acquired application such as app's marketing, support, terms of service and privacy statement URLs. The terms of service and privacy statement are surfaced to users through the user consent experience. Inherited from servicePrincipal.
managerApplications	Guid collection	The collection of application IDs designated as managers of this agent identity blueprint principal's backing agentIdentityBlueprint. Read-only; the value is server-managed and reflects the **managerApplications** of the backing agentIdentityBlueprint. To change the managers, an owner or administrator must update the **managerApplications** property on the backing agentIdentityBlueprint **in the tenant where it's registered**. For multitenant agent identity blueprints, admins in a tenant where the blueprint is only consumed can't make this change — they must ask an owner or administrator in the blueprint's home tenant. Not nullable. Returned only on `$select`.

Property	Type	Description
publishedPermissionScopes	permissionScope collection	The delegated permissions exposed by the application. For more information, see the **oauth2PermissionScopes** property on the agent identity blueprint entity's **api** property. Not nullable. Inherited from servicePrincipal.
publisherName	String	The name of the Microsoft Entra tenant that published the application. Inherited from servicePrincipal.
servicePrincipalNames	String collection	Contains the list of **identifiersUris**, copied over from the associated agent identity blueprint. More values can be added to hybrid agent identity blueprint. These values can be used to identify the permissions exposed by this app within Microsoft Entra ID. Not nullable. **Property blocked on Agent Identity Blueprint Principal.** Inherited from servicePrincipal.
servicePrincipalType	String	Identifies if the agent identity blueprint principal represents an application. This is set by Microsoft Entra ID internally. For an agent identity blueprint principal that represents an application this is set as **Application**. Inherited from servicePrincipal.
signInAudience	String	Specifies the Microsoft accounts that are supported for the current agent identity blueprint. Read-only. Supported values are: `AzureADMyOrg`, `AzureADMultipleOrgs`, `AzureADandPersonalMicrosoftAccount`, and `PersonalMicrosoftAccount`. Inherited from servicePrincipal.
tags	String collection	Custom strings that can be used to categorize and identify the agent identity blueprint principal. Not nullable. The value is the union of strings set here and on the associated agent identity blueprint entity's **tags** property. Inherited from servicePrincipal.
verifiedPublisher	verifiedPublisher	Specifies the verified publisher of the application that's linked to this agent identity blueprint principal. Inherited from servicePrincipal.

Relationships

Expand table

Relationship	Type	Description
appManagementPolicies	appManagementPolicy collection	The appManagementPolicy applied to this agent identity blueprint principal. Inherited from microsoft.graph.servicePrincipal
appRoleAssignedTo	appRoleAssignment collection	App role assignments for this agent identity blueprint principal, granted to users, groups, and other service principals. Supports `$expand`. Inherited from microsoft.graph.servicePrincipal
appRoleAssignments	appRoleAssignment collection	App role assignment for another app or service, granted to this agent identity blueprint principal. Supports `$expand`. Inherited from microsoft.graph.servicePrincipal
createdObjects	directoryObject collection	Directory objects created by this agent identity blueprint principal. Read-only. Nullable. Inherited from microsoft.graph.servicePrincipal
memberOf	directoryObject collection	Roles that this agent identity blueprint principal is a member of. HTTP Methods: GET Read-only. Nullable. Supports `$expand`. Inherited from microsoft.graph.servicePrincipal
oauth2PermissionGrants	oAuth2PermissionGrant collection	Delegated permission grants authorizing this agent identity blueprint principal to access an API on behalf of a signed-in user. Read-only. Nullable. Inherited from microsoft.graph.servicePrincipal
ownedObjects	directoryObject collection	Directory objects that are owned by this agent identity blueprint principal. Read-only. Nullable. Supports `$expand` and `$filter` (`/$count eq 0`, `/$count ne 0`, `/$count eq 1`, `/$count ne 1`). Inherited from microsoft.graph.servicePrincipal
owners	directoryObject collection	Directory objects that are owners of this agent identity blueprint principal. The owners are a set of nonadmin users or servicePrincipals who are allowed to modify this object. Supports `$expand` and `$filter` (`/$count eq 0`, `/$count ne 0`, `/$count eq 1`, `/$count ne 1`). Inherited from microsoft.graph.servicePrincipal
sponsors	directoryObject collection	The sponsors for this agent identity blueprint principal. Sponsors are users or service principals who can authorize and manage the lifecycle of agent identity instances.

JSON representation

The following JSON representation shows the resource type. Only a subset of all properties are returned by default. All other properties can only be retrieved using `$select`.

JSON


```
{
  "@odata.type": "#microsoft.graph.agentIdentityBlueprintPrincipal",
  "id": "String (identifier)",
  "accountEnabled": "Boolean",
  "createdByAppId": "String",
  "appDescription": "String",
  "appDisplayName": "String",
  "appId": "String",
  "appOwnerOrganizationId": "Guid",
  "appRoleAssignmentRequired": "Boolean",
  "disabledByMicrosoftStatus": "String",
  "displayName": "String",
  "publisherName": "String",
  "servicePrincipalNames": [
    "String"
  ],
  "servicePrincipalType": "String",
  "signInAudience": "String",
  "tags": [
    "String"
  ],
  "appRoles": [
    {
      "@odata.type": "microsoft.graph.appRole"
    }
  ],
  "info": {
    "@odata.type": "microsoft.graph.informationalUrl"
  },
  "managerApplications": [
    "Guid"
  ],
  "publishedPermissionScopes": [
    {
      "@odata.type": "microsoft.graph.permissionScope"
    }
  ],
  "verifiedPublisher": {
    "@odata.type": "microsoft.graph.verifiedPublisher"
  }
}
```


Last updated on 09/11/2026

agentIdentity resource type

Namespace: microsoft.graph

Represents an agent identity in Microsoft Entra ID. An agent identity is an account used by AI agents to authenticate within the Microsoft Entra ID ecosystem.

Inherits from servicePrincipal.

This resource is an open type that allows additional properties beyond those documented here.

Methods

Expand table

Method	Return type	Description
List	agentidentity collection	Get a list of the agentidentity objects and their properties.
Create	agentidentity	Create a new agentidentity object.
Get	agentIdentity	Read the properties and relationships of agentIdentity object.
Update	agentIdentity	Update the properties of an agentIdentity object.
Delete	None	Delete an agentIdentity object.
App role assignments		
List appRoleAssignedTo	appRoleAssignment collection	Get the users, groups, and agent identities assigned app roles for this agent identity.
List appRoleAssignments	appRoleAssignment collection	Get the app roles that this agent identity is assigned.
Create appRoleAssignment	appRoleAssignment	Create a new appRoleAssignment object.
Delete appRoleAssignment	appRoleAssignment	Delete an existing appRoleAssignment object.
Delegated permission grants		

Method	Return type	Description
List oauth2PermissionGrants	oAuth2PermissionGrant collection	Get the delegated permission grants authorizing this agent identity to access an API on behalf of a signed-in user.
Deleted items		
List	directoryObject collection	Retrieve a list of recently deleted agent identities.
Get	directoryObject	Retrieve the properties of a recently deleted agent identity.
Restore	directoryObject	Restore a recently deleted agent identity.
Permanently delete	None	Permanently delete an agent identity.
Directory objects		
List ownedObjects	directoryObject collection	Get directory objects owned by this agent identity.
Memberships		
List direct memberships	directoryObject collection	Get the groups that this agent identity is a direct member of.
List transitive memberships	directoryObject collection	Get the groups that this agent identity is a member of. This operation is transitive and includes the groups that this agent identity is a nested member of.
Owners		
List owners	directoryObject collection	Get the owners of this agent identity.
Add owners	directoryObject	Add owners by posting to the owners collection.
Remove owners	None	Remove a directoryObject object.
Sponsors		
List sponsors	directoryObject collection	Get the sponsors for this agent identity.
Add sponsors	directoryObject	Add sponsors by posting to the sponsors collection.
Remove sponsors	None	Remove a directoryObject object.

Properties

> **Important**

While this resource inherits from **servicePrincipal**, some properties are not applicable.

Expand table

Property	Type	Description
odata.type	String	`#microsoft.graph.agentIdentity`. Distinguishes this object as an agent identity. Can be used to identify this object as an agent identity, instead of another kind of service principal.
accountEnabled	Boolean	`true` if the agent identity account is enabled; otherwise, `false`. If set to `false`, then no users are able to sign in to this app, even if they're assigned to it. Inherited from servicePrincipal.
agentIdentityBlueprintId	String	The **appId** of the agent identity blueprint that defines the configuration for this agent identity.
customSecurityAttributes	customSecurityAttributeValue	An open complex type that holds the value of a custom security attribute that is assigned to a directory object. Nullable. Requires `$select` to retrieve. Inherited from servicePrincipal.
createdByAppId	String	The **appId** of the application that created this agent identity. Set internally by Microsoft Entra ID. Read-only. Inherited from servicePrincipal.
createdDateTime	DateTimeOffset	The date and time the agent identity was created. Read-only. Inherited from servicePrincipal.
disabledByMicrosoftStatus	String	Specifies whether Microsoft has disabled the registered Agent Identity Blueprint. The possible values are: `null` (default value), `NotDisabled`, and `DisabledDueToViolationOfServicesAgreement` (reasons may include suspicious, abusive, or malicious activity, or a violation of the Microsoft Services Agreement). Inherited from servicePrincipal.
displayName	String	The display name for the agent identity. Inherited from servicePrincipal.
id	String	The unique identifier for the agent identity. Inherited from directoryObject. Key. Not nullable. Read-only. Inherited from entity.

Property	Type	Description
managerApplications	Guid collection	The collection of application IDs designated as managers of this agent identity's backing agentIdentityBlueprint. Read-only; the value is server-managed and reflects the **managerApplications** of the backing agentIdentityBlueprint. To change the managers, an owner or administrator must update the **managerApplications** property on the backing agentIdentityBlueprint **in the tenant where it's registered**. For multitenant agent identity blueprints, admins in a tenant where the blueprint is only consumed can't make this change — they must ask an owner or administrator in the blueprint's home tenant. Not nullable. Returned only on `$select`.
servicePrincipalType	String	Set to **ServiceIdentity** for all agent identities. Inherited from servicePrincipal.
tags	String collection	Custom strings that can be used to categorize and identify the agent identity. Not nullable. The value is the union of strings set here and on the associated Agent Identity Blueprint entity's **tags** property. Inherited from servicePrincipal.

Relationships

Expand table

Relationship	Type	Description
appRoleAssignedTo	appRoleAssignment collection	App role assignments for this app or service, granted to users, groups, and other agent identities. Supports `$expand`. Inherited from microsoft.graph.servicePrincipal
appRoleAssignments	appRoleAssignment collection	App role assignment for another app or service, granted to this agent identity. Supports `$expand`. Inherited from microsoft.graph.servicePrincipal
createdObjects	directoryObject collection	Directory objects created by this agent identity. Read-only. Nullable. Inherited from microsoft.graph.servicePrincipal

Relationship	Type	Description
memberOf	directoryObject collection	Roles that this agent identity is a member of. HTTP Methods: GET Read-only. Nullable. Supports `$expand`. Inherited from microsoft.graph.servicePrincipal
oauth2PermissionGrants	oAuth2PermissionGrant collection	Delegated permission grants authorizing this agent identity to access an API on behalf of a signed-in user. Read-only. Nullable. Inherited from microsoft.graph.servicePrincipal
ownedObjects	directoryObject collection	Directory objects that are owned by this agent identity. Read-only. Nullable. Supports `$expand` and `$filter` (`/$count eq 0`, `/$count ne 0`, `/$count eq 1`, `/$count ne 1`). Inherited from microsoft.graph.servicePrincipal
owners	directoryObject collection	Directory objects that are owners of this agent identity. The owners are a set of nonadmin users or agent identities who are allowed to modify this object. Supports `$expand` and `$filter` (`/$count eq 0`, `/$count ne 0`, `/$count eq 1`, `/$count ne 1`). Inherited from microsoft.graph.servicePrincipal
sponsors	directoryObject collection	The sponsors for this agent identity.

JSON representation

The following JSON representation shows the resource type. Only a subset of all properties are returned by default. All other properties can only be retrieved using $select.

JSON


```
{
  "@odata.type": "#microsoft.graph.agentIdentity",
  "id": "String (identifier)",
  "accountEnabled": "Boolean",
  "agentIdentityBlueprintId": "String",
  "createdByAppId": "String",
  "createdDateTime": "String (timestamp)",
  "disabledByMicrosoftStatus": "String",
  "displayName": "String",
  "managerApplications": [
    "Guid"
  ],
  "servicePrincipalType": "String",
  "tags": [
    "String"
```

```
  ]
}
```

Last updated on 09/11/2026

Microsoft agent identity platform error codes

This article provides a comprehensive reference for error codes you might encounter when working with the Microsoft agent identity platform.

Handling error codes in your application

The OAuth2.0 spec provides guidance on how to handle errors during authentication using the `error` portion of the error response. For more information on handling error code and the possible `error` field values, see the Microsoft identity platform and OAuth 2.0 error codes documentation.

Quota and limit errors

These errors occur when you exceed the tenant quota or the maximum number of agent identity blueprints or agent identities.

Expand table

Error code	Description
`Agent_Directory_QuotaExceeded`	Agent identity blueprints and/or agent identities have exceeded 95% of the tenant resource quota. To create more, you must permanently delete unneeded blueprints or identities.
`AgentBlueprint_LimitExceeded`	You've reached the maximum number of agent identity blueprints allowed including active and soft-deleted items. To create more, you must permanently delete unneeded blueprints.
`AgentIdentity_LimitExceeded`	You've reached the maximum number of agent identities allowed including active and soft-deleted entries. To add more, you must permanently delete unneeded agent identities.

Agent identity blueprint errors

These errors occur when a request includes properties or API versions that agent identity blueprints don't support.

Expand table

Error code	Description
`AgentBlueprint_IncompatibleProperty`	A property specified in the request is incompatible with agent identity blueprints and can't be set.
`AgentBlueprint_IncompatibleProperty_NullPropertyName`	A property in the request is incompatible with agent identity blueprints and can't be set.
`AgentBlueprint_NotSupportedOnApiVersion`	Agent identity blueprints aren't supported on the API version used in this request.

Agent identity blueprint principal errors

These errors occur when a request includes properties, API versions, or parent resources that agent identity blueprint principals don't support.

Expand table

Error code	Description
`AgentBlueprintPrincipal_AgentIdentity_IncompatibleProperty`	A property specified in the request is incompatible with agent identity and can't be set.
`AgentBlueprintPrincipal_IncompatibleProperty`	A property specified in the request is incompatible with agent identity blueprint principals and can't be set.
`AgentBlueprintPrincipal_NotSupportedOnApiVersion`	Agent identity blueprint principals aren't supported on the API version used in this request.
`AgentBlueprintPrincipal_RequireAgentBlueprint`	Agent identity blueprint principals can only be created for Agent Blueprints.

Agent identity errors

These errors occur when a request references a missing blueprint principal, an incompatible parent type, or unsupported credentials or API versions for agent identities.

Expand table

Error code	Description
`AgentIdentity_AgentBlueprintPrincipalDoesNotExist`	The required agent identity blueprint principal doesn't exist for the specified agent identity blueprint ID.
`AgentIdentity_CredentialsNotSupported`	Credentials are not supported for agent identities. All credentials must be added to the agent identity blueprint.
`AgentIdentity_IncompatibleParentType`	The specified Application (AppId) isn't an Agent Blueprint. The *AgentIdentityBlueprintId* must be set to the *AppId* of a valid agent identity blueprint.
`AgentIdentity_NotSupportedOnApiVersion`	Agent identities aren't supported on the API version used in this request.

Agent identity creation errors

These errors occur when the calling principal isn't allowed to create the requested agent identity.

Expand table

Error code	Description
`Error_AgentBlueprintCannotCreateAssociatedIdentity`	Agent identity blueprints can't create agent identities that are associated with another agent identity blueprint. To create this agent identity, either use the agent identity blueprint that's associated with the agent identity, or perform the operation with a different principal that has the required roles/permissions to create agent identities.
`Error_AgentIdentitiesCreatingAgentIdentitiesNotAllowed`	Agent identities can't create other agent identities. To create an agent identity, use the associated agent identity blueprint principal or nonagent blueprint service principal with the required permissions.
`Error_AgentIdentitySelfCreateRequired`	Applications can only create agent identities under themselves. The provided *AgentIdentityBlueprintId* doesn't match the calling application's *AppId*.

Get help

If you have a question or can't find what you're looking for, see Support and help options for developers to learn about other ways you can get help.

Last updated on 08/14/2026